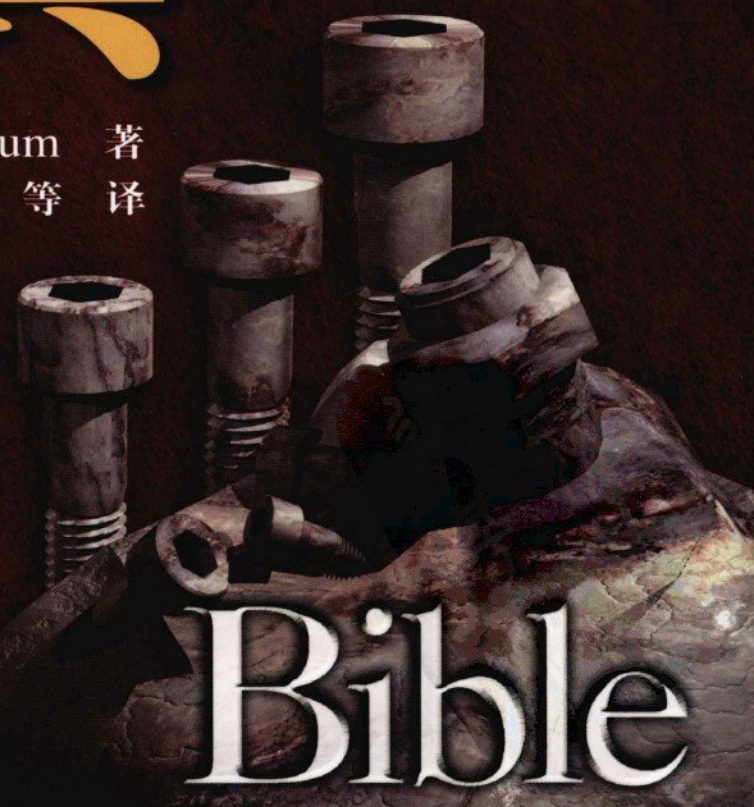
WILEY

丰富 权威 实用

# Linux命令行和shell脚本编程宝典

[美] Richard Blum 著
苏丽 张妍婧 侯晓敏 等 译



精通命令行，不再需要GUI

精通所有Linux shell

实现脚本化和自动化，迅速获得结果

Bible

人民邮电出版社
POSTS & TELECOM PRESS

## 一本书让您掌握Linux shell的所有命令行

基础部分需要学习的内容很多，本宝典不仅简明扼要地介绍了开源Linux软件的结构，并且展示了如何摒弃笨重的Linux图形用户界面，而使用一种更加快速而有效的方式——命令行和自动化脚本。您将学习如何在不使用GUI的情况下管理文件系统的文件；启动和终止程序；使用数据库，甚至执行Web编程，本书将教会您一切。

- 了解Linux桌面和各种命令行参数；
- 了解文件系统导航、文件处理，以及bash shell命令的基础；
- 编写shell脚本以自动化例行功能和报告；
- 管理嵌套循环和结构化命令；
- 监控程序、管理文件许可、进行查询；
- 以后台模式和计划作业方式运行脚本；
- 使用sed、gawk和正则表达式；
- 探讨所有其他shell，包括ash、tcsh、ksh、korn和zsh。

## 作者简介

Richard Blum作为一名系统和网络管理员，已经在IT界经历了19年的风风雨雨。他管理过UNIX、Linux、Novell和Microsoft服务器，曾帮助设计和维护使用Cisco交换机和路由器大型网络，共涉及3500多名用户。他使用Linux服务器和shell脚本执行自动网络监控，并可以在所有UNIX shell环境中编写shell脚本。他撰写了许多图书，包括《Professional Linux Programming》（Wrox）和《Linux For Dummies 8th Edition》（Wiley）。

WILEY
www.wiley.com

封面设计：胡平利

分类建议：计算机 / 操作系统 / Linux
人民邮电出版社网址：www.ptpress.com.cn

ISBN 978-7-115-19777-1
9 787115 197771 >
ISBN 978-7-115-19777-1/TP
定价：69.00 元

# Linux命令行和shell脚本编程宝典

[美] Richard Blum 著
苏丽 张妍婧 侯晓敏 等 译

人民邮电出版社
北京

**图书在版编目（CIP）数据**

Linux命令行和shell脚本编程宝典 / （美）布卢姆（Blum, R.）著；苏丽等译. —北京：人民邮电出版社，2009.6
ISBN 978-7-115-19777-1

Ⅰ. L… Ⅱ. ①布…②苏… Ⅲ. Linux操作系统—程序设计 Ⅳ. TP316.89

中国版本图书馆CIP数据核字（2009）第019023号

版 权 声 明

Richard Blum
Linux Command Line and shell Scripting Bible
Copyright © 2008 by Wiley Publishing, Inc., Indianapolis, Indiana
All rights reserved. This translation published under license.
Authorized translation from the English language edition published by Wiley Publishing, Inc..
**本书中文简体字版由 Wiley Publishing 公司授权人民邮电出版社出版，专有版权属于人民邮电出版社。**

**Linux 命令行和 shell 脚本编程宝典**

◆ 著 [美] Richard Blum
译 苏 丽 张妍婧 侯晓敏等
责任编辑 陈 昇

◆ 人民邮电出版社出版发行 北京市崇文区夕照寺街 14 号
邮编 100061 电子函件 315@ptpress.com.cn
网址 http://www.ptpress.com.cn
北京顺义振华印刷厂印刷

◆ 开本：787×1092 1/16
印张：32
字数：982 千字 2009 年 6 月第 1 版
印数：1 – 3 500 册 2009 年 6 月北京第 1 次印刷

著作权合同登记号 图字：01-2009-0382 号
ISBN 978-7-115-19777-1/TP

定价：69.00 元

**读者服务热线：(010)67132705 印装质量热线：(010)67129223**
**反盗版热线：(010)67171154**

# 内容提要

在 Linux 系统中，命令和 shell 脚本是非常重要的内容，它们的功能非常强大，使用它们可以完成各种操作。本书分为 5 部分。第 1 部分介绍 Linux shell 的基础知识；第 2 部分介绍 shell 脚本；第 3 部分介绍 shell 脚本编程的高级内容；第 4 部分介绍如何使用 Linux 环境中的各种 shell 编写 shell 脚本；第 5 部分演示如何在实际情况中使用 shell 脚本。

本书非常全面地介绍了 Linux 命令行和 shell 命令，并且提供了很多实践教程和实例应用程序信息，为学习的内容提供了相关环境，是 Linux 的爱好者以及 Linux 系统管理员的首选。

献给所有帮助我进步的人们，包括我的父母、我的亲友、我的老师、我的同事以及 Internet 上的匿名发帖人。三人行，必有我师，学无止境。

# 关于作者

**Richard Blum** 作为一名系统和网络管理员，已经在 IT 界经历了 19 年的风风雨雨。在此期间，他管理过 Unix、Linux、Novell 和 Microsoft 服务器，曾帮助设计和维护使用 Cisco 交换机和路由器的大型网络，共涉及 3 500 多名用户。他使用 Linux 服务器和 shell 脚本执行自动网络监控，并可以在所有 Unix shell 环境中编写 shell 脚本。

Rich拥有普渡大学电子工程学学士学位，并拥有管理信息系统方面的理学硕士学位。他编写过好几本Linux图书，包括《Sendmail for Linux》、《Running qmail》、《Postfix》、《Open Source E-mail Security》、《Network Performance Open Source Toolkit》和《Professional Assembly Language Programming》。他还是《Professional Linux Programming》和《Linux For Dummies》第8版的合著者。

# 致　谢

非常感谢 John Wiley & Sons 的团队为本项目作出的杰出贡献，感谢策划编辑 Jenny Watson 向我提供了编写本书的机会。还要感谢开发编辑 Tom Dinse 保证计划的顺利实施，并使本书更加漂亮。技术编辑 John Kennedy 对本书进行了仔细的检查，并提出了大量建议，这大大提升了内容质量。谢谢 John，感谢你的努力和勤奋。我还要感谢 Waterside Productions 公司的 Carole McClendon，是他为我安排了这次机会，并对我的写作事业给予了极大帮助。

最后，我要感谢我的父母 Mike 和 Joyce Blum，他们在养育我的同时还给予我极大的奉献和支持，以及我的妻子 Barbara 和女儿（Katie Jane 和 Jessica）对我的关爱、容忍和理解，尤其是在我编写本书时。

# 前　言

欢迎阅读本书。像所有的宝典系列丛书一样，本书将提供实践教程和实例应用程序信息，还将提供参考资料和背景信息，为学习的内容提供相关环境。本书非常全面地介绍了 Linux 命令行和 shell 命令。学习完本书之后，您将能够编写自己的 shell 脚本，自动执行 Linux 系统上的任何任务。

## 本书的读者对象

如果您是 Linux 环境的系统管理员，那么了解如何编写 shell 脚本将使您受益颇多。本书并没有详细介绍安装 Linux 系统的过程，而是介绍运行 Linux 之后，如何自动化执行一些例行的管理任务。这就是 shell 脚本的用途，也正是本书的目的所在。本书将说明如何使用 shell 脚本自动化地管理任务，从监控系统数据和数据文件到为上级生成报告。

如果您是家用 Linux 环境的爱好者，您也能从本书中受益。如今，很容易在预构建 widget 的图形世界中迷失。大部分桌面 Linux 发行版都尽量向典型用户隐藏系统内核。但是，很多时候需要了解内部究竟发生了什么。本书将展示如何访问 Linux 命令行提示符，介绍进入提示符之后应该做些什么。通过命令行执行一些简单的任务（如文件管理）往往比用图形界面更快。在命令行中可以使用很多命令，本书将展示如何使用它们。

## 本书的内容组织形式

本书将介绍创建 shell 脚本的各种基本 Linux 命令行。本书分为 5 部分，每部分以前一部分内容为基础。

第 1 部分假设您已经有了 Linux 系统或者正打算使用 Linux 系统。第 1 章“Linux shell 入门”介绍了 Linux 系统的大致组成部分以及 shell 如何装入。学习完 Linux 系统的基础知识之后，将继续介绍：

- 使用终端模拟包访问 shell（第 2 章）；
- 介绍基本 shell 命令（第 3 章）；
- 使用更高级的 shell 命令查看系统信息（第 4 章）；
- 处理 shell 变量以操作数据（第 5 章）；
- 了解 Linux 文件系统和安全（第 6 章）；
- 了解如何使用 Linux 编辑器开始编写 shell 脚本（第 7 章）。

在第 2 部分，您将开始学习如何编写 shell 脚本：

- 了解如何创建和运行 shell 脚本（第 8 章）；
- 使用 shell 脚本改变程序流（第 9 章）；
- 迭代代码片段（第 10 章）；
- 使用脚本处理用户数据（第 11 章）；
- 了解各种存储和显示脚本数据的方法（第 12 章）；
- 控制 shell 脚本在系统中运行的时间和方式（第 13 章）。

第 3 部分将深入探讨 shell 脚本编程的高级内容：

- 创建自己的函数并在脚本中使用它（第 14 章）；
- 了解与脚本用户交互的各种方法（第 15 章）；
- 使用高级 Linux 命令筛选和解析数据文件（第 16 章）；

- 使用正则表达式定义数据（第 17 章）；
- 了解操作脚本数据的高级方法（第 18 章）；
- 了解如何用原数据生成报告（第 19 章）。

在第 4 部分，您将学习如何使用 Linux 环境中的各种 shell 编写 shell 脚本：

- 为 ash 或 dash shell 编写脚本（第 20 章）；
- 了解使用 tcsh shell 编写脚本有何不同（第 21 章）；
- 使用 ksh93 shell 处理浮点型数字（第 22 章）；
- 使用 zsh shell 中的高级网络和数学特性（第 23 章）。

本书的最后一个部分（第 5 部分）演示如何在现实情况中使用 shell 脚本：

- 了解如何在 shell 脚本中使用常见的开源数据库（第 24 章）；
- 学习如何从网站提取数据以及在系统间发送数据（第 25 章）；
- 使用电子邮件向外部用户发送通知和报告（第 26 章）；
- 编写 shell 脚本，使日常系统管理功能自动化（第 27 章）。

## 约定与特色

本书有很多不同的组织和印刷特点，以便帮助您充分利用这些信息。

在本书中，特殊字体表示代码和命令。命令和代码以等宽字体显示。代码行显示如下：

```
$ cat test2
#!/bin/bash
# testing a bad command
if asdfg
then
   echo "it didn't work"
fi
echo "we're outside of the if statement"
$ ./test2
./test2: line 3: asdfg: command not found
we're outside of the if statement
$
```

## 注意和警告

如果希望引起您对某些重要内容的注意，这些信息将出现在注意或警告中。

注意：注意提供有用的附加信息或辅助信息，但不属于当前讨论的范围。

警告：该信息很重要，使用特殊字体与正文分离。警告提供需要小心的内容信息，包括可能给数据或系统造成不便或潜在危险的信息。

## 最低系统要求

本书从一般性的角度剖析 Linux，因此任何 Linux 系统都适用于该书。本书的很多内容都以 bash shell 为例，因为它是大部分 Linux 系统的默认 shell。

## 未来方向

学完本书之后，您将能够随意地使用 Linux 命令完成日常的 Linux 工作。在千变万化的 Linux 世界中，最好时刻留意最新的开发成果。Linux 发行版经常更改，如添加新特性和移除旧特性。要始终获得最新的 Linux 知识，应该时刻留意相关信息。找一个好的 Linux 论坛，观察 Linux 世界发生的事情。流行的 Linux 新闻站点很多，如 Slashdot 和 Distrowatch，它们都能提供有关 Linux 发展的最新消息。

# 目　录

## 第 1 部分　Linux 命令行


第 1 章　Linux shell 入门 …… 3
1.1　什么是 Linux …… 3
1.1.1　深入研究 Linux 内核 …… 4
1.1.2　GNU 实用程序 …… 9
1.1.3　Linux 桌面环境 …… 10
1.2　Linux 发行版 …… 13
1.2.1　核心 Linux 发行版 …… 13
1.2.2　特定 Linux 发行版 …… 14
1.2.3　Linux LiveCD …… 14
1.3　小结 …… 15
第 2 章　了解 shell …… 17
2.1　终端模拟 …… 17
2.1.1　图形功能 …… 18
2.1.2　键盘 …… 20
2.2　terminfo 数据库 …… 21
2.3　Linux 控制台 …… 23
2.4　xterm 终端 …… 24
2.4.1　命令行参数 …… 24
2.4.2　xterm 主菜单 …… 25
2.4.3　VT 选项菜单 …… 26
2.4.4　VT 字体菜单 …… 28
2.5　Konsole 终端 …… 29
2.5.1　命令行参数 …… 29
2.5.2　会话 …… 30
2.5.3　菜单栏 …… 31
2.6　GNOME 终端 …… 34
2.6.1　命令行参数 …… 34
2.6.2　选项卡 …… 35
2.6.3　菜单栏 …… 35
2.7　小结 …… 37
第 3 章　基本的 bash shell 命令 …… 38
3.1　启动 shell …… 38
3.2　shell 提示符 …… 39
3.3　bash 手册 …… 40
3.4　文件系统导航 …… 41
3.4.1　Linux 文件系统 …… 41
3.4.2　浏览目录 …… 43
3.5　文件和目录列表 …… 44
3.5.1　基本列表 …… 44
3.5.2　修改显示的信息 …… 45
3.5.3　完整的参数列表 …… 46
3.5.4　过滤列表输出 …… 48
3.6　文件处理 …… 48
3.6.1　创建文件 …… 48
3.6.2　复制文件 …… 49
3.6.3　链接文件 …… 50
3.6.4　重命名文件 …… 51
3.6.5　删除文件 …… 52
3.7　目录处理 …… 53
3.7.1　创建目录 …… 53
3.7.2　删除目录 …… 53
3.8　查看文件内容 …… 54
3.8.1　查看文件统计数据 …… 54
3.8.2　查看文件类型 …… 54
3.8.3　查看整个文件 …… 55
3.8.4　查看部分文件 …… 57
3.9　小结 …… 58
第 4 章　更多 bash shell 命令 …… 59
4.1　监控程序 …… 59
4.1.1　进程查看 …… 59
4.1.2　实时进程监控 …… 64
4.1.3　停止进程 …… 66
4.2　监控磁盘空间 …… 67
4.2.1　挂载介质 …… 68
4.2.2　使用 df 命令 …… 70
4.2.3　使用 du 命令 …… 71
4.3　操作数据文件 …… 71
4.3.1　数据排序 …… 72
4.3.2　搜索数据 …… 74
4.3.3　压缩数据 …… 75
4.3.4　归档数据 …… 78
4.4　小结 …… 79
第 5 章　使用 Linux 环境变量 …… 80
5.1　什么是环境变量 …… 80

5.1.1　全局环境变量 …… 80
5.1.2　本地环境变量 …… 81
5.2　设置环境变量 …… 83
5.2.1　设置本地环境变量 …… 83
5.2.2　设置全局环境变量 …… 84
5.3　移除环境变量 …… 84
5.4　默认的 shell 环境变量 …… 85
5.5　设置 PATH 环境变量 …… 87
5.6　定位系统环境变量 …… 88
5.6.1　登录 shell …… 88
5.6.2　交互式 shell …… 90
5.6.3　非交互式 shell …… 92
5.7　变量数组 …… 92
5.8　使用命令别名 …… 93
5.9　小结 …… 94
**第 6 章　理解 Linux 文件权限** …… 96
6.1　Linux 安全性 …… 96
6.1.1　/etc/passwd 文件 …… 96
6.1.2　/etc/shadow 文件 …… 98
6.1.3　添加新用户 …… 98
6.1.4　删除用户 …… 100
6.1.5　修改用户 …… 100
6.2　使用 Linux 用户组 …… 102
6.2.1　/etc/group 文件 …… 103
6.2.2　创建新用户组 …… 103
6.2.3　修改用户组 …… 104
6.3　解码文件权限 …… 104
6.3.1　使用文件权限符号 …… 105
6.3.2　默认文件权限 …… 105
6.4　修改安全设置 …… 107
6.4.1　修改权限 …… 107
6.4.2　修改所有者 …… 108
6.5　共享文件 …… 108
6.6　小结 …… 109
**第 7 章　使用编辑器** …… 111
7.1　vim 编辑器 …… 111
7.1.1　vim 基本用法 …… 111
7.1.2　编辑数据 …… 113
7.1.3　复制和粘贴 …… 113
7.1.4　搜索和替换 …… 114
7.2　emacs 编辑器 …… 114
7.2.1　在控制台中使用 emacs …… 114
7.2.2　在 X Windows 中使用 emacs …… 118
7.3　KDE 系列编辑器 …… 119
7.3.1　KWrite 编辑器 …… 119
7.3.2　Kate 编辑器 …… 121
7.4　GNOME 编辑器 …… 124
7.4.1　启动 gedit …… 124
7.4.2　基本 gedit 特性 …… 125
7.4.3　设置首选项 …… 125
7.5　小结 …… 128

**第 2 部分　shell 脚本的基础知识**

**第 8 章　基本脚本编译** …… 131
8.1　使用多条命令 …… 131
8.2　创建脚本文件 …… 131
8.3　显示消息 …… 133
8.4　使用变量 …… 134
8.4.1　环境变量 …… 134
8.4.2　用户变量 …… 135
8.4.3　反引号 …… 136
8.5　重定向输入输出 …… 137
8.5.1　输出重定向 …… 137
8.5.2　输入重定向 …… 137
8.6　管道 …… 138
8.7　数学计算 …… 140
8.7.1　expr 命令 …… 140
8.7.2　使用括号 …… 141
8.7.3　浮点解决方案 …… 142
8.8　退出脚本 …… 145
8.8.1　核对退出状态 …… 145
8.8.2　退出命令 …… 146
8.9　小结 …… 147
**第 9 章　使用结构化命令** …… 148
9.1　使用 if-then 语句 …… 148
9.2　if-then-else 语句 …… 150
9.3　嵌套 if 语句 …… 150
9.4　test 命令 …… 151
9.4.1　数值比较 …… 151
9.4.2　字符串比较 …… 152
9.4.3　文件比较 …… 155
9.5　复合条件检查 …… 161
9.6　if-then 的高级特征 …… 161
9.6.1　使用双圆括号 …… 162
9.6.2　使用双方括号 …… 162
9.7　case 命令 …… 163
9.8　小结 …… 164
**第 10 章　更多结构化命令** …… 165
10.1　for 命令 …… 165
10.1.1　读取列表中的值 …… 165

10.1.2 读取列表中的复杂值 …… 166
10.1.3 从变量读取列表 …… 168
10.1.4 读取命令中的值 …… 168
10.1.5 改变字段分隔符 …… 169
10.1.6 使用通配符读取目录 …… 170
10.2 C 式的 for 命令 …… 171
10.2.1 C 语言中的 for 命令 …… 171
10.2.2 使用多个变量 …… 172
10.3 while 命令 …… 173
10.3.1 while 的基本格式 …… 173
10.3.2 使用多条测试命令 …… 174
10.4 until 命令 …… 175
10.5 嵌套循环 …… 176
10.6 文件数据的循环 …… 178
10.7 控制循环 …… 178
10.7.1 break 命令 …… 179
10.7.2 continue 命令 …… 181
10.8 处理循环的输出 …… 183
10.9 小结 …… 184
**第 11 章 处理用户输入** …… 185
11.1 命令行参数 …… 185
11.1.1 读取参数 …… 185
11.1.2 读取程序名称 …… 187
11.1.3 测试参数 …… 188
11.2 特殊的参数变量 …… 188
11.2.1 参数计数 …… 188
11.2.2 获取所有数据 …… 190
11.3 移位 …… 190
11.4 处理选项 …… 191
11.4.1 找出选项 …… 191
11.4.2 使用 getopt 命令 …… 194
11.4.3 更高级的 getopts 命令 …… 196
11.5 标准化选项 …… 197
11.6 获取用户输入 …… 198
11.6.1 基本读取 …… 198
11.6.2 计时 …… 199
11.6.3 默读 …… 200
11.6.4 读取文件 …… 200
11.7 小结 …… 201
**第 12 章 显示数据** …… 202
12.1 了解输入和输出 …… 202
12.1.1 标准文件描述符 …… 202
12.1.2 重定向错误 …… 204
12.2 在脚本中重定向输出 …… 205
12.2.1 临时重定向 …… 205
12.2.2 永久重定向 …… 206
12.3 在脚本中重定向输入 …… 207
12.4 创建自己的重定向 …… 207
12.4.1 创建输出文件描述符 …… 207
12.4.2 重定向文件描述符 …… 208
12.4.3 创建输入文件描述符 …… 208
12.4.4 创建读取/写入文件描述符 …… 209
12.4.5 关闭文件描述符 …… 210
12.5 列出开放文件描述符 …… 210
12.6 禁止命令输出 …… 212
12.7 使用临时文件 …… 212
12.7.1 创建本地临时文件 …… 213
12.7.2 在/temp 中创建临时文件 …… 213
12.7.3 创建临时目录 …… 214
12.8 记录消息 …… 215
12.9 小结 …… 216
**第 13 章 脚本控制** …… 217
13.1 处理信号 …… 217
13.1.1 Linux 信号回顾 …… 217
13.1.2 生成信号 …… 218
13.1.3 捕获信号 …… 219
13.1.4 捕获脚本退出 …… 220
13.1.5 移除捕获 …… 220
13.2 以后台模式运行脚本 …… 221
13.2.1 以后台模式运行 …… 221
13.2.2 运行多个后台作业 …… 222
13.2.3 退出终端 …… 222
13.3 在不使用控制台的情况下运行脚本 …… 222
13.4 作业控制 …… 223
13.4.1 查看作业 …… 223
13.4.2 重新启动停止的作业 …… 225
13.5 变得更好 …… 225
13.5.1 nice 命令 …… 226
13.5.2 renice 命令 …… 226
13.6 准确无误地运行 …… 227
13.6.1 使用 at 命令调度作业 …… 227
13.6.2 使用 batch 命令 …… 229
13.6.3 调度定期脚本 …… 229
13.7 从头开始 …… 230
13.7.1 在启动时启动脚本 …… 231
13.7.2 随新 shell 一起启动 …… 232
13.8 小结 …… 232

## 第 3 部分　高级 shell 脚本编程


第 14 章　创建函数……235
14.1　基本脚本函数……235
14.1.1　创建函数……235
14.1.2　使用函数……236
14.2　返回值……237
14.2.1　默认退出状态……237
14.2.2　使用 return 命令……238
14.2.3　使用函数输出……239
14.3　在函数中使用变量……239
14.3.1　向函数传递参数……240
14.3.2　在函数中处理变量……241
14.4　数组变量与函数……243
14.4.1　向函数传递数组……243
14.4.2　从函数返回数组……244
14.5　函数递归……245
14.6　创建库……246
14.7　在命令行中使用函数……247
14.7.1　在命令行创建函数……247
14.7.2　在.bashrc 文件中定义函数……248
14.8　小结……249
第 15 章　在脚本中添加颜色……250
15.1　创建文本菜单……250
15.1.1　创建菜单布局……250
15.1.2　创建菜单函数……251
15.1.3　添加菜单逻辑……252
15.1.4　将其全部组合在一起……252
15.1.5　使用 select 命令……253
15.2　添加颜色……254
15.2.1　ANSI 转义码……254
15.2.2　显示 ANSI 转义码……255
15.2.3　在脚本中使用颜色……256
15.3　制作窗口……257
15.3.1　dialog 软件包……257
15.3.2　dialog 选项……261
15.3.3　在脚本中使用 dialog 命令……263
15.4　获取图形……265
15.4.1　KDE 环境……265
15.4.2　GNOME 环境……267
15.5　小结……270
第 16 章　sed 和 gawk 介绍……271
16.1　文本处理……271
16.1.1　sed 编辑器……271
16.1.2　gawk 程序……273
16.2　sed 编辑器基础知识……278
16.2.1　更多替换选项……278
16.2.2　使用地址……279
16.2.3　删除行……281
16.2.4　插入和附加文本……282
16.2.5　更改行……283
16.2.6　变换命令……284
16.2.7　打印命令温习……284
16.2.8　将文件用于 sed……286
16.3　小结……288
第 17 章　正则表达式……289
17.1　正则表达式是什么……289
17.1.1　正则表达式的定义……289
17.1.2　正则表达式的类型……290
17.2　定义 BRE 模式……290
17.2.1　纯文本……290
17.2.2　特殊字符……291
17.2.3　定位符……292
17.2.4　点字符……293
17.2.5　字符类……294
17.2.6　否定字符类……295
17.2.7　使用范围……296
17.2.8　特殊字符类……296
17.2.9　星号……297
17.3　扩展的正则表达式……298
17.3.1　问号……298
17.3.2　加号……299
17.3.3　使用大括号……299
17.3.4　管道符号……300
17.3.5　将表达式分组……300
17.4　正则表达式实战……301
17.4.1　计算目录文件……301
17.4.2　验证电话号码……302
17.4.3　解析电子邮件地址……303
17.5　小结……304
第 18 章　高级 sed 编程……306
18.1　多行命令……306
18.1.1　next 命令……306
18.1.2　多行删除命令……309
18.1.3　多行打印命令……310
18.2　保留空间……310
18.3　否定命令……311
18.4　更改命令流……313
18.4.1　分支……313
18.4.2　测试……314

18.5 模式替换……315
18.5.1 与号……315
18.5.2 替换个别单词……315
18.6 在脚本中使用 sed……316
18.6.1 使用包装器……316
18.6.2 重定向 sed 输出……317
18.7 创建 sed 工具……317
18.7.1 双倍行距……318
18.7.2 对可能有空行的文件使用双倍行距……318
18.7.3 对文件中的行记数……319
18.7.4 打印最后几行……319
18.7.5 删除行……320
18.7.6 删除 HTML 标记……322
18.8 小结……323
**第 19 章 高级 gawk 编程**……324
19.1 使用变量……324
19.1.1 内置变量……324
19.1.2 用户定义的变量……328
19.2 使用数组……329
19.2.1 定义数组变量……329
19.2.2 在数组变量中递归……330
19.2.3 删除数组变量……330
19.3 使用模式……331
19.3.1 正则表达式……331
19.3.2 匹配操作符……331
19.3.3 数学表达式……332
19.4 结构化命令……332
19.4.1 if 语句……333
19.4.2 while 语句……334
19.4.3 do-while 语句……335
19.4.4 for 语句……335
19.5 格式化打印……335
19.6 内置函数……337
19.6.1 数学函数……337
19.6.2 字符串函数……338
19.6.3 时间函数……339
19.7 用户定义的函数……340
19.7.1 定义函数……340
19.7.2 使用自己的函数……340
19.7.3 创建函数库……341
19.8 小结……341

**第 4 部分 可选的 Linux shell**

**第 20 章 ash shell**……345
20.1 什么是 ash shell……345
20.2 原始 ash shell……346
20.2.1 原始 ash 命令行参数……346
20.2.2 原始 ash 内置命令……347
20.2.3 ash shell 文件……349
20.3 dash shell……349
20.3.1 dash 命令行参数……349
20.3.2 dash 环境变量……350
20.3.3 dash 内置命令……352
20.4 在 dash 中编写脚本……355
20.4.1 创建 ash 和 dash 脚本……355
20.4.2 失效的特性……355
20.5 小结……359
**第 21 章 tcsh shell**……360
21.1 什么是 tcsh shell……360
21.2 tcash shell 的组成部分……360
21.2.1 tcsh 命令行参数……361
21.2.2 tcsh 文件……361
21.2.3 tcsh 登录文件……362
21.2.4 shell 启动文件……362
21.2.5 退出文件……363
21.2.6 tcsh 环境变量……364
21.2.7 shell 变量……364
21.2.8 环境变量……367
21.2.9 在 tcsh 中设置变量……369
21.2.10 使用 set 命令……369
21.2.11 使用 setenv 命令……369
21.2.12 tcsh 内置命令……370
21.3 在 tcsh 中编写脚本……372
21.3.1 处理变量……372
21.3.2 数组变量……372
21.3.3 处理数学运算……372
21.3.4 结构化命令……373
21.3.5 if 语句……373
21.3.6 foreach 语句……375
21.3.7 while 语句……375
21.3.8 switch 命令……375
21.4 小结……376
**第 22 章 Korn shell**……378
22.1 Korn shell 的历史……378
22.2 ksh93 shell 的组成部分……378
22.2.1 命令行参数……379
22.2.2 默认文件……380
22.2.3 环境变量……380
22.2.4 内置命令……385

22.3 在 ksh93 shell 中编写脚本……387
22.3.1 数学运算……387
22.3.2 结构化命令……389
22.3.3 命令重定向……391
22.3.4 规程函数……391
22.4 小结……392
**第 23 章 zsh shell**……393
23.1 zsh shell 的历史……393
23.2 zsh shell 的组成部分……393
23.2.1 shell 选项……394
23.2.2 zsh shell 文件……396
23.2.3 环境变量……398
23.2.4 内置命令……401
23.3 使用 zsh 编写脚本……405
23.3.1 数学运算……405
23.3.2 结构化命令……406
23.3.3 函数……406
23.4 小结……408

**第 5 部分 高级主题**

**第 24 章 使用数据库**……411
24.1 MySQL 数据库……411
24.1.1 安装 MySQL……411
24.1.2 完成 MySQL 配置……413
24.1.3 MySQL 客户端接口……414
24.1.4 创建 MySQL 数据库对象……417
24.2 PostgreSQL 数据库……419
24.2.1 安装 PostgreSQL……419
24.2.2 PostgreSQL 命令接口……421
24.2.3 创建 PostgreSQL 数据库对象……422
24.3 处理表……424
24.3.1 创建表……424
24.3.2 插入和删除数据……425
24.3.3 查询数据……426
24.4 在脚本中使用数据库……427
24.4.1 连接到数据库……427
24.4.2 向服务器发送命令……428
24.4.3 格式化数据……431
24.5 小结……432
**第 25 章 使用 Web**……433
25.1 Lynx 程序……433
25.1.1 安装 Lynx……434
25.1.2 Lynx 命令行……434
25.1.3 Lynx 配置文件……438
25.1.4 Lynx 环境变量……439
25.1.5 从 Lynx 捕获数据……439
25.2 cURL 程序……441
25.2.1 安装 cURL……441
25.2.2 cURL 命令行……442
25.2.3 研究 curl……444
25.3 使用 zsh 连接网络……445
25.3.1 TCP 模块……445
25.3.2 客户端/服务器模型……446
25.3.3 使用 zsh 进行客户端/服务器编程……446
25.4 小结……449
**第 26 章 使用电子邮件**……450
26.1 Linux 电子邮件基础知识……450
26.1.1 Linux 中的电子邮件……450
26.1.2 邮件传输代理……451
26.1.3 邮件分发代理……452
26.1.4 邮件用户代理……452
26.2 设置服务器……454
26.2.1 sendmail……455
26.2.2 Postfix……456
26.3 使用 Mailx 发送消息……459
26.4 Mutt 程序……461
26.4.1 安装 Mutt……461
26.4.2 Mutt 命令行……462
26.4.3 使用 Mutt……462
26.5 小结……463
**第 27 章 管理员使用的 shell 脚本**……465
27.1 监视系统统计信息……465
27.1.1 监视磁盘空闲空间……465
27.1.2 谁在霸占磁盘资源……467
27.1.3 监视 CPU 和内存使用情况……469
27.2 执行备份……474
27.2.1 归档数据文件……474
27.2.2 脱机存储备份文件……477
27.3 小结……478
**附录 A bash 命令快速指南**……479
A.1 内置命令……479
A.2 bash 命令……480
A.3 环境变量……481
**附录 B sed 和 gawk 快速指南**……484
B.1 sed 编辑器……484
B.1.1 启动 sed 编辑器……484
B.1.2 sed 命令……484

B.2 gawk 程序……487
B.2.1 gawk 命令格式……487
B.2.2 使用 gawk……487
B.2.3 gawk 变量……488
B.2.4 gawk 程序功能……489
附录 C shell 版本比较……491
C.1 变量……491
C.1.1 环境变量……491
C.1.2 用户定义的变量……491
C.1.3 数组变量……492
C.2 结构化命令……492
C.2.1 if-then、while 和 until 语句……492
C.2.2 for 语句……493
C.3 数学计算……493

PART 1

# Linux 命令行

本部分内容：

第 1 章
Linux shell 入门

第 2 章
了解 shell

第 3 章
基本的 bash shell 命令

第 4 章
更多 bash shell 命令

第 5 章
使用 Linux 环境变量

第 6 章
理解 Linux 文件权限

第 7 章
使用编辑器

# 第 1 章
# Linux shell 入门

CHAPTER 1

本章内容：

- 什么是 Linux；
- Linux 内核的组成部分；
- 探索 Linux 桌面；
- 了解 Linux 发行版。

在深入讨论 Linux 命令行和 shell 之前，首先要了解什么是 Linux，Linux 从何而来，以及 Linux 如何工作。本章将简单介绍什么是 Linux，然后说明 shell 和命令行在整个 Linux 中的地位。

## 1.1 什么是 Linux

如果您以前没有接触过 Linux，可能会奇怪为什么 Linux 有这么多版本。查看 Linux 软件包时，您肯定会被发行版（distribution）、LiveCD 和 GNU 这些术语弄得头昏眼花。第一次涉足 Linux 世界很可能就此成为一段不堪回首的往事。在开始学习命令和脚本之前，本章先为您揭开 Linux 系统的神秘面纱。

对于初学者，首先要了解 Linux 由以下 4 个主要部分组成：

- Linux 内核；
- GNU 实用程序；
- 图形桌面环境；
- 应用软件。

在 Linux 系统中，这 4 个部分各自承担着不同的责任，每个部分都是不可或缺的。图 1-1 展示了一个基本图表，该图表说明这 4 个部分如何共同作用以形成一个完整的 Linux 系统。

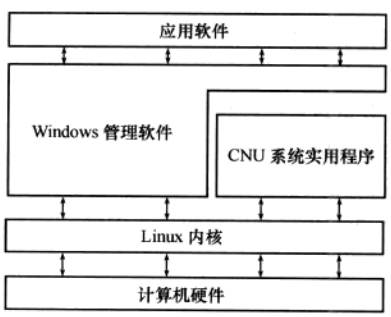



图 1-1 Linux 系统

本节将详细介绍这 4 个主要组成部分，并概述它们如何共同作用以形成完整的 Linux 系统。

### 1.1.1 深入研究 Linux 内核

Linux 系统的核心称为**内核**（kernel）。内核控制计算机系统上的所有硬件和软件，并在必要时分配硬件，在需要时执行软件。

如果您一直关注 Linux，那么肯定听过 Linus Torvalds 的大名。Linus 在赫尔辛基大学读书时创建了第一个 Linux 内核软件。当时很多大学都使用 Unix 系统作为操作系统，Linus 一开始只是打算创建一个 Unix 系统的复制品。

开发出 Linux 内核之后，Linus 将其发布到 Internet 社区，希望社区能够提供一些改进的建议。正是这个简单的过程引发了计算机操作系统的革命。很快，Linus 便收到了来自世界各地的建议，有的来自学生，有的来自专业程序员。

如果每个人都能更改内核的编程代码，结果必然会是一团糟。为了方便起见，Linus 成为了所有改进建议的汇集中心，最终由他决定是否在内核中加入建议的代码。如今的 Linux 内核代码改进仍然使用该原则，只不过控制内核代码的不再仅仅是 Linus，而是包括他在内的一个开发人员团队。

内核主要有以下 4 种功能：

- 系统内存管理；
- 软件程序管理；
- 硬件管理；
- 文件系统管理。

接下来的几节将详细介绍每种功能。

**1. 系统内存管理**

内存管理是操作系统内核的主要功能之一。内核不仅可以管理服务器上可用的物理内存，而且能够创建并管理虚拟内存，或者说非实际存在的内存。

内存管理必须要使用硬盘空间，该空间称为**交换空间**（swap space）。内核不断地在交换空间和实际物理内存之间交换虚拟内存位置的内容。这样系统认为可用的内存比实际存在的内存多（如图 1-2 所示）。

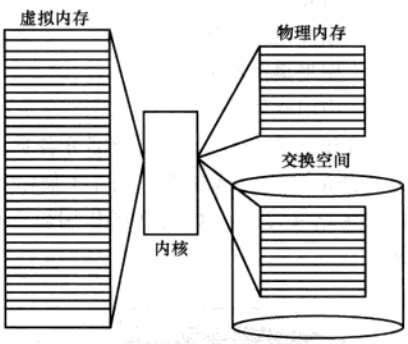


图 1-2 Linux 系统内存图

将内存位置分组为多个数据块，此操作称为**分页**（page）。内核定位物理内存或交换空间中的每个内存分页，然后维护一个内存分页表，此表说明位于物理内存的分页和交换到磁盘的分页。

内核跟踪使用的内存分页，并自动将一段时间没有访问的内存分页复制到交换空间区域（称为**换出**），即使内存仍然够用也会这样做。程序需要访问已经换出的内存分页时，内核必须换出另一个内存分页，以在物理内存中为该内存分页腾出空间，然后从交换空间换入需要的分页。显然，该过程比较耗时，可能会降低运行速度。在 Linux 系统运行期间，为运行应用程序换出内存分页的过程时刻都在进行。

查看特殊文件/proc/meminfo，可以看到 Linux 系统中虚拟内存的当前状态。/proc/meminfo 项的示例如下：

```
# cat /proc/meminfo
MemTotal:         255392 kB
MemFree:            4336 kB
Buffers:            1236 kB
Cached:            48212 kB
SwapCached:         1028 kB
Active:           182932 kB
```

```
Inactive:          44388 kB
HighTotal:             0 kB
HighFree:              0 kB
LowTotal:         255392 kB
LowFree:            4336 kB
SwapTotal:        524280 kB
SwapFree:         514528 kB
Dirty:               456 kB
Writeback:             0 kB
AnonPages:        176940 kB
Mapped:            40168 kB
Slab:              16080 kB
SReclaimable:       4048 kB
SUnreclaim:        12032 kB
PageTables:         4048 kB
NFS_Unstable:          0 kB
Bounce:                0 kB
CommitLimit:      651976 kB
Committed_AS:     442296 kB
VmallocTotal:     770040 kB
VmallocUsed:        3112 kB
VmallocChunk:     766764 kB
HugePages_Total:       0
HugePages_Free:        0
HugePages_Rsvd:        0
Hugepagesize:       4096 kB

#
```

Mem:一行说明该 Linux 服务器的物理内存为 256MB，还剩下 4 MB 的空间没有使用（空闲空间）。从该输出还可以看到，该系统的交换空间内存为 512MB。

默认情况下，在 Linux 系统上运行的每个进程都有自己专用的内存分页。一个进程不能访问另一个进程正在使用的内存分页。内核也有自己的内存区域。出于安全性考虑，任何进程都不得访问内核进程使用的内存。

要促进数据共享，可以创建共享内存分页。通用共享内存区域可以供多个进程读取和写入。内核维护和管理共享内存区域，并允许单个进程访问共享区域。

使用特殊命令 ipcs 可以查看系统当前的共享内存分页。以下是 ipcs 命令的输出示例：

```
# ipcs -m
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x00000000 0          rich       600        52228          6       dest
0x395ec51c 1          oracle     640        5787648        6

#
```

每个共享内存段都对应有一个创建该内存段的所有者。每个内存段还对应有一个标准的 Linux 权限设置，可以设置该内存段对于其他用户的可用性。使用键值允许其他用户获得访问该共享内存段的权限。

**2. 软件程序管理**

Linux 操作系统将正在运行的程序称为**进程**。进程可以在前台运行（在显示器上显示），也可以在后台运行。内核控制 Linux 系统如何管理在系统中运行的所有进程。

内核创建第一个进程，称为**初始进程**（init process），该进程可在系统上启动所有其他进程。内核启动时，它将初始进程加载到虚拟内存中。内核每启动一个其他进程，都将在虚拟内存中为其分配一个唯一的空间，用于存储该进程使用的数据和代码。

有些 Linux 实现包含一个进程表，可以在引导时自动启动。在 Linux 系统中，该表通常位于特殊

文件/etc/inittabs 中。

Linux 操作系统使用具有**运行级别**功能的初始系统。运行级别可以指示初始进程仅运行哪些特定的进程，正如/etc/inittabs 文件定义的那样。Linux 操作系统中有 5 个初始运行级别。

运行级别 1 只启动基本系统进程，以及一个控制台终端进程，这称为单用户模式。单用户模式通常用于紧急情况下（如某些内容损坏）的系统维护。显然，在这种模式下，只允许一个人（通常是管理员）登录系统操作数据。

标准的初始运行级别是 3。在该运行级别下，可以启动大部分应用软件，如网络支持软件。Linux 中另一个常用的运行级别是 5。在该运行级别下，系统可以启动图形化 X Window 软件，并允许使用图形化桌面窗口登录。

通过控制初始运行级别，Linux 系统可以控制整个系统功能。将系统级别从 3 更改为 5，系统可以从基于控制台的系统更改为高级的图形化 X Window 系统。

稍后（第 4 章）将介绍如何使用 ps 命令查看 Linux 系统中当前运行的进程。使用 ps 命令得到的输出如下：

```
$ ps ax
  PID TTY          STAT     TIME COMMAND
    1 ?            S        0:03 init
    2 ?            SW       0:00 [kflushd]
    3 ?            SW       0:00 [kupdate]
    4 ?            SW       0:00 [kpiod]
    5 ?            SW       0:00 [kswapd]
  243 ?            SW       0:00 [portmap]
  295 ?            S        0:00 syslogd
  305 ?            S        0:00 klogd
  320 ?            S        0:00 /usr/sbin/atd
  335 ?            S        0:00 crond
  350 ?            S        0:00 inetd
  365 ?            SW       0:00 [lpd]
  403 ttyS0        S        0:00 gpm -t ms
  418 ?            S        0:00 httpd
  423 ?            S        0:00 httpd
  424 ?            SW       0:00 [httpd]
  425 ?            SW       0:00 [httpd]
  426 ?            SW       0:00 [httpd]
  427 ?            SW       0:00 [httpd]
  428 ?            SW       0:00 [httpd]
  429 ?            SW       0:00 [httpd]
  430 ?            SW       0:00 [httpd]
  436 ?            SW       0:00 [httpd]
  437 ?            SW       0:00 [httpd]
  438 ?            SW       0:00 [httpd]
  470 ?            S        0:02 xfs -port -1
  485 ?            SW       0:00 [smbd]
  495 ?            S        0:00 nmbd -D
  533 ?            SW       0:00 [postmaster]
  538 tty1         SW       0:00 [mingetty]
  539 tty2         SW       0:00 [mingetty]
  540 tty3         SW       0:00 [mingetty]
  541 tty4         SW       0:00 [mingetty]
  542 tty5         SW       0:00 [mingetty]
  543 tty6         SW       0:00 [mingetty]
  544 ?            SW       0:00 [prefdm]
  549 ?            SW       0:00 [prefdm]
  559 ?            S        0:02 [kwm]
  585 ?            S        0:06 kikbd
  594 ?            S        0:00 kwmsound
  595 ?            S        0:03 kpanel
```

```
  596 ?            S        0:02 kfm
  597 ?            S        0:00 krootwm
  598 ?            S        0:01 kbgndwm
  611 ?            S        0:00 kcmlaptop -daemon
  666 ?            S        0:00 /usr/libexec/postfix/master
  668 ?            S        0:00 qmgr -l -t fifo -u
  787 ?            S        0:00 pickup -l -t fifo
  790 ?            S        0:00 telnetd: 192.168.1.2 [vt100]
  791 pts/0        S        0:00 login -- rich
  792 pts/0        S        0:00 -bash
  805 pts/0        R        0:00 ps ax
$
```

输出的第一列是**进程 ID**（或称为 PID）。注意，第一个进程是前面介绍过的初始进程，Linux 系统分配的 PID 为 1。所有在初始进程之后启动的进程均按数字顺序分配 PID。任意两个进程的 PID 均不同。

第三列是进程的当前状态（S 表示休眠；SW 表示休眠并等待；R 表示运行）。进程名在最后一列显示。方括号中的进程表示该进程不活动，因而已经从内存换出到磁盘交换空间中。可以看到，有些进程已经换出，但大部分运行的进程没有换出。

### 3. 硬件管理

内核的另一个职责是硬件管理。Linux 系统需要与之通信的设备都必须在内核代码中插入驱动程序代码。驱动程序代码使内核能够向设备传输数据，它的作用就像是应用程序与硬件之间的中间人。在 Linux 内核中插入设备驱动程序代码有两种方法：

- 在内核中编译驱动程序；
- 向内核添加驱动程序模块。

以前，插入设备驱动程序代码的唯一方式是重新编译内核。每次向系统添加一个新设备时，都必须重新编译内核代码。随着 Linux 内核支持硬件的增多，这种方法越来越低效。幸运的是，Linux 开发人员想出了一种更好的方法向运行内核插入驱动程序代码。

开发人员开发了内核模块，可以在运行内核中插入驱动程序代码而无需重新编译内核。此外，当设备使用完毕时，还可以从内核中移除内核模块。这极大地简化并扩展了 Linux 对硬件的使用。

Linux 系统将硬件设备标识为特殊文件，称为**设备文件**。设备文件分为 3 类：

- 字符；
- 块；
- 网络。

字符设备文件用于那些一次仅处理一个字符的设备。大部分调制解调器和终端类型都识别为字符文件。块文件用于那些一次可处理大量数据块的设备，如磁盘驱动器。

网络文件类型用于那些使用数据包发送和接收数据的设备。这包括网卡和特殊的回路设备，允许 Linux 系统使用通用网络编程协议与自身通信。

Linux 为系统的每个设备创建特殊文件，称为节点。所有与设备的通信都通过设备节点完成。每个节点都有一个唯一的数字对，Linux 内核使用该数字对标识该节点。数字对包含主设备号和次设备号。类似的设备使用相同的主设备号分组。次设备号用于标识主设备组中的具体设备。以下是几个 Linux 服务器设备文件的示例：

```
$ ls -al sda* ttyS*
 brw-rw----   1 root   disk      8,   0 May  5  2006 sda
 brw-rw----   1 root   disk      8,   1 May  5  2006 sda1
 brw-rw----   1 root   disk      8,  10 May  5  2006 sda10
 brw-rw----   1 root   disk      8,  11 May  5  2006 sda11
 brw-rw----   1 root   disk      8,  12 May  5  2006 sda12
 brw-rw----   1 root   disk      8,  13 May  5  2006 sda13
 brw-rw----   1 root   disk      8,  14 May  5  2006 sda14
 brw-rw----   1 root   disk      8,  15 May  5  2006 sda15
```

```
brw-rw----   1 root   disk      8,   2 May  5  2006 sda2
brw-rw----   1 root   disk      8,   3 May  5  2006 sda3
brw-rw----   1 root   disk      8,   4 May  5  2006 sda4
brw-rw----   1 root   disk      8,   5 May  5  2006 sda5
brw-rw----   1 root   disk      8,   6 May  5  2006 sda6
brw-rw----   1 root   disk      8,   7 May  5  2006 sda7
brw-rw----   1 root   disk      8,   8 May  5  2006 sda8
brw-rw----   1 root   disk      8,   9 May  5  2006 sda9
crw-------   1 root   tty       4, 64 Jun 29 16:09 ttyS0
crw-------   1 root   tty       4, 65 May  5  2006 ttyS1
crw-------   1 root   tty       4, 66 May  5  2006 ttyS2
crw-------    1 root   tty       4, 67 May  5  2006 ttyS3
$
```

不同的 Linux 发行版使用不同的设备名称处理设备。在该发行版中，sda 设备是第一个 SCSI 硬盘，ttyS 设备是标准的 IBM PC COM 端口。该列表显示了在示例 Linux 系统上创建的所有 sda 设备。并非所有设备都实际使用，创建它们是为了方便管理员在需要时使用。列表同样显示了创建的所有 ttyS 设备。

第 5 列是主设备节点号。注意，所有 sda 设备都使用相同的设备节点 8，所有的 ttyS 设备都使用 4。第 6 列是次设备节点号。每个主号中的设备都具有唯一的次设备节点号。

第 1 列表示设备文件的权限。权限的第一个字符表示文件的类型。注意，SCSI 硬盘文件都标记为块（b）设备，COM 端口设备文件都标记为字符（c）设备。

**4. 文件系统管理**

与其他操作系统不同，Linux 内核可以使用不同类型的文件系统与硬盘传输数据。除了本身的十几个文件系统外，Linux 还可以通过其他操作系统（如 Microsoft Windows）使用的文件系统传输数据。并编译内核以支持系统将使用的所有文件系统。表 1-1 列出了 Linux 系统可以用来读取和写入数据的文件系统。

Linux 服务器访问的任何硬盘都必须使用表 1-1 中列出的文件系统类型之一进行格式化。

**表 1-1　　Linux 文件系统**

| 文件系统 | 描述 |
|---|---|
| ext | Linux 扩展文件系统——原始 Linux 文件系统 |
| ext2 | 第二代扩展文件系统，提供比 ext 更高级的功能 |
| ext3 | 第三代扩展文件系统，支持日志功能 |
| hpfs | OS/2 高性能文件系统 |
| jfs | IBM 的日志文件系统 |
| iso9660 | ISO 9660 文件系统（CD-ROM） |
| minix | MINIX 文件系统 |
| msdos | Microsoft FAT16 |
| ncp | Netware 文件系统 |
| nfs | 网络文件系统 |
| ntfs | 支持 Microsoft NT 文件系统 |
| proc | 访问系统信息 |
| ReliserFS | 高级 Linux 文件系统，性能更佳且支持磁盘恢复 |
| smb | Samba SMB 文件系统，支持网络访问 |
| sysv | 旧式 Unix 文件系统 |
| ufs | BSD 文件系统 |
| umsdos | 类似 Unix 的文件系统，寄存在 MSDOS 上 |
| vfat | Windows 95 文件系统（FAT32） |
| XFS | 高性能 64 位日志文件系统 |

Linux 内核使用虚拟文件系统（Virtual File System，VFS）与每个文件系统进行连接。这为内核与其他文件系统类型的通信提供了一个标准接口。挂载和使用每个文件系统时，VFS 在内存中缓存相关

信息。

### 1.1.2 GNU 实用程序

除了使用内核控制硬件设备外，计算机操作系统需要实用程序执行标准功能，如控制文件和程序。尽管 Linus 创建了 Linux 系统内核，但他没有任何系统实用程序来运行它。幸运的是，就在他孜孜不倦地工作时，Internet 上还有一群人正在尝试开发一套标准的计算机系统实用程序，用于模拟流行的 Unix 操作系统。

GNU 组织（GNU 代表 GNU's Not Unix）开发了一个完整的 Unix 实用程序集，但是没有任何内核系统去运行它们。这些实用程序的开发基于一种新的软件思想，称为开源软件（OSS）。

OSS 允许程序员开发软件并向全世界发布，没有任何附加的许可费用。任何人都可以使用它，修改它，或者将它整合到自己的系统中，无需缴纳任何许可费用。Linus 的 Linux 内核与 GNU 操作系统实用程序的结合诞生了一个完整的、功能强大的免费操作系统。

尽管 Linux 内核和 GNU 实用程序一般统称为 Linux，但在 Internet 上有时也能看到某些 Linux 人士称其为 GNU/Linux 系统，以对 GNU 组织做出的贡献表达敬意。

#### 1. 核心 GNU 实用程序

GNU 项目的主要目的是使 Unix 系统管理员有一个类似 Unix 的环境。这个侧重点使得项目移植了许多常用的 Unix 系统命令行实用程序。提供给 Linux 系统的核心实用程序包称为 coreutils 软件包。

GNU coreutils 软件包由 3 部分组成：

- 处理文件的实用程序；
- 操作文本的实用程序；
- 管理进程的实用程序。

对于 Linux 系统管理员和程序员来说，这 3 个主要的实用程序组中包含的各种实用程序简直是无价之宝。本书将详细介绍 GNU coreutils 软件包中包含的所有实用程序。

#### 2. shell

GNU/Linux shell 是一个特殊的交互式实用程序。它为用户提供了一种启动程序、管理文件系统中的文件和管理运行在 Linux 系统上的进程的方式。shell 的核心是命令提示符。命令提示符是 shell 用于交互的组成部分。它允许输入文本命令，解释命令，然后在内核中执行命令。

shell 包含一组内部命令，此命令组可以用来控制文件复制、文件移动、文件重命名，以及显示当前运行在系统上的程序等操作。除了内部命令外，shell 还允许在命令提示符处输入程序的名称。shell 可以将程序名称传递到内核以启动该程序。

可以在 Linux 系统上使用的 Linux shell 很多。不同的 shell 有不同的特征，有些可能对创建脚本更有用，有些则可能更适合于管理进程。所有 Linux 发行版中默认使用的 shell 是 bash shell。bash shell 是 GNU 项目开发的，用于取代标准的 Unix shell，也就是 Bourne shell（以它的创建者命名）。bash shell 的名字实际上是一个文字游戏，它表示 Bourne again shell。

除了 bash shell 之外，本书还将介绍其他几种常用的 shell。表 1-2 列出了我们将介绍的几种 shell。

表 1-2 Linux shell

| shell | 描述 |
|---|---|
| ash | 一个简单的轻量级 shell，运行于低内存环境，但与 bash shell 完全兼容 |
| korn | 与 Bourne shell 兼容的编程 shell，但支持高级编程功能，如关联数组和浮点算法 |
| tcsh | 在 shell 脚本中吸收了 C 编程语言元素的 shell |
| zsh | 吸收了 bash、tcsh、korn 等 shell 功能的高级 shell，还提供高级编程功能、共享历史文件和主题提示符 |

大部分 Linux 发行版包含多个 shell，但是通常只挑选其中一个作为默认 shell。如果您的 Linux 发

行版包括多个 shell，可以逐个试用找到最适合自己的 shell。

### 1.1.3　Linux 桌面环境

早期的 Linux（20 世纪 90 年代初）只有一个连接 Linux 操作系统的简单文本界面。该文本界面允许管理员启动程序、控制程序操作，以及在系统范围内移动文件。

随着 Microsoft Windows 的流行，计算机用户希望改进旧的文本界面。这促使 OSS 社区进行了更多开发，于是就出现了 Linux 图形桌面。

Linux 最大的特点就是能够以多种方式完成某一任务，这一特点在图形桌面中表现得淋漓尽致。在 Linux 中，有很多图形桌面供您选择。以下几节将介绍几个常用的桌面。

#### 1. X Windows 系统

控制视频环境的两个基本元素是 PC 中的显卡和监视器。要在计算机中显示高级图形，Linux 软件需要了解如何与这两个元素对话。X Windows 软件是展现图形的核心元素。

X Windows 软件是一个低级别的程序，能与 PC 中的显卡和监视器直接交互，并控制 Linux 应用程序如何在计算机中展示高级窗口和图形。

Linux 并不是唯一使用 X Windows 的操作系统，X Windows 针对不同的操作系统编写了很多版本。对于 Linux 来说，只有两个软件包能执行 X Windows。

XFree86 软件包是这两个软件包中较早的一个，在很长一段时间内，它也是唯一一个可用于 Linux 的 X Windows 包。从它的名称可以看出，这是 X Windows 软件的一个免费开源版本。

最近，Linux 世界又出现了一个新面孔，这个新的软件包称为 X.org。同样，它也是 X Windows 系统的开源软件。它的应用越来越广泛，许多 Linux 发行版开始使用它替代旧的 XFree86 系统。

这两个软件包的工作原理相同，都控制着 Linux 如何使用显卡在监视器上显示内容。要实现这一点，首先必须针对特定的系统进行配置，这应该在安装 Linux 时自动完成。

第一次安装 Linux 发行版时，它将检测显卡和监视器，然后创建一个 X Windows 配置文件，其中包含了必需的信息。在安装期间，您可能会注意到有一段时间安装程序会扫描监视器以寻找支持的视频模式。有时该过程会导致监视器白屏几秒钟。因为显卡和监视器的型号实在太多了，所以完成该过程可能需要花点时间。

对于这个过程，大部分自定义 Linux 发行版都能应付自如，它们可以自动检测视频硬件和设置，而不会丢给您一大堆技术难题。

不幸的是，有时安装无法自动检测所使用的视频设置，尤其是遇到一些较新也更加复杂的显卡时。更糟的是，如果无法找到显卡设置，有些 Linux 发行版将安装失败。还有一些发行版可能会在安装过程中询问一些问题，以帮助手动收集必要的信息。还有一些发行版默认使用最低标准，不考虑您的视频环境，生成通用的屏幕图像。

让事情更加复杂的是，为了能够享受高分辨率游戏，许多 PC 用户拥有高级显卡，如 3-D 加速卡。过去，如果安装 Linux，此类显卡将引起很多问题。但最近显卡公司正在帮忙解决这个问题，它们提供了 Linux 驱动程序。现在许多自定义 Linux 发行版都包含特定显卡的驱动程序。

核心 X Windows 软件仅仅生成一个图形显示环境。尽管这对于运行单个应用程序而言很不错，但是对于日常的计算机使用并不很实用。没有任何桌面环境允许用户操纵文件或启动程序。要实现这些功能，需要在 X Windows 系统软件上安装桌面环境。

#### 2. KDE 桌面

KDE 桌面环境于 1996 年首次发行，它是一个开源项目，该项目旨在创建类似于 Micorsoft Windows 环境的图形桌面。Windows 用户会发现 KDE 桌面集成了所有熟悉的功能。图 1-3 展示了在 Linux 上运行的 KDE 桌面示例。

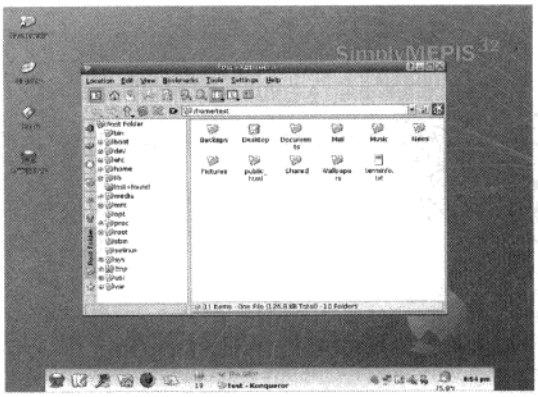


图 1-3 SimplyMEPIS Linux 系统上的 KDE 桌面

KDE 桌面允许在桌面区域放置应用程序和文件图标。如果单击应用程序图标，Linux 系统将启动该应用程序。如果单击文件图标，KDE 桌面将确定使用哪种应用程序处理该文件。

桌面底部的工具栏称为 Panel。Panel 由 4 个部分组成：

- **K 菜单**：类似于 Windows 的“开始”菜单，K 菜单包含启动所安装应用程序的链接。
- **程序快捷键**：可以直接从 Panel 启动应用程序的快速链接。
- **任务栏**：任务栏显示当前在桌面运行的应用程序的图标。
- **Applets**：一些小型应用程序，在 Panel 中存在对应的图标，图标通常随应用程序的信息而改变。

Panel 的所有功能都与 Windows 中的类似。除了桌面功能外，KDE 项目还创建了大量可以在 KDE 环境中运行的应用程序。这些应用程序如表 1-3 所示。（您可能注意到了，KDE 应用程序的名称倾向于使用大写 K。）

表 1-3 KDE 应用程序

| 应用程序 | 描述 | 应用程序 | 描述 |
|---|---|---|---|
| amaroK | 音频文件播放器 | Koffice | 办公应用程序套件 |
| digiKam | 数码相机软件 | Konqueror | 文件和 Web 浏览器 |
| K3b | CD 刻录软件 | Kontact | 个人信息管理器 |
| Kaffeine | 视频播放器 | Kopete | 即时通信客户端 |
| Kmail | 电子邮件客户端 | | |

该表只列举了 KDE 项目创建的部分应用程序，KDE 桌面中还有很多其他的应用程序。

### 3. GNOME 桌面

GNU 网络对象模型环境（GNU Network Object Model Environment，GNOME）是另一个流行的 Linux 桌面环境。GNOME 于 1999 年首次发行，已经成为许多 Linux 发行版（包括最常用的 Red Hat Linux）默认的桌面环境。

GNOME 选择了与标准 Microsoft Windows 完全不同的外观，但是仍然包含了许多 Windows 用户熟悉的特性，包括：

- 桌面图标区域；
- 两个面板区域；
- 拖放功能。

图 1-4 展示 Fedora Linux 发行版使用的标准 GNOME 桌面。

为了抗衡 KDE，GNOME 开发人员也创建了许多可以集成在 GNOME 桌面的图形应用程序，如表 1-4 所示。

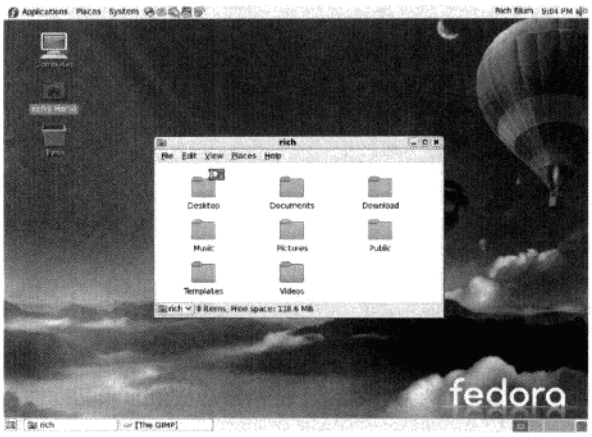



图 1-4　Fedora Linux 系统中的 GNOME 桌面

**表 1-4　　GNOME 应用程序**

| 应用程序 | 描述 | 应用程序 | 描述 |
|---|---|---|---|
| epiphany | Web 浏览器 | gnome-terminal | 终端模拟器 |
| evince | 文档查看器 | nautilus | 图形文件管理器 |
| gcalc-tool | 计算器 | nautilus-cd-burner | CD 刻录工具 |
| gedit | GNOME 文本编辑器 | sound juicer | 音频 CD 转换工具 |
| gnome-panel | 启动应用程序的桌面面板 | tomboy | 笔记软件 |
| gnome-nettool | 网络诊断工具 | totem | 多媒体播放器 |

可以看到，有很多应用程序可在 GNOME 桌面运行。除了这些应用程序之外，大部分使用 GNOME 桌面的 Linux 发行版也集成了 KDE 库，这样就可以在 GNOME 桌面上运行 KDE 应用程序。

**4. 其他桌面**

图形桌面环境的缺点在于，它需要大量系统资源才能正常运行。在早期的 Linux 版本中，它的特点和卖点在于能够在旧的、配置较低的 PC 上运行，而较新的 Microsoft 桌面产品做不到这一点。但是，随着 KDE 和 GNOME 桌面的流行，这个特点也逐渐消失，因为运行 KDE 和 GNOME 桌面所需的内存与最新的 Microsoft 桌面环境相差无几。

如果您的 PC 较老，也不要气馁。Linux 开发人员已经联合起来找回了 Linux 的根：他们创建了几个面向低内存的图形桌面应用程序，这些应用程序只提供基本的功能，在老式的 PC 上也能很好地运行。

尽管这些图形桌面没有太多相应的应用程序，但是同样能运行许多基本的图形应用程序，支持文字处理、电子表格、数据库、绘画和多媒体等功能。

表 1-5 展示了一些更小的 Linux 图形桌面环境，它们可以在配置较低的 PC 和手提电脑中使用。

**表 1-5　　其他 Linux 图形桌面**

| 桌面 | 描述 |
|---|---|
| fluxbox | 一个简陋的桌面，不包括 Panel，只有一个弹出菜单供启动应用程序使用 |
| xfce | 类似于 KDE 桌面，但是图形化较低，用于低内存环境 |
| fvwm | 支持一些高级桌面功能，如虚拟桌面和虚拟 Panel，但可在低内存环境运行 |
| fvwm95 | 衍生自 fvwm，外观类似 Windows 95 桌面 |

这些图形桌面环境不如 KDE 和 GNOME 那么花哨，但是都提供了基本的图形功能。图 1-5 展示了在 SimplyMEPIS antiX 发行版中使用的 fluxbox 桌面。

如果您的 PC 较老，可以尝试一个使用这些桌面的 Linux 发行版，看看会发生什么。结果可能会

好得出人意料。

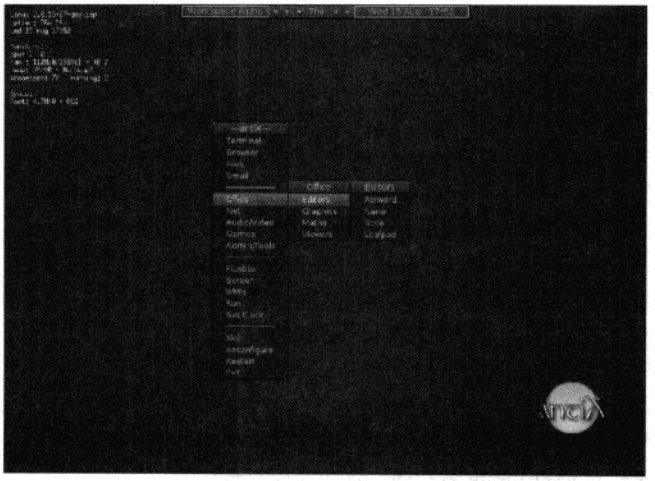


图 1-5 在 SimplyMEPIS antiX 发行版中使用的 fluxbox 桌面

## 1.2 Linux 发行版

现在我们已经介绍了完整 Linux 系统所需的 4 个主要组件，您可能想知道如何将这些组件结合到一起以形成一个 Linux 系统。幸运的是，这个工作已经有人帮我们完成了。

完整的 Linux 系统包称为**发行版**。现在有许多发行版可供选择，可以满足各种计算需求。大部分发行版是针对某一特定的用户群定制的，比如针对商业用户、多媒体爱好者、软件开发人员或普通家庭用户。每种定制的发行版都包括支持特定功能所需的软件包，如多媒体爱好者所需的视频和音频编辑软件，或者软件开发人员所需的编译器和集成开发环境（IDE）。

各种 Linux 发行版一般可分为以下 3 种类型：

- 完整核心 Linux 发行版；
- 特定发行版；
- LiveCD 测试发行版。

接下来的几节将介绍这几种类型的 Linux 发行版，并针对每种类型提供一些 Linux 发行版示例。

### 1.2.1 核心 Linux 发行版

核心 Linux 发行版包括内核、一个或多个图形桌面环境，以及几乎所有的 Linux 应用程序，它针对内核进行了预编译。只需该发行版即可完整安装 Linux。表 1-6 展示了一些比较常见的核心 Linux 发行版。

表 1-6 核心 Linux 发行版

| 发行版 | 描述 |
|---|---|
| Slackware | 原始 Linux 发行版集合之一，广受 Linux 发烧友的好评 |
| Red Hat | 商用发行版，主要供 Internet 服务器使用 |
| Fedora | 从 Red Hat 分离出来的部分，主要针对家庭用户 |
| Gentoo | 针对高级 Linux 用户设计的发行版，仅包含 Linux 源代码 |
| Mandriva | 主要针对家庭用户设计（以前称为 Mandrake） |
| openSuSe | 有商用和家用两个版本（现归 Novell 所有） |
| Debian | 在 Linux 专家和商用 Linux 产品中很流行 |

在早期的 Linux 版本中，发行版使用一组软盘发布。您必须下载文件组，然后将它们复制到磁盘。这通常需要 20 张或以上的磁盘才能完成整个发布！不必说，这真是往事不堪回首。

如今，家用计算机普遍都内置了 CD 和 DVD 播放器，于是 Linux 就使用 CD 集或单张 DVD 发布。这使 Linux 的安装变得更加容易。

但是，新手在安装核心 Linux 发行版时还是常常会遇到问题。为了满足 Linux 用户可能出现的各种需求，一个发行版常常包括很多应用软件，从高端的 Internet 数据库服务器到常见的游戏。由于可用于 Linux 的应用程序如此之多，以至于一个完整的发行版通常需要 4 张或以上的 CD。

对于 Linux 发烧友来说，有如此多的选择实在再好不过了，但是对于 Linux 初级用户则是一场噩梦。大部分发行版在安装过程中会询问一长串问题，以确定默认加载的应用程序，连接到 PC 的硬件，以及如何配置硬件。新手常常搞不清楚这些问题。结果，他们要么在计算机上加载的程序太多，要么加载的不够，以至于后来计算机不能正常工作。

幸运的是，对于新手有一种更简单的安装 Linux 的方法。

### 1.2.2　特定 Linux 发行版

后来逐渐发展出一组新的 Linux 发行版。这些发行版通常以一个主要的发行版为基础，但是只包含对特定领域有用的应用程序子集。

除了提供专门的软件（如对商业用户只提供办公产品）之外，定制的 Linux 发行版还试图为 Linux 初级用户提供帮助——自动检测和自动配置常用的硬件设备。这使安装 Linux 的过程变得更加轻松。

表 1-7 展示了一些特定的 Linux 发行版及其目标领域。

**表 1-7　　特定 Linux 发行版**

| 发行版 | 描述 |
|---|---|
| Linspire | 商用 Linux 包，外观配置类似于 Windows |
| Xandros | 为新手配置的商业 Linux 包 |
| SimplyMEPIS | 免费的家用发行版 |
| Ubuntu | 供学校和家庭使用的免费发行版 |
| PCLinuxOS | 供家庭和办公使用的免费发行版 |
| dyne:bolic | 针对音频和 MIDI 应用程序设计的免费发行版 |
| Puppy Linux | 在老式 PC 上运行良好的免费小型发行版 |

这仅仅是一小部分特定 Linux 发行版。此外还有数百种特定 Linux 发行版，以及不断在 Internet 上出现的更多版本。无论哪个领域，都有专门为您设计的 Linux 发行版。

许多特定 Linux 发行版以 Debian Linux 发行版为基础。它们使用的安装文件与 Debian 相同，但是软件包只是完整 Debian 系统的一小部分。

### 1.2.3　Linux LiveCD

Linux 世界中出现的新生事物是可引导的 Linux CD 发行版。无需安装即可了解 Linux 系统的全貌。除了标准的硬盘启动之外，大部分现代 PC 都可以从 CD 启动。为了利用这一点，一些 Linux 发行版创建了可引导的 CD，其中包含一个示例 Linux 系统（称为 **Linux LiveCD**）。由于单张 CD 的大小有限，该示例无法包含整个 Linux 系统，但其中包含的软件数量之多一定会让您感到惊讶。这样一来，无需在硬盘上安装即可从 CD 启动 PC，并运行 Linux 发行版。

这是测试各种 Linux 发行版的绝佳方式，它丝毫不会弄乱您的 PC。只需要放入 CD 然后启动即可！所有 Linux 软件都直接通过 CD 运行。可以从 Internet 上下载的 Linux LiveCD 很多，您可以下载并将其刻成 CD，以进行测试。

表 1-8 展示了一些常用的 Linux LiveCD。

表 1-8 Linux LiveCD 发行版

| 发行版 | 描述 |
| --- | --- |
| Knoppix | 德语 Linux 版本，开发的第一个 Linux LiveCD |
| SimplyMEPIS | 为初级 Linux 家庭用户设计 |
| PCLinuxOS | 完整的 LiveCD Linux 发行版 |
| Ubuntu | 世界范围的 Linux 项目，针对多种语言设计 |
| Slax | 以 Slackware Linux 为基础的 Live Linux CD |
| Puppy Linux | 针对老式 PC 设计的，功能完整的 Linux |

您可能注意到该表似曾相识。这是因为许多特定 Linux 发行版也有 Linux LiveCD 版本。有些 Linux LiveCD 发行版（如 Ubuntu）允许直接从 LiveCD 安装 Linux 发行版。这使您可以用 CD 引导，测试 Linux 发行版。如果需要，也可以在硬盘上安装。该功能非常方便且非常人性化。

尽管有很多优点，但 Linux LiveCD 也有一些缺点。因为所有内容都通过 CD 访问，因此应用程序运行得很慢，使用较老的低端计算机和 CD 驱动器时更是如此。同时，由于无法写入 CD，因此对 Linux 系统所做的任何更改在下一次启动时都将不复存在。

但在 Linux LiveCD 中加入了一些先进的技术帮助解决这些问题。这些技术包括：

- 将 Linux 系统文件从 CD 复制到内存；
- 将系统文件复制到硬盘中；
- 在 USB 记忆棒中存储系统设置；
- 在 USB 记忆棒中存储用户设置。

有些 Linux LiveCD（如 Puppy Linux）包含的 Linux 系统文件数量非常少，可以在 CD 启动时直接将它们复制到内存。这样一来，启动 Linux 之后就可以立即取出 CD。这不仅使应用程序运行的速度更快（因为应用程序从内存运行的速度更快），而且让 CD 驱动器空闲下来，可以通过 Puppy Linux 中包含的软件处理音频 CD 或播放视频 DVD。

其他 Linux LiveCD 也使用类似的方法允许在启动后取出 CD。这需要将核心 Linux 文件作为一个文件复制到 Windows 硬盘。CD 启动后，它将查找该文件并从中读取系统文件。dyne：bolic Linux LiveCD 使用的就是这项技术，称为停靠（docking）。当然，在从 CD 启动之前，必须将系统文件复制到硬盘。

从 Linux CD 会话存储数据的一项常用技术是，使用常见的 USB 记忆棒（也称为闪存或拇指驱动器）。几乎所有的 Linux LiveCD 都可以识别插入式 USB 记忆棒（即使它是针对 Windows 格式化的）并可以从中读取和写入文件。这样您就可以启动 Linux LiveCD，使用 Linux 应用程序创建文件，在记忆棒中存储文件，然后通过 Windows 应用程序（或从不同的计算机）访问它们。听起来不错吧？

## 1.3 小结

本章讨论了 Linux 系统的起源和工作原理。Linux 内核是该系统的核心，控制着内存、程序和硬件交互的方式。GNU 实用程序也是 Linux 系统中的重要部分。Linux shell 是 GNU 核心实用程序的一部分，也是本书的重点。本章还讨论了 Linux 的最后一个组成部分——Linux 桌面环境。随着时间的推移很多事物都发生了变化，Linux 目前已经支持许多图形桌面环境。

接下来，本章讨论了各种 Linux 发行版。Linux 发行版将各个 Linux 系统的组成部分绑定到一个软件包中，以便于在 PC 上进行安装。Linux 发行版分为包含几乎所有应用程序的完整 Linux 发行版，以

及仅包含特定功能应用程序的特定 Linux 发行版。Linux LiveCD 构成了另一类 Linux 发行版，使您能轻松测试 Linux 而无需在硬盘上安装。

在下一章中，我们将介绍如何开始命令行和 shell 脚本的学习。您将会看到，怎样才能从图形桌面环境获得 Linux shell 实用工具。其中有些内容比较难。

# 第 2 章
# 了解 shell

CHAPTER 2

本章内容：

- 讨论终端模拟；
- 探讨 terminfo 文件；
- 了解 xterm；
- 研究 Konsole；
- 熟悉 GNOME 终端。

在 Linux 的发展早期，唯一能用的工具就是 shell。系统管理员、程序员和系统用户都通过 Linux 控制台终端输入文本命令，并查看文本输出。而今，有了图形桌面环境，在系统上找到 shell 提示符都变得困难起来。本章将介绍如何提供命令行环境，然后全面介绍各种 Linux 发行版都包含的终端模拟包。

## 2.1 终端模拟

在图形桌面出现之前，与 Unix 系统交互的唯一方式是使用 shell 提供的文本**命令行界面**（Command Line Interface，CLI）。CLI 只允许文本输入，并且只能显示文本和基本的图形输出。

由于这种限制，不需要太高级的输出设备。通常一个简单的非智能终端就能与 Unix 系统交互。非智能终端无外乎监视器和键盘（尽管后来鼠标的出现让终端智能起来），它们通过通信电缆（通常是多线串行电缆）连接到 Unix 系统。这种简单的组合能够轻松地向 Unix 系统输入文本数据并查看文本结果。

如您所见，如今的 Linux 环境已经完全不同。几乎所有 Linux 发行版都使用某种图形桌面环境。但是，要访问 shell，仍然需要文本显示器与 CLI 交互。现在就只剩下一个问题了：由于这些新的图形 Linux 桌面特性，很难在 Linux 发行版中找到 CLI。

一种进入 CLI 的方法是让 Linux 系统退出图形桌面模式并采用文本模式。这样一来，监视器上便只能出现一个简单的 shell CLI，就像图形桌面出现之前那样。该模式称为 **Linux 控制台**，因为它模拟以前电缆连接的控制台终端，是连接到 Linux 系统的直接接口。

进入 Linux 控制台的另一种方式是使用图形 Linux 桌面环境中的**终端模拟包**（terminal emulation package）。终端模拟包模拟非智能终端的工作方式，所有操作都可以在桌面的图形窗口内完成。图 2-1 展示了图形 Linux 桌面环境中运行终端模拟器的示例。

每一个终端模拟包都可以模拟一个或多个特定的非智能终端。如果需要使用 Linux 中的 shell，那么您必须要了解一些有关终端模拟的知识。

了解旧式非智能终端的核心功能将有助于在使用图形终端模拟器时确定应该选择哪种模拟类型，也有助于发挥这些功能的全部潜力。非智能终端中使用的主要功能可以分为两个方面：图形功能和键盘。本节将介绍这两个功能，并讨论它们与各种终端模拟器之间的关系。

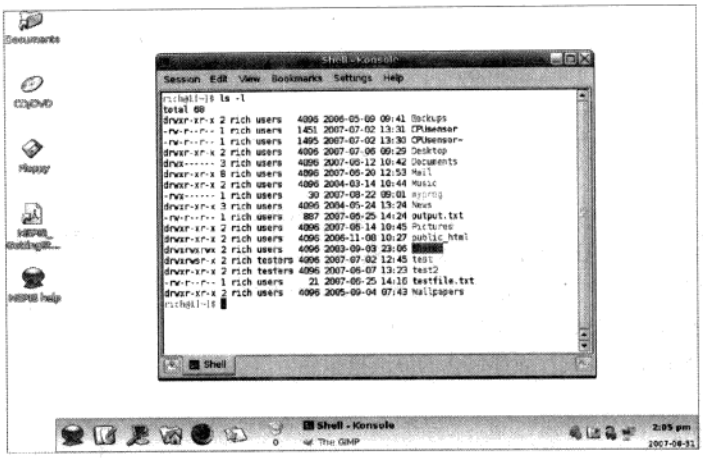



图 2-1　一个在 Linux 桌面运行的简单终端模拟器

## 2.1.1　图形功能

终端模拟最重要的部分是它如何在监视器上显示信息。一听到“文本模式”，我们就应该知道不需要担心图形的问题。但是，即使最基本的非智能终端也支持某些屏幕操作（比如清屏和在屏幕的特定位置显示文本等）。

本节介绍各种终端类型特有的图形功能，以及终端模拟包能够提供的功能。

**1. 字符集**

所有的终端都必须在屏幕上显示字符（否则文本模式将毫无用处）。难点在于显示什么字符，Linux 系统需要发送什么代码才能显示它们。**字符集**是一组二进制命令，Linux 系统需要将它发送到监视器以显示字符。各种终端模拟器包支持的字符集如下所示。

- **ASCII（美国信息交换标准码）**：该字符集包括使用 7 位代码存储的英语字符，由 128 个英语字母（包括大写和小写）、数字和特殊符号组成。该字符集由美国国家标准协会（ANSI）采用（US-ASCII）。在终端模拟器中，它常常被称为 ANSI 字符集。
- **ISO-8859-1（通常称为 Latin-1）**：它是 ASCII 字符集的扩展，由国际标准化组织（ISO）开发。它使用 8 位代码支持标准的 ASCII 字符以及大多数西欧语言的字符。Latin-1 字符集在多国终端模拟包中很常见。
- **ISO-8859-2**：支持东欧语言字符的 ISO 字符集。
- **ISO-8859-6**：支持阿拉伯语言字符的 ISO 字符集。
- **ISO-8859-7**：支持希腊语言字符的 ISO 字符集。
- **ISO-8859-8**：支持希伯来语言字符的 ISO 字符集。
- **ISO-10646（通常称为 Unicode）**：ISO 2 字节字符集，包含大多数英语和非英语语言代码。该字符集包含了所有 ISO-8859-x 系列字符集中定义的代码。Unicode 字符集很快在开源应用程序中流行起来。

目前在英语国家中使用最广的字符集无疑是 Latin-1 字符集。Unicode 字符集正日益崛起，有朝一日很可能成为字符集的新标准。大多数流行的终端模拟器都允许选择要使用的字符集。

**2. 控制码**

除了显示字符外，终端必须有能力控制监视器和键盘的特殊功能，比如光标在屏幕上的位置。这种能力通过**控制码**系统实现。控制码是一种字符集中没有的特殊代码，它通知终端执行特殊的不可显示的操作。

常见的控制码功能有回车（让光标返回到行开头）、换行（将光标放到下一个水平行）、水平制表符（将光标移动预设的空格数）、方向键（上、下、左、右）和向上翻页/向下翻页键。这些代码主要模拟控制监视器上光标位置的功能，此外还有一些其他代码，比如清除整个屏幕，甚至还能模拟铃声（旧式打字机的结束铃声）。

控制码还可以用于控制非智能终端的通信功能。非智能终端通过某种通信通道（一般是串行通信电缆）连接到计算机系统。有时需要在通信通道上控制数据，因此开发人员设计出专门用于数据通信的特殊控制码。虽然这些代码在现代终端模拟器中不是必需的部分，但大多数模拟器仍然支持这些代码，以保证兼容性。此类代码最常见的是 XON 和 XOFF 码，分别可以启动和终止向终端传输数据。

**3. 块模式图形**

随着非智能终端的日益流行，制造商开始尝试基本的图形功能。Unix 中曾流行一时的“图形”非智能终端是 DEC VT 系列终端。非智能终端的转折点是 1978 年发布的 DEC VT100。DEC VT100 终端是第一个支持完整 ANSI 字符集的终端，包括块模式图形字符。

ANSI 字符集包含的代码不仅使监视器能够显示文本，而且还能显示基本的图形符号，比如边框、线条和块。20 世纪 80 年代 Unix 操作中最流行的非智能终端之一无疑是 VT102，VT102 是 VT100 的升级版。大多数终端模拟程序都模拟 VT102 显示的操作，都支持所有的 ANSI 代码，以创建块模式图形。

**4. 向量图形**

Tektronix 公司生产了一系列流行的终端，使用的显示方法称为向量图形。向量图形与块模式图形的 DEC 方法不同，它使用一系列线条（向量）生成所有的屏幕图（包括字符）。Tektronix 4010 终端是其产品中应用最广的图形非智能终端，许多终端模拟包仍然模拟该终端的功能。

4010 终端显示图像的方式是使用电子束绘制一系列向量，类似于使用铅笔绘画。由于向量图不使用点生成线条，因此与面向点的图形终端相比，向量图能绘制精度更高的几何图形。这一功能广受数学家和科学家的好评。

终端模拟器使用软件模拟 Tektronix 4010 终端的向量图绘制功能。对于需要绘制精确图形的人，以及仍然运行使用向量图例程的应用程序绘制复杂图表的人来说，这也是一个很受欢迎的功能。

**5. 显示缓冲**

图形显示的关键在于终端缓冲数据的能力。缓冲数据需要在终端中有额外的内存，以存储当前不需要在监视器上显示的字符。

DEC VT 系列终端使用的数据缓冲方式有两种：

- ✦ 数据滚动出主显示窗口时缓冲数据（称为历史数据）；
- ✦ 缓冲一个完整的显示窗口（称为替代屏幕）。

第一种类型的缓冲称为**滚动区域**（scroll region）。滚动区域是终端在数据滚动出屏幕时用来“记住”数据的内存容量。标准的 DEC VT102 终端包含 25 行字符的查看区域。当终端显示一行新的字符时，上一行便向上滚动。当终端显示到达最后一行时，下一行将最上面的一行挤出显示区。

VT102 终端的内存能够保存刚刚滚动出显示区的 64 行字符。用户可以锁定当前屏幕显示，使用方向键滚动回之前“滚动出”显示区的行。终端模拟包允许使用滚动条或鼠标滚轮滚动保存的数据，无需锁定显示区。当然，为了完整的模拟兼容性，大多数终端模拟包也允许锁定显示区，并使用箭头

和向上/向下翻页滚动保存的数据。

第二种类型的缓冲称为**替代屏幕**（alternative screen）。通常，终端直接向监视器的普通显示区写入数据。后来开发了一种方法，可以使用两个屏幕区域存储数据，简单地实现了动画。这需要使用控制码通知终端向替代屏幕而不是当前显示屏幕写入数据，该数据保存在内存中。另一个控制码可以通知终端交替在监视器中显示普通屏幕数据和保存在替代屏幕的数据，这个过程几乎是瞬间完成的。通过在替代屏幕区中存储连续数据页面并在稍后进行显示，可以粗略地模拟动态图形。

模拟 VT00 系列的终端能够支持替代屏幕方式。

**6. 彩色**

早在黑白（或绿色）非智能终端的年代，程序员们就已经开始尝试各种显示数据的方法。大多数终端都支持特殊的控制码，以生成以下特殊文本：

- 粗体字符；
- 下划线字符；
- 反向显示（白底黑字）；
- 闪烁；
- 组合以上特性。

以前，如果想引起他人的注意，可以使用粗体、闪烁、反向显示文本。而现在的选择众多，让人眼花缭乱。

随着彩色终端的出现，程序员增加了特殊的控制码，以使用各种颜色和亮度显示文本。ANSI 字符集包括用于指定在监视器上显示的前景文本和背景文本颜色的控制码。大多数终端模拟器都支持 ANSI 颜色控制码。

## 2.1.2 键盘

终端的内容不仅仅是监视器操作而已。如果您曾经接触过各种非智能终端，您肯定知道键盘上通常包含不同的键。对于终端模拟包而言，要模拟特定非智能终端上的特定键是很困难的。

PC 键盘的生产者不可能给非智能终端中的每个键都建造一个对应的键。有些 PC 制造商曾尝试建造特殊的键以用于特定功能，但最终 PC 键盘键还是标准化了。

终端模拟包若想完全模拟特定类型的非智能终端，它必须重新映射 PC 键盘上不存在的非智能终端键。这种重新映射功能很容易让人感到困惑，尤其在不同的系统对同一个键使用不同的控制码时。

终端模拟包中常见的特殊键包括以下几种。

- **BREAK**：向主机发送一串零。这通常用于中断 shell 当前执行的程序。
- **SCROLL LOCK**：也称为 NO SCROLL，终止显示器输出。有些终端使用内存保存显示的内容，因此在启动 SCROLL LOCK 时，用户可以回滚到之前查看过的信息。
- **Repeat**：与另一个键一起按下时，将导致终端向主机重复发送另一个键的值。
- **Return**：通常用于向主机发送回车字符。最常见的用法是表示命令结束，请求主机进行处理（现在在 PC 键盘上称为 Enter 键）。
- **Delete**：虽然这只是一个简单的功能，但 Delete 键着实让终端模拟包吃尽了苦头。有些终端删除当前光标位置的字符，而有些则删除前一个字符。为了解决这个难题，PC 键盘包含了两个删除键，Backspace 键和 Delete 键。
- **方向键**：通常用于将光标移动到特定位置。例如，滚动通过列表。
- **功能键**：一组专用的键，在程序中可以分配唯一的值，类似于 PC 的 F1 到 F12。DEC VT 系列终端实际上有两组功能键，F1 到 F20 和 PF1 到 PF4。

键盘模拟是终端模拟包中的关键部分。不幸的是，很多应用程序需要用户单击特定键才能实现特

定功能。我曾见过许多通信包使用老式 DEC 终端的 PF1 到 PF4 键，这在终端模拟包中通常很难找到。

## 2.2 terminfo 数据库

有了能够模拟各种终端的终端模拟包之后，需要用一种方式告诉 Linux 系统所模拟的究竟是哪种终端。Linux 需要了解与终端模拟器通信时应该使用哪种控制码。要做到这些，需要使用环境变量（参阅第 5 章）和一组特殊的文件集——**terminfo 数据库**。

terminfo 数据库是一组标识 Linux 系统中可以使用的各种终端特征的文件。Linux 系统在 terminfo 数据库目录下为每种终端类型使用一个单独的文件存储 terminfo 数据。该目录的位置通常随发行版的不同而不同。常见的位置有：`/usr/share/terminfo`、`/etc/terminfo` 和`/lib/terminfo`。

为了组织的方便（通常有很多不同的 terminfo 文件），terminfo 数据库目录中包含以不同字母为名称的目录。特定终端的文件存储在与该终端名的字母对应的目录下。

终端文件是编译文本文件得到的二进制文件。编译的文本文件中包含定义屏幕功能的代码字，以及在终端实现功能所需的控制码。

由于 terminfo 数据库文件是二进制的，因此无法在这些文件中看到代码。但是，可以使用 `infocmp` 命令将二进制内容转换为文本。使用该命令的示例如下：

```
$ infocmp vt100
# Reconstructed via infocmp from file: /lib/terminfo/v/vt100
vt100|vt100-am|dec vt100 (w/advanced video),
        am, msgr, xenl, xon,
        cols#80, it#8, lines#24, vt#3,
        acsc=''aaffggjjkkllmmnnooppqqrrssttuuvvwwxxyyzz{{||}}~~,
        bel=^G, blink=\E[5m$<2>, bold=\E[1m$<2>,
        clear=\E[H\E[J$<50>, cr=^M, csr=\E[%i%p1%d;%p2%dr,
        cub=\E[%p1%dD, cub1=^H, cud=\E[%p1%dB, cud1=^J,
        cuf=\E[%p1%dC, cuf1=\E[C$<2>,
        cup=\E[%i%p1%d;%p2%dH$<5>, cuu=\E[%p1%dA,
        cuu1=\E[A$<2>, ed=\E[J$<50>, el=\E[K$<3>, el1=\E[1K$<3>,
        enacs=\E(B\E)0, home=\E[H, ht=^I, hts=\EH, ind=^J, ka1=\E0q,
        ka3=\E0s, kb2=\E0r, kbs=^H, kc1=\E0p, kc3=\E0n, kcub1=\E0D,
        kcud1=\E0B, kcuf1=\E0C, kcuu1=\E0A, kent=\E0M, kf0=\E0y,
        kf1=\E0P, kf10=\E0x, kf2=\E0Q, kf3=\E0R, kf4=\E0S, kf5=\E0t,
        kf6=\E0u, kf7=\E0v, kf8=\E0l, kf9=\E0w, rc=\E8,
        rev=\E[7m$<2>, ri=\EM$<5>, rmacs=^O, rmam=\E[?7l,
        rmkx=\E[?1l\E>, rmso=\E[m$<2>, rmul=\E[m$<2>,
        rs2=\E>\E[?3l\E[?4l\E[?5l\E[?7h\E[?8h,
        sc=\E7,
        sgr0=\E[m\017$<2>, smacs=^N, smam=\E[?7h, smkx=\E[?1h\E=,
        smso=\E[7m$<2>, smul=\E[4m$<2>, tbc=\E[3g,
$
```

terminfo 条目定义终端名称（本例中为 vt100），以及与该终端名称关联的任何别名。注意，第一行显示提取值的终端文件所在的位置。

之后，`infocmp` 命令列出终端定义的功能，以及用于模拟每个功能的控制码。有些功能有可能启用也有可能禁用（比如 am、auto-right-margin 特性）。如果功能出现在列表中，则终端定义启动该功能。其他功能必须定义特定的控制码序列才能执行任务（比如清除监视器显示）。表 2-1 展示了 vt100 的 terminfo 定义文件中列出的部分功能。

表 2-1　　terminfo 功能代码

| 代码 | 描述 |
| --- | --- |
| am | 设置右侧自动边缘调整 |
| msgr | 以突出模式安全移动光标 |

续表

| 代码 | 描述 |
| --- | --- |
| Xenl | 忽略 80 列之后的换行符 |
| xon | 终端使用 XON/XOFF 字符进行流控制 |
| cols#80 | 一行 80 列 |
| it#8 | 将字符集移动 8 个空格 |
| lines#24 | 屏幕显示 24 行 |
| vt#3 | 虚拟终端数 3 |
| bel | 用于模拟铃声的控制码 |
| blink | 用于生成闪烁文本的控制码 |
| bold | 用于生成粗体文本的控制码 |
| clear | 用于清除屏幕的控制码 |
| cr | 用户输入回车的控制码 |
| csr | 用于更改滚动范围的控制码 |
| cub | 将一个字符向左移动，但不删除任何字符 |
| cub1 | 将光标回退一格 |
| cud | 将光标下移一行 |
| cud1 | 将光标下移一行的控制码 |
| cuf | 将一个字符向右移动，但不删除任何字符 |
| cuf1 | 将光标向右移动一格但不删除任何字符的控制码 |
| cup | 移动到显示器第一行第二列的控制码 |
| cuu | 将光标上移一行 |
| cuu1 | 将光标上移一行的控制码 |
| ed | 清除到屏幕底部 |
| el | 清除到一行的结尾 |
| el1 | 清除到一行的开头 |
| enacs | 启用备选字符集 |
| home | 将光标移动到主位置（第一行第二列）的控制码（与 cup 相同） |
| ht | 跳过字符 |
| hts | 在当前列的每一行设置跳过 |
| ind | 向上滚动文本 |
| ka1 | 小键盘的左上键 |
| ka3 | 小键盘的右上键 |
| kb2 | 小键盘的中心键 |
| kbs | 删除键 |
| kc1 | 小键盘的左下键 |
| kc3 | 小键盘的右下键 |
| kcub1 | 左方向键 |
| kcud1 | 下方向键控制码 |
| kcuf1 | 右方向键 |
| kcuu1 | 上方向键 |
| kent | 回车键 |
| kf0 | F0 键 |
| kf1 | F1 键 |
| kf10 | F10 键 |
| rc | 将光标还原到上次保存的位置 |
| rev | 反向显示模式 |

续表

| 代码 | 描述 |
| --- | --- |
| ri | 向下滚动文本 |
| rmacs | 终止备选字符集 |
| rmam | 关闭自动边缘调整 |
| rmkx | 退出键盘传输模式 |
| rmso | 退出突出模式 |
| rmul | 退出下划线模式 |
| rs2 | 重置 |
| sc | 保存当前光标位置 |
| sgr | 定义视频属性 |
| sgr0 | 关闭所有属性 |
| smacs | 启动备选字符集 |
| smam | 打开自动边缘调整 |
| smkx | 启动键盘传输模式 |
| smso | 启动突出模式 |
| smul | 启动下划线模式 |
| tbc | 清除所有制表位 |

Linux shell 使用 TERM 环境变量定义使用 terminfo 数据库中的那个终端模拟设置进行特定会话。当 TERM 设置为 vt100 时，shell 就知道应该使用与 vt100 terminfo 数据库条目关联的控制码向终端模拟器发送控制码。要查看 TERM 环境变量，可以通过 CLI 回显它：

```
$ echo $TERM
xterm
$
```

该示例表明当前终端类型设置为 terminfo 数据库中的 xterm 条目。

## 2.3 Linux 控制台

在 Linux 早期，启动系统时可以在监视器上看到一个登录提示，仅此而已。如前所述，这称为 Linux 控制台。只有这个地方才能输入系统命令。

现代 Linux 系统中，Linux 系统在启动时自动创建几个**虚拟控制台**（virtual console）。虚拟控制台是 Linux 系统内存中运行的终端会话。Linux 系统启动 7 个（有时更多）虚拟控制台而不是使用 6 个连接到 PC 的非智能终端，使用一个 PC 键盘和监视器可以访问虚拟控制台。

在大多数 Linux 发行版中，可以使用一个简单的键击组合访问虚拟控制台。一般使用 Ctl+Alt 组合键，然后再按功能键（F1 键到 F8 键），选择想使用的虚拟控制台。功能键 F1 生成虚拟控制台 1，F2 键生成虚拟控制台 2，依此类推。

前 6 个虚拟控制台使用全屏文本终端模拟器显示文本登录屏幕，如图 2-2 所示。

图 2-2 Linux 控制台登录屏幕

使用用户 ID 和密码登录后，将进入 Linux bash shell CLI。在 Linux 控制台无法运行任何图形程序。只能使用文本程序在 Linux 文本控制台上显示。

登录到虚拟控制台后，可以在保持该控制台激活状态的同时切换到另一个虚拟控制台，而不会丢失活动会话。可以在所有的虚拟控制台之间切换，运行多个活动会话。

最后两个虚拟控制台通常保留供 X Windows 图形桌面使用。有些发行版只分配其中一个，因此必须使用 Ctl+Alt+F7 组合键和 Ctl+Alt+F8 组合键查看发行版使用的是哪个虚拟控制台。大多数发行版在启动完成后自动切换到其中一个图形虚拟控制台，提供完整的图形登录和桌面体验。

如果总是要登录到文本虚拟终端会话然后再切换到图形虚拟会话，您会感到很乏味。幸运的是，有一种方法可以在 Linux 系统的图形和文本模式之间切换：从图形桌面会话中访问 shell CLI 的常用方法是终端模拟包。以下几节将介绍能在图形窗口中提供终端模拟的最常用的几个软件包。

## 2.4　xterm 终端

最古老最基础的 X Windows 终端模拟包是 xterm。xterm 包在 X Window 出现之初便已经存在了，默认包含在大多数 X Window 包中。

xterm 包提供基本的 VT102/220 终端模拟 CLI 和图形 Tektronix 4014 环境（类似于 4010 环境）。尽管 xterm 是一个完整的终端模拟包，但运行它不需要太多的资源（如内存）。由于这种特性，xterm 包在针对老硬件设计的 Linux 发行版中仍然很常见。有些图形桌面环境（比如 fluxbox）用它作为默认的终端模拟包。

尽管没有太多炫目的特性，但 xterm 包提供了一个非常好的功能，即模拟 VT220 终端。最新的 xterm 版本甚至能模拟 VT 系列的颜色控制码，因而能够在脚本中使用颜色（将在第 15 章讨论）。

图 2-3 展示在图形 Linux 桌面上运行的 xterm。

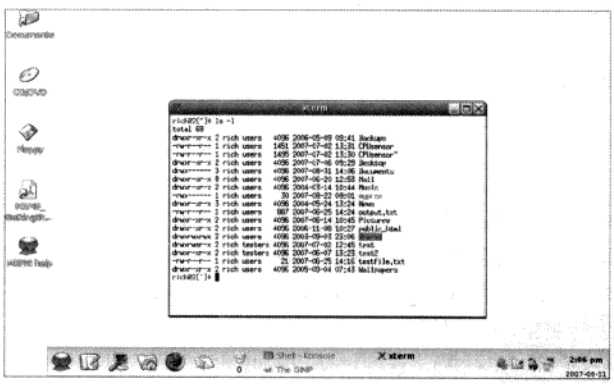



图 2-3　基本的 xterm 显示

xterm 包支持使用命令行参数和 4 个简单的图形菜单设置各项特性。以下几节讨论这些特性以及更改它们的方式。

### 2.4.1　命令行参数

xterm 命令行参数列表包含的内容很多。有许多特性可供选择以自定义终端模拟特性，比如启用或禁用各 VT 模拟。

xterm 命令行参数使用加号（+）和减号（–）表示如何设置特性。加号表示该特性应该还原为默认设置。减号表示将特性设置为非默认值。表 2-2 列出部分可以使用命令行参数设置的常见特性。

表 2-2 xterm 命令行参数

| 参数 | 描述 |
| --- | --- |
| 132 | 默认情况下，xterm 不允许每行 132 个字符的模式 |
| ah | 总是加亮文本光标 |
| aw | 启用自动换行 |
| bc | 启用文本光标闪烁 |
| bg color | 指定背景使用的颜色 |
| cm | 禁用 ANSI 颜色更改控制码的识别 |
| fb font | 指定粗体文本使用的字体 |
| fg color | 指定前景文本使用的颜色 |
| fn font | 指定文本使用的字体 |
| fw font | 指定宽度文本使用的字体 |
| hc color | 指定高亮文本使用的颜色 |
| j | 使用跳动滚动，即一次滚动多行 |
| l | 将屏幕数据记录到日志文件 |
| lf filename | 指定屏幕记录使用的文件名 |
| mb | 当光标到达行末尾时发出铃声 |
| ms color | 指定文本光标使用的颜色 |
| name name | 指定标题栏中显示的应用程序名称 |
| rv | 通过交换背景和前景颜色启用反向显示 |
| sb | 使用边滚动栏滚动保存的滚动数据 |
| t | 以 Tektronix 模式启动 xterm |
| tb | 指定 xterm 在顶部显示工具栏 |

要特别注意的一点是，并非所有 xterm 实现都支持所有这些命令行参数。如果在系统中启动了 xterm，可以使用-help 参数确定 xterm 实现了哪些参数。

## 2.4.2 xterm 主菜单

xterm 主菜单包括同时适用于 VT102 和 Tektronix 窗口的配置项。在 xterm 会话窗口中，按 Ctrl 键并单击一次鼠标按钮（右手鼠标的左键，左手鼠标的右键）即可访问主菜单。图 2-4 展示了 xterm 主菜单的外观。

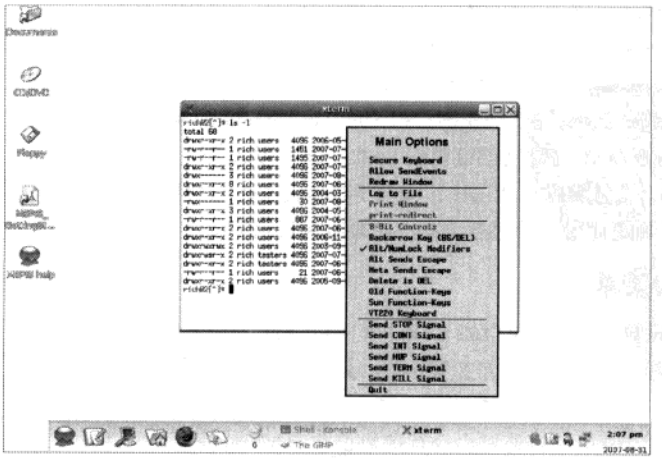



图 2-4 xterm 主菜单

xterm 主菜单有 4 个部分，下面几节将分别介绍它们。

**1. X 事件命令**

X 事件命令部分包含的特性用于管理 xterm 与 X Window 显示交互的方式。

- **Toolbar**：如果 xterm 安装支持工具栏，则该条目可以在 xterm 窗口中启用或禁用工具栏的显示（与 `tb` 命令行参数的作用一样）。
- **Secure Keyboard**：将键盘按键限制在特定的 xterm 窗口。这在输入密码时非常有用，可以确保密码不会被其他窗口劫持。
- **Allow SendEvents**：允许 xterm 窗口接收其他 X Window 应用程序生成的 X Window 事件。
- **Redraw Window**：指示 X Window 刷新 xterm 窗口。

同样，某些 xterm 实现可能不支持部分特性。如果不支持，菜单上将显示灰色。

**2. 输出捕获**

xterm 包能够捕获窗口中显示的数据——将它记录到一个文件中或者发送到 X Window 中定义的默认打印机。本节将讨论的特性有以下几种。

- **Log to file**：将 xterm 窗口中显示的所有数据发送到一个文本文件。
- **Print window**：将当前窗口中显示的所有数据发送到默认的 X Window 打印机。
- **Redirect to printer**：将 xterm 窗口中显示的所有数据发送到默认的 X Window 打印机。必须关闭该特性才能停止打印数据。

如果在显示区域使用图形字符或控制字符（比如彩色文本），则捕获功能可能会出现问题。所有发送到显示区域的字符（包括控制字符）都将存储到日志文件或发送到打印机。

xterm 打印特性假定在 X Window 系统中定义了默认打印机。如果没有定义打印机，则该特性在菜单中将显示为灰色。

**3. 键盘设置**

键盘设置部分包含的功能可以自定义 xterm 将键盘字符发送到主机系统的方式。

- **8-bit controls**：发送用于 VT220 终端的 8 位控制码，而不是 7 位 ASCII 控制码。
- **Backarrow key**：切换向后方向键的功能——发送一个退格字符或一个删除字符。
- **Alt/Numlock Modifies**：控制 Alt 键或 Numlock 键是否更改 PC 数字键盘的行为。
- **Alt Sends Escape**：Alt 键和另一个按键一同发送一个转义控制码。
- **Meta sends Escape**：控制功能键是否发送一个两字符控制码，包括转义控制码。
- **Delete is DEL**：PC Delete 键发送一个删除字符而不是退格字符。
- **Old Function keys**：PC 功能键模拟 DEC VT100 功能键。
- **HP Function keys**：PC 功能键模拟 HP 终端功能键。
- **Sun Funciton keys**：PC 功能键模拟 Sun Workstation 功能键。
- **VT220 keyboard**：PC 功能键模拟 DEC VT220 功能键。

正如您看到的，设置键盘首选项往往依赖于具体使用的应用程序和/或工作的环境。这还涉及许多个人的喜好。至于到底使用哪种键盘设置，以最适合自己情况的设置为准。

### 2.4.3 VT 选项菜单

VT 选项菜单设置 xterm 在 VT102 模拟中使用的特性。按住 trl 键然后单击第二个鼠标按钮即可访问 VT 选项。通常第二个鼠标按钮是鼠标中键。如果使用的是双按钮鼠标，大多数 Linux X Window 配置在同时单击鼠标左键和右键时模拟鼠标中键。图 2-5 展示了 VT 选项菜单的外观。

从图 2-5 中可以看到，许多通过命令行参数设置的 VT 特性也可以通过 VT 选项菜单设置。这生成了许多可用的选项。VT 选项分为 3 个命令集，我们将在下面几节进行介绍。

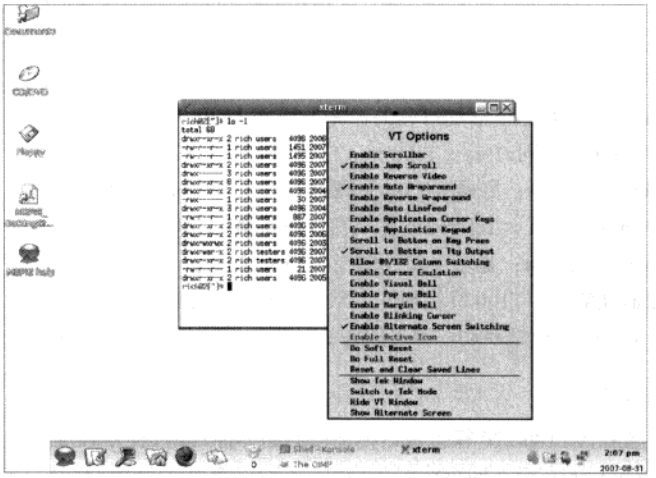



图 2-5 xterm VT 选项菜单

### 1. VT 特性

VT 特性命令更改 xterm 如何实现 VT102/220 模拟的特性。这些特性包括：

- Enable Scrollbar；
- Enable Jump Scrollbar；
- Enable Reverse Video；
- Enable Auto Wraparound；
- Enable Reverse Wraparound；
- Enable Auto Linefeed；
- Enable Application Cursor Keys；
- Enable Application Keypad；
- Scroll to Bottom on Keypress；
- Scroll to Bottom on TTY Output；
- Allow 80/132 Column Switching；
- Select to Clipboard；
- Enable Visual Bell；
- Enable Pop on Bell；
- Enable Margin Bell；
- Enable Blinking Cursor；
- Enable Alternate Screen Swithching；
- Enable Active Icon。

可以通过单击菜单的特性启用或禁用这些功能。启用的功能旁边将有一个复选标记。

### 2. VT 命令

VT 命令部分向 xterm 模拟窗口发送特定的重置命令。这些命令包括：

- Do Soft Reset；
- Do Full Reset；
- Reset and Clear Saved Lines。

Do Soft Reset 发送控制码重置屏幕区域。如果程序对滚动区域的设置不正确，则该命令非常方便。

Do Full Reset 清除屏幕，重置所有制表符位置，并将在会话期间设置的终端模式特性重置到初始状态。Reset and Clear Saved Lines 命令执行完全重置，同时清除滚动区域历史文件。

**3. 当前屏幕命令**

当前屏幕命令部分将影响当前活动屏幕的命令发送到 xterm 模拟器。

- **Show Tek Window**：同时显示 Tektronix 终端窗口和 VT100 终端窗口。
- **Switch to Tek Window**：隐藏 VT100 终端窗口，显示 Tektronix 终端窗口。
- **Hide VT Window**：显示 Tektronix 终端窗口时隐藏 VT100 终端窗口。
- **Show Alternate Screen**：显示当前存储在 VT100 备选屏幕区域的数据。

xterm 终端模拟器能够以 VT100 终端模式（默认）启动，也能够以 Tektronix 终端模式启动（使用 `t` 命令行参数）。无论以哪种模型启动，都可以在会话期间使用该菜单区域切换到另一个模式。

## 2.4.4　VT 字体菜单

VT 字体菜单设置在 VT100/220 模拟窗口中使用的字体样式。按住 Ctrl 键并单击第三个鼠标键（右手鼠标的右键或左手鼠标的左键）即可访问该菜单。图 2-6 展示了 VT 字体菜单的外观。

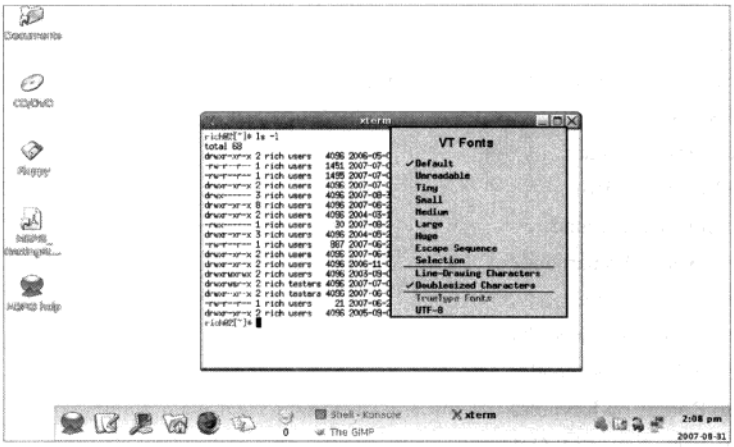



图 2-6　xterm VT 字体菜单

以下几节介绍 VT 字体菜单，共 3 部分。

**1. 设置字体**

这些菜单选项设置 xterm 窗口中使用的字体大小。可用的大小有：

- Default；
- Unreadable；
- Tiny；
- Small；
- Medium；
- Large；
- Huge；
- Escape the Sequence；
- Selection。

Default 字体是在当前 X Window 窗体中显示文本所使用的标准大小的字体。Unreadable 字体形如其名，它将 xterm 窗口收缩到很小，几乎没法使用。但是，如果希望将窗口在桌面最小化而不是在系

统最小化，则该字体非常实用。Large 和 Huge 字体选项为视力障碍的用户提供大字体。

Escape the Sequence 选项将字体设置为 VT100 设置字体控制码最后一次设置的字体。Selection 选项可以使用一个特殊的字体名称保存当前字体。

**2. 显示字体**

该菜单选项部分定义用于创建文本的字符类型。可用的选项有以下两种。

- **Line Drawing Characters**：通知 Linux 系统生成 ANSI 图形行而不是从选择的字体中使用行字符；
- **Doublesized characters**：通知 Linux 系统将字体扩大为普通大小的两倍。

Line Drawing Characters 可用于确定在文本模式绘图时使用的图形特性类型。可以使用字体源提供的字符，也可以使用 DEC VT100 控制码提供的字符。

**3. 指定字体**

该菜单部分提供创建字符所使用的字体类型选项：

- TrueType Fonts；
- UTF-8 Fonts。

TrueType 字体在图形环境中很常见。字符在一行中占有的空间不同，与它们的自然大小成正比。因此，字母 i 在一行中占用的空间比字母 m 要少。对于不支持外国字符的应用程序，可以使用 UTF-8 字体临时切换到 Unicode 字符集。

## 2.5 Konsole 终端

KDE 桌面项目创建了自己的终端模拟包，名为 **Konsole**。Konsole 包整合了基本的 xterm 特性以及一些更高级的类似 Windows 应用程序特性。本节介绍 Konsole 终端的特性，并说明如何使用这些特性。

### 2.5.1 命令行参数

Linux 发布版通常会提供一种方法直接从图形桌面菜单系统启动应用程序。如果您的发布版不支持这项功能，则可以通过以下命令手动启动 Konsole：

```
konsole parameters
```

与 xterm 一样，Konsole 包在新会话中使用命令行参数设置特性。表 2-3 展示了可用的 Konsole 命令行参数。

表 2-3　　Konsole 命令行参数

| 参数 | 描述 |
|---|---|
| -e command | 执行命令而不是 shell |
| --keytab file | 使用指定的密钥文件定义密钥映射 |
| --keytabs | 列出所有可用的密钥表 |
| --ls | 在登录屏幕启动 Konsole 会话 |
| --name name | 设置 Konsole 标题栏中出现的名称 |
| --noclose | 在最后一个会话关闭后禁止关闭 Konsole 窗口 |
| --noframe | 启动 Konsole，而不启动窗体 |
| --nohist | 禁止 Konsole 在会话中保存滚动历史数据 |
| --nomenubar | 启动 Konsol，而不启动标准的菜单栏选项 |
| --noresize | 禁止更改 Konsole 窗口区域的大小 |
| --notabbar | 启动 Konsole，而不启动会话的标准选项卡区域 |

续表

| 参数 | 描述 |
| --- | --- |
| --noxft | 启动 Konsole，而不支持更小的字体 |
| --profile file | 启动 Konsole，使用指定文件中保存的设置 |
| --profiles | 列出所有可用的 Konsole 配置文件 |
| --schema name | 使用指定的模式名称或文件启动 Konsole |
| --schemata | 列出 Konsole 中可用的模式 |
| -T title | 设置 Konsole 窗口标题 |
| --type type | 使用指定的类型启动 Konsole 会话 |
| --types | 列出所有可用的 Konsole 会话类型 |
| --vt_sz CxL | 指定终端列（C）和行（L） |
| --workdir dir | 指定 Konsole 存储临时文件的工作目录 |

## 2.5.2 会话

启动 Konsole 时，注意到它有一个选项卡窗口，其中一个选项卡打开终端模拟会话。这是默认的会话，通常是标准的 bash shell CLI。可以为 Konsole 定义许多不同的会话，甚至能同时打开多个会话。

Konsole 的默认配置包括几个可以通过 Konsole 窗口访问的不同类型的会话：

- 使用 xterm 终端模拟的标准 shell 会话；
- 使用文本模式终端模拟的 Linux console 会话；
- 作为 Linux 超级用户登录的 shell 会话（必须应用密码）；
- Midnight Commander 文件管理会话（如果发行版中已安装）；
- 使用 Python 交互 CLI 的会话，可测试使用 Python 语言编写的程序（如果发行版中安装了 Python）；
- 保存的书签区域中启动的 shell（稍后将在本章的“菜单栏”一节讨论）。

Konsole 允许同时使用多个会话。每个会话都包含在各自的选项卡窗口中。Konsole 在每个会话窗口放置一个选项卡，以方便在会话之间进行切换。您可能发现了，选项卡不是在窗口区域的顶部就是在底部。这对于需要在会话中编辑代码，而又需要在另一个会话中测试的程序员来说非常好。在 Konsole 中，可以轻松地在各个活动会话之间来回切换。图 2-7 展示了一个带有 3 个活动会话的 Konsole 窗口。

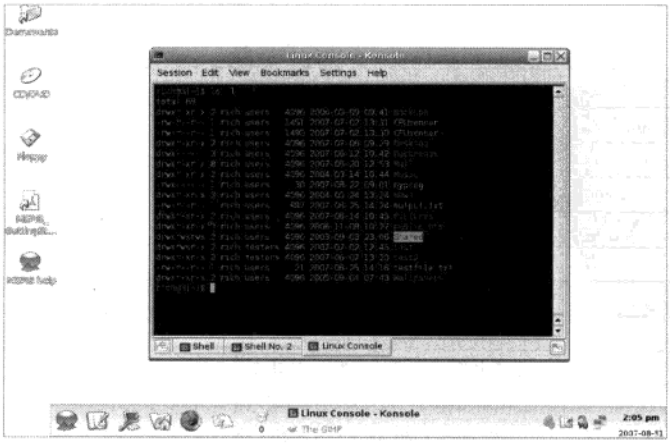



图 2-7 带有 3 个活动会话的 Konsole 终端模拟器

在选项卡区域的左边有一个新的会话按钮。单击该按钮可以自动使用标准 shell 启动一个新的会话。单击该按钮不放，可以看到一个会话菜单，该菜单允许选择要启动的新会话的类型。

与 xterm 终端模拟器类似，Konsole 也提供一个简单的菜单，在活动会话区域右键单击即可出现。如果在会话区域右键单击，则会出现一个带有以下选项的菜单。

- **Set Selection End**：在光标到鼠标指针位置之间选择会话窗口区域。
- **Copy**：将选定的文本复制到剪贴板。
- **Paste**：将剪贴板的内容粘贴到选定区域。
- **Send Singal**：将 Linux 控制信号发送到系统。
- **Detach session**：将变为选项卡的会话移动到新的 Konsole 会话窗口（仅在有多个会话活动时可用）。
- **Rename session**：更改会话的 X Windows 名称。
- **Bookmarks**：在当前会话位置添加一个会话书签。以后可以在另一个会话中再调用该书签返回到相同的目录。
- **Close session**：终止会话。如果这是 Konsole 窗口中的最后一个会话，则 Konsole 将关闭。

Konsole 还提供另一种快速访问新会话菜单的方式，即按住 Ctrl 键并在会话区域中右键单击。

除了会话选项卡和新的会话按钮之外，默认情况下，Konsole 还使用一个菜单栏提供附加功能，以便修改并保存 Konsole 会话。

### 2.5.3 菜单栏

默认的 Konsole 设置使用一个菜单栏轻松查看和更改会话的特性与选项。菜单栏由 6 个条目组成，将在以下几节介绍。

#### 1. Session

Session 菜单栏条目提供了另一个启动新会话类型的位置。除了列出标准会话类型以启动新会话外，它还包括以下条目：

- 启动一个新 Konsole 窗口，同时启动一个默认会话；
- 打印当前会话屏幕；
- 关闭当前会话；
- 退出 Konsole 应用程序。

从菜单中选择其中一个会话类型时，新的会话将作为一个新的选项卡式窗口出现在 Konsole 窗口中。

#### 2. Edit

Edit 菜单栏提供处理会话中文本的选项。

- **Copy**：将选定文本（使用鼠标选中）复制到系统剪贴板。
- **Paste**：将系统剪贴板中的当前文本粘贴到当前光标位置。如果文本包含新行字符，则交由 shell 处理。
- **Send Signal**：将 Linux 控制信号发送到系统。可用的控制信号有以下几种。
  - **STOP**：终止当前运行的程序；
  - **CONT**：中断时继续；
  - **HUP**：重置当前运行的程序；
  - **INT**：中断当前运行的程序；
  - **TERM**：终止当前会话；
  - **KILL**：销毁当前会话；

- **USR1**：用户定义的信号 1；
- **USR2**：用户定义的信号 2。

- **ZModem Upload**：使用 ZModem 协议将文件上传到系统。
- **Clear terminal**：清除当前会话窗口的所有文本。
- **Reset and Clear Terminal**：发送控制码重置终端模拟器，并清除当前会话窗口。
- **Find in History**：在会话输出文本的上几行查找文本字符串。
- **Find Next**：在会话输出文本的上几行查找下一次出现的文本字符串。
- **Find Previous**：在会话输出文本的上几行查找上一次出现的文本字符串。
- **Save History As**：将当前历史数据保存为文件。
- **Clear History**：清除会话输出文本的上几行。
- **Clear All Histories**：清楚所有会话输出文本的上几行。

Konsole 为每个活动会话保留一个历史区域。历史区域包含那些滚动出终端模拟器的输出文本行。默认情况下，Konsole 在历史区域中保留最近的 1000 个输出行。可以在查看区域使用滚动栏滚动查看历史区域，也可以按住 Shift 键和向上方向键逐行滚动，或者按住 Page Up 键按页滚动（一页 24 行）。

### 3. View

View 菜单栏包括控制 Konsole 窗口中的各个会话的条目。这些选项包括以下几种。

- **Detach Session**：从 Konsole 窗口中移除当前会话，并使用当前会话作为默认会话启动一个新的 Konsole 窗口。仅当有多个活动会话时可用。
- **Rename Session**：更改当前会话的名称。新名称出现在会话选项卡中，以便轻松标识选项卡。
- **Monitor for Activity**：设置会话，以便在屏幕区域出现新文本时会话选项卡显示一个特殊图标。这样便可以在等待应用程序输出时切换到另一个会话，然后在输出完成时获得通知。该特性在开和关之间切换。
- **Monitor for Silence**：设置会话，使会话选项卡在持续 10 秒没有在屏幕区域出现新文本时显示一个特殊图标。这样便可以在等待应用程序输出完成前（比如编译一个大型应用程序）切换到另一个会话。该特性在开和关之间切换。
- **Send Input to All Sessions**：将一个会话中键入的文本发送到所有活动会话。
- **Move Session Left**：将当前会话选项卡向窗口列表左移。
- **Move Session Right**：将当前会话选项卡向窗口列表右移。

在标准菜单选项卡后，View 菜单栏包含了一个当前活动会话列表。可以选择特定的会话图标在会话之间进行切换。

### 4. Bookmarks

Bookmarks 菜单项提供了一种管理 Konsole 窗口书签设置的方法。书签可以在活动会话中保存目录位置，然后可以在同一个会话或者新的会话中轻松返回该位置。您有没有遇到过这样的情况：深入 Linux 系统的目录查找某个内容，但退出后又忘记了该内容所在的位置？Bookmarks 菜单可以解决这个问题。在到达所需的目录位置后，添加一个新书签。想返回时，只需要在 Bookmarks 中查看新的书签，它可以自动将目录更改为所需的目录。Bookmarks 菜单项包括以下几种。

- **Add Bookmark**：在当前目录位置创建一个新书签。
- **Edit Bookmarks**：编辑现有书签。
- **New Bookmark Folder**：为书签创建一个新的存储文件夹。

Konsole 中存储的书签数量没有限制，但是书签过多可能会比较混乱。默认情况下，它们都在

Bookmark 区域的同一级别显示。若要整理书签，可以创建新的书签文件夹，使用 Edit Bookmarks 选项将各个书签移动到新的文件夹。

5. Settings

Settings 菜单栏区域用于自定义特定会话的显示形式。该区域包括以下几部分。

- **Hide Menubar**：从 Konsole 窗口移除菜单栏。
- **Tab bar**：将窗口选项卡放在窗口的上方或下方，或者隐藏它们。
- **Scrollbar**：将滚动条放置在窗口的右边或左边，或者隐藏它们。
- **Fullscreen**：切换全屏模式（类似于 Linux 控制台）和窗口模式。
- **Bell**：设置铃声控制码的动作。该选项可设置可听到的声音，通知 Linux 系统，闪动 Konsole 窗口，或者什么也不做。
- **Font**：设置字符字体、样式和大小。
- **Encoding**：选择要在终端模拟中使用的字符集。
- **Keyboard**：选择用于会话的键盘映射文件。
- **Schema**：选择用于背景和文本颜色的颜色方案。
- **Size**：设置会话窗口中可见的行和列的数量。
- **History**：设置历史滚动区域中保存的数据量（如果有数据）。
- **Save as Default**：将当前会话配置作为默认值保存。
- **Save Session Profile**：将当前打开的会话保存为配置文件，以供将来重新调用。
- **Configure Notifications**：设置特定会话事件的动作。
- **Configure Shortcuts**：为 Konsole 命令创建键盘快捷键。
- **Configure Konsole**：创建自定义 Konsole 模式和会话。

您会发现，大多数设置在关于终端模拟器的介绍中已经出现过。Konsole 可以选择字符集、键盘布局和颜色，甚至能够控制铃声控制码在 Konsole 会话窗口中的模拟方式。

Configure Notifications 区域非常先进，它可以将出现在会话中的 5 个特定事件与 6 个不同的动作关联。当其中一个事件发生时，就会采取相应的行动。

Configure Konsole 设置可以对会话特性进行高级设置，包括创建新的颜色模式。图 2-8 展示了 Configure Konsole 的主对话框。

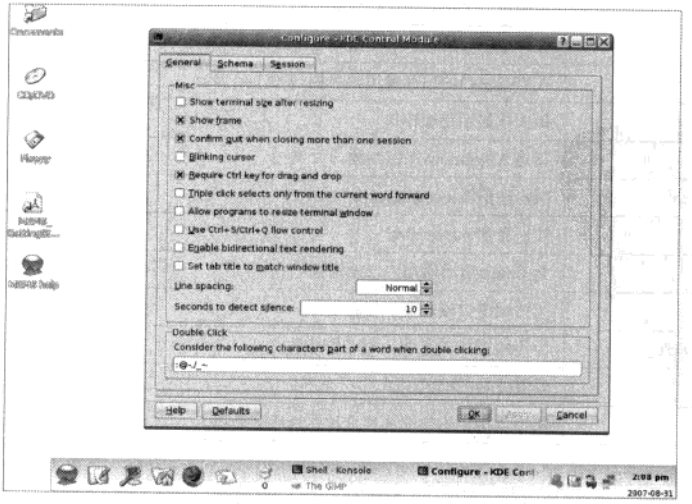



图 2-8 Konsole 配置对话框

在配置对话框中有 3 个选项卡。

- **General**：可以设置各种终端模拟特性，比如闪烁光标等。还可以使用控制码来运行程序以调整终端窗口大小，设置行间隔，设置会话从活动到不再活动之间的秒数。
- **Schema**：可以保存会话的颜色方案，以供将来在会话中使用。
- **Session**：可以配置新的和现有的 Konsole 会话。可以配置使用标准 shell 或特定命令（比如编辑器）启动的新会话。

6. Help

Help 菜单项提供完整的 Konsole 手册（如果 Linux 发行版中安装了 KDE 手册），“每日提示”特性可以在每次启动 Konsole 时提供一些鲜为人知的快捷键和技巧，此外还有一个标准的 About Konsole 对话框。

## 2.6　GNOME 终端

同样，GNOME 也有自己的终端模拟程序。GNOME 终端软件包的特性与 Konsole 和 xterm 有许多相似之处。本节将介绍配置和使用 GNOME 终端的各个部分。

### 2.6.1　命令行参数

GNOME 终端应用程序也提供了大量的命令行参数，启动后可以通过这些参数控制它的行为。表 2-4 列出了可用的参数。

表 2-4　　GNOME 终端命令行参数

| 参数 | 描述 |
|---|---|
| `-e command` | 在默认终端窗口中执行参数 |
| `-x` | 在默认终端窗口中执行该参数之后的整个命令行内容 |
| `--window` | 在默认终端窗口中打开一个新窗口。可以添加多个`--window`参数启动多个窗口 |
| `--window-with-profile=` | 使用指定配置文件打开一个新窗口。此参数也可以向命令行多次添加 |
| `--tab` | 在上次打开的终端窗口中打开一个新的选项卡化的终端 |
| `--tab-with-profile=` | 使用指定配置文件在上次打开的终端窗口中打开一个新的选项卡化的终端 |
| `--role` | 为最后一个指定的窗口设置角色 |
| `--show-menubar` | 在终端窗口的顶部启动菜单栏 |
| `--hide-menubar` | 在终端窗口的顶部禁用菜单栏 |
| `--full-screen` | 最大化显示终端窗口 |
| `--geometry=` | 指定 X Window 几何参数 |
| `--disable-factory` | 禁止注册激活 nameserver |
| `--use-factory` | 注册激活 nameserver |
| `--startup-id=` | 为 Linux 启动通知协议设置 ID |
| `-t, --title=` | 为终端窗口设置窗口标题 |
| `--working-directory=` | 为终端窗口设置默认的工作目录 |
| `--zoom=` | 设置终端的缩放因子 |
| `--active` | 将上次指定的终端选项卡设置为活动选项卡 |

GNOME 终端命令行参数在 GNOME 终端启动时自动设置许多特性。但是，大多数特性还是可以在启动后在 GNOME 终端窗口中设置。

### 2.6.2 选项卡

GNOME 终端将每个会话称为一个“选项卡”，因为它使用选项卡跟踪运行在窗口中的多个会话。图 2-9 展示了带有 3 个活动会话选项卡的 GNOME 终端窗口。

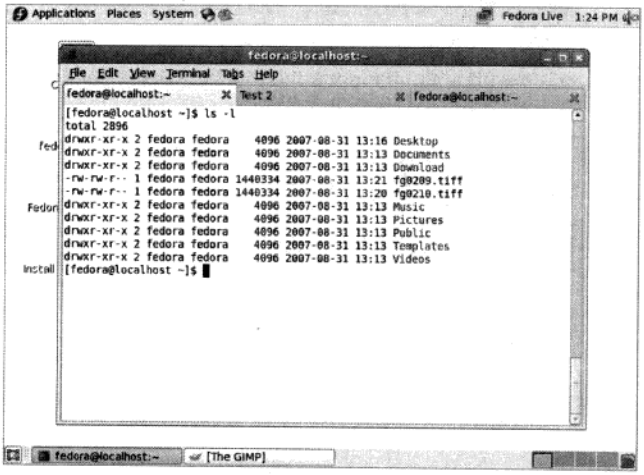



图 2-9 带有 3 个活动会话的 GNOME 终端

可以在选项卡窗口右键单击以查看选项卡菜单。这个快捷菜单提供了一些可以在选项卡会话中执行的操作。

- **Open Terminal**：打开一个新的 GNOME 终端窗口，该窗口带有一个默认选项卡会话。
- **Open Tab**：在现有 GNOME 终端窗口中打开一个新的会话选项卡。
- **Close Tab**：关闭当前会话选项卡。
- **Copy**：将当前会话选项卡中选中的文本复制到剪贴板。
- **Paste**：将剪贴板中的数据粘贴到当前会话选项卡的当前光标位置。
- **Change Profile**：更改当前会话选项卡的配置文件。
- **Edit Current Profile**：编辑当前会话选项卡的配置文件。
- **Show Menubar**：切换菜单栏的隐藏或显示状态。

快捷菜单可以轻松访问终端窗口的标准菜单栏中常用的动作。

### 2.6.3 菜单栏

GNOME 终端的主要操作使用菜单栏执行。菜单栏包括所有定义 GNOME 终端所需的配置和自定义选项。以下几节介绍菜单栏中的各种选项。

1. File

File 菜单选项包括创建和管理终端选项卡的各种选项。

- **Open Termianl**：在新的 GNOME 终端窗口中启动一个新的 shell 会话。
- **Open Tab**：在现有 GNOME 终端窗口的新选项卡上启动一个新的 shell 会话。
- **New Profile...**：自定义选项卡会话并保存为一个配置文件，以供将来重新调用。
- **Close Tab**：关闭窗口中的当前选项卡。
- **Close Window**：关闭当前 GNOME 终端会话，关闭所有活动选项卡。

大多数 File 菜单中的选项还可以通过右键单击会话选项卡区域查看。New Profile 选项可以自定义

会话选项卡设置，并保存它们供以后使用。

New Profile 首先请求一个新配置文件的名称，然后生成 Editing Profile 对话框，如图 2-10 所示。

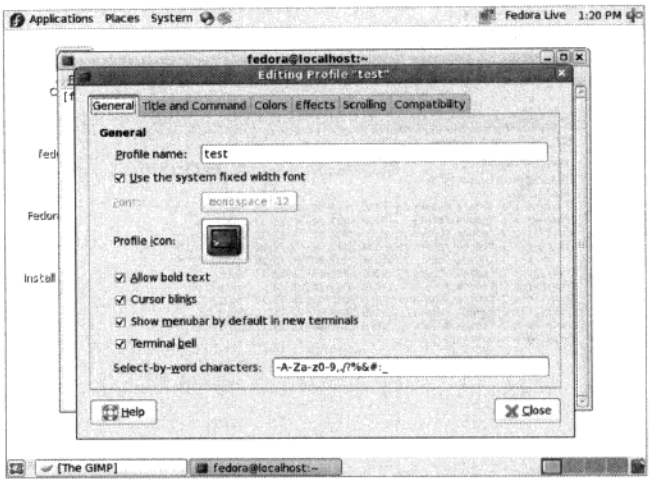



图 2-10　GNOME 终端 Editing Profile 对话框

该区域中，可以设置会话的终端模拟特性。它包括 6 个选项：

- **General**：提供常规设置，比如字体、铃声和菜单栏。
- **Title and Command**：可以设置会话选项卡的标题（在选项卡显示），并确定会话是否使用特殊命令而不是 shell 进行启动。
- **Colors**：设置会话选项卡使用的前景颜色和背景颜色。
- **Effects**：设置会话选项卡的背景图，或者让它透明以便透过会话选项卡查看桌面。
- **Scrolling**：控制是否创建滚动区域及其大小。
- **Compatibility**：设置 Backspace 和 Delete 键将向系统发送哪些控制码。

配置好配置文件后，可以在打开新会话选项卡时指定它。

2. Edit

Edit 菜单项包含可以在选项卡中处理文本的选项。可以使用鼠标在选项卡窗口的任何地方复制和粘贴文本。使用这些菜单可以轻松将命令行输出复制到剪贴板，然后导入到编辑器。还可以将另一个 GNOME 应用程序的文本复制到选项卡会话。

- **Copy**：将选择的文本复制到 GNOME 剪贴板。
- **Paste**：将 GNOME 剪贴板中的文本复制到选项卡会话。
- **Profiles...**：添加、删除或修改 GNOME 终端中的配置文件。
- **Keyboard Shortcuts...**：创建键组合，快速访问 GNOME 终端特性。
- **Current Profile...**：提供编辑当前会话选项卡所用配置文件的快捷方法。

配置文件编辑特性是一个强大的工具，它可以自定义几个配置文件，然后在更改会话时更改配置。

3. View

View 菜单选项控制会话选项卡显示的方式。它们包括以下选项。

- **Show Menubar**：显示或隐藏菜单栏；
- **Full Screen**：将 GNOME 终端窗口放大为整个桌面大小；
- **Zoom In**：放大会话窗口的字体；
- **Zoom Out**：缩小会话窗口的字体；

- **Normal Size**：将会话字体返回默认大小。

如果隐藏了菜单栏，在任何会话选项卡中单击右键，然后切换 Show Menubar 选项就可以轻松显示窗口。

4. Terminal

Terminal 菜单选项包含控制选项卡会话终端模拟特性的选项。它们包括以下选项。

- **Change Profile**：切换到会话选项卡中另一个配置好的配置文件。
- **Set Title...**：在会话选项卡上设置标题以便识别。
- **Set Character Encoding**：选择用于发送和显示字符的字符集。
- **Reset**：将重置控制码发送到 Linux 系统。
- **Reset and Clear**：将重置控制码发送到 Linux 系统，并清除任何当前出现在选项卡区域中的文本。

字符编码提供一个包含许多可供选择的字符集的列表。在必须处理非英语语言时，这非常方便。

5. Tabs

Tabs 菜单项提供控制选项卡位置和选择激活哪个选项卡的选项。

- **Previous Tab**：激活列表中的前一个选项卡。
- **Next Tab**：激活列表中的下一个选项卡。
- **Move Tab to the Left**：将当前选项卡拖动到上一个选项卡前方。
- **Move Tab to the Right**：将当前选项卡拖动到下一个选项卡前方。
- **Detach Tab**：移除选项卡并使用该选项卡会话启动一个新的 GNOME 终端窗口。
- **The Tab list**：列出当前在终端窗口中运行的会话选项卡。选择一个选项卡可快速切换到该会话。

该部分可以管理选项卡，一次打开了多个选项卡时非常方便。

6. Help

Help 菜单选项提供完整的 GNOME 终端手册，可以搜索 GNOME 终端中使用的各个选项和特性。

## 2.7 小结

要学习 Linux 命令行命令，首先要能访问命令行。在图形界面中，有时候这很难做到。本章介绍了从图形桌面环境访问 Linux 命令行时需要考虑的事情。文章首先介绍了终端模拟，然后展示了为保证 Linux 系统能与终端模拟包正常通信，以及正常显示文本和图形而必须了解的特性。

介绍完终端模拟器之后，我们讨论了 3 种不同的终端模拟器。xterm 终端模拟器包是第一个可用于 Linux 的模拟器包。它模拟 VT102 和 Tektronix 4014 终端。KDE 桌面项目创建了 Konsole 终端模拟包。它提供了许多高级功能，比如在同一个窗口包含多个会话，使用控制台和 xterm 会话，从而完全控制终端模拟参数。

最后，本章讨论了 GNOME 桌面项目的 GNOME 终端模拟包。GNOME 终端也可以在一个窗口中包含多个终端会话，此外还提供设置各种终端特性的便捷方式。

在下一章中，将开始介绍 Linux 命令行命令。另外，将逐一介绍导航 Linux 文件系统，以及创建、删除和操作文件所需的各种命令。

CHAPTER 3

# 第 3 章
# 基本的 bash shell 命令

本章内容：

- 查看 bash shell；
- 阅读手册；
- 浏览文件系统；
- 处理文件和目录；
- 查看文件内容。

所有的 Linux 发行版中默认使用的 shell 都是 GNU bash shell。本章介绍 bash shell 中的基本特性，并逐步介绍如何使用 bash shell 提供的基本命令处理 Linux 文件和目录。如果您已经能够熟练的在 Linux 环境中处理文件和目录，则可以跳过本章，继续学习第 4 章中更加高级的命令。

## 3.1 启动 shell

GNU bash shell 是一个可以交互访问 Linux 系统的程序。它的运行方式与常规程序相同，通常在用户登录终端时启动。系统启动的 shell 取决于用户 ID 配置。

/etc/passwd 文件包含一个所有系统用户账户组成的列表，以及一些关于每个用户的基本配置信息。以下是一个 /etc/passwd 文件中的示例条目：

```
rich:x:501:501:Rich Blum:/home/rich:/bin/bash
```

每个条目都有 7 个数据字段，使用冒号分隔。系统使用这些字段中的数据向用户分配具体的特性。这些字段有：

- 用户名；
- 用户密码（如果密码存储在其他文件中，则为占位符）；
- 用户的系统用户 ID 编号；
- 用户的系统组 ID 编号；
- 用户的全名；
- 用户的默认主目录；
- 用户的默认 shell 程序。

大部分字段将在第 6 章中详细讨论。现在只针对指定的几个 shell 程序进行重点论述。

大部分 Linux 系统在启动用户命令行接口（CLI）环境时使用默认的 bash shell。bash 程序还可以使用命令行参数修改启动的 shell 类型。表 3-1 列出了 bash 中可用于定义所用 shell 类型的命令行。

表 3-1 bash 命令行参数

| 参数 | 描述 |
| --- | --- |
| -c string | 从字符串读取命令并进行处理 |
| -r | 启动受限制 shell，将用户限制在默认目录 |
| -i | 启动交互 shell，允许用户的输入 |
| -s | 从标准输入中读取命令 |

默认情况下，当 bash shell 启动时，它将自动执行位于用户主目录的.bashrc 文件中的命令。许多 Linux 发行版还使用该文件加载一个包含系统所有命令和设置的通用文件。该通用文件通常加载到 /etc/bashrc 文件。该文件经常用于设置各种应用程序中使用的环境变量（在第 5 章介绍）。

## 3.2 shell 提示符

启动终端模拟包或者从 Linux 控制台登录后，便可以访问 shell CLI **提示符**。提示符是通往 shell 的大门，是输入 shell 命令的地方。

bash shell 的默认提示符是美元符号（$）。该符号表示 shell 等待输入文本。但是，也可以改变 shell 所用的提示符格式。不同的 Linux 发行版使用不同的提示符格式。在我的 SimplyMEPIS Linux 系统中，bash shell 提示符如下：

```
rich@1[~]$
```

在我的 Fedora Linux 系统中，提示符如下：

```
[rich@testbox ~]$
```

可以配置提示符使之提供环境的基本信息。上述第一个示例在提示符中显示了 3 条信息：

- 启动 shell 的用户名；
- 当前虚拟控制台编号；
- 当前目录（波浪号是主目录的简写表示法）。

第二个示例提供了类似的信息，不同之处在于，它使用主机名取代虚拟控制台编号。控制命令行提示符的环境变量有两个。

- PS1：控制默认命令行提示符的格式；
- PS2：控制第二层命令行提示符的格式。

对于在 shell 中的初次数据输入，shell 使用默认的 PS1 提示符。如果输入一个需要附加信息的命令，shell 将显示 PS2 环境变量指定的第二层提示符。

要显示提示符的当前设置，可以使用 echo 命令：

```
rich@1[~]$ echo $PS1
\u@\l[\W]\$
rich@1[~]$ echo $PS2
>
rich@1[~]$
```

提示符环境变量的格式看起来可能很奇怪。shell 使用特殊字符表示命令行提示符中的元素。表 3-2 展示您可以在提示符字符串中使用的特殊字符。

表 3-2 bash shell 提示符字符

| 字符 | 描述 |
| --- | --- |
| \a | 铃声字符 |
| \d | 格式为“日 月 年”的日期 |
| \e | ASCII 转义字符 |

续表

| 字符 | 描述 |
| --- | --- |
| \h | 本地主机名 |
| \H | 完全合格的限定域主机名 |
| \j | shell 当前管理的作业数 |
| \l | shell 终端设备名的基本名称 |
| \n | ASCII 换行字符 |
| \r | ASCII 回车 |
| \s | shell 的名称 |
| \t | 格式为“小时:分钟:秒”的 24 小时制的当前时间 |
| \T | 格式为“小时:分钟:秒”的 12 小时制的当前时间 |
| \@ | 格式为 am/pm 的 12 小时制的当前时间 |
| \u | 当前用户的用户名 |
| \v | bash shell 的版本 |
| \V | bash shell 的发布级别 |
| \w | 当前工作目录 |
| \W | 当前工作目录的基本名称 |
| \! | 该命令的 bash shell 历史数 |
| \# | 该命令的命令数量 |
| \$ | 如果是普通用户，则为美元符号；如果超级用户，则为英镑符号 |
| \nnn | 对应于八进制值 nnn 的字符 |
| \\ | 斜杠 |
| \[ | 控制码序列的开头 |
| \] | 控制码序列的结尾 |

注意，所有特殊提示符字符均以反斜杠（\）开头。这将提示符字符与普通文本区分开来。在前面的示例中，提示符既包含了提示符字符又包含了普通字符（@符号和中括号）。您可以在提示符中使用任何提示符字符的组合。要创建一个新的提示符，只需给 PSI 变量赋值一个新的字符串：

```
[rich@testbox ~]$ PS1="[\t][\u]\$"
[14:40:32][rich]$
```

新的 shell 提示符现在可以显示当前时间和用户名。新的 PS1 定义只在 shell 会话期间有效。启动一个新的 shell 时，将重新加载默认的 shell 提示符定义。第 5 章将介绍如何更改所有 shell 会话的默认 shell 提示符。

## 3.3 bash 手册

大部分 Linux 发行版都包含一个在线手册，用于查询 shell 命令的信息以及发行版中包含的众多其他 GNU 实用程序的信息。熟悉手册非常重要，对于实用程序的使用很有价值，尤其有助于了解各种命令行参数的作用。

man 命令可以访问存储在 Linux 系统中的手册页面。输入 man 命令，后跟特定的实用程序名称可以打开针对该实用程序的手册条目。图 3-1 展示了查询 date 命令手册页面的示例。

手册页面将有关命令的信息分为几个部分，如表 3-3 所示。

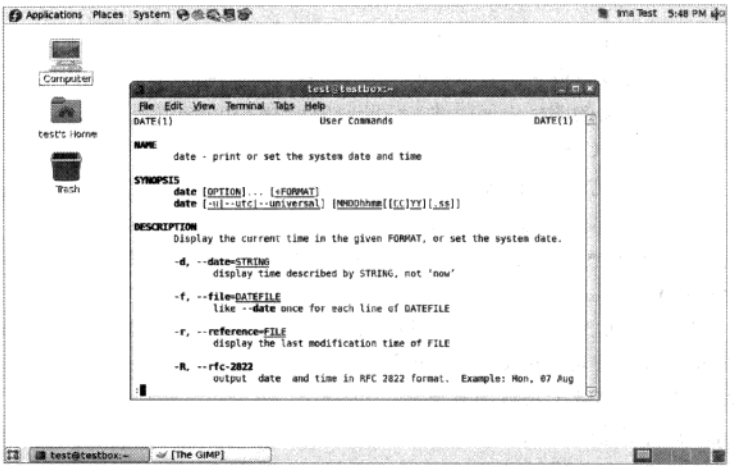



图 3-1 展示 Linux date 命令的手册页面

表 3-3 Linux 手册页面格式

| 部分 | 描述 | 部分 | 描述 |
|---|---|---|---|
| Name | 展示命令行名称和简短的描述 | Reporting bugs | 提供报告 bug 的信息 |
| Synopsls | 展示命令行的格式 | Copyright | 提供命令代码版权状况的信息 |
| Description | 描述每个命令选项 | See Also | 为您提供类似命令 |
| Author | 提供该命令开发人员的信息 | | |

按空格键可以翻页，也可以使用方向键滚动页面文本（要使用该功能，您的终端模拟包必须支持方向键功能）。

要查看 bash shell 的信息，可以使用以下命令查看手册页面：

```
$ man bash
```

该命令可以展示 bash shell 的整个手册页面。在构建脚本时该命令非常有用，因为您不必参考相关书籍或 Internet 网站以查询命令的具体格式。会话中可以方便的查看该手册。

## 3.4 文件系统导航

从 shell 提示符中可以看到，启动 shell 会话时，所在的位置一般都是主目录。通常情况下，需要离开主目录转到 Linux 系统的其他区域。本节介绍如何使用命令行命令实现这一点。在这之前，我们先大致了解一下 Linux 文件系统，以明确我们的目的。

### 3.4.1 Linux 文件系统

Linux 系统新手可能很难弄清楚它引用文件和目录的方式，对于习惯于 Microsoft Winodws 操作系统方式的人来说更是如此。在深入研究 Linux 系统之前，首先了解一下它的构建方式。

第一个不同之处在于，Linux 的路径名中不使用驱动器盘符。在 Windows 中，PC 上安装的物理驱动器决定了文件的路径名。Windows 为每个物理磁盘驱动器分配一个盘符，每个驱动器包含自己的目录结构，以便访问存储的文件。

例如，在 Windows 中，经常可以见到以下形式的文件路径：

```
c:\Documents and Settings\Rich\My Documents\test.doc.
```

该路径表示文件 `test.doc` 存储在目录 `My Documents` 中，而该目录本身又位于 `rich` 目录中。

rich 目录则包含在 Documents and Settings 下，而 Documents and Settings 位于盘符为 C 的硬盘分区（通常是 PC 的第一个硬盘）上。

Windows 文件路径可以告诉您包含 test.doc 文件的确切物理磁盘分区。如果您希望在软盘上存储文件，则应该单击 A 驱动器的图标，使用的文件路径自动为 a:\test.doc。该路径表示文件位于盘符为 A 的驱动器（通常是 PC 软盘）的根目录。

Linux 使用的不是这种方法。Linux 在一个目录结构中存储文件，该目录成为虚拟目录。虚拟目录包含 PC 上安装的所有存储设备的文件路径，它们都合并为一个目录结构。

Linux 虚拟目录结构包含一个基本目录，成为根。根目录以下的目录和文件根据到达它们的目录路径列出，这一点与 Windows 类似。

Tip 提示：注意到 Linux 使用正斜杠（/）而不是反斜杠（\）表示文件路径的目录。Linux 中的反斜杠字符表示转义字符，如果在文件路径中使用会造成很多问题。习惯了 Windows 环境的人可能需要一些时间来适应。

例如，Linux 文件路径/home/rich/Documents/test.doc 表示文件 test.doc 在目录 Documents 中，Documents 属于目录 rich，而目录 rich 包含在主目录中。关于该文件存储在 PC 中的哪个物理磁盘，路径不提供任何相关信息。

Linux 虚拟目录最神奇的地方在于结合每个存储设备的方式。安装在 Linux PC 中的第一个硬盘称为**根驱动器**。根驱动器包含虚拟目录的核心。其他目录都是从这里开始创建的。

在根驱动器中，Linux 创建一些名为**挂载点**（mount point）的特殊目录。挂载点是虚拟目录中用于分配额外存储设备的目录。

虚拟目录使文件和目录都出现在这些挂载点目录中，即使它们实际上存储在另一个驱动器中。

系统文件往往存储在根驱动器中，而用户文件则存储在其他驱动器中，如图 3-2 所示。

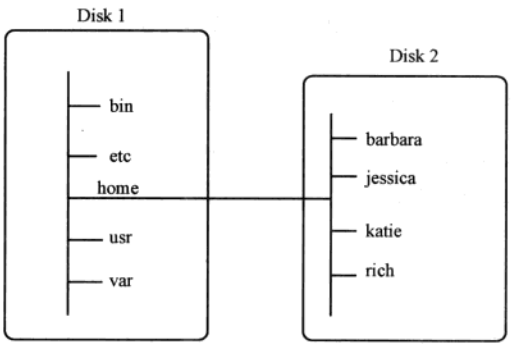



图 3-2 Linux 文件结构

在图 3-2 中可以看到，PC 上有两个硬盘。一个硬盘与虚拟目录的根关联（使用一个正斜杠表示）。其他硬盘可以挂载到虚拟目录结构的任何地方。在本例中，第二个硬盘挂载到位置/home，这也是用户目录所在的位置。

Linux 文件系统结构从 Unix 文件结构演化而来。不幸的是，由于 Unix 版本的不同，Unxi 文件结构这些年来变得十分复杂。如今，似乎任何 Unix 或 Linux 系统的文件系统结构都不相同。但是，对于共同的功能使用的目录名称大致相同。表 3-4 列出了一些常见的 Linux 虚拟目录名称。

表 3-4 通用 Linux 目录名称

| 目录 | 用法 |
| --- | --- |
| / | 虚拟目录的根目录。通常此处没有文件 |
| /bin | 二进制目录，存储许多 GNU 用户级别的实用程序 |

续表

| 目录 | 用法 |
| --- | --- |
| /boot | 引导目录，存储引导文件 |
| /dev | 设备目录，Linux 在该目录中创建设备节点 |
| /etc | 系统配置文件目录 |
| /home | 主目录，Linux 在该目录中创建用户目录 |
| /lib | 库目录，存储系统和应用程序库文件 |
| /media | 媒体目录，可移动媒体设备常用的挂载点 |
| /mnt | 挂载目录，另一个可移动媒体设备常用的挂载点 |
| /opt | 可选目录，常用于存储可选软件包 |
| /root | 根主目录 |
| /sbin | 系统二进制目录，存储许多 GNU 管理级别的实用程序 |
| /tmp | 临时目录，可以在该目录中创建和销毁临时工作文件 |
| /usr | 用户安装软件的目录 |
| /var | 可变目录，用于经常更改的文件，比如日志文件 |

启动一个新的 shell 提示符之后，会话将在主目录中启动，该目录是分配给用户账户的唯一目录。创建用户账户时，系统通常给账户分配一个唯一的目录（参阅第 6 章）。

在 Windows 中，您可能习惯于使用图形界面浏览目录结构。要在 CLI 浏览虚拟目录，则需要学习使用 cd 命令。

## 3.4.2 浏览目录

更改目录命令（cd）可以用于将 shell 会话移动到 Linux 文件系统中的另一个目录。该命令的格式非常简单：

```
cd destination
```

cd 命令只有一个参数 destination，指定目标目录。如果没有指定 cd 命令的 destination 参数，则将回到主目录。

destination 参数可以使用两种不同的方法表示：

- 绝对文件路径；
- 相对文件路径。

以下几节描述这两种方法之间的区别。

### 1. 绝对文件路径

可以在虚拟目录中使用**绝对文件路径**引用目录名称。绝对文件路径定义目录在虚拟文件结构中的确切位置，它以虚拟目录的根为开头。类似于目录的完整名称。

因此，如果要引用包含在 usr 目录中的 lib 目录的 apache 目录，则可以使用绝对文件路径：

```
/usr/lib/apache
```

使用绝对文件路径，可以准确了解您的目标目录。要使用绝对文件路径移动到文件系统中的特定位置，只需在 cd 命令中指定完整的路径名：

```
rich@l[~]$cd /etc
rich@l[etc]$
```

从提示符可以看出，cd 命令后 shell 的新目录现在是/etc。

您可以使用绝对文件路径移动到整个 Linux 目录结构的任何层次：

```
rich@l[~]$ cd /usr/lib/apache
rich@l[apache]$
```

但是，如果只需要在主目录结构中操作，频繁使用绝对路径则显得多此一举。例如，如果已经位

于目录/home/rich中，则为了转到Documents目录而输入命令：

```
cd /home/rich/Documents
```

会显得很多余。幸运的是，还有一种更简单的解决方案。

**2. 相对文件路径**

**相对文件路径**可以相对当前的位置指定目标文件路径，无需从根目录开始。相对文件路径的开头不是表示根目录的正斜杠。

相对文件路径以目录名称（如果要进入当前目录下面的目录）或者表示当前目录位置相对位置的特殊字符开始。使用的两种特殊字符是：

- 点（.）表示当前目录；
- 双点（..）表示父目录。

双点字符在导航目录层次时非常方便。例如，如果位于主目录的Documents目录中，并且需要转到Desktop目录，该目录也在主目录下，则可以使用以下命令：

```
rich@1[Documents]$ cd ../Desktop
rich@1[Desktop]$
```

双点字符将上升一个目录层次到达主目录，然后/Desktop部分将下降到Desktop目录。导航时可以根据需要使用任何数量的双点字符。例如，如果当前位于主目录（/home/rich），希望转到/etc目录，则可以键入：

```
rich@1[~]$ cd ../../etc
rich@1[etc]$
```

当然，在这种情况下，使用相对路径需要键入的字符比绝对路径要多，后者只需要键入/etc！

## 3.5 文件和目录列表

shell最基本的功能是查看系统上目前存在哪些文件。列表命令（ls）即是查看文件的工具。本节介绍ls命令以及所有格式化列表信息的选项。

### 3.5.1 基本列表

ls命令能够以最基本的形式显示当前目录中的文件和目录：

```
$ ls
4rich  Desktop   Download Music   Pictures store       store.zip test
backup Documents Drivers  myprog  Public   store.sql   Templates Videos
$
```

注意，ls命令按字母顺序列出文件和目录（以列而不是行列出）。如果使用了支持颜色的终端模拟器，则ls命令还可以用不同的颜色显示不同类型的条目。LS_COLORS环境变量控制该特性。由于终端模拟器的功能不同，不同的Linux发行版中该环境变量也不同。

如果没有彩色终端模拟器，则可以将-F参数与ls命令配置使用，轻松区分文件和目录。使用-F参数生成的输出如下：

```
$ ls -F
4rich/     Documents/  Music/    Public/   store.zip   Videos/
backup.zip Download/   myprog*   store/    Templates/
Desktop/   Drivers/    Pictures/ store.sql test
$
```

-F参数现在使用正斜杠标记目录，以帮助在列表中区分它们。与此类似，它使用星号标记可执行文件（比如上面的myprog文件），以帮助您轻松找到可以在系统中运行的文件。

基本的ls命令可能会产生一些误导。它显示包含在当前目录中的文件和目录，但是不一定显示所有内容。Linux经常使用**隐藏文件**存储配置信息。在Linux中，隐藏文件的文件名以句点开始。这些文件使用默认的ls无法查看（因此称为隐藏文件）。

要将隐藏文件与普通文件和目录一起显示，可以使用-a 参数。图 3-3 展示使用-a 参数和 ls 命令的示例。

图 3-3 将-a 参数与 ls 命令一同使用

看起来完全不一样了。对于从图形桌面登录系统的用户来说，在主目录将看到许多隐藏配置文件。该示例来自一个登录到 GHOME 桌面会话的用户。可以看到，有 3 个文件以.bash 开头。这些文件是 bash shell 环境使用的隐藏文件。这些特性将在第 5 章详细介绍。

-R 参数是另一个 ls 参数。它显示包含在当前目录的所有目录中的文件。如果当前目录中有许多目录，则显示的内容将会很长。以下是-R 参数生成的简单输出示例：

```
$ ls -F -R
.:
file1 test1/ test2/

./test1:
myprog1* myprog2*

./test2:
$
```

注意最开头，-R 参数显示当前目录的内容，即一个文件（file1）和两个目录（test1 和 test2）。在这之后，它浏览这两个目录，列出目录中包含的所有文件。test1 目录有两个文件（myprog1 和 myprog2），而 test2 目录则没有任何文件。如果在 test1 和 test2 目录中还有子目录，则-R 参数将继续浏览下去。因此，在大型目录结构中该参数将生成大量的输出。

## 3.5.2 修改显示的信息

从基本的列表中可以看到，ls 命令无法生成有关每个文件的详细信息。要列出详细信息，可以使用另一个常用的参数-l。-l 参数生成长列表格式，提供目录中的每个文件的详细信息：

```
$ ls -l
total 2064
drwxrwxr-x    2 rich rich      4096 2007-08-24 22:04 4rich
-rw-r--r--    1 rich rich   1766205 2007-08-24 15:34 backup.zip
drwxr-xr-x    3 rich rich      4096 2007-08-31 22:24 Desktop
drwxr-xr-x    2 rich rich      4096 2001-11-01 04:06 Documents
drwxr-xr-x    2 rich rich      4096 2001-11-01 04:06 Download
drwxrwxr-x    2 rich rich      4096 2007-07-26 18:25 Drivers
drwxr-xr-x    2 rich rich      4096 2001-11-01 04:06 Music
```

```
-rwxr--r--    1 rich rich        30 2007-08-23 21:42 myprog
drwxr-xr-x    2 rich rich      4096 2001-11-01 04:06 Pictures
drwxr-xr-x    2 rich rich      4096 2001-11-01 04:06 Public
drwxrwxr-x    5 rich rich      4096 2007-08-24 22:04 store
-rw-rw-r--    1 rich rich     98772 2007-08-24 15:30 store.sql
-rw-r--r--    1 rich rich    107507 2007-08-13 15:45 store.zip
drwxr-xr-x    2 rich rich      4096 2001-11-01 04:06 Templates
drwxr-xr-x    2 rich rich      4096 2001-11-01 04:06 Videos
[rich@testbox ~]$
```

长列表格式使用一行列出目录中的每个文件或目录。除了文件名之外，它还显示其他有用的信息。输出的第一行显示目录中所含数据块总数。之后，每一行都包含每个文件（或目录）的以下信息：

- 文件类型（比如目录（d）、文件（-）、字符设备（c）、块设备（b））；
- 文件的权限（参阅第 6 章）；
- 指向该文件的硬链接数量（参阅本章的“链接文件”一节）；
- 文件所有者的用户名称；
- 文件所属的组的名称；
- 文件的字节大小；
- 上次修改文件的时间；
- 文件或目录名称。

`-l` 参数是一个强大的工具。有了这些信息，您可以查看系统任何文件或目录的任何信息。

### 3.5.3 完整的参数列表

在文件管理过程中，`ls` 命令可以提供许多参数。如果对 `ls` 使用 `man` 命令，则可以看到有好几页可以用来修改 `ls` 命令输出的参数。

`ls` 命令使用两种命令行参数：

- 单字母参数；
- 全字（长）参数。

单字母参数的前面总是有一个破折号。全字参数的含义更加一目了然，前面带有两个破折号。许多参数既有单字母参数版本也有全字参数版本，有些则只有一种类型。表 3-5 列出了一些常用的参数，可以帮助您更方便地使用 bash `ls` 命令。

表 3-5 一些常用的 ls 命令参数

| 单字母 | 全字 | 描述 |
|---|---|---|
| -a | --all | 不要忽略以句点开头的条目 |
| -A | --almost -all | 不要列出.和..文件 |
| | --author | 打印每个文件的作者 |
| -b | --escape | 打印不可打印字符的八进制值 |
| | --block -size=size | 使用 size-byte 块计算块大小 |
| -B | --ignore -backups | 不要列出带波浪号的条目（波浪号表示备份副本） |
| -c | | 根据最后一次修改的时间排序 |
| -C | | 按列列出条目 |
| | --color=when | 何时使用颜色（总是，从不，自动） |
| -d | --directory | 列出目录条目而不是内容，不要废弃符号链接 |
| -F | --classify | 向条目附加文件类型指示符 |
| | --file-type | 仅向某些文件类型（非可执行文件）附加文件类型指示符 |

续表

| 单字母 | 全字 | 描述 |
|---|---|---|
| | --format=word | 以交叉、逗号、水平、长、单列、详细、垂直等形式格式化输出 |
| -g | | 列出文件所有者以外的详细信息 |
| | --group -directories -first | 在文件之前列出所有目录 |
| -G | --no -group | 在长列表中不显示组名称 |
| -h | --human -readable | 打印大小，K 表示千字节，M 表示兆字节，G 表示十亿字节 |
| | --si | 与-h 相同，但是使用的权重为 1 000 而不是 1 024 |
| -i | --inode | 显示每个文件的索引号（inode） |
| -l | | 显示长列表格式 |
| -L | --dereference | 显示链接文件的源文件信息 |
| -n | --numeric -uid -gid | 显示用户 ID 和组 ID 而不是名称 |
| -o | | 在长列表中不显示所有者名称 |
| -r | --reverse | 显示文件和目录时逆向排序 |
| -R | --recursive | 递归列出子目录的内容 |
| -s | --size | 打印每个文件的块大小 |
| -S | --sort=size | 按照文件大小排列输出 |
| -t | --sort=time | 按照文件修改时间排列输出 |
| -u | | 显示文件上一次访问时间而不是修改时间 |
| -U | --sort=none | 输出列表不排序 |
| -v | --sort=version | 根据文件版本排列输出 |
| -x | | 按行而不是列列出条目 |
| -X | --sort=extension | 按文件扩展名排列输出 |

如果需要，一次也可以使用多个参数。双破折号参数必须分别列出，而单破折号参数则可以在一个破折号后组成一个字符串。常见的组合是，使用-a 参数列出所有文件，使用-i 参数类出每个文件的**索引节点（inode）**，使用-l 参数生成长列表，使用-s 参数列出文件的块大小。文件或目录的索引节点是一个独特的标识数，由内核分配给文件系统的每个对象。组合这些参数可以得到一个易于记忆的-sail 参数：

```
$ ls -sail
total 2360
301860    8 drwx------ 36 rich rich    4096 2007-09-03 15:12 .
 65473    8 drwxr-xr-x  6 root root    4096 2007-07-29 14:20 ..
360621    8 drwxrwxr-x  2 rich rich    4096 2007-08-24 22:04 4rich
301862    8 -rw-r--r--  1 rich rich     124 2007-02-12 10:18 .bashrc
361443    8 drwxrwxr-x  4 rich rich    4096 2007-07-26 20:31 .ccache
301879    8 drwxr-xr-x  3 rich rich    4096 2007-07-26 18:25 .config
301871    8 drwxr-xr-x  3 rich rich    4096 2007-08-31 22:24 Desktop
301870    8 -rw-------  1 rich rich      26 2001-11-01 04:06 .dmrc
301872    8 drwxr-xr-x  2 rich rich    4096 2001-11-01 04:06 Download
360207    8 drwxrwxr-x  2 rich rich    4096 2007-07-26 18:25 Drivers
301882    8 drwx------  5 rich rich    4096 2007-09-02 23:40 .gconf
301883    8 drwx------  2 rich rich    4096 2007-09-02 23:43 .gconfd
360338    8 drwx------  3 rich rich    4096 2007-08-06 23:06 .gftp
```

除了常见的-l 参数输出信息之外，每一行还有两个额外的数字。列表中的第一个数字是文件或目录的索引节点编号。第二个数字是文件的块大小。第三个条目是文件类型以及文件的权限。我们将在第 6 章详细介绍。

在这之后，分别是一个指向该文件的硬链接数（将在“链接文件”一节讨论）、文件的所有者、

文件所属的组、文件的大小（字节为单位）、默认显示最后一次修改时间的时间戳，最后是实际的文件名。

### 3.5.4 过滤列表输出

通过示例可以看出，默认情况下 ls 命令列出目录的所有文件。有时候这样做有些多余，尤其在您只需要查找某一个文件的信息时。

幸运的是，ls 命令还提供了一种在命令行定义过滤器的方法。它使用过滤器确定应该在输出中显示哪些文件或目录。

过滤器的工作方式类似于一个简单的文本匹配字符串。在命令行参数之后包含过滤器的方式如下：

```
$ ls -l myprog
-rwxr--r-- 1 rich rich 30 2007-08-23 21:42 myprog
$
```

如果将特定文件的名称指定为过滤器，则 ls 命令将仅显示该文件的信息。有时候您可能不知道要查找的文件的确切名称。这时 ls 命令可以使用标准的通配符字符，并使用它们匹配过滤器中的模式。

- 问号表示一个字符；
- 星号表示零个或多个字符。

问号可以用来替代过滤字符串中的一个字符。例如：

```
$ ls -l mypro?
-rw-rw-r-- 1 rich rich  0 2007-09-03 16:38 myprob
-rwxr--r-- 1 rich rich 30 2007-08-23 21:42 myprog
$
```

过滤器 mypro?与目录中的两个文件匹配。与此类似，星号可以用来匹配零个或多个字符：

```
$ ls -l myprob*
-rw-rw-r-- 1 rich rich 0 2007-09-03 16:38 myprob
-rw-rw-r-- 1 rich rich 0 2007-09-03 16:40 myproblem
$
```

星号匹配 myprob 文件中的零个字符，但在 myproblem 文件中则匹配 3 个字符。

如果不确定文件的名称，使用该特性搜索文件将非常便捷。

## 3.6 文件处理

bash shell 提供许多操作 Linux 文件系统文件的命令。本节介绍通过 CLI 处理文件所需的基本命令，以满足您所有的文件处理需求。

### 3.6.1 创建文件

有时可能会遇到需要创建一个空文件的情况。比如，应用程序可能希望在写入日志文件前先显示该文件。在这种情况下，可以使用 touch 命令轻松创建一个空文件。

```
$ touch test1
$ ls -il test1
1954793 -rw-r--r-- 1 rich   rich          0 Sep 1 09:35 test1
$
```

touch 命令创建一个您指定的新文件，并将您的用户名作为文件所有者。由于我对 ls 命令使用了-il 参数，因此列表中的第一项显示分配给文件的索引节点号。Linux 系统中的每个文件都有一个唯一的索引节点号。

注意，文件大小为零，因为 touch 命令创建的是一个空文件。touch 命令还可以用于更改现有文件的访问时间和修改时间，而不改变文件的内容。

```
$ touch test1
$ ls -l test1
-rw-r--r--    1 rich        rich          0 Sep 1 09:37 test1
$
```

test1 的修改时间现在已经更新。如果只需要更改访问时间，则可以使用-a 参数。若只需要更改修改时间，则使用-m 参数。默认情况下，touch 使用当前时间。也可以指定时间——使用-t 参数加上指定的时间戳。

```
$ touch -t 200812251200 test1
$ ls -l test1
-rw-r--r--    1 rich        rich          0 Dec 25 2008 test1
$
```

现在文件的修改时间已经从当前时间更改为将来的某个时间。

### 3.6.2 复制文件

对于系统管理员来说，经常需要将文件和目录从文件系统的一个位置复制到另一个位置。cp 命令可以提供这种功能。

cp 命令最基本的形式是使用两个参数：源对象和目标对象：

```
cp source destination
```

如果 source 和 destination 参数都是文件名，cp 命令将源文件复制为一个新文件，使用 destination 参数指定的文件名。新文件就跟新创建的文件一样，具有更新过的文件创建时间和最后修改时间：

```
$ cp test1 test2
$ ls -il
total 0
1954793 -rw-r--r--     1 rich    rich          0 Dec 25  2008 test1
1954794 -rw-r--r--     1 rich    rich          0 Sep  1 09:39 test2
$
```

新文件 test2 的索引节点号不同，表示这是一个全新的文件。可以看到，test2 文件的修改时间与它的创建时间相同。

如果 destination 文件已经存在，则 cp 命令将提示您是否覆盖该文件：

```
$ cp test1 test2
cp: overwrite 'test2'? y
$
```

如果不回答 y，则将取消文件复制操作。您还可以将文件复制到现有目录：

```
$ cp test1 dir1
$ ls -il dir1
total 0
1954887 -rw-r--r--     1 rich    rich          0 Sep 6 09:42 test1
$
```

新文件现在位于 dir1 目录下，文件名与源文件相同。这些示例均使用相对路径名称，但是对源对象和目标对象使用绝对路径也没有问题。

要将文件复制到当前所在的目录，可以使用句点符号：

```
$ cp /home/rich/dir1/test1 .
cp: overwrite './test1'?
```

与大部分命令一样，cp 命令也有几个可以提供帮助的命令行参数，如表 3-6 所示。

表 3-6 cp 命令参数

| 参数 | 描述 |
| --- | --- |
| -a | 归档文件，保留其属性 |
| -b | 为每个现有 destination 文件创建一个备份，而不是覆盖它 |
| -d | 保留 |

续表

| 参数 | 描述 |
| --- | --- |
| -f | 强制覆盖现有 destination 文件，没有提示 |
| -i | 覆盖 destination 文件前提示 |
| -l | 创建文件链接而不是复制文件 |
| -p | 保留文件属性（如果可能） |
| -r | 递归复制文件 |
| -R | 递归复制目录 |
| -s | 创建符号链接而不是复制文件 |
| -S | 重写备份特性 |
| -u | 仅在 source 文件的日期和时间比 destination 新时复制 source 文件（更新） |
| -v | 冗长模式，解释发生的事件 |
| -x | 将复制限定在当前文件系统 |

使用-p 参数可以使复制文件的访问时间和修改时间与源文件一致。

```
$ cp -p test1 test3
$ ls -il
total 4
1954886 drwxr-xr-x     2 rich    rich      4096 Sep  1 09:42 dir1/
1954793 -rw-r--r--     1 rich    rich         0 Dec 25  2008 test1
1954794 -rw-r--r--     1 rich    rich         0 Sep  1 09:39 test2
1954888 -rw-r--r--     1 rich    rich         0 Dec 25  2008 test3
$
```

尽管 test3 文件是一个全新的文件，但它的时间戳与源文件 test1 一样。

-R 参数非常强大。它可以使用一个命令递归复制整个目录的内容：

```
$ cp -R dir1 dir2
$ ls -l
total 8
drwxr-xr-x    2 rich    rich      4096 Sep  6 09:42 dir1/
drwxr-xr-x    2 rich    rich      4096 Sep  6 09:45 dir2/
-rw-r--r--    1 rich    rich         0 Dec 25  2008 test1
-rw-r--r--    1 rich    rich         0 Sep  6 09:39 test2
-rw-r--r--    1 rich    rich         0 Dec 25  2008 test3
$
```

生成的 dir2 是 dir1 的完整复制。cp 命令中还可以使用通配符：

```
$ cp -f test* dir2
$ ls -al dir2
total 12
drwxr-xr-x    2 rich    rich      4096 Sep  6 10:55 ./
drwxr-xr-x    4 rich    rich      4096 Sep  6 10:46 ../
-rw-r--r--    1 rich    rich         0 Dec 25  2008 test1
-rw-r--r--    1 rich    rich         0 Sep  6 10:55 test2
-rw-r--r--    1 rich    rich         0 Dec 25  2008 test3
$
```

该命令将所有以 test 开始的文件复制到 dir2。我使用-f 参数强制重写所有已经存在于目录的 test1 文件，没有提示。

### 3.6.3 链接文件

您可能已经注意到了，cp 命令中有些参数涉及到链接文件。这是 Linux 文件系统中一个非常好的特性。如果需要在系统中维护同一个文件的两个（或两个以上）副本，您不一定要使用两个物理副本，可以使用一个物理副本和多个虚拟副本，这种虚拟副本称为**链接**。链接是目录中的占位符，指向文件的真实位置。Linux 中有两种不同的文件链接类型：

- ✦ 符号链接（又称软链接）；
- ✦ 硬链接。

硬链接创建一个单独的文件，其中包括有关源文件的信息和位置。引用该硬链接文件的效果跟引用源文件一样：

```
$ cp -l test1 test4
$ ls -il
total 16
1954886 drwxr-xr-x     2 rich    rich     4096 Sep  1 09:42 dir1/
1954889 drwxr-xr-x     2 rich    rich     4096 Sep  1 09:45 dir2/
1954793 -rw-r--r--     2 rich    rich        0 Sep  1 09:51 test1
1954794 -rw-r--r--     1 rich    rich        0 Sep  1 09:39 test2
1954888 -rw-r--r--     1 rich    rich        0 Dec 25  2008 test3
1954793 -rw-r--r--     2 rich    rich        0 Sep  1 09:51 test4
$
```

-l 参数创建 test1 文件的一个硬链接，名为 test4。执行文件列表时会发现，test1 和 test4 的索引节点号相同，这表示实际上它们是同一个文件。还注意到，链接编号（列表第三项）显示这两个文件都有两个链接。

注意：在同一个物理介质的文件之间只能创建一个硬链接。不能在不同挂载点下的文件之间创建硬链接。在这种情况下，您必须使用软链接。

另一方面，-s 参数创建一个符号链接（又称软链接）：

```
$ cp -s test1 test5
$ ls -il test*
total 16
1954793 -rw-r--r--     2 rich    rich        6 Sep  1 09:51 test1
1954794 -rw-r--r--     1 rich    rich        0 Sep  1 09:39 test2
1954888 -rw-r--r--     1 rich    rich        0 Dec 25  2008 test3
1954793 -rw-r--r--     2 rich    rich        6 Sep  1 09:51 test4
1954891 lrwxrwxrwx     1 rich    rich        5 Sep  1 09:56 test5 -> test1
$
```

在文件列表中要注意几件事情。首先，注意新的 test5 文件的索引节点号与 test1 文件不同，这表示 Linux 系统将其视为不同的文件。其次，文件大小不同。链接文件只需要存储有关源文件的信息，而不是文件中的实际数据。列表的文件名部分显示了这两个文件的关系。

提示：如果需要链接文件，除了 cp 命令之外还可以使用 ln 命令。默认情况下，ln 命令创建的是硬链接。如果需要创建软链接，还需要使用-s 参数。

在复制链接文件时要十分小心。如果使用 cp 命令复制一个链接到另一个源文件的文件，则所做的只是再次生成了源文件的一个副本。这可能导致混淆。实际上，您可以再创建一个源文件的链接而不是复制链接文件。对同一个文件生成多个链接没有任何问题。但是，不应该创建其他软链接文件的软链接。因为这会创建一个链接链条，不仅会导致混淆，而且很容易遭到破坏，可能导致各种问题。

### 3.6.4 重命名文件

在 Linux 中，重命名文件的过程称为**移动**。mv 命令可以将文件和目录移动到另一个位置：

```
$ mv test2 test6
$ ls -il test*
1954793 -rw-r--r--     2 rich    rich        6 Sep  1 09:51 test1
1954888 -rw-r--r--     1 rich    rich        0 Dec 25  2008 test3
1954793 -rw-r--r--     2 rich    rich        6 Sep  1 09:51 test4
1954891 lrwxrwxrwx     1 rich    rich        5 Sep  1 09:56 test5 -> test1
1954794 -rw-r--r--     1 rich    rich        0 Sep  1 09:39 test6
$
```

注意，移动文件将更改文件名但保留原来的索引节点号和时间戳。移动带有软链接的文件存在问题：

```
$ mv test1 test8
$ ls -il test*
total 16
1954888 -rw-r--r--     1 rich    rich      0 Dec 25  2008 test3
1954793 -rw-r--r--     2 rich    rich      6 Sep  1 09:51 test4
1954891 lrwxrwxrwx     1 rich    rich      5 Sep  1 09:56 test5 -> test1
1954794 -rw-r--r--     1 rich    rich      0 Sep  1 09:39 test6
1954793 -rw-r--r--     2 rich    rich      6 Sep  1 09:51 test8
[rich@test2 clsc]$ mv test8 test1
```

使用硬链接的 test4 文件使用相同的索引节点号，这没有问题。但是，test5 文件现在指向一个无效的文件，不再是一个有效链接。

还可以使用 mv 命令移动目录：

```
$ mv dir2 dir4
```

整个目录的内容都没有变化。唯一改变的是目录的名称。

### 3.6.5 删除文件

在使用 Linux 的过程中经常需要删除现有的文件。删除文件的目的可能是为了整理文件系统，也可能是为了卸载软件包。

在 Linux 中，删除过程称为**移除**。bash shell 中移除文件的命令是 rm。rm 命令的基本格式非常简单：

```
$ rm -i test2
rm: remove 'test2'? y
$ ls -l
total 16
drwxr-xr-x    2 rich    rich      4096 Sep  1 09:42 dir1/
drwxr-xr-x    2 rich    rich      4096 Sep  1 09:45 dir2/
-rw-r--r--    2 rich    rich         6 Sep  1 09:51 test1
-rw-r--r--    1 rich    rich         0 Dec 25  2008 test3
-rw-r--r--    2 rich    rich         6 Sep  1 09:51 test4
lrwxrwxrwx    1 rich    rich         5 Sep  1 09:56 test5 -> test1
$
```

注意，该命令会提示您是否确定移除该文件。bash shell 中没有回收站。一旦移除文件则无法还原。

现在介绍删除带链接文件时会出现的趣事：

```
$ rm test1
$ ls -l
total 12
drwxr-xr-x    2 rich    rich 4096 Sep  1 09:42 dir1/
drwxr-xr-x    2 rich    rich 4096 Sep  1 09:45 dir2/
-rw-r--r--    1 rich    rich    0 Dec 25  2008 test3
-rw-r--r--    1 rich    rich    6 Sep  1 09:51 test4
lrwxrwxrwx    1 rich    rich    5 Sep  1 09:56 test5 -> test1
$ cat test4
hello
$ cat test5
cat: test5: No such file or directory
$
```

我移除了 test1 文件，它包含一个指向 test4 文件的硬链接和一个指向 test5 文件的软链接。注意发生了什么。两个链接文件仍然显示，尽管 test1 文件已经不再了（在我的彩色终端中 test5 文件名使用红色显示）。当我查看硬链接的 test4 文件的内容时，仍然可以显示。当我查看软链接的 test5 文件的内容时，bash 指出不存在该文件。

记住，硬链接文件使用的索引节点编号与源文件相同。在移除了最后一个链接文件之前，硬链接文件将一直维护索引节点编号，并保留数据！所有软链接的处理方式则是：底层文件不在了，那么链接指向的内容自然也消失了。在处理链接文件时务必记住该特性。

rm 命令的另一个特性是，如果需要移除很多文件并且不希望收到提示，则可以使用-f 参数强制

移除。但务必小心使用！

Tip 提示：与复制文件一样，可以对 rm 命令使用通配符。再次提醒您这样做时要小心，因为您移除的所有内容（即使是无意中移除的）都无法还原。

## 3.7 目录处理

在 Linux 中，有些命令既可用于文件又可用于目录（比如 cp 命令），还有些则只能用于目录处理。若要创建一个新的目录，需要使用一个特殊的命令，我们将在本节介绍。移除目录也值得一提，因此本节也将顺便介绍。

### 3.7.1 创建目录

在 Linux 创建新目录只需使用 mkdir 命令即可：

```
$ mkdir dir3
$ ls -il
total 16
1954886 drwxr-xr-x     2 rich    rich     4096 Sep  1 09:42 dir1/
1954889 drwxr-xr-x     2 rich    rich     4096 Sep  1 10:55 dir2/
1954893 drwxr-xr-x     2 rich    rich     4096 Sep  1 11:01 dir3/
1954888 -rw-r--r--     1 rich    rich        0 Dec 25  2008 test3
1954793 -rw-r--r--     1 rich    rich        6 Sep  1 09:51 test4
$
```

系统将创建一个新的目录并分配一个新的索引节点编号。

### 3.7.2 删除目录

移除目录则比较复杂，但也有因可循。开始删除目录时，可能会出现很多问题。bash shell 尽可能帮助我们避免意外的发生。移除目录的基本命令是 rmdir：

```
$ rmdir dir3
$ rmdir dir1
rmdir: dir1: Directory not empty
$
```

默认情况下，rmdir 命令仅用于移除空目录。由于 dir1 目录中存在一个文件，因此 rmdir 命令将拒绝移除该目录。可以使用--ignore-fail-on-non-empty 参数移除非空目录。

前面介绍过的 rm 命令可以帮助我们处理目录。

如果不带参数的使用它（就像对文件一样），您可能会失望：

```
$ rm dir1
rm: dir1: is a directory
$
```

但是，如果确实需要移除目录，则可以使用-r 参数递归移除目录中的文件，然后再移除目录本身：

```
$ rm -r dir2
rm: descend into directory 'dir2'? y
rm: remove 'dir2/test1'? y
rm: remove 'dir2/test3'? y
rm: remove 'dir2/test4'? y
rm: remove directory 'dir2'? y
$
```

尽管这样做是可行的，但是比较麻烦。注意，您必须验证需要移除的每个文件。对于带有很多文件和子目录的目录而言，这样做非常麻烦。

移除整个目录和内容的终极解决方案是，使用 rm 命令以及-r 和-f 参数：

```
$ rm -rf dir2
$
```

就是这样。没有警告，没有技巧，只是另一个 shell 提示符而已。当然，这也是一个非常危险的工

具，尤其是作为超级用户登录时。请谨慎使用，使用前务必三思而行，确定这样做不会出现问题。

注意：在上一个示例中，您可能注意到我将两个命令行参数连接到一起，只使用了一个破折号。这是 bash shell 的一个特性，该特征允许我们组合命令行参数，以简化输入。

## 3.8 查看文件内容

到目前为止，除了如何查看文件内容外，我们已经介绍了有关文件的其他内容。有几个命令不需要使用编辑器（参阅第 7 章）即可查看文件内容。本章介绍其中几个文件查看命令。

### 3.8.1 查看文件统计数据

我们已经介绍过可以提供许多有关文件信息的 ls 命令。但是，仍然有很多信息无法使用 ls 命令查看（至少不能一次性查看）。

stat 命令提供文件系统中文件状态的完整摘要：

```
$ stat test10
  File: "test10"
  Size: 6               Blocks: 8              Regular File
Device: 306h/774d       Inode: 1954891         Links: 2
Access: (0644/-rw-r--r--) Uid: ( 501/ rich) Gid: ( 501/ rich)
Access: Sat Sep 1 12:10:25 2007
Modify: Sat Sep 1 12:11:17 2007
Change: Sat Sep 1 12:16:42 2007
$
```

stat 命令的结果可以显示目标文件的所有信息，甚至包括存储该文件的设备的主设备号和次级设备号。

### 3.8.2 查看文件类型

在 stat 命令生成的所有信息之中，仍然缺少了一条信息——文件类型。在准备列出 1000 字节的文件之前，最好能了解该文件的类型。如果尝试列出二进制文件，您的监视器上将出现很多乱码，甚至会锁定终端模拟器。

file 命令是一个方便的实用程序。它可以查看文件内部并确定文件的类型：

```
$ file test1
test1: ASCII text
$ file myscript
myscript: Bourne shell script text executable
$ file myprog
myprog: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV),
dynamically linked (uses shared libs), not stripped
$
```

file 命令将文件分为以下 3 类：

- **文本文件**：包含可打印字符的文件；
- **可执行文件**：可以在系统运行的文件；
- **数据文件**：包括不可打印的二进制字符的文件，但也不可以在系统运行。

第一个示例是一个文本文件。file 命令不仅能确定该文件包含文本，而且能确定文本的字符码格式。第二个示例是一个文本脚本文件。尽管该文件也是文本文件，但由于它又是一个脚本文件，因此可以在系统上执行（运行）。最后一个示例是一个二进制可执行程序。file 命令确定编译该程序的平台以及它需要的库类型。如果有一个来自未知源的二进制可执行程序，则该命令非常便利。

### 3.8.3 查看整个文件

如果有一个很大的文本文件，您可能需要查看它的内容。Linux 中有几个命令可以帮助您做到这一点。

**1. cat 命令**

cat 命令是一个显示文本文件内容的便捷工具：

```
$ cat test1
hello

This is a test file.

That we'll use to      test the cat command.
$
```

没什么新鲜的内容，只是文本文件的内容而已。cat 命令可以使用几个参数。

-n 参数为所有行编号：

```
$ cat -n test1
     1 hello
     2
     3 This is a test file.
     4
     5
     6 That we'll use to         test the cat command.
$
```

该特性在查看脚本时很有用。如果只需要为带文本的行编号，则可以使用-b 参数：

```
$ cat -b test1
     1 hello

     2 This is a test file.

     3 That we'll use to         test the cat command.
$
Fancy! If you need to compress multiple blank lines into a single
blank line, use the -s parameter:
$ cat -s test1
hello

This is a test file.

That we'll use to      test the cat command.
$
```

最后，如果不希望出现跳格字符，则可以使用-T 参数：

```
$ cat -T test1
hello

This is a test file.

That we'll use to^Itest the cat command.
$
```

-T 参数将使用^I 字符组合替换文本中的任何跳格符。

对于大的文件，cat 命令可能不太适用。因为文件中的文本会快速在监视器上不停滚动。幸运的是，还有一种简单的方法可以解决该问题。

**2. more 命令**

cat 命令主要的缺点在于启动之后无法控制。为了解决这个问题，开发人员创建了 more 命令。more 命令显示文本文件，但是在显示完每页数据时停止。more 输出示例如图 3-4 所示。

图 3-4　使用 more 命令显示文本文件

注意，在图 3-4 的屏幕下方，more 命令显示了一个标记，该标记说明您仍然在 more 应用程序中，并表明您离文件开头的距离。more 命令有一个提示符。这时有很多选项供您选择，如表 3-7 所示。

表 3-7　　more 命令选项

| 选项 | 描述 |
|---|---|
| H | 显示帮助菜单 |
| spacebar | 显示文件文本的下一页 |
| z | 显示文件文本的下一页 |
| ENTER | 显示文件文本的下一行 |
| d | 显示一半屏幕的文件文本（11 行） |
| q | 退出程序 |
| s | 向前跳过文本的下一行 |
| f | 向前跳过一屏幕的文本 |
| b | 向后跳过一屏幕的文本 |
| /expression | 在文件中搜索 expression 代表的文本 |
| n | 搜索上次指定的表达式中下一次出现的位置 |
| ' | 转到指定表达式第一次出现的位置 |
| !cmd | 执行 shell 命令 |
| v | 在当前行启动 vi 编辑器 |
| CTRL-L | 在文件的当前位置重绘屏幕 |
| = | 显示文件的当前行号 |
| . | 重复上一个命令 |

more 命令可以在文本文件中执行一些基本的移动操作。若需要更多高级性能，可以尝试使用 less 命令。

### 3. less 命令

从名称看来，似乎该命令不会比 more 命令更加高级，但实际上，less 命令的名称只是一个文字游戏，它是 more 命令的高级版本（less 这个名称来自俗语“越简单就越丰富”，即 less is more）。它提供一些便利的功能，用于向前或向后滚动文本文件，还提供了一些高级的搜索功能。

less 命令还可以在命令读完整个文件之前显示文件内容。这项功能是 cat 和 more 命令所没有的，因而可以轻松应付大型文件。

less 命令的操作与 more 命令基本相同，一次显示一屏幕的文本。图 3-5 展示了一个 less 命令的输出。

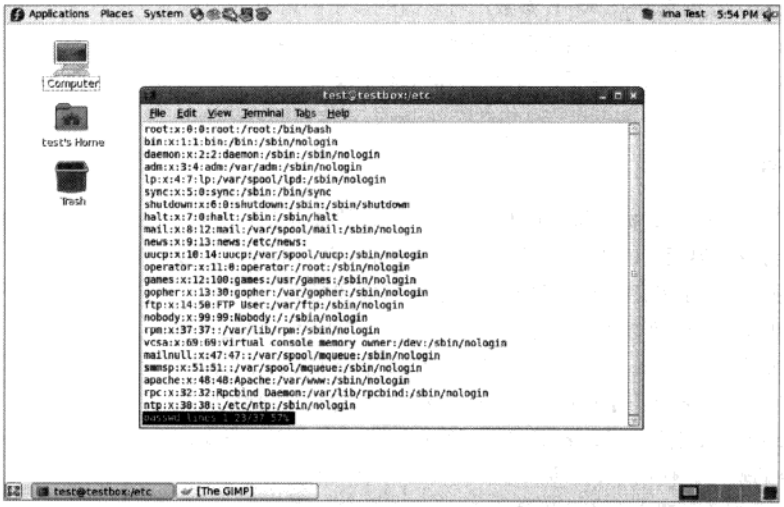



图 3-5 使用 less 命令查看文件

注意，less 命令在提示符提供更多的信息，它显示了文件的总行数以及当前显示的行数范围。less 命令除了支持 more 命令的命令集外，还增加了许多选项。若要查看所有可用选项，可以查看 less 命令的 man 页面。其中一个特性集使 less 命令能够识别向上和向下方向键，以及向上翻页和向下翻页键（假定您使用的是相应的终端）。这能够完全控制查看文件的方式。

## 3.8.4 查看部分文件

需要查看的数据有时正好位于文本文件的开头或结尾。如果信息位于大文件的开头，则需要等待 cat 或 more 命令加载了整个文件之后才能查看。如果信息位于文件结尾（比如日志文件），则需要翻过许多文本之后才能到达。幸运的是，Linux 有专门的命令可以解决这些问题。

### 1. tail 命令

tail 命令显示最后一组文件行。默认情况下，它显示文件的最后 10 行，但是也可以使用命令行参数更改该数字，如表 3-8 所示。

表 3-8 tail 命令行参数

| 参数 | 描述 |
|---|---|
| -c bytes | 显示文件最后 bytes 个字节 |
| -n lines | 显示文件最后 lines 行 |
| -f | 保持 tail 程序的激活状态，并继续显示添加到文件的新行 |
| --pid=PID | 与-f 一起使用，跟踪文件直到进程 ID 为 PID 的进程终止 |
| -s sec | 与-f 一起使用，迭代之间休眠 sec 秒 |
| -v | 始终显示给定文件名的输出标题 |
| -q | 绝不显示给定文件名的输出标题 |

-f 参数是 tail 命令中一个非常好的特性。它允许您在其他进程使用文件时查看该文件。当文本

文件中出现新内容时，tail 命令保持激活状态并继续显示新行。这也是实时监控系统日志文件的一种好方式。

**2. head 命令**

与 tail 命令类似，head 命令显示开头的一组文件行。默认情况下，它显示文件的前 10 行。与 tail 命令类似，它也支持-c 和-n 参数，可以改变显示的内容。

一般情况下，文件的开头不会更改，因此 head 命令不支持-f 参数特性。head 命令是一个方便的工具，在不确定文件内容的情况下，它可以帮助您查看文件开头，而不需要多此一举地显示整个文件。

## 3.9 小结

本章介绍了通过 shell 提示符处理 Linux 文件系统的基本方法。首先讨论了 bash shell，介绍如何与 shell 交互。命令行界面（CLI）使用提示符字符串指示什么时候可以输入命令。可以自定义提示符字符串显示与系统有关的有用信息、登录 ID，甚至日期和时间。

bash shell 提供许多可以用于创建和操作文件的实用程序。在处理文件之前，最好了解 Linux 如何存储它们。本章介绍了 Linux 虚拟目录的基础知识并显示了 Linux 如何引用存储媒体设备。介绍完 Linux 文件系统之后，本章还介绍了如何使用 cd 命令在虚拟目录之间移动。

介绍了如何进入目录之后，本章演示了如何使用 ls 命令列出文件和子目录。有许多参数可以自定义 ls 命令的输出。使用 ls 命令可以获得文件和目录的各种信息。

touch 命令对于创建空文件以及更改现有文件的访问时间和修改时间非常有用。本章还讨论了如何使用 cp 命令将现有文件从一个位置复制到另一个位置。讨论了使用链接文件代替复制文件的步骤，这是一种便捷的方式，可以在两个位置存储相同的文件而不需要进行复制。cp 命令和 ln 命令都具有该功能。

接下来，我们学习了如何使用 mv 命令在 Linux 中重命名文件（称为移动），以及如何使用 rm 命令删除文件（称为移除）。还学习了如何使用 mkdir 和 rmdir 命令对目录执行这两项任务。

本章的最后介绍如何查看文件的内容。使用 cat、more 和 less 命令可以轻松查看文件的所有内容，使用 tail 和 head 命令可以进入文件查看一小部分内容。

下一章将继续讨论 bash shell 命令。我们将学习更加高级的管理命令，帮助您轻松管理 Linux 系统。

# 第 4 章
# 更多 bash shell 命令

CHAPTER 4

本章内容：

- 管理进程；
- 获取磁盘统计信息；
- 挂载新磁盘；
- 排序数据；
- 归档数据。

第 3 章介绍了在 Linux 文件系统中搜索的基础知识，还介绍了如何使用文件和目录。文件和目录的管理是 Linux shell 的一个主要功能。但在开始介绍脚本编程前，我们还需要考虑一些其他东西。本章将深入介绍 Linux 系统管理命令，向您展示如何使用命令行命令来了解 Linux 系统。然后本章会给出一些便捷的命令，使用这些命令可以处理系统上的数据文件。

## 4.1 监控程序

作为 Linux 系统管理员，最困难的工作之一就是跟踪正在系统中运行的程序——尤其是现在，图形桌面使用很多的程序，只是为了生成一个桌面环境。系统中总是运行着很多程序。

幸运的是，有一些命令行工具可帮助您轻松完成任务。本节将介绍一些您需要了解并用来管理 Linux 系统程序的基本工具。

### 4.1.1 进程查看

当一个程序在系统中运行，就称之为一个**进程**。要检查这些进程，就需要熟悉 ps 命令。它是出色的实用工具。它可以生成关于运行在系统中的所有程序的大量信息。

不幸的是，强大的功能导致其复杂性（以多种参数的形式出现），使得 ps 命令成为可能最难掌握的命令之一。大多数系统管理员发现 ps 命令的一部分参数便足够生成他们需要的数据，然后他们就会一直使用这些参数。

然而，这也说明基本 ps 命令本身并不能提供太多信息。

```
$ ps
  PID TTY          TIME CMD
 3081 pts/    0 00:00:00 bash
 3209 pts/    0 00:00:00 ps
$
```

默认情况下，ps 命令只显示属于当前用户并正在当前终端中运行的进程。在本例中，我只运行了 bash shell（记住，shell 也只是运行在系统中的另一个程序），当然也运行了 ps 命令。

ps命令的基本输出显示了程序的进程ID（PID），其运行的终端（TTY）和进程使用的CPU时间。

注意：ps命令的复杂性（也是此命令之所以复杂的原因）是它同时有两个版本。每个版本有自己不同的命令行参数集，用来控制信息的显示内容和显示方式。最近，Linux开发人员已经将两套ps命令格式合并成单一ps程序（当然加入了他们自己的风格）。

Linux系统中使用的GNU ps命令支持3种不同的命令行参数：

- Unix类型参数，前面有一短划线；
- BSD类型参数，前面没有短划线；
- GNU长参数，前面有双短划线。

下面几节将介绍此3种不同的参数类型并提供使用示例。

### 1. Unix类型参数

Unix类型参数起源于贝尔实验室发明的AT&T Unix系统使用的原始ps命令。参数如表4-1所示。

表4-1 ps命令的Unix参数

| 参数 | 描述 |
|---|---|
| -A | 显示所有进程 |
| -N | 显示特定参数的结果的补集 |
| -a | 显示除会话标题和无终端进程外的所有进程 |
| -d | 显示除会话标题外的所有进程 |
| -e | 显示所有进程 |
| -C *cmslist* | 显示包含在cmdlist列表中的进程 |
| -G *grplist* | 显示grplist列表中具有组ID的进程 |
| -U *userlist* | 显示userlist列表中的userid拥有的进程 |
| -g *grplist* | 根据会话或包含在grplist中的groupid来显示进程 |
| -p *pidlist* | 显示pidlist列表中具有PID的进程 |
| -s *sesslist* | 显示sesslist列表中具有会话ID的进程 |
| -t *ttylist* | 显示ttylist列表中具有终端ID的进程 |
| -u *userlist* | 根据userlist列表中的有效userid来显示进程 |
| -F | 使用额外完整输出 |
| -O *format* | 显示format列表中的特定列和默认列 |
| -M | 显示关于进程的安全信息 |
| -c | 显示关于进程的额外调度信息 |
| -f | 显示完整的格式列表 |
| -j | 显示作业信息 |
| -l | 显示长列表 |
| -o *format* | 只显示format中列出的特定列 |
| -y | 不显示进程标记 |
| -Z | 显示安全上下文信息 |
| -H | 以层级格式显示进程（显示父进程） |
| -n *namelist* | 定义在WCHAN列中显示的值 |
| -w | 使用宽输出格式，使显示宽度不受限制 |

续表

| 参数 | 描述 |
| --- | --- |
| -L | 显示进程线程 |
| -V | 显示 ps 的版本 |

上面已经列出了许多参数，但请记住，这只是其中一部分！使用 ps 命令的关键不是记住所有可用的参数，而是仅记住您认为最有用的参数。大多数 Linux 系统管理员都有自己的一组需要记住的常用参数，用于提取相关信息。例如，如果您需要查看系统上运行的一切程序，可以使用-ef 参数组合（ps 命令允许这样组合参数）：

```
$ ps -ef
UID        PID PPID C STIME TTY          TIME CMD
root         1    0 0 11:29 ?        00:00:01 init [5]
root         2    0 0 11:29 ?        00:00:00 [kthreadd]
root         3    2 0 11:29 ?        00:00:00 [migration/0]
root         4    2 0 11:29 ?        00:00:00 [ksoftirqd/0]
root         5    2 0 11:29 ?        00:00:00 [watchdog/0]
root         6    2 0 11:29 ?        00:00:00 [events/0]
root         7    2 0 11:29 ?        00:00:00 [khelper]
root        47    2 0 11:29 ?        00:00:00 [kblockd/0]
root        48    2 0 11:29 ?        00:00:00 [kacpid]
68        2349    1 0 11:30 ?        00:00:00 hald
root      2489    1 0 11:30 tty1     00:00:00 /sbin/mingetty tty1
root      2490    1 0 11:30 tty2     00:00:00 /sbin/mingetty tty2
root      2491    1 0 11:30 tty3     00:00:00 /sbin/mingetty tty3
root      2492    1 0 11:30 tty4     00:00:00 /sbin/mingetty tty4
root      2493    1 0 11:30 tty5     00:00:00 /sbin/mingetty tty5
root      2494    1 0 11:30 tty6     00:00:00 /sbin/mingetty tty6
root      2956    1 0 11:42 ?        00:00:00 /usr/sbin/httpd
apache    2958 2956 0 11:42 ?        00:00:00 /usr/sbin/httpd
apache    2959 2956 0 11:42 ?        00:00:00 /usr/sbin/httpd
root      2995    1 0 11:43 ?        00:00:00 auditd
root      2997 2995 0 11:43 ?        00:00:00 /sbin/audispd
root      3078 1981 0 12:00 ?        00:00:00 sshd: rich [priv]
rich      3080 3078 0 12:00 ?        00:00:00 sshd: rich@pts/0
rich      3081 3080 0 12:00 pts/0    00:00:00 -bash
rich      4445 3081 3 13:48 pts/0    00:00:00 ps -ef
$
```

为节省空间，我从输出中删去了一些行，但是您可以看到，Linux 系统中运行了许许多多的进程。本例使用了两参数，-e 参数用于显示系统上运行的所有进程，-f 参数用于扩展输出以显示一些有用的信息列。

- UID：负责启动进程的用户；
- PID：进程的 ID；
- PPID：父进程的 PID（如果某个进程由另一个进程启动）；
- C：进程存续期的处理器利用率；
- STIME：进程启动时的系统时间；
- TTY：进程从中启动的终端设备；
- TIME：运行进程所需的累计 CPU 时间；
- CMD：启动程序的名称。

这将产生一些有用的信息，这也是许多系统管理希望了解的。要查看更多信息，可以使用-l 参数，它将生成长格式的输出：

```
$ ps -l
F S  UID PID PPID C PRI NI ADDR SZ WCHAN TTY      TIME   CMD
0 S  500 3081 3080 0 80   0 -  1173 wait pts/0   00:00:00 bash
```

```
0 R  500 4463 3081 1 80   0 -  1116 - pts/0      00:00:00 ps
$
```

注意使用-l 参数时显示的额外列。

- F：内核分配给进程的系统标记；
- S：进程的状态（O= 在处理器上运行；S= 睡眠；R= 可运行，等待运行；Z= 死进程，进程已终止，但父进程不可用；T= 进程已停止）；
- PRI：进程的优先级（数字越大表示优先级越低）；
- NI：nice value，用于判断优先级；
- ADDR：进程的内存地址；
- SZ：换出进程大致需要的交换空间；
- WCHAN：进程睡眠时所在的内核函数的地址。

另外，需要记住一个极为方便的参数-H。-H 参数将进程组织为分层格式，可以显示哪些流程是随其他进程一起启动的。以下内容选自-efH-格式列表：

```
$ ps -efH
UID        PID  PPID C STIME TTY          TIME CMD
root      3078  1981 0 12:00 ?        00:00:00   sshd: rich [priv]
rich      3080  3078 0 12:00 ?        00:00:00     sshd: rich@pts/0
rich      3081  3080 0 12:00 pts/0    00:00:00       -bash
rich      4803  3081 1 14:31 pts/0    00:00:00         ps -efH
```

注意 CMD 列输出中的变化。这显示了正在运行的进程的层次结构。首先，sshd 进程由根用户启动（这是 Secure shell（SSH）服务器会话，它将监听远程 SSH 连接）。接下来，当从远程终端连接到系统时，主 SSH 进程产生了一个终端进程（pts/0），而该进程又产生了一个 bash shell。

然后，我执行了 ps 命令，它作为子进程显示在 bash 进程中。在多用户系统中，如果需要排除失控进程的故障，或者尝试跟踪它们所属的 userid 或终端，这将是一款极为有用的工具。

### 2. BSD 类型参数

现在，您已经了解了 Unix 参数，现在来看看 BSD 类型参数。伯克利软件发行版（Berkeley Software Distribution，BSD）是由加州大学伯克利分校开发的 Unix 版本。它与 AT&T Unix 系统之间存在许多细微的差异，因此引发了多年的 Unix 纷争。BSD 版本的 ps 命令参数如表 4-2 所示。

表 4-2　　ps 命令的 BSD 参数

| 参数 | 描述 |
|---|---|
| T | 显示与该终端相关的所有进程 |
| a | 显示与任何终端相关的所有进程 |
| g | 显示包括会话标题在内的所有进程 |
| r | 只显示正在运行的进程 |
| x | 显示所有进程，包括未分配终端设备的进程 |
| U *userlist* | 显示 userlist 列表中的 userid 拥有的进程 |
| p *pidlist* | 显示在 pidlist 列表中拥有 PID 的进程 |
| t *ttylist* | 显示与 ttylist 列表中某个终端相关的进程 |
| O *format* | 列出 format 中的具体列，与标准列一并显示 |
| X | 以寄存器格式显示数据 |
| Z | 在输出中包含安全信息 |
| j | 显示作业信息 |
| l | 显示长格式 |
| o *format* | 只显示 format 中指定的列 |
| s | 使用信号格式 |

续表

| 参数 | 描述 |
| --- | --- |
| u | 使用面向用户的格式 |
| v | 使用虚拟内存格式 |
| N *namelist* | 定义在 WCHAN 列中使用的值 |
| O *order* | 定义显示信息列的顺序 |
| S | 针对父进程中的子进程的合计数值信息，比如说 CPU 和内存使用率 |
| c | 显示真实命令名称（用于启动进程的程序名） |
| e | 显示命令使用的任何环境变量 |
| f | 以层次结构显示进程，显示哪个进程启动了哪个进程 |
| h | 不显示标题信息 |
| k *sort* | 定义输出排序使用的列 |
| n | 使用数值表示用户和组 ID，以及 WCHAN 信息 |
| w | 为较宽的终端生成宽输出 |
| H | 将线程作为进程显示 |
| m | 在进程后显示线程 |
| L | 列出所有格式说明符 |
| V | 显示 ps 的版本 |

如您所见，Unix 类型与 BSD 类型的参数之间存在许多相同之处。多数信息采用两种参数类型均可获取。多数情况下，使用的参数类型取决于您的习惯和喜好（例如，您在使用 Linux 之前已经习惯于 BSD 环境）。

使用 BSD 类型参数时，ps 命令会自动更改输出以模拟 BSD 格式。下面是一个使用参数 l 的例子：

```
$ ps l
F UID   PID PPID PRI  NI  VSZ  RSS WCHAN  STAT   TTY       TIME COMMAND
0 500  3081 3080  20   0 4692 1432 wait   Ss     pts/0     0:00 -bash
0 500  5104 3081  20   0 4468  844 -      R+     pts/0     0:00 ps l
$
```

注意，虽然许多输出列都与我们使用 Unix 类型参数时的情况一样，但也有一些不同之处。

- VSZ：进程在内存中的大小，以 KB 为单位；
- RSS：进程使用过的且未被换出的物理内存；
- STAT：由两个字符组成的状态码，用于表示当前进程的状态。

许多系统管理员都比较喜欢 BSD 类型的参数 l，因为它能为进程生成更加详细的状态码（STAT 列）。与单字符的 Unix 类型输出相比，两个字符的状态码能够更加准确的定义进程的具体状态。

第一个字符使用与 Unix 类型的 S 输出列相同的值，显示进程正在睡眠、运行还是等待。第二个字符进一步定义进程的状态。

- <：进程正以高优先级运行。
- N：进程正以低优先级运行。
- L：进程在内存中存在锁定页面。
- s：进程是会话领导者（session leader）。
- l：进程是多线程的。
- +：进程正在前台运行。

从上面那个简单的例子可以看出，bash 命令正在睡眠，但它是一个会话领导者（它是我的会话

中的主进程），而 ps 命令正在系统的前台中运行。

### 3. GNU 长参数

最后，GNU 开发人员在参数组合中添加了一些选项，为新的、经过改进的 ps 命令加入了他们自己的风格。一些 GNU 长参数沿用现有的 Unix 或 BSD 类型的参数，而其他长参数则提供了一些新特性。表 4-3 列出了可用的 GNU 长参数。

表 4-3　　ps 命令的 GNU 参数

| 参数 | 描述 |
|---|---|
| --deselect | 显示在命令行中列出的进程之外的所有其他进程 |
| --Group *grplist* | 显示组 ID 在 *grplist* 中列出的进程 |
| --User *userlist* | 显示用户 ID 在 *userlist* 中列出的进程 |
| --group *grplist* | 显示有效组 ID 在 *grplist* 中列出的进程 |
| --pid *pidlist* | 显示进程 ID 在 *pidlist* 中列出的进程 |
| --ppid *pidlist* | 显示父进程 ID 在 *pidlist* 中列出的进程 |
| --sid *sidlist* | 显示会话 ID 在 *sidlist* 中列出的进程 |
| --tty *ttylist* | 显示终端设备 ID 在 *ttylist* 中列出的进程 |
| --user *userlist* | 显示有效用户 ID 在 *userlist* 中列出的进程 |
| --format *format* | 只显示在 *format* 中指定的列 |
| --context | 显示额外的安全信息 |
| --cols *n* | 将屏幕宽度设置为 *n* 列 |
| --columns *n* | 将屏幕宽度设置为 *n* 列 |
| --cumulative | 包含已停止子进程的信息 |
| --forest | 在层次结构清单中显示进程，以显示父进程 |
| --headers | 对每页输出重新列头（column headers） |
| --no-headers | 不显示列头 |
| --lines *n* | 将屏幕高度设置为 *n* 行 |
| --rows *n* | 将屏幕高度设置为 *n* 行 |
| --sort order | 定义用于排序输出的列 |
| --width *n* | 将屏幕宽度设置为 *n* 列 |
| --help | 显示帮助信息 |
| --info | 显示调试信息 |
| --version | 显示 ps 程序的版本 |

您可以将 GNU 长参数与 Unix 或 BSD 类型的参数结合使用，以定制自己的显示。在 GNU 长参数中，我非常喜欢的一个特性就是--forest 参数。它用于显示进程的层次结构信息，但是会使用 ASCII 字符来绘制图表。

```
  1981 ?        00:00:00 sshd
  3078 ?        00:00:00  \_sshd
  3080 ?        00:00:00      \_sshd
  3081 pts/0    00:00:00          \_bash
 16676 pts/0    00:00:00              \_ps
```

这种格式使得跟踪子进程与父进程成为了小菜一碟。

## 4.1.2　实时进程监控

ps 命令对于收集在系统中运行的进程的信息非常实用，但它同时也有一个缺点。ps 命令只能显示特定时间点的信息。如果您需要查明经常在内存中换进换出的进程的趋势，那么 ps 命令就无能为

力了。

然而，top 命令可以解决此问题。top 命令用于显示与 ps 命令相似的进程信息，不过它采用的是实时模式。图 4-1 显示实际运行中的 top 命令。

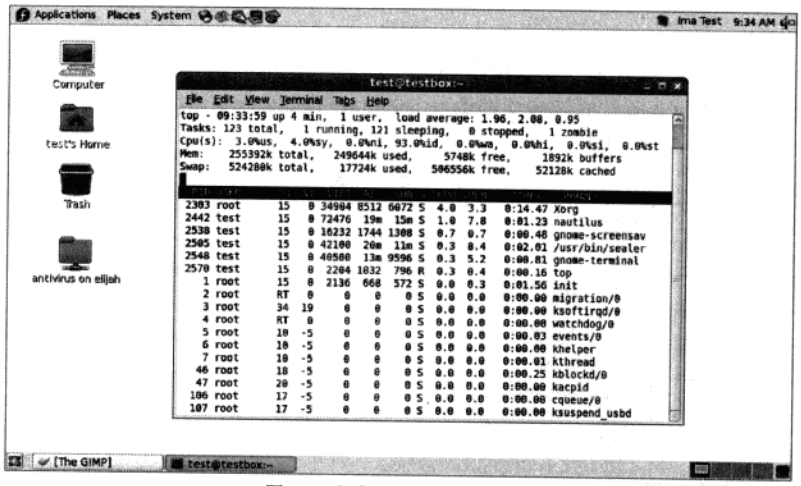



图 4-1 运行 top 命令时的输出

输出的第一部分显示了一般系统信息。第一行显示了当前时间，系统已正常运行的时间，已登录的用户数量，以及系统的平均负载。

平均负载包括 3 个值，分别是 1 分钟平均负载值、5 分钟平均负载值和 15 分钟平均负载值。值越高则系统负载越大。短期活动促使 1 分钟负载值冲高是很常见的。如果 15 分钟负载值处于高位，则您的系统可能遇到了麻烦。

注意：在 Linux 系统管理中，定义确切的高负载平均值是需要技巧的。该值依赖于平时在系统中运行的程序和硬件配置。同一情况，在某个系统中为高值，而在另一个系统中则可能是正常值。一般而言，如果平均负载值超过了 2，则表示您的系统趋于繁忙。

第二行显示一般进程信息（在 top 中称为**任务**）：有多少进程正在运行、睡眠、已停止和僵死（已经运行完毕，但父进程未响应）。

下一行显示 CPU 一般信息。top 命令会根据进程所有者（用户或系统进程）和进程的状态（运行、空闲或等待）将 CPU 利用率划分成若干个类别。

接下来的两行详细给出了系统内存的状态。第一行显示系统物理内存的状态，内存总量、当前使用的内存量和空闲内存量。第二行显示系统交换内存区域（如果已经安装）的状态，包括与前面相同的信息。

最后，下一部分显示当前正在运行的进程的详细列表，其中一些信息列与 ps 命令输出相似。

- PID：进程的 ID；
- USER：进程所有者的用户名；
- PR：进程的优先级；
- NI：进程的 nice value；
- VIRT：进程所使用的虚拟内存总量；
- RES：进程当前正使用的物理内存量；
- SHR：进程与其他进程共享的内存量；

- S：进程状态（D = 可中断睡眠，R = 运行中，S = 睡眠，T = 已跟踪或已停止，Z = 僵死）；
- %CPU：进程正使用的 CPU 时间份额；
- %MEM：进程正使用的可用物理内存份额；
- TIME+：进程启动后已使用的总 CPU 时间；
- COMMAND：进程的命令行名称（已启动程序）。

默认情况下，当启动 top 时，它会根据%CPU 的值对进程进行排序。可以在运行 top 时使用交互式命令来更改排序顺序。各交互式命令都是一个字符，可以在 top 运行过程中按下该键，更改程序的行为。这些命令如表 4-4 所示。

表 4-4　top 交互式命令

| 命令 | 描述 |
|---|---|
| 1 | 在单个 CPU 与 Symmetric Multiprocessor (SMP)状态之间切换 |
| b | 启用表中重要数字粗体显示 |
| I | 切换 Irix/Solaris 模式 |
| z | 配置表的颜色 |
| l | 切换平均负载信息行显示 |
| t | 切换 CPU 信息行显示 |
| m | 切换 MEM 和 SWAP 信息行显示 |
| f | 添加或删除不同的信息列 |
| o | 更改信息列的显示顺序 |
| F 或 O | 选择对进程进行排序的字段（默认为%CPU） |
| <或> | 将排序字段左移 1 列（<）或右移 1 列（>） |
| r | 在正常排序或反序排序之间切换 |
| h | 切换线程显示 |
| c | 切换显示进程的命令名或完整命令行（包括参数） |
| i | 切换空闲进程显示 |
| S | 切换显示累计 CPU 时间或相对 CPU 时间 |
| x | 切换排序字段高亮显示 |
| y | 切换运行中任务的高亮显示 |
| z | 切换颜色和 mono 模式 |
| b | 切换 x 和 y 模式的粗体模式 |
| u | 显示特定用户的进程 |
| n 或# | 设置要显示进程的数量 |
| k | 杀死特定的进程（仅适用于进程所有者和根用户） |
| r | 更改特定进程的优先级（renice）（仅适用于进程所有者和根用户） |
| d 或 s | 更改更新间隔（默认为 3 秒） |
| W | 将当前设置写入配置文件 |
| q | 退出 top 命令 |

可见，有很多方法可以控制 top 命令输出。使用此工具，您可以经常发现系统中的异常进程。当然，发现异常进程后，下一个任务就是停止它，这正是下一节的主题。

### 4.1.3　停止进程

知道何时及如何停止进程是系统管理员的一项关键素质。有时，进程会进入挂起状态，这时需要稍作调整让它恢复运行或停止运行。另一种情况是，进程不受 CPU 控制且拒绝停止。在这两种情况下，

都需要通过一个命令来控制进程。Linux 沿用了 Unix 中的进程间通信方法。

在 Linux 中，各进程使用**信号**相互通信。进程信号是进程可识别的预定义信息，进程可以选择忽略它或做出响应。开发人员规划进程如何处理信号。大多数精心编写的应用程序都能够接收标准 Unix 进程信号，并对此做出响应。这些信号如表 4-5 所示。

表 4-5 Linux 进程信号

| 信号 | 名称 | 描述 |
|---|---|---|
| 1 | HUP | 挂起 |
| 2 | INT | 中断 |
| 3 | QUIT | 停止运行 |
| 9 | KILL | 强制终止 |
| 11 | SEGV | 段违例 |
| 15 | TERM | 条件终止 |
| 17 | STOP | 强制停止，但未终止 |
| 18 | TSTP | 停止或暂停，但继续在后台运行 |
| 19 | CONT | STOP 或 TSTP 之后恢复执行 |

Linux 提供了两个命令用于发送进程信号给运行中的进程。

**1. kill 命令**

kill 命令允许根据进程 ID（PID）将信号发送给相关进程。默认情况下，kill 命令会发送一个 TERM 命令给命令行中列出的所有 PID。遗憾的是，您只能使用进程 PID，而不是它的命令名称，这使 kill 命令有时难以使用。

要发送进程信号，您必须是该进程的所有者或已作为根用户登录。

```
$ kill 3940
-bash: kill: (3940) - Operation not permitted
$
```

TERM 信号将友好地通知进程停止运行。遗憾的是，如果某个进程已失控，它极有可能会忽略该请求。当您需要一些强制措施时，可以使用-s 参数来指定其他信号（使用它们的名称或信号编号）。

常用的流程是首先尝试 TERM 信号。如果进程忽略该信号，则尝试 INT 或 HUP 信号。如果进程能够识别这些信号，那么它会在关闭之前停止执行以前的操作。最具强制力的信号是 KILL 信号。当进程接收到此信号，它会立即停止运行。这有时会对文件造成破坏。

从下面的例子中可以看到，kill 命令没有与之相关的输出。

```
# kill -s HUP 3940
#
```

要验证命令是否有效，需要执行另一个 ps 或 top 命令来查看异常进程是否已停止。

**2. killall 命令**

killall 命令非常强大，它可以根据进程的名称（而非 PID）来停止进程。killall 命令还允许使用通配符，它对于处理出错的系统非常有用。

警告：作为根用户登录时，使用 killall 命令时要格外小心。若通配符使用不当，则很容易意外停止重要的系统进程。这可能会导致文件系统受损。

## 4.2 监控磁盘空间

系统管理员的另一个重要任务是跟踪系统的磁盘使用情况。无论是运行简单的 Linux 桌面还是

大型 Linux 服务器，都需要了解可供应用程序使用的空间。

您可以使用一些命令行命令来管理 Linux 系统上的介质环境。本节介绍管理系统时最常用的一些核心命令。

## 4.2.1　挂载介质

如第 3 章所述，Linux 文件系统将所有介质磁盘合并到一个虚拟目录中。使用新介质磁盘之前，需要将它放在一个虚拟目录中。这项任务称为**挂载**。

在如今的图形桌面世界中，大多数 Linux 发行版都能够自动挂载特定类型的**可移动介质**。显然，可移动介质设备就是可以轻易从 PC 中移除的介质，比如 CD-ROM、软盘和 USB 记忆棒。

如果您未使用支持自动挂载和卸载可移动介质的发行版，则需要手动完成这些操作。本节介绍帮助您管理可移动介质设备的 Linux 命令行命令。

### 1. mount 命令

mount 命令用于挂载介质。默认情况下，mount 命令显示系统当前挂载的介质设备列表。

```
$ mount
/dev/mapper/VolGroup00-LogVol00 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
devpts on /dev/pts type devpts (rw,gid=5,mode=620)
/dev/sda1 on /boot type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
none on /proc/sys/fs/binfmt_misc type binfmt_misc (rw)
sunrpc on /var/lib/nfs/rpc_pipefs type rpc_pipefs (rw)
/dev/sdb1 on /media/disk type vfat
(rw,nosuid,nodev,uhelper=hal,shortname=lower,uid=503)
$
```

mount 命令提供了 4 条信息：

- 介质的设备位置；
- 介质在虚拟目录中的挂载点；
- 文件系统类型；
- 已挂载介质的访问状态。

上例中的最后一个条目是 GNOME 桌面在/media/disk 挂载点自动挂载的 USB 记忆棒。vfat 文件系统类型显示它已在 Microsoft Windows PC 上经过格式化。

要手动在虚拟目录中挂载介质设备，需要作为根用户登录。用于手动挂载介质设备的基本命令如下：

```
mount -t type device directory
```

type 参数定义格式化磁盘的文件系统类型。Linux 可以识别许多不同的文件系统类型。如果您需要与 Windows PC 共享可移动介质设备，则最可能的类型包括以下几种。

- vfat：Windows 长文件系统。
- ntfs：Windows 高级文件系统，在 Windows NT、XP 和 Vista 中使用。
- iso9660：标准 CD-ROM 文件系统。

大多数 USB 记忆棒和软盘都使用 vfat 文件系统进行格式化。如果需要挂载某个数据 CD，则需要使用 iso9660 文件系统类型。

接下来的两个参数定义介质设备文件的位置，挂载点在虚拟目录中的位置。例如，要将设备/dev/sdb1 上的 USB 记忆棒手动挂载到/media/disk 位置，可以使用以下命令：

```
mount -t vfat /dev/sdb1 /media/disk
```

在虚拟目录中挂载介质设备之后，根用户将拥有对该设备的完全访问权限，但其他用户的访问则受限。可以使用目录权限（第 6 章将介绍此内容）来控制可以访问设备的用户。

如果需要使用 mount 命令中比较特殊的特性，则可用的参数如表 4-6 所示。

表 4-6 mount 命令参数

| 参数 | 描述 |
| --- | --- |
| -a | 挂载/etc/fstab 文件中指定的所有文件系统 |
| -f | 使 mount 命令模拟挂载某个设备，但并不实际挂载 |
| -F | 和-a 参数连用时，同时挂载所有文件系统 |
| -v | 详细模式，解释挂载设备所需的所有步骤 |
| -I | 不使用/sbin/mount.filesystem 中的任何文件系统帮助类文件 |
| -l | 自动为 ext2、ext3 或 XFS 文件系统添加文件系统标签 |
| -n | 挂载设备，而不在/etc/mstab 挂载设备文件中注册 |
| -p num | 对于加密的挂载，从文件描述符 num 中读取 passphrase |
| -s | 忽略文件系统不支持的挂载选项 |
| -r | 以只读方式挂载设备 |
| -w | 以读写方式挂载设备（默认） |
| -L label | 以指定的 label 挂载设备 |
| -U uuid | 以指定的 uuid 挂载设 |
| -0 | 和-a 参数连用时，限制使用的文件系统集 |
| -o | 在文件系统中添加特定选项 |

-o 选项允许使用一列逗号分隔额外选项挂载文件系统。常用的选项包括以下几种。

- ro：以只读形式挂载。
- rw：只读写形式挂载。
- user：允许普通用户挂载文件系统。
- check=none：挂载文件系统，而不执行完整性检查。
- loop：挂载某个文件。

最近，人们流行在 Linux 中将 CD 作为.iso 文件分发。.iso 文件是 CD 在单个文件中的完整映像。大多数 CD 刻录软件包都可以根据.iso 文件创建新 CD。Mount 命令的一个特性是，可以直接将.iso 文件挂载到 Linxu 虚拟目录中，而无需将其刻录到 CD 中。这可以通过在 loop 选项中使用-o 参数来完成：

```
$ mkdir mnt
$ su
Password:
# mount -t iso9660 -o loop MEPIS-KDE4-LIVE-DVD_32.iso mnt
# ls -l mnt
total 16
-r--r--r-- 1 root root 702 2007-08-03 08:49 about
dr-xr-xr-x 3 root root 2048 2007-07-29 14:30 boot
-r--r--r-- 1 root root 2048 2007-08-09 22:36 boot.catalog
-r--r--r-- 1 root root 894 2004-01-23 13:22 cdrom.ico
-r--r--r-- 1 root root 5229 2006-07-07 18:07 MCWL
dr-xr-xr-x 2 root root 2048 2007-08-09 22:32 mepis
dr-xr-xr-x 2 root root 2048 2007-04-03 16:44 OSX
-r--r--r-- 1 root root 107 2007-08-09 22:36 version
# cd mnt/boot
# ls -l
total 4399
dr-xr-xr-x 2 root root 2048 2007-06-29 09:00 grub
-r--r--r-- 1 root root 2392512 2007-07-29 12:53 initrd.gz
-r--r--r-- 1 root root 94760 2007-06-14 14:56 memtest
-r--r--r-- 1 root root 2014704 2007-07-29 14:26 vmlinuz
#
```

mount 命令挂载了.iso CD 映像文件，就像它是真正的 CD 一样，并允许我们操作它的文件系统。

### 2. umount 命令

要移除可移动介质设备，您不应简单地从系统中移除它。相反，应该总是首先**卸载**它。

提示：Linux 不允许您弹出已挂载的 CD。如果您在从驱动器中移除 CD 时遇到了麻烦，则该 CD 极有可能仍然挂载在虚拟目录中。首先卸载它，然后再尝试弹出它。

用于卸载设备的命令是 umount（是的，该命令中没有“n”，这有时确实会造成混淆）。umount 命令的格式相当简单：

```
umount [directory | device ]
```

umount 命令允许您通过设备位置或挂载目录名称来定义介质设备。如果设备中包含任何打开的文件，则系统将不允许您卸载它。

```
[root@testbox boot]# umount /home/rich/mnt
umount: /home/rich/mnt: device is busy
umount: /home/rich/mnt: device is busy
[root@testbox boot]# cd /home/rich
[root@testbox rich]# umount /home/rich/mnt
[root@testbox rich]# ls -l mnt
total 0
[root@testbox rich]#
```

在本例中，尽管我没有使用已挂载.iso 镜像文件中的文件，但我仍然在文件系统结构的某个目录中，因此 umount 命令不允许我卸载镜像文件。当移除镜像文件的文件系统之后，便能成功卸载镜像文件。

## 4.2.2　使用 df 命令

有时，需要了解某个设备上有多少可用磁盘空间。df 命令允许我们方便地查看所有已挂载磁盘的使用情况：

```
$ df
Filesystem         1K-blocks    Used Available Use% Mounted on
/dev/sda2          18251068 7703964   9605024  45% /
/dev/sda1            101086   18680     77187  20% /boot
tmpfs                119536       0    119536   0% /dev/shm
/dev/sdb1            127462  113892     13570  90% /media/disk
$
```

df 命令显示每个包含数据的已挂载文件系统。从之前的 mount 命令中可以看出，一些挂载设备用于内部系统目的。该命令显示：

- 设备位置；
- 能保存的数据块数量（每块 1 024 字节）；
- 已使用的数据块数量（每块 1 024 字节）；
- 可用的数据块数量（每块 1 024 字节）；
- 已使用空间的百分比；
- 设备挂载点。

df 命令也提供了一些不同的命令行参数，但大部分都很少使用。其中一个较为常用的参数是-h，它用于以可读形式显示磁盘空间，通常使用 M 表示兆字节，使用 G 表示吉字节：

```
$ df -h
Filesystem         Size Used Avail Use% Mounted on
/dev/sdb2           18G 7.4G  9.2G  45% /
/dev/sda1           99M  19M   76M  20% /boot
tmpfs              117M     0  117M   0% /dev/shm
/dev/sdb1          125M 112M   14M  90% /media/disk
$
```

现在，命令不再使用块数量来表示磁盘空间大小，它使用“普通”格式显示所有的磁盘大小。df 命令对于排除系统中的磁盘空间问题非常有用。

注意：记住，Linux 系统始终会在后台运行一些处理文件的进程。df 命令提供的值反映 Linux 系统认可的该时间点的当前值。运行中的进程可能已创建或删除了某个文件，只是还没有释放它。这个值将不会包括在空闲空间计算中。

### 4.2.3 使用 du 命令

借助 df 命令，可以方便地了解磁盘是否已空间不足。系统管理员的下一个问题是需要了解磁盘空间不足时该怎么办。

帮助我们解决这个问题的命令就是 du 命令。du 命令显示特定目录的磁盘使用情况（默认为当前目录）。这是判断系统是否存在磁盘占用大户的快捷方法。

默认情况下，du 命令显示当前目录下的所有的文件、目录和子目录，并显示各文件或目录占用了多大空间。对于标准目录，这是很长的一个清单。下面给出了 du 命令显示的部分清单：

```
$ du
484     ./.gstreamer-0.10
8       ./Templates
8       ./Download
8       ./.ccache/7/0
24      ./.ccache/7
368     ./.ccache/a/d
384     ./.ccache/a
424     ./.ccache
8       ./Public
8       ./.gphpedit/plugins
32      ./.gphpedit
72      ./.gconfd
128     ./.nautilus/metafiles
384     ./.nautilus
72      ./.bittorrent/data/metainfo
20      ./.bittorrent/data/resume
144     ./.bittorrent/data
152     ./.bittorrent
8       ./Videos
8       ./Music
16      ./.config/gtk-2.0
40      ./.config
8       ./Documents
```

各行右侧的数字表示各文件或目录占用的数据块数量。注意，清单从目录底部向上依次列出包含在目录中的文件和子目录。

就其自身而言，du 命令并没有多大的用处。它虽然能够显示各文件和目录占用的磁盘空间，但是当您要查找某个文件或目录，则需要查看一页又一页的信息。

du 命令提供了一些命令行参数，可以提高清单的可读性。

- -c：生成所有已列出文件的总计。
- -h：以可读形式输出大小，使用 K 表示千字节，M 表示兆字节，G 表示吉字节。
- -s：汇总各自变量。

系统管理员下一步将使用文件处理命令来操作大量数据。这正是下一节的内容。

## 4.3 操作数据文件

当您拥有大量数据时，处理信息并对它加以利用往往变得非常困难。正如上一节中 du 命令用法

所示，使用系统命令时很容易出现数据过载的情况。

Linux 系统提供了一些命令行工具，用于帮助我们管理大量数据。本节介绍每个系统管理（以及任何日常 Linux 用户）都应该了解的基本命令。

### 4.3.1　数据排序

在操作大量数据时，一个常用的命令就是 sort 命令。sort 命令的作用是对数据进行排序。

默认情况下，sort 命令使用您为会话指定的默认语言的标准排序规则对文本文件中的数据行进行排序。

```
$ cat file1
one
two
three
four
five
$ sort file1
five
four
one
three
two
$
```

相当简单。但是事情并非总是像它们表面上那么简单。看看下面这个例子：

```
$ cat file2
1
2
100
45
3
10
145
75
$ sort file2
1
10
100
145
2
3
45
75
$
```

如果您希望按数字顺序对上述数字排序，那么您会失望了。默认情况下，sort 命令将数字解释为字符，并执行标准的字符排序，因此生成的输出可能不是您所期望的。要解决此问题，可以使用-n 参数，它会通知 sort 命令将数字识别为数字（而不是字符），并根据数值大小对它们进行排序：

```
$ sort -n file2
1
2
3
10
45
75
100
145
$
```

现在，情况好多了！另一个常用的参数是-M，即按月份排序。Linux 日志文件通常会在每行开始处包含一个时间信息戳，以指示事件的发生时间：

```
Sep 13 07:10:09 testbox smartd[2718]: Device: /dev/sda, opened
```

如果使用默认排序对使用时间戳日期的文件进行排序，则会获得以下结果：

```
$ sort file3
Apr
Aug
Dec
Feb
Jan
Jul
Jun
Mar
May
Nov
Oct
Sep
$
```

这不是您所希望的。如果您使用-M 参数，则 sort 命令会识别由 3 个字符组成的月份缩写，并对它们进行恰当排序：

```
$ sort -M file3
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
$
```

还有其他一些排序参数可以使用，如表 4-7 所示。

**表 4-7　　sort 命令参数**

| 单线 | 双线 | 描述 |
|---|---|---|
| -b | --ignore-leading-blanks | 排序时忽略前面的空格 |
| -C | --check=quiet | 不执行排序，但不报告数据是否无序排列 |
| -c | --check | 不执行排序，但检查数据是否已经排序。若未排序，则报告 |
| -d | --dictionary-order | 只考虑空格和字母数字；不考虑特殊字符 |
| -f | --ignore-case | 默认情况下，大写字母排在前面。此参数将忽略大小写 |
| -g | --general-numeric-sort | 使用普通数值进行排序 |
| -i | --ignore-nonprinting | 在排序中忽略非打印字符 |
| -k | --key=POS1[,POS2] | 根据 POS1 位置进行排序，并在 POS2 位置结束（如果指定了该参数） |
| -M | --month-sort | 使用由 3 个字符组成的月份缩写按月份排序 |
| -m | --merge | 合并两个已经执行排序的数据文件 |
| -n | --numeric-sort | 按字符串数值进行排序 |
| -o | -output=file | 将结果写入指定的文件 |
| -R | --random-sort | 根据随机散列值排序 |
|  | --random-source=FILE | 指定-R 参数使用的随机字节文件 |
| -r | --reverse | 反转排列（降序，而不是升序） |
| -S | --buffer-size=SIZE | 指定要使用的内存量 |
| -s | --stable | 禁用 last-resort comparison |

续表

| 单线 | 双线 | 描述 |
| --- | --- | --- |
| -T | --temporary-direction=DIR | 指定存储临时工作文件的位置 |
| -t | --field-separator=SEP | 指定用于区分关键位置的字符 |
| -u | --unique | 带-c参数，检查严格排序；无-c参数，只输出两个相似行中的第一行 |
| -z | --zero-terminated | 使用NULL字符结束所有行，而不是换行符 |

-k和-t参数可以方便对使用字段的数据进行排序，比如说/etc/passwd文件。-t参数用于指定字段分隔符，-k参数用于指定排序的字段。例如，要根据userid数值对密码文件进行排序，可以这样操作：

```
$ sort -t ':' -k 3 -n /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
news:x:9:13:news:/etc/news:
uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
gopher:x:13:30:gopher:/var/gopher:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
```

现在，命令已经根据第三个字段，即userid数值，对数据进行了排序。

-n参数用于对数值输出进行排序，比如du命令的输出：

```
$ du -sh * | sort -nr
1008k    mrtg-2.9.29.tar.gz
972k     bldg1
888k     fbs2.pdf
760k     Printtest
680k     rsync-2.6.6.tar.gz
660k     code
516k     fig1001.tiff
496k     test
496k     php-common-4.0.4pl1-6mdk.i586.rpm
448k     MesaGLUT-6.5.1.tar.gz
400k     plp
```

注意，-r选项还可以按降序对数值进行排序，因此您可以方便地查看哪些文件在目录中占的空间最多。

### 4.3.2 搜索数据

经常需要查找大文件中的特定数据行。可以使用grep命令来完成搜索任务，而不必在整个文件手动查找。grep命令的命令行格式如下：

```
grep [options] pattern [file]
```

grep命令将在输入或您指定的文件中搜索包含与指定模式（pattern）匹配的字符的数据行。grep命令的输出是包含匹配模式的数据行。

下面给出了使用grep命令搜索“数据排序”一节中使用的file1文件的两个简单示例：

```
$ grep three file1
three
$ grep t file1
two
```

```
three
$
```

第一个例子在 file1 文件中搜索与模式 three 匹配的文本。grep 命令输出了包含匹配模式的数据行。第二个例子在 file1 文件中搜索与模式 t 匹配的文本。在本例中，共有两行数据与指定模式相匹配，并且都已经显示。

由于 grep 命令广受欢迎，因此在其生命期中经历了许多改进。grep 命令中添加了许多特性。如果您在手册中查阅 grep 命令，会发现它的功能有多么强大。

如果您希望执行反转搜索（输出不匹配模式的数据行），可以使用-v 参数：

```
$ grep -v t file1
one
four
five
$
```

如果您希望获取与模式匹配的数据所在的行号，可以使用-n 参数：

```
$ grep -n t file1
2:two
3:three
$
```

如果您只需要了解有多少行包含匹配模式，可以使用-c 参数：

```
$ grep -c t file1
2
$
```

如果您需要指定多个匹配模式，可以使用-e 参数指定各模式：

```
$ grep -e t -e f file1
two
three
four
five
$
```

本例输出包含字符 t 或字符 f 的行。

默认情况下，grep 命令使用基本 Unix 类型的正则表达式来匹配模式。Unix 类型的正则表达式使用特殊字符定义如何查找匹配模式。

交叉参考：有关正则表达式的详细信息，请参阅第 17 章。

下面给出了在 grep 搜索中使用正则表达式的例子：

```
$ grep [tf] file1
two
three
four
five
$
```

正则表达式中的方括号表明 grep 命令应查找包含 t 或 f 字符的数据。如果未使用正则表达式，则 grep 将搜索与字符串 tf 匹配的文本。

egrep 命令是 grep 命令的一个分支，它允许您指定 POSIX 扩展的正则表达式，即使用多个字符指定匹配模式（详细信息请参阅第 17 章）。fgrep 命令则允许您使用一列由换行符分隔的固定字符串值来指定匹配模式。这允许您将一列字符串放置在一个文件中，然后在 fgrep 命令中使用该列表搜索大文件中的字符串。

### 4.3.3 压缩数据

如果您是一名 Microsoft Windows 用户，那么肯定使用过 zip 文件。由于非常流行，Microsoft 最终将该特性整合到了 Windows XP 操作系统中。zip 工具允许您方便地将大文件（文本或可执行文件）压

缩为较小的文件，以节省空间。

Linux 提供了一些文件压缩工具。这听起来非常不错，但是在尝试下载文件时，它经常会造成混淆或混乱。表 4-8 列出了 Linux 中可用的文件压缩工具。

表 4-8　　Linux 文件压缩工具

| 工具 | 文件扩展名 | 描述 |
| --- | --- | --- |
| bzip2 | .bz2 | 使用 Burrows-Wheeler 块排序文本压缩算法和 Huffman 编码 |
| compress | .Z | 原 Unix 文件压缩工具；现在已经很少使用了 |
| gzip | .gz | GNU 项目的压缩工具；使用 Lempel-Ziv 编码 |
| zip | .zip | Unix 版本的 Windows PKZIP 程序 |

大多数 Linux 系统都没有安装 compress 文件压缩工具。如果您下载了扩展名为 .Z 的文件，则可以安装 compress 包（在许多 Linux 发行版中被称为 ncompress），然后使用 uncompress 命令解压缩文件。

### 1. bzip2 工具

bzip2 工具是一个相对较新的压缩包，它在压缩大型二进制文件方面正日益普及。bzip2 包中的工具包括以下几种。

- bzip2：用于压缩文件；
- bzcat：用于显示已压缩文本文件的内容；
- bunzip2：用于解压 .bz2 文件；
- bzip2recover：用于尝试恢复受损的压缩文件。

默认情况下，bzip2 命令尝试压缩原始文件，并将它替换为压缩后的文件，压缩后的文件名相同，但使用 .bz2 扩展名：

```
$ ls -l myprog
-rwxrwxr-x 1 rich rich 4882 2007-09-13 11:29 myprog
$ bzip2 myprog
$ ls -l my*
-rwxrwxr-x 1 rich rich 2378 2007-09-13 11:29 myprog.bz2
$
```

myprog 程序的初始大小为 4882 字节，经过 bzip2 压缩后，现在只有 2 378 字节。此外，需要注意，bzip2 命令会自动使用 .bz2 扩展名对原始文件进行重命名，扩展名指示了文件所使用的压缩技术。

要解压文件，可以使用 bunzip2 命令：

```
$ bunzip2 myprog.bz2
$ ls -l myprog
-rwxrwxr-x 1 rich rich 4882 2007-09-13 11:29 myprog
$
```

如您所见，解压后的文件将返回初始文件大小。压缩文本文件之后，您将无法使用标准的 cat、more 或 less 命令来查看数据。相反，您需要使用 bzcat 命令：

```
$ bzcat test.bz2
This is a test text file.
The quick brown fox jumps over the lazy dog.
This is the end of the test text file.
$
```

bzcat 命令用于显示压缩文件内部的文本，而无需解压实际的文件：

### 2. gzip 工具

目前为止，gzip 工具是 Linux 中最流行的文件压缩工具。gzip 包是由 GNU 项目创建的，其用意是创建一个免费版本的 Unix compress 工具。该包包含以下文件。

- gzip：用于压缩文件；
- gzcat：用于显示压缩后的文本文件的内容；
- gunzip：用于解压文件。

这些工具采用与 bzip2 工具相同的方式：

```
$ gzip myprog
$ ls -l my*
-rwxrwxr-x 1 rich rich 2197 2007-09-13 11:29 myprog.gz
$
```

gzip 命令将压缩在命令行中指定的文件。还可以指定多个文件名，或者甚至使用通配符一次压缩多个文件：

```
$ gzip my*
$ ls -l my*
-rwxr--r--    1 rich    rich          103 Sep  6 13:43 myprog.c.gz
-rwxr-xr-x    1 rich    rich         5178 Sep  6 13:43 myprog.gz
-rwxr--r--    1 rich    rich           59 Sep  6 13:46 myscript.gz
-rwxr--r--    1 rich    rich           60 Sep  6 13:44 myscript~.gz
$
```

gzip 命令将压缩目录中匹配通配符模式的所有文件。

### 3. zip 工具

zip 工具与 Phil Katz 为 MS-DOS 和 Windows 创建的流行的 PKZIP 包兼容。Linux zip 包中包含 4 个工具。

- zip：用于创建包含列出文件和目录的压缩文件。
- zipcloak：用于创建包含列出文件和目录的加密压缩文件。
- zipnote：用于提取 zip 文件中注释。
- zipsplit：用于将 zip 文件分割成指定大小的多个小文件（用于将大 zip 文件复制到软盘中）。
- unzip：用于提取压缩 zip 文件中的文件和目录。

要查看 zip 工具中可用的所有选项，只需在命令行输入该命令：

```
$ zip
Copyright (C) 1990-2005 Info-ZIP
Type 'zip "-L"' for software license.
Zip 2.31 (March 8th 2005). Usage:
zip [-options] [-b path] [-t mmddyyyy] [-n suffixes] [zipfile list]
[-xi list]
  The default action is to add or replace zipfile entries from list,
which can include the special name - to compress standard input.
 If zipfile and list are omitted, zip compresses stdin to stdout.
-f freshen: only changed files  -u update: only changed or new files
-d delete entries in zipfile    -m move into zipfile (delete files)
-r recurse into directories     -j junk directory names
-0 store only                   -l convert LF to CR LF
-1 compress faster              -9 compress better
-q quiet operation              -v verbose operation
-c add one-line comments        -z add zipfile comment
-@ read names from stdin        -o make file as old as latest entry
-x exclude the following names  -i include only the following names
-F fix zipfile (-FF try harder) -D do not add directory entries
-A adjust self-extracting exe   -J junk zipfile prefix (unzipsfx)
-T test zipfile integrity       -X eXclude eXtra file attributes
-y store symbolic links as the link instead of the referenced file
-R PKZIP recursion (see manual)
-e encrypt                      -n don't compress these suffixes
$
```

zip 工具的强大之处在于能够将文件的整个目录压缩为一个文件。这是归档完整目录结构的理想方法：

```
$ zip -r testzip test
```

```
  adding: test/ (stored 0%)
  adding: test/test1/ (stored 0%)
  adding: test/test1/myprog2 (stored 0%)
  adding: test/test1/myprog1 (stored 0%)
  adding: test/myprog.c (deflated 39%)
  adding: test/file3 (deflated 2%)
  adding: test/file4 (stored 0%)
  adding: test/test2/ (stored 0%)
  adding: test/file1.gz (stored 0%)
  adding: test/file2 (deflated 4%)
  adding: test/myprog.gz (stored 0%)
$
```

本例将创建一个名称为 testzip.zip 的 zip 文件，并递归 test 目录，将发现的各文件和目录添加到该 zip 文件中。从输出中可以看到，并非 zip 文件中的所有文件都可压缩。zip 工具将自动判断为各文件使用的最佳压缩类型。

警告：当您使用 zip 命令中的递归特性时，文件将存储在 zip 文件中的相同目录结构中。包含在子目录中的文件将保存在相同子目录中的 zip 文件中。在解压文件时，您必须格外小心，unzip 命令将在新位置重建整个目录结构。当您拥有许多子目录和文件时，这会造成混淆。

### 4.3.4　归档数据

虽然 zip 命令在压缩和归档数据方面格外出色，但它不是 Unix 和 Linux 世界中的标准工具。目前，Unix 和 Linux 中最流行的归档工具是 tar 命令。

最初，tar 命令用于将文件归档到磁带设备中。但是，它还可以将输出写入到文件中，这已成为在 Linux 中归档文件的流行方式：

tar 命令格式如下：

```
tar function [options] object1 object2 ...
```

function 参数定义 tar 命令的职责，如表 4-9 所示。

表 4-9　　tar 命令功能

| 功能 | 长名称 | 描述 |
|---|---|---|
| -A | --concatenate | 将已有 tar 归档文件添加到另一个已有归档文件中 |
| -c | --create | 创建一个新 tar 归档文件 |
| -d | --diff | 检查 tar 归档文件和文件系统之间的差异 |
| | --delete | 从已有 tar 归档文件中删除 |
| -r | --append | 将文件添加到已有 tar 归档文件的末尾 |
| -t | --list | 列出现有 tar 归档文件的内容 |
| -u | --update | 更新现有 tar 归档文件中的文件 |
| -x | --extract | 解压现有归档文件 |

各功能使用**选项**定义 tar 归档文件的特定行为。表 4-10 列出了 tar 命令的常用选项。

表 4-10　　tar 命令选项

| 选项 | 描述 |
|---|---|
| -C dir | 切换到指定目录 |
| -f file | 输出结果到文件（或设备）file |
| -j | 将输出重定向到 bzip2 命令进行压缩 |
| -p | 保留所有文件权限 |
| -v | 在处理过程中列出文件 |
| -z | 将输出重定向到 gzip 命令进行压缩 |

这些选项通常结合在一起创建以下场景：

```
tar -cvf test.tar test/ test2/
```

这将创建一个名称为 test.tar 的归档文件，其中包含 test1 目录和 test2 目录的内容。

```
tar -tf test.tar
```

这将列出（但不解压）tar 文件 test.tar 的内容。

```
tar -xvf test.tar
```

这将解压 test.tar 文件的内容。如果 tar 文件在创建时拥有目录结构，则整个目录结构将在当前目录中被重建。

如您所见，tar 命令可以方便地将整个目录结构创建为归档文件。在 Linux 世界中，这是为开源应用程序分发源代码文件的一种常见方法。

提示：如果您下载了开源软件，您将看到以 .tgz 结尾的文件名。它们是经过 gzip 压缩的 tar 文件，可以使用 tar -zxvf filename.tgz 对它们进行解压。

## 4.4 小结

本章讨论了供 Linux 系统管理员和程序员使用的一些比较高级的 bash 命令。ps 和 top 命令对于确定系统的状态至关重要，它们允许您查看正在运行的应用程序以及它们占用的资源。

在可移动介质时代，系统管理员的另一项常见任务是挂载存储设备。mount 命令允许您将存储设备挂载到 Linux 虚拟目录结构中。要移除设备，您可以使用 umount 命令。

最后，本章讨论了各种数据处理工具。sort 工具可以方便地对大数据文件进行排序，以帮助您组织数据；grep 工具允许您快速搜索大数据文件中的特定信息。Linux 还提供了一些不同的文件压缩工具，包括 bzip2、gzip 和 zip。每个工具都允许您压缩大文件，以帮助节省文件系统的空间。Linux tar 工具是将目录结构归档到单个文件中的流行方法，该文件可以方便地迁移到另一个系统中。

下一章将讨论 Linux 环境变量。环境变量允许您在脚本中访问关于系统的信息，并提供了一种方便的方法用于在脚本中保存数据。

CHAPTER 5

# 第5章 使用Linux环境变量

本章内容：

- 使用环境变量；
- 设置自己的环境变量；
- 高级变量技巧；
- 使用别名。

Linux环境变量有助于定义Linux shell体验。但是，它们对于新的Linux用户却是一个不易理解的话题。许多程序和脚本都使用环境变量来获取系统信息并存储临时数据和配置信息。Linux系统中的许多地方都设置了环境变量。并且，了解它们的具体位置相当重要。本章将带领您漫游Linux环境变量的世界，您将了解它们的位置、使用方法，以及如何创建自己的环境变量。最后，本章将介绍一个与此相关的话题：在您的shell会话中定义和使用别名。

## 5.1 什么是环境变量

bash shell使用一种称为**环境变量**的特性来存储关于shell会话和工作环境的信息（环境变量的名称由此而来）。该特性还允许您将数据存储在内存中，以便于在shell中运行的程序或脚本访问它们。这是一种便捷的、用于存储持久性数据的方式，这些数据可以标识用户账户、系统、shell或任何需要存储的内容的特性。

bash shell中共有两种类型的环境变量：

- 全局变量；
- 本地变量。

本节介绍环境变量的各种类型，以及如何检查和使用它们。

注意：虽然bash shell使用的具体环境变量是一致的，但不同的Linux发行版通常添加了自己的环境变量。本章介绍的环境变量示例可能与您的发行版中可用的环境变量稍微有所不同。如果您遇到未在本章中介绍的环境变量，请查阅您的Linux发行版的相关文档。

### 5.1.1 全局环境变量

全局环境变量在shell会话和该shell产生的任何子进程中都可见。本地变量只在创建它们的shell中可见。在产生需要父进程信息的子进程的应用程序中，这便体现了全局变量的重要作用。

Linux 系统会在您开始 bash 会话时设置一些全局环境变量（有关此时启动的变量的详细信息，请参阅本章稍后的“定位系统环境变量”一节）。系统环境变量始终使用全大写字母来区别于普通用户环境变量。

要查看全局环境变量，请使用 printenv 命令：

```
$ printenv
HOSTNAME=testbox.localdomain
TERM=xterm
SHELL=/bin/bash
HISTSIZE=1000
SSH_CLIENT=192.168.1.2 1358 22
OLDPWD=/home/rich/test/test1
SSH_TTY=/dev/pts/0
USER=rich
LS_COLORS=no=00:fi=00:di=00;34:ln=00;36:pi=40;33:so=00;35:
bd=40;33;01:cd=40;33;01:or=01;05;37;41:mi=01;05;37;41:ex=00;32:
*.cmd=00;32:*.exe=00;32:*.com=00;32:*.btm=00;32:*.bat=00;32:
*.sh=00;32:*.csh=00;32:*.tar=00;31:*.tgz=00;31:*.arj=00;31:
*.taz=00;31:*.lzh=00;31:*.zip=00;31:*.z=00;31:*.Z=00;31:
*.gz=00;31:*.bz2=00;31:*.bz=00;31:*.tz=00;31:*.rpm=00;31:
*.cpio=00;31:*.jpg=00;35:*.gif=00;35:*.bmp=00;35:*.xbm=00;35:
*.xpm=00;35:*.png=00;35:*.tif=00;35:
MAIL=/var/spool/mail/rich
PATH=/usr/kerberos/bin:/usr/lib/ccache:/usr/local/bin:/bin:/usr/bin:
/home/rich/bin
INPUTRC=/etc/inputrc
PWD=/home/rich
LANG=en_US.UTF-8
SSH_ASKPASS=/usr/libexec/openssh/gnome-ssh-askpass
SHLVL=1
HOME=/home/rich
LOGNAME=rich
CVS_RSH=ssh
SSH_CONNECTION=192.168.1.2 1358 192.168.1.4 22
LESSOPEN=|/usr/bin/lesspipe.sh %s
G_BROKEN_FILENAMES=1
=/usr/bin/printenv
$
```

如您所见，bash shell 已设置了许多全局环境变量。其中大多数由系统在登录过程中设置。

要显示某个环境变量的值，请使用 echo 命令。当引用某个环境变量时，您必须在环境变量名称前添加一个美元符号：

```
$ echo $HOME
/home/rich
$
```

如前所述，全局环境变量对于在当前 shell 会话中运行的子进程同样可用：

```
$ bash
$ echo $HOME
/home/rich
$
```

在本例中，使用 bash 命令开始一个新 shell 后，我显示了 HOME 环境变量的当前值，该值由系统在我登录主 shell 时设置。可以肯定，该值在子 shell 进程中也可用。

### 5.1.2 本地环境变量

顾名思义，本地环境变量只在定义它们的本地进程中可见。不要搞混本地环境变量，它们与全局变量同样重要。事实上，Linux 系统默认还为您定义了标准的本地环境变量。

查看本地环境变量列表需要使用一点小技巧。遗憾的是，还没有任何命令可以只显示本地环境变

量。set命令显示特定进程的所有环境变量集。但是，这其中也包括了全局环境变量。

下面给出了一个set命令的输出示例：

```
$ set
BASH=/bin/bash
BASH_ARGC=()
BASH_ARGV=()
BASH_LINENO=()
BASH_SOURCE=()
BASH_VERSINFO=([0]="3" [1]="2" [2]="9" [3]="1" [4]="release"
[5]="i686-redhat-linux-gnu")
BASH_VERSION='3.2.9(1)-release'
COLORS=/etc/DIR_COLORS.xterm
COLUMNS=80
CVS_RSH=ssh
DIRSTACK=()
EUID=500
GROUPS=()
G_BROKEN_FILENAMES=1
HISTFILE=/home/rich/.bash_history
HISTFILESIZE=1000
HISTSIZE=1000
HOME=/home/rich
HOSTNAME=testbox.localdomain
HOSTTYPE=i686
IFS=$' \t\n'
INPUTRC=/etc/inputrc
LANG=en_US.UTF-8
LESSOPEN='|/usr/bin/lesspipe.sh %s'
LINES=24
LOGNAME=rich
LS_COLORS='no=00:fi=00:di=00;34:ln=00;36:pi=40;33:so=00;35:bd=40;33;
01:cd=40;33;01:or=01;05;37;41:mi=01;05;37;41:ex=00;32:*.cmd=00;32:
*.exe=00;32:*.com=00;32:*.btm=00;32:*.bat=00;32:*.sh=00;32:
*.csh=00;32:*.tar=00;31:*.tgz=00;31:*.arj=00;31:*.taz=00;31:
*.lzh=00;31:*.zip=00;31:*.z=00;31:*.Z=00;31:*.gz=00;31:*.bz2=00;31:
*.bz=00;31:*.tz=00;31:*.rpm=00;31:*.cpio=00;31:*.jpg=00;35:
*.gif=00;35:*.bmp=00;35:*.xbm=00;35:*.xpm=00;35:*.png=00;35:
*.tif=00;35:'
MACHTYPE=i686-redhat-linux-gnu
MAIL=/var/spool/mail/rich
MAILCHECK=60
OPTERR=1
OPTIND=1
OSTYPE=linux-gnu
PATH=/usr/kerberos/bin:/usr/lib/ccache:/usr/local/bin:/bin:/usr/bin:
/home/rich/bin
PIPESTATUS=([0]="0")
PPID=3702
PROMPT_COMMAND='echo -ne
"\033]0;${USER}@${HOSTNAME%%.*}:${PWD/#$HOME/~}"; echo -ne "\007"'
PS1='[\u@\h \W]\$ '
PS2='> '
PS4='+ '
PWD=/home/rich
SHELL=/bin/bash
SHELLOPTS=braceexpand:emacs:hashall:histexpand:history:
interactive-comments:monitor
SHLVL=2
SSH_ASKPASS=/usr/libexec/openssh/gnome-ssh-askpass
SSH_CLIENT='192.168.1.2 1358 22'
SSH_CONNECTION='192.168.1.2 1358 192.168.1.4 22'
SSH_TTY=/dev/pts/0
```

```
TERM=xterm
UID=500
USER=rich
_=-H
consoletype=pty
$
```

不难发现，通过 printenv 看到的所有全局环境变量都出现在了 set 命令的输出中。但是，其中还包括一些额外的环境变量。它们是本地环境变量。

## 5.2 设置环境变量

您可以在 bash shell 中直接设置自己的环境变量。本节介绍如何创建自己的环境变量，并通过交互式 shell 或 shell 脚本程序来引用它们。

### 5.2.1 设置本地环境变量

启动 bash shell（或产生 shell 脚本）之后，您将能够创建在 shell 进程中可见的本地变量。您可以为环境变量分配一个数值或字符串，方法是使用等号将变量指定为具体值：

```
$ test=testing
$ echo $test
testing
$
```

这非常简单！现在，当您需要引用测试环境变量的值时，只需通过名称$test 来引用它。

如果您要指定包含空格的字符串值，则需要使用单引号来指示字符的起始位置。

```
$ test=testing a long string
-bash: a: command not found
$ test='testing a long string'
$ echo $test
testing a long string
$
```

如果没有单引号，bash shell 将假定下一个字符是另一个要处理的命令。注意，我使用小写字母来定义本地环境变量，而目前为止出现的系统环境变量都使用了大写字母。

这是 bash shell 中的一个标准约定。如果创建新环境变量，则建议（但不要求）使用小写字母。这将有助于区分个人环境变量与系统环境变量。

警告：环境变量名称、等号和值之间没有空格，这一点尤为重要。如果您在其中添加了任何空格，bash shell 会将值解释为一个单独的命令：

```
$ test2 = test
-bash: test2: command not found
$
```

设置本地变量之后，可以在 shell 进程的任何地方使用它。但是，如果产生了另外一个 shell，则不能在子 shell 中使用它：

```
$ bash
$ echo $test

$ exit
exit
$ echo $test
testing a long string
$
```

在本例中，我启动了一个子 shell。如您所见，test 环境变量在子 shell 中不可用（它包含一个空值）。当我退出子 shell 并返回初始 shell 之后，本地环境变量仍然可用。

类似地，如果在子进程中设置了本地环境变量，则一旦离开子进程，本地环境变量将不再可用：

```
$ bash
```

```
$ test=testing
$ echo $test
testing
$ exit
exit
$ echo $test

$
```

当返回父 shell 时，子 shell 中的测试环境变量集已经不再存在。

### 5.2.2 设置全局环境变量

全局环境变量在任何由设置全局环境变量的进程创建的子进程中都可见。创建全局环境变量的方法是创建一个本地环境变量，然后再将它导出到全局环境中。

这将通过使用 export 命令来完成：

```
$ echo $test
testing a long string
$ export test
$ bash
$ echo $test
testing a long string
$
```

导出本地环境变量 test 之后，我开始了一个新 shell 进程并查看了 test 环境变量的值。这次，环境变量的值仍然存在，因为 export 命令已让它具有全局性。

警告：在导出本地环境变量时，您不必使用美元符号来引用变量的名称。

## 5.3 移除环境变量

当然，如果能够创建新的环境变量，那么也应该能够删除已有的环境变量。这可以通过 unset 命令来完成：

```
$ echo $test
testing
$ unset test
$ echo $test

$
```

在 unset 命令中引用环境变量时，请记住不要使用美元符号。

在操作全局环境变量时，需要稍微使用一些技巧。如果您在子进程中使用 unset 移除全局环境变量，则该操作只对子进程有效。全局环境变量在父进程中仍然可用：

```
$ test=testing
$ export test
$ bash
$ echo $test
testing
$ unset test
$ echo $test

$ exit
exit
$ echo $test
testing
$
```

在本例中，我设置了一个名称为 test 的本地环境变量，然后将它导出为全局环境变量。随后，我开始了一个子进程，并检查以确保全局环境变量 test 仍然可用。接下来，仍然在子进程中，我使

用 unset 命令移除全局环境变量 test，然后退出子进程。现在，返回初始父 shell，检查 test 环境变量的值，发现它仍然有效。

## 5.4 默认的 shell 环境变量

bash shell 默认将使用一些特定的环境变量来定义系统环境。您随时都可以依靠这些在 Linux 系统中设置的变量。由于 bash shell 派生自原 Unix Bourn shell，因此它还包括最初在该 shell 中定义的环境变量。

表 5-1 列出了 bash shell 提供的与原 Unix Bourn shell 兼容的环境变量。

表 5-1 bash shell Bourne 变量

| 变量 | 描述 |
|---|---|
| CDPATH | 冒号分隔的目录列表，用作 cd 命令的搜索路径 |
| HOME | 当前用户的主目录 |
| IFS | 用于分隔字段的字符列表，shell 使用它们分隔文本字符串 |
| MAIL | 当前用户邮箱的文件名。对于新邮件，bash shell 将检查该文件 |
| MAILPATH | 当前用户邮箱的多个文件名，由冒号分隔。对于新邮件，bash shell 将检查该列表中的各个文件 |
| OPTARG | getopts 命令处理的最后一个选项参数的值 |
| OPTIND | getopts 命令处理的最后一个选项参数的索引值 |
| PTAH | 冒号分隔的目录列表，shell 将在这些目录中查找命令 |
| PS1 | 主 shell 命令行界面提示字符串 |
| PS2 | 次 shell 命令行界面提示字符串 |

目前，此列表中最重要的环境变量是 PATH 环境变量。当您在 shell 命令行界面（CLI）中输入命令时，shell 必须搜索系统以找到程序。PATH 环境变量定义了它搜索命令的目录。在我的 Linux 系统中，PATH 变量显示如下：

```
$ echo $PATH
/usr/kerberos/bin:/usr/lib/ccache:/usr/local/bin:/bin:/usr/bin:
/home/rich/bin
$
```

可以看到，它为 shell 指定了 6 个在其中查找命令的目录。PATH 中的每个目录都由冒号分隔。PATH 变量的末尾没有任何标记指示目录清单结束。您可以方便地在 PATH 中加入其他目录，只需在其后添加一个冒号和新目录。PATH 还显示了 shell 查找命令的顺序。

除了默认的 Bourne 环境变量之外，bash shell 还提供了一些自己的变量，如表 5-2 所示。

表 5-2 bash shell 环境变量

| 变量 | 描述 |
|---|---|
| BASH | 执行当前 bash shell 实例的完整路径名称 |
| BASH_ENV | 设置后，各 bash 脚本都将在运行之前尝试执行此变量定义的启动文件 |
| BASH_VERSION | 当前 bash shell 实例的版本号 |
| BASH_VERSINFO | 包含当前 bash shell 实例的主要及次要版本号的变量数组 |
| COLUMNS | 包含当前 bash shell 实例所使用的终端的终端宽度 |
| COMP_CWORD | 变量 COMP_CWORD 中的索引，它包含当前光标的位置 |
| COMP_LINE | 当前命令行 |
| COMP_POINT | 当前光标相对于当前命令开始处的位置的索引 |

续表

| 变量 | 描述 |
| --- | --- |
| COMP_WORDS | 包含当前命令行上的各单词的变量数组 |
| COMPREPLY | 包含可能由 shell 函数生成的完成代码的变量数组 |
| DIRSTACK | 包含目录栈当前内容的变量数组 |
| EUID | 当前用户数值形式的有效用户 ID |
| FCEDIT | fc 命令使用的默认编辑器 |
| FIGNORE | 冒号分隔的后缀列表，执行文件名完成（filename completion）时忽略 |
| FUNCNAME | 当前正在执行的 shell 函数的名称 |
| GLOBIGNORE | 冒号分隔的模式列表，用于定义文件扩展忽略的文件名集合 |
| GROUPS | 包含当前用户所在用户组的变量数组 |
| Histchars | 最多 3 个字符，用于控制历史扩展 |
| HISTCMD | 当前命令的历史记录数量 |
| HISTCONTROL | 控制 shell 历史列表中输入的命令 |
| HISTFILE | 保存 shell 历史记录的文件的名称（默认为.bash history） |
| HISTFILESIZE | 历史文件中可保存的最大行数 |
| HISTIGNORE | 冒号分隔的模式列表，用于定义哪些命令将被历史文件忽略 |
| HISTSIZE | 存储在历史文件中的最大命令数量 |
| HOSTFILE | 包含当 shell 需要完成主机名时应该读取的文件的名称 |
| HOSTNAME | 当前主机的名称 |
| HOSTTYPE | 描述运行 bash shell 的机器的字符串 |
| IGNOREEOF | shell 在退出之前必须接受的连续 EOF 字符的数量。如果该值不存在，则默认值为 1 |
| INPUTRC | Readline 初始化文件的名称（默认为.inputrc） |
| LANG | shell 的地区类别 |
| LC_ALL | 覆盖 LANG 变量，定义地区类别 |
| LC_COLLATE | 设置对字符串值进行排序时所使用的排序顺序 |
| LC_CTYPE | 确定在文件名扩展和模式匹配中使用的字符译码 |
| LC_MESSAGES | 确定在解释双引号字符串前出现美元符号时使用的地区设置 |
| LC_NUMERIC | 确定格式化数字时使用的地区设置 |
| LINENO | 当前执行脚本中的行号 |
| LINES | 定义终端上可用的行数 |
| MACHTYPE | 以 *cpu-company-system* 格式定义系统类型的字符串 |
| MAILCHECK | shell 应该检查新邮件的频率（以秒为单位，默认为 60 秒） |
| OLDPWD | shell 中刚使用过的工作目录 |
| OPTERR | 如果设置为 1，则 bash shell 将显示 getopts 命令生成的错误 |
| OSTYPE | 定义运行 shell 的操作系统 |
| PIPESTATUS | 包含前台进程中的进程退出状态值列表的变量数组 |
| POSIXLY_CORRECT | 如果设置了此变量，则 bash 将以 POSIX 模式启动 |

续表

| 变量 | 描述 |
| --- | --- |
| PPID | bash 父进程的进程 ID（PID） |
| PROMPT_COMMAND | 如果设置了该变量，则表示在显示主提示符之前执行的命令 |
| PS3 | select 命令使用的提示符 |
| PS4 | 回显命令行之前显示的提示符（如果使用了 bash –x 参数） |
| PWD | 当前工作目录 |
| RANDOM | 返回 0 到 32767 之间的随机数。为此变量分配的值将作为随机数生成程序的 seed |
| REPLY | read 命令的默认变量 |
| SECONDS | 自 shell 启动之后的秒数。为该变量分配值会将计时器重设为该值 |
| SHELLOPTS | 已启用的 bash shell 选项列表，由冒号分隔 |
| SHLVL | 指示 shell 级别，每次开始一个新 bash shell 时，该变量都增加 1 |
| TIMEFORMAT | 指定 shell 如何显示时间值的格式 |
| TMOUT | select 和 read 命令应该等待输入的时间（以秒为单位）。默认值 0 表示无限等待 |
| UID | 当前用户数值形式的真实用户 id |

您可能会注意到，当使用 set 命令时，并非所有的默认环境变量都会显示出来。其原因是，虽然有很多默认环境变量，但并非所有变量都需要包含值。

## 5.5 设置 PATH 环境变量

似乎 Linux 系统上发生的最大问题是由 PATH 环境变量造成的。它定义 shell 在哪里查找您在命令行中输入的命令。如果它无法找到命令，则会生成一个错误消息：

```
$ myprog
-bash: myprog: command not found
$
```

问题是，应用程序经常将它们的可执行程序放置在 PATH 环境变量以外的目录中。技巧是确保您的 PATH 环境变量包含应用程序所在的所有目录。

您可以在现有 PATH 环境变量中添加新搜索目录，而不用重新构建它。PATH 中列出的各目录由冒号分隔。您只需要引用原始 PATH 值，并在字符串中添加任何新目录。这看上去应如下所示：

```
$ echo $PATH
/usr/kerberos/bin:/usr/lib/ccache:/usr/local/bin:/bin:/usr/bin:/home
/rich/bin
$ PATH=$PATH:/home/rich/test
$ echo $PATH
/usr/kerberos/bin:/usr/lib/ccache:/usr/local/bin:/bin:/usr/bin:/home
/rich/bin:/home/rich/test
$ myprog
The factorial of 5 is 120.
$
```

通过将目录添加到 PATH 环境变量中，您现在可以在虚拟目录结构的任何位置执行您的程序：

```
[rich@testbox ~]$ cd /etc
[rich@testbox etc]$ myprog
The factorial of 5 is 120
[rich@testbox etc]$
```

程序员经常用的一个技巧就是在他们的 PATH 环境变量中包含一个点符号。单个点符号代表当前目录（参阅第 3 章）：

```
[rich@testbox ~]$ PATH=$PATH:.
[rich@testbox ~]$ cd test2
[rich@testbox test2]$ myprog2
The factorial of 6 is 720
[rich@testbox test2]$
```

在下一节中，您将了解如何在系统中对环境变量执行永久更改，以便能始终执行您的程序。

## 5.6 定位系统环境变量

Linxu 系统使用环境变量在程序和脚本中标识自己。这为获取程序的系统信息提供了一种简便的方法。如何设置环境变量需要一定的技巧。

通过登录 Linux 系统启动 bash shell 时，bash 默认将检查一些文件以执行命令。这些文件被称为**启动文件**。bash 处理的启动文件依赖于启动 bash shell 的方法。可采用三种方法来启动 bash shell：

- 在登录时作为默认登录 shell；
- 作为非登录 shell 的交互式 shell；
- 作为非交互式 shell 运行脚本。

以下几节描述 bash shell 在各启动方法中执行的启动文件。

### 5.6.1 登录 shell

当登录到 Linux 系统中时，bash shell 将作为登录 shell 启动。登录 shell 将查找 4 个不同的启动文件来处理其中的命令。bash shell 处理文件的顺序如下：

- /etc/profile；
- $HOME/.bash_profile；
- $HOME/.bash_login；
- $HOME/.profile。

/etc/profile 文件是 bash shell 在系统上的主默认启动文件。系统上的每一个用户在登录时都将执行此启动文件。另外 3 个启动文件特定于各个用户，并且可以根据各用户的需求自定义它们。让我们仔细看看这些文件。

**1. /etc/profile 文件**

/etc/profile 文件是 bash shell 的主默认启动文件。当您登录 Linux 系统时，bash 将执行 /etc/profile 启动文件中的命令。不同 Linux 发行版在此文件中放置的命令也不尽相同。在我的 Linux 系统中，它看上去如下所示：

```
$ cat /etc/profile
# /etc/profile

# System wide environment and startup programs, for login setup
# Functions and aliases go in /etc/bashrc

pathmunge () {
        if ! echo $PATH | /bin/egrep -q "(^|:)$1($|:)" ; then
           if [ "$2" = "after" ] ; then
              PATH=$PATH:$1
           else
              PATH=$1:$PATH
           fi
        fi
}

# ksh workaround
```

```
if [ -z "$EUID" -a -x /usr/bin/id ]; then
        EUID='id -u'
        UID='id -ru'
fi

# Path manipulation
if [ "$EUID" = "0" ]; then
        pathmunge /sbin
        pathmunge /usr/sbin
        pathmunge /usr/local/sbin
fi

# No core files by default
ulimit -S -c 0 > /dev/null 2>&1

if [ -x /usr/bin/id ]; then
        USER="'id -un'"
        LOGNAME=$USER
        MAIL="/var/spool/mail/$USER"
fi

HOSTNAME='/bin/hostname'
HISTSIZE=1000

if [ -z "$INPUTRC" -a ! -f "$HOME/.inputrc" ]; then
    INPUTRC=/etc/inputrc
fi

export PATH USER LOGNAME MAIL HOSTNAME HISTSIZE INPUTRC

for i in /etc/profile.d/*.sh ; do
    if [ -r "$i" ]; then
        . $i
    fi
done

unset i
unset pathmunge
$
```

您在此文件中看到的大多数命令和脚本都将在第 8 章中详细说明。现在需要重点注意的是在此启动文件中设置的环境变量。注意文件底部的 export 一行：

```
export PATH USER LOGNAME MAIL HOSTNAME HISTSIZE INPUTRC
```

这将确保这些环境变量对于由登录 shell 生成的子进程都可用。

profile 文件还使用了另外一个技巧性的特性。它就是能迭代/etc/profile.d 目录中的任何文件的 for 语句。（for 语句将在第 10 章中讨论。）这使 Linux 系统提供了一个放置特定于应用程序的启动文件（这些文件将在您登录时由 shell 执行）的地方。在我的 Linux 系统中，profile.d 目录中包含以下文件：

```
$ ls -l /etc/profile.d
total 168
-rw-r--r-- 1 root root   88 2007-03-15 10:08 ccache.csh
-rw-r--r-- 1 root root   87 2007-03-15 10:08 ccache.sh
-rw-r--r-- 1 root root  764 2007-02-26 08:04 colorls.csh
-rw-r--r-- 1 root root  713 2007-02-26 08:04 colorls.sh
-rw-r--r-- 1 root root   80 2007-07-11 09:00 cvs.csh
-rw-r--r-- 1 root root   78 2007-07-11 09:00 cvs.sh
-rw-r--r-- 1 root root  192 2004-09-09 01:17 glib2.csh
-rw-r--r-- 1 root root  192 2005-12-12 00:58 glib2.sh
-rw-r--r-- 1 root root   58 2007-03-20 05:17 gnome-ssh-askpass.csh
-rw-r--r-- 1 root root   70 2007-03-20 05:17 gnome-ssh-askpass.sh
```

```
-rw-r--r-- 1 root root  218 2004-09-09 03:12 krb5-devel.csh
-rw-r--r-- 1 root root  229 2006-01-19 13:05 krb5-devel.sh
-rw-r--r-- 1 root root  218 2004-09-09 03:12 krb5-workstation.csh
-rw-r--r-- 1 root root  229 2006-01-19 13:05 krb5-workstation.sh
-rwxr-xr-x 1 root root 3006 2007-06-25 12:57 lang.csh
-rwxr-xr-x 1 root root 3329 2007-06-25 12:57 lang.sh
-rw-r--r-- 1 root root  122 2007-02-07 07:55 less.csh
-rw-r--r-- 1 root root  108 2007-02-07 07:55 less.sh
-rw-r--r-- 1 root root   74 2007-06-27 05:11 vim.csh
-rw-r--r-- 1 root root  248 2007-06-27 05:11 vim.sh
-rwxr-xr-x 1 root root  170 2007-01-22 05:48 which-2.sh
$
```

您会注意到这些文件与系统中的特定应用程序最为相关。大多数应用程序都创建了两个启动文件，一个用于bash shell（使用.sh扩展），另一个用于csh（使用.csh扩展）。稍后，我们将在第21章中讨论这两种shell之间的不同。

lang.csh和lang.sh文件将尝试确定系统上使用的默认语言字符集，并适当设置LANG环境变量。

**2. $HOME启动文件**

其余3个启动文件具备相同的功能——提供特定于用户的启动文件，用于定义特定于用户的环境变量。大多数Linux发行版都只使用这3者之一：

- $HOME/.bash_profile；
- $HOME/.bash_login；
- $HOME/.profile。

注意，所有3个文件都以点号开头，这表示它们是隐藏文件（它们不会显示在普通的ls命令清单中）。由于它们位于用户的HOME目录中，因此各用户可以编辑这些文件并添加自己的、对于他们启动的每一个bash shell会话都为活动状态的环境变量。

我的Linux系统包含以下.bash_profile文件：

```
$ cat .bash_profile
# .bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
        . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/bin

export PATH
$
```

.bash profile启动文件首先将检查HOME目录中是否存在另一个名称为.bashrc的启动文件（我们将在下面的“交互式shell”一节中详细讨论这方面内容）。如果存在，则启动文件将执行其中的命令。接下来，启动文件将添加一个目录到PATH环境变量中，目的是提供在HOME目录中放置可执行文件的公共位置。

## 5.6.2　交互式shell

如果启动了一个bash shell而没有登录系统（比如说，只在CLI提示符中键入bash），则您启动的就是一个**交互式shell**。交互式shell与登录shell的行为不同，但它仍然提供了一个CLI提示符供您输入命令。

如果 bash 作为交互式 shell 启动，则它不会处理/etc/profile 文件。相反，它将检查用户 HOME 目录中的.bashrc 文件。

在我的 Linux 发行版中，该文件看上去如下所示：

```
[rich@testbox ~]$ cat .bashrc
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi

# User specific aliases and functions
[rich@testbox ~]$
```

.bashrc 文件执行两个任务。首先，它检查/etc 目录中的公共 bashrc 文件。其次，它为用户输入个人别名（稍后将在“使用命令别名”一节中讨论）和私有脚本函数（在第 14 章中介绍）提供了地方。

公共/etc/bashrc 文件可由系统上启动了交互式 shell 会话的任何人运行。在我的 Linux 发行版中，它看上去如下所示：

```
$ cat /etc/bashrc
# /etc/bashrc

# System wide functions and aliases
# Environment stuff goes in /etc/profile

# By default, we want this to get set.
# Even for non-interactive, non-login shells.
if [ $UID -gt 99 ] && [ "'id -gn'" = "'id -un'" ]; then
        umask 002
else
        umask 022
fi

# are we an interactive shell?
if [ "$PS1" ]; then
    case $TERM in
        xterm*)
                if [ -e /etc/sysconfig/bash-prompt-xterm ]; then
                        PROMPT_COMMAND=/etc/sysconfig/bash-promptxterm
                else
                PROMPT_COMMAND='echo -ne
"\033]0;${USER}@${HOSTNAME%%.*}:${PWD/#$HOME/~}"; echo -ne "\007"'
                fi
                ;;
        screen)
                if [ -e /etc/sysconfig/bash-prompt-screen ]; then
                        PROMPT_COMMAND=/etc/sysconfig/bash-promptscreen
                else
                PROMPT_COMMAND='echo -ne
"\033 ${USER}@${HOSTNAME%%.*}:${PWD/#$HOME/~}"; echo -ne "\033\\"'
                fi
                ;;
        *)
                [ -e /etc/sysconfig/bash-prompt-default ] &&
PROMPT_COMMAND=/etc/sysconfig/bash-prompt-default
            ;;
    esac
    # Turn on checkwinsize
    shopt -s checkwinsize
    [ "$PS1" = "\\s-\\v\\$ " ] && PS1="[\u@\h \W]\\$ "
```

```
fi

if ! shopt -q login_shell ; then # We're not a login shell
        # Need to redefine pathmunge, it get's undefined at the end
of /etc/profile
    pathmunge () {
                if ! echo $PATH | /bin/egrep -q "(^|:)$1($|:)" ; then
                        if [ "$2" = "after" ] ; then
                                PATH=$PATH:$1
                        else
                                PATH=$1:$PATH
                        fi
                fi
        }

        for i in /etc/profile.d/*.sh; do
                if [ -r "$i" ]; then
                        . $i
        fi
        done
        unset i
        unset pathmunge
fi
# vim:ts=4:sw=4
$
```

默认文件将设置一些环境变量。但是需要注意，它没有使用 export 命令将它们设置为全局性质。请记住，每次启动新交互式 shell 时都会运行交互式 shell 启动文件；因此，任何子 shell 都将自动执行交互式 shell 启动文件。

您还将注意到，/etc/bashrc 文件也执行了位于/etc/profile.d 目录中的、特定于应用程序的启动文件。

### 5.6.3 非交互式 shell

shell 的最后一种类型是非交互式 shell。这是系统开始执行 shell 脚本的 shell。它的不同之处在于不用担心 CLI 提示符。但是，您仍然希望在每次启动系统中的脚本时运行特定的启动命令。

为适应这种情况，bash shell 提供了 BASH_ENV 环境变量。当 shell 开始一个非交互式 shell 进程时，它将检查该环境变量表示的待执行启动文件的名称。如果该变量有值，则 shell 将执行该文件中的命令。在我的 Linux 发行版中，此环境变量并非默认设置。

## 5.7 变量数组

环境变量的一个非常好的特性就是能够当作**数组**使用。数组是能保存多个值的变量。数组中的值既可以分别引用，也可以作为整体引用。

要为某个环境变量设置多个值，只需将它们列出在圆括号中，各值以空格分隔：

```
$ mytest=(one two three four five)
$
```

这并没有太多令人兴奋的地方。如果您尝试将数组作为普通环境变量显示，那么结果将令您十分失望：

```
$ echo $mytest
one
$
```

只有数组的第一个值将被显示出来。要引用某个单独的数组元素，您必须使用一个数值索引值，

它表示该元素在数组中的位置。索引值包含在方括号中：

```
$ echo ${mytest[2]}
three
$
```

警告：环境变量数组的索引值以 0 开始。这经常会造成混淆。

要显示整个数组变量，可以使用星号通配符作为索引值：

```
$ echo ${mytest[*]}
one two three four five
$
```

您还可以更改单个索引位置的值：

```
$ mytest[2]=seven
$ echo ${mytest[*]}
one two seven four five
$
```

您甚至可以使用 unset 命令移除数组中的某个值，但是需要格外小心，因为这需要一定的技巧。查看以下示例：

```
$ unset mytest[2]
$ echo ${mytest[*]}
one two four five
$
$ echo ${mytest[2]}

$ echo ${mytest[3]}
four
$
```

该示例使用 unset 命令移除索引 2 位置上的值。当您显示该数组时，会发现其他索引值都下移了一位。但是，如果明确指定显示索引 2 位置上的数据，则会看到该位置为空。

最后，可以通过在 unset 命令中使用数组名称来移除整个数组：

```
$ unset mytest
$ echo ${mytest[*]}

$
```

有时，变量数组只令事情变得更加复杂，因此它们在 shell 脚本编程中并不常用。它们并不适合移植到其他 shell 环境，这对于为不同 shell 执行大量 shell 编程会带来不利的影响。虽然实际中不乏使用数组的 bash 系统环境变量（如 BASH_VERSINFO），但是总的来说，您不可能经常使用这些环境变量。

## 5.8 使用命令别名

虽然 shell 命令别名本质上不是环境变量，但它的行为却与之基本无异。**命令别名**允许您为公共命令（以及它们的参数）创建别名，以尽可能减少录入工作。

您的 Linux 发行版极有可能已经设置了一些公共命令别名。要查看活动别名的列表，请使用带-p 参数的 alias 命令：

```
$ alias -p
alias l.='ls -d .* --color=tty'
alias ll='ls -l --color=tty'
alias ls='ls --color=tty'
alias vi='vim'
alias which='alias | /usr/bin/which --tty-only -read-alias--show-dot --show-tilde'
$
```

注意，在我的 Linux 发行版中，它们使用一个别名覆盖了标准的 ls 命令。它自动提供了一个 --color 参数，指示终端支持颜色模式清单。

您可以使用 alias 命令创建自己的别名：

```
[rich@testbox ~]$ alias li='ls -il'
[rich@testbox ~]$ li
total 989292
 360621 drwxrwxr-x 2 rich rich       4096 2007-08-24 22:04 4rich
301871 drwxr-xr-x 4 rich rich        4096 2007-09-18 08:38 Desktop
 301875 drwxr-xr-x 2 rich rich       4096 2001-11-01 01:10 Documents
 301872 drwxr-xr-x 2 rich rich       4096 2001-11-01 04:06 Download
 360207 drwxrwxr-x 2 rich rich       4096 2007-07-26 18:25 Drivers
327362 drwxrwxr-x 2 rich rich        4096 2007-09-18 08:38 mnt
 301876 drwxr-xr-x 2 rich rich       4096 2001-11-01 04:06 Music
 301942 -rw-rw-r-- 1 rich rich          0 2007-09-03 16:38 myprob
 301963 -rw-rw-r-- 1 rich rich          0 2007-09-03 16:40 myproblem
 301974 -rwxr--r-- 1 rich rich         30 2007-08-23 21:42 myprog
 301877 drwxr-xr-x 2 rich rich       4096 2001-11-01 04:06 Pictures
 301874 drwxr-xr-x 2 rich rich       4096 2001-11-01 04:06 Public
 360262 drwxrwxr-x 5 rich rich       4096 2007-08-24 22:04 store
```

定义了别名值之后，可以随时在 shell（包括 shell 脚本）中使用它。

命令别名与本地环境变量的行为相似。它们只对于定义范围内的 shell 进程有效：

```
$ alias li='ls -il'
$ bash
$ li
bash: li: command not found
$
```

当然，现在您已经了解了解决此问题的方法。在启动新的交互式 shell 时，bash shell 始终会读取 $HOME/.bashrc 启动文件。这是放置命令别名语句的绝佳位置（正如我们在.bashrc 文件注释中指出的那样）。

## 5.9 小结

本章深入搜索了 Linux 环境变量的世界。全局环境变量可以在由它们定义范围内的进程产生的任何子进程中访问。本地环境变量只能在它们定义范围内的进程中访问。

Linux 系统同时使用全局环境变量和本地环境变量存储关于系统环境的信息。您可以在 shell 命令行界面和 shell 脚本中访问此信息。bash shell 使用原始 Unix Bourne shell 中定义的系统环境变量，以及大量新环境变量。PATH 环境变量定义 bash shell 用于查找可执行命令的搜索模式。您可以修改 PATH 环境变量，在其中添加自己的目录，或者甚至当前的目录符号，使程序运行更加简单。

还可以创建自己的全局和本地环境变量，供自己使用。创建某个环境变量之后，可以在 shell 会话的整个有效期时访问它。

bash shell 在启动时会执行一些启动文件。这些启动文件可以包含环境变量定义，为各 bash 会话设置标准环境变量。登录 Linux 系统之后，bash shell 将访问/etc/profile 启动文件，以及针对各个用户的 3 个本地启动文件：$HOME/.bash_profile、$HOME/.bash_login 和 $HOME/.profile。用户可以定制这些文件，在其中包含环境变量和启动脚本，以满足他们自己的使用需要。

bash shell 还提供了环境变量数组。这些环境变量可以在一个变量中包含多个值。可以通过引用索

引值来分别访问各个值，也可以通过引用整个环境变量数组名称来访问整个数组。

最后，本章讨论了命令别名的使用。虽然它们不是环境变量，但命令别名的行为类似于环境变量。它们允许为某个命令和它的参数定义别名。您不再需要键入冗长的命令和参数，可以将它们指定为简单的别名，然后就可以随时在 shell 会话中使用它们。

下一章将探究 Linux 文件权限的世界。对于 Linux 初级用户，这可能是最难的一个话题。但是，为了编写优秀的 shell 脚本，需要理解文件权限的工作原理，并且能够在自己的 Linux 系统中使用它们。

CHAPTER 6

# 第 6 章
# 理解 Linux 文件权限

本章内容：

- 理解 Linux 安全性；
- 解码文件权限；
- 操作 Linux 用户组。

任何完整的系统都应该具有某种形式的安全性。必须通过某种机制来保护文件不被未授权的用户查看或修改。Linux 系统遵循 Unix 的文件权限方法，它允许各用户和用户组根据一组针对各文件和目录的安全设置来访问文件。本章讨论如何使用 Linux 文件安全系统来根据需要保护数据和共享数据。

## 6.1 Linux 安全性

Linux 安全系统的核心是**用户账户**。访问 Linux 系统的每个人都应该拥有已分配的、唯一的用户账户。用户拥有的对系统中对象的权限依赖他们登录的用户账户。

可以使用**用户 ID**（通常称为 UID）来跟踪用户权限，账户将在创建时被分配 UID。UID 是针对每个用户的唯一数值。但是，您不能使用您的 UID 登录 Linux 系统。相反，您需要使用**登录名**。登录名是由不多于 8 个字符（可以是数字或字母）组成的文本字符串，用户使用它（以及相关密码）登录系统。

Linux 系统使用特定的文件和实用工具来跟踪和管理系统上的用户账户。在讨论文件权限之前，我们需要介绍 Linux 处理用户账户的方式。本节描述用户账户需要的文件和实用工具，以便于您理解如何在操作文件权限时使用它们。

### 6.1.1 /etc/passwd 文件

Linux 系统使用一个特殊文件将登录名与相应的 UID 值匹配。这就是/etc/passwd 文件。/etc/passwd 文件包含几条关于用户的信息。在我的 Linux 系统中，/etc/passwd 文件看上去如下所示：

```
$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
```

```
news:x:9:13:news:/etc/news:
uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
gopher:x:13:30:gopher:/var/gopher:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:99:99:Nobody:/:/sbin/nologin
rpm:x:37:37::/var/lib/rpm:/sbin/nologin
vcsa:x:69:69:virtual console memory owner:/dev:/sbin/nologin
mailnull:x:47:47::/var/spool/mqueue:/sbin/nologin
smmsp:x:51:51::/var/spool/mqueue:/sbin/nologin
apache:x:48:48:Apache:/var/www:/sbin/nologin
rpc:x:32:32:Rpcbind Daemon:/var/lib/rpcbind:/sbin/nologin
ntp:x:38:38::/etc/ntp:/sbin/nologin
nscd:x:28:28:NSCD Daemon:/:/sbin/nologin
tcpdump:x:72:72::/:/sbin/nologin
dbus:x:81:81:System message bus:/:/sbin/nologin
avahi:x:70:70:Avahi daemon:/:/sbin/nologin
hsqldb:x:96:96::/var/lib/hsqldb:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
rpcuser:x:29:29:RPC Service User:/var/lib/nfs:/sbin/nologin
nfsnobody:x:65534:65534:Anonymous NFS User:/var/lib/nfs:/sbin/nologin
haldaemon:x:68:68:HAL daemon:/:/sbin/nologin
xfs:x:43:43:X Font Server:/etc/X11/fs:/sbin/nologin
gdm:x:42:42::/var/gdm:/sbin/nologin
rich:x:500:500:Rich Blum:/home/rich:/bin/bash
mama:x:501:501:Mama:/home/mama:/bin/bash
katie:x:502:502:katie:/home/katie:/bin/bash
jessica:x:503:503:Jessica:/home/jessica:/bin/bash
mysql:x:27:27:MySQL Server:/var/lib/mysql:/bin/bash
$
```

根用户账户是 Linux 系统的管理员，它的 UID 始终为 0。Linux 系统为各种非实际用户的函数创建了大量用户账户，称为**系统账户**。系统账户是一个特殊的账户，在系统中运行的服务使用它们访问系统资源。在后台模式中运行的所有服务都需要作为系统用户账户登录 Linux 系统。

在安全性成为重大问题之前，这些服务通常只使用根用户账户登录。遗憾的是，如果某个未授权个人闯入其中一个服务，则他可以立即获取根用户那样对系统的访问权限。为防止这种情况，在后台中运行的每一个服务都有自己的用户账户用于登录。这样，如果不怀好意者欺骗了某个服务，那么他仍然不能获得对整个系统的访问权限。

Linux 将 500 以下的 UID 保留用作系统账户。一些服务甚至需要特定的 UID 才能正常工作。当您为普通用户创建账户时，大多数 Linux 系统都会从 500 开始分配第一个可用的 UID（不过并非所有的 Linux 发行版都是这样）。

您可能已经注意到/etc/passwd 文件中包含的信息远远不止用户的登录名和 UID。/etc/passwd 文件中的字段包含以下信息：

- 登录用户名；
- 用户的密码；
- 用户账户的数值 UID；
- 用户账户的数值用户组 ID（GID）；
- 用户账户的文本描述（称为注释字段）；
- 用户 HOME 目录的位置；
- 用户的默认 shell。

/etc/passwd 文件中的密码字段被设置为一个 x。这并不意味着所有用户账户都拥有相同的密码。在早期的 Linux 系统中，/etc/passwd 文件包含一个加密版本的用户密码。但是，由于大多数

程序都需要访问/etc/passwd 文件来获取用户信息，因此这也成了一个安全问题。随着能够方便地解密经过加密的密码的软件的出现，恶意攻击者尝试破解存储在/etc/passwd 文件中的密码已不再那么困难。Linux 开发人员需要重新思考该策略。

现在，大多数 Linux 系统都将用户密码保存在一个单独的文件中（称为**影子**文件，位于/etc/shadow）。只有一些特殊程序（如登录程序）才允许访问该文件。

/etc/passwd 文件是一个标准的文本文件。可以使用任何文本编辑器直接在/etc/passwd 文件中手动执行用户管理函数（如添加、修改或删除用户账户）。但是，这是一种非常危险的操作。如果/etc/passwd 文件被破坏，则系统将不能读取它，并且它将阻止任何人（甚至是根用户）登录系统。相反，使用标准的 Linux 用户管理实用工具来执行所有的用户管理功能则相对比较安全。

### 6.1.2 /etc/shadow 文件

/etc/shadow 文件为 Linux 系统管理密码的方式提供了更多的控制。只有根用户才能访问/etc/shadow 文件，这比/etc/passwd 文件更加安全。

对于系统上的每个用户账户，/etc/shadow 文件都包含一个相应的记录。记录如下所示：

```
rich:$1$.FfcK0ns$f1UgiyHQ25wrB/hykCn020:11627:0:99999:7:::
```

每个/etc/shadow 文件记录中都有 9 个字段：

- 与/etc/passwd 文件中的登录名相对应的登录名；
- 经过加密的密码；
- 密码上次修改时距 1970 年 1 月 1 日的天数；
- 距能够修改密码时的最少天数；
- 距必须修改密码时的天数；
- 距密码逾期并警告用户修改密码的天数；
- 从密码逾期开始到账户被禁用的天数；
- 用户账户已被禁用的天数（存储为自 1970 年 1 月 1 日开始的天数）；
- 保留供未来使用的字段。

使用影子密码系统，Linux 系统将对用户密码拥有更加精细的控制。它可以控制用户必须多久修改一次密码，以及当密码未修改时何时禁用账户。

### 6.1.3 添加新用户

useradd 是用于将新用户添加到 Linux 中的主要工具。此命令为创建新用户账户并同时建立用户的 HOME 目录结构提供了一种简便的方法。useradd 命令使用系统默认值结合命令行参数来定义用户账户。要查看 Linux 发行版所使用的系统默认值，请输入带-D 参数的 useradd 命令：

```
# /usr/sbin/useradd -D
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
CREATE_MAIL_SPOOL=yes
#
```

注意：一些 Linux 发行版将 Linux 用户和用户组实用工具放置在/usr/sbin 目录中，该目录可能不在您的 PATH 环境变量中。如果您的 Linux 发行版属于这种情况，请将该目录添加到 PATH 变量中，或使用绝对文件路径来运行它。

如果在创建新用户账户时未在命令行中指定各属性，则-D 参数将显示 useradd 命令所使用的默认值。本例显示了以下默认值：

- 新用户将被添加到用户组 ID 为 100 的公共用户组中。
- 新用户将在/home/loginname 目录下创建一个 HOME 账户。
- 当密码逾期时，账户将不会被禁用。
- 新账户不会在设定日期被设置为逾期。
- 新账户将使用 bash shell 作为默认 shell。
- 系统会将/etc/skel 目录中的内容复制到用户的 HOME 目录。
- 系统将在邮件目录中创建一个文件用于用户账户接收邮件。

倒数第二个值非常有趣。useradd 命令允许管理员创建一个默认的 HOME 目录配置，然后使用它作为模板来创建新用户的 HOME 目录。这允许您将系统的默认文件自动放置在每一个新用户的 HOME 目录中。在我的 Linux 系统中，/etc/skel 目录包含以下文件：

```
# ls -al /etc/skel
total 48
drwxr-xr-x   2 root root  4096 2001-11-01 00:23 .
drwxr-xr-x 107 root root 12288 2007-09-20 16:53 ..
-rw-r--r--   1 root root    33 2007-02-12 10:18 .bash_logout
-rw-r--r--   1 root root   176 2007-02-12 10:18 .bash_profile
-rw-r--r--   1 root root   124 2007-02-12 10:18 .bashrc
#
```

在第 5 章中已经认识了这些文件。它们是 bash shell 环境的标准启动文件。系统会自动将这些默认文件复制到您为每个用户创建的 HOME 目录中。

要测试此特性，可以使用默认的系统参数创建一个新用户，然后再查看新用户的 HOME 目录：

```
# /usr/sbin/useradd test
# ls -al /home/test
total 40
drwx------ 2 test test 4096 2007-09-20 18:23 .
drwxr-xr-x 7 root root 4096 2007-09-20 18:23 ..
-rw-r--r-- 1 test test   33 2007-09-20 18:23 .bash_logout
-rw-r--r-- 1 test test  176 2007-09-20 18:23 .bash_profile
-rw-r--r-- 1 test test  124 2007-09-20 18:23 .bashrc
#
```

不出所料，useradd 命令使用/etc/skel 目录中的文件创建了一个新 HOME 目录。

如果您希望在创建新用户时覆盖默认值，那么可以使用命令行参数来执行此操作。这些命令行参数如表 6-1 所示。

表 6-1 useradd 命令行参数

| 参数 | 描述 |
|---|---|
| -c *comment* | 添加文本到新用户的注释字段 |
| -d *home_dir* | 为主目录指定不同于登录名的名称 |
| -e *expire_date* | 指定账户将要逾期的日期，采用 YYYY-MM-DD 格式 |
| -f *inactive_days* | 指定密码逾期后，账户被禁用需要的天数。值 0 表示密码逾期时立即禁用账户；值-1 表示禁用此特性 |
| -g *initial_group* | 指定用户登录组的用户组名称或 GID |
| -G *group...* | 指定用户属于的一个或多个补充用户组 |
| -k | 将/etc/skel 目录内容复制到用户的 HOME 目录（还必须使用-m） |
| -m | 创建用户的 HOME 目录 |
| -M | 不创建用户的 HOME 目录（当默认设置为创建时使用） |
| -n | 使用与用户登录名相同的名称创建一个新用户组 |
| -r | 创建一个系统账户 |
| -p *passwd* | 为用户账户指定一个默认密码 |

续表

| 参数 | 描述 |
| --- | --- |
| -s *shell* | 指定默认登录 shell |
| -u *uid* | 为账户指定唯一的 UID |

如您所见，通过使用命令行参数，在创建新用户账户时，可以覆盖所有的系统默认值。然而，如果总是需要覆盖一个值，更改系统默认值更容易。

通过使用-D参数和表示需要修改的值的参数，可以修改系统默认的新用户值。这些参数如表6-2所示。

表6-2 useradd更改默认值参数

| 参数 | 描述 |
| --- | --- |
| -b *default_home* | 修改用户的 HOME 目录被创建的位置 |
| -e *expiration_date* | 修改新账户的逾期日期 |
| -f *inactive* | 修改从密码逾期至账户被禁用的天数 |
| -g *group* | 修改所使用的默认用户组名称或 GID |
| -s *shell* | 修改默认登录 shell |

修改这些默认值非常简单：

```
# useradd -D -s /bin/tsch
# useradd -D
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/tsch
SKEL=/etc/skel
CREATE_MAIL_SPOOL=yes
#
```

现在，useradd命令将使用tsch shell作为您创建的新用户账户的默认登录shell。

### 6.1.4 删除用户

如果您希望从系统中删除用户，则需要使用 userdel 命令。默认情况下，userdel 只删除/etc/passwd文件中的用户信息。它不会删除账户在系统中拥有的任何文件。

如果您使用-r参数，则userdel将删除用户的HOME目录和用户的邮件目录。但是，在系统中删除的用户账户可能仍然拥有其他文件。这在某些环境中可能是一个问题。

下面给出了一个使用userdel命令删除已有用户账户的例子：

```
# /usr/sbin/userdel -r test
# ls -al /home/test
ls: cannot access /home/test: No such file or directory
#
```

使用-r参数之后，用户的原/home/test目录将不复存在。

警告：在拥有大量用户的环境中使用-r参数时需要格外小心。您不可能知道某个用户是否在他/她的 HOME 目录中存储了其他人或其他程序需要使用的重要文件。在删除用户的 HOME 目录之前，务必进行检查！

### 6.1.5 修改用户

Linux为修改已有用户账户的信息提供了一些不同的实用工具。表6-3列出了这些实用工具。

每个实用工具都提供了一个特定的函数，用于修改关于用户账户的信息。以下几节将分别介绍这些实用工具。

表 6-3 用户账户修改实用工具

| 命令 | 描述 |
| --- | --- |
| usermod | 编辑用户账户字段，以及指定主要和次要用户组成员 |
| passwd | 修改已有用户的密码 |
| chpasswd | 读取登录名和密码对文件，并更新密码 |
| chage | 修改密码的逾期日期 |
| chfn | 修改用户账户的注释信息 |
| chsh | 修改用户账户的默认 shell |

### 1. usermod

usermod 是最强大的用户账户修改实用工具。它提供了各种选项用于修改/etc/passwd 文件中的大多数字段。为此，您只需要使用与要修改的值对应的命令行参数。这些参数大多与 useradd 参数相同（例如，-c 用于修改注释字段；-e 用于修改逾期日期；-g 用于修改默认登录用户组）。但是，还有其他一些参数也非常简便易用：

- -l 用于修改用户账户的登录名；
- -L 用于锁定账户，使用户无法登录；
- -p 用于修改账户的密码；
- -U 用于解除账户锁定，使用户能够登录。

-L 参数尤为实用。使用它锁定账户之后，用户将无法登录，而不需要删除账户和用户的数据。要使账户返回正常，只需使用-U 参数。

### 2. passwd 和 chpasswd

passwd 命令可用于快速修改用户的密码：

```
# passwd test
Changing password for user test.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
#
```

如果您只使用 passwd 命令自身，那么它将修改您自己的密码。系统中的任何用户都可以修改他们自己的密码，但是只有根用户可以修改其他人的密码。

-e 选项用于强制用户在下次登录时修改密码，它非常方便。这允许您将用户的密码设置为一个简单的值，然后再强制他们将密码修改为可以记住的较为复杂的值。

如果需要对系统上的大量用户执行批量密码修改，那么可以使用 chpasswd 命令。chpasswd 命令从标准输入读取登录名和密码对列表（由冒号分隔），并自动对密码进行加密，然后为用户账户设置它们。

### 3. chsh、chfn 和 chage

chsh、chfn 和 chage 实用工具专用于特定的功能。chsh 命令允许快速修改用户的默认登录 shell。必须使用 shell 的完全路径名，而不能仅是 shell 名称：

```
# chsh -s /bin/csh test
Changing shell for test.
Shell changed.
#
```

chfn 命令提供了一个标准方法，用于将信息存储到/etc/passwd 文件的注释字段中。与只插入随机文本，如名称、昵称，或者甚至保留注释字段为空的传统方式不同，chfn 命令使用在 Unix finger 命令中使用的具体信息来存储注释字段中的信息。finger 命令允许您方便地查找关于 Linux 系统上的用户信息。

```
# finger rich
```

```
Login: rich                    Name: Rich Blum
Directory: /home/rich          shell: /bin/bash
On since Thu Sep 20 18:03 (EDT) on pts/0 from 192.168.1.2
No mail.
No Plan.
#
```

注意：基于安全性考虑，许多Linux系统管理员都在他们的系统上禁用了finger命令。

如果使用不带参数的chfn命令，那么它将询问适合输入到注释字段中的值：

```
# chfn test
Changing finger information for test.
Name []: Ima Test
Office []: Director of Technology
Office Phone []: (123)555-1234
Home Phone []: (123)555-9876

Finger information changed.
# finger test
Login: test                    Name: Ima Test
Directory: /home/test          shell: /bin/csh
Office: Director of Technology Office Phone: (123)555-1234
Home Phone: (123)555-9876
Never logged in.
No mail.
No Plan.
#
```

如果现在查看/etc/passwd文件中的条目，则它应如下所示：

```
# grep test /etc/passwd
test:x:504:504:Ima Test,Director of Technology,(123)555-
1234,(123)555-9876:/home/test:/bin/csh
#
```

所有的finger信息都整齐地保存在/etc/passwd文件条目中。

最后，chage命令将帮助我们管理用户账户的密码变化过程。许多参数可用于设置各个值，如表6-4所示。

表6-4 chage命令参数

| 参数 | 描述 |
|---|---|
| -d | 设置距密码上次被修改时的天数 |
| -E | 设置密码将逾期的日期 |
| -I | 设置账户因密码逾期而被锁定后处于不活动状态的天数 |
| -m | 设置两次密码修改之间的最小间隔天数 |
| -W | 设置警告消息在密码逾期前多少天出现 |

可以采用以下两种方法来表示chage日期值：

- YYYY-MM-DD格式的日期；
- 以数值表示的距1970年1月1日的天数。

chage命令的一个有用的特性是允许您为账户设置逾期日期。使用该特性，您可以创建一个临时的用户账户，它会自动在设定日期失效，而不需要您亲自去删除它们！逾期账户类似于被锁定的账户。账户仍然存在，但是用户无法使用它登录。

## 6.2 使用Linux用户组

用户账户对于控制各用户的安全性极为有效，但是它们在允许用户共享资源方面则稍有欠缺。为

实现此目的，Linux 系统使用了另一个安全概念，即**用户组**。

用户组权限允许多个用户共享系统上某个对象的公共权限集，例如文件、目录或设备（有关此方面的更多信息，请参阅“解码文件权限”一节）。

各 Linux 发行版在处理默认用户组成员的方式方面稍有不同。有些 Linux 发行版只创建一个用户组，其中包含所有用户账户，即它的成员。如果您的 Linux 发行版采取这种方式，则需要格外小心，因为系统上的所有其他用户可能都能读取您的文件。其他发行版为每个用户都创建一个单独的用户账户，以提供更多的安全性。

每个用户组都有一个唯一的 GID。与 UID 类似，GID 是系统上独一无二的数值。除了 GID 之外，每个用户组还有一个唯一的用户组名称。可以使用一些用户组实用工具来创建和管理自己在 Linux 系统上的用户组。本节讨论如何存储用户组信息，以及如何使用用户组工具来创建新用户组和修改已有用户组。

## 6.2.1 /etc/group 文件

与用户账户相同，用户组信息存储在系统上的某个文件中。/etc/group 文件包含关于系统所使用的各用户组的信息。在我的 Linux 系统上，/etc/group 文件中的一些例子如下所示：

```
root:x:0:root
bin:x:1:root,bin,daemon
daemon:x:2:root,bin,daemon
sys:x:3:root,bin,adm
adm:x:4:root,adm,daemon
rich:x:500:
mama:x:501:
katie:x:502:
jessica:x:503:
mysql:x:27:
test:x:504:
```

与 UID 类似，GID 的分配需要使用特殊格式。用于系统账户的用户组将分配 500 以下的 GID，而用户的用户组将从 500 开始分配 GID。/etc/group 文件使用 4 个字段：

- 用户组名称；
- 用户组密码；
- GID；
- 属于用户组的用户账户列表。

通过使用密码，用户组密码将允许非用户组成员临时成为用户组的成员。此特性并不常用，但它确实存在。

您不应该采取编辑/etc/group 文件的方式来将用户添加到用户组中。相反，您应该使用 usermod 命令（参阅前面的“Linux 安全性”一节）将用户账户添加到用户组中。在能够将用户添加到不同用户组中之前，您必须创建用户组。

注意：用户账户的列表可能会造成误导。您会注意到，列表中的一些用户组并没有拥有任何用户。这不是因为它们没有任何成员。当用户使用某个用户组作为/etc/passwd 文件里的默认用户组，用户账户不会作为成员出现在/etc/group 文件中。多年以来，已经不止一个系统管理员对此产生了混淆！

## 6.2.2 创建新用户组

groupadd 命令允许您在系统中创建新用户组：

```
# /usr/sbin/groupadd shared
# tail /etc/group
haldaemon:x:68:
xfs:x:43:
gdm:x:42:
```

```
rich:x:500:
mama:x:501:
katie:x:502:
jessica:x:503:
mysql:x:27:
test:x:504:
shared:x:505:
#
```

当您创建新用户组时，系统默认不会为它分配任何用户。groupadd 命令并未提供将用户账户添加到用户组的选项。要添加新用户，需使用 usermod 命令：

```
# /usr/sbin/usermod -G shared rich
# /usr/sbin/usermod -G shared test
# tail /etc/group
haldaemon:x:68:
xfs:x:43:
gdm:x:42:
rich:x:500:
mama:x:501:
katie:x:502:
jessica:x:503:
mysql:x:27:
test:x:504:
shared:x:505:rich, test
#
```

共享用户组现在已包含两个成员，分别是 test 和 rich。usermod 命令中的-G 参数用于将新用户组追加到用户账户的用户组列表末尾。

警告：在为用户账户指定用户组时需要格外小心。如果您使用了-G 参数，则指定的用户组名称将替代用户账户的默认用户组。-G 参数会将用户组添加到用户所属的用户组列表中，以保证默认用户组的完整性。

### 6.2.3 修改用户组

从/etc/group 文件中可以看到，其中并没有太多关于您要修改的用户组的信息。groupmod 命令允许您修改已有用户组的 GID（使用-g 参数）或用户组名称（使用-n 参数）：

```
# /usr/sbin/groupmod -n sharing shared
# tail /etc/group
haldaemon:x:68:
xfs:x:43:
gdm:x:42:
rich:x:500:
mama:x:501:
katie:x:502:
jessica:x:503:
mysql:x:27:
test:x:504:
sharing:x:505:test,rich
#
```

在修改用户组的名称时，GID 和用户组成员将仍然保持不变，仅有用户组名称会发生变化。由于所有安全性权限都基于 GID，因此您可以随时修改用户组的名称，而不会对文件的安全性造成不利的影响。

## 6.3 解码文件权限

现在，您已经对用户和用户组有了一定的了解。接下来，我们将解码您在使用 ls 命令时遇到的神秘的文件权限。本节讨论如何译解权限以及它们从何而来。

### 6.3.1 使用文件权限符号

您是否还记得第 3 章的内容，ls 命令允许我们查看 Linux 系统上的文件、目录和设备的文件权限：

```
$ ls -l
total 68
-rw-rw-r-- 1 rich rich   50 2007-09-13 07:49 file1.gz
-rw-rw-r-- 1 rich rich   23 2007-09-13 07:50 file2
-rw-rw-r-- 1 rich rich   48 2007-09-13 07:56 file3
-rw-rw-r-- 1 rich rich   34 2007-09-13 08:59 file4
-rwxrwxr-x 1 rich rich 4882 2007-09-18 13:58 myprog
-rw-rw-r-- 1 rich rich  237 2007-09-18 13:58 myprog.c
drwxrwxr-x 2 rich rich 4096 2007-09-03 15:12 test1
drwxrwxr-x 2 rich rich 4096 2007-09-03 15:12 test2
$
```

输出清单中的第一个字段是描述文件和目录的权限的代码。字段中的第一个字符定义了对象的类型：

- -表示文件；
- d 表示目录；
- l 表示链接；
- c 表示字符设备；
- b 表示块设备；
- n 表示网络设备。

在它后面是 3 组字符。每组的 3 个字符都定义一个访问权限：

- r 表示对象读权限；
- w 表示对象写权限；
- x 表示对象执行权限。

如果拒绝了某个权限，则该位置会出现一个破折号。这 3 组字符分别与 3 种级别的对象安全性相关：

- 对象的所有者；
- 拥有对象的用户组；
- 系统上的其他任何人。

详细分解如图 6-1 所示。

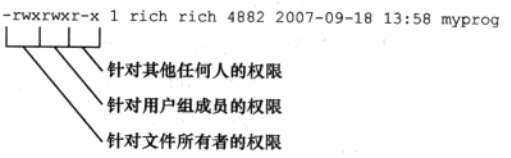



图 6-1 Linux 文件权限

讨论这一点的最简单的方法是举例说明，并逐步拆解文件权限：

```
-rwxrwxr-x 1 rich rich 4882 2007-09-18 13:58 myprog
```

文件 myprog 拥有以下权限字符集：

- rwx 针对文件所有者（设置为登录名 rich）；
- rwx 针对文件用户组所有者（设置为用户组名称 rich）；
- r-x 针对系统上的其他任何人。

这些权限表示用户登录名 rich 可以读取、写入和执行文件（被认为是完全权限）。同样，用户组 rich 中的成员也能读取、写入和执行文件。但是，不在 rich 用户组中的任何人只能读取和执行文件；w 被替换为破折号，表示写入权限未分配给此安全级别。

### 6.3.2 默认文件权限

您可能希望知道这些文件权限从何而来。答案是 umask。umask 命令为您创建的任何文件或目录设置默认权限：

```
$ touch newfile
$ ls -al newfile
-rw-r--r--    1 rich        rich           0 Sep 20 19:16 newfile
$
```

touch命令使用分配给我的用户账户的默认权限创建了文件。umask命令显示并设置默认权限：

```
$ umask
0022
$
```

遗憾的是，umask命令设置并非足够清晰，并且尝试理解它的确切工作原理也使事情变得更加复杂。第一个数字表示一种特殊的安全特性，称为**粘着位**。稍后，我们将在本章的“共享文件”一节中详细讨论这方面的内容。

随后的3个数字表示文件或目录的八进制umask值。要理解umask的工作原理，您首先需要理解八进制模式安全设置。

**八进制模式**安全设置将3个rwx权限值转换为一个3位的二进制值，由一个八进制值表示。在二进制表示中，每个位置都是一个二进制位。因此，如果读取权限是唯一的权限集，则值应该是r--，相关的二进制值为100，这表示八进制值为4。表6-5列出了可能的组合。

表6-5 Linux文件权限代码

| 权限 | 二进制 | 八进制 | 描述 |
|---|---|---|---|
| --- | 000 | 0 | 无权限 |
| --x | 001 | 1 | 只有执行权限 |
| -w- | 010 | 2 | 只有写入权限 |
| -wx | 011 | 3 | 写入和执行权限 |
| r-- | 100 | 4 | 只有读取权限 |
| r-x | 101 | 5 | 读取和执行权限 |
| rw- | 110 | 6 | 读取和写入权限 |
| rwx | 111 | 7 | 读取、写入和执行权限 |

八进制模式使用八进制权限并依次列出了3种安全级别（用户、用户组和任何人）的权限值。因此，八进制模式值664表示用户和用户组拥有读取和写入权限，而其他任何人只拥有读取权限。

现在，您已经知道了八进制模式权限，这使umask值的含义变得更加扑朔迷离。在我的Linux系统上，默认umask的八进制值显示为0022，但是，我创建的文件的八进制模式权限为644。这是如何发生的呢？

umask值只是一个掩码。它掩去您不希望为安全级别指定的权限。现在，我们需要研究一些八进制算法来继续探索之旅。

umask值是从对象完整权限集中减去的值。文件的完整权限是模式666（所有人都有读取/写入权限），而目录的完整权限是777（所有人都有读取/写入/执行权限）。

因此，在示例中，文件的初始权限为666，而应用022的umask之后，文件许可变为了644。

umask值通常设置在/etc/profile启动文件中（参阅第5章）。可以使用umask命令指定不同的默认umask设置：

```
$ umask 026
$ touch newfile2
$ ls -l newfile2
-rw-r-----    1 rich        rich           0 Sep 20 19:46 newfile2
$
```

通过将umask值设置为026，默认文件权限将变为640，因此，新文件现在只允许用户组成员读取，而系统上的其他人对该文件不具有任何权限。

umask值还可用于设置新目录：

```
$ mkdir newdir
$ ls -l
drwxr-x--x    2 rich        rich           4096 Sep 20 20:11 newdir/
```

由于目录的默认权限为777，因此通过umask生成的权限与新文件不同。026 umask值将从777

中减去，得出的目录权限设置为 751。

## 6.4 修改安全设置

如果已经创建了一个文件或目录，并且需要修改它的安全设置，Linux 提供了一些不同的实用工具可实现此目的。本节介绍如何修改文件或目录的已有权限、默认所有者和默认用户组设置。

### 6.4.1 修改权限

chmod 命令允许修改文件和目录的安全设置。chmod 命令的格式如下：

```
chmod options mode file
```

mode 参数允许您使用八进制或符号模式来设定安全设置。八进制模式设置相当简单；只需根据需要为文件指定标准的 3 位八进制代码：

```
$ chmod 760 newfile
$ ls -l newfile
-rwxrw----    1 rich        rich          0 Sep 20 19:16 newfile*
$
```

八进制文件权限将自动应用于指定的文件。符号模式权限的实现则没有这么容易。

chmod 命令采取了与 3 组 3 个字符的普通字符串不同的方法。使用符号模式指定权限的格式如下：

```
[ugoa...][[+-=][rwxXstugo...]
```

这种方式非常完美，不是吗？第一组字符串定义了新权限适用的对象：

- u 表示用户；
- g 表示用户组；
- o 表示其他（其他任何人）；
- a 表示上述所有。

接下来，使用一个符号表示您希望在已有权限中添加权限（+），从已有权限中减去权限（-），还是要为权限赋值（=）。

最后，第三个符号是为设置使用的权限。您可能会注意到，其中并不仅包括普通的 rwx 值。其他的设置包括：

- x 用于指定执行权限，仅当对象为目录或已经拥有执行权限时有效；
- s 用于设置正在执行的 UID 或 GID；
- t 用于保存程序文本；
- u 用于将权限设置为所有者的权限；
- g 用于将权限设置为用户组的权限；
- o 用于将权限设置为其他人的权限。

这些权限的使用方法如下：

```
$ chmod o+r newfile
$ ls -l newfile
-rwxrw-r--    1 rich        rich          0 Sep 20 19:16 newfile*
$
```

o+r 条目将读取权限添加到任何人（everyone）安全级别已经拥有的权限中。

```
$ chmod u-x newfile
$ ls -l newfile
-rw-rw-r--    1 rich        rich          0 Sep 20 19:16 newfile
$
```

u-x 条目将删除用户已有拥有的执行权限。注意，ls 命令设置通过在文件名中添加星号来指示该文件是否拥有执行权限。

options 参数提供了一些额外的特性来扩充 chmod 命令的行为。-R 参数将循环执行文件和目录修改。您可以在指定文件名时使用通配符，以便于在一个命令中修改多个文件的权限。

### 6.4.2 修改所有者

有时需要修改文件的所有者，例如，某人离开组织或开发人员创建的应用程序需要在生产阶段由系统账户拥有。Linux 提供了两个命令来完成此操作。chown 命令可以方便地修改文件的所有者，chgrp 命令允许您修改文件的默认用户组。

chown 命令的格式如下：

```
chown options owner[.group] file
```

您可以为文件的新所有者指定登录名或数值 UID：

```
# chown dan newfile
# ls -l newfile
-rw-rw-r--    1 dan         rich          0 Sep 20 19:16 newfile*
#
```

非常简单。chown 命令还允许您修改文件的用户和用户组：

```
# chown dan.dan newfile
# ls -l newfile
-rw-rw-r--    1 dan         dan           0 Sep 20 19:16 newfile*
#
```

如果您确实需要加入一些技巧，那么可以只修改文件的默认用户组：

```
# chown .rich newfile
# ls -l newfile
-rw-rw-r--    1 dan         rich          0 Sep 20 19:16 newfile*
#
```

最后，如果您的 Linux 系统使用与用户登录名相匹配的用户组名称，则可以在一条命令中同时修改这两者：

```
# chown test. newfile
# ls -l newfile
-rw-rw-r--    1 test        test          0 Sep 20 19:16 newfile*
#
```

chown 命令使用了一些不同的选项参数。-R 参数允许您使用通配符对子目录和文件执行递归式修改。-h 参数还可以修改任何与文件具有符号关联的文件的所有者：

注意：只有根用户可以修改文件的所有者。任何用户都可修改文件的默认用户组，但是用户必须是与修改操作相关的用户组中的成员。

chgrp 命令提供了一种简单的方法来仅修改文件或目录的默认用户组：

```
$ chgrp shared newfile
$ ls -l newfile
-rw-rw-r-- 1 rich      shared        0 Sep 20 19:16 newfile*
$
```

现在，共享用户组中的任何成员都可以对文件执行写入操作。这是在 Linux 系统上共享文件的一种方法。但是，在系统上的一组人中共享文件有时需要一些技巧。下一节将讨论如何实现它。

## 6.5 共享文件

您可能已经知道，创建用户组是在 Linux 系统上共享文件访问的一种方法。但是，对于完全的文件共享环境，事情却不是这么简单。

您已经从“解码文件权限”一节中了解到，在创建新文件时，Linux 使用默认 UID 和 GID 为新文件分配文件权限。为允许其他人访问文件，您需要修改任何人（everyone）安全用户组的安全权限，或者为文件分配一个不同的、包含其他用户的默认用户组。

对于大型环境，如果您希望在多个人之间创建和共享文档，那么这样操作将非常痛苦。幸运的是，可以通过一个简单的解决方案来解决此问题。

Linux 为每个文件和目录都存储了 3 个额外的信息位：

- **set user id（SUID）**：当文件由用户执行时，程序将在文件所有者的权限下运行。

- **set group id（SGID）**：对于文件，程序将在文件用户组的权限下运行。对于目录，目录中创建的新文件使用目录用户组作为默认用户组。
- **粘着位**：进程结束后，文件仍然保留（粘着）在内存中。

SGID 位对于共享文件非常重要。通过启用 SGID 位，您可以强制在共享目录中创建的所有新文件都由目录的用户组和现在各个用户的用户组拥有。

可以使用 chmod 命令来设置 SGID。它将被添加到标准 3 位八进制值的开头（组成四位八进制值），或者，也可以在符号模式中使用符号 s。

如果使用八进制模式，则需要知道这些位的排列方式，如表 6-6 所示。

表 6-6　　chmod SUID、SGID 和粘着位八进制值

| 二进制 | 八进制 | 描述 |
|---|---|---|
| 000 | 0 | 清空所有位 |
| 001 | 1 | 设置粘着位 |
| 010 | 2 | 设置 SGID 位 |
| 011 | 3 | 设置 SGID 和粘着位 |
| 100 | 4 | 设置 SUID 位 |
| 101 | 5 | 设置 SUID 和粘着位 |
| 110 | 6 | 设置 SUID 和 SGID 位 |
| 111 | 7 | 设置所有位 |

因此，要创建一个始终为所有新文件设置了目录用户组的共享目录，您只需要为目录设置 SGID：

```
$ mkdir testdir
$ ls -l
drwxrwxr-x    2 rich        rich          4096 Sep 20 23:12 testdir/
$ chgrp shared testdir
$ chmod g+s testdir
$ ls -l
drwxrwsr-x    2 rich        shared        4096 Sep 20 23:12 testdir/
$ umask 002
$ cd testdir
$ touch testfile
$ ls -l
total 0
-rw-rw-r--    1 rich        shared           0 Sep 20 23:13 testfile
$
```

第一步是使用 mkdir 命令创建一个希望共享的目录。接下来，使用 chgrp 命令将目录的默认用户组修改为包含需要共享文件的成员的用户组。最后，为目录设置 SGID，以确保在该目录中创建的任何文件都使用共享用户组名称为默认用户组。

要让此环境能够正常运行，所有用户组成员都需要设置各自的 umask 值，使用户组成员能够写入文件。这也是我为何要将 umask 修改为 002 的原因。

完成所有这些任务之后，我可以进入共享目录并创建新文件。不出所料，新文件使用目录的默认用户组，而不是我的用户账户的默认用户组。现在，共享用户组中的任何用户都可以访问此文件。

## 6.6 小结

本章讨论了在系统上管理 Linux 安全需要了解的一些命令行命令。Linux 使用用户 ID 和用户组 ID 的机制来保护对文件、目录和设备的访问。Linux 将与用户账户相关的信息存储在/etc/passwd 文件中，并将与用户组相关的信息存储在/etc/group 文件中。每个用户都分配了一个唯一的数值用户 ID

和文本登录名，用于标识系统中的用户。用户组也分配了一个唯一的数值用户组 ID 和文本用户组名称。用户组可以包含一个或多个允许共享系统资源访问的用户。

可以通过一些命令来管理用户账户和用户组。useradd 命令允许您创建新用户账户，groupadd 命令允许您创建新用户组账户。要修改已有的用户账户，可以使用 usermod 命令。与此类似，groupmod 命令用于修改用户组账户信息。

Linux 使用一种复杂的位机制来确定文件和目录的访问权限。每个文件都包含 3 个保护安全级别：文件的所有者、可访问文件的默认用户组和一个针对系统上其他任何人的级别。每个安全级别都由 3 个访问位定义：读取、写入和执行。这 3 个位的组合通常写作符号 rwx，它们分别表示读取、写入和执行。如果要拒绝某个权限，则将它的符号替换为一个破折号（例如，r--表示只读权限）。

符号权限通常由八进制值表示，3 个二进制位被组合到一个八进制位中，并通过 3 个八进制位来表示 3 个安全级别。umask 命令用于为在系统上创建的文件和目录设定默认安全设置。系统管理员通常在 /etc/profile 文件中设置默认 umask 值，但是，您可以随时使用 umask 命令来修改您的 umask 值。

chmod 命令用于修改文件和目录的安全设置。只有文件的所有者可以修改文件或目录的权限。但是，根用户可以修改系统上任何文件或目录的安全设置。可以使用 chown 和 chgrp 命令来修改文件的默认所有者和用户组。

最后，本章在结束部分讨论了如何使用 set GID 位来创建共享目录。SGID 位强制在某个目录中创建的任何新文件或目录都使用父目录（而不是创建它们的用户）的默认用户组名称。这为在系统上的各用户之间共享文件提供了一个简单的方法。

现在，您已经了解了 Linux 文件安全的方方面面，接下来可以开始创建一些程序了。但是，在开始编写代码之前，我们还有一个元素需要讨论：编辑器。如果您计划编写 shell 脚本，则需要一个合适的环境来创建作品。下一章将讨论您可以在不同 Linux 环境中使用的各种文本编辑器。

# 第 7 章
# 使用编辑器

CHAPTER 7

本章内容：

- 使用 vim 编辑器；
- 理解 emacs；
- 熟悉 KDE；
- GNOME 编辑器。

在开始编写 shell 脚本之前，您需要了解如何使用至少一种 Linux 中的编辑器。只有熟练掌握了这些出色的特性，如搜索、剪切和粘贴，您才能更加快速地开发 shell 脚本。本章讨论 Linux 世界中的一些的主要的文本编辑器。

## 7.1 vim 编辑器

如果您在命令行模式中工作，则至少需要熟悉在 Linux 控制台下运行的一种文本编辑器。vi 编辑器是在 Unix 系统中使用的原始编辑器。它使用控制台图形模式模拟了一个文本编辑窗口，允许您直观查看文件、在文件中移动，以及插入、编辑和替换文本。

虽然它可能是当今世界上最为复杂的编辑器（至少在憎恨它的人看来是这样），但它提供了许多实用的特性，使它成为 Unix 管理员手中长盛不衰的利器。

当 GNU 项目将 vi 编辑器移植到开源领域中时，他们选择对它做出一番改进。由于它不再类似于 Unix 世界中的原始 vi 编辑器，因此他们还将它重新命名为 vim。

几乎所有的 Linux 发行版都使用别名（参阅第 5 章）vi 来表示 vim：

```
$ alias vi
alias vi='vim'
$
```

本节介绍使用 vim 编辑器来编辑文本 shell 脚本文件的基础知识。

### 7.1.1 vim 基本用法

vim 编辑器处理内存缓冲中的数据。要启动 vim 编辑器，只需键入 vim 命令（或者别名 vi）和希望编辑的文件的名称：

```
$ vim myprog.c
```

如果您在启动 vim 时没有指定文件名，或者文件不存在，则 vim 将打开一个新的缓冲区域来进行编辑。如果您在命令行中指定了一个已有文件，则 vim 会将该文件的整个内容读取到已经准备进行编辑的缓冲区中，如图 7-1 所示。

vim 编辑器将检测会话的终端类型（参阅第 2 章），并使用全屏模式来使用整个控制台窗口表示编辑区。

初始 vim 编辑窗口显示了文件的内容（如果有内容），并在窗口底部显示了一个消息行。如果文件内容未占据整个屏幕，则 vim 会在不属于文件的行上放置一个波浪号（如图 7-1 所示）。

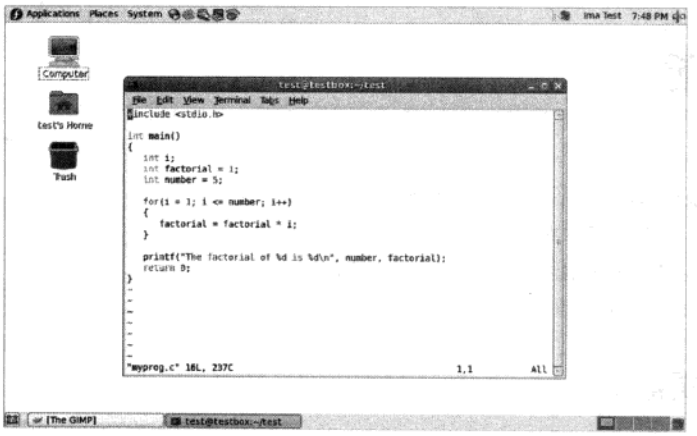



图 7-1　vim 主窗口

底部的消息行指示关于被编辑文件的信息，这取决于该文件的状态和 vim 安装中的默认设置。如果该文件是新文件，则消息行显示的内容为 [New File]。

vim 编辑器有两种运行模式：

- 正常模式；
- 插入模式。

当第一次打开某个文件（或创建新文件）进行编辑时，vim 编辑器将进入正常模式。在正常模式中，vim 编辑器会将击键解释为命令（稍后将详细介绍）。

在插入模式中，vim 会将每次击键插入到缓冲的当前光标位置。要进入插入模式，按 i 键即可。要离开插入模式并返回正常模式，按键盘上的 Esc 键即可。

在正常模式中，可以使用方向键在文本区中移动光标（只要 vim 正确检测到您的终端类型）。如果碰巧您的终端连接未定义方向键，这也不是完全没有希望。vim 命令包含用于移动光标的命令：

- h 用于左移一个字符。
- j 用于下移一行（文本中的下一行）。
- k 用于上移一行（文本中的上一行）。
- l 用于右移一个字符。

在大文本文件中逐行移动难免会让人觉得单调乏味。幸运的是，vim 提供了一些命令，可以帮助提高操作速度：

- PageDown（或 Ctl+f）用于下移一个屏幕的数据；
- PageUp（或 Ctl+b）用于上移一个屏幕的数据；
- G 用于移动到缓冲的最后一行；
- num G 用于移动到缓冲中的第 num 行；
- gg 用于移动到缓冲的第一行。

vim 编辑器在正常模式中有一个特殊的特性，即命令行模式。命令行模式提供了一个交互式的命令行，您可以在其中输入额外的命令来控制 vim 中的操作。要进入命令行模式，请在正常模式中按冒

号键。光标将移动到消息行并出现一个冒号，您可以在其中输入命令。

在命令行模式中，您可以通过一些命令来将缓冲保存到文件，并退出 vim：

- 如果未对缓冲数据执行任何修改，q 将直接退出；
- q!退出并放弃对缓冲数据的任何修改；
- w filename 将文件另存为指定的文件名；
- wq 将缓冲数据保存到文件并退出。

了解了一些基本的 vim 命令之后，您可能理解了为何一些人对 vim 编辑器深恶痛绝。为了能够充分发挥 vim 的作用，您必须了解一些隐晦的命令。但是，熟悉了一些基本的 vim 命令之后，您可以快速在命令行中直接编辑文件，不管您处于哪种类型的环境。

### 7.1.2 编辑数据

在插入模式中，可以将数据插入到缓冲中，但是，有时需要在进入缓冲之后添加或删除数据。在普通模式中，vim 编辑器提供了一些命令用于编辑缓冲中的数据。表 7-1 列出了一些常用的 vim 命令。

一些编辑命令还允许您使用数值修改程序来指示执行命令的次数。例如，命令 2x 将从当前光标位置开始删除两个字符，命令 5dd 将从当前光标位置开始删除 5 行。

表 7-1 vim 编辑命令

| 命令 | 描述 |
|---|---|
| x | 删除当前光标位置的字符 |
| dd | 删除当前光标位置的行 |
| dw | 删除当前光标位置的单词 |
| d$ | 删除当前光标位置到行末的所有内容 |
| J | 删除当前光标位置所在行末的断开线 |
| a | 将数据附加在当前光标位置后面 |
| A | 将数据附加在当前光标位置所在行后面 |
| r *char* | 将当前光标位置的单个字符替换为 *char* |
| R *text* | 将当前光标位置的数据覆盖为 *text*，直到您按 Esc 键为止 |

警告：尝试在 vim 中使用 PC 键盘的 Backspace 键或 Delete 键时需要格外小心。vim 编辑器通常会将 Delete 键识别为 x 命令的功能，即删除当前光标位置的字符。通常，vim 编辑器不会识别 Backspace 键。

### 7.1.3 复制和粘贴

现代编辑器的标准特性是能够剪切或复制数据，然后再将它粘贴到文档的其他地方。vim 编辑器提供了一种方法来实现此操作。

剪切和粘贴相对较为简单。您已经从表 7-1 中了解了从缓冲中删除数据的命令。但是，当 vim 删除数据时，它实际上将它保存在一个单独的寄存器中。您可以通过使用 p 命令来检索该数据。

例如，可以使用 dd 命令来删除一个文本行，然后将光标移到缓冲中的目标位置，然后使用 p 命令即可。p 命令会将文本插入到当前光标位置所在行的后面。可以使用任何删除文本的命令来实现此目的。

复制文本稍微复杂一点。vim 中的复制命令是 y（用于移出）。您可以在 y 命令中使用与 d 命令相同的第二个字符（yw 用于移出单词，y$用于移出当前位置至行尾的文本）。移出文本之后，将光标移到文本中的目标位置，然后使用 p 命令。移出的文本现在将显示在该位置。

移出操作的麻烦在于，您不知道发生了什么，因为您不会影响移出的文本。在粘贴移出的文本之前，您不知道移出了什么。但是，vim 提供了另外一种特性，可以帮助我们解决移出问题。

**可视模式**会在您移动光标时突出显示文本。可以通过可视模式来选择要移出进行粘贴的文本。要进入可视模式，将光标移动到要开始进行移出的位置，然后按 v 键。您会注意到，光标所在位置的文本已经突出显示。接下来，移动光标选定要移出的文本（您甚至可以选中多行内容来移出多行文本）。移动光标时，vim 将突出显示移出区域中的文本。选中要复制的文本之后，按 y 键激活移出（yank）命令。现在，您已经将文本保存在寄存器中，将光标移动出要粘贴的位置，然后使用 p 命令即可以完成粘贴操作。

### 7.1.4　搜索和替换

借助 vim 搜索命令，您可以方便地搜索缓冲中的数据。要输入搜索字符串，按正斜杠（/）键即可。光标将到达消息行，并且 vim 将显示一个正斜杠。输入您要查找的内容，然后按 Enter 键。vim 编辑器将通过以下 3 个操作之一进行响应：

- 如果单词出现在当前光标所在位置的后面，它将跳到文本首次出现的位置。
- 如果单词未出现在当前光标所在位置的后面，它将搜索整个文件并跳到文本在文件中首次出现的位置（并通过消息说明）。
- 生成一个错误消息，声明未在文件中找到该文本。

要继续搜索相同的单词，按正斜杠字符，然后按 Enter 键，或者也可以使用 n 键（表示查找下一个）。

替换命令允许快速将文本中的某个单词替换为另一个单词。要使用替换命令，必须处于命令行模式。替换命令的格式如下：

```
:s/old/new/
```

vim 编辑器将跳转到文本 old 第一次出现的位置，并将它替换为文本 new。可以对替换命令做少许修改，以便替换多处文本：

- :s/old/new/g 用于替换一行中的所有 old；
- :#,#s/old/new/g 用于替换两行之间的所有 old；
- :%s/old/new/g 用于替换整个文件中的所有 old；
- :%s/old/new/gc 用于替换整个文件中的所有 old，但每次替换时都会弹出提示。

如您所见，作为一款命令行文本编辑器，vim 提供了不少高级特性。由于任何 Linux 发行版都包含该编辑器，因此了解 vim 编辑器的基本用法是大有裨益的，您可以随心所欲地编辑脚本，而不用担心自己使用什么样的操作系统，或者是否有合适的工具可用。

## 7.2　emacs 编辑器

emacs 编辑器是一款极为流行的编辑器，它甚至在 Unix 之前就已经出现了。开发人员对它喜爱有加，不但它移植到 Unix 环境中，还将它移植到了 Linux 环境中。emacs 编辑器最初以控制台编辑器的形式出现，与 vi 极为类似，但已经开始向图形世界迁移。

emacs 编辑器仍然提供了最初的控制台模式编辑器，但它现在可以通过图形 X Windows 窗口在图形环境中编辑文本。通常，当您在命令行中启动 emacs 编辑器时，它将判断您是否有可用的 X Windows 会话，并且已启动图形模式。如果没有，它将以控制台模式启动。

本节将介绍控制台模式和图形模式这两种 emacs 编辑器，以便于您了解如何选择适合自己的编辑器。

### 7.2.1　在控制台中使用 emacs

控制台模式版本的 emacs 是另一种使用大量键命令执行编辑功能的编辑器。emacs 编辑器使用的键命令涉及 Control 键（PC 键盘上的 Ctrl 键）和 Meta 键。在大多数 PC 终端模拟器包中，Meta 键与 PC 的 Alt 键相对应。官方 emacs 文档将 Ctrl 键简称为 C-，并将 Meta 键简称为 M-。因此，如果您输

入 Ctrl+x 组合键，则文档将显示 C-x。在本书中，我们也采用这种表示方法，以避免产生混淆。

### 1. emacs 基本用法

要使用 emacs 编辑文件，请在命令行中输入：

```
$ emacs myprog.c
```

此时将打开 emacs 控制台模式窗口，其中显示了一个简短介绍及帮助界面。不用担心；当您按某个键时，emacs 会将文件加载到活动缓冲中并显示文本，如图 7-2 所示。

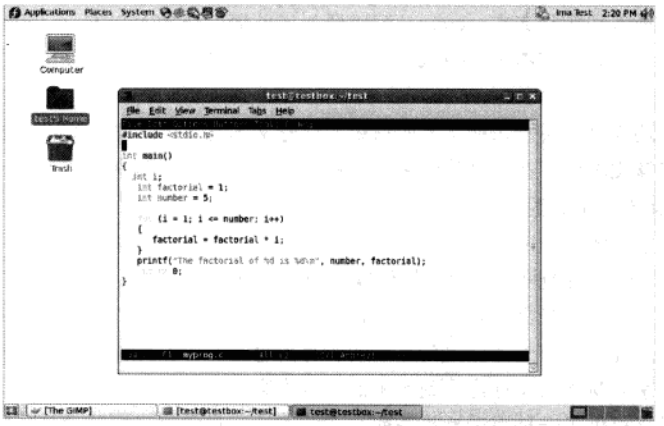



图 7-2 使用控制台模式的 emacs 编辑器编辑文件

您会注意到，控制台模式窗口顶部显示了一个典型的菜单栏。遗憾的是，您不能在控制台模式中使用菜单，它仅支持图形模式。

在 vim 编辑器中，您需要不断进入和退出插入模式，以便于在输入命令和插入文本之间切换，而 emacs 与此不同，它只拥有一个模式。如果键入了一个可打印的字符，emacs 会将它插入到当前光标位置。如果键入一条命令，emacs 则会执行该命令。

要在缓冲区域中移动光标，可以使用方向键、PageUp 和 PageDown 键，前提是 emacs 已正确检测到您的终端模拟器。如果没有，可以通过一些命令来移动光标：

- C-p 用于向上移动一行（文本中的前一行）。
- C-b 用于向左（后）移动一个字符。
- C-f 用于向右（前）移动一个字符。
- C-n 用于向下移动一行（文本中的后一行）。

还可以通过一些命令使光标在文本中跳跃较长距离：

- M-f 用于向右（前）移至下一个单词。
- M-b 用于向左（后）移至前一个单词。
- C-a 用于移至当前行的开始处。
- C-e 用于移至当前行的结尾处。
- M-a 用于移至当前句的开始处。
- M-e 用于移至当前句的结尾处。
- M-v 用于将数据向后移动一个屏幕。
- C-v 用于将数据向前移动一个屏幕。
- M-< 用于移至文本的第一行。
- M-> 用于移至文本的最后一行。

还有一些命令用于将编辑器缓冲保存回文件，并退出 emacs：

- C-x C-s 用于将当前缓冲内容保存到文件中。
- C-z 用于退出 emacs，但仍然让它在会话中运行，以便能够返回 emacs。
- C-x C-c 用于退出 emacs 并退出程序。

您会注意到，其中两个命令需要两个键命令。C-x 命令被称为扩展命令。这提供了另一组完整的操作命令。

**2. 编辑数据**

对于在缓冲中插入和删除文本，emacs 编辑器相当便捷。要插入文本，只需将光标移动到目标位置并键入文本即可。在删除文本时，emacs 使用 Backspace 键删除当前光标位置之前的字符，并使用 Delete 键删除当前光标位置的字符。

emacs 编辑器还提供了用于杀死文本的命令。删除文本与杀死文本之间的差异在于，当您杀死文本时，emacs 会将该文本保存在一个临时区域，您可以在其中重新检索文本（请参阅“复制和粘贴”一节）。而删除的文本将永远消失。

以下命令用于杀死缓冲中的文本：

- M-Backspace 用于杀死当前光标位置之前的单词。
- M-d 用于杀死当前光标位置之后的单词。
- C-k 用于杀死从当前光标位置到行末的所有内容。
- M-k 用于杀死从当前光标位置到句末的所有内容。

emacs 编辑器还提供了一种用于批量杀死文本的方法。只需将光标移动到目标区域的开始处，并按 C-@或 C-Spacebar 键。然后将光标移动到目标区域的结束处，并按 C-w 键。这两个位置之间的所有文本将被杀死。

如果您在杀死文本时不小心出错，那么可以使用 C-u 命令来撤销杀死命令，并将数据返回杀死前的状态。

**3. 复制和粘贴**

您已经了解了如何剪切 emacs 缓冲区中的数据；现在，我们将介绍如何将它粘贴到别处。遗憾的是，如果您使用 vim 编辑器，则这种流程会让您在使用 emacs 编辑器时感到混淆。非常巧合的是，在 emacs 中粘贴数据也被称为移出。在 vim 编辑器中，复制被称为移出，这对于同时使用两种编辑器的用户来说会比较难以记住。

使用杀死命令杀死数据之后，将光标移动到要粘贴数据的位置，然后使用 C-y 命令。此操作会将文本移出临时区并将它粘贴到当前光标位置。C-y 命令用于移出上次杀死的文本。如果您已经执行了多个杀死命令，则可以 M-y 命令在它们之间循环切换。

要复制文本，只需将它移回被杀死的位置，然后移动到新位置并再次使用 C-y 命令。您可以任意多次地移出文本。

**4. 搜索和替换**

在 emacs 编辑器中搜索文本可以借助 C-s 和 C-r 命令来完成。C-s 命令用于在缓冲区中执行从当前光标位置到缓冲结束位置的向前搜索，而 C-r 命令用于在缓冲区中执行从当前光标位置到缓冲开始位置的向后搜索。

当您输入 C-s 或 C-r 命令时，行底会弹出一个提示，询问您要搜索的文本。emacs 可以执行两种类型的搜索。

在**增量**搜索中，emacs 编辑器将在您键入单词时以实时模式执行文本搜索。键入第一个字母时，它会突出显示缓冲中存在的所有该字母。键入第二个字母时，它会突出显示缓冲中所有这两个字母的组合，依此类推直到您完成要搜索的文本。

在**非增量**搜索中，在 C-s 或 C-r 命令后按 Enter 键。这会将搜索查询锁定到行底区域，并允许您

在搜索之前键入要搜索的完整文本。

要将已有文本字符串替换为新的文本字符串，需要使用 M-x 命令。此命令需要一个带参数的文本命令。

该文本命令是 replace-string。键入命令之后，按 Enter 键，emacs 将询问您要替换的已有文本字符串。输入该字符串之后，再次按 Enter 键，这时 emacs 会询问您新的替换文本字符串。

**5. 在 emacs 中使用缓冲**

emacs 编辑器拥有多个缓冲区，因此允许您同时编辑多个文件。您可以将文件加载到缓冲中，然后在编辑时在各缓冲之间来回切换。

要在 emacs 中将新文件加载到缓冲中，可以使用 C-x C-f 组合键。这是 emacs 的读档（find a file）模式。它会将您带到窗口的底行并允许您输入要编辑的文件的名称。如果您不知道该文件的名称或位置，只需按 Enter 键即可。这将在编辑窗口中弹出一个文件浏览器，如图 7-3 所示。

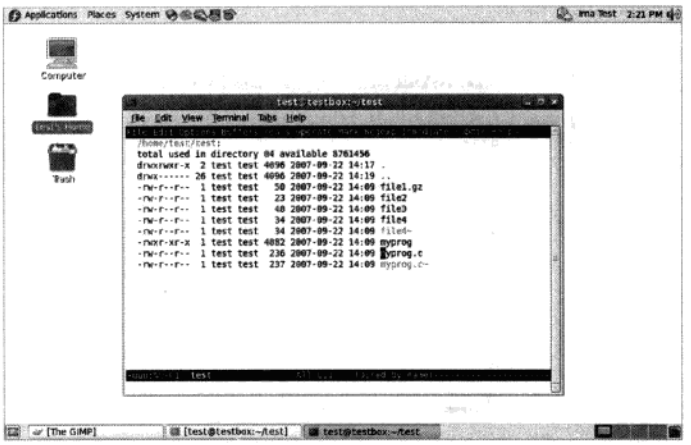



图 7-3 emacs 的读档模式浏览器

在文件浏览器中，可以浏览到要编辑的文件。要返回上一个目录层次，移动到两点条目，并按 Enter 键。要进入某个目录，移动到该目录并按 Enter 键。发现要编辑的文件时，只需按 Enter 键，这时 emacs 会将它加载到新缓冲区中。

可以通过按 C-x C-b 扩展命令组合来列出活动的缓冲区。emacs 编辑器将分离编辑器窗口并在底部窗口中显示缓冲列表。除了主要编辑缓冲区之外，emacs 始终提供另外两个缓冲区：

- 一个暂存区，称为*scratch*；
- 一个消息区，称为*Messages*。

暂存区允许您输入 LISP 编程命令，以及输入您自己的注释。消息区将在操作过程中显示 emacs 生成的消息。如果在使用 emacs 时出现了任何错误，则它们将出现在消息区中。

可以采用两种方式在窗口中切换到不同的缓冲区：

- C-x o 用于切换到缓冲清单窗口。使用方向键移动到所需的缓冲区，然后按 Enter 键。
- C-x b 用于键入希望切换到的缓冲区的名称。

选择切换到缓冲清单窗口时，emacs 将在新窗口区域中打开缓冲区。emacs 编辑器允许在一个会话中打开多个窗口。下一节将讨论如何在 emacs 中管理多个窗口。

**6. 在控制台模式 emacs 中使用多个窗口**

早在图形窗口想法出现的多年之前，控制台模式 emacs 编辑器就已经开发。但是，它在当时已经非常先进，因为它可以支持在主 emacs 窗口中打开多个编辑窗口。

您可以借助下面两个命令之一将 emacs 编辑窗口分隔成多个窗口：

- `C-x 2` 用于将窗口水平分隔成两个窗口。
- `C-x 3` 用于将窗口垂直分隔成两个窗口。

要从一个窗口移动到另一个窗口，可以使用 `C-x o` 命令。您会注意到，当创建新窗口时，emacs 将在新窗口中使用原始窗口中的缓冲区。移动到新窗口之后，可以使用 `C-x C-f` 命令加载新文件，或者使用其中一个命令切换到新窗口中的不同缓冲区。

要关闭某个窗口，先切换到它并使用 `C-x 0`（数字 0）命令。如果您要关闭除当前窗口之外的其他所有窗口，可以使用 `C-x 1`（数字 1）命令。

### 7.2.2　在 X Windows 中使用 emacs

如果在 X Windows 环境（如 KDE 或 GNOME 桌面）中使用 emacs，它将以图形模式启动，如图 7-4 所示。

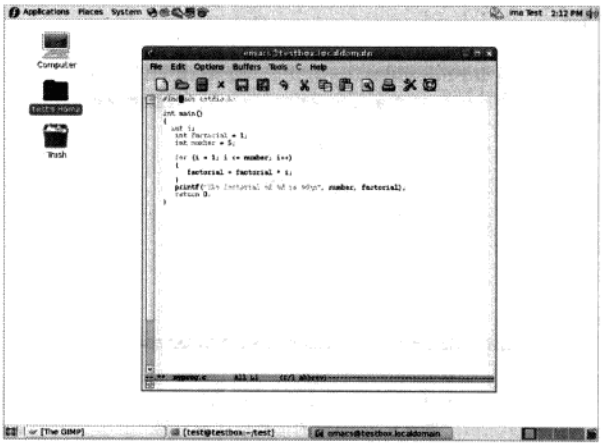



图 7-4　emacs 图形窗口

如果您已经在控制台模式中使用了 emacs，则应该对 X Windows 模式相当熟悉。所有键命令都有各自的菜单栏项目。emacs 菜单栏包含以下项目：

- File 允许您在窗口中打开文件，创建新窗口，关闭窗口，保存缓冲和打印缓冲。
- Edit 允许您将选定文本剪切和复制到剪贴板，将剪贴板数据粘贴到当前光标位置，搜索文本和替换文本。
- Options 提供对许多 emacs 特性的设置，例如高亮显示、自动换行、光标类型和设置字体。
- Buffers 列出当前可用的缓冲，并允许您方便地在缓冲区之间切换。
- Tools 提供对 emacs 中的高级特性的访问，例如命令行界面访问、拼写检查、比较文件之间的文本（称为 diff）、发送电子邮件消息、日历和计算器。
- Help 提供 emacs 在线手册，提供特定 emacs 函数方面的帮助。

除了普通图形 emacs 菜单栏项目之外，编辑器缓冲经常还有一个特定于文件类型的单独项目。在图 7-4 中，我打开了一个 C 程序，因此提供了一个 C 菜单项目，允许对 C 语法高亮以及从命令提示符处编译、运行和调试代码进行高级设置。

图形 emacs 窗口是原始的控制台应用程序向图形世界迁移的一个例子。目前，随着许多 Linux 发行版都提供了图形桌面（甚至在不需要它们的服务器上），图形编辑器正在日益普及。两大流行的 Linux 桌面环境（KDE 和 GNOME）也已经针对它们的环境提供了图形文本编辑器，本章的其余部分将介绍

这方面内容。

## 7.3 KDE 系列编辑器

如果您的 Linux 发行版使用 KDE 桌面（参阅第 1 章），那么在考虑文本编辑器时有许多选择。KDE 项目官方支持两种不同的文本编辑器。

- KWrite：一种单屏文本编辑包；
- Kate：特性全面的多窗口文本编辑包。

这两种编辑器都是图形编辑器，它们包含许多高级特性。Kate 编辑器提供较多的高级特性，并具备标准文本编辑器一般没有优良特性。本节将分别描述这两种编辑器，并展示可对您的 shell 脚本编辑提供帮助的一些特性。

### 7.3.1 KWrite 编辑器

KDE 环境中的基本编辑器是 KWrite。它提供简单的单词处理样式的文本编辑，以及对代码语法高亮和编辑的支持。默认的 KWrite 编辑窗口如图 7-5 所示。

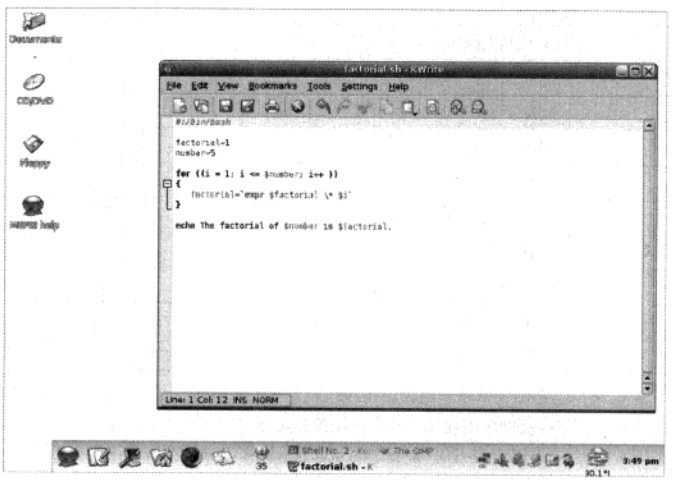



图 7-5 正在编辑 shell 脚本程序的默认 KWrite 窗口

您无法从图 7-5 中看出端倪，但 KWrite 编辑器可以识别若干种不同类别的编程语言，并使用颜色编码来区分常量、函数和注释。此外，`for` 循环提供了一个图标用于链接左右括号。这被称为折叠标记（folding marker）。通过单击该图标，可以将函数折叠成单行。在开发大型应用程序时，这是一项极有用的特性。

KWrite 窗口提供了完整的剪切和粘贴功能，即使用鼠标和方向键来实现。就像在 word processor 中一样，可以突出显示并剪切（或复制）缓冲区中的任何文本，并将它粘贴到任何其他位置。

要使用 KWrite 编辑文件，可以从桌面上的 KDE 菜单系统中选择 KWrite（一些 Linux 发行版甚至为它创建了一个控制板图标），或者从命令行提示符启动它。

```
$ kwrite factorial.sh
```

`kwrite` 命令提供了一些命令行参数，您可以使用这些参数来定制命令的启动方式：

- `--stdin` 指定 KWrite 从标准输入设备中读取数据，而不是从文件中。
- `--encoding` 为该文件指定使用的字符编码类型。
- `--line` 指定在编辑器窗口中打开的文件的起始行数。

- `--column` 指定在编辑器窗口中打开的文件的起始列数。

KWrite 编辑器在编辑窗口顶部提供了一个菜单栏和一个工作栏，允许您选择特性并更改 KWrite 编辑器的配置设置。

该菜单栏包含以下项目：

- **File** 用于载入、保存、打印和导出文件中的文本。
- **Edit** 用于操作缓冲区中的文本。
- **View** 用于管理文本在编辑器窗口中的显示方式。
- **Bookmarks** 用于处理返回文本特定位置的指针。
- **Tools** 包含操作文本的专门特性。
- **Setting** 用于配置编辑器处理文本的方式。
- **Help** 用于获取关于编辑器和命令的信息。

Edit 菜单栏项目提供的命令可以满足您所有的文本编辑需要。您不再需要记住含义模糊的键命令（KWrite 也支持这种方式），而只需选择 Edit 菜单栏中的项目，如表 7-2 所示。

表 7-2　　KWrite 的 Edit 菜单项目

| 项目 | 描述 |
|---|---|
| Undo | 返回上次操作之前的状态 |
| Redo | 返回上次撤销操作之前的状态 |
| Cut | 删除所选文本并将它放置在剪贴板中 |
| Copy | 将所选文本复制到剪贴板 |
| Copy as HTML | 将所选文本作为 HTML 代码复制到剪贴板 |
| Paste | 在当前光标位置插入剪贴板中的当前内容 |
| Select All | 选择编辑器中的所有文本 |
| Deselect | 取消当前选择的任何文本 |
| Block Selection Mode | 启用时，允许您选择两列之间的文本，而不整行 |
| Overwrite Mode | 将插入模式切换为覆盖模式，将文本替换为新键入的文本，而不是仅插入新文本 |
| Find | 打开 Find Text 对话框，允许您定制文本搜索 |
| Find Next | 在缓冲区中向后重复上一次查找操作 |
| Find Previous | 在缓冲区中向前重复上一次查找操作 |
| Replace | 打开 Replace With 对话框，允许您定制文本搜索和替换 |
| Go to Line | 打开 Goto 对话框，允许您输入行号。光标将移动到指定行 |

Find Text 对话框，如图 7-6 所示，是用于在缓冲区中搜索文本的一种强大的工具。

Find Tex 对话框使用当前光标位置处的单词作为要搜索的默认文本值。它还支持您在搜索中使用正则表达式（在第 17 章中介绍）。您还可以使用一些复选框来定制搜索，例如，指定搜索是否区分大小写，或者是否只查找整个单词，而不是查找单词中的文本。

Tools 菜单栏项目为操作缓冲区中的文本提供了一些方便的特性。表 7-3 描述了 KWrite 中可用的工具。

表 7-3　　KWrite 工具

| 工具 | 描述 |
|---|---|
| Read Only Mode | 锁定文本，这样便不能在编辑器执行更改 |
| Filetype | 选择在文本中使用的文件类型（filetype）方案 |
| Highlighting | 根据内容高亮显示文本，例如程序代码或配置文件 |

续表

| 工具 | 描述 |
| --- | --- |
| Indentation | 自动根据选择缩进行 |
| Encoding | 设定文本使用的字符集编码 |
| Spelling | 从文本开头就启动拼写检查程序 |
| Spelling（from cursor） | 从当前光标位置启动拼写检查程序 |
| Spellcheck Selection | 仅对所选文本部分启动拼写检查程序 |
| Indent | 段落缩进增加 1 |
| Unindent | 段落缩进减少 1 |
| Clean Indentation | 将所有段落缩进返回到初始设置 |
| Align | 强制当前行或所选行返回到默认缩进设置 |
| Comment | 根据检测到的文件类型在当前行添加注释符号 |
| Uncomment | 根据检测到的文件类型从当前行删除注释符号 |
| Uppercase | 将所选文本或当前光标位置的字符设置为大写字母 |
| Lowercase | 将所选文本或当前光标位置的字符设置为小写字母 |
| Capitalize | 将所选文本或当前光标位置的单词的首字母设置为大写 |
| Join Lines | 将所选行或当前光标位置的行与下一行合并为一行 |
| Word Wrap Document | 在文本中启用自动换行。如果某行超过了编辑器窗口的边界，则该行将自动转换到下一行 |

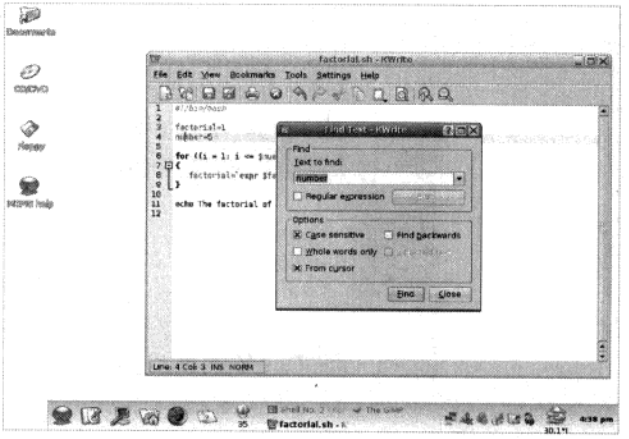



图 7-6 KWrite 的 Find Tex 对话框

简单文本编辑器提供了许多工具！

Settings 菜单包含 Configure Editor 对话框，如图 7-7 所示。

Configuration 对话框使用左侧图标供您选择在 KWrite 中要配置的特性。选择某个图标时，对话框右侧将显示该特性的配置设置。

Appearance 特性允许您设置文本在文本编辑器窗口中的显示方式。您可以启用自动换行、行号（对程序员十分有用）和当前的文件夹标记。借助 Fonts & Colors 特性，可以定制编辑器的完整颜色方案，确定程序代码中各类别的文本分别使用什么颜色。

### 7.3.2 Kate 编辑器

Kate 编辑器是 KDE 项目的旗舰级编辑器。它使用与 KWrite 应用程序相同的文本编辑器（因此大多数特性也是相同的），但它将许多其他特性整合到了一个包中。

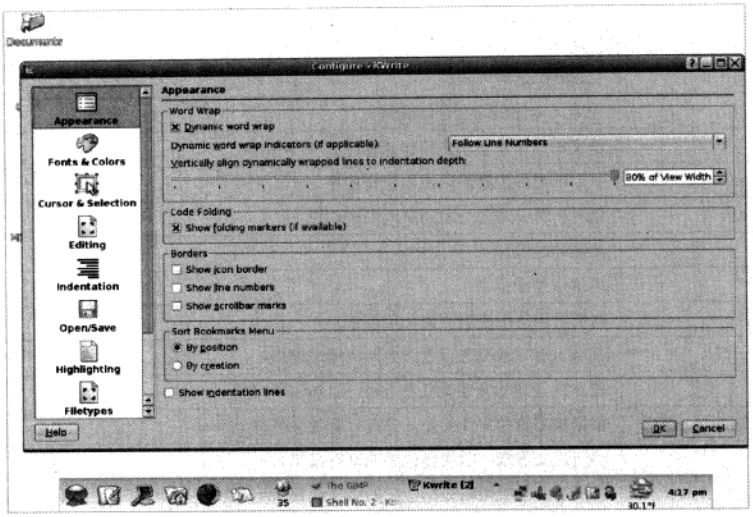



图 7-7　KWrite Configure Editor 对话框

启动 Kate 编辑器时注意到第一件事情就是，编辑器并未正常启动！相反，将看到一个对话框，如图 7-8 所示。

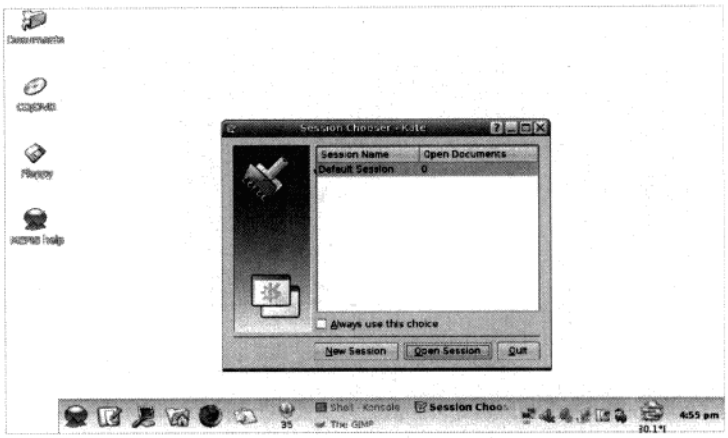



图 7-8　Kate 会话对话框

Kate 编辑器处理会话中的文件。您可以在一个会话中打开多个文件，并且可以保存多个会话。启动 Kate 时，它会询问您要返回哪个会话。选择 Kate 会话时，它会记住您曾经打开过的文档，并在下一次启动 Kate 时显示它们。

选择某个会话之后，将看到主 Kate 编辑器窗口，如图 7-9 所示。

左侧窗体显示了当前在会话中打开的文档。您可以通过单击文档名称在各文档之间切换。要编辑新文件，单击左侧的 Filesystem Browser 选项卡。左侧窗体现在已成为一个完整的图形文件系统浏览器，它允许您以图形方式浏览并定位到您的文件。

在 Kate 编辑器中，我最喜欢的一个特性就是它的内置终端窗口，如图 7-10 所示。

文本编辑器窗口底部的 Terminal 选项卡用于启动 Kate 的内置终端模拟器（使用 KDE Konsole 终端模拟器）。这特性将水平分隔当前编辑窗口，创建一个新窗口并在其中运行 Konsole。现在，您可以

输入命令行命令、启动程序或检查系统设置，而无需离开编辑器！要关闭终端窗口，只需在命令提示行中键入 exit 即可。

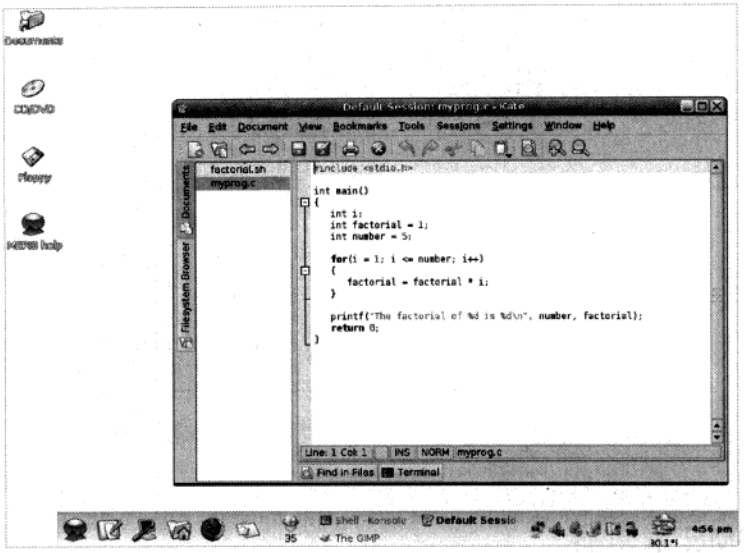



图 7-9 主 Kate 编辑窗口

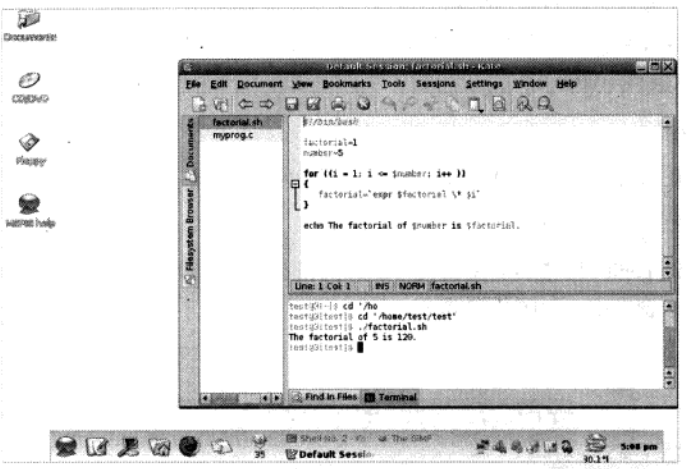



图 7-10 Kate 内置终端窗口

从终端特性中可以看出，Kate 还支持多个窗口。Window 菜单栏项目提供了以下选项：

- 使用当前会话创建一个新的 Kate 窗口；
- 垂直分隔当前窗口以创建一个新窗口；
- 水平分隔当前窗口以创建一个新窗口；
- 关闭当前窗口。

要在 Kate 中设置配置信息，选择 Settings 菜单栏项目下方的 Configure Kate 项。此时将打开 Configuration 对话框，如图 7-11 所示。

您会注意到 Editor 设置区域与 Kwrite 完全相同。这是因为两款编辑器共享相同的文本编辑器引擎。Application 设置区域允许您配置 Kate 项目的设置，例如控制会话（如图 7-11 所示）、文档列表和文件

系统浏览器。Kate 还支持外部插件应用程序，该选项也在此处激活。

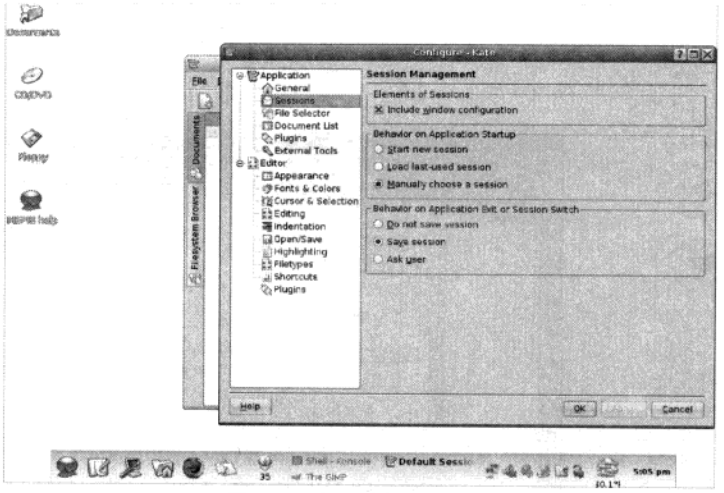



图 7-11　Kate 配置设置对话框

## 7.4　GNOME 编辑器

如果您的 Linux 系统使用 GNOME 桌面环境，那么它也提供了一款图形文本编辑器。gedit 文本编辑器是一款基本的文本编辑器，它附带少量高级特性供娱乐之用。本节介绍 gedit 编辑器的特性，并演示如何在 shell 脚本编程中使用它。

### 7.4.1　*启动 gedit*

大多数 GNOME 桌面环境都在 Accessories Panel 菜单项中包含了 gedit。如果未在其中找到 gedit，则可以通过命令行提示来启动它：

```
$ gedit factorial.sh myprog.c
```

以多个文件启动 gedit 时，它会将所有文件加载到相互独立的缓冲中，并将各文件显示在主编辑器窗口中的各个选项卡窗口中，如图 7-12 所示。

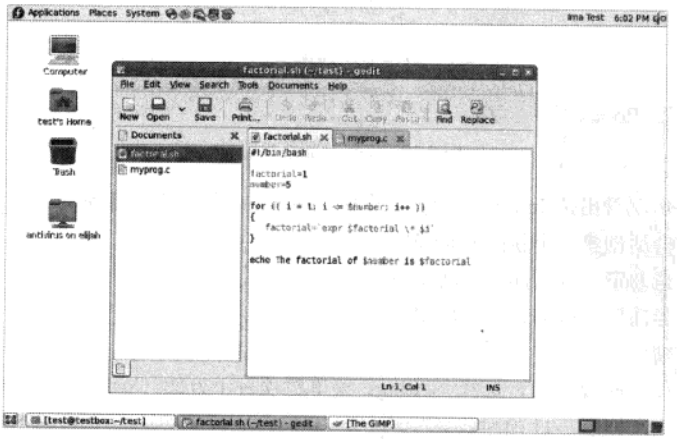



图 7-12　gedit 主编辑器窗口

gedit 主编辑器窗口左侧的窗体显示当前正在编辑的文档。右侧窗体显示包含缓冲文本的选项卡窗

口。如果将鼠标指标移动到各选项卡上，则会出现一个对话框，其中显示了该文件的完整路径名、MIME 类型和它使用的字符集编码。

### 7.4.2 基本 gedit 特性

除了编辑器窗口之外，还可以使用 gedit 提供的菜单栏和工具栏来设置特性和配置设置。工具栏提供了对菜单栏项的快速访问。可用的菜单栏项包括：

- File（文件）用于处理新文件、保存现有文件和打印文件。
- Edit（编辑）用于操作活动缓冲区中的文本，以及设置编辑器首选项。
- View（查看）用于设置在窗口中显示的编辑器特性，以及设置文本高亮模式。
- Search（搜索）用于查找和替换活动编辑器缓冲区中的文本。
- Tools（工具）用于访问 gedit 中安装的插件工具。
- Documents（文档）用于管理在缓冲区中打开的文件。
- Help（帮助）用于访问完整 gedit 手册。

这些内容不足以令人吃惊。File 菜单提供了一个 Open Location 选项，允许使用 WWW 世界中流行的标准统一资源标识符（URI）格式从网络中打开文件。此格式标识用于访问文件的协议（如 HTTP 或 FTP）、文件所在的服务器和该文件在服务器上的完整访问路径。

Edit 菜单包含标准的剪切、复制和粘贴功能，以及一个优异的特性，允许方便地在文本中输入不同格式的日期和时间。Search 菜单提供了一个标准的查找功能，可以在对话框中输入要查找的文本，以及选择查找方式的功能（匹配大小写、匹配整个单词和搜索方向）。它还提供了一个以实时模式运行的增量搜索特性，支持在输入字符的同时查找文本。

### 7.4.3 设置首选项

Edit 菜单包含一个 Preferences 项，它将调用 gedit Preferences 对话框，如图 7-13 所示。

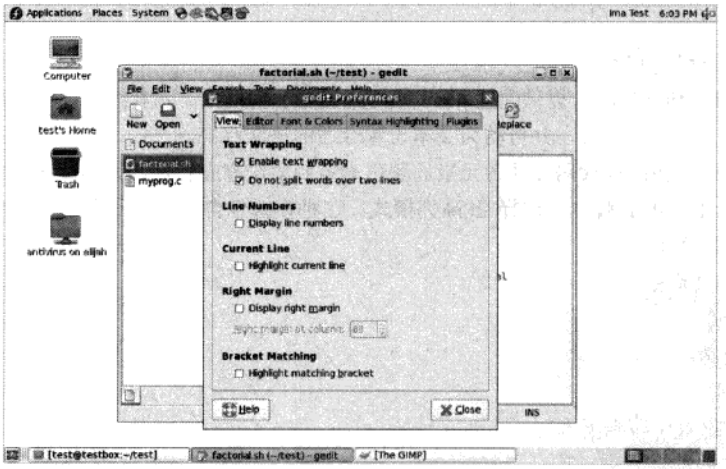



图 7-13 gedit Preferences 对话框

在该对话框中，可以定制 gedit 编辑器的操作。Preferences 对话框包含 5 个选项卡区，用于设置编辑器的特性和行为。

#### 1. View

View 选项卡提供 gedit 如何在编辑器窗口中显示文本的选项：

- Text Wrapping 确定如何处理编辑器中的一行较长的文本。Enabling text wrapping 选项将指定

行较长的文本换至编辑器的下一行。Do Not Split Words Over Two Lines 选项将防止在长单词中自动插入连字符，以防止它们被分离成两行。

- Line Numbers 在编辑器窗口最左侧显示行号。
- Current Line 高亮显示光标当前所在行，使您能够轻易找到光标位置。
- Right Margin 将启用右侧页边空白，允许您设置编辑器窗口中应显示的列数。默认值为 80 列。
- Bracket Matching，启用时，它将高亮显示编程代码中的括号对，允许您轻易匹配 `if-then` 语句、`for` 循环、`while` 循环和其他代码元素中的括号。

行编号和括号匹配特性为程序员提供了一个用于排除文本编辑器中不常出现代码的环境。

### 2. Editor

Editor 选项卡提供 gedit 编辑器如何处理制表符和缩进以及如何保存文件的选项：

- Tab Stops 设置按 Tab 键时跳过的空格数。默认值为 8。该特性还包含一个复选框，当被选中时，它将插入空格以代替制表符跳行操作。
- Automatic Indentation，启用时，gedit 将自动缩进文本中的行以表示段落和代码元素（如 `if-then` 语句和循环）。
- File Saving 提供两个用于保存文件的特性：在编辑窗口中打开时是否为文件创建一个备份副本，以及是否以预定间隔自动保存文件。

自动保存特性是确保更改以常规方式保存并防止崩溃或断电造成的灾难的有效方法。

### 3. Font&Colors

Font&Colors 选项卡允许您配置两个项目（不足为奇）：

- Font 允许选择 Monospace 10 的默认字体，或者从对话框中选择自定义字体和字体大小。
- Colors 允许选择文本、背景、选中文本和选择颜色使用的默认颜色方案，或者为各类别选择自定义颜色。

gedit 的默认颜色匹配为桌面选择的标准 GNOME 桌面方案。这些颜色将变化以匹配您为桌面选择的方案。

### 4. Syntax Highlighting

Syntax Highlighting 选项卡提供配置 gedit 如何高亮显示编程代码元素的选项。gedit 编辑器能够检测文本文件中包含的编程语言，并自动为文本设置合适的语法高亮。

您还可以通过选择编程代码文件各元素的颜色来定制语法高亮特性。元素将根据您选择的编程代码类型而变化。对于 shell 脚本，选择 sh 高亮模式。这种模式包含以下代码元素的颜色方案：

- 引号中的文本；
- 内置 shell 命令；
- 常用命令；
- shell 函数；
- 行注释；
- 多行字符串（两个版本）；
- shell 运算符；
- shell 标点；
- shell 重定向；
- Self；
- 两类字符串；
- 两类变量。

在选择 shell 脚本编辑环境时，这为您提供了足够的控制。您可以定制自己的工作区，以及设置喜

好的代码高亮颜色。

### 5. Plugins

Plugins 选项卡提供对 gedit 中使用的插件的控制。插件是可以与 gedit 接合以提供额外功能的单独程序。Plugins 选项卡如图 7-14 所示。

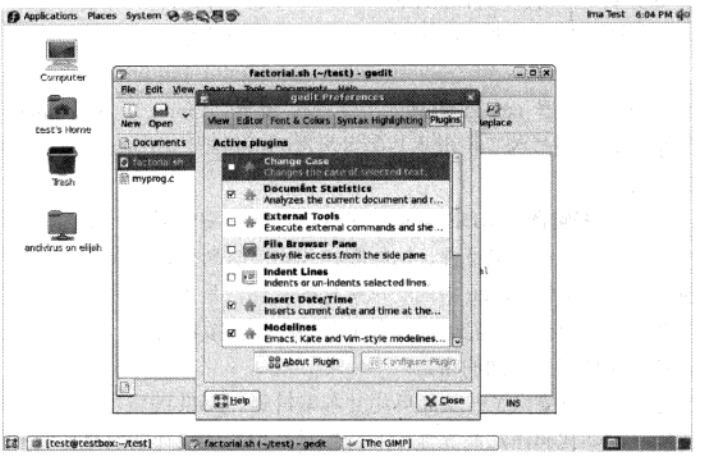



图 7-14 gedit Plugins Preferences 选项卡

gedit 中有若干种可用的插件，但并非所有插件都是默认安装的。表 7-4 介绍了 gedit 中当前可用的插件。

表 7-4 gedit 插件

| 插件 | 描述 |
|---|---|
| Change Case | 更改所选文本的大小写 |
| Document Statistics | 报告单词、行、字符和非空字符的数量 |
| External Tools | 在编辑器中提供一个 shell 环境，用于执行命令和脚本 |
| File Browser Pane | 提供一个简单的文件浏览器，便于选择要编辑的文件 |
| Indent Lines | 提供高级行缩进和不缩进功能 |
| Insert Date/Time | 在当前光标位置插入各种格式的当前日期和时间 |
| Modelines | 在编辑器窗口底部提供 emacs 样式的消息行 |
| Python Console | 在编辑器窗口底部提供一个交互式控制台，用于使用 Python 编程语言输入命令 |
| Snippets | 允许您保存经常使用的文本段，以方便在文本的任何地方检索 |
| Sort | 对整个文件或所选文本快速排序 |
| Spell Checker | 为文本文件提供字典拼写检查 |
| Tag List | 提供经常使用的字符串列表，您可以方便地在文本中输入它们 |
| User name | 在当前光标位置插入当前用户的登录名 |

已启用的插件将在它们的名称旁边显示一个选中标记。某些插件，例如 External Tools 插件，在您选择之后还提供了一些额外的配置特性。它允许您设置启动终端的快捷键，gedit 将在其中显示输出，以及用于启动 shell 会话的命令。

遗憾的是，并非所有插件都安装在 gedit 菜单栏的相同位置中。一些插件出现在 Tools 菜单栏项目中(如 Spell Checker 和 External Tools 插件)，而还有一些插件显示在 Edit 菜单栏项目中(如 Change Case 和 Insert Date/Time 插件)。

## 7.5　小结

创建 shell 脚本时，需要某种类型的文本编辑器。Linux 环境支持若干种流行的文本编辑器。Unix 世界中最流行的编辑器 vi 已经迁移到 Linux 世界中，即 vim 编辑器。vim 编辑器通过控制台提供了简单的文本编辑功能，它使用基本的全屏图形模式。vim 编辑器提供了许多高级编辑器特性，例如文本搜索和替换。

另一款流行的 Unix 编辑器 emacs 也进入了 Linux 世界。Linux 版本的 emacs 支持控制台和 X Windows 图形两种模式，从而成为了旧世界与新世界之间的桥梁。emacs 编辑器提供了多个缓冲区，允许您同时编辑多个文件。

KDE 项目创建了两个编辑器在 KDE 桌面中使用。KWrite 编辑器是一款简单的编辑器，它提供了基本的文本编辑特性，以及一些高级特性，比如编程代码语法高亮、行编号和代码折叠。Kate 编辑器为程序员提供了较多的高级特性。Kate 中的一项极佳的特性就是内置的终端窗口。您可以直接在 Kate 编辑器中打开一个命令行界面会话，而无需打开单独的终端模拟器窗口。Kate 编辑器还允许您打开多个文件，从而为各打开文件提供不同的窗口。

GNOME 项目还为程序员提供了一款简单的文本编辑器。gedit 编辑器是一款基本的文本编辑器，它提供一些高级特性，例如代码语法高亮显示和行编号，但它的设计初衷是作为一款基本编辑器。为了充实 gedit 编辑器，开发人员创建了一些插件，这些插件扩展了 gedit 中可用的特性。当前的插件包括一个拼写检查器、一个终端模拟器和一个文件浏览器。

至此，介绍如何使用 Linux 中的命令行的背景知识章节已经结束。本书的下一部分将探索 shell 脚本的世界。下一章将介绍如何创建 shell 脚本文件，以及如何在 Linux 系统上运行它。它还将介绍 shell 脚本的基础知识，允许您将多个命令串连成可以运行的脚本，从而创建简单的程序。

PART

2

# shell 脚本的基础知识

本部分内容：

第 8 章
基本脚本编译

第 9 章
使用结构化命令

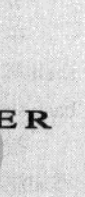
第 10 章
更多结构化命令

第 11 章
处理用户输入

第 12 章
显示数据

第 13 章
脚本控制

# 第 8 章
# 基本脚本编译




本章内容：

- 构建脚本；
- 将命令串连起来；
- 存储变量；
- 数学计算；
- 重定向输出；
- 检查代码。

我们已经介绍了 Linux 系统和命令行的基础知识，现在开始编写代码。本章讲述编写 shell 脚本的基础知识。在编写 shell 脚本之前需要了解这些基本概念。

## 8.1 使用多条命令

到目前为止，我们已经知道如何使用 shell 的命令行界面提示来输入命令和查看命令结果。shell 脚本的关键是可以输入多条命令，处理每一条命令的结果，甚至可以将一条命令的结果传递给另一条命令。shell 可以将多条命令连接起来形成单一的步骤。

如果想将两条命令一起运行，可以将它们输入到同一提示行中，并用分号将它们隔开：

```
$ date ; who
Mon Sep  24 19:44:35 EST 2007
rich     :0         2007-09-24 18:23 (console)
rich     pts/1      2007-09-24 18:24
rich     pts/0      2007-09-24 18:42
barbara  pts/2      2007-09-24 19:30
katie    pts/3      2007-09-24 19:39
$
```

祝贺您，您刚才编写了一个 shell 脚本！这个简单的脚本仅仅使用了两个 bash shell 命令。date 命令首先运行，显示当前的日期和时间，接下来是 who 命令的输出，显示当前登录到该系统的用户。采用这种技术，可以将任意多条命令连接在一起，命令行的最大字符数不超过 255 个字符。

虽然该技术适用于小脚本，但是它有一个主要缺点，即每次想要运行命令时，都必须在命令提示符处输入整条命令。相对于在命令行手动输入命令，可以将命令合并到一个简单的文本文件中。需要运行命令时，只需运行文本文件即可。

## 8.2 创建脚本文件

要将 shell 命令放置到一个文本文件中，首先需要使用文本编辑器（参阅第 7 章）创建一个文件，然后将命令输入到该文件中。

创建 shell 脚本文件时，必须在文件的第一行指明所使用的 shell。格式如下：

```
#!/bin/bash
```

在普通的 shell 脚本行中，英镑符号（#）用作注释行。shell 不处理 shell 脚本中的注释行。但是，shell 脚本文件的第一行是个特例，#后面跟着感叹号告诉 shell 运行下列脚本的 shell（是的，可以使用 bash shell，但用另一个 shell 运行脚本）。

指明 shell 以后，将命令输入到文件中的每一行，后面紧跟一个回车。如前所述，使用#符号可以添加注释。如本例所示：

```
#!/bin/bash
# This script displays the date and who's logged on
date
who
```

这就是脚本文件的所有内容。如果想将两条命令一起运行，可以将它们放在同一行，并用分号将其隔开，但在 shell 脚本中，可以分行列出每条命令。shell 将按照文件中命令的顺序处理命令。

另外要注意的是，我以英镑符号开头又添加了一行，并且添加了注释。shell 不会解析以#（以#!开头的第一行除外）开头的行。这是一个注释脚本内容的好方法，因此，两年后回顾时会很轻松地回忆起当时所做的工作。

将该脚本保存为 test1 文件，就几乎准备好了。之所以说“几乎”是因为在运行新 shell 脚本文件之前还有几件事情要做。

如果现在试图运行该文件，将会略有失望地看到如下内容：

```
$ test1
bash: test1: command not found
$
```

第一个要越过的障碍就是使 bash shell 找到脚本文件。如果您还记得第 5 章的内容，shell 使用名为 PATH 的环境变量查找命令。查看 PATH 环境变量将发现问题所在：

```
$ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin:/usr/X11R6/bin
$
```

PATH 环境变量设置为只可以在少数目录中查找命令。要使 shell 找到 test1 脚本，需要执行下列任意一种操作：

- 将 shell 脚本文件所在的目录添加到 PATH 环境变量中。
- 在命令行提示符中使用文件的绝对路径或相对路径来引用该 shell 脚本。

提示：有些 Linux 发行版将$HOME/bin 目录添加到 PATH 环境变量中。这就在每个用户的 HOME 目录下创建了一个放置文件的目录，通过这个目录 shell 可以找到要执行的文件。

本例使用第二种方法，将脚本文件的存储路径准确地告诉 shell。记住，要引用当前目录中的文件，可以在 shell 中使用单点操作符：

```
$ ./test1
bash: ./test1: Permission denied
$
```

现在，shell 轻松地找到了 shell 脚本文件，但还有一个问题。shell 指出我没有执行该文件的权限。快速查看该文件的权限可以显示这里遇到的问题：

```
$ ls -l test1
-rw-r--r--    1 rich        rich          73 Sep 24 19:56 test1
$
```

创建新的 test1 文件时，umask 值确定了新文件的默认权限设置。由于 umask 设为 022（参阅第 6 章），系统以只读和只写的权限创建了该文件。

下一步就是赋予自己执行该文件的权限，使用 chmod 命令（参阅第 6 章）：

```
$ chmod u+x test1
```

```
$ ./test1
Mon Sep  24 19:58:35 EST 2007
rich     :0       2007-09-24 18:23 (console)
rich     pts/1    2007-09-24 18:24
rich     pts/0    2007-09-24 18:42
barbara  pts/2    2007-09-24 19:30
katie    pts/3    2007-09-24 19:39
$
```

成功！现在具备了所有的条件，可以执行这个新 shell 脚本文件了。

## 8.3 显示消息

大多数 shell 命令生成自己的输出，这些输出在运行该脚本的控制台监视器上显示。有时候想添加自己的文本消息以帮助脚本用户了解脚本中发生了什么。使用 echo 命令可以完成此操作。如果在 echo 命令后添加字符串，echo 命令就能显示一个简单的文本字符串：

```
$ echo This is a test
This is a test
$
```

注意，默认情况下不需要使用引号来标记想要显示的字符串。但如果在字符串中使用了引号，有时就会出现问题：

```
$ echo Let's see if this'll work
Lets see if thisll work
$
```

echo 命令既可以用双引号也可以用单引号来标记文本字符串。如果要在字符串中使用它们，需要在文本使用一种引号类型，然后用另一种类型标记字符串：

```
$ echo "This is a test to see if you're paying attention"
This is a test to see if you're paying attention
$ echo 'Rich says "scripting is easy".'
Rich says "scripting is easy".
$
```

现在所有的引号都正确地显示在输出结果中。

在 shell 脚本中需要显示额外信息的任何位置都可以添加 echo 语句：

```
$ cat test1
#!/bin/bash
# This script displays the date and who's logged on
echo The time and date are:
date
echo "Let's see who's logged into the system:"
who
$
```

运行脚本，产生输出结果：

```
$ chmod u+x test1
$ ./test1
The time and date are:
Mon Sep 24 20:09:35 EST 2007
Let's see who's logged into the system:
rich     :0       2007-09-24 18:23 (console)
rich     pts/1    2007-09-24 18:24
rich     pts/0    2007-09-24 18:42
barbara  pts/2    2007-09-24 19:30
katie    pts/3    2007-09-24 19:39
$
```

很好，但是如果想要 echo 命令结果和 echo 文本字符串在同一行，那该如何做呢？只需对 echo 语句使用-n 参数即可。现在把第一个 echo 语句行改成：

```
echo -n "The time and date are: "
```

需要用引号包围字符串，同时确保在该字符串末尾有个空格。命令的输出正好从字符串输出停止的位置开始。输出结果如下：

```
$ ./test1
The time and date are: Mon Sep 24 20:11:35 EST 2007
Let's see who's logged into the system:
rich     :0       2007-09-24 18:23 (console)
rich     pts/1    2007-09-24 18:24
rich     pts/0    2007-09-24 18:42
barbara  pts/2    2007-09-24 19:30
katie    pts/3    2007-09-24 19:39
$
```

很好！echo 命令是与用户交互的 shell 脚本关键部分。您可以发现在许多情况下都要使用它，尤其是想显示脚本变量的值时。让我们看看下面的内容。

## 8.4 使用变量

从 shell 脚本中运行单独的命令很有用，但是它有局限性。通常想要在 shell 命令中结合其他数据以处理信息，为此可以使用**变量**。变量允许在 shell 脚本中暂存信息，以便与脚本中其他命令一起使用。本节将展示在 shell 脚本中如何使用变量。

### 8.4.1 环境变量

您已经看到了一种 Linux 变量如何运行。第 5 章描述了 Linux 系统中可用的环境变量。也可以从 shell 脚本中访问这些变量的值。

shell 维护跟踪特定系统信息的环境变量，这些系统信息包括系统的名称、登录到系统中的用户名、用户的系统 ID（称为 UID）、用户的默认根目录，以及 shell 查找程序使用的搜索路径。通过使用 set 命令，可以显示活动环境变量的完整列表：

```
$ set
BACKSPACE=Delete
BASH=/bin/bash
EUID=1000
HISTFILE=/home/rich/.bash_history
HISTFILESIZE=1000
HISTSIZE=1000
HOME=/home/rich
HOSTNAME=testing
HOSTTYPE=i586
LANG=en
LANGUAGE=en_US:en
LINES=24
LOGNAME=rich
...
```

通过使用以美元符号开头的环境变量可以从脚本中引进这些环境变量。下面的脚本展示了这一点：

```
$ cat test2
#!/bin/bash
# display user information from the system.
echo "User info for userid: $USER"
echo UID: $UID
echo HOME: $HOME
$
```

环境变量$USER、$UID 和$HOME 用来显示关于登录用户的相关信息。输出结果如下所示：

```
$chmod u+x test2
$ ./test2
User info for userid: rich
UID: 501
```

```
HOME: /home/rich
$
```

注意，当脚本运行时，echo 命令中的环境变量被它们的当前值所替代。还要注意的是，可以在第一个字符串中将系统变量$USER 放在双引号内，shell 脚本仍然能够明白我们的意图。但是使用这种方法有一个缺陷。看看下面的例子会发生什么：

```
$ echo "The cost of the item is $15"
The cost of the item is 5
```

这显然不是我们想要的。无论何时当脚本在引号内发现英镑符号时，它就会认为您引用了一个变量。在本例中，脚本想要显示变量$1 (没有定义过)，然后是数字 5。要显示一个实际的美元符号，必须在它的前面加上反斜杠字符:

```
$ echo "The cost of the item is \$15"
The cost of the item is $15
```

这样就好多了。反斜杠可以使 shell 脚本将美元符号翻译为实际的美元符号，而不是一个变量。下一节将讲述如何在脚本中创建自己的变量。

注意：您可能也看到过使用${variable}格式来引用变量。包围变量名的大括号通常用来将变量名与美元符号区分开来。

## 8.4.2 用户变量

除了环境变量，shell 脚本允许在脚本中设置和使用自己的变量。设置变量可以暂时存储数据并在脚本中使用它们，这使 shell 脚本更像是一个真正的计算机程序。

用户变量可以是由不超过 20 个字符的字母、数字或下划线组成的文本字符串。用户变量区分大小写，所以变量 Var1 与变量 var1 是不同的。这个小的规则常常给脚本程序员新手带来困扰。

值通过等号赋值给用户变量。在变量、等号和变量值之间不允许有空格(初学者的另一个困惑点)。下面是给用户变量赋值的几个例子：

```
var1=10
var2=-57
var3=testing
var4="still more testing"
```

shell 脚本自动为变量值确定数据类型。脚本中定义的变量在 shell 脚本的生命周期内保留它们的值，但是当 shell 脚本完成时就被删除了。

像系统变量一样，用户变量也可以使用美元符号引用：

```
$ cat test3
#!/bin/bash
# testing variables
days=10
guest="Katie"
echo "$guest checked in $days days ago"
days=5
guest="Jessica"
echo "$guest checked in $days days ago"
$
```

运行脚本产生如下输出结果：

```
$ chmod u+x test3
$ ./test3
Katie checked in 10 days ago
Jessica checked in 5 days ago
$
```

每次引用变量时，它会生成当前赋给它的那个值。有一点必须记住，即用美元符号引用变量值，但是当引用变量给它赋值时，不能使用美元符号。下面是一个例子：

```
$ cat test4
#!/bin/bash
```

```
# assigning a variable value to another variable

value1=10
value2=$value1
echo The resulting value is $value2
$
```

当在赋值语句中使用变量 value1 的值时，必须仍使用美元符号。此代码产生的输出如下所示：

```
$ chmod u+x test4
$ ./test4
The resulting value is 10
$
```

如果忘了使用美元符号，value2 赋值行如下所示：

```
Value2=value1
```

得到以下输出结果：

```
$ ./test4
The resulting value is value1
$
```

没有美元符号，shell 就会把变量名理解为一个普通的文本字符串，这很有可能不是您想要的。

### 8.4.3 反引号

shell 脚本一个最有用的功能是普通的反引号字符，在 Linux 世界中通常称为反引号（`）。注意，这与用于字符串的普通单引号字符不同。因为在 shell 脚本之外很少用到它，所以您甚至都不知道它在键盘的位置。您应该熟悉它，因为它是许多 shell 脚本的关键组成部分（提示：在美式键盘上，它通常与波浪号（～）在同一个键上）。

反引号允许将 shell 命令的输出赋值给变量。尽管这看起来好像意义不大，但是在脚本编程中，它是主要的构件。

必须将整个命令行命令用反引号包围起来：

```
testing=`date`
```

shell 运行带有反引号符号的命令并把结果赋值给变量 testing。这里是使用标准 shell 命令的输出创建变量的一个例子：

```
$ cat test5
#!/bin/bash
# using the backtick character
testing=`date`
echo "The date and time are: " $testing
$
```

变量 testing 接收到 date 命令的输出，并且用 echo 语句来显示它。运行 shell 脚本生成下列输出：

```
$ chmod u+x test5
$ ./test5
The date and time are: Mon Sep 24 20:23:25 EDT 2007
$
```

这还不是这个例子中最令人兴奋的（可以简单地把命令置于 echo 语句中），但是用变量捕获到命令输出后，就可以用它做任何事情了。

这里有一个常用的例子，说明如何用反引号来捕获当前日期，并用它在脚本中创建唯一的文件名：

```
#!/bin/bash
# copy the /usr/bin directory listing to a log file
today=`date +%y%m%d`
ls /usr/bin -al > log.$today
```

格式化 date 命令的输出结果赋值给了 today 变量。这是一种为了日志文件名提取日期信息的常用技术。+%y%m%d 格式指示 date 命令使用两位数来显示年、月、日。

```
$ date +%y%m%d
```

```
070922
$
```

脚本为变量赋值，然后变量被用作文件名的一部分。文件本身包含目录列表的重定向输出（将在后面的“重定向输出”一节中讨论）。运行脚本之后，就能够在目录下看到一个新的文件：

```
-rw-r--r--    1 rich        rich          769 Sep 22 10:15 log.070922
```

该日志文件以$today 变量的值作为部分文件名出现在目录中。日志文件的内容是目录/user/bin 的目录列表。如果脚本文件第二天运行，日志文件名将是 log.070923，这样新的一天创建一个新的文件。

## 8.5 重定向输入输出

有时想通过命令保存输出，而不是只在显示器上显示输出。bash shell 提供了几种操作符，能够将命令的输出**重定向**到另一个位置（如一个文件）。重定向既能用于输入也能用于输出，重定向一个文件到命令输入。本节介绍在脚本中使用重定向需要些什么。

### 8.5.1 输出重定向

重定向的最基本类型是通过一条命令将输出发送到文件中。bash shell 重定向为此使用大于号：

```
Command > outputfile
```

相反，任何命令中应该显示在显示器上的内容都存储到了一个指定的输出文件中：

```
$ date > test6
$ ls -l test6
-rw-r--r--    1 rich        rich          29 Sep 24 17:56 test6
$ cat test6
Tue Sep 24 17:56:58 EDT 2007
$
```

重定向操作符创建了文件 test6（使用了默认的 umask 设置），并且将 date 命令的输出重定向到文件 test6。如果输出文件已经存在，那么重定向操作符就会用新的文件数据重写已经存在的文件：

```
$ who > test6
$ cat test6
rich     pts/0    Sep 24 17:55
$
```

现在，文件 test6 的内容包含了 who 命令的输出。

有时可能需要将命令的输出附加到现有文件，而不是重写此文件的内容。例如，如果正在创建记录系统中活动的日志文件。在这种情况下，可以用两个大于号（>>）附加数据：

```
$ date >> test6
$ cat test6
rich     pts/0    Sep 24 17:55
Tue Sep 24 18:02:14 EDT 2007
$
```

文件 test6 仍然包含先前 who 命令处理的原始数据，现在还包含了来自于 date 命令的新输出。

### 8.5.2 输入重定向

输入重定向是输出重定向的相反过程。输入重定向不是将命令的输出重定向到一个文件中，而是将一个文件的内容重定向到一条命令中。

输入重定向符号是小于号（<）：

```
command < inputfile
```

记住这一点的简单方法是命令总是位于在命令行的开始位置，重定向符号指向数据流动的方向。小于号表明数据从输入文件流向命令。

这里是一个使用 wc 命令的输入重定向示例：

```
$ wc < test6
      2      11      60
$
```

wc 命令对数据中的文本计数。默认情况下它生成 3 个值：

- 文本的行数；
- 文本的单词数；
- 文本的字节数。

通过将文本文件重定向到 wc 命令中，很快就会获得文件的行数、单词数和字节数。上面的例子显示文件 test6 有两行、11 个单词和 60 个字节。

输入重定向还有另一种方法，称为**内置输入重定向**（inline input redirection）。这种方法允许在命令行中而非文件中为输入重定向指定数据。起初看起来有些奇怪，但是有一些这样的应用（例如后面“数学计算”一节中介绍的应用程序）。

内置输入重定向符号是两个小于号（<<）。除了这个符号，还必须指定一个文本标记来说明输入数据的开始和结尾。文本标记可以使用任何字符串值，但是在数据的开始和结尾处必须相同：

```
command << marker
data
marker
```

在命令行中使用内置重定向时，shell 会采用次提示来提示数据，次提示在 **PS2** 环境变量（参阅第 5 章）中定义。使用时命令如下：

```
$ wc << EOF
> test string 1
> test string 2
> test string 3
> EOF
      3       9      42
$
```

次提示会为更多的数据继续提示直到输入文本标记。wc 命令计算内置输入重定向提供的数据的行、单词、字节的数目。

## 8.6 管道

有时需要将一个命令的输出发送至另一个命令的输入。使用重定向可以做到这一点，但是这有些笨拙：

```
$ rpm -qa > rpm.list
$ sort < rpm.list
a2ps-4.13b-65.fc7
acl-2.2.39-3.1.fc7
alacarte-0.11.3-3.fc7
alsa-lib-1.0.14-3.fc7
alsa-utils-1.0.14-1.fc7
anacron-2.3-47.fc7
apr-1.2.8-6
apr-util-1.2.8-7
aspell-0.60.5-3.fc7
aspell-en-6.0-7.fc7
at-3.1.10-13.fc7
atk-1.18.0-1.fc7
at-spi-1.18.1-1.fc7
...
```

rpm 命令用 Red Hat 软件包管理系统（RPM）（如所示的 Fedora 系统）管理安装在系统中的软件包。当与参数-qa 一起使用时，它会生成已安装软件包的列表，但是未必以特定的顺序显示。如果您正在寻找一个特定的软件包或软件包的组，使用 rpm 命令的输出很难找到。

使用标准的输出重定向，能够将 rpm 命令的输出重定向到 rpm.list 的文件。命令完成后，rpm.list 文件包含系统中所有安装的软件包列表。接下来，使用输入重定向将 rpm.list 文件的内容发送到 sort 命令，按软件包名称的字母顺序排列软件包。

这种方法有用，但却是一种生成信息的比较笨拙的方法。可以将输出重定向到另一条命令，而不是将命令的输出重定向到一个文件。这个过程称为**管道传送**。管道的符号是竖条操作符（|）：

```
command1 | command2
```

管道传送链接命令以提供更详细的输出。不要认为管道传送是一个一个地运行两个命令。Linux 系统实际上同时运行两条命令，并在系统内部把它们连接在一起。第一条命令生成输出时，输出就立即发送给第二条命令。没有使用中间文件或者缓冲区来传递数据。

现在，使用管道就可以很轻松地将 rpm 命令的输出直接传送给 sort 命令，产生的结果如下所示：

```
$ rpm -qa | sort
a2ps-4.13b-65.fc7
acl-2.2.39-3.1.fc7
alacarte-0.11.3-3.fc7
alsa-lib-1.0.14-3.fc7
alsa-utils-1.0.14-1.fc7
anacron-2.3-47.fc7
apr-1.2.8-6
apr-util-1.2.8-7
aspell-0.60.5-3.fc7
aspell-en-6.0-7.fc7
at-3.1.10-13.fc7
atk-1.18.0-1.fc7
at-spi-1.18.1-1.fc7
...
```

除非您能快速阅览，否则很有可能跟不上此命令生成的输出。由于管道功能操作的实时性，rpm 命令一产生数据，sort 命令就立刻进行排序。当 rpm 命令完成输出数据时，sort 命令也排序完成并将结果显示在显示器上。

命令中可以使用数量不限的管道数（行长度最多是 255 个字符）。可以连续将命令的输出传送给其他命令以改进操作。

在上例中，由于 sort 命令的输出急剧增大，可以使用一种文本分页命令（如 less 或 more 命令），将输出强制停留在每一屏的数据上：

```
$ rpm -qa | sort | more
```

此指令序列运行 rpm 命令，将输出管道传送给 sort 命令，然后将输出再管道传送给 more 命令去显示数据，显示一屏信息停止一下。这就在继续之前允许暂停并读取显示的内容，如图 8-1 所示。

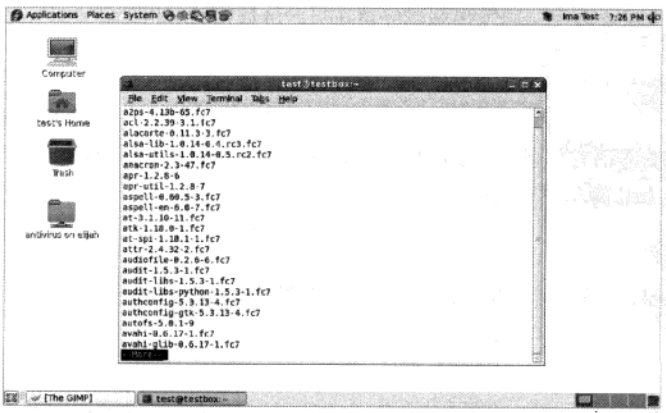



图 8-1 使用管道传送将数据传送给 more 命令

要想做得更漂亮，可以将重定向和管道传送结合使用，以将输出保存到文件中：

```
$ rpm -qa | sort > rpm.list
$more rpm.list
ammonite-1.0.0-1mdk
anacron-2.3-7mdk
apache-1.3.19-3mdk
apache-common-1.3.19-3mdk
apache-conf-1.3.19-3mdk
apache-mod_perl-1.3.19_1.25-3mdk
apache-modules-1.3.19-3mdk
apache-suexec-1.3.19-3mdk
arts-2.1.1-7mdk
ash-0.2-24mdk
aspell-0.32.5-4mdk
aspell-en-0.32.5-4mdk
at-3.1.7-21mdk
...
```

正如预料的那样，文件 `rpm.list` 中的数据已经进行了排序!

到目前为止，管道的一个最受欢迎的用法是用管道传送命令的结果，这些命令产生出长输出给 `more` 命令。这对 `ls` 命令尤其常见，如图 8-2 所示：

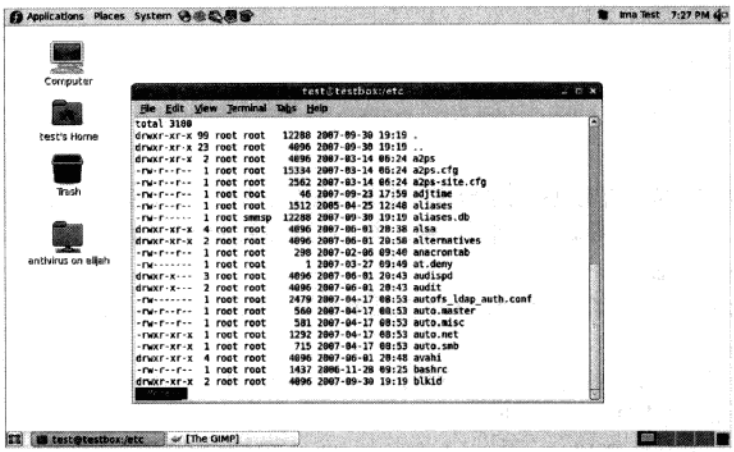



图 8-2 与 ls 命令一起使用 more 命令

`ls -1` 命令产生目录下所有文件的列表。对于含有很多文件的目录来说，这个列表相当长。通过把输出管道传送给 `more` 命令，将输出强制停留在每一屏数据的结束处。

## 8.7 数学计算

另一个对任何编程语言都至关重要的功能是处理数字的能力。不幸的是，对于 shell 脚本来说，这个过程有点笨拙。shell 脚本中有两种不同的方法执行数学计算。

### 8.7.1 expr 命令

最初，Bourne shell 提供一种专有命令来计算数学等式。`expr` 命令允许处理命令行中的等式，但是很笨拙：

```
$ expr 1+5
6
```

`expr` 命令能够区分一些不同的数学操作符和字符串操作符，如表 8-1 所示。

表 8-1 expr 命令操作符

| 操作符 | 描述 |
|---|---|
| *ARG1 \| ARG2* | 如果两个参数都不为空或都不为 0，返回 *AEG1*；否则，返回 *ARG2* |
| *ARG1&ARG2* | 如果两个参数都不为空或都不为 0，返回 *AEG1*；否则，返回 0 |
| *ARG1<ARG2* | 如果 *ARG1* 小于 *ARG2*，返回 1；否则，返回 0 |
| *ARG1<=ARG2* | 如果 *ARG1* 小于等于 *ARG2*，返回 1；否则，返回 0 |
| *ARG1=ARG2* | 如果 *ARG1* 等于 *ARG2*，返回 1；否则，返回 0 |
| *ARG1!=ARG2* | 如果 *ARG1* 不等于 *ARG2*，返回 1；否则，返回 0 |
| *ARG1>=ARG2* | 如果 *ARG1* 大于等于 *ARG2*，返回 1；否则，返回 0 |
| *ARG1>ARG2* | 如果 *ARG1* 大于 *ARG2*，返回 1；否则，返回 0 |
| *ARG1+ARG2* | 返回 *ARG1* 与 *ARG2* 的和 |
| *ARG1-ARG2* | 返回 *ARG1* 与 *ARG2* 的差 |
| *ARG1*ARG2* | 返回 *ARG1* 与 *ARG2* 的乘积 |
| *ARG1/ARG2* | 返回 *ARG1* 除以 *ARG2* 的商 |
| *ARG1%ARG2* | 返回 *ARG1* 除以 *ARG2* 的余数 |
| *STRING:REGEXP* | 如果 *REGEXP* 匹配 *STRING* 中的一个模式，返回该模式 |
| match *STRING REGEXP* | 如果 *REGEXP* 匹配 *STRING* 中的一个模式，返回该模式 |
| substr *STRING POS LENGTH* | 从 *POS* 位置起始（始于 1），返回长度为 *LENGTH* 的字符 |
| index *STRING CHARS* | 返回在 *STRING* 中找到 *CHARS* 的位置，否则返回 0 |
| length *STRING* | 返回字符串 *STRING* 的长度 |
| + *TOKEN* | 将 *TOKEN* 解释为一个字符串，即使它是一个关键字 |
| (*EXPRESSION*) | 返回 *EXPRESSION* 的值 |

在脚本或命令行中使用标准操作符会出现问题，尽管在 expr 命令中它们可以正常运行。shell 中许多 expr 命令操作符（如星号）有其他含义。在 expr 命令中使用它们会生成一些奇怪的结果：

```
$ expr 5 * 2
expr: syntax error
$
```

要解决此问题，在传送给 expr 命令的字符可能被错误解析前，需要使用 shell 转义字符（反斜杠）来识别它们：

```
$ expr 5 \* 2
10
$
```

现在将变得很难看！在 shell 脚本中使用 expr 命令也很麻烦：

```
#!/bin/bash
# An example of using the expr command
var1=10
var2=20
var3='expr $var2 / $var1'
echo The result is $var3
```

为了把一个数学等式的结果赋值给一个变量，必须使用字符反引号从 expr 命令中提取输出：

```
$ ./test6
The result is 2
$
```

幸运的是，bash shell 对处理数学计算符有改进措施。

## 8.7.2 使用括号

bash shell 包含 expr 命令，使其与 Bourne shell 保持兼容。然而，它也提供了一种计算数学等式更简单的方法。在 bash 中，为一个变量指定一个数学值时，可以用美元符号和方括号（$[operation]）把数学等式括起来：

```
$ var1=$[1 + 5]
$ echo $var1
```

```
6
$ var2 = $[$var1 * 2]
$ echo $var2
12
$
```

与使用 expr 命令相比，使用括号使 shell 数学运算变得更简单。此技巧同样使用于 shell 脚本：

```
$ cat test7
#!/bin/bash
var1=100
var2=50
var3=45
var4=$[$var1 * ($var2 - $var3)]
echo The final result is $var4
$
```

运行此脚本生成如下输出：

```
$ chmod u+x test7
$ ./test7
The final result is 500
$
```

还要注意的是，使用方括号方法计算等式时，不必担心 shell 错误理解乘法符号或者其他符号。shell 知道它不是一个通配符，因为它在方括号里面。

在 bash shell 脚本中计算数学等式有一个主要限制。看看这个例子：

```
$ cat test8
#!/bin/bash
var1=100
var2=45
var3=$[$var1 / $var2]
echo The final result is $var3
$
```

现在运行它并查看结果：

```
$ chmod u+x test8
$ ./test8
The final result is 2
$
```

Bash shell 的数学操作只支持整数算法。如果想做任何类型的实际的计算，这是一个很大的限制。

注意：Z shell（zsh）支持浮点算术运算。如果在 shell 脚本中需要浮点计算，可以考虑使用 z shell（在第 23 章中介绍）。

### 8.7.3 浮点解决方案

有几种克服 bash 整数限制的解决方案。最普遍的解决方法是使用内置的 bash 计算器（称为 bc）。

**1. bc 的基本知识**

bash 计算器实际上是一种编程语言，该语言允许在命令行中输入浮点表达式，然后解释表达式并计算它们，最后返回结果。bash 计算器可以识别：

- 数字（整数和浮点）；
- 变量（简单变量和数组）；
- 注释（以英镑符号开始的行或 C 语言的/* */对）；
- 表达式；
- 编程语句（如 if-then 语句）；
- 函数。

可以使用 bc 命令从 shell 提示符访问 bash 计算器：

```
$ bc
bc 1.06
```

```
Copyright 1991-1994, 1997, 1998, 2000 Free Software Foundation, Inc.
This is free software with ABSOLUTELY NO WARRANTY.
For details type 'warranty'.
12 * 5.4
64.8
3.156 * (3 + 5)
25.248
quit
$
```

这个例子以输入表达式 12*5.4 开始。bash 计算器返回结果。计算器对每一个后续键入的表达式都进行计算并显示结果。要退出 bash 计算器，必须输入 quit。

浮点算术被称为 scale 的内置变量控制。必须把这个值设置为想要的十进制小数位数，否则得不到想要的结果：

```
$ bc -q
3.44 / 5
0
scale=4
3.44 / 5
.6880
quit
$
```

scale 变量的默认值是 0。在设置 scale 值之前，bash 计算器提供 0 小数位的答案。将 scale 变量值设为 4 之后，bash 计算器显示为 4 位小数位的答案。-q 命令行参数禁止 bash 计算器的冗余欢迎标语。

除了普通的数字，bash 计算器也识别变量：

```
$ bc -q
var1=10
var1 * 4
40
var2 = var1 / 5
print var2
2
quit
$
```

一旦定义了变量值，可以在整个 bash 计算器会话中使用该变量。print 语句允许打印变量和数字。

**2. 在脚本中使用 bc**

现在您可能想知道在 shell 脚本中，bash 计算器如何帮助我们进行浮点计算。还记得反引号字符吗？是的，可以使用反引号运行 bc 命令，并且把输出赋值给一个变量！使用的基本格式为：

```
variable='echo "options; expression" | bc'
```

第一部分 options 允许设置变量。如果需要设置多个变量，使用分号将它们隔开。expression 参数定义了使用 bc 计算的数学表达式。这看起来有点古怪，但它运行起来很快。下面是一个在脚本中运用的例子：

```
$ cat test9
#!/bin/bash
var1='echo " scale=4; 3.44 / 5" | bc'
echo The answer is $var1
$
```

这个例子设置了 scale 变量为 4 位小数，然后为表达式指定了一种特定的计算。运行此脚本生成如下输出：

```
$ chmod u+x test9
$ ./test9
The answer is .6880
$
```

现在好了，不会再受限于仅在表达式中使用数字了。可以使用在 shell 脚本中定义的变量：

```
$ cat test10
#!/bin/bash
var1=100
var2=45
var3='echo "scale=4; $var1 / $var2" | bc'
echo The answer for this is $var3
$
```

这个脚本定义了两个变量，这两个变量用在发送给 bc 命令的表达式中。记住使用美元符号表示变量的值，而不是变量本身。该脚本的输出结果是：

```
$ ./test10
The answer for this is 2.2222
$
```

当然，变量被赋值后，该变量就能用于其他计算：

```
$ cat test11
#!/bin/bash
var1=20
var2=3.14159
var3='echo "scale=4; $var1 * $var1" | bc'
var4='echo "scale=4; $var3 * $var2" | bc'
echo The final result is $var4
$
```

对于简短的计算这种方法很好，但是有时数字会很多。如果有多个计算，试图在同一行列出多个表达式，就会出现混乱。

这个问题有一个简单的解决方法。bc 命令能够识别输入重定向，允许把一个文件重定向到 bc 命令中处理。然而，这同样容易混淆，因为需要在文件中存储表达式。

还可以使用内置输入重定向方法，而不使用文件重定向。内置输入重定向允许从命令行直接重定向数据。在 shell 脚本中，可以把输出赋值给变量。如下所示：

```
variable='bc << EOF
options
statements
expressions
EOF
'
```

EOF 文本字符串表明内置重定向数据的开始和末尾。记住仍然需要反引号字符把 bc 命令的输出结果赋值给变量。

现在可以把所有单独的 bash 计算器元素放到脚本的单独行中。这是一个在脚本中使用该技术的例子：

```
$ cat test12
#!/bin/bash
var1=10.46
var2=43.67
var3=33.2
var4=71

var5='bc << EOF
scale = 4
a1 = ( $var1 * $var2)
b1 = ($var3 * $var4)
a1 + b1
EOF
'

echo The final answer for this mess is $var5
$
```

使用该技术，可以在脚本中把每个操作和表达式放在单独的行中，使其更加清晰并便于阅读。EOF 字符串标明要重定向到 bc 命令的数据的开始和结束。当然，需要使用反引号字符来标明要赋值给变量的命令。

在本例中，您会注意到在 bash 计算器中可以给变量赋值。重要的是要记住，在 bash 计算器内创建的变量只在 bash 计算器内有效，而不能用在 shell 脚本中。

## 8.8 退出脚本

至今，我们的示例脚本中，命令终止得都很突然。执行最后一条命令时，我们只是结束了脚本。我们还可以用一种更优雅的方式来完成。

为了向 shell 表明，命令已经处理完毕，每条运行在 shell 中的命令都使用一个**退出状态**。这个退出状态是一个介于 0 和 255 之间的整数值，当命令运行完成时，命令就会把退出状态传递给 shell。可以捕捉这个值并在您的脚本中使用它。

### 8.8.1 核对退出状态

Linux 提供$?特殊变量来保存最后一条命令执行结束的退出状态。如果想核对一条命令的退出状态，必须在这条命令运行完成之后立即查看或使用变量$?。它会改变为 shell 执行的最后一条命令的退出状态值：

```
$ date
Sat Sep 29 10:01:30 EDT 2007
$ echo $?
0
$
```

按照惯例，一条命令成功完成的退出状态是 0。如果命令执行错误，那么退出状态就会是一个正整数：

```
$ asdfg
-bash: asdfg: command not found
$ echo $?
127
$
```

无效命令返回的退出状态值为 127。对于 Linux 的错误退出状态码没有标准约定。但是有些准则可以使用，如表 8-2 所示。

表 8-2 Linux 退出状态代码

| 代码 | 描述 | 代码 | 描述 |
|---|---|---|---|
| 0 | 命令成功完成 | 128 | 无效的退出参数 |
| 1 | 通常的未知错误 | 128+x | 使用 Linux 信号 x 的致命错误 |
| 2 | 误用 shell 命令 | 130 | 使用 Ctrl –C 终止的命令 |
| 126 | 命令无法执行 | 255 | 规范外的退出状态 |
| 127 | 没有找到命令 | | |

退出状态值为 126 表明用户没有权限执行这条命令：

```
$ ./myprog.c
-bash: ./myprog.c: Permission denied
$ echo $?
126
$
```

如果为命令提供一个无效的参数，就会遇到另一个常见错误：

```
$ date %t
date: invalid date '%t'
$ echo $?
1
$
```

这产生普通的退出状态代码 1，表明在命令中出现了未知的错误。

## 8.8.2 退出命令

默认情况下，您的 shell 脚本以脚本中最后一条命令的退出状态退出：

```
$ ./test6
The result is 2
$ echo $?
0
$
```

可以把它改为返回您自己的退出状态代码。exit 命令允许在脚本结束时，指定一个退出状态：

```
$ cat test13
#!/bin/bash
# testing the exit status
var1=10
var2=30
var3=$[ $var1 + var2 ]
echo The answer is $var3
exit 5
$
```

检查脚本的退出状态时，会得到作为 exit 命令参数的参数值：

```
$ chmod u+x test13
$ ./test13
The answer is 40
$ echo $?
5
$
```

也可以在 exit 命令参数中使用变量：

```
$ cat test14
#!/bin/bash
# testing the exit status
var1=10
var2=30
var3=$[ $var1 + var2 ]
exit $var3
$
```

运行该命令时，生成如下退出状态：

```
$ chmod u+x test14
$ ./test14
$ echo $?
40
$
```

应该谨慎使用这个功能，因为退出状态代码最高是 255。看看下面这个例子会发生什么情况：

```
$ cat test14b
#!/bin/bash
# testing the exit status
var1=10
var2=30
var3=$[ $var1 * var2 ]
echo The value is $var3
exit $var3
$
```

现在运行它，会得到：

```
$ ./test14b
The value is 300
$ echo $?
44
$
```

退出状态代码降低到适合 0～255 范围之内的数。shell 通过使用模计算做到了这一点。一个值的模是除操作之后的余数。运算结果就是那个特定的数被 256 除了之后的余数。本例中结果值是 300，

余数是 44，44 就是退出状态码。

在第 9 章中，将学习如何使用 `if-then` 语句来核对命令返回的错误状态，以检查这条命令是否成功运行。

## 8.9 小结

bash shell 脚本允许在脚本中把命令串在一起。创建脚本的最基本方法是在命令行中使用分号隔开多条命令。shell 按顺序执行每条命令，把每条命令的输出显示在显示器上。

也可以创建一个 shell 脚本文件，在文件中放置多条命令，shell 能按序执行命令。shell 脚本文件必须定义用来运行脚本的 shell。这在脚本文件的第一行定义，使用#!符号，然后是 shell 的完整路径。

在 shell 脚本中，通过在变量前使用美元符号来引用环境变量的值。也可以在脚本中定义自己的变量，并为变量赋值，甚至可以使用反引号将命令的输出结果赋值给变量。在变量名称前使用美元符号，就可以在脚本中使用变量值。

bash shell 允许标准行为命令的输入重定向和输出重定向。可以使用大于号对显示器上列出的任何命令的结果重定向到一个文件，大于号后面紧跟捕获输出文件的文件名。可以使用两个大于号把输出数据追加到一个已存在的文件中。小于号用来把输入重定向到一条命令。您也可以把输入从文件重定向到一条命令。

Linux 管道命令（竖条符号）允许把一条命令的输出直接重定向到另一条命令的输入。Linux 系统同时运行两条命令，把第一条命令的输出结果发送给第二条命令的输入，而不需要使用重定向文件。

bash shell 提供了两种方法执行 shell 脚本中的数学运算。`expr` 命令是一种执行整数运算的简单方法。在 bash shell 中，通过将等式放在方括号中并以美元符号开头，可以执行基本的数学计算。要执行浮点运算，需要使用 `bc` 计算命令，从内置数据重定向输入，并把输出保存到用户变量中。

最后，本章讨论了在 shell 脚本中如何使用退出状态。shell 中运行的每条命令都产生一个退出状态。退出状态是一个介于 0～255 之间的整数，这个整数表明命令是否成功完成，如果不成功，可能的原因是什么。退出状态为 0 表明命令成功完成。可以在 shell 脚本中使用退出命令，指定一个表明脚本完成的特定退出状态。

到目前为止，在您的脚本中，命令已经能够一条一条地顺序执行。下一章，将学习如何使用逻辑流控制来更改在脚本中执行的命令。

CHAPTER 9

# 第 9 章
# 使用结构化命令

本章内容：

- 改变命令流；
- 使用 if-then 逻辑；
- 嵌套 if-then；
- 测试条件；
- 高级 if-then 功能。

在第 8 章介绍的 shell 脚本中，shell 按照命令在脚本中出现的顺序处理每一条命令。想要所有的命令按照合适的顺序执行时，这种方式就很好。然而并不是所有的程序都这样运行。

许多程序在脚本命令之间需要某种逻辑流控制。这就意味着 shell 在一定的情况下运行特定的命令，但是它有能力在不同的情况下执行其他命令。有一类命令，允许脚本根据变量值的条件或者其他命令的结果跳过一些命令或者循环执行这些命令。这类命令通常被称为**结构化命令**。

结构化命令允许改变程序操作流，在某些条件下执行一些命令，而在其他条件下则跳过其他命令。许多结构化命令可以在 bash shell 中使用，这些命令我们将单独介绍它们。本章将介绍 `if-then` 语句。

## 9.1 使用 if-then 语句

最基本的结构化命令类型就是 `if-then` 语句。`if-then` 语句的格式如下：

```
if command
then
    commands
fi
```

如果习惯了在其他编程语言中使用 `if-then` 语句，这种格式可能会使您感到困惑。在其他编程语言中，`if` 语句后面的对象是一个值为 `TRUE` 或 `FALSE` 的等式。bash shell 脚本中的 `if` 语句不是这样的。

bash shell 中的 `if` 语句运行在 `if` 行定义的命令。如果命令的退出状态（参阅第 8 章）是 0（成功执行命令），将执行 `then` 后面的所有命令。如果命令的退出状态是 0 以外的其他值，那么 `then` 后面的命令将不会执行，bash shell 会移动到脚本的下一条命令。

一个简单示例将说明这一概念：

```
$ cat test1
#!/bin/bash
# testing the if statement
if date
then
   echo "it worked"
fi
```

该脚本使用 if 行的 date 命令。如果命令成功执行，echo 语句将显示文本字符串。从命令行运行该脚本将会得到如下结果：

```
$ ./test1
Sat Sep 29 14:09:24 EDT 2007
it worked
$
```

shell 执行 if 行的 date 命令。因为退出状态是零，所以也执行在 then 部分列出的 echo 语句。以下是另一个示例：

```
$ cat test2
#!/bin/bash
# testing a bad command
if asdfg
then
   echo "it didn't work"
fi
echo "we're outside of the if statement"
$ ./test2
./test2: line 3: asdfg: command not found
we're outside of the if statement
$
```

在本例中，我故意使用了一条不会在 if 语句行运行的命令。这是一条错误的命令，它会产生一个非零的退出状态，bash shell 跳过了 then 部分中的 echo 语句。还要注意，运行 if 语句中的命令所产生的错误信息仍然出现在脚本的输出结果中。有时可能想要避免这种情况发生。第 12 章将介绍如何避免这种情况发生。

并不限制在 then 部分只列一条命令，可以像在 shell 脚本中的其他部分那样列出多条命令。bash shell 会将这些命令看作一个整体，如果 if 语句行的命令返回的退出状态为 0，就会执行所有命令；如果命令返回的是非零退出状态，就会跳过这些命令：

```
$ cat test3
#!/bin/bash
# testing multiple commands in the then section
testuser=rich
if grep $testuser /etc/passwd
then
   echo The bash files for user $testuser are:
   ls -a /home/$testuser/.b*
fi
```

if 语句行使用 grep 命令搜索/etc/passwd 文件，查看系统是否正在使用一个特定的用户名。如果一个用户拥有该登录名，脚本会显示一些文本，然后列出用户 HOME 目录下的 bash 文件：

```
$ ./test3
rich:x:500:500:Rich Blum:/home/rich:/bin/bash
The files for user rich are:
/home/rich/.bash_history    /home/rich/.bash_profile
/home/rich/.bash_logout     /home/rich/.bashrc
$
```

然而，如果将变量 testuser 设置成系统中不存在的一个用户，什么都不会列出：

```
$ ./test3
$
```

这不是很好。如果我们能显示一些信息说明系统中不存在这个用户名，那就好了。那么，我们可以使用 if-then 语句的另一个功能。

注意：在一些脚本中，可能会看到 if-then 语句的另一种形式：

```
if command; then
   commands
fi
```

在计值的命令末尾使用一个分号，可以将 then 语句包含在此行中，这更像是在其他一些编程语言中处理 if-then 语句的方式。

## 9.2 if-then-else 语句

在 if-then 语句中，无论命令是否成功仅有一个选择。如果命令返回一个非零退出状态代码，bash shell 会移动到脚本中的下一条命令。在这种情况下，如果能够执行另一个命令集就好了。这正是 if-then-else 语句之所以存在的原因。

if-then-else 语句在语句中提供另一组命令：

```
if command
then
   commands
else
   commands
fi
```

如果 if 语句行的命令返回的退出状态码为 0，就会执行在 then 部分列出的命令，就像在普通的 if-then 语句中一样。如果 if 语句行的命令返回非零退出状态码，bash shell 将会执行 else 部分的命令。

现在可以修改 test 脚本，使其看起来如下所示：

```
$ cat test4
#!/bin/bash
# testing the else section
testuser=badtest
if grep $testuser /etc/passwd
then
   echo The files for user $testuser are:
   ls -a /home/$testuser/.b*
else
   echo "The user name $testuser doesn't exist on this system"
fi
$ ./test4
The user name badtest doesn't exist on this system
$
```

这样就显得更加友好。像 then 部分一样，else 部分也能包含多条命令。fi 语句表明 else 部分结束。

## 9.3 嵌套 if 语句

有时需要在脚本代码中检查几种情况。这时可以不必编写单独的 if-then 语句，可以使用 else 部分的另一种版本，称为 elif。

elif 以另一个 if-then 语句继续 else 部分：

```
if command1
then
   commands
elif command2
then
   more commands
fi
```

elif 语句行提供另一条要评估的命令，与原始的 if 语句行相似。如果 elif 命令返回的退出状态码为 0，bash 执行第二个 then 语句部分的命令。

可以把多个 elif 语句串在一起，创建一个大的 if-then-elif 组：

```
if command1
then
   command set 1
elif command2
then
```

```
    command set 2
elif command3
then
    command set 3
elif command4
then
    command set 4
fi
```

命令块的执行依赖于哪条命令返回的退出状态码为 0。记住，bash shell 会按顺序执行 if 语句，只有第一个返回 0 退出状态的命令会导致 then 部分命令被执行。稍后在“case 命令”一节，将会介绍如何使用 case 命令，而不必嵌套许多 if-then 语句。

## 9.4 test 命令

到目前为止，已经看到 if 语句行都是普通的 shell 命令。您可能想知道，bash 的 if-then 语句是否有能力评估任何条件，而不只是一条命令的退出状态码。

答案是否定的，它不能。然而，在 bash shell 中有一个灵巧的工具，使用 if-then 语句可以帮助我们评估其他事情。

test 命令提供一种检测 if-then 语句中不同条件的方法。如果 test 命令中列出的条件评估值为 true，test 命令以 0 退出状态代码退出，这使 if-then 语句使用与其他编程语言中的 if-then 语句一样的方法运行。如果条件为 false，则 test 命令退出，使得 if-then 语句失败。

test 命令的格式非常简单：

```
test condition
```

*condition* 是一系列 test 命令评估的参数和值。在 if-then 语句中使用时，test 命令如下所示：

```
if test condition
then
    commands
fi
```

bash shell 提供一种在 if-then 语句中声明 test 命令的另一种方法：

```
if [ condition ]
then
    commands
fi
```

方括号定义在 test 命令中使用的条件。注意，在前半个方括号的后面和后半个方括号的前面必须都有一个空格，否则会得到错误信息。

test 命令能够评估以下 3 类条件：

- 数值比较；
- 字符串比较；
- 文件比较。

下一节介绍在 if-then 语句中每种类型的使用方法。

### 9.4.1 数值比较

使用 test 命令的最常用方法是比较两个数值。表 9-1 显示了用于测试两个值的条件参数列表。

表 9-1 test 数值比较

| 比较 | 描述 | 比较 | 描述 |
|---|---|---|---|
| *n1* -eq *n2* | 检查 *n1* 是否等于 *n2* | *n1* -le *n2* | 检查 *n1* 是否小于或等于 *n2* |
| *n1* -ge *n2* | 检查 *n1* 是否大于或等于 *n2* | *n1* -lt *n2* | 检查 *n1* 是否小于 *n2* |
| *n1* -gt *n2* | 检查 *n1* 是否大于 *n2* | *n1* -ne *n2* | 检查 *n1* 是否不等于 *n2* |

数值测试条件能用于估计数字和变量。下面是一个例子：

```
$ cat test5
#!/bin/bash
# using numeric test comparisons
val1=10
val2=11

if [ $val1 -gt 5 ]
then
   echo "The test value $val1 is greater than 5"
fi

if [ $val1 -eq $val2 ]
then
   echo "The values are equal"
else
   echo "The values are different"
fi
```

第一个测试条件：

```
if  [ $val1 -gt 5 ]
```

测试变量 val1 的值是否大于 5。第二个 test 条件：

```
if  [ $val1 -eq $val2 ]
```

测试变量 val1 的值是否等于变量 val2 的值。运行该脚本并查看结果：

```
$ ./test5
The test value 10 is greater than 5
The values are different
$
```

这两个数值测试条件都像预期的一样被评估。

但 test 数值条件有一个限制。尝试该脚本：

```
$ cat test6
#!/bin/bash
# testing floating point numbers
val1=' echo "scale=4; 10 / 3 " | bc'
echo "The test value is $val1"
if [ $val1 -gt 3 ]
then
   echo "The result is larger than 3"
fi
$ ./test6
The test value is 3.3333
./test6: line 5: [: 3.3333: integer expression expected
$
```

这个例子使用 bash 计算器来生成一个浮点值，并将它存储在变量 val1 中。接下来，它使用 test 命令评估这个值。很明显这里出错了。

在第 8 章中，您已经学习了如何使用 bash shell 处理浮点值；这个脚本中仍然有一个问题。test 命令无法处理存储在变量 val1 中的浮点值。

记住，bash shell 只能处理整数数字。使用 bash 计算器时，我们只是欺骗 shell 把浮点值作为字符串值存储于一个变量。如果只是想使用 echo 语句显示结果，这种方法很好用。但是这在面向数值的函数（如数值测试条件）中不起作用。底线是不能在 test 命令中使用整数。

### 9.4.2 字符串比较

test 命令允许对字符串值进行比较。执行字符串的比较可能会很复杂，正如您将看到的那样。表 9-2 显示了用于评估两个字符串值的比较函数。

表 9-2 test 命令字符串比较

| 比较 | 描述 | 比较 | 描述 |
|---|---|---|---|
| *str1 = str2* | 检查 *str1* 与 *str2* 是否相同 | *str1 > str2* | 检查 *str1* 是否大于 *str2* |
| *str1 != str2* | 检查 *str1* 与 *str2* 是否不同 | *-n str1* | 检查 *str1* 的长度是否大于 0 |
| *str1 < str2* | 检查 *str1* 是否小于 *str2* | *-z str1* | 检查 *str1* 的长度是否为 0 |

下面几节将介绍几种可用的不同字符串比较。

### 1. 字符串相等

字符串相等和不相等条件大都是不言而喻的。很容易得知两个字符串值是相等还是不相等：

```
$cat test7
#!/bin/bash
# testing string equality
testuser=rich

if [ $USER = $testuser ]
then
   echo "Welcome $testuser"
fi
$ ./test7
Welcome rich
$
```

类似地，使用不相等字符串比较：

```
$ cat test8
#!/bin/bash
# testing string equality
testuser=baduser

if [ $USER != $testuser ]
then
   echo "This isn't $testuser"
else
   echo "Welcome $testuser"
fi
$ ./test8
This isn't baduser
$
```

比较字符串是否相等时，测试比较将所有标点符号和大写都考虑在内。

### 2. 字符串顺序

尝试比较一个字符串比另一个字符串是小还是大时，事情就开始变得复杂了。使用 test 命令的大于或者小于功能时，shell 程序员通常会遇到两个问题。

- 大于和小于符号一定要转义，否则 shell 会将它们当作重定向符号，将字符串值看作文件名。
- 大于和小于顺序与在 sort 命令中的顺序不同。

第一条能导致很大的错误，而且编写脚本时通常不会发现该问题。下面是一个典型的示例，shell 脚本程序员新手有时会出现这种问题：

```
$ cat badtest
#!/bin/bash
# mis-using string comparisons

val1=baseball
val2=hockey

if [ $val1 > $val2 ]
then
   echo "$val1 is greater than $val2"
else
```

```
   echo "$val1 is less than $val2"
fi
$ ./badtest
baseball is greater than hockey
$ ls -l hockey
-rw-r--r--    1 rich        rich          0 Sep 30 19:08 hockey
$
```

在脚本中只单独使用大于号，没有生成错误，但是结果不对。脚本将大于号理解为输出重定向，所以创建了一个 hockey 文件。由于重定向成功完成，所以测试命令返回退出状态代码 0，if 语句认为事情已成功完成。

要解决这个问题，需要适当地转义大于号：

```
$ cat test9
#!/bin/bash
# mis-using string comparisons

val1=baseball
val2=hockey

if [ $val1 \> $val2 ]
then
   echo "$val1 is greater than $val2"
else
   echo "$val1 is less than $val2"
fi
$ ./test9
baseball is less than hockey
$
```

现在，答案更贴近所希望的字符串比较的行。

第二条更加微妙一点，除非使用大写字母和小写字母，否则不会遇到这种问题。sort 命令处理大些字母的方法与 test 命令处理大写字母的方法相反。我们在脚本中测试一下这个功能：

```
$ cat test9b
#!/bin/bash
# testing string sort order
val1=Testing
val2=testing

if [ $val1 \> $val2 ]
then
   echo "$val1 is greater than $val2"
else
   echo "$val1 is less than $val2"
fi
$ ./test9b
Testing is less than testing
$ sort testfile
testing
Testing
$
```

在 test 命令中，大写字母小于小写字母。然而，当在文件中放置相同的字符串并使用 sort 命令时，首先显示小写字母。这是由于每条命令使用的排序技术不同。test 命令使用标准的 ASCII 排序，使用每个字母的 ASCII 数值来决定排列顺序。sort 命令使用为系统当前语言设置定义的排列顺序。对于英语来说，当前设置指定小写字母排列顺序在大写字母之前。

警告：注意，test 命令使用标准的数学比较符来进行字符串比较，而使用文本代码进行数值比较。这是一个许多程序员都试图反过来使用的功能。如果使用数学比较符号进行数值比较，shell 会将它们解释为字符串，可能不会产生正确的结果。

### 3. 字符串大小

评估一个变量是否包含数据时，使用-n 和-z 比较很方便：

```
$ cat test10
#!/bin/bash
# testing string length
val1=testing
val2=''

if [ -n $val1 ]
then
   echo "The string '$val1' is not empty"
else
   echo "The string '$val1' is empty"
fi

if [ -z $val2 ]
then
   echo "The string '$val2' is empty"
else
   echo "The string '$val2' is not empty"
fi

if [ -z $val3 ]
then
   echo "The string '$val3' is empty"
else
   echo "The string '$val3' is not empty"
fi
$ ./test10
The string 'testing' is not empty
The string '' is empty
The string '' is empty
$
```

这个例子创建了两个字符串变量。变量 val1 包含一个字符串，变量 val2 作为一个空字符串创建。进行了如下比较：

```
if  [ -n $val1 ]
```

确定变量 val1 的长度是否为非零值，如果是非零值，那么将处理 then 部分：

```
if  [ -z $val2 ]
```

确定变量 val2 的长度是否为 0，如果是 0，那么将处理 then 部分：

```
if  [ -z $val3 ]
```

确定变量 val3 的长度是否为 0。这个变量没有在 shell 脚本中定义，所以字符串长度仍然为零，即使它没有定义。

**警告：空变量和没有初始化的变量可能会对 shell 脚本测试产生灾难性的影响。如果不确定变量的内容，在数值比较或者字符串比较中使用它之前，最好使用-n 或者-z 测试一下它是否有值。**

## 9.4.3 文件比较

测试比较的最后一类，也很可能是 shell 脚本中最强大和最常使用的一类比较。test 命令能够测试 Linux 文件系统上的文件状态和路径。表 9-3 列出了这些比较。

表 9-3　　test 命令文件比较

| 比较 | 描述 |
| --- | --- |
| -d *file* | 检查 *file* 是否存在并且是一个目录 |
| -e *file* | 检查 *file* 是否存在 |

续表

| 比较 | 描述 |
| --- | --- |
| -f *file* | 检查 *file* 是否存在并且是一个文件 |
| -r *file* | 检查 *file* 是否存在并且可读 |
| -s *file* | 检查 *file* 是否存在并且不为空 |
| -w *file* | 检查 *file* 是否存在并且可写 |
| -x *file* | 检查 *file* 是否存在并且可执行 |
| -O *file* | 检查 *file* 是否存在并且被当前用户拥有 |
| -G *file* | 检查 *file* 是否存在并且默认组是否为当前用户组 |
| *file1* -nt *file2* | 检查 *file1* 是否比 *file2* 新 |
| *file1* -ot *file2* | 检查 *file1* 是否比 *file2* 旧 |

这些条件使您能够在 shell 脚本中检查文件系统中文件，它们经常用于访问文件的脚本。由于它们使用得非常频繁，下面将逐个它们。

**1. 检查目录**

-d 测试检查指定的文件是否作为目录存在于系统中。如果想将文件写到一个目录下，或者在试图改变到目录位置之前，这通常是一个很好的操作：

```
$ cat test11
#!/bin/bash
# look before you leap
if [ -d $HOME ]
then
   echo "Your HOME directory exists"
   cd $HOME
   ls -a
else
   echo "There's a problem with your HOME directory"
fi
$ ./test11
Your HOME directory exists
.              Drivers            .ICEauthority          .recently
..             .emacs.d           joomla                 .redhat
4rich          .evolution         .lesshst               rpm.list
backup.zip     .fontconfig        .metacity              .spamassassin
.bash_history  .gconf             mnt                    store
.bash_logout   .gconfd            .mozilla               store.sql
.bash_profile  .gftp              Music                  store.zip
.bashrc        .gimp-2.2          myprob                 Templates
.bittorrent    .gnome             myproblem              test
.ccache        .gnome2            myprog                 test1
.config        .gnome2 private    .mysql_history         .thumbnails
Desktop        .gphpedit          .nautilus              .Trash
.dmrc          .gstreamer-0.10    .openoffice.org2.0     Videos
Documents      .gtk-bookmarks     Pictures               .viminfo
Download       .gtkrc-1.2-gnome2  Public
$
```

这个示例代码使用-d 测试条件，查看根目录下是否存在用户 HOME 目录。如果存在，继续使用 cd 命令更改到 HOME 目录，并执行目录列表。

**2. 检查对象是否存在**

在脚本中使用文件或目录之前，-e 比较能够检查它们是否存在：

```
$ cat test12
#!/bin/bash
# checking if a directory exists
if [ -e $HOME ]
```

```
then
   echo "OK on the directory, now let's check the file"
   # checking if a file exists
   if [ -e $HOME/testing ]
   then
      # the file exists, append data to it
      echo "Appending date to existing file"
      date >> $HOME/testing
   else
      # the file doesn't exist, create a new file
      echo "Creating new file"
      date > $HOME/testing
   fi
else
   echo "Sorry, you don't have a HOME directory"
fi
$ ./test12
OK on the directory, now let's check the file
Creating new file
$ ./test12
OK on the directory, now let's check the file
Appending date to existing file
$
```

第一个检查使用-e 比较确定用户是否有 HOME 目录。如果有，下一个-e 比较检查确定 HOME 目录下是否存在文件 testing。如果该文件不存在，shell 脚本使用一个大于重定向符号，用于从 date 命令的输出创建一个新文件。第二次运行该 shell 脚本时，使用了两个大于号，所以只是将 date 的输出附加到已存在的文件中。

**3. 检查文件**

-e 比较既适用于文件也适用于目录。要确定指定的对象是一个文件，必须使用-f 比较：

```
$ cat test13
#!/bin/bash
# check if a file
if [ -e $HOME ]
then
   echo "The object exists, is it a file?"
   if [ -f $HOME ]
   then
      echo "Yes, it's a file!"
   else
      echo "No, it's not a file!"
      if [ -f $HOME/.bash_history ]
      then
         echo "But this is a file!"
      fi
   fi
else
   echo "Sorry, the object doesn't exist"
fi
$ ./test13
The object exists, is it a file?
No, it's not a file!
But this is a file!
$
```

这个小脚本进行了全面检查！首先，使用-e 比较来检查$HOME 是否存在。如果存在，使用-f 检查它是否是个文件。如果不是文件（当然不是文件），使用-f 比较检查$HOME/.bash-history 是否是一个文件，它确实是个文件。

**4. 是否能读**

在尝试从文件中读取数据之前，通常首先检查一下是否能读取文件。使用-r 比较完成此工作：

```
$ cat test14
#!/bin/bash
# testing if you can read a file
pwfile=/etc/shadow

# first, test if the file exists, and is a file
if [ -f $pwfile ]
then
   # now test if you can read it
   if [ -r $pwfile ]
   then
      tail $pwfile
   else
      echo "Sorry, I'm unable to read the $pwfile file"
   fi
else
   echo "Sorry, the file $file doesn't exist"
fi
$ ./test14
Sorry, I'm unable to read the /etc/shadow file
$
```

文件/ect/shadow包含系统用户的加密密码，所以系统中的普通用户不能读取这个文件。-r比较确定我没有读取这个文件的权限，所以测试命令失败，bash shell执行if-then语句的else部分。

**5. 检查空文件**

应该使用-s比较来检查文件是否为空，尤其是想要删除文件时。注意，-s比较成功时，它表明文件包含数据：

```
$ cat test15
#!/bin/bash
# testing if a file is empty
file=t15test
touch $file

if [ -s $file ]
then
   echo "The $file file exists and has data in it"
else
   echo "The $file exists and is empty"
fi

date > $file
if [ -s $file ]
then
   echo "The $file file has data in it"
else
   echo "The $file is still empty"
fi
$./test15
The t15test exists and is empty
The t15test file has data in it
$
```

touch命令创建了该文件，但在文件中没有放置任何数据。使用了date命令并将输出重定向到文件之后，-s比较表明文件中有数据。

**6. 检查是否能够向文件中写入数据**

-w比较确定是否有权限向文件中写入数据：

```
$ cat test16
#!/bin/bash
# checking if a file is writeable

logfile=$HOME/t16test
```

```
touch $logfile
chmod u-w $logfile
now='date +%Y%m%d-%H%M'

if [ -w $logfile ]
then
   echo "The program ran at: $now" > $logfile
   echo "The fist attempt succeeded"
else
   echo "The first attempt failed"
fi

chmod u+w $logfile
if [ -w $logfile ]
then
   echo "The program ran at: $now" > $logfile
   echo "The second attempt succeeded"
else
   echo "The second attempt failed"
fi
$ ./test16
The first attempt failed
The second attempt succeeded
$ cat $HOME/t16test
The program ran at: 20070930-1602
$
```

这是一个相当忙的 shell 脚本！首先，在 HOME 目录中定义日志文件，并在变量 logfile 中存储该文件名，创建了该文件，然后使用 chmod 命令删除用户的写权限。接下来，使用 date 命令创建了变量 now 并存储了时间戳。这些都做完之后，它检查对新的日志文件是否有写权限（刚刚把写权限删除）。因为没有写权限，您应该看到显示的失败信息。

在此之后，脚本再次使用 chmod 命令将写权限重新授权给用户，并再次试图向文件中写入数据。这次写入成功了。

**7. 检查是否能够运行文件**

-x 比较是确定您是否有指定文件的运行权限的一个简单方法。尽管对于大多数命令来说，这可能不必要，但是如果从 shell 脚本中运行很多脚本，它就会很方便：

```
$ cat test17
#!/bin/bash
# testing file execution
if [ -x test16 ]
then
   echo "You can run the script:"
   ./test16
else
   echo "Sorry, you are unable to execute the script"
fi
$ ./test17
You can run the script:
The first attempt failed
The second attempt succeeded
$ chmod u-x test16
$ ./test17
Sorry, you are unable to execute the script
$
```

该 shell 脚本示例使用-x 比较来检查是否有执行脚本 test16 的权限。如果有权限，就会运行该脚本（注意，在 shell 脚本中，必须要有合适的路径来执行不位于您 PATH 中的脚本）。第一次成功运行脚本 test16 之后，改变对它的权限，再试一次。这次-x 比较失败，因为您没有执行脚本 test16 的权限。

**8. 检查所有权**

-O 比较可以容易地检查您是否是文件的所有者：

```
$ cat test18
#!/bin/bash
# check file ownsership
```

```
if [ -O /etc/passwd ]
then
   echo "You're the owner of the /etc/passwd file"
else
   echo "Sorry, you're not the owner of the /etc/passwd file"
fi
$ ./test18
Sorry, you're not the owner of the /etc/passwd file
$ su
Password:
# ./test18
You're the owner of the /etc/passwd file
#
```

该脚本使用-O比较来检查运行脚本的用户是否是文件/etc/passwd的所有者。第一次，脚本在普通用户的账户下运行，所以检查失败。第二次，使用su命令成为根用户，检查成功。

-G比较检查文件的默认组，如果它与用户的默认组相匹配，它就成功。这个可能有点混乱，因为-G比较只检查默认组，而不是检查用户所属的所有组。下面是一个例子：

```
$ cat test19
#!/bin/bash
# check file group test

if [ -G $HOME/testing ]
then
   echo "You're in the same group as the file"
else
   echo "The file is not owned by your group"
fi
$ ls -l $HOME/testing
-rw-rw-r-- 1 rich rich 58 2007-09-30 15:51 /home/rich/testing
$ ./test19
You're in the same group as the file
$ chgrp sharing $HOME/testing
$ ./test19
The file is not owned by your group
$
```

第一次运行脚本时，文件$HOME/testing在rich组，-G比较成功。接下来，将组更改为sharing组，我也是其中的一个成员。但-G比较失败，因为它只能比较默认组，而不是其他所属的任何组。

**9. 检查文件日期**

最后一个比较是比较两个文件的创建时间。编写脚本安装软件时，这很方便。有时可能不想安装一个比系统中已安装的文件更旧的文件。

-nt比较确定一个文件是否比另一个文件新。如果一个文件更新，它就有一个更近的文件创建时间。-ot比较确定一个文件是否比另一个文件更旧。如果文件更旧，它就会有一个比较久远的创建时间：

```
$ cat test20
#!/bin/bash
# testing file dates

if [ ./test19 -nt ./test18 ]
then
   echo "The test19 file is newer than test18"
else
   echo "The test18 file is newer than test19"
fi

if [ ./test17 -ot ./test19 ]
then
   echo "The test17 file is older than the test19 file"
fi
$ ./test20
The test19 file is newer than test18
The test17 file is older than the test19 file
$ ls -l test17 test18 test19
-rwxrw-r-- 1 rich rich 167 2007-09-30 16:31 test17
```

```
-rwxrw-r-- 1 rich rich 185 2007-09-30 17:46 test18
-rwxrw-r-- 1 rich rich 167 2007-09-30 17:50 test19
$
```

运行脚本的目录不同，在比较中使用文件路径就不同。如果正在检查的文件可以四处移动，就会产生问题。另一个问题是，这两个比较都不能用于检查文件是否一开始就存在。测试这个检查：

```
$ cat test21
#!/bin/bash
# testing file dates

if [ ./badfile1 -nt ./badfile2 ]
then
   echo "The badfile1 file is newer than badfile2"
else
   echo "The badfile2 file is newer than badfile1"
fi
$ ./test21
The badfile2 file is newer than badfile1
$
```

这个小示例表明，如果文件不存在，-nt 比较只是返回一个失败条件。要在-nt 或-ot 比较中使用文件，必须首先确定该文件存在。

## 9.5 复合条件检查

if-then 语句可以使用布尔逻辑来合并检查条件。可以使用两个布尔操作符：

- [ *condition1* ] && [ *condition2* ]
- [ *condition1* ] || [ *condition2* ]

第一个布尔操作使用 AND 布尔操作符来合并两个条件，两个条件必须都满足才能执行 then 部分。

第二个布尔操作使用 OR 布尔操作符来合并两个条件。如果任意一个条件评估为 true，那么就会执行 then 部分。

```
$ cat test22
#!/bin/bash
# testing compound comparisons

if [ -d $HOME ] && [ -w $HOME/testing ]
then
   echo "The file exists and you can write to it"
else
   echo "I can't write to the file"
fi
$ ./test22
I can't write to the file
$ touch $HOME/testing
$ ./test22
The file exists and you can write to it
$
```

使用 AND 布尔操作符，两个比较必须都满足。第一个比较检查用户的 HOME 目录是否存在。第二个比较检查在用户的根目录下是否有一个 testing 文件，并且用户是否对该文件有写权限。如果这两个比较中的任意一个失败，if 语句就失败，shell 将执行 else 部分。如果这两个比较都成功，if 语句成功，shell 将执行 then 部分。

## 9.6 if-then 的高级特征

在 bash shell 中最近新增加了两个比较新的功能，它们提供了可以在 if-then 语句中使用的高级

功能：

- 双圆括号表示数学表达数；
- 双方括号表示高级字符串处理函数。

下面几节将详细介绍每种功能。

## 9.6.1 使用双圆括号

**双圆括号**命令允许在比较中包含高级数学公式。test 命令只允许在比较中进行简单的算术操作。双圆括号命令提供更多的数学符号，这些符号是其他语言程序员习惯使用的符号。双圆括号命令的格式是：

```
(( expression ))
```

术语 expression 可以是任何的数学赋值表达式或数学比较表达式。除了 test 命令使用的标准数学操作符，表 9-4 显示了在双括号命令中可以使用的其他操作符列表。

**表 9-4 双圆括号命令符号**

| 符号 | 描述 | 符号 | 描述 |
|---|---|---|---|
| *val++* | 后增量 | << | 逐位左移 |
| *val--* | 后减量 | >> | 逐位右移 |
| *++val* | 前增量 | & | 逐位布尔逻辑与 |
| *--val* | 前减量 | \| | 逐位布尔逻辑或 |
| ! | 逻辑否定 | && | 逻辑与 |
| ~ | 逐位取反 | \|\| | 逻辑或 |
| ** | 取幂 | | |

可以在 if 语句中使用双圆括号命令，也可以使用普通命令在脚本中进行赋值：

```
$ cat test23
#!/bin/bash
# using double parenthesis

val1=10

if (( $val1 ** 2 >90 ))
then
   (( val2 = $val1 ** 2 ))
   echo "The square of $val1 is $val2"
fi
$ ./test23
The square of 10 is 100
$
```

注意，在双圆括号内的表达式中，不必转义大于号。这是双圆括号提供的另一个高级功能。

## 9.6.2 使用双方括号

**双方括号**命令为字符串比较提供高级功能。双方括号命令的格式是：

```
[[ expression ]]
```

双方括号包围的 expreesion 使用在 test 命令中使用的标准字符串比较。但是它提供了 test 命令没有的另一个功能，即**模式匹配**。

在模式匹配中，可以定义与字符串值相匹配的正则表达式（在第 17 章中将详细介绍）：

```
$ cat test24
#!/bin/bash
# using pattern matching
if [[ $USER == r* ]]
then
   echo "Hello $USER"
else
   echo "Sorry, I don't know you"
```

```
fi
$ ./test24
Hello rich
$
```

双方括号命令匹配环境变量$USER，查看它是否以字母 r 开头。如果是，则比较成功，shell 执行 then 部分的命令。

## 9.7 case 命令

您经常会发现自己试图评估一个变量的值，并在一组可能的值中寻找特定的值。在这种情况下，不得不编写冗长的 if-then-else 语句，如下所示：

```
$ cat test25
#!/bin/bash
# looking for a possible value

if [ $USER = "rich" ]
then
   echo "Welcome $USER"
   echo "Please enjoy your visit"
elif [ $USER = barbara ]
then
   echo "Welcome $USER"
   echo "Please enjoy your visit"
elif [ $USER = testing ]
then
   echo "Special testing account"
elif [ $USER = jessica ]
then
   echo "Don't forget to logout when you're done"
else
   echo "Sorry, you're not allowed here"
fi
$ ./test25
Welcome rich
Please enjoy your visit
$
```

elif 语句继续 if-then 检查，为单个比较变量查找特定的值。

可以使用 case 命令，而不是编写所有的 elif 语句来继续检查相同的变量值。case 命令以列表导向格式检查单个变量的多个值：

```
case variable in
pattern1 | pattern2) commands1;;
pattern3) commands2;;
*) default commands;;
esac
```

case 命令将指定的变量与不同的模式进行比较。如果变量与模式匹配，shell 执行为该模式指定的命令。可以在一行中列出多个模式，使用竖条操作符将每个模式分开。星号是与任何列出的模式都不匹配的所有值。下面是一个使用 case 命令转换 if-then-else 程序的示例：

```
$ cat test26
#!/bin/bash
# using the case command

case $USER in
rich | barbara)
   echo "Welcome, $USER"
   echo "Please enjoy your visit";;
testing)
```

```
    echo "Special testing account";;
jessica)
    echo "Don't forget to log off when you're done";;
*)
    echo "Sorry, you're not allowed here";;
esac
$ ./test26
Welcome, rich
Please enjoy your visit
$
```

case 命令提供了一种为每个可能的变量值指定不同选项的更清楚的方法。

## 9.8 小结

结构化命令允许改变 shell 脚本中的普通执行流。最基本的结构化命令是 if-then 语句。该语句允许评估一条命令，并基于评估命令的结果执行其他命令。

如果指定的命令也失败了，可以扩展 if-then 语句以包含 bash shell 执行的一组命令。只有当被估计的命令返回非零退出状态码时，if-then-else 语句才允许执行命令。

也可以使用 elif 语句将 if-then-else 语句连接起来。elif 与使用 else if 语句等同，如果被估计的原始命令失败，可以提供额外的检查。

在大多数脚本中，不是想估计一条命令，而是想估计一个条件，例如数值、字符串的内容，或者文件或目录的状态。test 命令提供评估所有这些条件的一种简单方法。如果条件评估为 true，test 命令就为 if-then 语句生成一个零退出状态代码。如果条件评估为 false，test 命令会为 if-then 语句生成一个非零退出状态代码。

方括号是一个特殊的 bash 命令，它是 test 命令的同义词。在 if-then 语句中可以使用方括号包围测试条件，来检查数字、字符串和文件条件。

双圆括号命令允许使用附加操作符执行高级数学评估，双方括号允许执行高级字符串模式匹配评估。

最后，本章讨论了 case 命令，它是执行多条 if-then-else 命令并在值列表中检查单个变量值的快速方法。

下一章通过研究 shell 循环命令来继续讨论结构化命令。for 命令和 while 命令允许创建在指定时间段内重复的循环。

# 第 10 章
# 更多结构化命令

本章内容：

- 使用 for 语句循环；
- 使用 until 语句迭代；
- 使用 while 语句；
- 结合循环；
- 重定向循环输出。

在前一章，已经看到如何通过检验命令的输出和变量的值来操作 shell 脚本程序中的流。本章将继续学习控制 shell 脚本流的结构化命令。您将会看到如何执行重复的过程和命令，这些过程和命令能够通过一组命令循环，直到满足一个特定条件。本章讨论和演示 bash shell 循环命令 `for`、`while` 和 `until`。

## 10.1 for 命令

重复一系列的命令是一种常见的编程实践。通常需要重复一组命令直到满足一个特定的条件。例如，处理目录下的所有文件、系统中的所有用户，或者文本文件中的所有行。

bash shell 提供了 `for` 命令，用于创建通过一系列值重复的循环。每次重复使用系列中的一个值执行一个定义的命令集。bash shell `for` 命令的基本格式为：

```
for var in list
do
   commands
done
```

在参数 `list` 中提供一系列用于迭代的值。指定列表中的值有几种不同的方法。

在每次迭代中，变量 `var` 包含列表的当前值。第一次迭代使用列表中的第一项，第二次迭代使用第二项，依次类推直到列表中的所有项都被使用为止。

进入 `do` 和 `done` 语句之间的命令可以是一条或多条的标准 bash shell 命令。在命令中，变量`$var` 包含当前迭代的列表项值。

注意：如果愿意，可以将 `do` 语句与 `for` 语句放在同一行，但是必须使用分号将它与列表项分开：`for var in list; do`。

如前所述，在列表中指定值有几种不同的方法。下面几节将介绍这几种方法。

### 10.1.1 读取列表中的值

`for` 命令的最基本使用方法是通过在 `for` 命令中定义一列值来迭代：

```
$ cat test1
#!/bin/bash
# basic for command
for test in Alabama Alaska Arizona Arkansas California Colorado
do
   echo The next state is $test
done
$ ./test1
The next state is Alabama
The next state is Alaska
The next state is Arizona
The next state is Arkansas
The next state is California
The next state is Colorado
$
```

每次 for 命令通过提供的值列表进行迭代时，它将列表中的下一个值赋值给变量 test。变量$test 可以像在 for 命令语句中使用其他脚本变量一样使用。最后一次迭代之后，变量$test 在 shell 脚本的其他部分中仍然有效。它仍然是迭代的最后一个值（除非改变它的值）：

```
$ cat test1b
#!/bin/bash
# testing the for variable after the looping

for test in Alabama Alaska Arizona Arkansas California Colorado
do
   echo "The next state is $test"
done
echo "The last state we visited was $test"
test=Connecticut
echo "Wait, now we're visiting $test"
$ ./test1b
The next state is Alabama
The next state is Alaska
The next state is Arizona
The next state is Arkansas
The next state is California
The next state is Colorado
The last state we visited was Colorado
Wait, now we're visiting Connecticut
$
```

变量$test 保留它的值，并且允许更改该值，并在 for 命令循环之外像任何其他的变量一样使用它。

### 10.1.2 读取列表中的复杂值

事情并不总是像 for 循环看起来那样简单。有时会遇到产生问题的数据。这儿有一个典型的示例，说明可以给 shell 脚本程序员带来麻烦的问题：

```
$ cat badtest1
#!/bin/bash
# another example of how not to use the for command

for test in I don't know if this'll work
do
   echo "word:$test"
done
$ ./badtest1
word:I
word:dont know if thisll
word:work
$
```

shell 看到列表值当中的单引号，并试图用它们来定义一个单独的数据值，它的确破坏了这个过程。

有两种方法解决这个问题：

- 使用转义字符（反斜杠符号）来转义单引号；
- 使用双引号来定义使用单引号的值。

两种方法都不是很好，但每种解决方法都有助于解决该问题：

```
$ cat test2
#!/bin/bash
# another example of how not to use the for command
for test in I don\'t know if "this'll" work
do
   echo "word:$test"
done
$ ./test2
word:I
word:don't
word:know
word:if
word:this'll
word:work
$
```

在第一个问题值中，添加了反斜杠符号来转义 don't 值中的单引号。在第二个问题值中，使用了双引号把 this'll 包围起来。这两种方法都能正常运行并识别值。

可能遇到的另一个问题是运行多字值。记住，for 循环认为每个值都用空格分隔。如果有包含空格的数据值，就会遇到另一个问题：

```
$ cat badtest2
#!/bin/bash
# another example of how not to use the for command

for test in Nevada New Hampshire New Mexico New York North Carolina
do
   echo "Now going to $test"
done
$ ./badtest1
Now going to Nevada
Now going to New
Now going to Hampshire
Now going to New
Now going to Mexico
Now going to New
Now going to York
Now going to North
Now going to Carolina
$
```

很遗憾，那不是我们想要的。for 命令用空格分隔列表中的每个值。如果在个别的数据值中有空格，必须使用双引号将它们包围起来。

```
$ cat test3
#!/bin/bash
# an example of how to properly define values

for test in Nevada "New Hampshire" "New Mexico" "New York"
do
   echo "Now going to $test"
done
$ ./test3
Now going to Nevada
Now going to New Hampshire
Now going to New Mexico
Now going to New York
$
```

现在 for 命令可以很好地区分不同的值。还要注意，使用双引号包围值时，shell 不会将双引号作为值的一部分。

### 10.1.3　从变量读取列表

shell 脚本中经常发生的是积累了存储于变量中的一个值列表，然后需要通过列表迭代。也可以使用 for 命令做到这一点：

```
$ cat test4
#!/bin/bash
# using a variable to hold the list

list="Alabama Alaska Arizona Arkansas Colorado"
list=$list" Connecticut"

for state in $list
do
   echo "Have you ever visited $state?"
done
$ ./test4
Have you ever visited Alabama?
Have you ever visited Alaska?
Have you ever visited Arizona?
Have you ever visited Arkansas?
Have you ever visited Colorado?
Have you ever visited Connecticut?
$
```

变量$list 包含用于迭代的标准的文本值列表。注意，代码可以使用另一种赋值语句向包含在 $list 变量中的已存在列表添加（或连接）一条项目。这是一种将文本添加到一个已存在的、存储在一个变量中的文本字符串末尾的常用方法。

### 10.1.4　读取命令中的值

生成列表中使用的值的另一种方法是使用命令的输出。可以使用反引号字符来执行生成输出的任何命令，然后在 for 命令中使用命令的输出：

```
$ cat test5
#!/bin/bash
# reading values from a file

file="states"

for state in 'cat $file'
do
   echo "Visit beautiful $state"
done
$ cat states
Alabama
Alaska
Arizona
Arkansas
Colorado
Connecticut
Delaware
Florida
Georgia
$ ./test5
Visit beautiful Alabama
Visit beautiful Alaska
Visit beautiful Arizona
Visit beautiful Arkansas
```

```
Visit beautiful Colorado
Visit beautiful Connecticut
Visit beautiful Delaware
Visit beautiful Florida
Visit beautiful Georgia
$
```

本例子使用了 cat 命令显示文件 states 的内容。注意，文件 states 每行只包含一个状态，不用空格分隔。假设每行只有一个状态，for 命令仍然一次转义 cat 命令的一行输出结果。然而，这并不能解决数据中有空格的问题。如果列出的状态中包含一个空格，for 命令仍然将每个单词作为一个单独的值。这是有原因的，下一节将会介绍。

注意：代码 test5 示例将文件名赋值给一个变量，只使用了文件名而没有使用路径。这就需要文件和脚本在同一路径下。如果不是这样，就需要使用完整的路径名（绝对路径或相对路径）来引用文件位置。

### 10.1.5 改变字段分隔符

引起这个问题的原因是称为**内部字段分隔符**（internal field separator，IFS）的特殊环境变量。环境变量 IFS 定义了 bash shell 用作字段分隔符的字符列表。默认情况下，bash shell 将下面的字符看作字段分隔符：

- 空格；
- 制表符；
- 换行符。

如果 bash shell 在数据中遇到这几种字符中的一种，它就会认为您正在列表中启动新的数据字段。当处理能够包含空格的数据（如文件名）时，就会产生干扰，正如前面的示例所示。

要解决该问题，可以在 shell 脚本中暂时更改环境变量 IFS 的值，限制 bash shell 看作是字段分隔符的字符。然而，这样做有点奇怪。例如，如果想将 IFS 的值更改为只识别换行符，需要这样做：

```
IFS=$'\n'
```

在脚本中添加这条语句，通知 bash shell 在数据值中忽略空格和制表符。将该语句应用于先前的脚本，产生的结果如下：

```
$ cat test5b
#!/bin/bash
# reading values from a file

file="states"

IFS=$'\n'
for state in 'cat $file'
do
  echo "Visit beautiful $state"
done
$ ./test5b
Visit beautiful Alabama
Visit beautiful Alaska
Visit beautiful Arizona
Visit beautiful Arkansas
Visit beautiful Colorado
Visit beautiful Connecticut
Visit beautiful Delaware
Visit beautiful Florida
Visit beautiful Georgia
Visit beautiful New York
Visit beautiful New Hampshire
Visit beautiful North Carolina
$
```

现在 shell 脚本可以在列表中使用包含空格的值了。

警告：当使用较长的脚本时,可能在一个位置更改了 IFS 的值，但忘记了该值，并在脚本的其他地方假设为默认值。一个安全的做法是，在更改原始 IFS 值之前将它保存起来，当需要运用时即可恢复。

该技术可以写成如下代码：

```
IFS.OLD=$IFS
IFS=$'\n'
‹use the new IFS value in code›
IFS=$IFS.OLD
```

这就确保了在接下来的脚本操作中，IFS 值会返回默认值。

环境变量 IFS 有其他的很好的应用。如果想迭代文件中用冒号分隔的值(如在文件/etc/passwd中)。需要做的就是将 IFS 的值设置为冒号：

```
IFS=:
```

如果想指定多个 IFS 字符，只需将它们在赋值行中串连起来即可：

```
IFS=$'\n':;"
```

该赋值使用了换行符、冒号、分号和双引号字符作为字段分隔符。对于使用的 IFS 字符如何解析数据没有限制。

### 10.1.6 使用通配符读取目录

最后，可以使用 for 命令自动迭代文件的目录。为此，必须在文件或路径名中使用通配符。这就迫使 shell 使用**文件通配**（file globbing）。文件通配是生成与指定通配符匹配的文件或路径名的过程。

当不知道目录中所有的文件名时，这个功能非常好：

```
$ cat test6
#!/bin/bash
# iterate through all the files in a directory

for file in /home/rich/test/*
do

   if [ -d "$file" ]
   then
      echo "$file is a directory"
   elif [ -f "$file" ]
   then
      echo "$file is a file"
   fi
done
$ ./test6
/home/rich/test/dir1 is a directory
/home/rich/test/myprog.c is a file
/home/rich/test/myprog is a file
/home/rich/test/myscript is a file
/home/rich/test/newdir is a directory
/home/rich/test/newfile is a file
/home/rich/test/newfile2 is a file
/home/rich/test/testdir is a directory
/home/rich/test/testing is a file
/home/rich/test/testprog is a file
/home/rich/test/testprog.c is a file
$
```

for 命令迭代/home/rich/test/*列表的结果。代码使用 test 命令（使用方括号方法）测试每个项目，查看它是目录（使用-d 参数）还是文件（使用-f 参数）。(参阅第 9 章)

注意，在本例子中对 if 语句测试中做了一些不同的处理：

```
if [ -d "$file" ]
```

在 Linux 中，包含空格的路径和文件名是合法的。要容纳它们，应该使用双引号将变量$file 包围起来。如果不这样做，遇到包含空格的路径或文件名时就会出错：

```
./test6: line 6: [: too many arguments
./test6: line 9: [: too many arguments
```

bash shell 将 test 命令中的其他词语理解为参数，这会生成错误。

通过在 for 命令中列出一系列目录通配符，可以在同一个 for 语句中将目录搜索方法和列表方法结合起来：

```
$ cat test7
#!/bin/bash
# iterating through multiple directories

for file in /home/rich/.b* /home/rich/badtest
do
   if [ -d "$file" ]
   then
      echo "$file is a directory"
   elif [ -f "$file" ]
   then
      echo "$file is a file"
   else
      echo "$file doesn't exist"
   fi
done
$ ./test7
/home/rich/.backup.timestamp is a file
/home/rich/.bash_history is a file
/home/rich/.bash_logout is a file
/home/rich/.bash_profile is a file
/home/rich/.bashrc is a file
/home/rich/badtest doesn't exist
$
```

for 语句首先使用文件通配迭代通配符生成的文件列表，然后迭代列表中的下一个文件。可以结合列表中任意数目的通配符项进行迭代。

警告：可以在列表数据中输入任何值，即使文件或目录不存在，for 语句会试图处理放在列表中的任何东西。但遇到文件和目录时，这可能是一个问题。无法知道是否在迭代一个不存在的目录：在试图处理文件或目录之前检测一下通常是个好方法。

## 10.2 C 式的 for 命令

如果使用 C 语言编程进行过编程，可能会对 bash shell 使用 for 命令的方法感到惊讶。在 C 语言中，for 循环通常定义一个变量，该变量在每次迭代过程中自动改变值。通常程序员将该变量用作计数器，每次迭代计数器要么增加一个要么减少一个。bash 的 for 命令也能提供这种功能。本节将介绍如何在 bash shell 脚本中使用 C 式的 for 命令。

### 10.2.1 C 语言中的 for 命令

C 语言中的 for 命令有一种特定的方法指定一个变量，即必需保持 true 值用于继续迭代的条件，和一种每次迭代改变变量的方法。当特定的条件变为 false，for 循环结束。条件式使用标准的数学符号定义。以 C 语言代码为例：

```
for (i = 0; i < 10; i++)
{
   printf("The next number is %d\n", i);
}
```

该代码生成一个简单的迭代循环，变量 i 用作计数器。第一部分将默认值赋值给变量。中间部分定义循环迭代的条件。当定义的条件变为 false 时，for 循环停止迭代。最后一部分定义迭代过程。每次迭代之后，将执行在最后一部分中定义的表达式。在本例中，变量 i 每次迭代之后都增加 1。

bash shell 也支持看起来与 C 式的 for 循环类似的 for 循环版本，尽管二者有些细微的差别，其中几点差别连程序员也很混淆。这是 bash 中 C 式的 for 循环的基本格式：

```
for (( variable assignment ; condition ; iteration process ))
```

对于 bash shell 脚本程序员来说，C 式 for 循环的格式能令人混淆，因为它使用 C 式的变量引用而不是 shell 式的变量引用。这里是 C 式的 for 命令：

```
for (( a=1; a<10; a++ ))
```

注意，有几项不遵循标准 bash shell 的 for 方法：

- 变量的赋值可以包含空格；
- 条件中的变量不以美元符号做前缀；
- 迭代处理式不使用 expr 命令格式。

shell 开发者创建了这种更加接近 C 式 for 命令的格式。尽管这对于 C 语言程序员来说非常好，但它却使 shell 编程专家感到恐慌。在脚本中使用 C 式的 for 循环要谨慎。

这里是一个在 bash shell 程序中使用 C 式 for 循环的例子：

```
$ cat test8
#!/bin/bash
# testing the C-style for loop

for (( i=1; i <= 10; i++ ))
do
   echo "The next number is $i"
done
$ ./test8
The next number is 1
The next number is 2
The next number is 3
The next number is 4
The next number is 5
The next number is 6
The next number is 7
The next number is 8
The next number is 9
The next number is 10
$
```

for 循环使用在 for 循环中定义的变量迭代命令（在本示例中是字母 i）。在每次迭代中，变量 $i 都包含 for 循环中分配的值。每次迭代之后，循环迭代过程就会应用到变量，在本示例中，变量增加 1。

### 10.2.2 使用多个变量

C 式的 for 命令也允许使用多个变量迭代。循环分别处理每个变量，允许为每个变量定义不同的迭代过程。虽然可以使用多个变量，但只可以在 for 循环中定义一个条件：

```
$ cat test9
#!/bin/bash
# multiple variables

for (( a=1, b=10; a <= 10; a++, b-- ))
do
   echo "$a - $b"
done
$ ./test9
```

```
1 - 10
2 - 9
3 - 8
4 - 7
5 - 6
6 - 5
7 - 4
8 - 3
9 - 2
10 - 1
$
```

变量 a 和 b 初始化为不同的值，也定义了不同的迭代处理。每次迭代循环增加变量 a 的值而减少变量 b 的值。

## 10.3 while 命令

while 命令有点像 if-then 语句和 for 循环间的结合。while 命令允许定义要测试的命令，然后只要定义的测试命令返回 0 退出状态代码，就循环一组命令。它在每次迭代开始时检查测试命令。测试命令返回非零退出状态代码时，while 命令停止执行命令集。

### 10.3.1 while 的基本格式

while 命令的格式是：

```
while test command
do
 other commands
done
```

在 while 命令中定义的 test 命令与在 if-then 语句中定义的格式一样（参阅第 9 章）。就像在 if-then 语句中一样，可以使用任何常规的 shell 命令，或者可以使用 test 命令检测条件（如变量值）。

while 命令的关键是指定的 test 命令的退出状态必须根据循环中命令的运行情况改变。如果退出状态不改变改变，while 循环就会陷入无限的循环中。

最常见的情况是使用 test 命令括号来检验用于循环命令的 shell 变量的值：

```
$ cat test10
#!/bin/bash
# while command test

var1=10
while [ $var1 -gt 0 ]
do
   echo $var1
   var1=$[ $var1 - 1 ]
done
$ ./test10
10
9
8
7
6
5
4
3
2
1
$
```

while 命令定义测试条件来检验每次迭代：

```
while [ $var1 -gt 0 ]
```

只要测试条件为 true，while 命令会继续循环定义的命令。在这些命令中，用于检测条件的变量一定要改变，否则就会无穷循环。该例中使用 shell 算术将变量值减 1：

```
var1=$[ $var1 -1 ]
```

测试条件不再为 true 时，while 循环停止。

### 10.3.2 使用多条测试命令

有点奇怪的情况是，while 命令允许在 while 语句行定义多条 test 命令。只有最后一条测试命令的退出状态是用来决定循环是何时停止的。如果不仔细的话，就会生成有趣的结果，这是一个例子：

```
$ cat test11
#!/bin/bash
# testing a multicommand while loop

var1=10

while echo $var1
      [ $var1 -ge 0 ]
do
   echo "This is inside the loop"
   var1=$[ $var1 - 1 ]
done
$ ./test11
10
This is inside the loop
9
This is inside the loop
8
This is inside the loop
7
This is inside the loop
6
This is inside the loop
5
This is inside the loop
4
This is inside the loop
3
This is inside the loop
2
This is inside the loop
1
This is inside the loop
0
This is inside the loop
-1
$
```

仔细看看在本例中发生了什么。在 while 语句中定义了两个测试命令：

```
While echo $var1
      [ $var1 -ge 0 ]
```

第一条测试命令简单地显示了变量 var1 的当前值。第二条命令使用测试命令决定变量 var1 的值。一个 echo 语句在循环中列出了一条简单的信息，表明循环在进行。注意运行本例时输出是如何结束的：

```
This is inside the loop
-1
$
```

当变量 var1 的值等于零时，while 循环执行 echo 语句。接下来，用于下一次迭代的测试条件被执行。执行 echo 测试命令，显示变量 var1 的值，它现在小于零。直到 shell 执行 test 测试命令，

while 循环才终止。

这表明，在多命令 while 语句中，所有的测试命令在每次迭代中都执行，包含测试命令失败的最后一次迭代。注意这一点。

## 10.4 until 命令

until 命令刚好与 while 命令相反。until 命令需要制定一条测试命令，这条命令通常产生一个非 0 的退出状态。只要测试命令的退出状态非 0，bash shell 就会执行列在循环当中的命令。一旦测试条件返回 0 退出状态，循环停止。

如您所料，until 命令的格式是：

```
until test commands
do
   other commands
done
```

与 while 命令相似，可以在 until 命令语句中使用多条测试命令。只有最后一条命令的退出状态能够决定 bash shell 是否执行其他定义的命令。

下面是一个使用 until 命令的例子：

```
$ cat test12
#!/bin/bash
# using the until command

var1=100

until [ $var1 -eq 0 ]
do
   echo $var1
   var1=$[ $var1 - 25 ]
done
$ ./test12
100
75
50
25
$
```

这个例子测试了变量 var1 以决定 until 循环何时停止。一旦变量的值等于 0，until 命令就停止循环。在 until 命令中使用多条测试命令时，要注意的问题与使用 while 命令要注意的问题相同：

```
$ cat test13
#!/bin/bash
# using the until command

var1=100

until echo $var1
      [ $var1 -eq 0 ]
do
   echo Inside the loop: $var1
   var1=$[ $var1 - 25 ]
done
$ ./test13
100
Inside the loop: 100
75
Inside the loop: 75
50
Inside the loop: 50
```

```
25
Inside the loop: 25
0
$
```

shell 执行指定的测试命令，并且只有当最后一条命令为 true 时才停止。

## 10.5　嵌套循环

一条循环语句可以在循环中使用任何类型的命令，包括其他循环命令。这称为**嵌套循环**。使用嵌套循环时要谨慎，因为是在一个迭代内部执行另一个迭代，它增加了正在运行的命令的运行次数。如果在脚本中不注意就会引发问题。

这是一个在 for 循环中嵌套另一个 for 循环的简单示例：

```
$ cat test14
#!/bin/bash
# nesting for loops

for (( a = 1; a <= 3; a++ ))
do
   echo "Starting loop $a:"
   for (( b = 1; b <= 3; b++ ))
   do
      echo " Inside loop: $b"
   done
done
$ ./test14
Starting loop 1:
   Inside loop: 1
   Inside loop: 2
   Inside loop: 3
Starting loop 2:
   Inside loop: 1
   Inside loop: 2
   Inside loop: 3
Starting loop 3:
   Inside loop: 1
   Inside loop: 2
   Inside loop: 3
$
```

嵌套循环（也称为内循环）对每次外部循环迭代都迭代所有的值。注意，两个循环的 do 命令和 done 命令之间没区别。bash shell 知道，当第一条 done 命令执行时，它指的是内循环而不是外循环。

这也同样适用与混合循环命令时，例如，将 for 循环放在 while 循环内：

```
$ cat test15
#!/bin/bash
# placing a for loop inside a while loop

var1=5

while [ $var1 -ge 0 ]
do
   echo "Outer loop: $var1"
   for (( var2 = 1; $var2 < 3; var2++ ))
   do
      var3=$[ $var1 * $var2 ]
      echo " Inner loop: $var1 * $var2 = $var3"
   done
   var1=$[ $var1 - 1 ]
done
```

```
$ ./test15
Outer loop: 5
  Inner loop: 5 * 1 = 5
  Inner loop: 5 * 2 = 10
Outer loop: 4
  Inner loop: 4 * 1 = 4
  Inner loop: 4 * 2 = 8
Outer loop: 3
  Inner loop: 3 * 1 = 3
  Inner loop: 3 * 2 = 6
Outer loop: 2
  Inner loop: 2 * 1 = 2
  Inner loop: 2 * 2 = 4
Outer loop: 1
  Inner loop: 1 * 1 = 1
  Inner loop: 1 * 2 = 2
Outer loop: 0
  Inner loop: 0 * 1 = 0
  Inner loop: 0 * 2 = 0
$
```

同样，shell 能够区分内部 for 循环的 do 命令和 done 命令之间的命令和外部 while 循环中相同的命令。

如果真的想测试一下大脑，甚至可以将 until 和 while 循环结合起来：

```
$ cat test16
#!/bin/bash
# using until and while loops

var1=3

until [ $var1 -eq 0 ]
do
   echo "Outer loop: $var1"
   var2=1
   while [ $var2 -lt 5 ]
   do
      var3='echo "scale=4; $var1 / $var2" | bc'
      echo " Inner loop: $var1 / $var2 = $var3"
      var2=$[ $var2 + 1 ]
   done
   var1=$[ $var1 - 1 ]
done
$ ./test16
Outer loop: 3
   Inner loop: 3 / 1 = 3.0000
   Inner loop: 3 / 2 = 1.5000
   Inner loop: 3 / 3 = 1.0000
   Inner loop: 3 / 4 = .7500
Outer loop: 2
   Inner loop: 2 / 1 = 2.0000
   Inner loop: 2 / 2 = 1.0000
   Inner loop: 2 / 3 = .6666
   Inner loop: 2 / 4 = .5000
Outer loop: 1
   Inner loop: 1 / 1 = 1.0000
   Inner loop: 1 / 2 = .5000
   Inner loop: 1 / 3 = .3333
   Inner loop: 1 / 4 = .2500
$
```

外部 until 循环以值 3 开始直到变量值等于 0。内部 while 循环以值 1 开始，只要值小于 5 就继续。每一次循环必需更改测试条件的值，否则循环就会无限进行。

## 10.6 文件数据的循环

通常需要迭代存储在文件内部的项。这就需要结合两种介绍过的技术：

- 使用嵌套循环；
- 更改环境变量 IFS。

通过更改环境变量 IFS，可以迫使 for 命令将文件中的每行作为单独的一项来处理，即使数据包含空格。提取了文件中的个别行之后，您可能还必须再循环以提取其包含的数据。

这个问题的典型示例是在 /etc/passwd 文件中处理数据。这就需要逐行迭代文件 /etc/passwd，然后将变量 IFS 的值更改为冒号，以便能将每行中的单个组成部分分离出来。

下面是一个示例：

```
#!/bin/bash
# changing the IFS value
IFS.OLD=$IFS
IFS=$'\n'
for entry in 'cat /etc/passwd'
do
   echo "Values in $entry -"
   IFS=:
   for value in $entry
   do
      echo " $value"
   done
done
$
```

该脚本使用两个不同的 IFS 值解析数据。第一个 IFS 值解析文件/etc/passwd 中的单独行。接下来的内部 for 循环将 IFS 值更改为冒号，它允许解析文件/etc/passwd 行中的单独值。

运行该脚本时，会得到下面的输出：

```
Values in rich:x:501:501:Rich Blum:/home/rich:/bin/bash -
   rich
   x
   501
   501
   Rich Blum
   /home/rich
   /bin/bash
Values in katie:x:502:502:Katie Blum:/home/katie:/bin/bash -
   katie
   x
   506
   509
   Katie Blum
   /home/katie
   /bin/bash
```

内部循环解析/etc/passwd 项的每个单独值。这也是处理用逗号分隔的数据的一种好方法，也是导入电子表数据的一种常用方法。

## 10.7 控制循环

您可能会认为循环开始之后就不得不继续循环直到它完成所有的迭代。其实不然。有两条命令可以帮助我们控制循环内部的发生的事情：

- break 命令；

✦ continue 命令。

对于如何控制循环操作，每条命令有不同的使用方法。下面几节将介绍如何使用这两个命令来控制循环操作。

## 10.7.1 break 命令

break 命令是在处理过程中跳出循环的一种简单方法。可以使用 break 命令退出任何类型的循环，包括 while 循环和 until 循环。

有几种可以使用 break 命令的情况。这部分依次说明每一种方法。

**1. 跳出单循环**

shell 执行 break 命令时，它试图跳出当前正在处理的循环：

```
$ cat test17
#!/bin/bash
# breaking out of a for loop

for var1 in 1 2 3 4 5 6 7 8 9 10
do
   if [ $var1 -eq 5 ]
   then
      break
   fi
   echo "Iteration number: $var1"
done
echo "The for loop is completed"
$ ./test17
Iteration number: 1
Iteration number: 2
Iteration number: 3
Iteration number: 4
The for loop is completed
$
```

for 循环通常迭代列表中指定的所有值。然而，当 if-then 条件满足时，shell 执行 break 命令，终止 for 循环。

这种技术页同样适用于 while 循环和 until 循环：

```
$ cat test18
#!/bin/bash
# breaking out of a while loop

var1=1

while [ $var1 -lt 10 ]
do
   if [ $var1 -eq 5 ]
   then
      break
   fi
   echo "Iteration: $var1"
   var1=$[ $var1 + 1 ]
done
echo "The while loop is completed"
$ ./test18
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
The while loop is completed
$
```

当 if-then 条件满足时，执行 break 命令，while 循环终止。

### 2. 跳出内循环

使用多循环时 break 命令自动终止您所在的最里面的内部循环：

```
$ cat test19
#!/bin/bash
# breaking out of an inner loop

for (( a = 1; a < 4; a++ ))
do
   echo "Outer loop: $a"
   for (( b = 1; b < 100; b++ ))
   do
      if [ $b -eq 5 ]
      then
         break
      fi
      echo " Inner loop: $b"
   done
done
$ ./test19
Outer loop: 1
   Inner loop: 1
   Inner loop: 2
   Inner loop: 3
   Inner loop: 4
Outer loop: 2
   Inner loop: 1
   Inner loop: 2
   Inner loop: 3
   Inner loop: 4
Outer loop: 3
   Inner loop: 1
   Inner loop: 2
   Inner loop: 3
   Inner loop: 4
$
```

内循环的 for 语句规定迭代到变量 b 的值等于 100。然而，内循环的 if-then 语句指定变量 b 的值等于 5 时，执行 break 命令。注意，即使内循环被 break 命令终止，外循环会继续按指定进行。

### 3. 跳出外循环

可能有时处于内循环但需要停止外循环。break 命令包括单独的命令行参数值：

```
break n
```

*n* 表明要跳出的循环级别。默认情况下，*n* 是 1，代表跳出当前循环。如果将 *n* 设置为 2，break 命令将停止外循环的下一级循环：

```
$ cat test20
#!/bin/bash
# breaking out of an outer loop

for (( a = 1; a < 4; a++ ))
do
   echo "Outer loop: $a"
   for (( b = 1; b < 100; b++ ))
   do
      if [ $b -gt 4 ]
      then
         break 2
      fi
      echo " Inner loop: $b"
   done
```

```
done
$ ./test20
Outer loop: 1
   Inner loop: 1
   Inner loop: 2
   Inner loop: 3
   Inner loop: 4
$
```

现在，当 shell 执行 break 命令时外循环停止。

### 10.7.2 continue 命令

continue 命令是一种提前停止循环内命令，而不完全终止循环的方法。这就允许在循环中设置 shell 不执行命令的条件。下面是一个在 for 循环中使用 continue 命令的简单示例：

```
$ cat test21
#!/bin/bash
# using the continue command

for (( var1 = 1; var1 < 15; var1++ ))
do
   if [ $var1 -gt 5 ] && [ $var1 -lt 10 ]
   then
      continue
   fi
   echo "Iteration number: $var1"
done
$ ./test21
Iteration number: 1
Iteration number: 2
Iteration number: 3
Iteration number: 4
Iteration number: 5
Iteration number: 10
Iteration number: 11
Iteration number: 12
Iteration number: 13
Iteration number: 14
$
```

if-then 语句的条件（值大于 5 小于 10）满足时，shell 执行 continue 命令，跳过该循环中余下的命令，但是循环继续进行。如果 if-then 语句的条件不再满足，那么循环就恢复正常。

在 while 循环和 until 循环中可以使用 continue 命令，但是要特别谨慎。记住，当 shell 执行 continue 命令时，它跳过余下的命令。如果您正在其中的一个条件中增加测试条件变量，就会发生糟糕的事情：

```
$ cat badtest3
#!/bin/bash
# improperly using the continue command in a while loop

var1=0
while echo "while iteration: $var1"
      [ $var1 -lt 15 ]
do
   if [ $var1 -gt 5 ] && [ $var1 -lt 10 ]
   then
      continue
   fi
   echo " Inside iteration number: $var1"
   var1=$[ $var1 + 1 ]
done
$ ./badtest3 | more
```

```
while iteration: 0
   Inside iteration number: 0
while iteration: 1
   Inside iteration number: 1
while iteration: 2
   Inside iteration number: 2
while iteration: 3
   Inside iteration number: 3
while iteration: 4
   Inside iteration number: 4
while iteration: 5
   Inside iteration number: 5
while iteration: 6
while iteration: 6
while iteration: 6
while iteration: 6
while iteration: 6
while iteration: 6
while iteration: 6
while iteration: 6
while iteration: 6
while iteration: 6
while iteration: 6
$
```

要确保将该脚本的输出重定向给 more 命令，这样才能够分屏查看。一切都运行正常，直到 if-then 条件满足，shell 执行 continue 命令。当 shell 执行 continue 命令时，它跳过 while 循环中余下的命令。不幸的是，那就是我增加计数器变量的地方，这个计数器变量用在 while 测试命令中测试。那就意味着变量没有增加，正如持续不断的输出显示所示。

与使用 break 命令一样，continue 命令也允许使用命令行参数指定要继续的循环级别：

```
continue n
```

其中，*n* 定义了要继续的循环级别。下面是一个继续外部 for 循环的示例：

```
$ cat test22
#!/bin/bash
# continuing an outer loop

for (( a = 1; a <= 5; a++ ))
do
   echo "Iteration $a:"
   for (( b = 1; b < 3; b++ ))
   do
      if [ $a -gt 2 ] && [ $a -lt 4 ]
      then
         continue 2
      fi
      var3=$[ $a * $b ]
      echo " The result of $a * $b is $var3"
   done
done
$ ./test22
Iteration 1:
   The result of 1 * 1 is 1
   The result of 1 * 2 is 2
Iteration 2:
   The result of 2 * 1 is 2
   The result of 2 * 2 is 4
Iteration 3:
Iteration 4:
   The result of 4 * 1 is 4
   The result of 4 * 2 is 8
Iteration 5:
```

```
    The result of 5 * 1 is 5
    The result of 5 * 2 is 10
$
```

if-then 语句：

```
if [ $a -gt 2 ] && [ $a -lt 4 ]
      then
         continue 2
      fi
```

使用 continue 命令停止处理循环内部的命令，但是继续外部循环。注意在脚本输出中，迭代的值为 3 时不处理任何内循环语句，因为 continue 命令停止了处理，但是继续外循环处理。

## 10.8 处理循环的输出

最后，可以在 shell 脚本中使用管道或者重定向循环输出结果。可以通过在 done 命令的末尾添加处理命令来做到这一点：

```
for file in /home/rich/*
do
   if [ -d "$file" ]
   then
      echo "$file is a directory"
   elif
      echo "$file is a file"
   fi
done > output.txt
```

shell 将 for 命令的结果重定向到文件 output.txt，而不是显示在监视器上。

下面是一个将 for 命令的输出结果重定向到文件的例子：

```
$ cat test23
#!/bin/bash
# redirecting the for output to a file

for (( a = 1; a < 10; a++ ))
do
   echo "The number is $a"
done > test23.txt
echo "The command is finished."
$ ./test23
The command is finished.
$ cat test23.txt
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9
$
```

shell 创建了文件 test23.txt，只将 for 命令的输出重定向到文件。但是，shell 正常显示了 for 命令之后的 echo 语句。

同样的技术也可用于将循环的输出管道传送给其他命令：

```
$ cat test24
#!/bin/bash
# piping a loop to another command

for state in "North Dakota" Connecticut Illinois Alabama Tennessee
```

```
do
   echo "$state is the next place to go"
done | sort
echo "This completes our travels"
$ ./test24
Alabama is the next place to go
Connecticut is the next place to go
Illinois is the next place to go
North Dakota is the next place to go
Tennessee is the next place to go
This completes our travels
$
```

state值没有在for命令列表中以任何特定的顺序中列出。for命令的输出结果被管道传送给sort命令，它会改变for命令输出的顺序。运行脚本确实显示输出在脚本中进行了恰当的排序。

## 10.9 小结

循环是编程不可缺少的一部分。bash shell提供了3种不同的循环命令，可以用在脚本中。for命令能够迭代值列表，无论是命令行内部提供的还是包含在变量中的，或者使用文件通配从通配符中提取文件和目录的名称。

while命令提供一种基于命令的条件进行循环的一种方法，可以使用普通命令或测试命令来测试变量条件。只要命令（或条件）生成零退出状态，while循环将继续重复指定的命令集。

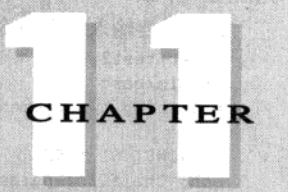
until命令也提供一种重复命令的方法，但是它是基于一条命令（或者条件）生成非零退出状态。这个特性允许我们在重复停止之前设定一个必需满足的条件。

可以在shell脚本中结合循环生成多层循环。bash shell提供continue命令和break命令，允许我们根据循环内的不同值改变正常的循环处理流。

bash shell也允许我们使用标准命令重定向和管道传送更改循环的输出。可以使用重定向将循环的输出重定向到文件，或者将循环的输出管道传送给另一条命令。这提供了很多特性，可以使用这些特性控制shell脚本的执行。

下一章将讨论如何与shell脚本用户进行交互。通常shell脚本不完全是齐全的。当运行它们时，需要提供给它们某种外部数据。下一章将介绍几种不同的方法，使用这些方法可以为脚本提供实时的数据进行处理。

# 第 11 章
# 处理用户输入




本章内容：

- 使用命令行参数；
- 设置选项；
- 获取用户输入。

前面已经学习了编写同 Linux 系统上的数据、变量和文件进行交互的脚本的方法。有时编写的脚本需要能够与运行脚本的人员进行交互。bash shell 提供了一些不同的方法来从用户处获取数据，这些方法包括命令行参数（添加在命令后面的数据值）、命令行选项（修改命令行为的单字符值）和直接读取键盘输入。本章将讨论如何将这些方法集成到 bash shell 脚本中，以便从运行脚本的人员处获取数据。

## 11.1 命令行参数

向 shell 脚本传递数据的最基本方式是使用**命令行参数**（command line parameters）。使用命令行参数可以在执行脚本时向命令行中添加数据值。

```
$ ./addem 10 30
```

这个例子向脚本 addem 传递了两个命令行参数（10 和 30）。脚本将使用特殊变量来处理命令行参数。下面几节将介绍在 bash shell 脚本中如何使用命令行参数。

### 11.1.1 读取参数

bash shell 将在命令行中输入的所有参数赋值给一些特殊变量，这些变量称为**位置参数**（positional parameter）。其中还包括 shell 执行的程序的名称。位置参数通过标准数字表示，其中$0 为程序名称，$1 为第一个参数，$2 为第二个参数，依此类推，直到$9 为第九个参数。

下面的简单示例展示了在 shell 脚本中使用了一个命令行参数：

```
$ cat test1
#!/bin/bash
# using one command line parameter

factorial=1
for (( number = 1; number <= $1 ; number++ ))
do
   factorial=$[ $factorial * $number ]
done
echo The factorial of $1 is $factorial
$ ./test1 5
The factorial of 5 is 120
$
```

可以在 shell 脚本中像使用其他变量一样使用变量$1。shell 脚本自动将命令行参数赋值给变量$1。无需用户干预。

如果需要输入更多命令行参数，那么必须在命令行中使用空格分隔每个参数：

```
$ cat test2
#!/bin/bash
# testing two command line parameters

total=$[ $1 * $2 ]
echo The first paramerer is $1.
echo The second parameter is $2.
echo The total value is $total.
$ ./test2 2 5
The first paramerer is 2.
The second parameter is 5.
The total value is 10.
$
```

shell 将每个参数赋值给适当的变量。

在本例中，使用的两个命令行参数都是数值。也可以在命令行中使用文本字符串。

```
$ cat test3
#!/bin/bash
# testing string parameters

echo Hello $1, glad to meet you.
$ ./test3 Rich
Hello Rich, glad to meet you.
$
```

shell 将在命令行中输入的字符串传递给脚本。但是如果要传递一个包含空格的文本字符串，则会出现问题。

```
$ ./test3 Rich Blum
Hello Rich, glad to meet you.
$
```

记住，每个参数都是通过空格分隔的，所以 shell 会将空格解析为两个值的分隔标志。要想在参数值中包含空格，就必须使用引号（单引号和双引号均可）：

```
$ ./test3 'Rich Blum'
Hello Rich Blum, glad to meet you.
$ ./test3 "Rich Blum"
Hello Rich Blum, glad to meet you.
$
```

注意，引号不作为数据的一部分，它们只是界定数据的起始和结束。

如果脚本需要的命令行参数多于 9 个，那么可以继续在命令行中添加命令行参数，但是变量名称会稍有变化。在第 9 个变量之后，必须使用大括号将变量括起来，如${10}。示例如下：

```
$ cat test4
#!/bin/bash
# handling lots of parameters

total=$[ ${10} * ${11} ]
echo The tenth parameter is ${10}
echo The eleventh parameter is ${11}
echo The total is $total
$ ./test4 1 2 3 4 5 6 7 8 9 10 11 12
The tenth parameter is 10
The eleventh parameter is 11
The total is 110
$
```

通过这一方法，可以根据需要为脚本添加任意数量的命令行参数。

### 11.1.2 读取程序名称

使用参数$0 可以确定 shell 从命令行启动的程序的名称。在编写具有多个功能的工具时，这种方法非常方便。但是，也有可能会遇到一些小问题。请看下面的小例子：

```
$ cat test5
#!/bin/bash
# testing the $0 parameter

echo The command entered is: $0
$ ./test5
The command entered is: ./test5
$ /home/rich/test5
The command entered is: /home/rich/test5
$
```

传递给变量$0 的字符串实际上是程序的完整路径，而不仅仅是程序的名称。

如果希望编写的脚本能够根据通过命令行运行的脚本的名称执行不同功能，必须进行一些额外的处理。需要去除通过命令行运行脚本时使用的所有路径。

幸运的是，有一个便利的小命令可以用来获取程序名称。basename 命令只返回程序名称，不带路径。将示例脚本进行如下修改，并查看其运行结果：

```
$ cat test5b
#!/bin/bash
# using basename with the $0 parameter

name='basename $0'
echo The command entered is: $name
$ ./test5b
The command entered is: test5b
$ /home/rich/test5b
The command entered is: test5b
$
```

现在好多了。可以使用这个方法编写基于使用的脚本的名称来执行不同功能的脚本。如下面的例子所示：

```
$ cat test6
#!/bin/bash
# testing a multi-function script

name='basename $0'
if [ $name = "addem" ]
then
   total=$[ $1 + $2 ]
elif [ $name = "multem" ]
then
   total=$[ $1 * $2 ]
fi
echo The calculated value is $total
$ chmod u+x test6
$ cp test6 addem
$ ln -s test6 multem
$ ls -l
-rwxr--r--    1 rich        rich          211 Oct 15 18:00 addem*
lrwxrwxrwx    1 rich        rich            5 Oct 15 18:01 multem -> test6*
-rwxr--r--    1 rich        rich          211 Oct 15 18:00 test6*
$ ./addem 2 5
The calculated value is 7
$ ./multem 2 5
The calculated value is 10
$
```

该示例从 test6 代码创建了两个文件名，一个通过复制文件创建，另一个通过使用链接创建。两种情况下，脚本都是先判断脚本的 basename，然后根据 basename 执行适当的函数。

### 11.1.3　测试参数

在 shell 脚本中使用命令行参数时要非常小心。如果执行脚本时缺少参数，会产生很糟的结果：

```
$ ./addem 2
./addem: line 8: 2 + : syntax error: operand expected (error
token is " ")
The calculated value is
$
```

当脚本认为参数变量中包含数据，但实际上其中并没有数据时，极有可能会生成脚本错误消息。这种编写脚本方法很不好。好的方法是对参数进行检查来保证使用参数前确实存在数据：

```
$ cat test7
#!/bin/bash
# testing parameters before use

if [ -n "$1" ]
then
   echo Hello $1, glad to meet you.
else
   echo "Sorry, you didn't identify yourself."
fi
$ ./test7 Rich
Hello Rich, glad to meet you.
$ ./test7
Sorry, you didn't identify yourself.
$
```

在本例中，在 test 命令中使用了-n 参数来检查命令行参数中是否存在数据。下一节将介绍检查命令行参数的另一个方法。

## 11.2　特殊的参数变量

在 bash shell 中有一些特殊的变量用于跟踪命令行参数。本节将介绍这些变量和它们的使用方法。

### 11.2.1　参数计数

在上一节中学习过，在脚本中使用命令行参数前对它们进行验证是非常好的习惯。对于使用多个命令行参数的脚本，这是件很乏味的事情。

可以不必测试每个参数，而只对命令行中输入的参数个数进行计数。为此，bash shell 提供了一个特殊变量。

特殊变量$#中存储执行脚本时包含的命令行参数的个数。在脚本中的任意位置都可以使用这个特殊变量，就像使用普通变量一样：

```
$ cat test8
#!/bin/bash
# getting the number of parameters

echo There were $# parameters supplied.
$ ./test8
There were 0 parameters supplied.
$ ./test8 1 2 3 4 5
There were 5 parameters supplied.
$ ./test8 1 2 3 4 5 6 7 8 9 10
There were 10 parameters supplied.
$ ./test8 "Rich Blum"
```

```
There were 1 parameters supplied.
$
```

现在可以在使用参数前测试现有的参数个数：

```
$ cat test9
#!/bin/bash
# testing parameters

if [ $# -ne 2 ]
then
   echo Usage: test9 a b
else
   total=$[ $1 + $2 ]
   echo The total is $total
fi
$ ./test9
Usage: test9 a b
$ ./test9 10
Usage: test9 a b
$ ./test9 10 15
The total is 25
$ ./test9 10 15 20
Usage: test9 a b
$
```

上面的 if-then 语句使用 test 命令对命令行中提供的参数个数执行一个数值测试。如果提供的参数个数不正确，可以打印一个表示脚本的正确使用方式的错误消息。

这个变量还提供了一个很酷的方法来获取命令行中的最后一个参数，使用这种方法时不必知道使用的参数个数。但是，需要使用一点小技巧。

稍微思考一下，可能会认为因为$#变量包含参数个数值，所以使用变量${$#}就可以得到最后一个命令行参数值。试一下，看会得到什么结果：

```
$ cat badtest1
#!/bin/bash
# testing grabbing last parameter

echo The last parameter was ${$#}
$ ./badtest1 10
The last parameter was 15354
$
```

发生了什么呢？显然，产生了错误。结果表明不能在大括号中使用美元符号，必须将美元符号替换为感叹号。很怪异，但确实有用：

```
$ cat test10
#!/bin/bash
# grabbing the last parameter

params=$#
echo The last parameter is $params
echo The last paramerer is ${!#}
$ ./test10 1 2 3 4 5
The last parameter is 5
The last paramerer is 5
$ ./test10
The last parameter is 0
The last parameter is ./test10
$
```

完美极了。这个测试还将$#变量值赋值给了变量 params，然后按特殊命令行参数变量的格式使用变量 params。$#与 params 这两种方式都有效。还要注意很重要的一点，当命令行中没有任何参数时，$#的值为零，变量 params 的值也为零，但变量${!#}将返回命令行中使用的脚本的名称。

### 11.2.2　获取所有数据

有些情况下，会需要获取命令行中提供的所有参数，并对它们进行迭代。此时，不必使用$#变量确定命令行中参数的个数，然后循环所有参数，而是可以使用其他一些特殊变量。

变量$*和$@为所有参数提供了“一站式购物”服务。这两个变量都是在一个变量中包含所有命令行参数。

变量$*将命令行中提供的所有参数作为一个单词处理。这个单词中包含出现在命令行中的每一个参数值。本质上，变量$*不是将参数视为多个对象，而是将它们看作一个参数。

而另一方面，变量$@将命令行中提供的所有参数作为同一个字符串中的多个单词处理。允许对其中的值进行迭代，分隔开所提供的不同参数。通常使用 for 命令来进行这样的迭代。

这两个变量的操作方式很容易产生混淆。如果只从字面值来看，看不出任何区别：

```
$ cat test11
#!/bin/bash
# testing $* and $@

echo "Using the \$* method: $*"
echo "Using the \$@ method: $@"
$ ./test11 rich barbara katie jessica
Using the $* method: rich barbara katie jessica
Using the $@ method: rich barbara katie jessica
$
```

表面看来，两个变量生成相同的输入，一次显示提供的所有命令行参数。

现在请再看一个例子，该例子将展示区别所在：

```
$ cat test12
#!/bin/bash
# testing $* and $@

count=1
for param in "$*"
do
   echo "\$* Parameter #$count = $param"
   count=$[ $count + 1 ]
done

count=1
for param in "$@"
do
   echo "\$@ Parameter #$count = $param"
   count=$[ $count + 1 ]
done
$ ./test12 rich barbara katie jessica
$* Parameter #1 = rich barbara katie jessica
$@ Parameter #1 = rich
$@ Parameter #2 = barbara
$@ Parameter #3 = katie
$@ Parameter #4 = jessica
$
```

现在明白不同之处了吧。通过使用 for 命令迭代特殊变量，可以看到这两个变量处理命令行参数的区别。变量$*将所有参数视为一个单词，而变量$@分别对待每个参数。这是迭代命令行参数的好方法。

## 11.3　移位

bash shell 工具集中的另一个工具是 shift 命令。bash shell 提供 shift 命令来帮助操作命令行参数。顾名思义，shift 命令能够改变命令行参数的相对位置。

使用 shift 命令时，默认将每个参数变量左移一个位置。于是，变量$3 的值移给变量$2，变量$2 的值移给变量$1，而变量$1 的值被丢弃（注意变量$0 的值和程序名称都保持不变）。

这是迭代命令行参数的另一种好办法，尤其是在不清楚参数数目的情况下。可以先对第一个参数进行操作，然后对参数进行一次左移，再对第一个参数进行操作。

请看下面的简短演示：

```
$ cat test13
#!/bin/bash
# demonstrating the shift command

count=1
while [ -n "$1" ]
do
   echo "Parameter #$count = $1"
   count=$[ $count + 1 ]
   shift
done
$ ./test13 rich barbara katie jessica
Parameter #1 = rich
Parameter #2 = barbara
Parameter #3 = katie
Parameter #4 = jessica
$
```

脚本执行一个 while 循环，测试第一个参数值的长度。当地一个参数值长度为 0 时，结束循环。测试第一个参数后，使用 shift 命令将所有参数左移一个位置。

当然，可以通过为 shift 命令提供一个参数来实行多位移变化。提供需要移动的位置数目即可：

```
$ cat test14
#!/bin/bash
# demonstrating a multi-position shift

echo "The original parameters: $*"
shift 2
echo "Here's the new first parameter: $1"
$ ./test14 1 2 3 4 5
The original parameters: 1 2 3 4 5
Here's the new first parameter: 3
$
```

通过在 shift 命令中设置值，可以很容易的跳过不需要的参数。

警告：使用 shift 命令时要谨慎。将某一参数移位掉后，该参数值就丢失了，不能恢复。

## 11.4 处理选项

如果您是从头开始阅读本书的话，就会看到一些 bash 命令提供参数和选项。**选项**是由破折号引导的单个字母，它更改命令的行为。本节将介绍 3 种在 shell 脚本中处理选项的方法。

### 11.4.1 找出选项

从表面看，命令行选项没有什么特殊之处。它们跟随在命令行中的脚本名称之后，与命令行参数一样。实际上，如果需要的话，可以像处理命令行参数一样处理命令行选项。

#### 1. 处理简单选项

在前面的 test13 脚本中，您看到了使用 shift 命令处理提供给脚本程序的命令行参数的方法。可以使用同样的方法处理命令行选项。

在抽取每个参数时，使用 case 语句判断参数是否符合选项格式：

```
$ cat test15
#!/bin/bash
# extracting command line options as parameters

while [ -n "$1" ]
do
   case "$1" in
   -a) echo "Found the -a option" ;;
   -b) echo "Found the -b option";;
   -c) echo "Found the -c option" ;;
   *) echo "$1 is not an option";;
   esac
   shift
done
$ ./test15 -a -b -c -d
Found the -a option
Found the -b option
Found the -c option
-d is not an option
$
```

case 语句检查每个参数是否为有效的选项。当找到一个选项时，就在 case 语句中运行适当的命令。无论选项在命令行中以什么顺序出现，这种方法都有效：

```
$ ./test15 -d -c -a
-d is not an option
Found the -c option
Found the -a option
$
```

case 语句在命令行参数中每找出一个选项，就会对选项进行处理。如果命令行中包含其他参数，可以在 case 语句的 catch-all 部分包含处理它们的命令。

**2. 从参数中分离选项**

执行 shell 脚本时经常会遇到既需要使用选项又需要使用参数的情况。在 Linux 中的标准方式是通过特殊字符码将二者分开，这个特殊字符码告诉脚本选项结束和普通参数开始的位置。

对于 Linux，这个特殊字符是双破折号（--）。shell 使用双破折号指示选项列表的结束。发现双破折号后，脚本就能够安全的将剩余的命令行参数作为参数而不是选项处理。

要想检查双破折号，只要在 case 语句中增加一个条目即可：

```
$ cat test16
#!/bin/bash
# extracting options and parameters

while [ -n "$1" ]
do
   case "$1" in
   -a) echo "Found the -a option" ;;
   -b) echo "Found the -b option";;
   -c) echo "Found the -c option" ;;
   --) shift
       break ;;
   *) echo "$1 is not an option";;
   esac
   shift
done

count=1
for param in $@
do
   echo "Parameter #$count: $param"
   count=$[ $count + 1 ]
done
$
```

这个脚本使用 break 命令使得遇到双破折号时从 while 循环中跳出。因为提前从循环中跳出，所以需要保证加入一个 shift 命令将双破折号从参数变量中丢弃。

第一个测试，尝试使用普通选项和参数集运行脚本：

```
$ ./test16 -c -a -b test1 test2 test3
Found the -c option
Found the -a option
Found the -b option
test1 is not an option
test2 is not an option
test3 is not an option
$
```

结果显示脚本将所有命令行参数作为选项处理。接下来，再进行一次测试，但此次使用双破折号将命令行中选项和参数分开：

```
$ ./test16 -c -a -b -- test1 test2 test3
Found the -c option
Found the -a option
Found the -b option
Parameter #1: test1
Parameter #2: test2
Parameter #3: test3
$
```

当脚本到达双破折号时，停止处理选项，并将其余的参数视为命令行参数。

### 3. 处理带值的选项

有些选项需要附加的参数值。这种情况下，命令行形式与下面的格式类似：

```
$ ./testing -a test1 -b -c -d test2
```

要求脚本必须能够监测命令行选项何时需要附加参数，并能够正确进行处理。下面是一个例子：

```
$ cat test17
#!/bin/bash
# extracting command line options and values

while [ -n "$1" ]
do
   case "$1" in
   -a) echo "Found the -a option";;
   -b) param="$2"
       echo "Found the -b option, with parameter value $param"
       shift 2;;
   -c) echo "Found the -c option";;
   --) shift
       break;;
   *) echo "$1 is not an option";;
   esac
   shift
done

count=1
for param in "$@"
do
   echo "Parameter #$count: $param"
   count=$[ $count + 1 ]
done
$ ./test17 -a -b test1 -d
Found the -a option
Found the -b option, with parameter value test1
-d is not an option
$
```

在本例中，case 语句定义了 3 个选项的处理。其中-b 选项还需要附加参数值。由于要处理的参

数是$1，所以知道附加参数值位于$2 中（因为所有参数在处理后都会被移位）。于是从变量$2 中抽取参数值。当然，因为这个选项占用了两个参数位置，所以需要设置 shift 命令来移位两个位置。

与基本特征一样，无论选项以何种顺序排列，这个过程都有效（只要记得为每个选项包含适当的选项参数）：

```
$ ./test17 -b test1 -a -d
Found the -b option, with parameter value test1
Found the -a option
-d is not an option
$
```

现在您拥有了在 shell 脚本中处理命令行选项的基本能力，但仍具有局限性。例如，如果试图在一个参数中结合多个选项，以前的方法就无法工作：

```
$ ./test17 -ac
-ac is not an option
$
```

在 Linux 中联合使用多个选项非常常见，而且如果想要使脚本人性化，也需要为用户提供这一特性。幸运的是，有一种处理选项的方法可供使用。

## 11.4.2、使用 getopt 命令

getopt 命令是个不错的工具，在处理命令行选项和参数时非常方便。它对命令行参数进行重新组织，使其更便于在脚本中解析。

### 1. 命令格式

getopt 命令可以接受任意形式的命令行选项和参数列表，并自动将这些选项和参数转换为适当的格式。命令格式如下：

```
getopt options optstring parameters
```

**选项字符串**（optstring）是处理的关键。它定义命令行中的有效选项字母。它还定义哪些选项字母需要参数值。

首先，在选项字符串中列出将在脚本中用到的每个命令行选项字母。然后，在每个需要参数值的选项字母后面放置一个冒号。getopt 命令根据定义的选项字符串解析提供的参数。

请看下面的 getopt 示例：

```
$ getopt ab:cd -a -b test1 -cd test2 test3
 -a -b test1 -c -d -- test2 test3
$
```

其中的选项字符串定义了四个有效选项字母，a、b、c 和 d。还定义选项字母 b 需要一个参数值。当执行 getopt 命令时，会监测提供的参数列表，然后基于提供的选项字符串对列表进行解析。注意，解析时自动将-cd 选项分割成两个不同的选项，并插入双破折号来分隔行中的额外参数。

如果指定的选项不包含在选项字符串中，getopt 命令会默认生成一个错误消息：

```
$ getopt ab:cd -a -b test1 -cde test2 test3
getopt: invalid option -- e
 -a -b test1 -c -d -- test2 test3
$
```

如果希望忽略这个错误消息，可以在命令中使用-q 选项：

```
$ getopt -q ab:cd -a -b test1 -cde test2 test3
 -a -b 'test1' -c -d -- 'test2' 'test3'
$
```

注意，getopt 命令选项必须列在选项字符串前面。现在就可以在脚本中使用这个命令来处理命令行选项。

### 2. 在脚本中使用 getopt

可以在脚本中使用 getopt 命令格式化为脚本输入的任意命令行选项或参数。这里要使用一点小

技巧。

这个技巧是将现有的命令行选项和参数替换为 getopt 命令生成的格式化形式。方法是使用 set 命令。

在第 5 章中查看 set 命令。set 命令可以处理 shell 中的各种变量。第 5 章介绍了如何使用 set 命令显示所有系统环境变量。

set 命令的一个选项是双破折号，表示将命令行参数变量替换为 set 命令的命令行中的值。

于是，这个技巧便是：将原始脚本命令行参数送给 getopt 命令，然后将 getopt 命令的输出送给 set 命令，以便将原始命令行参数替换为通过 getopt 格式化过的更精细的形式。如下所示：

```
set -- 'getopts -q ab:cd "$@"'
```

现在原始的命令行参数变量就被替换成了 getopt 命令的输出，getopt 命令将命令行参数进行了格式化。

使用这个方法，就可以编写处理命令行参数的脚本：

```
$ cat test18
#!/bin/bash
# extracting command line options and values with getopt

set -- 'getopt -q ab:c "$@"'
while [ -n "$1" ]
do
   case "$1" in
   -a) echo "Found the -a option" ;;
   -b) param="$2"
       echo "Found the -b option, with parameter value $param"
       shift ;;
   -c) echo "Found the -c option" ;;
   --) shift
       break;;
   *) echo "$1 is not an option";;
   esac
   shift
done

count=1
for param in "$@"
do
   echo "Parameter #$count: $param"
   count=$[ $count + 1 ]
done
$
```

可以看到这段脚本与 test17 中的脚本基本相同。唯一的变动是增加了 getopt 命令来格式化命令行参数。

现在执行带有复杂选项的脚本时，情况就好多了：

```
$ ./test18 -ac
Found the -a option
Found the -c option
$
```

当然，所有原有特性仍都正常工作：

```
$ ./test18 -a -b test1 -cd test2 test3 test4
Found the -a option
Found the -b option, with parameter value 'test1'
Found the -c option
Parameter #1: 'test2'
Parameter #2: 'test3'
Parameter #3: 'test4'
$
```

现在看起来非常完美。但是，还有一个小 bug 潜伏在 getopt 命令中。请看下面的例子：

```
$ ./test18 -a -b test1 -cd "test2 test3" test4
Found the -a option
Found the -b option, with parameter value 'test1'
Found the -c option
Parameter #1: 'test2
Parameter #2: test3'
Parameter #3: 'test4'
$
```

getopt 命令不能很好的处理带有空格的参数值。它将空格解析为参数分隔符，而不是将双引号引起来的两个值合并为一个参数。幸运的是，有另一种方法可以解决这个问题。

### 11.4.3　更高级的 getopts 命令

bash shell 中有一个 getopts 命令（注意它是复数形式）。它与它的堂兄弟 getopt 很相像，但具有一些扩展特性。

getopt 为在命令行中找到的所有待处理选项和参数生成一个输出，而与 getopt 不同，getopts 命令顺序的对现有的 shell 参数变量进行处理。

每调用一次 getopts，它只处理在命令中监测到的参数中的一个。处理完所有参数后，以大于零的退出状态退出。因此，getopts 非常适宜用在循环中解析所有命令行参数。

getopts 命令的格式为：

```
getopts optstring variable
```

其中 optstring 值与 getopt 命令中使用的相似。在选项字符串中列出有效选项字母，如果选项字母需要参数值后面加上一个冒号。如果要禁止输出错误消息，那么使选项字符串以冒号开头。getopts 命令将当前参数放在命令行中定义的 variable 中。

getopts 命令使用两个环境变量。环境变量 OPTARG 中包含需要参数值的选项要使用的值。环境变量 OPTIND 包含的值表示 getopts 停止处理时在参数列表中的位置。这样，处理完选项后可以继续处理其他命令行参数。

请看一个使用 getopts 命令的简单例子：

```
$ cat test19
#!/bin/bash
# simple demonstration of the getopts command

while getopts :ab:c opt
do
   case "$opt" in
   a) echo "Found the -a option" ;;
   b) echo "Found the -b option, with value $OPTARG";;
   c) echo "Found the -c option" ;;
   *) echo "Unknown option: $opt";;
   esac
done
$ ./test19 -ab test1 -c
Found the -a option
Found the -b option, with value test1
Found the -c option
$
```

while 语句定义 getopts 命令，指定要查找哪些命令行选项以及每次迭代中存储它们的变量名。

在本例中，可以看到 case 语句的使用有些特殊之处。当 getopts 命令解析命令行选项时，会将打头的破折号去掉，所以在 case 定义中不需要破折号。

getopts 命令具有许多很好的特性。首先，现在可以在参数值中包含空格：

```
$ ./test19 -b "test1 test2" -a
Found the -b option, with value test1 test2
```

```
Found the -a option
$
```

另一个好特性是选项字母和参数值中间可以没有空格：

```
$ ./test19 -abtest1
Found the -a option
Found the -b option, with value test1
$
```

getopts 命令正确的将-b 选项后面的 test1 值解析出来。getopts 命令还有一个好特性是将在命令行中找到的未定义的选项都绑定为单一的输出——问号：

```
$ ./test19 -d
Unknown option: ?
$ ./test19 -acde
Found the -a option
Found the -c option
Unknown option: ?
Unknown option: ?
$
```

任何没有在选项字符串值中定义的选项字母都将以问号形式传递给代码。

getopts 命令知道何时停止处理选项，将参数留给您处理。getopts 每个处理选项，环境变量 OPTIND 的值会增加 1。当到达 getopts 处理的末尾时，可以使用 shift 命令和 OPTIND 值进行操作来移动到参数：

```
$ cat test20
#!/bin/bash
# processing options and parameters with getopts

while getopts :ab:cd opt
do
   case "$opt" in
   a) echo "Found the -a option" ;;
   b) echo "Found the -b option, with value $OPTARG";;
   c) echo "Found the -c option";;
   d) echo "Found the -d option";;
   *) echo "Unknown option: $opt";;
   esac
done
shift $[ $OPTIND - 1 ]

count=1
for param in "$@"
do
   echo "Parameter $count: $param"
   count=$[ $count + 1 ]
done
$ ./test20 -a -b test1 -d test2 test3 test4
Found the -a option
Found the -b option, with value test1
Found the -d option
Parameter 1: test2
Parameter 2: test3
Parameter 3: test4
$
```

现在拥有了可以用于所有 shell 脚本的功能完整的命令行选项和参数处理工具。

## 11.5 标准化选项

创建 shell 脚本时，您显然拥有对 shell 脚本的控制权。完全由您决定选择使用哪些字母选项，以及如何使用它们。

但是，在 Linux 世界中有一些字母选项具有某种标准含义。如果在 shell 脚本中使用这些选项，会使脚本变得更加便于使用。

表 11-1 中列出了 Linux 中使用的一些命令行选项的常用含义。

表 11-1　　常用的 Linux 命令行选项

| 选项 | 描述 | 选项 | 描述 |
|---|---|---|---|
| -a | 显示所有对象 | -n | 使用非交互式（批量）模式 |
| -c | 生成计数 | -o | 指定一个输出文件来重定向输出 |
| -d | 指定目录 | -q | 以 quiet 模式执行 |
| -e | 展开对象 | -r | 递归处理目录和文件 |
| -f | 指定读取数据的文件 | -s | 以 silent 模式执行 |
| -h | 显示命令的帮助信息 | -v | 生成 verbose 输出 |
| -i | 忽略大小写 | -x | 排除和拒绝 |
| -l | 生成长格式的输出 | -y | 设置所有提问的回答为 yes |

在本书后面，可以在对各种 bash 命令进行介绍时体会这些选项含义。选项使用相同的含义有助于用户和脚本之间的交互，不必为查看手册而烦恼。

## 11.6　获取用户输入

虽然提供命令行选项和参数是从脚本用户处获取数据的好方法，但有时脚本需要具有更高的交互性。有时在脚本执行过程中需要询问一个问题并等待执行脚本的人员的应答。bash shell 提供的 read 命令可以解决这个问题。

### 11.6.1　基本读取

read 命令接受标准输入（键盘）的输入，或其他文件描述符（参阅第 12 章）的输入。得到输入后，read 命令将数据放入一个标准变量中。下面是 read 命令的最简单形式：

```
$ cat test21
#!/bin/bash
# testing the read command

echo -n "Enter your name: "
read name
echo "Hello $name, welcome to my program."
$ ./test21
Enter your name: Rich Blum
Hello Rich Blum, welcome to my program.
$
```

非常简单。注意生成提示的 echo 命令使用了-n 选项。该选项抑制字符串结尾的新行符，允许脚本用户在字符串后面立即输入数据，而不是在下一行中输入。这使得脚本看起来更整齐。

事实上，在 read 命令包含-p 选项，允许在 read 命令行中直接指定一个提示：

```
$ cat test22
#!/bin/bash
# testing the read -p option

read -p "Please enter your age:" age
days=$[ $age * 365 ]
echo "That makes you over $days days old!"
$ ./test22
Please enter your age:10
That makes you over 3650 days old!
$
```

可以看到，第一个例子中输入我的名字时，read 命令将我的名字中的姓和名都赋值给了同一个变量。read 命令将提示后面输入的所有数据赋值给单个变量，或者也可以指定多个变量。输入的每个数据值都被赋值给列表中的下一个变量。如果变量列表中变量数目少于数据数目，则剩余的数据都赋值给最后一个变量：

```
$ cat test23
#!/bin/bash
# entering multiple variables

read -p "Enter your name: " first last
echo "Checking data for $last, $first..."
$ ./test23
Enter your name: Rich Blum
Checking data for Blum, Rich...
$
```

在 read 命令行中也可以不指定变量。如果不指定变量，那么 read 命令会将接收到的数据放置在环境变量 REPLY 中：

```
$ cat test24
#!/bin/bash
# testing the REPLY environment variable

read -p "Enter a number: "
factorial=1
for (( count=1; count <= $REPLY; count++ ))
do
   factorial=$[ $factorial * $count ]
done
echo "The factorial of $REPLY is $factorial"
$ ./test24
Enter a number: 5
The factorial of 5 is 120
$
```

环境变量 REPLY 中包含输入的所有数据，可以像使用其他变量一样在 shell 脚本中使用环境变量 REPLY。

### 11.6.2 计时

使用 read 命令存在着潜在危险。脚本很可能会停下来一直等待脚本用户输入数据。如果无论是否输入数据脚本都必须继续执行，那么可以使用-t 选项指定一个计时器。-t 选项指定 read 命令等待输入的秒数。当计时器计时数满时，read 命令返回一个非零退出状态：

```
$ cat test25
#!/bin/bash
# timing the data entry

if read -t 5 -p "Please enter your name: " name
then
   echo "Hello $name, welcome to my script"
else
   echo
   echo "Sorry, too slow!"
fi
$ ./test25
Please enter your name: Rich
Hello Rich, welcome to my script
$ ./test25
Please enter your name:
Sorry, too slow!
$
```

因为如果计时器计时数满，read 命令会以非零状态退出，所以很容易使用标准的结构语句进行

跟踪，如 if-then 语句或 while 循环。在本例中，当计时器计时数满时，if 语句失败，shell 会执行 else 部分的命令。

除了输入时间计时，还可以设置 read 命令计数输入的字符。当输入的字符数目达到预定数目时，自动退出，并将输入的数据赋值给变量：

```
$ cat test26
#!/bin/bash
# getting just one character of input

read -n1 -p "Do you want to continue [Y/N]? " answer
case $answer in
Y | y) echo
       echo "fine, continue on...";;
N | n) echo
       echo OK, goodbye
       exit;;
esac
echo "This is the end of the script"
$ ./test26
Do you want to continue [Y/N]? Y
fine, continue on...
This is the end of the script
$ ./test26
Do you want to continue [Y/N]? n
OK, goodbye
$
```

该例子使用了-n 选项，后接数值 1，指示 read 命令只要接收到一个字符就退出。只要按下一个字符进行回答，read 命令立即接受输入并将其传给变量。无需按回车键。

### 11.6.3　默读

有时会需要脚本用户进行输入，但不希望输入的数据显示在监视器上。典型的例子就是输入密码，当然还有很多其他类型的数据需要隐藏。

-s 选项能够使 read 命令中输入的数据不显示在监视器上（实际上，数据是显示的，只是 read 命令将文本颜色设置成了与背景相同的颜色）。下面是一个在脚本中使用-s 选项的示例：

```
$ cat test27
#!/bin/bash
# hiding input data from the monitor

read -s -p "Enter your password: " pass
echo
echo "Is your password really $pass?"
$ ./test27
Enter your password:
Is your password really T3st1ng?
$
```

在输入提示后面输入的数据没有显示在监视器上，但正确赋值给了变量，供在脚本中使用。

### 11.6.4　读取文件

最后，还可以使用 read 命令读取 Linux 系统上存储在文件中的数据。每调用一次 read 命令，都会读取文件中的一行文本。当文件中没有可读的行时，read 命令将以非零退出状态退出。

读取文件的关键是如何将文件中的数据传送给 read 命令。最常用的方法是对文件使用 cat 命令并通过管道将结果直接传给包含 read 命令的 while 命令。如下面的例子所示：

```
$ cat test28
#!/bin/bash
# reading data from a file
```

```
count=1
cat test | while read line
do
   echo "Line $count: $line"
   count=$[ $count + 1]
done
echo "Finished processing the file"
$ cat test
The quick brown dog jumps over the lazy fox.
This is a test, this is only a test.
O Romeo, Romeo! wherefore art thou Romeo?
$ ./test28
Line 1: The quick brown dog jumps over the lazy fox.
Line 2: This is a test, this is only a test.
Line 3: O Romeo, Romeo! wherefore art thou Romeo?
Finished processing the file
$
```

例子中 while 命令使用 read 命令不断循环处理文件中的每一行，直到 read 命令以非零退出状态退出。

## 11.7 小结

本章讨论了获取脚本用户数据的 3 种不同方法。命令行参数允许用户在执行脚本时直接在命令行中输入数据。脚本使用位置参数获取命令行参数，并将命令行参数赋值给变量。

shift 命令通过对位置参数进行移位来操作命令行参数。使用该命令时，不必知道可用参数的个数就可以很容易的对参数进行迭代。

处理命令行参数时，可以使用 3 种特殊变量。shell 将变量$#设置为命令行中输入的参数个数。变量$*中以一个字符串的形式包含所有参数，变量$@以单词的形式包含所有参数。当处理长参数列表时，这些变量使用起来很方便。

除了参数，脚本用户还可以使用命令行选项传递信息给脚本。命令行选项是前面带有一个破折号的单个字母。设置不同的选项可以改变脚本的行为。bash shell 提供 3 种方式处理命令行选项。

第一种方式是以处理命令行参数的方式处理选项。使用位置参数变量迭代选项，处理出现在命令行中的每个选项。

另一种处理命令行参数的方式是使用 getopt 命令。这个命令将命令行选项和参数转化成脚本可以处理的标准格式。getopt 命令允许指定将哪些字母作为选项识别，哪些选项需要附加参数值。getopt 命令处理标准的命令行参数，并将选项和参数按适当的顺序进行输出。

bash shell 还包含一个 getopts 命令（注意为复数形式）。getopts 命令提供对命令行参数的更高级处理。允许多值参数，标识脚本未定义的选项。

最后一种获取脚本用户数据的方式是使用 read 命令。read 命令允许脚本以交互式的方式请求并等待用户信息。read 命令将脚本用户输入的所有数据都放在可以在脚本中使用一个或多个变量中。

可以为 read 命令设置许多选项来自定义输入给脚本的数据，如使用隐藏数据条目、应用计时数据条目和请求指定数目的输入字符。

下一章，将深入研究 bash shell 脚本输出数据的方法。至此，已经学习了如何在监视器上显示数据和如何将数据重定向到文件。接下来将探讨一些其他选项，使用这些选项不仅可以将数据定位到指定的位置，还可以将特定类型的数据定位到指定的位置。这将使 shell 脚本看起来更专业！

CHAPTER 12

# 第 12 章 显示数据

本章内容：

- 回顾重定向；
- 标准输入和输出；
- 报告错误；
- 丢弃数据；
- 创建日志文件。

到目前为止，本书介绍的脚本显示信息方式有两种：将数据回显到监视器和将数据重定向到文件。第 8 章介绍了如何将命令的输出重定向到文件。本章将继续讨论该主题，向您介绍如何将脚本输出重定向到 Linux 系统的各个位置。

## 12.1 了解输入和输出

到目前为止，您已经学习了两种显示脚本输出的方法：

- 在监视器屏幕上显示输出；
- 将输出重定向到文件。

这两种方法要么生成全部数据输出，要么没有任何输出。但有时需要在监视器上显示一部分数据，而在文件中显示另一部分数据。在这种情况下，最好能了解 Linux 处理输入和输出的方式，以便让脚本输出到正确的位置。

以下几节将介绍如何使用标准的 Linux 输入和输出系统满足您的需求，以帮助您将脚本输出定向到特定位置。

### 12.1.1 标准文件描述符

Linux 系统将每个对象当作文件处理。这包括输入和输出过程。Linux **使用文件描述符**（file descriptor）标识每个文件对象。文件描述符是一个非负整数，可以唯一地标识会话中打开的文件。每个进程中最多可以有 9 个打开文件的描述符。bash shell 为特殊需要保留前 3 个文件描述符（0、1 和 2），如表 12-1 所示。

表 12-1 Linux 标准文件描述符

| 文件描述符 | 缩写 | 描述 |
| --- | --- | --- |
| 0 | STDIN | 标准输入 |
| 1 | STDOUT | 标准输出 |
| 2 | STDERR | 标准错误 |

这 3 个特殊文件描述符处理脚本的输入和输出。shell 使用它们将 shell 中的默认输入和输出定向到相应位置（默认位置通常是监视器）。以下几节将详细介绍每种标准文件描述符。

### 1. STDIN

STDIN 文件描述符引用 shell 的标准输入。对于终端接口，标准输入是键盘。shell 通过 STDIN 文件描述符从键盘接受输入，并在键入时处理每个字符。

使用输入重定向符号（<）时，Linux 将使用重定向引用的文件替换标准的输入文件描述符。它读取文件并获取数据，就像是通过键盘键入的一样。

许多 bash 命令通过 STDIN 接收输入，尤其在命令行没有指定文件时。以下是使用 cat 命令通过 STDIN 输入数据的示例：

```
$ cat
this is a test
this is a test
this is a second test.
this is a second test.
```

在命令行输入 cat 命令时，它将通过 STDIN 接收输入。输入每行命令时，cat 命令将回显该行。

但是，也可以使用 STDIN 重定向符号强制 cat 命令从 STDIN 以外的其他文件接收输入：

```
$ cat < testfile
This is the first line.
This is the second line.
This is the third line.
$
```

现在 cat 命令使用包含在 testfile 文件中的行作为输入。可以使用该技术将数据输入到任何从 STDIN 接收数据的 shell 命令。

### 2. STDOUT

STDOUT 文件描述符引用 shell 的标准输出。在终端接口，标准输出是终端监视器。shell 的所有输出（包括 shell 中运行的程序和脚本）都定向到标准输出，也就是监视器。

许多 bash 命令默认将输出定向到 STDOUT 文件描述符。如第 8 章所示，可以使用输出重定向更改默认设置：

```
$ ls -l > test2
$ cat test2
total 20
-rw-rw-r-- 1 rich rich 53 2007-10-26 11:30 test
-rw-rw-r-- 1 rich rich  0 2007-10-26 11:32 test2
-rw-rw-r-- 1 rich rich 73 2007-10-26 11:23 testfile
$
```

使用输出重定向符号，所有通常定向到监视器的输出现在都重定向到 shell 指定的重定向文件。

还可以使用>>符号向文件添加数据：

```
$ who >> test2
$ cat test2
total 20
-rw-rw-r-- 1 rich rich 53 2007-10-26 11:30 test
-rw-rw-r-- 1 rich rich  0 2007-10-26 11:32 test2
-rw-rw-r-- 1 rich rich 73 2007-10-26 11:23 testfile
rich     pts/0        2007-10-27 15:34 (192.168.1.2)
$
```

who 命令生成的输出将添加到已经存在于 test2 文件中的数据。

但是，如果使用脚本的标准输出重定向，您可能会遇到问题。脚本中可能发生的问题示例如下：

```
$ ls -al badfile > test3
ls: cannot access badfile: No such file or directory
$ cat test3
$
```

当命令生成错误消息时，shell 不会将错误消息重定向到输出重定向文件。shell 创建了输出重定向

文件，但是错误消息显示在监视器屏幕上。

shell 将错误消息与正常输出分开处理。如果创建一个以后台模式运行的 shell 脚本，则通常必须依赖将发送到日志文件的输出消息。如果使用这项技术，那么在生成错误消息时，将无法在日志文件中显示。因此必须换一种方法。

#### 3. STDERR

shell 使用特殊的 STDERR 文件描述符处理错误消息。STDERR 文件描述符引用 shell 的标准错误输出。这是 shell 发送错误消息（shell 或 shell 中运行的程序和脚本所生成）的目的地。

默认情况下，STDERR 文件描述符与 STDOUT 文件描述符指向相同的位置（即使它们分配了不同的文件描述符值）。这意味着，默认情况下，所有错误消息都显示在监视器上。

但是，从示例中可以看到，重定向 STDOUT 不会自动重定向 STDERR。处理脚本时，常常需要更改该行为，尤其是需要在日志文件中记录错误消息时。

### 12.1.2　重定向错误

您已经了解了如何使用重定向符号重定向 STDOUT 数据。重定向 STDERR 数据的方式也没什么不同，只需在使用重定向符号时定义 STDERR 文件描述符即可。有好几种方法进行重定向数据。

#### 1. 仅重定向错误

如表 12-1 所示，STDERR 文件描述符的值设置为 2。通过替换位于重定向符号正前方的这个文件描述符，可以仅重定向错误消息。该值必须位于重定向符号的正前方，否则将无效：

```
$ ls -al badfile 2> test4
$ cat test4
ls: cannot access badfile: No such file or directory
$
```

现在再运行命令，错误消息将不会显示在监视器上。命令生成的任何错误消息将包含在输出文件中。使用这种方法，shell 仅重定向错误消息，不包括普通数据。这里还有一个示例，该示例在同一输出中混合了 STDOUT 和 STDERR 消息：

```
$ ls -al test badtest test2 2> test5
-rw-rw-r-- 1 rich rich 158 2007-10-26 11:32 test2
$ cat test5
ls: cannot access test: No such file or directory
ls: cannot access badtest: No such file or directory
$
```

ls 命令的普通 STDOUT 输出仍然定向到默认的 STDOUT 文件描述符，也就是监视器。由于命令将文件描述符 2 输出（STDERR）重定向到输出文件，因此 shell 将所有生成的错误消息发送到指定的重定向文件。

#### 2. 重定向错误和数据

如果需要同时重定向错误和普通数据，则必须使用两个重定向符号。必须在希望重定向的数据前面放置相应的文件描述符，然后将它们指向相应的输出文件以保存数据：

```
$ ls -al test test2 test3 badtest 2> test6 1> test7
$ cat test6
ls: cannot access test: No such file or directory
ls: cannot access badtest: No such file or directory
$ cat test7
-rw-rw-r-- 1 rich rich 158 2007-10-26 11:32 test2
-rw-rw-r-- 1 rich rich   0 2007-10-26 11:33 test3
$
```

通过 1>符号，shell 将定向到 STDOUT 的 ls 命令的普通输出重定向 test7 文件。通过 2>符号，将应该定向到 STDEER 的错误消息重定向到 test6 文件。

可以使用这项技术将脚本中发生的任何错误消息与普通脚本输出分离开来。这样就能够轻松标识

错误，而不会受到成千上万行普通输出数据的影响。

此外，如果愿意，也可以将 STDEER 和 STDOUT 输出重定向到同一个输出文件。为此，bash shell 提供了一个特殊的重定向符号，即&>符号：

```
$ ls -al test test2 test3 badtest &> test7
$ cat test7
ls: cannot access test: No such file or directory
ls: cannot access badtest: No such file or directory
-rw-rw-r-- 1 rich rich 158 2007-10-26 11:32 test2
-rw-rw-r-- 1 rich rich   0 2007-10-26 11:33 test3
$
```

使用&>符号，命令生成的所有输出都发送到同一位置，包括数据和错误。您会发现其中一个错误消息的顺序与期望的不同。badtest 文件（列出的最后一个文件）的错误消息排在输出文件的第二位。bash shell 自动使错误消息的优先级高于标准输出。这使您能够一起查看错误消息，而不用在整个输出文件中查找。

## 12.2 在脚本中重定向输出

在脚本中使用 STDOUT 和 STDEER 文件描述符在多个位置生成输出，只要重定向相应的文件描述符即可。使用脚本重定向输出的方法有两种：

- 临时重定向每一行；
- 在脚本中永久重定向所有命令。

以下几节介绍这两种方法如何工作。

### 12.2.1 临时重定向

如果想要故意在脚本中生成错误消息，可以将单个输出行重定向到 STDEER。需要做的只是使用输出重定向符号将输出重定向到 STDEER 文件描述符。重定向到某个文件描述符时，必须在文件描述符编号前面添加&号：

```
echo "This is an error message" >&2
```

该行在脚本的 STDEER 文件描述符指向的地方而不是普通的 STDOUT 上显示以上文本。以下是使用这项特性的脚本示例：

```
$ cat test8
#!/bin/bash
# testing STDERR messages

echo "This is an error" >&2
echo "This is normal output"
$
```

如果像往常一样运行脚本，并没有什么不同之处：

```
$ ./test8
This is an error
This is normal output
$
```

记住，默认情况下，Linux 将 STDOUT 输出定向到 STDOUT。但是，如果运行脚本时重定向 STDERR，任何使用脚本定向到 STDERR 的文本将被重定向：

```
$ ./test8 2> test9
This is normal output
$ cat test9
This is an error
$
```

非常好！使用 STDOUT 显示的文本出现在监控器上，而发送到 STDERR 的 echo 语句文本重定向

到输出文件。

该方法对于在脚本中生成错误消息非常好。如果有人使用您的脚本，他们可以使用 STDERR 文件描述符轻松地重定向错误消息，如上文所示。

### 12.2.2　永久重定向

如果在脚本中重定向许多数据，那么重定向每个 echo 语句就不太方便了。这种情况下，可以使用 exec 命令通知 shell 在脚本执行期间重定向特定的文件描述符：

```
$ cat test10
#!/bin/bash
# redirecting all output to a file
exec 1>testout

echo "This is a test of redirecting all output"
echo "from a script to another file."
echo "without having to redirect every individual line"
$ ./test10
$ cat testout
This is a test of redirecting all output
from a script to another file.
without having to redirect every individual line
$
```

exec 命令启动一个新 shell，并将 STDOUT 文件描述符重定向到一个文件。所有定向到 STDOUT 的脚本输出都将重定向到这个文件。

还可以在脚本中间重定向 STDOUT：

```
$ cat test11
#!/bin/bash
# redirecting output to different locations

exec 2>testerror

echo "This is the start of the script"
echo "now reidirecting all output to another location"

exec 1>testout

echo "This output should go to the testout file"
echo "but this should go to the testerror file" >&2
$ ./test11
This is the start of the script
now reidirecting all output to another location
$ cat testout
This output should go to the testout file
$ cat testerror
but this should go to the testerror file
$
```

脚本使用 exec 命令将任何定向到 STDERR 的输出重定向到文件 testerror。接下来，脚本使用 echo 命令向 STDOUT 显示几行。之后，再次使用 exec 命令将 STDOUT 重定向到 testout 文件。注意，即使重定向了 STDOUT，也可以指定 echo 语句的输出定向到 STDERR，在本例中 STDERR 仍然重定向到 testerror 文件。

如果只想将部分脚本的输出重定向到另一个位置（如错误日志），该特性将十分方便。但在使用它时会遇到一个问题。

重定向 STDOUT 或 STDERR 之后，就无法轻松地将它们重定向到原来的位置。如果重定向位置需要来回切换，则需要学习一个技巧。本章后面的“创建自己的重定向”一节将介绍这个技巧以及如何在 shell 脚本中使用它。

## 12.3 在脚本中重定向输入

重定向 STDOUT 和 STDERR 的技术同样可以用来重定向键盘的 STDIN。exec 命令可以帮助您将 STDIN 重定向到 Linux 系统的文件：

```
exec  0<  testfile
```

该命令告知 shell 它应该从文件 testfile 而不是 STDIN 获取输入。在脚本请求输出的任何时间都可以使用这种重定向方式。以下是一个示例：

```
$ cat test12
#!/bin/bash
# redirecting file input

exec 0< testfile
count=1

while read line
do
   echo "Line #$count: $line"
   count=$[ $count + 1 ]
done
$ ./test12
Line #1: This is the first line.
Line #2: This is the second line.
Line #3: This is the third line.
$
```

第 11 章介绍了如何使用 read 命令读取用户通过键盘输入的数据。通过将 STDIN 重定向到文件，在 read 命令尝试从 STDIN 读取命令时，它将从文件而不是键盘获取数据。

这是一项非常好的技术，可以读取文件的数据并使用脚本处理。Linux 系统管理员常见的一个任务是，从日志文件读取数据并进行处理。这是执行该任务最简单的方式。

## 12.4 创建自己的重定向

在脚本中重定向输入和输出时，并不局限于 3 种默认的文件描述符。如前所述，在 shell 中最多可以有 9 个打开的文件描述符。其他 6 个文件描述符的编号从 3～8。可以将这些文件描述符应用到任何文件，然后在脚本中使用它们。本节将介绍如何在脚本中使用这 6 个文件描述符。

### 12.4.1 创建输出文件描述符

使用 exec 命令为输出分配文件描述符。与标准的文件描述符一样，向文件位置分配了备选文件描述符之后，该重定向将是永久性的，除非重新分配。以下是一个在脚本中使用备选文件描述符的简单示例：

```
$ cat test13
#!/bin/bash
# using an alternative file descriptor

exec 3>test13out

echo "This should display on the monitor"
echo "and this should be stored in the file" >&3
echo "Then this should be back on the monitor"
$ ./test13
This should display on the monitor
Then this should be back on the monitor
```

```
$ cat test13out
and this should be stored in the file
$
```

脚本使用 exec 命令将文件描述符 3 重定向到一个备选文件位置。当脚本执行 echo 语句时，它们将显示在 STDOUT 上，正如期望的那样。但是，重定向到文件描述符 3 的 echo 语句定向到备选文件。这使您能够将普通输出定向到监视器，将特殊信息重定向到文件（如日志文件）。

### 12.4.2 重定向文件描述符

现在介绍恢复重定向文件描述符的技巧。您可以向标准文件描述符分配一个备选文件描述符，反之亦然。这就是说，可以将 STDOUT 的原位置重定向到备选文件描述符，然后将该文件描述符重定向回 STDOUT。听起来似乎有些复杂，实际上非常简单。希望以下这个简单的示例能让您更加清楚：

```
$ cat test14
#!/bin/bash
# storing STDOUT, then coming back to it

exec 3>&1
exec 1>test14out

echo "This should store in the output file"
echo "along with this line."

exec 1>&3

echo "Now things should be back to normal"
$ ./test14
Now things should be back to normal
$ cat test14out
This should store in the output file
along with this line.
$
```

本例有些复杂，让我们一步步来看。首先，脚本将文件描述符 3 重定向到文件描述符 1 的当前位置，也就是 STDOUT。这表明任何发送到文件描述符 3 的输出都将出现在监视器上。

第二个 exec 命令将 STDOUT 重定向到一个文件。shell 现在将任何直接发送到 STDOUT 的输出重定向到输出文件。但是，文件描述符 3 仍然指向 STDOUT 原来的位置，也就是监视器。如果这时将输出数据发送到文件描述符 3，它仍然将出现在显示器上，即使重定向了 STDOUT。

向 STDOUT（指向文件）发送一些输出后，脚本将 STDOUT 重定向到文件描述符 3 的当前位置，文件描述符 3 仍然设置为监视器。这就是说，现在 STDOUT 指向它原来的位置，即监视器。

虽然这种方法可能很容易搞混淆，但它是一种临时重定向脚本文件输出然后将输出设置回普通设置的常见方式。

### 12.4.3 创建输入文件描述符

可以使用重定向输出文件描述符的方式重定向输入文件描述符。在重定向到文件之前，将 STDIN 文件描述符位置保存到另一个文件描述符，然后在完成了文件读取操作之后，可以将 STDIN 恢复为原来的位置：

```
$ cat test15
#!/bin/bash
# redirecting input file descriptors

exec 6<&0

exec 0< testfile

count=1
```

```
while read line
do
   echo "Line #$count: $line"
   count=$[ $count + 1 ]
done
exec 0<&6
read -p "Are you done now? " answer
case $answer in
Y|y) echo "Goodbye";;
N|n) echo "Sorry, this is the end.";;
esac
$ ./test15
Line #1: This is the first line.
Line #2: This is the second line.
Line #3: This is the third line.
Are you done now? y
Goodbye
$
```

在本例中，使用文件描述符 6 保存 STDIN 的位置。然后脚本将 STDIN 重定向到文件。read 命令的所有输入都来自重定向后的 STDIN，现在也就是输入文件。

读取完所有行之后，脚本通过将 STDIN 重定向到文件描述符 6，将它设置回原来的位置。脚本使用另一个 read 命令进行测试以确保 STDIN 回到了普通设置（这次等待键盘的输入）。

### 12.4.4 创建读取/写入文件描述符

听起来可能有些奇怪，但是确实可以为输入和输出打开同一个文件描述符。可以使用同一个文件描述符从一个文件读取数据，同时向这个文件写入数据。

但使用这种方法时必须十分小心。读取文件和向该文件写入数据时，shell 维护一个内部指针，指示文件内部的位置。读取和写入操作都发生在指针上次放置的地方。如果不小心，很可能造成非常有趣的结果。请看以下示例：

```
$ cat test16
#!/bin/bash
# testing input/output file descriptor

exec 3<> testfile
read line <&3
echo "Read: $line"
echo "This is a test line" >&3
$ cat testfile
This is the first line.
This is the second line.
This is the third line.
$ ./test16
Read: This is the first line.
$ cat testfile
This is the first line.
This is a test line
ine.
This is the third line.
$
```

本例使用 exec 命令将文件描述符 3 分配给文件 testfile 的输入输出操作。接下来，它应用分配的文件描述符使用 read 命令读取文件的第一行，然后在 STDOUT 中显示读取的数据行。之后，它使用 echo 语句向使用同一个文件描述符打开的文件写入一行数据。

运行脚本时，一开始没什么问题。输出显示读取了 testfile 文件的第一行。但是，如果在运行脚本之后显示 testfile 文件的内容，您会发现写入文件的数据覆盖了现有数据。

当脚本向文件写入数据时，它从文件指针所在的地方开始。read 命令读取第一行数据后，文件指针指向第二行数据的第一个字符。当 echo 语句将数据输出到文件时，它将数据放在文件指针的当前位置，覆盖了此处的数据。

### 12.4.5 关闭文件描述符

如果创建新的输入或输出文件描述符，shell 将在脚本退出时自动关闭它们。但有时需要在脚本结束前手动关闭文件描述符。

要关闭文件描述符，将它重定向到特殊符号&-。以下是它在脚本中的示例：

```
exec 3>&-
```

该语句关闭文件描述符 3，使之不再在脚本中使用。当试图使用关闭的文件描述符时，将出现以下情况：

```
$ cat badtest
#!/bin/bash
# testing closing file descriptors

exec 3> test17file

echo "This is a test line of data" >&3

exec 3>&-

echo "This won't work" >&3
$ ./badtest
./badtest: 3: Bad file descriptor
$
```

关闭文件描述符之后，就不能在脚本中向它写入任何数据，否则 shell 将生成一个错误消息。

关闭文件描述符时还需要注意另一件事情。如果稍后在脚本中打开同一个输出文件，shell 将使用新文件替换现有文件。这意味着如果您输出任何数据，它将覆盖现有文件。以下是该问题的示例：

```
$ cat test17
#!/bin/bash
# testing closing file descriptors

exec 3> test17file
echo "This is a test line of data" >&3
exec 3>&-

cat test17file

exec 3> test17file
echo "This'll be bad" >&3
$ ./test17
This is a test line of data
$ cat test17file
This'll be bad
$
```

将数据字符串发送到 test17file 并关闭文件描述符之后，脚本使用 cat 命令显示文件的内容。到目前为止没有问题。接下来，脚本重新打开输出文件并向其发送另一个数据字符串。当显示输出文件的内容时，您看到的所有内容只有第二个数据字符串。shell 覆盖了原来的输出文件。

## 12.5 列出开放文件描述符

因为只有 9 个文件描述符可用，您可能认为让所有内容都条理清晰是件容易的事情。但有时很难跟踪哪个文件描述符重定向到了哪个位置。为了帮助您理清线索，bash shell 提供了 lsof 命令。

lsof 命令列出整个 Linux 系统上所有的开放文件描述符。某种程度上说，这是一项有争议的功能，因为它向非系统管理员提供有关 Linux 系统的信息。因此，许多 Linux 系统都隐藏了该命令，以防用户无意中使用它。

在我的 Fedora Linux 系统中，lsof 命令位于/usr/sbin 目录中。要使用普通用户账户运行它，必须使用完全路径名引用它：

```
$ /usr/sbin/lsof
```

这会生成大量输出。它显示 Linux 系统上当前开放的每个文件的相关信息，包括所有后台运行的进程，以及所有登录到系统的用户账户。

有许多命令行参数和选项，可以帮助筛选 lsof 输出。最常用的是-p，该参数可以指定进程 ID（PID），还有-d，该参数可以指定要显示的文件描述符编号。

为了轻松确定进程的当前 PID，可以使用特殊环境变量$$，shell 将该变量设置为当前 PID。-a 选项可以用来连接其他两个选项的结果，以生成以下内容：

```
$ /usr/sbin/lsof -a -p $$ -d 0,1,2
COMMAND  PID USER   FD TYPE DEVICE SIZE NODE NAME
bash    3344 rich    0u CHR  136,0         2 /dev/pts/0
bash    3344 rich    1u CHR  136,0         2 /dev/pts/0
bash    3344 rich    2u CHR  136,0         2 /dev/pts/0
$
```

这显示了当前进程（bash shell）的默认文件描述符（0、1 和 2）。lsof 的默认输出包括几个信息列，如表 12-2 所示。

表 12-2 默认 lsof 输出

| 列 | 描述 |
|---|---|
| COMMAND | 进程中命令名称的前 9 个字符 |
| PID | 进程的进程 ID |
| USER | 拥有进程的用户的登录名 |
| FD | 文件描述符编号和访问类型（r-读取，w-写入，u-读取/写入） |
| TYPE | 文件类型（CHR-字符，BLK-块，DIR-目录，REG-常规文件） |
| DEVICE | 设备的设备编号（主要和次要） |
| SIZE | 如果可用，则为文件大小 |
| NODE | 本地文件的节点编号 |
| NAME | 文件的名称 |

与 STDIN、STDOUT 和 STDERR 关联的文件类型是字符模式。由于 STDIN、STDOUT 和 STDERR 文件描述符都指向终端，则输出文件的名称是终端的设备名。所有 3 个标准文件都可以用于读取和写入（尽管能够写入 STDIN 以及能够从 STDOUT 读取看起来很奇怪）。

现在，从打开了几个备选文件描述符的脚本内部查看 lsof 命令的结果：

```
$ cat test18
#!/bin/bash
# testing lsof with file descriptors

exec 3> test18file1
exec 6> test18file2
exec 7< testfile

/usr/sbin/lsof -a -p $$ -d0,1,2,3,6,7
$ ./test18
COMMAND PID USER   FD   TYPE  DEVICE SIZE NODE NAME
test18 3594 rich    0u   CHR   136,0         2 /dev/pts/0
test18 3594 rich    1u   CHR   136,0         2 /dev/pts/0
est18 3594 rich    2u  CHR   136,0          2 /dev/pts/0
```

```
18  3594 rich    3w  REG   253,0    0 360712 /home/rich/test18file1
18  3594 rich    6w  REG   253,0    0 360715 /home/rich/test18file2
18  3594 rich    7r  REG   253,0   73 360717 /home/rich/testfile
$
```

该脚本创建了3个备选文件描述符，两个用于输出(3和6)，一个用于输入(7)。当脚本运行lsof命令时，可以看到输出中的新文件描述符。我去掉了输出的第一部分，以便能看到文件名称的结果。文件名显示文件在文件描述符中使用的完整路径名。它显示每个文件的类型都是REG，表示它们是文件系统的常规文件。

## 12.6 禁止命令输出

有时候不希望显示任何脚本输出，例如，正在以后台进程方式运行脚本时（参阅第13章）。如果在后台运行时出现了错误消息，shell将通过电子邮件通知进程的所有者。这会很讨厌，尤其是运行的脚本生成了很多细小的错误时。

要解决该问题，可以将STDERR重定向到称为**空文件**（null file）的特殊文件。顾名思义，该文件不包含任何内容。shell输出到空文件的任何数据都不会保存，即全部丢失。

该文件在Linux系统的标准位置是/dev/null。任何重定向到该位置的数据都将丢失且不会显示：

```
$ ls -al > /dev/null
$ cat /dev/null
$
```

这是一种常见的禁止错误消息而不保存它们的方法：

```
$ ls -al badfile test16 2> /dev/null
-rwxr--r--    1 rich        rich          135 Oct 29 19:57 test16*
$
```

可以使用/dev/null文件作为输入文件用于输入重定向。因为/dev/null文件不包含任何内容，程序员经常使用它快速将数据从现有文件移除，无需移除文件并重新创建它：

```
$ cat testfile
This is the first line.
This is the second line.
This is the third line.
$ cat /dev/null > testfile
$ cat testfile
$
```

testfile文件仍然存在于系统中，但现在是空文件。这是一种常见的清除日志文件（该文件必须保留在原地以供应用程序运行）的方法。

## 12.7 使用临时文件

Linux系统保留了一个特殊的目录位置，以供临时文件使用。Linux使用/tmp目录处理不需要永久保存的文件。大部分Linux发行版的系统配置都是在启动时自动删除/tmp目录中的任何文件。

系统上的任何用户账户都有权读取和写入/tmp 目录中的文件。该特性可以帮助您轻松创建临时文件，而无需担心它们的清理问题。

创建临时文件还有一个特殊的命令。mktemp命令可以轻松在/tmp文件夹中创建一个唯一的临时文件。shell创建该文件但是不使用默认的umask值（参阅第6章）。相反，它仅向文件所有者分配读取和写入权限，并使您成为文件的所有者。创建文件之后，就可以通过脚本进行完整权限的读取和写入操作，但其他人都不能访问它（当然根用户除外）。

### 12.7.1 创建本地临时文件

默认情况下，mktemp 在本地目录创建文件。要使用 mktemp 命令在本地目录中创建临时文件，只需要指定一个文件名模板即可。模板包括文本文件名以及附加到文件名后的 6 个 X：

```
$ mktemp testing.XXXXXX
$ ls -al testing*
-rw-------    1 rich        rich          0 Oct 29 21:30 testing.UfIi13
$
```

mktemp 命令使用一个 6 字符代码替换 6 个 X，以确保文件名在目录中的唯一性。您可以创建多个临时文件，每个文件保证是唯一的：

```
$ mktemp testing.XXXXXX
testing.lDRLuV
$ mktemp testing.XXXXXX
testing.lVBtkW
$ mktemp testing.XXXXXX
testing.PgqNKG
$ ls -l testing*
-rw-------    1 rich        rich          0 Oct 29 21:57 testing.lDRLuV
-rw-------    1 rich        rich          0 Oct 29 21:57 testing.PgqNKG
-rw-------    1 rich        rich          0 Oct 29 21:30 testing.UfIi13
-rw-------    1 rich        rich          0 Oct 29 21:57 testing.lVBtkW
$
```

可以看出，mktemp 命令的输出是它所创建的文件的名称。在脚本中使用 mktemp 命令时，需要使用一个变量保存该文件名，以便稍后在脚本中引用：

```
$ cat test19
#!/bin/bash
# creating and using a temp file

tempfile='mktemp test19.XXXXXX'
exec 3>$tempfile

echo "This script writes to temp file $tempfile"

echo "This is the first line" >&3
echo "This is the second line." >&3
echo "This is the last line." >&3
exec 3>&-

echo "Done creating temp file. The contents are:"
cat $tempfile
rm -f $tempfile 2> /dev/null
$ ./test19
This script writes to temp file test19.vCHoya
Done creating temp file. The contents are:
This is the first line
This is the second line.
This is the last line.
$ ls -al test19*
-rwxr--r--    1 rich        rich          356 Oct 29 22:03 test19*
$
```

该脚本使用 mktemp 命令创建一个临时文件，并将文件名分配给$tempfile 变量。然后，它使用临时文件作为文件描述符 3 的输出重定向文件。当临时文件名在 STDOUT 上显示之后，它将几行数据写入临时文件，然后关闭文件描述符。最后，它显示临时文件的内容并使用 rm 命令删除它。

### 12.7.2 在/temp 中创建临时文件

-t 选项强迫 mktemp 在系统的临时文件夹中创建文件。但使用该特性时，mktemp 命令返回用于

创建临时文件的完整路径名，而不仅仅是文件名：

```
$ mktemp -t test.XXXXXX
/tmp/test.xG3374
$ ls -al /tmp/test*
-rw------- 1 rich rich 0 2007-10-29 18:41 /tmp/test.xG3374
$
```

由于 mktemp 命令返回完整路径名，因此可以从 Linux 系统的任何目录引用临时文件，无论系统将临时目录放在哪个位置：

```
$ cat test20
#!/bin/bash
# creating a temp file in /tmp

tempfile='mktemp -t tmp.XXXXXX'
echo "This is a test file." > $tempfile
echo "This is the second line of the test." >> $tempfile

echo "The temp file is located at: $tempfile"
cat $tempfile
rm -f $tempfile
$ ./test20
The temp file is located at: /tmp/tmp.Ma3390
This is a test file.
This is the second line of the test.
$
```

当 mktemp 创建临时文件时，它将完整路径名返回给环境变量。因此，之后可以在任何命令中使用该变量引用临时文件。

### 12.7.3 创建临时目录

-d 选项让 mktemp 命令创建一个临时目录而不是一个文件。然后可以将该目录用于任何目的，比如创建更多临时文件：

```
$ cat test21
#!/bin/bash
# using a temporary directory

tempdir='mktemp -d dir.XXXXXX'
cd $tempdir
tempfile1='mktemp temp.XXXXXX'
tempfile2='mktemp temp.XXXXXX'
exec 7> $tempfile1
exec 8> $tempfile2

echo "Sending data to directory $tempdir"
echo "This is a test line of data for $tempfile1" >&7
echo "This is a test line of data for $tempfile2" >&8
$ ./test21
Sending data to directory dir.ouT8S8
$ ls -al
total 72
drwxr-xr-x    3 rich        rich          4096 Oct 29 22:20 ./
drwxr-xr-x    9 rich        rich          4096 Oct 29 09:44 ../
drwx------    2 rich        rich          4096 Oct 29 22:20 dir.ouT8S8/
-rwxr--r--    1 rich        rich           338 Oct 29 22:20 test21*
$ cd dir.ouT8S8
[dir.ouT8S8]$ ls -al
total 16
drwx------    2 rich        rich          4096 Oct 29 22:20 ./
drwxr-xr-x    3 rich        rich          4096 Oct 29 22:20 ../
-rw-------    1 rich        rich            44 Oct 29 22:20 temp.N5F3O6
```

```
-rw-------    1 rich        rich            44 Oct 29 22:20 temp.SQslb7
[dir.ouT8S8]$ cat temp.N5F3O6
This is a test line of data for temp.N5F3O6
[dir.ouT8S8]$ cat temp.SQslb7
This is a test line of data for temp.SQslb7
[dir.ouT8S8]$
```

该脚本在当前目录创建一个目录，然后使用 cd 命令跳转到该目录，并创建两个临时文件。然后，给这两个临时文件分配文件描述符，并用于存储脚本的输出。

## 12.8 记录消息

有时很有必要将输出同时发送到监视器和日志文件。这种情况下不需要使用两次重定向，只需要使用特殊的 tee 命令即可。

tee 命令就像管道的 T 型接头。它将 STDIN 的数据同时发送到两个目的地。一个是 STDOUT，另一个是 tee 命令行指定的文件名：

```
tee filename
```

由于 tee 重定向来自 STDIN 的数据，因此可以与管道命令配置使用重定向任何命令的输出：

```
$ date | tee testfile
Mon Oct 29 18:56:21 EDT 2007
$ cat testfile
Mon Oct 29 18:56:21 EDT 2007
$
```

输出出现在 STDOUT 中，并且写入指定的文件。注意，默认情况下，tee 命令每次使用时都会覆盖输出文件：

```
$ who | tee testfile
rich     pts/0         2007-10-29 18:41 (192.168.1.2)
$ cat testfile
rich     pts/0         2007-10-29 18:41 (192.168.1.2)
$
```

如果希望向文件添加数据，则必须使用-a 选项：

```
$ date | tee -a testfile
Mon Oct 29 18:58:05 EDT 2007
$ cat testfile
rich     pts/0         2007-10-29 18:41 (192.168.1.2)
Mon Oct 29 18:58:05 EDT 2007
$
```

使用这项技术，可以将数据保存在文件中，同时将数据显示在监视器上供用户查看：

```
$ cat test22
#!/bin/bash
# using the tee command for logging

tempfile=test22file

echo "This is the start of the test" | tee $tempfile
echo "This is the second line of the test" | tee -a $tempfile
echo "This is the end of the test" | tee -a $tempfile
$ ./test22
This is the start of the test
This is the second line of the test
This is the end of the test
$ cat test22file
This is the start of the test
This is the second line of the test
This is the end of the test
$
```

现在可以在永久保存输出副本的同时，将输出显示给用户。

## 12.9 小结

了解 bash shell 处理输入和输出的方式将有助于创建脚本。可以控制脚本接收数据和显示数据的方式，以针对任何环境定制脚本。可以将脚本输入从标准输入（STDIN）重定向到系统中的任何文件，同样可以将脚本输出从标准输出（STDOUT）重定向到系统中的任何文件。

除了 STDOUT 之外，还可以通过重定向 STDERR 输出重定向脚本生成的任何错误消息。方法是重定向与 STDERR 输出关联的文件描述符，即文件描述符 2。可以将 STDERR 输出重定向到 STDOUT 输出的文件，或者一个完全不同的文件。这使您能够将脚本生成的普通消息与错误消息分离。

bash shell 允许创建自己的文件描述符，以便在脚本中使用。可以创建文件描述符 3 到 9，并将其分配给任何输出文件。创建文件描述符之后，可以使用标准的重定向符号将任何命令的输出重定向到该描述符。

bash shell 还可以将输入重定向到文件描述符，提供了一种将文件中包含的数据读取到脚本的简便方法。可以使用 lsof 命令显示 shell 中的活动文件描述符。

Linux 系统提供一个名为/dev/null 的特殊文件，以重定向不需要的输出。Linux 系统丢弃任何重定向到/dev/null 文件的输出。如果将/dev/null 文件的内容重定向到另一个文件，可以生成一个空文件。

mktemp 命令是 bash shell 的一个方便的功能，用它可以轻松创建临时文件和临时目录。您只需为 mktemp 命令指定一个模板，它就可以在每次调用时基于文件模板格式创建一个唯一的文件。还可以在 Linux 系统的/tmp 目录创建临时文件和临时目录，该目录是一个特殊位置，系统启动时不会保留其内容。

tee 命令是一个将输出同时发送到标准输出和日志文件的便捷方法。通过该命令，可以在监视器显示脚本的消息，同时将它们存储在日志文件中。

在第 13 章，将介绍如何控制和运行脚本。除了从命令行界面提示符运行之外，Linux 还提供了很多不同的方法运行脚本。您将了解如何安排脚本在特定的时间运行，以及如何在运行过程中暂停运行。

# 第 13 章
# 脚本控制

本章内容：

- 回顾信号功能；
- 隐藏在背景中；
- 在没有控制台的情况下运行；
- 做得更好；
- 准确无误地运行；
- 从头开始。

当开始构建更高级的脚本时，您也许想知道在 Linux 系统上如何运行并控制它们。到目前为止，我们运行脚本的唯一方式是以实时模式直接从命令行界面运行。这不是在 Linux 中运行脚本的唯一方式。还有许多可用的其他选项可以在 Linux 系统上运行 shell 脚本。本章将研究用于运行代码的各种不同方式。另外，有时脚本会遭遇循环中止问题，需要找出如何在不关闭 Linux 系统的情况下停止这段脚本。本章将介绍控制 shell 脚本在系统中的运行方式以及运行时间的不同方法。

## 13.1 处理信号

Linux 使用信号与系统上运行的进程进行通信。第 4 章介绍了不同的 Linux 信号，以及 Linux 如何使用这些信号停止、启动和终止进程。也可以使用这些信号控制 shell 脚本的运行，只需要让 shell 脚本在接收到来自于 Linux 系统的特定信号时执行命令即可。

### 13.1.1 Linux 信号回顾

系统和应用程序可以生成 30 多个 Linux 信号。表 13-1 列出了 Linux 编程中最常见的 Linux 系统信号。

表 13-1 Linux 信号

| 信号 | 值 | 描述 |
|---|---|---|
| 1 | SIGHUP | 挂起进程 |
| 2 | SIGINT | 中断进程 |
| 3 | SIGQUIT | 停止进程 |
| 9 | SIGKILL | 无条件终止进程 |
| 15 | SIGTERM | 如果可能的话终止进程 |
| 17 | SIGSTOP | 无条件停止，但不终止进程 |
| 18 | SIGTSTP | 停止或暂停进程，但不终止它 |
| 19 | SIGCONT | 重新启动停止的进程 |

默认情况下，bash shell 会忽略它接收的任何 SIGQUIT(3) 和 SIGTERM(15) 信号（以防止交互的 shell 意外终止）。但是，bash shell 处理它收到的任何 SIGHUP(1) 和 SIGINT(2) 信号。

如果 bash shell 收到了一个 SIGHUP 信号，它会退出。在退出之前，它将 SIGHUP 信号传递给 shell 启动的任意进程（如 shell 脚本）。收到 SIGNINT 信号，shell 会立即中断。Linux 内核停止向 shell 提供 CPU 上的处理时间。在这种情况下，shell 将 SIGINT 信号传递给由 shell 启动的任意进程以通知它们这种情况。

shell 将这些信号传递给 shell 脚本程序以对它们进行处理。但是，默认的 shell 脚本行为将忽略这些信号，这样会对脚本运行产生不良影响。为了避免发生这种情况，可以对脚本进行编程以识别信号，并执行命令以便脚本为信号结果做好准备。

### 13.1.2 生成信号

Bash shell 可以使用键盘上的组合键生成两个基本的 Linux 信号。如果需要停止或暂停失控的程序，那么这个功能就会派上用场。

**1. 中断进程**

使用 Ctrl+C 组合键可以生成 SIGINT 信号，并将其发送给当前正在 shell 中运行的任意进程。运行一个通常需要很长时间才能完成的命令并按 Ctrl+C 组合键，可以测试此操作。

```
$ sleep 100

$
```

Ctrl+C 组合键不会在监视器上生成任何输出，它只会停止当前在 shell 中运行的进程。sleep 命令在指定的秒数之内暂停操作。通常，命令提示符在计时器过期之前不会返回。在计时器过期之前按 Ctrl+C 组合键，可以使 sleep 命令提前终止。

**2. 暂停进程**

您还可以在过程中暂停进程而不终止它。有时这是一件危险的事情（例如，脚本锁定了一个关键系统文件时），但这可以查看脚本正在做什么而无需终止进程。

Ctrl+Z 组合键生成 SIGTSTP 信号，可以停止任何在 shell 中运行的进程。停止进程与终止进程不同，停止进程后程序仍然留在内存中，能够从停止的地方继续运行。在稍后的“作业控制”一节将学习如何重新启动停止的进程。

使用 Ctrl+Z 组合键时，shell 通知您进程已经停止：

```
$ sleep 100

[1]+ Stopped                 sleep 100
$
```

方括号中的数字是 shell 分配的**作业编号**（job number）。shell 以**作业**（job）形式引用 shell 中运行的每个进程，向每个作业分配唯一的作业编号。它向第一个启动的进程分配作业编号 1，向第二个进程分配作业编号 2，依此类推。

如果 shell 会话中有一个停止的作业，在退出 shell 时 bash 将发出警告：

```
$ exit
logout
There are stopped jobs.
$
```

可以使用我们介绍过的 ps 命令查看停止的作业：

```
$ ps au
USER PID   %CPU %MEM   VSZ  RSS TTY    STAT   START   TIME COMMAND
rich 20560  0.0   1.2 2688 1624 pts/0  S      05:15   0:00 -bash
rich 20605  0.2   0.4 1564  552 pts/0  T      05:22   0:00 sleep 100
rich 20606  0.0   0.5 2584  740 pts/0  R      05:22   0:00 ps au
$
```

ps 命令将停止作业的状态显示为 T，表示该命令正被跟踪或者已经停止。

如果想在停止作业仍处于活动状态时退出 shell，只需要再次键入 exit 命令。shell 将退出，并终

止停止的作业。还有一种方法，现在已经知道了停止作业的 PID，因此可以使用 kill 命令发送 SIGKILL 信号来终止它：

```
$ kill -9 20605
$
[1]+ Killed                  sleep 100
$
```

终止作业时，最初不会得到任何响应。但是，下一次使用某个生成 shell 提示符的操作时，将会看到一条消息，指示已经终止了作业。每次在 shell 生成提示符时，它将显示 shell 中已经更改了状态的作业的状态。终止作业之后，在下一次让 shell 生成提示符时，它将显示一条消息，指出作业在运行时被终止。

### 13.1.3 捕获信号

除了可以使脚本忽略信号之外，还可以在信号出现时捕获信号和执行其他命令。trap 命令可以指定能够通过 shell 脚本监控和拦截的 Linux 信号。如果脚本收到在 trap 命令中列出的信号，它将保护该信号不被 shell 处理，并在本地处理它。

trap 命令的格式如下：

```
trap commands signals
```

非常简单。在 trap 命令行中，只需要列出希望 shell 执行的命令，以及希望捕获的信号列表（以空格分隔）。指定信号可以通过它们的数值或 Linux 信号名实现。

以下是一个使用 trap 命令忽略 SIGINT 和 SIGTERM 信号的简单示例：

```
$ cat test1
#!/bin/bash
# testing output in a background job

trap "echo Haha" SIGINT SIGTERM
echo "This is a test program"
count=1
while [ $count -le 10 ]
do
   echo "Loop #$count"
   sleep 10
   count=$[ $count + 1 ]
done
echo "This is the end of the test program"
$
```

本例使用的 trap 命令在每次检测到 SIGINT 和 SIGTERM 信号时显示一个简单的文本消息。如果捕获了这些信号，在用户试图使用 bash shell 键盘 CRTL+C 命令停止程序时，脚本将不受影响：

```
$ ./test1
This is a test program
Loop #1
Haha
Loop #2
Loop #3
Haha
Loop #4
Loop #5
Loop #6
Loop #7
Haha
Loop #8
Loop #9
Loop #10
This is the end of the test program
$
```

每次使用 Crtl+C 组合键时，脚本执行在 trap 命令中指定的 echo 语句，而不是忽略信号并允许 shell 停止脚本。

### 13.1.4 捕获脚本退出

除了在 shell 脚本中捕获信号之外，还可以在 shell 脚本退出时捕获它们。这是一种在 shell 完成作业时执行命令的便捷方式。

要捕获 shell 脚本退出，只需要向 trap 命令添加 EXIT 信号：

```
$ cat test2
#!/bin/bash
# trapping the script exit

trap "echo byebye" EXIT

count=1
while [ $count -le 5 ]
do
   echo "Loop #$count"
   sleep 3
   count=$[ $count + 1 ]
done
$ ./test2
Loop #1
Loop #2
Loop #3
Loop #4
Loop #5
byebye
$
```

当脚本到达常规退出点时，就会触发 trap，shell 将执行在 trap 命令行中指定的命令。如果提前退出脚本，也能够捕获 EXIT：

```
$ ./test2
Loop #1
Loop #2
byebye

$
```

使用 CRTL+C 组合键发送 SIGINT 信号时，脚本将退出（因为捕获列表中没有列出该信号），但是在脚本退出之前，shell 将执行 trap 命令。

### 13.1.5 移除捕获

要移除捕获，使用破折号作为命令和想要恢复正常行为的信号列表：

```
$ cat test3
#!/bin/bash
# removing a set trap

trap "echo byebye" EXIT

count=1
while [ $count -le 5 ]
do
   echo "Loop #$count"
   sleep 3
   count=$[ $count + 1 ]
done
trap - EXIT
echo "I just removed the trap"
```

```
$ ./test3
Loop #1
Loop #2
Loop #3
Loop #4
Loop #5
I just removed the trap
$
```

信号捕获移除后，脚本将忽略信号。但是，如果在移除捕获之前收到信号，脚本仍将根据 trap 命令处理该信号：

```
$ ./test3
Loop #1
Loop #2
byebye

$
```

在本例中，使用 CRTL+C 组合键提前终止脚本。因为脚本是在移除捕获之前终止的，所以脚本将执行在捕获中指定的命令。

## 13.2 以后台模式运行脚本

有时直接从命令行界面运行 shell 脚本很不方便。有些脚本需要很长的时间才能处理，而您可能不希望让命令行界面等待太长的时间。当脚本运行时，您不能在终端窗口执行其他操作。幸运的是，该问题有一个很简单的解决方案。

使用 ps 命令，可以看到 Linux 系统上运行着许多不同的进程。很明显，所有这些进程都不是在终端监视器上运行的。这就是所谓的在**后台**运行进程。在后台模式中，进程运行时与终端会话的 STDIN、STDOUT 和 STDERR 无关（参阅第 12 章）。

您也可以使用 shell 脚本来利用该特性，允许进程在后台运行而不锁定终端会话。以下几节将介绍如何在 Linux 系统中以后台模式运行脚本。

### 13.2.1 以后台模式运行

以后台模式运行 shell 脚本的方式很简单。要通过命令行界面以后台模式运行 shell 脚本，只需要在命令后加上一个&符号：

```
$ ./test1 &
[1] 19555
$ This is test program
Loop #1
Loop #2

$ ls -l
total 8
-rwxr--r--    1 rich        rich          219 Nov 2 11:27 test1*
$ Loop #3
```

将&符号放在命令之后时，它将 bash shell 与命令相分离，并以独立的后台进程形式在系统上运行。显示的第一行如下所示：

```
[1] 19555
```

方括号中的数字是 shell 分配给后台进程的作业编号。后面的数字是 Linux 系统分配给进程的 PID。所有 Linux 系统上运行的进程都必须有一个唯一的 PID。

系统显示这些条目之后，将出现一个新的命令行界面提示符。您将返回 shell，执行的命令将以后台模式安全运行。

这时，可以在提示符处输出新的命令（如上例所示）。但是，后台进程仍在运行，它仍然使用终端监视器显示 STDOUT 和 STDERR 消息。从示例中可以看到，test1 脚本的输出与从 shell 运行的其他命令的输出混合在一起。

后台进程结束时，它将在终端显示一条消息：

```
[1]+ Done                    ./test1
```

这表示作业编号和作业状态（Done），以及用于启动该作业的命令。

### 13.2.2 运行多个后台作业

可以通过命令行提示符同时启动任何数量的后台作业：

```
$ ./test1 &
[1] 19582
$ This is test program
Loop #1
$ ./test1 &
[2] 19597
$ This is test program
Loop #1
$ ./test1 &
[3] 19612
$ This is test program
Loop #1
Loop #2
Loop #2
Loop #2
```

每次启动一个新作业时，Linux 系统将分配一个新作业编号和一个 PID。使用 ps 命令可以查看运行的所有脚本：

```
$ ps au
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME
COMMAND
rich 19498 0.0 1.2 2688 1628 pts/0 S 11:38 0:00 -bash
rich 19582 0.0 0.9 2276 1180 pts/0 S 11:55 0:00 /bin/bash ./test1
rich 9597  0.1 0.9 2276 1180 pts/0 S 11:55 0:00 /bin/bash ./test1
rich 19612 0.1 0.9 2276 1180 pts/0 S 11:55 0:00 /bin/bash ./test1
rich 19639 0.0 0.4 1564 552  pts/0 S 11:56 0:00 sleep 10
rich 19640 0.0 0.4 1564 552  pts/0 S 11:56 0:00 sleep 10
rich 19641 0.0 0.4 1564 552  pts/0 S 11:56 0:00 sleep 10
rich 19642 0.0 0.5 2588 744  pts/0 R 11:56 0:00 ps au
```

启动的每个后台进程都出现在 ps 命令的运行进程输出列表中。如果所有进程都在终端会话中显示输出，那么会变得一团糟。幸运的是，有一种简便的方法可以解决该问题，这将在下一节介绍。

### 13.2.3 退出终端

通过终端会话使用后台进程时必须非常小心。注意 ps 命令的输出，每个后台进程都连接着一个终端会话（pts/0）终端。如果终端会话退出，则后台进程也将退出。

如果与终端相关联的后台程序正在运行，有些终端模拟器会发出警告，而有些则不会。如果希望在注销控制台后脚本继续以后台模式运行，则需要执行一些其他操作。下一节将讨论该过程。

## 13.3 在不使用控制台的情况下运行脚本

有时需要从终端会话启动 shell 脚本，然后让脚本在结束之前以后台模式运行，即使退出终端会话也是如此。可以使用 nohup 命令实现。

nohup 命令运行另一个命令阻塞发送到进程的任何 SIGHUP 信号。这可以防止在退出终端会话时退出进程。

nohup 命令的格式如下：

```
$ nohup ./test1 &
[1] 19831
$ nohup: appending output to 'nohup.out'
$
```

与普通的后台进程一样，shell 给该命令分配一个作业编号，Linux 系统分配一个 PID 编号。不同之处在于，使用 nohup 命令时，关闭会话后脚本将忽略任何终端会话发送的 SIGHUP 信号。

因为 nohup 命令将进程与终端断开，所以进程没有 STDOUT 和 STDERR 输出链接。为了接收命令生成的任何输出，nohup 命令自动将 STDOUT 和 STDERR 消息重定向到称为 nohup.out 的文件。

nohup.out 文件包含通常发送到终端监视器的所有输出。进程运行完成后，可以打开 nohup.out 文件查看输出结果：

```
$ cat nohup.out
This is a test program
Loop #1
Loop #2
Loop #3
Loop #4
Loop #5
Loop #6
Loop #7
Loop #8
Loop #9
Loop #10
This is the end of the test program
$
```

nohup.out 文件中的输出与在命令行运行的进程输出没有区别。

警告：如果使用 nohup 运行另一个命令，输出将添加到现有的 nohup.out 文件。从同一个目录运行多个命令时要小心，因为所有的输出都将发送到同一个 nohup.out 文件，可能造成混淆。

## 13.4 作业控制

本章的前几节介绍了如何使用 CRTL+Z 组合键停止 shell 中运行的作业。停止作业后，Linux 系统会让您终止或重启它。可以使用 kill 命令终止进程。重启停止的进程需要向它发送 SIGCONT 信号。

重启、停止、终止和恢复作业的操作称为**作业控制**（job control）。使用作业控制可以完全控制进程在 shell 环境中运行的方式。

本节将介绍用于查看和控制 shell 中运行的作业的命令。

### 13.4.1 查看作业

作业控制的关键命令是 jobs 命令。jobs 命令可以查看 shell 处理的当前作业：

```
$ cat test4
#!/bin/bash
# testing job control

echo "This is a test program $$"
count=1
```

```
while [ $count -le 10 ]
do
   echo "Loop #$count"
   sleep 10
   count=$[ $count + 1 ]
done
echo "This is the end of the test program"
$ ./test4
This is a test program 29011
Loop #1

[1]+ Stopped                 ./test4
$ ./test4 > test4out &
[2] 28861
$
$ jobs
[1]+ Stopped                 ./test4
[2]- Running                 ./test4 >test4out &
$
```

脚本使用$$变量显示 Linux 系统分配给脚本的 PID，然后进入循环，每次迭代间隔 10 秒。在该例中，我通过命令行界面启动第一个脚本，然后使用 CRTL+Z 组合键停止它。接下来，我使用&符号以后台进程方式启动另一个作业。为简便起见，我将该脚本的输出重定向到一个文件，以便不会在监视器上显示。

两个作业启动之后，我使用 jobs 命令查看分配给 shell 的作业。jobs 命令既可以显示停止的作业也可以显示运行的作业，还包括作业编号以及作业中使用的命令。

jobs 命令使用一些不同的命令行参数，如表 13-2 所示。

**表 13-2　　作业命令参数**

| 参数 | 描述 |
|---|---|
| -l | 列出进程的 PID 和作业编号 |
| -n | 仅列出自上一次 shell 通知以来已经更改了状态的作业 |
| -p | 仅列出作业的 PID |
| -r | 仅列出运行的作业 |
| -s | 仅列出停止的作业 |

您可能已经注意到输出中的加号和减号。带有加号的作业被视为默认作业。如果命令行没有指定作业编号，则它应该是任何作业控制命令引用的作业。带有减号的作业是在处理完当前默认作业之后将成为默认作业的作业。在某一个时间点，只能有一个带有加号的作业，也只能有一个带有减号的作业，无论 shell 中运行了多少作业。

以下示例显示在移除默认作业后，排在下一个的作业如何接管默认状态：

```
$ ./test4
This is a test program 29075
Loop #1

[1]+ Stopped                 ./test4
$ ./test4
This is a test program 29090
Loop #1

[2]+ Stopped                 ./test4
$ ./test4
This is a test program 29105
Loop #1
```

```
[3]+ Stopped                 ./test4
$ jobs -l
[1] 29075 Stopped                 ./test4
[2]- 29090 Stopped                ./test4
[3]+ 29105 Stopped                ./test4
$ kill -9 29105
$ jobs -l
[1]- 29075 Stopped                ./test4
[2]+ 29090 Stopped                ./test4
$
```

在本例中，先启动然后停止了 3 个独立的进程。jobs 命令列表显示了这 3 个进程及其状态。注意，最后启动的进程是默认进程（带有加号的进程）。

然后，我使用 kill 命令将 SIGHUP 信号发送到默认进程。在接下来的作业列表中，之前带有减号的作业现在成为了默认作业。

### 13.4.2 重新启动停止的作业

在 bash 作业控制下，能够以后台进程或前台进程的形式重新启动任何停止的作业。前台进程将控制正在处理的终端，因此请谨慎使用该特性。

要以后台模式重新启动作业，可以使用带有作业编号的 bg 命令：

```
$ bg 2
[2]+ ./test4 &
Loop #2
$ Loop #3
Loop #4

$ jobs
[1]+ Stopped                 ./test4
[2]- Running                 ./test4 &
$ Loop #6
Loop #7
Loop #8
Loop #9
Loop #10
This is the end of the test program

[2]- Done                    ./test4
$
```

我以后台模式重新启动作业，因此出现了命令行界面提示符，使我能够继续处理其他命令。jobs 命令的输出现在显示作业的确正在运行（出现在监视器上的输出表明了这一点）。

要以前台模式重新启动作业，可以使用带有作业编号的 fg 命令：

```
$ jobs
[1]+ Stopped                 ./test4
$ fg 1
./test4
Loop #2
Loop #3
```

作业现在以前台模式运行，因此在作业完成前无法使用命令行界面提示符。

## 13.5 变得更好

在多任务操作系统（包括 Linux）中，内核负责为系统中运行的每个进程分配 CPU 时间。实际上，CPU 中一次只能运行一个进程，因此内核轮流向每个进程分配 CPU 时间。

默认情况下，从 shell 启动的所有进程在 Linux 系统上的**调度优先级**（scheduling priority）都相同。

调度优先级是内核相对其他进程分配个某一进程的CPU时间量。

调度优先级是一个整数值，从-20（最高优先级）到+20（最低优先级）。默认情况下，bash shell启动所有优先级为0的进程。

这表明只需要一点处理时间的简单脚本与需要运行几个小时的复杂数学算法得到的CPU时间段相同。

有时需要更改特定命令的优先级，可能要降低优先级以减少它占用的CPU资源，还可能要提高优先级，以便获得更多的处理时间。这可以使用nice命令实现。

### 13.5.1 nice命令

nice命令可以在启动命令时设置它的调度优先级。要让命令在更低的优先级下运行，只需使用nice的-n命令行选项指定一个新的优先级水平：

```
$ nice -n 10 ./test4 > test4out &
[1] 29476
$ ps al
  F UID   PID  PPID PRI  NI WCHAN  STAT TTY        TIME COMMAND
100 501 29459 29458  12   0 wait4  S    pts/0      0:00 -bash
000 501 29476 29459  15 10 wait4  SN   pts/0      0:00 /bin/bash .
000 501 29490 29476  15 10 nanosl SN   pts/0      0:00 sleep 10
000 501 29491 29459  14   0 -      R    pts/0      0:00 ps al
$
```

nice命令使脚本在更低的优先级运行。但是，如果试图增加其中一个命令的优先级，结果可能会出人意料：

```
$ nice -n -10 ./test4 > test4out &
[1] 29501
$ nice: cannot set priority: Permission denied

[1]+ Exit 1            nice -n -10 ./test4 >test4out
$
```

nice命令不允许普通系统用户增加命令的优先级。这是一个安全特性，防止用户以高优先级启动所有命令。

### 13.5.2 renice命令

有时需要更改已经在系统中运行的命令优先级。这就是renice命令的功能。它能够指定运行进程的PID以更改优先级：

```
$ ./test4 > test4out &
[1] 29504
$ ps al
  F   UID   PID  PPID  PRI NI WCHAN  STAT TTY       TIME COMMAND
100   501 29459 29458   12  0 wait4  S    pts/0      0:00 -bash
000   501 29504 29459    9  0 wait4  S    pts/0      0:00 /bin/bash .
000   501 29518 29504    9  0 nanosl S    pts/0      0:00 sleep 10
000   501 29519 29459   14  0 -      R    pts/0      0:00 ps al
$ renice 10 -p 29504
29504: old priority 0, new priority 10
$ ps al
  F   UID   PID  PPID  PRI NI WCHAN  STAT TTY       TIME COMMAND
100   501 29459 29458   16  0 wait4  S    pts/0      0:00 -bash
000   501 29504 29459   14 10 wait4  SN   pts/0      0:00 /bin/bash .
000   501 29535 29504    9  0 nanosl S    pts/0      0:00 sleep 10
000   501 29537 29459   14  0 -      R    pts/0      0:00 ps al
$
```

renice命令自动更新运行进程的调度优先级。与nice命令一样，renice命令也有几个

限制：

- 只能对拥有的进程使用 renice 命令。
- 只能使用 renice 命令将进程调至更低的优先级。
- 根用户可以使用 renice 命令将任何进程调至任何优先级。

如果要完全控制运行进程，则必须以根账户登录。

## 13.6 准确无误地运行

在开始处理脚本时，总会遇到需要在给定时间（通常是您不在的时候）运行脚本的情况。Linux 系统提供了 3 种方式在预定时间运行脚本：

- at 命令；
- batch 命令；
- cron 表格。

每种方法使用不同的技术调度运行脚本的时间和频率。以下几节将介绍这些方法。

### 13.6.1 使用 at 命令调度作业

at 命令允许指定 Linux 系统运行脚本的时间。at 命令将作业提交到一个队列，并指示 shell 在何时运行该作业。另一个命令 atd 以后台模式运行，并检查作业队列以运行作业。大部分 Linux 发行版都可以在启动时自动启动该特性。

atd 命令检查系统上的特殊目录（通常是/var/spool/at），以便运行使用 at 命令提交的作业。默认情况下，atd 命令每隔 60 秒检查一次该目录。如果存在作业，atd 命令检查运行作业的时间。如果时间与当前时间匹配，则 atd 命令将运行该作业。

以下几节将介绍如何使用 at 命令提交作业以便运行，以及如何管理作业。

**1. at 命令格式**

at 命令的基本格式很简单：

```
at [-f filename] time
```

默认情况下，at 命令将 STDIN 的输入提交到队列。可以使用-f 参数指定用于读取命令的文件名（脚本文件）。

time 参数指定希望 Linux 系统运行作业的时间。指定时间的方式可以发挥创造性。at 命令可以识别各种时间格式：

- 标准的小时和分钟格式，比如 10:15；
- AM/PM 指示符，比如 10:15PM；
- 具体指定的时间，比如 now、noon、midnight、teatime（4PM）。

如果指定一个已经过去的时间，at 命令将在第二天的那个时间运行作业。

除了指定运行作业的时间之外，还可以包括具体的日期，可以使用各种日期格式：

- 标准的日期格式，比如 MMDDYY、MM/DD/YY 或 DD.MM.YY；
- 文本日期格式，比如 Jul 4 或 Dec 25，有没有年份都可以；
- 还可以指定时间增量：
  - Now + 25 minutes
  - 10:15PM tomorrow
  - 10:15 + 7 days

使用 at 命令时，作业将提交到**作业队列**（job queue）中。作业队列保存 at 命令提交的作业以便处理。有 26 种不同的作业队列可用于不同的优先级水平。使用小写字母 a 到 z 引用作业

队列。

默认情况下，所有的 at 作业都提交到作业队列 a，即最高的优先级队列。如果希望以较低的优先级运行作业，则可以使用-q 参数指定字母。

**2. 获取作业输出**

作业在 Linux 系统运行时，没有与该作业关联的监视器。Linux 使用提交作业的用户的电子邮件地址作为 STDOUT 和 STDERR。任何以 STDOUT 和 STDERR 为目的地的输出都将通过邮件系统发送给用户。

以下是使用 at 命令调度作业运行的一个简单示例：

```
$ cat test5
#!/bin/bash
# testing the at command

time='date +%T'
echo "This script ran at $time"
echo "This is the end of the script" >&2
$ date
Sat Nov 3 12:06:04 EST 2007
$ at -f test5 12:07
warning: commands will be executed using /bin/sh
job 6 at 2007-11-03 12:07
$ mail
Mail version 8.1.1 6/6/93. Type ? for help.
"/var/spool/mail/rich": 1 message 1 new
>N 1 rich@testbox Sat Nov 3 12:07 14/474 "Output from your job "
&
Message 1:
From rich Sat Nov 3 12:07:00 2007
Delivered-To: rich@testbox
Subject: Output from your job 6
Date: Sat, 3 Nov 2007 12:07:00 -0500 (EST)
From: rich@testbox (Rich)
To: undisclosed-recipients:;

This script ran at 12:07:00
This is the end of the script

&
```

at 命令生成一个警告消息，指出系统使用什么 shell 运行脚本（默认 shell 分配给/bin/sh，对于 Linux 就是 bash shell），分配给作业的作业编号以及作业计划运行的时间。

作业完成时，监视器上不会显示任何内容，但是系统将生成一个电子邮件消息。电子邮件消息显示脚本生成的输出。如果脚本没有生成输出，则默认情况下不会生成电子邮件消息。可以使用 at 命令的-m 选项更改该特性。在脚本没有生成任何输出的情况下，该选项可以生成一个电子邮件消息，指示作业已完成。

**3. 列出排队的作业**

atq 命令能够查看系统中排队的作业：

```
$ at -f test5 10:15
warning: commands will be executed using /bin/sh
job 7 at 2007-11-04 10:15
$ at -f test5 4PM
warning: commands will be executed using /bin/sh
job 8 at 2007-11-03 16:00
$ at -f test5 1PM tomorrow
warning: commands will be executed using /bin/sh
job 9 at 2007-11-04 13:00
$ atq
```

```
7        2007-11-04 10:15 a
8        2007-11-03 16:00 a
9        2007-11-04 13:00 a
$
```

作业列表显示了作业编号、系统运行该作业的日期和时间，以及存储该作业的作业队列。

**4. 移除作业**

了解了作业队列中排队作业的相关信息之后，便可以使用 atrm 命令移除排队的作业：

```
$ atrm 8
$ atq
7        2007-11-04 10:15 a
9        2007-11-04 13:00 a
$
```

只需要指定希望移除的作业编号即可。只能移除您自己提交执行的作业。不能移除其他人提交的作业。

## 13.6.2 使用 batch 命令

batch 命令与 at 命令稍有不同。batch 命令的作用不是安排脚本在预设的时间运行，而是安排脚本在系统使用率低时运行。

如果 Linux 系统正处于高负载水平，batch 命令将延迟提交作业的运行，直到系统负载降低为止。这对于服务器是个很好的特性，因为服务器在白天和夜晚的负载水平可能大不相同。您可以安排脚本在低负载水平时运行，无需知道确切的时间。

batch 命令检查 Linux 系统当前的平均负载水平。如果平均负载低于 0.8，它将运行任何在作业队列中等待的作业。

batch 命令的命令格式如下：

```
batch [-f filename] [time]
```

与 at 命令类似，默认情况下 batch 命令从 STDIN 读取命令。可以使用-f 参数指定从中读取命令的文件。还可以指定 batch 命令应该尝试运行作业的最早时间（可选）。

## 13.6.3 调度定期脚本

使用 at 命令调度脚本在预设时间运行很不错，但如果需要脚本在每天、每周或每月在同一时间运行，该怎么办呢？这种情况下不需要不断地提交作业，可以使用 Linux 系统的另一种特性。

Linux 系统使用 cron 程序调度需定期运行的作业。cron 程序在后台运行，它从特殊表格（cron 表格）中查找需要调度运行的作业。

**1. cron 表格**

corn 表格使用特殊格式指定作业运行的时间。corn 表格的格式如下：

```
min hour dayofmonth month dayofweek command
```

corn 表格可以将条目指定为具体值、一定范围的值（如 1-5）或通配符（星号）。例如，如果希望每天 10:15 运行命令，可以使用以下 cron 表格项：

```
15 10 * * * command
```

dayofmonth、month 和 *dayofweek* 字段中使用的通配符表示 cron 将在每个月每一天的 10:15 执行该命令。要指定命令在每周一下午 4:15 运行，可以使用：

```
15 16 * * 1 command
```

可以将 *dayofweek* 项指定为 3 字符的文本值（mon、tue、wed、thu、fri、sat、sun）或数字值，使用 0 表示星期天，6 表示星期六。

下面是另一个例子，要在每个月第一天的中午 12 点执行命令，可以使用以下格式：

```
00 12 1 * * command
```

*dayofmonth* 项指定每个月的日期值（1-31）。

注意：精明的读者可能想知道如何设置一个命令在每个月的最后一天执行，因为不能将 *dayofmonth* 设置为涵盖所有月份。这个问题一直让 Linux 和 Unix 程序员苦恼不已，最后他们提出了一些解决方案。一种常用的方法是添加 if-then 语句，该语句使用 date 命令检查明天是否是 1 号：

```
00 12 * * * if [ 'date +%d -d tomorrow' = 01 ] ; then ; command
```

这将在每天 12 点查看本日是否是该月的最后一天，如果是，它将运行命令。

命令列表必须指定要运行的命令或 shell 脚本的完整路径名。可以根据需要添加任何命令行参数或者重定向符号，就像常规命令行一样：

```
15 10 * * * /home/rich/test4 > test4out
```

cron 程序使用提交作业的用户账户运行脚本。因此，您必须拥有相应的权限才能访问命令行列表中指定的命令和输出文件。

**2. 构建 cron 表格**

每个系统用户都可以拥有自己的 cron 表格（包括根用户）以运行安排好的作业。Linux 提供了 crontab 命令处理 cron 表格。要列出现有的 cron 表格，可使用-l 参数：

```
$ crontab -l
no crontab for rich
$
```

默认情况下，每个用户的 cron 表格文件都不存在。要向 cron 表格添加条目，可使用-e 参数。这样做时，crontab 命令自动启动 vi 编辑器（参阅第 7 章）处理现有的 cron 表格（如果没有，则创建一个空文件）。

**3. anacron 程序**

使用 cron 程序存在的唯一一个问题是，它假定 Linux 系统每天运行 24 个小时，每周运行 7 天。只有在服务器环境中运行 Linux 时才可能发生这种情况。

如果在使用 cron 调度作业运行时 Linux 系统处于关闭状态，则作业将无法运行。cron 程序无法在系统打开后重新运行错过的作业。为了解决该问题，许多 Linux 发行版还提供了 anacron 程序。

anacron 程序使用时间戳确定调度的作业是否在正确的时间间隔运行。如果它确定某个作业错过了调度的运行时间，它将自动尽快运行该作业。这意味着，如果 Linux 系统关闭了好几天，当它再次开启时，任何计划在系统关闭期间运行的作业都将自动运行。

该特性对于执行例行日志维护的脚本非常有用。如果系统在应该运行脚本时总是关机，则日志文件将得不到清理，文件大小可能会不断增大。使用 anacron 可以保证日志文件至少在每次系统启动时清理一次。

anacron 程序使用自己的表格（通常位于/etc/anacrontab）指定作业。在大部分 Linux 发行版上，该表格仅供根用户使用。anacron 表格的格式稍微不同于 cron 表格的格式：

```
period delay identifier command
```

period 项定义作业应该间隔多久运行一次，以天为单位。delay 项指定在 anacron 程序确定应该运行一个命令之后需要多长时间才会实际运行该命令。该选项可以为不同的命令设置不同的延迟值，以防止所有命令在一打开 Linux 系统时运行。

identifier 项是一个唯一的非空字符串，可以唯一地标识日志消息和错误电子邮件中的作业。

## 13.7　从头开始

启动 shell 脚本的最后一种方法是，让脚本在 Linux 系统一启动或者用户启动新的 bash shell 会话时便自动运行。在启动时启动脚本的方法通常保留供特殊的脚本使用，这些脚本执行系统功能，比如配置网络接口或者启动服务器进程。但是，如果您是 Linux 系统管理员，在每次 Linux 系统启动时也可能需要执行一些任务，比如重置自定义日志文件或者启动自定义应用程序等。

每次用户启动新的 bash shell 时（甚至在特定用户启动 bash shell 时），运行脚本的功能也很有用。经常会需要设置 shell 特性以进行 shell 会话，有时要确定已经设置了特定的文件。

本节将介绍如何配置 Linux 系统，以在启动时或者每次启动新 bash shell 时运行脚本。

### 13.7.1 在启动时启动脚本

在将 shell 脚本设置为启动时自动启动之前，需要了解一下 Linux 启动过程的工作方式。Linux 在启动时按照一定的顺序启动脚本，了解该过程将有助于您让脚本按照预期的方式执行。

**1. 启动过程**

打开 Linux 系统之后，Linux 内核加载到内存并运行。最先运行的是 `init` 程序。因为 `init` 程序（通常位于`/sbin/init`）总是第一个运行，因此内核总是向它分配 PID 1。然后，`init` 进程负责启动 Linux 系统的所有其他进程。

在启动过程中，`init` 程序读取`/etc/inittab` 文件。`inittab` 文件列出了 `init` 程序以不同的**运行级别**启动的脚本。Linux 运行级别定义 Linux 系统的操作状态。不同的运行级别启动不同的程序和脚本。表 13-3 列出了 Linux 运行级别。

表 13-3 Linux 运行级别

| 运行级别 | 描述 | 运行级别 | 描述 |
|---|---|---|---|
| 0 | 停止 | 4 | 未使用 |
| 1 | 单用户模式 | 5 | 多用户模式，有网络和图形 X Window 会话 |
| 2 | 多用户模式，通常没有网络支持 | 6 | 重启 |
| 3 | 完全多用户模式，有网络 | | |

每个运行级别都定义 `init` 程序启动或终止的脚本。图形 Linux 系统的常见运行级别是 5。大部分 Linux 发行版以运行级别 3 启动所有服务器软件，该级别允许多个用户登录系统。

Linux 系统通过 `rc` 脚本确定以哪种运行级别启动哪些程序。`rc` 脚本确定当前系统的运行级别并在该运行级别上运行相应的脚本。

Linux 系统使用**启动脚本**（startup script）启动应用程序。启动脚本是启动应用程序的 shell 脚本，为运行的程序提供必要的环境变量。

Linux 启动过程的这个部分使事情变得有些混乱，主要是因为不同的 Linux 发行版放置启动脚本的位置稍有不同。有些发行版将启动脚本放在`/etc/rc.d` 目录，每种运行级别使用不同的目录。还有些发行版使用`/etc/init.d` 目录，另一些则使用`/etc/init.d/rc.d` 目录。

通常快速查看一下`/etc` 目录结构便可以轻松确定您的发行版使用的是哪种格式。

**2. 定义脚本**

最好不要弄乱 Linux 发行版中的任何启动脚本文件。发行版通常提供一些工具自动在添加服务器应用程序时构建这些脚本，手动更改这些脚本可能会出现问题。

大部分 Linux 发行版专门提供一个本地启动文件，允许系统管理员输入在启动时运行的脚本。当然，该文件的名称和位置随 Linux 发行版的不同而不同。表 13-4 展示 3 种常见的 Linux 发行版中启动文件的位置。

表 13-4 Linux 本地启动文件的位置

| 发行版 | 文件位置 |
|---|---|
| Debian | `/etc/init.d/rc.local` |
| Fedora | `/etc/rc.d/rc.local` |
| openSuse | `/etc/init.d/boot.local` |

在本地启动文件内部，可以指定特定命令和语句，或者输入希望在系统启动时启动的脚本。记住，如果使用脚本，您必须指定该脚本的完整路径名，以便系统能够在启动时找到它。

警告：不同的 Linux 发行版可能在启动过程中的不同时间点执行本地启动脚本。有些脚本在启动某些功能（如网络支持）之前运行。参考您的 Linux 发行版文档，确定它在何时运行本地启动脚本。

### 13.7.2 随新 shell 一起启动

每个用户的主目录都包含两个文件，bash shell 使用这两个文件自动启动脚本和设置环境变量：

- .bash_profile 文件；
- .bashrc 文件。

bash shell 在由于新用户登录而运行新的 shell 时运行.bash_pfofile 文件。在该文件中放置任何您希望登录时运行的脚本。

bash shell 在每次启动新 shell 时（包括新用户登录时）运行.bashrc 文件。可以向主目录的.bashrc 文件添加一个简单的 echo 语句，然后启动一个新 shell 以测试该特性：

```
$ bash
This is a new shell!!
$
```

如果希望为系统的每个用户运行脚本，可以使用大部分 Linux 发行版提供的/etc/bashrc 文件（注意，bashrc 文件名前面没有点）。bash shell 在系统任何用户启动一个新 bash shell 时执行该文件中的语句。

## 13.8 小结

Linux 系统允许使用信号控制 shell 脚本。bash shell 可以接收信号，并能将信号传递到 shell 进程下运行的任何进程。Linux 信号可以轻松终止失控的进程或者临时暂停长时间运行的进程。

可以使用脚本的 trap 语句捕获信号和执行命令。该特性提供了一种简单的方式控制用户是否能够在运行时中断您的脚本。

默认情况下，在终端会话 shell 中运行脚本时，交互 shell 在脚本完成之前将会挂起。可以在命令名后面添加&符号，让脚本或命令以后台模式运行。以后台模式运行脚本或命令时，交互 shell 返回，您可以输入更多的命令。使用该方法运行的后台进程仍然与终端会话绑定。如果退出了终端会话，后台进程也将退出。

要避免这种情况，可使用 nohup 命令。该命令拦截任何会停止该进程的信号，例如，退出终端会话的信号。这样可以使脚本在终端会话退出时也能继续以后台模式运行。

以后台模式运行进程时，也可以控制进程的运行。jobs 命令可以查看从 shell 会话启动的进程。了解后台进程的作业 ID 之后，可以使用 kill 命令将 Linux 信号发送给该进程，或者使用 fg 命令将进程带回 shell 会话的前台。可以使用 Crtl+C 组合键挂起运行的前台进程，然后使用 bg 命令将其返回为后台模式。

nice 和 renice 命令能够更改进程的优先级别。如果进程获得较低的优先级别，则分配给它的 CPU 时间较少。在运行可能占用很多 CPU 时间的长进程时，该命令非常方便。

除了在运行时控制进程外，还可以确定进程何时在系统中启动。不仅可以从命令行界面提示符直接运行脚本，还可以安排进程在可选的时间运行。有好几种方法可以实现这一点。at 和 batch 命令可以在预设置的时间运行脚本。cron 程序可以以定期的时间间隔运行脚本。

最后，Linux 系统提供了一些脚本文件，用于让脚本在系统启动或者用户启动新的 bash shell 时运行。rc.local（对于 openSuse 是 boot.local）文件可以列出每次系统启动时启动的脚本。这可以帮助系统管理员在系统启动时运行特定脚本，以进行系统维护。与此类似，.bash_profile 和.bashrc 文件位于每个用户的主目录，提供一个位置放置与新 shell 一起运行的脚本和命令。.bash_profile 文件在每次用户登录系统时运行脚本，.bashrc 文件在每个新 shell 启动时运行脚本。

在下一章中，我们将介绍如何编写脚本函数。脚本函数只需要编写一次代码块即可在脚本的多个位置使用。

PART

3

# 高级 shell 脚本编程

本部分内容：

第 14 章
创建函数

第 15 章
在脚本中添加颜色

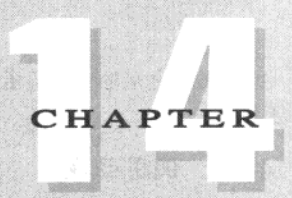
第 16 章
sed 和 gawk 介绍

第 17 章
正则表达式

第 18 章
高级 sed 编程

第 19 章
高级 gawk 编程

# 第 14 章
# 创建函数




本章内容：

- 创建函数；
- 使用参数；
- 共享函数。

shell 脚本的编写通常有多处使用相同的代码。如果这些代码比较短小，那么通常并不费事。但是，如果有大段代码需要重复编写，那会让人厌烦。bash shell 脚本支持用户自定义函数，可以帮助解决这个问题。shell 脚本代码可封装在函数内，而函数可在脚本任意处使用。本章引导读者学习创建 shell 脚本函数，并演示如何在其他 shell 脚本应用程序中使用这些函数。

## 14.1 基本脚本函数

编写比较复杂的 shell 脚本时，完成具体任务的代码可能需要重复使用。这些代码有的比较简单，例如显示文本消息和从脚本用户获取回应；有的代码是比较复杂的计算，是较大进程的组成部分并在脚本中多次使用。

在每种情况下，重复编写相同的脚本代码块令人厌烦。重复代码块最好只需编写一次，然后在脚本的任意位置就可以引用这个代码块，而不必重复编写。

bash shell 提供满足这种要求的特性。**函数**是被赋予名称的脚本代码块，可以在代码的任意位置重用。每当需要在脚本中使用这样的代码块时，只需引用该代码块被赋予的函数名称（称为**调用**该函数）。本节讲述在 shell 脚本中如何创建和使用函数。

### 14.1.1 创建函数

在 bash shell 脚本中创建函数可以使用两种格式。一种格式是使用关键字 function，后跟代码块的函数名：

```
function name {
    commands
}
```

name 属性定义了该函数的唯一名称。脚本中自定义的每个函数都必须赋予唯一的名称。

commands 是组成函数的一条或多条 bash shell 命令。当调用函数时，就像在普通脚本中一样，bash shell 按照各条命令在函数中出现的顺序依次执行。

在 bash shell 脚本中定义函数还有另一个格式，它更接近一般编程语言定义函数的方式：

```
name() {
commands
}
```

函数名之后的空白圆括号表明这是在定义函数。该格式的函数命名规则与前面的 shell 脚本函数格式相同。

### 14.1.2 使用函数

要在脚本中使用函数，相应函数名需要在脚本行中指定，这与其他 shell 命令的用法一样：

```
$ cat test1
#!/bin/bash
# using a function in a script

function func1 {
   echo "This is an example of a function"
}

count=1
while [ $count -le 5 ]
do
   func1
   count=$[ $count + 1 ]
done
echo "This is the end of the loop"
func1
echo "Now this is the end of the script"
$ ./test1
This is an example of a function
This is an example of a function
This is an example of a function
This is an example of a function
This is an example of a function
This is the end of the loop
This is an example of a function
Now this is the end of the script
$
```

函数名 func1 每次被引用时，bash shell 返回 func1 函数的定义并执行该函数定义的全部命令。

函数定义不一定位于 shell 脚本的起始部分，但是应当小心使用。如果在函数定义之前使用函数，会得到错误消息：

```
$ cat test2
#!/bin/bash
# using a function located in the middle of a script

count=1
echo "This line comes before the function definition"

function func1 {
   echo "This is an example of a function"
}

while [ $count -le 5 ]
do
   func1
   count=$[ $count + 1 ]
done
echo "This is the end of the loop"
func2
echo "Now this is the end of the script"
```

```
function func2 {
   echo "This is an example of a function"
}
$ ./test2
This line comes before the function definition
This is an example of a function
This is an example of a function
This is an example of a function
This is an example of a function
This is an example of a function
This is the end of the loop
./test2: func2: command not found
Now this is the end of the script
$
```

这个脚本的第一个函数 func1，定义在两条语句之后，能够正常工作。在脚本中使用函数 func1 时，shell 知道这个函数的位置。

但是该脚本试图在 func2 定义之前使用它。当脚本到达使用该函数的位置时，由于函数 func2 尚未定义，所以有错误消息生成。

函数的命名也需要注意。请记住，每个函数名必须唯一，否则会出问题。如果重新定义函数，那么新定义将取代函数原先的定义，这不会引发错误消息。

```
$ cat test3
#!/bin/bash
# testing using a duplicate function name

function func1 {
echo "This is the first definition of the function name"
}

func1

function func1 {
   echo "This is a repeat of the same function name"
}

func1
echo "This is the end of the script"
$ ./test3
This is the first definition of the function name
This is a repeat of the same function name
This is the end of the script
$
```

函数 func1 原先的定义能够正常工作。但是在第二个定义之后，对该函数的任何调用将使用第二个定义。

## 14.2 返回值

bash shell 将函数看作小型脚本，并以退出状态结束（参阅第 8 章）。函数退出状态有 3 种生成方式。

### 14.2.1 默认退出状态

默认情况下，函数的退出状态是函数的最后一条命令返回的退出状态。函数执行完毕之后，可以使用标准变量$?来确定函数的退出状态：

```
$ cat test4
#!/bin/bash
# testing the exit status of a function
```

```
func1() {
   echo "trying to display a non-existent file"
   ls -l badfile
}

echo "testing the function:"
func1
echo "The exit status is: $?"
$ ./test4
testing the function:
trying to display a non-existent file
ls: badfile: No such file or directory
The exit status is: 1
$
```

该函数的退出状态是 1，因为它的最后一条命令执行失败。但是，这样没有办法知道该函数的其他命令是否成功执行。请看下例：

```
$ cat test4b
#!/bin/bash
# testing the exit status of a function

func1() {
  ls -l badfile
   echo "This was a test of a bad command"
}

echo "testing the function:"
func1
echo "The exit status is: $?"
$ ./test4b
testing the function:
ls: badfile: No such file or directory
This was a test of a bad command
The exit status is: 0
$
```

这时，由于函数结尾是能够完全成功执行的 echo 语句，虽然该函数有一条命令运行失败，但是函数的退出状态仍然是 0。所以，使用函数的默认退出状态是一种危险的做法。幸运的是还有另外两种解决方法。

### 14.2.2 使用 return 命令

bash shell 使用 return 命令以特定退出状态退出函数。return 命令可以使用单个整数值来定义函数退出状态，提供了一种通过编程设置函数退出状态的简单方法：

```
$ cat test5
#!/bin/bash
# using the return command in a function

function dbl {
   read -p "Enter a value: " value
   echo "doubling the value"
   return $[ $value * 2 ]
}

dbl
echo "The new value is $?"
$
```

函数 dbl 将变量$value 中的用户输入值变为双倍后，使用 return 命令返回结果。脚本使用变量$?显示该结果。

但是使用这种方法从函数返回数据时，必须注意避免两个容易发生的错误：

- 请记住在函数完成后尽快提取返回值。
- 请记住退出状态的取值范围是 0～255。

如果在通过使用变量$?提取函数值之前执行了其他命令，那么该函数的返回值将丢失（请记住，变量$?返回最近已执行命令的退出状态）。

第二个情况说明了这种方法的限制。由于退出状态必须小于 256，所以函数结果生成的整数值必须小于 256。数据大于 256 将返回错误值：

```
$ ./test5
Enter a value: 200
doubling the value
The new value is 1
$
```

如果想要返回更大的整数值或字符串，那么不要使用这种返回值方法。替代方法将在下一节讲述。

### 14.2.3 使用函数输出

正如命令输出可以捕获并存放到 shell 变量中一样，函数的输出也可以捕获并存放到 shell 变量中。这种方法可以从函数获取任意类型的输出并给变量赋值：

```
result='db1'
```

这条命令将函数 db1 的输出赋予 shell 变量$result。在脚本中使用这种方法的示例如下：

```
$ cat test5b
#!/bin/bash
# using the echo to return a value

function dbl {
   read -p "Enter a value: " value
   echo $[ $value * 2 ]
}

result='dbl'
echo "The new value is $result"
$ ./test5b
Enter a value: 200
The new value is 400
$ ./test5b
Enter a value: 1000
The new value is 2000
$
```

这个新函数使用 echo 语句显示计算结果。该脚本仅捕获函数 db1 的输出，而不查看结果的退出状态。

本例展示了一个窍门。函数 db1 实际输出两条消息。read 命令输出一条较短的消息向用户询问取值。bash shell 脚本比较聪明，不会将此消息作为 STDOUT 输出的组成部分，而将其忽略。假如使用 echo 语句生成这条消息，那么 shell 变量将捕获这条消息，对于输出值也是如此。

注意：这个方法也可以返回浮点数和字符串值，所以这种方法能够非常灵活地从函数返回数据。

## 14.3 在函数中使用变量

从上一节的 test5 示例可以看出，函数内变量$value 存放需要处理的数据。在函数中使用变量，应当多注意如何定义和处理这些变量。这是 shell 脚本问题的一个常见原因。本节探究 shell 脚本函数内外变量处理的一些方法。

### 14.3.1 向函数传递参数

正如前面的“返回值”一节所述，bash shell 将函数作为小型脚本处理。这意味着可以仿照普通脚本（参阅第 11 章）向函数传递参数。

函数可以使用标准参数环境变量来表示命令行传递给函数的参数。例如，函数名在变量$0 中定义，函数命令行的其他参数使用变量$1 和$2 等定义。专用变量$#可以用来确定传递给函数的参数数目。

在脚本中指定函数时，必须在函数所在命令行提供参数，如下所示：

```
func1 $value1 10
```

这样，该函数才可以使用参数环境变量获取参数值。使用这种方法向函数传递值，如下所示：

```
$ cat test6
#!/bin/bash
# passing parameters to a function

function addem {
   if [ $# -eq 0 ] || [ $# -gt 2 ]
   then
      echo -1
   elif [ $# -eq 1 ]
   then
      echo $[ $1 + $1 ]
   else
      echo $[ $1 + $2 ]
   fi
}

echo -n "Adding 10 and 15: "
value='addem 10 15'
echo $value
echo -n "Let's try adding just one number: "
value='addem 10'
echo $value
echo -n "Now trying adding no numbers: "
value='addem'
echo $value
echo -n "Finally, try adding three numbers: "
value='addem 10 15 20'
echo $value
$ ./test6
Adding 10 and 15: 25
Let's try adding just one number: 20
Now trying adding no numbers: -1
Finally, try adding three numbers: -1
$
```

脚本 test6 的函数 addem 首先检查传递给它的参数数目。如果没有任何参数，或者参数数目超过两个，那么返回值-1。如果只有一个参数，那么这个参数与自己相加作为结果。如果有两个参数，那么两个参数相加作为结果。

由于函数为自己的参数值使用专用的参数环境变量，所以函数无法从脚本命令行直接访问脚本参数值。下面的例子将出错：

```
$ cat badtest1
#!/bin/bash
# trying to access script parameters inside a function

function badfunc1 {
   echo $[ $1 * $2 ]
}

if [ $# -eq 2 ]
```

```
then
   value='badfunc1'
   echo "The result is $value"
else
   echo "Usage: badtest1 a b"
fi
$ ./badtest1
Usage: badtest1 a b
$ ./badtest1 10 15
./badtest1: * : syntax error: operand expected (error token is "*
")
The result is
$
```

该函数使用的变量$1 和$2，不同于脚本主代码的变量$1 和$2。如果想在函数中使用这些值，那么必须在调用该函数时手动传递这些数据：

```
$ cat test7
#!/bin/bash
# trying to access script parameters inside a function

function func7 {
   echo $[ $1 * $2 ]
}

if [ $# -eq 2 ]
then
   value='func7 $1 $2'
   echo "The result is $value"
else
   echo "Usage: badtest1 a b"
fi
$ ./test7
Usage: badtest1 a b
$ ./test7 10 15
The result is 150
$
```

通过将变量$1 和$2 传递给函数，函数可以像使用其他参数一样使用这两个变量。

### 14.3.2 在函数中处理变量

变量**作用域**是 shell 脚本程序员遇到的常见问题。作用域是变量的可见区域。函数内定义的变量与普通变量有不同的作用域，前者能被脚本外部定义的变量所覆盖。

函数使用两种类型的变量：

- ✦ 全局变量；
- ✦ 局部变量。

下面两节介绍在函数中如何使用这两类变量。

#### 1. 全局变量

**全局变量**是在 shell 脚本内处处有效的变量。如果脚本的主代码定义了全局变量，那么在函数内部可以获取这个变量的值。同样，如果在函数内部定义了全局变量，那么脚本的主代码也可以获取该变量的值。

默认情况下，脚本中定义的变量都是全局变量。在函数外部定义的变量，在函数内部仍能正常访问：

```
$ cat test8
#!/bin/bash
# using a global variable to pass a value

function dbl {
   value=$[ $value * 2 ]
```

```
}

read -p "Enter a value: " value
dbl
echo "The new value is: $value"
$ ./test8
Enter a value: 450
The new value is: 900
$
```

变量$value在函数外部定义，并在函数外部赋值。当调用函数db1时，这个变量及其数据在函数内部仍然有效。该变量在函数内部赋值之后，它的新数据在脚本引用该变量时仍然有效。

这种做法有风险，尤其对多个shell脚本共用的函数。这种做法要求程序员确切清楚函数中使用了哪些变量，包括那些用于计算值且不返回脚本的所有变量。下面的例子说明这种做法的风险：

```
$ cat badtest2
#!/bin/bash
# demonstrating a bad use of variables

function func1 {
   temp=$[ $value + 5 ]
   result=$[ $temp * 2 ]
}

temp=4
value=6

func1
echo "The result is $result"
if [ $temp -gt $value ]
then
   echo "temp is larger"
else
   echo "temp is smaller"
fi
$ ./badtest2
The result is 22
temp is larger
$
```

因为变量$temp已在函数内部使用，而又在函数外部赋值，所以生成的结果可能出人意料。这个问题从函数内部容易解决，具体方法将在下一节讲述。

**2. 局部变量**

与全局变量相对，函数内部使用的变量可以称为局部变量。这只需在变量声明前面冠以local关键字：

```
local temp
```

也可以给局部变量赋值语句冠以local关键字：

```
local temp=$[ $value + 5 ]
```

关键字local确保变量仅在函数内部使用。如果脚本在函数外部有同名变量，那么shell将能区分开这两个变量。这样就可以很容易地将函数变量与脚本变量分开，而只共享需要的变量：

```
$ cat test9
#!/bin/bash
# demonstrating the local keyword

function func1 {
   local temp=$[ $value + 5 ]
   result=$[ $temp * 2 ]
}

temp=4
```

```
value=6

func1
echo "The result is $result"
if [ $temp -gt $value ]
then
   echo "temp is larger"
else
   echo "temp is smaller"
fi
$ ./test9
The result is 22
temp is smaller
$
```

这样，在函数 func1 内部使用变量$temp 时，脚本主代码变量$temp 的值不会受到影响。

## 14.4 数组变量与函数

第 5 章讨论了使用一个数组变量存放多个数据的高级方法。函数使用数组变量值需要一些技巧，还有一些特别注意事项。本节将帮助读者掌握相关方法。

### 14.4.1 向函数传递数组

向脚本函数传递数组变量的技术不太容易理解。如果试图将数组变量作为单个参数传递，是无法正常工作的：

```
$ cat badtest3
#!/bin/bash
# trying to pass an array variable

function testit {
   echo "The parameters are: $@"
   thisarray=$1
   echo "The received array is ${thisarray[*]}"
}

myarray=(1 2 3 4 5)
echo "The original array is: ${myarray[*]}"
testit $myarray
$ ./badtest3
The original array is: 1 2 3 4 5
The parameters are: 1
./badtest3: thisarray[*]: bad array subscript
The received array is
$
```

如果试图将数组变量作为函数参数使用，那么函数只提取数组变量的第一个取值。

要解决这个问题，必须将数组变量拆分为单个元素，然后使用这些元素的值作为函数参数。函数内部可以再将这些参数重组为新数组变量，如下例所示：

```
$ cat test10
#!/bin/bash
# array variable to function test

function testit {
   local newarray
   newarray=('echo "$@"')
   echo "The new array value is: ${newarray[*]}"
}

myarray=(1 2 3 4 5)
```

```
echo "The original array is ${myarray[*]}"
testit ${myarray[*]}
$ ./test10
The original array is 1 2 3 4 5
The new array value is: 1 2 3 4 5
$
```

该脚本使用变量$myarray存放所有数组值，并将它们都放在函数命令行。函数命令行随后根据命令行参数重建数组变量。这样，函数内部可以像使用其他数组一样使用这个数组：

```
$ cat test11
#!/bin/bash
# adding values in an array

function addarray {
   local sum=0
   local newarray
   newarray=('echo "$@"')
   for value in ${newarray[*]}
   do
      sum=$[ $sum + $value ]
   done
   echo $sum
}

myarray=(1 2 3 4 5)
echo "The original array is: ${myarray[*]}"
arg1='echo ${myarray[*]}'
result='addarray $arg1'
echo "The result is $result"
$ ./test11
The original array is: 1 2 3 4 5
The result is 15
$
```

函数addarray对数组数据递推求和。任意数目的值都可以放在数组变量myarray中，并且函数addarray将对其求和。

### 14.4.2 从函数返回数组

类似方法也可用于从函数向shell脚本回传数组变量。函数使用echo语句以恰当顺序输出数组各元素的值，然后脚本必须将这些数据重组为新数组变量：

```
$ cat test12
#!/bin/bash
# returning an array value

function arraydblr {
   local origarray
   local newarray
   local elements
   local i
   origarray=('echo "$@"')
   newarray=('echo "$@"')
   elements=$[ $# - 1 ]
   for (( i = 0; i <= $elements; i++ ))
   {
      newarray[$i]=$[ ${origarray[$i]} * 2 ]
   }
   echo ${newarray[*]}
}

myarray=(1 2 3 4 5)
echo "The original array is: ${myarray[*]}"
```

```
arg1='echo ${myarray[*]}'
result=('arraydblr $arg1')
echo "The new array is: ${result[*]}"
$ ./test12
The original array is: 1 2 3 4 5
The new array is: 2 4 6 8 10
```

这个脚本用变量$arg1 将数组值传递给函数 arraydblr。函数 arraydblr 将该数组重组为新数组变量，并将后者复制后作为输出数组变量。然后，该函数对数组变量的元素数据做递推，每个元素值双倍后，将其存放到复制的函数数组变量中。

之后，函数 arraydblr 使用 echo 语句输出数组变量的各元素值。脚本将函数 arraydblr 的输出按照数据重组为新数组变量。

## 14.5 函数递归

**自给**（self-containment）是局部函数变量的一个特性。自给函数除了脚本通过命令行传递的变量，不使用函数之外的任何资源。

这种特性使函数能够以**递归**方式调用。递归调用函数是指函数调用自身进行求解。通常，递归函数有基值，函数最终递推到达该值。许多高级数学算法使用递归将复杂等式的递归层次反复降低，直到到达基值指定的层次。

递归算法的一个经典示例是计算阶乘。一个数的阶乘是这个数乘以它前面的所有数的积。因此，要计算 5 的阶乘，求解下列等式：

```
5! = 1 * 2 * 3 * 4 * 5 = 120
```

这个等式使用递归可以归结为：

```
x! = x * (x-1)!
```

或者用自然语言描述为 x 的阶乘等于 x 乘以 x-1 的阶乘。这可以表述为简单的递归脚本：

```
function factorial {
   if [ $1 -eq 1 ]
   then
      echo 1
   else
      local temp=$[ $1 - 1 ]
      local result='factorial $temp'
      echo $[ $result * $1 ]
   fi
}
```

该阶乘函数调用自身计算阶乘：

```
$ cat test13
#!/bin/bash
# using recursion

function factorial {
   if [ $1 -eq 1 ]
   then
      echo 1
   else
      local temp=$[ $1 - 1 ]
      local result='factorial $temp'
      echo $[ $result * $1 ]
   fi
}

read -p "Enter value: " value
result='factorial $value'
echo "The factorial of $value is: $result"
```

```
$ ./test13
Enter value: 5
The factorial of 5 is: 120
$
```

使用递归函数很简单。

## 14.6 创建库

显然，函数在单个脚本中有助于减少输入量，但是如果多个脚本碰巧使用同样的函数又该怎样？如果每个脚本都对相同的函数做出定义，且各个脚本对该函数只调用一次，那么上述函数定义和调用的做法看似没有减少重复代码。

这种问题有个解决方法。bash shell 可以创建函数的**库文件**，然后可以在不同脚本中引用该库文件。

这种方法首先要创建公共库文件，包含多个脚本需要调用的函数。一个简单库文件称为 myfuncs，定义了 3 个简单函数，如下所示：

```
$ cat myfuncs
# my script functions

function addem {
   echo $[ $1 + $2 ]
}

function multem {
   echo $[ $1 * $2 ]
}

function divem {
   if [ $2 -ne 0 ]
   then
      echo $[ $1 / $2 ]
   else
      echo -1
   fi
}
$
```

下一步是将库文件 myfuncs 包含进需要调用库函数的脚本文件。这是难点所在。

其难点在于 shell 函数的作用域。与环境变量一样，shell 函数仅在其定义所处的 shell 会话中有效。如果从 shell 命令行界面运行 myfuncs 脚本，那么 shell 将打开一个新 shell，并在该新 shell 中运行此脚本。这将为新 shell 定义 3 个函数，但是当试图运行调用这些库函数的另一个脚本时，库函数并不能使用。

这也适用于脚本。如果试图将库文件作为普通脚本文件运行，那么这些函数不会出现在脚本文件中：

```
$ cat badtest4
#!/bin/bash
# using a library file the wrong way
./myfuncs

result='addem 10 15'
echo "The result is $result"
$ ./badtest4
./badtest3: addem: command not found
The result is
$
```

使用函数库的关键是 source 命令。source 命令在当前 shell 环境中执行命令，而非创建新 shell 来执行命令。使用 source 命令在 shell 脚本内部运行库文件脚本。这样脚本可以使用这些函数。

source 有一个短小的别名，称为**点操作符**。为了在 shell 脚本中调用 myfuncs 库文件，只需添

加下列命令行：

```
. ./myfuncs
```

该示例假定 myfuncs 库文件与 shell 脚本在同一目录。如果二者不在同一目录，那么应当使用恰当的路径来访问库文件。下例创建一个使用 myfuncs 库文件的脚本：

```
$ cat test14
#!/bin/bash
# using functions defined in a library file
. ./myfuncs

value1=10
value2=5
result1='addem $value1 $value2'
result2='multem $value1 $value2'
result3='divem $value1 $value2'
echo "The result of adding them is: $result1"
echo "The result of multiplying them is: $result2"
echo "The result of dividing them is: $result3"
$ ./test14
The result of adding them is: 15
The result of multiplying them is: 50
The result of dividing them is: 2
$
```

该脚本成功调用了在 myfuncs 库文件中定义的函数。

## 14.7 在命令行中使用函数

脚本函数可以用于创建比较复杂的操作。有时希望能够在命令行界面直接使用这些函数。

脚本函数不仅可以用作 shell 脚本命令，也可以用作命令行界面的命令。这是个很好的特性。因为一旦在 shell 中定义了函数，那么就可以从系统的任意目录使用这个函数。不必担心 PATH 环境变量是否包含函数文件所在目录。关键是能使 shell 识别出这个函数。这有两种方法。

### 14.7.1 在命令行创建函数

shell 在键盘输入命令时解释命令，函数可以直接在命令行定义。这有两种方法。

第一种方法将函数定义在一行命令中：

```
$ function divem { echo $[ $1 / $2 ]; }
$ divem 100 5
20
$
```

在命令行中定义函数时，每条命令的结尾必须包含分号，这样 shell 才知道命令在哪里分开：

```
$ function doubleit { read -p "Enter value: " value; echo $[
 $ value * 2 ]; }
$ doubleit
Enter value: 20
40
$
```

第二种方法是使用多行命令定义函数。这样，bash shell 使用次提示符提示输入更多命令。使用这种方法不需要在每条命令的结尾添加分号，只需按 ENTER 键：

```
$ function multem {
> echo $[ $1 * $2 ]
> }
$ multem 2 5
10
$
```

在函数末尾使用大括号时，shell 知道定义函数结束。

警告：使用命令行来创建函数应当十分小心。如果自定义函数与内置命令或其他命令同名，那么这个自定义函数将覆盖原有命令。

### 14.7.2 在.bashrc文件中定义函数

显然，直接在命令行定义shell函数的缺点是一旦退出shell，函数定义将失效。对于复杂函数，这多少是个问题。

比较省事的方法是将函数定义放在shell每次启动都能重新载入的地方。

.bashrc文件就是这样的地方。每次无论bash shell是交互式启动，还是从已有shell启动新shell，都会在主目录下查找这个文件

**1. 直接定义函数**

在主目录下的.bashrc文件中可以直接定义函数。大多数Linux发行版都已经在.bashrc文件中定义了一些内容，应当小心不要删除这些内容。只需在已有文件的末尾添加自定义函数，如下所示：

```
$ cat .bashrc
# .bashrc

# Source global definitions
if [ -r /etc/bashrc ]; then
        . /etc/bashrc
fi

function addem {
echo $[ $1 + $2 ]
}
$
```

该函数将在下一次启动新bash shell时生效。这样，该函数就可以在系统任意位置使用了。

**2. 提供函数文件**

正如在shell脚本中的做法一样，可以使用source命令（或称点操作符）将现有库文件的函数包含进.bashrc脚本：

```
$ cat .bashrc
# .bashrc

# Source global definitions
if [ -r /etc/bashrc ]; then
        . /etc/bashrc
fi

. /home/rich/libraries/myfuncs
$
```

确保包含库文件的准确路径名，这样bash shell才能够找到该库文件。再次启动shell之后，该库的所有函数将都能在命令行界面使用：

```
$ addem 10 5
15
$ multem 10 5
50
$ divem 10 5
2
$
```

更好的是，shell还将全部已定义函数传递给子shell进程。这样，对于所有从shell会话运行的脚本，自定义函数随之可用。为了验证这个优点，可以编写有自定义函数调用的脚本，且对自定义函数不加定义或供给，如下所示：

```
$ cat test15
#!/bin/bash
# using a function defined in the .bashrc file
```

```
value1=10
value2=5
result1='addem $value1 $value2'
result2='multem $value1 $value2'
result3='divem $value1 $value2'
echo "The result of adding them is: $result1"
echo "The result of multiplying them is: $result2"
echo "The result of dividing them is: $result3"
$ ./test15
The result of adding them is: 15
The result of multiplying them is: 50
The result of dividing them is: 2
$
```

由此可见，即使不提供库文件，相关函数仍然可以在 shell 脚本中正常工作。

## 14.8 小结

shell 脚本函数支持将脚本中重复出现的代码归到一处。这样，不必重复编写代码块，而是创建包含该代码块的函数，然后在脚本中引用其函数名。每当 bash shell 检查到脚本引用了函数名，它就跳转到该函数的代码块。

甚至可以创建有返回值的脚本函数。这种特性支持创建与脚本互动的函数，能够返回数值型和字符型数据。脚本函数返回数值型数据，可以借助函数最后一条命令的退出状态，也可以使用 `return` 命令。该命令支持以编程方式，根据函数结果将函数退出状态设置为特定值。

函数还可以使用标准 `echo` 语句返回数据。就像在其他 shell 命令中一样，可以使用反引号符号来捕获输出数据。这种方法能够从函数返回任意类型的数据，包括字符串和浮点数。

函数内部可以使用 shell 变量，给变量赋值或者从已有变量提取数据。这种特性能够支持脚本主代码与脚本函数之间交换任意类型的数据。函数还支持定义局部变量，这些变量只能在函数代码块内部使用。局部变量支持创建自给函数。这样的函数不会影响脚本主代码的任何变量和流程。

函数可以调用别的函数，也可以调用自身。当函数调用自身时，该操作称为递归。递归函数常常有一个基值，它是函数递归的结束值。函数持续调用自身并不断减小某个参数的取值，直到该参数取值到达基值。

如果 shell 脚本使用了大量函数，那么可以创建脚本函数的库文件。通过使用 `source` 命令或它的别名点操作符，可以将库文件包含进任何 shell 脚本文件。这称为提供库文件。shell 不运行库文件，而是使函数在运行该脚本的 shell 内可以使用。同样方法可以创建用在普通 shell 命令行的函数。函数可以直接在命令行定义；也可以将函数定义添加到 `.bashrc` 文件，这样每个新启动的 shell 会话都可以使用这些函数。这种简便方法可用于创建不受 `PATH` 环境变量设置制约的工具。

下一章将讨论在脚本中使用文本图形的问题。在这个图形化界面的时代，有时纯文本界面显得土里土气。为给脚本添加简单图形特性，bash shell 提供了易用的方法，来帮助增加脚本运行的趣味。

CHAPTER 15

# 第 15 章
# 在脚本中添加颜色

本章内容：

- 创建文本菜单；
- 使脚本丰富多彩；
- 文本窗口小部件；
- 使用 X Windows 图形。

多年来，shell 脚本一直被认为是单调乏味的。但是，如果计划在图形环境中运行脚本，就未必是这样。除 read 和 echo 语句之外，还有其他许多与脚本用户交互的方法。本章将深入介绍几种不同的方法，这些方法可以增加交互式脚本的活力，使其看上去更符合潮流。

## 15.1 创建文本菜单

创建交互式 shell 脚本最常见的方法是利用菜单。为用户提供各种选项有助于指导他们了解脚本的确切功能。

菜单脚本通常会清除显示区域，然后显示一个可用的选项列表。客户可以通过按下为各选项指定的相关联的字母或数字选择一个选项。图 15-1 给出了一个菜单的布局示例。

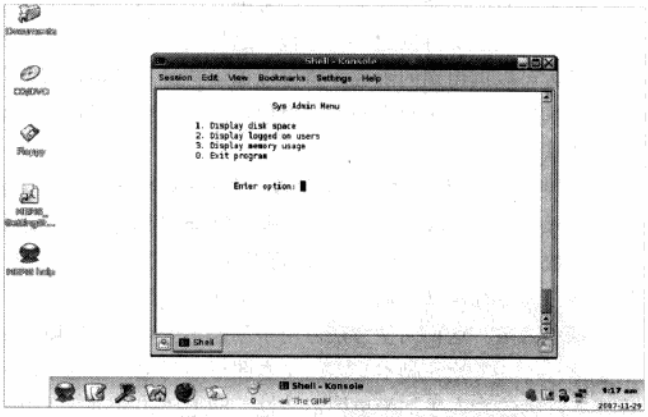



图 15-1 从 shell 脚本显示一个菜单

shell 脚本菜单的核心是 case 命令（参阅第 9 章）。case 命令根据用户在菜单中选择的字母执行特定的命令。

接下来几节将讨论创建基于菜单的 shell 脚本的步骤。

### 15.1.1 创建菜单布局

显然，创建菜单的第一步是确定菜单中要显示的元素和各元素的布局方式。

在创建菜单之前，一般要先清除监视器显示。这样可以在空白的环境中显示菜单而没有令人分心的文本。

clear 命令使用终端会话（参阅第 2 章）的信息数据清除显示在监视器上的所有文本。在 clear 命令之后，可以使用 echo 命令显示菜单元素。

默认情况下，echo 命令仅显示可打印文本字符。在创建菜单项时，通常使用不可打印项，例如制表符和换行符。要在 echo 命令中包含这些字符，必须使用-e 选项。这样，命令：

```
echo -e "1.\tDisplay disk space"
```

生成的输出行如下：

```
1.      Display disk space
```

在设置菜单项的格式时，这一点特别有用。仅使用几个 echo 命令，就可以创建外观很好的菜单：

```
clear
echo
echo -e "\t\t\tSys Admin Menu\n"
echo -e "\t1. Display disk space"
echo -e "\t2. Display logged on users"
echo -e "\t3. Display memory usage"
echo -e "\t0. Exit menu\n\n"
echo -en "\t\tEnter option: "
```

最后一行中的-en 选项将显示该行，而不在行尾添加换行符。这样可以使该菜单看上去更专业，因为光标会停留在该行的行尾，等待客户的输入。

创建菜单的最后一部分是获取客户的输入。获取客户的输入要用 read 命令来完成(参阅第 11 章)。由于仅期望单字母输入，因此最好是在 read 命令中使用-n 选项仅获取一个字符。这允许用户输入一个数字而不必按 Enter 键：

```
read -n 1 option
```

接下来，需要创建菜单函数。

### 15.1.2 创建菜单函数

shell 脚本菜单选项作为一组独立的函数创建更容易。这允许您创建一个容易理解的简明的 case 命令。

为此，需要为每个菜单选项创建独立的 shell 函数。创建菜单 shell 脚本的第一步是确定脚本要执行的函数，并将其作为代码中独立的函数来设计。

常见的做法是为未实现的函数创建**桩函数**（stub function）。桩函数是尚未包含任何命令，或者可能仅有一个 echo 语句说明此处最终应该放置的内容的函数：

```
function diskspace {
   clear
   echo "This is where the diskspace commands will go"
}
```

这样可在调试各个函数时可以使菜单顺利操作。不必为要使用菜单的所有函数进行编码。您会注意到，这些函数的开头是 clear 命令。这样能够使您在不显示菜单的空白监视器屏幕上启动函数。

在 shell 脚本菜单中有助于解决问题的一种做法是将菜单布局本身作为函数来创建：

```
function menu {
   clear
   echo
   echo -e "\t\t\tSys Admin Menu\n"
   echo -e "\t1. Display disk space"
   echo -e "\t2. Display logged on users"
   echo -e "\t3. Display memory usage"
   echo -e "\t0. Exit program\n\n"
   echo -en "\t\tEnter option: "
   read -n 1 option
}
```

这样只需调用 menu 函数就可以很容易地随时再显示该菜单。

### 15.1.3　添加菜单逻辑

现在，已经有了菜单布局和菜单函数，所需要做的只是创建编程逻辑将二者组合在一起。如前所述，这需要 case 命令。

case 命令根据从菜单选择的字符调用合适的函数。使用默认的 case 命令字符（星号）捕获任意错误的菜单输入始终是一个好办法。

在典型菜单中使用 case 命令如下所示：

```
menu
case $option in
0)
   break ;;
1)
   diskspace ;;
2)
   whoseon ;;
3)
   memusage ;;
*)
   clear
   echo "Sorry, wrong selection";;
esac
```

这段代码首先使用菜单函数清屏并显示菜单。菜单函数中的 read 命令暂停，等待用户在键盘上敲击一个字符。用户敲击了字符之后，case 命令启动。case 命令根据返回的字符调用合适的函数。函数执行完之后，case 命令退出。

### 15.1.4　将其全部组合在一起

现在，您已经看到了构成 shell 脚本菜单的全部部件，让我们将它们组合在一起，看一看它们如何相互影响。下面是一个完整菜单脚本的示例：

```
$ cat menu1
#!/bin/bash
# simple script menu

function diskspace {
   clear
   df -k
}

function whoseon {
   clear
   who
}

function memusage {
   clear
   cat /proc/meminfo
}

function menu {
   clear
   echo
   echo -e "\t\t\tSys Admin Menu\n"
   echo -e "\t1. Display disk space"
   echo -e "\t2. Display logged on users"
   echo -e "\t3. Display memory usage"
   echo -e "\t0. Exit program\n\n"
```

```
    echo -en "\t\tEnter option: "
    read -n 1 option
}

while [ 1 ]
do
    menu
    case $option in
    0)
        break ;;
    1)
        diskspace ;;
    2)
        whoseon ;;
    3)
        memusage ;;
    *)
        clear
        echo "Sorry, wrong selection";;
    esac
    echo -en "\n\n\t\t\tHit any key to continue"
    read -n 1 line
done
clear
$
```

该菜单使用常见命令创建 3 个函数，获取关于 Linux 系统的管理信息。这里使用了一个 `while` 循环来不断地重复菜单，直到用户选择选项 0。选项 0 将使用 `break` 命令跳出 `while` 循环。

该模板同样适用于创建任何 shell 脚本菜单界面。它提供了与用户交互的一种简单方法。

### 15.1.5 使用 select 命令

您可能已经注意到，创建文本菜单的大多数问题就是创建菜单布局和获取输入的答案。bash shell 提供了一个便利的小工具，可以自动完成所有这些工作。

`select` 命令允许从单命令行创建菜单，然后获取输入的答案并自动处理它。`select` 命令的格式是：

```
select variable in list
do
    commands
done
```

列表参数是用空格隔开的构建菜单的文本项列表。`select` 命令将列表中的每一项显示为一个编号选项，然后为选择显示一个特殊的提示符（由 PS3 环境变量定义）。

下面是 `select` 命令的一个简单示例：

```
$ cat smenu1
#!/bin/bash
# using select in the menu

function diskspace {
    clear
    df -k
}

function whoseon {
    clear
    who
}

function memusage {
    clear
```

```
    cat /proc/meminfo
}

PS3="Enter option: "
select option in "Display disk space" "Display logged on users"
"Display memory usage" "Exit program"
do
    case $option in
    "Exit program")
          break ;;
    "Display disk space")
          diskspace ;;
    "Display logged on users")
          whoseon ;;
    "Display memory usage")
          memusage ;;
    *)
          clear
          echo "Sorry, wrong selection";;
    esac
done
clear
$
```

运行该程序时，它自动生成下面的菜单：

```
$ ./smenu1
1) Display disk space       3) Display memory usage
2) Display logged on users  4) Exit program
Enter option:
```

使用 `select` 命令时，要记住：存储在变量中的结果值是整个文本字符串，而不是与该菜单项相关联的数字。文本字符串值才是需要在 `case` 语句中比较的内容。

## 15.2　添加颜色

在 Unix 的早期，脚本程序员只能使用单色的终端，因此交互式 shell 脚本不必担心要使用颜色。而现在，由于各种终端模拟包的出现，为交互式脚本添加颜色和特殊效果几乎成为必需的。本节将讨论和演示如何向脚本输出添加特殊文本特性。

### 15.2.1　ANSI 转义码

大多数终端模拟软件能够识别设置显示输出格式的 ANSI 转义码。ANSI 转义码以**控制序列指示器**（control sequence indicator，CSI）开头，后面跟表示要在显示器上执行的操作的数据。CSI 告诉终端该数据代表一个转义码。

有些 ANSI 转义码可以用于将光标定位在显示器上的指定位置，擦除部分显示，以及控制显示的格式（我们目前感兴趣的内容）。要控制显示格式，必须使用**选择图形再现**（Select Graphic Rendition，SGR）转义码。SGE 转义码的格式为：

```
CSIn[;k]m
```

该代码中的 m 代表 SGR 转义码。n 和 k 参数定义所使用的显示控制。可以仅指定一个参数或者同时指定两个，中间用分号隔开。显示控制参数有 3 类：

- ✦ 效果控制代码；
- ✦ 前景色控制代码；
- ✦ 背景色控制代码。

表 15-1 列出了现有的效果控制代码。

表 15-1 ANSI SGR 效果控制代码

| 代码 | 描述 | 代码 | 描述 |
|---|---|---|---|
| 0 | 重置为普通模式 | 5 | 使用慢闪烁 |
| 1 | 设置为强亮度 | 6 | 使用快闪烁 |
| 2 | 设置为弱亮度 | 7 | 背景、前景色反转 |
| 3 | 使用斜体 | 8 | 将前景色设置为背景色（文字不可见） |
| 4 | 使用单下划线 | | |

因此，要将显示设置为使用倾斜的字体，要发送代码：

```
CSI3m
```

如果要将显示设置为使用斜体和闪烁，可以发送代码：

```
CSI3;5m
```

前景色控制码和背景色控制码都使用两位数代码。前景色使用 3 开头的一个两位数的值，而背景色使用 4 开头的两位数的值。其中的第二位数字表示具体颜色。表 15-2 列出了现有的颜色控制码。

表 15-2 ANSI 颜色控制代码

| 代码 | 描述 | 代码 | 描述 |
|---|---|---|---|
| 0 | 黑色 | 4 | 蓝色 |
| 1 | 红色 | 5 | 洋红色 |
| 2 | 绿色 | 6 | 青色 |
| 3 | 黄色 | 7 | 白色 |

因此，要指定白色前景，发送代码：

```
CSI37m
```

该代码指定 3 表示前景色，7 表示白色。要设置为白色的背景，可以使用代码：

```
CSI47m
```

在各控制码中，可以将这两种属性组合在一起。因此，要将背景色设置为黑色，前景色设置为红色，可以发送代码：

```
CSI31;40m
```

接下来介绍如何将实际代码发送到终端。

## 15.2.2 显示 ANSI 转义码

现在，您已经了解了 ANSI 转义码，可能想要知道如何在 shell 脚本中的使用它们。可以使用 echo 命令，像发送普通文本一样，将 ANSI 转义码发送到终端会话。唯一比较难的部分是创建 CSI 字符。

CSI 字符通常是一个两字符序列。这个序列是 ESC ASCII 值，后跟左方括号字符。在脚本中创建 ESC ASCII 值可能是一个难题。

显然，不能仅敲击键盘上的 Esc 键，因为大多数文本编辑器将其解释为其他内容。幸运的是，有一个大多数编辑器都能够识别的常用键序列允许在脚本中插入 ESC ASCII 值。这就是 Ctrl-V 组合键，后跟 Esc 键。在输入此组合键时，字符^[出现。

Note 注意：在使用 ANSI 转义码的脚本中常常会看到^[字符。看到该字符组合时，记住它是使用 Ctrl-V Esc 组合键生成的。

因此，一个完整的 ANSI 转义控制代码看上去应该是这样：

```
^[[0m
```

这样可以指定效果控制代码 0，它将把显示重置为默认设置。

可以在命令行界面提示符下使用 ANSI 转义控制码进行测试：

```
$ echo ^[[41mThis is a test
This is a test
$
```

当然，您无法从书上的例子判断，但是这个语句能够将终端屏幕上的背景设置为红色。文本以默认的前景色显示而将红色作为背景色。如果在终端模拟器中完成，就会注意到在 shell 打印出 echo 命令中的文本之后，新的提示符仍然使用该颜色控制码。这是要记住的重要一点。

警告：ANSI 颜色控制码会保持有效，直到另一个 ANSI 颜色控制码改变输出。

要解决这一问题，通常较好的方法是使用重置控制码（0）将终端重置为正常显示：

```
$ echo ^[[41mThis is a test^[[0m
This is a test
$
```

这次测试之后，新的 shell 提示符将恢复其原来的颜色方案。

这一点同样适用于效果控制码：

```
$ echo ^[[1mThis is a test^[[0m
This is a test
$
```

echo 命令显示的文本在终端模拟器中应该用粗体显示，然后命令行界面提示符应该恢复正常。

注意：某些终端模拟器不支持效果控制码。例如，我的 PuTTY 终端模拟器软件不能闪烁显示用闪烁转义控制码标记的文本。相反，它仅用灰色背景表示加闪烁的文本。

如果需要同时设置背景色和前景色，ANSI 转义控制码允许在一个转义序列中指定两个代码：

```
$ echo "^[[33;44mThis is a test^[[0m"
This is a test
$
```

这个命令仅使用一个转义控制码序列将前景色设置为黄色，背景色设置为蓝色。

警告：在一个 echo 命令中放置两个转义控制码时，重要的是要用双引号将代码字符串引起来。没有双引号，echo 命令就不能正确地解释转义码，进而会产生错误消息。

## 15.2.3 在脚本中使用颜色

现在可以将 ANSI 转义控制码添加到 shell 脚本中，以控制脚本输出的颜色格式。在使用菜单时尤为便利，因为可以通过控制颜色将客户的注意力集中在菜单的某一特定部分。

然而使用 ANSI 转义控制码创建脚本时必须谨慎。记住，无论何时当终端模拟器遇到控制码，它都会处理。使用 cat 命令对含有 ANSI 转义控制码的脚本进行列表时，这尤其危险。cat 命令将在显示器上回显该代码，然后终端模拟器解释这些代码，并改变显示。在改变很多显示特性的长脚本中，这会是一件十分恼人的事情。

下面是一个脚本菜单修改版，它利用颜色控制代码使菜单看起来更活泼一些：

```
$ cat menu2
#!/bin/bash
# menu using colors

function diskspace {
   clear
   df -k
}

function whoseon {
   clear
   who
}

function memusage {
   clear
   cat /proc/meminfo
}
```

```
function menu {
   clear
   echo
   echo -e "\t\t\tSys Admin Menu\n"
   echo -e "\t1. Display disk space"
   echo -e "\t2. Display logged on users"
   echo -e "\t3. Display memory usage"
   echo -e "^[[1m\t0. Exit program\n\n^[[0m^[[44;33m"
   echo -en "\t\tEnter option: "
   read -n 1 option
}

echo "^[[44;33m"
while [ 1 ]
do
   menu
   case $option in
   0)
      break ;;
   1)
      diskspace ;;
   2)
      whoseon ;;
   3)
      memusage ;;
   *)
    clear
    echo -e "^[[5m\t\t\tSorry, wrong selection^[[0m^[[44;33m";;
   esac
   echo -en "\n\n\t\t\tHit any key to continue"
   read -n 1 line
done
echo "^[[0m"
clear
$
```

菜单在蓝色的背景上以黄色的文本样式显示。注意：在使用 clear 命令之前，重要的是要设置前景色和背景色；否则 clear 命令将用默认的颜色给终端模拟屏幕着色，从而使菜单项与背景的其他部分不协调。

在脚本中使用粗体（1）或闪烁（5）控制代码突出显示重要文本时，由于没有设置非粗体或非闪烁的控制代码，所以必须将前景色和背景色更改默认设置。遗憾的是，重置控制代码仅将显示设置回默认值，而非您之前使用的颜色。

## 15.3 制作窗口

使用文本菜单和颜色是朝正确方向迈出的一步，但是在交互式脚本中仍然有很多不足，特别是当我们尝试将它们与图形化的 Windows 世界相对比时。对我们来说幸运的一点是，在这个开源的世界，一些非常聪明的人可以帮助我们找到出路。

dialog 软件包是一个极好的小工具，最初由 Savio Lam 设计，现今由 Thomas E. Dickey 维护。这个软件包利用 ANSI 转义控制码在文本环境中重新创建了标准 Windows 对话框。可以很方便地将这些对话框合并到 shell 脚本中来与脚本用户进行交互。本节将讨论该 dialog 软件包，说明在 shell 脚本中如何应用它。

### 15.3.1 dialog 软件包

dialog 命令使用命令行参数确定生成的 Windows 小部件的类型。小部件（widget）是一类 Windows 元素的 dialog 软件包术语。dialog 软件包目前支持的小部件类型如表 15-3 所示。

表 15-3　对话框小部件

| 小部件 | 描述 |
| --- | --- |
| calendar | 提供了一个可以在其中选择日期的日历 |
| checklist | 显示多个项目，这些项目均可以被打开或关闭 |
| form | 允许建立一个可以填写标签和文本字段的表单 |
| fselect | 提供一个文件选择窗口以浏览文件 |
| gauge | 显示一个说明完成百分比的仪表 |
| infobox | 显示信息而不等待响应 |
| inputbox | 显示单一的文本窗体框以输入文本 |
| inputmenu | 提供一个可编辑的菜单 |
| menu | 显示一系列可供选择的选项 |
| msgbox | 显示消息并要求用户选择 OK 按钮 |
| pause | 显示一个说明指定暂停阶段状态的仪表 |
| passwordbox | 显示一个可以隐藏输入文本的单一文本框 |
| passwordform | 显示一个带标签和隐藏文本字段的表单 |
| radiolist | 提供一组菜单项，但是只可选择一个项目 |
| tailbox | 利用 tail 命令在滚动的窗口中显示文件中的文本 |
| tailboxbg | 与 tailbox 相同，但是需在背景模式下操作 |
| texbox | 在滚动窗口中显示文件的内容 |
| timebox | 提供一个用来选择时、分、秒的窗口 |
| yesno | 提供一个含 Yes/No 按钮的简单消息 |

从表 15-3 可以看出，有许多不同的小部件可供选择。这样就可以毫不费力地使脚本具备更专业的外观。

要在命令行上具体指定一个特定的小部件，需要使用双破折号格式：

```
dialog --widget parameters
```

其中，widget 表示的是表 15-3 中所示的小部件的名称，parameters 定义该小部件窗口的大小以及它所需的任何文本。

每个对话框小部件均以两种形式提供输出：

- 使用 STDERR；
- 使用退出代码状态。

dialog 命令的退出代码状态决定用户所选择的按钮，如果选择 OK 或 Yes 按钮，该 dialog 命令将返回 0 退出状态。如果选择 Cancel 或 No 按钮，该 dialog 命令返回 1 退出状态。可以使用标准 $? 变量确定在该对话框小部件中选择的按钮。

如果小部件返回任何数据，例如一个菜单选项，dialog 命令将该数据发送到 STDERR。可以使用标准的 bash shell 技术将 STDERR 输出重定向至另一个文件或文件描述符：

```
dialog --inputbox "Enter your age:" 10 20 2>age.txt
```

此命令将文本框中输入的文本重定向到 age.txt 文件。

下面几节将介绍一些例子，说明在 shell 脚本中使用较多的对话框小部件。

### 1. msgbox 小部件

msgbox 小部件是最常见的对话框类型。它在窗口中显示简单的信息，并等待用户单击 OK 按钮后关闭。使用 msgbox 小部件所要求的格式是：

```
dialog --msgbox text height width
```

text 参数是任意要放置在窗口中的字符串。dialog 命令将使用 height 和 width 参数自动调整文本以适应所创建的窗口大小。如果要在窗口顶部放置一个标题，也可以使用--title 参数以及标题文本。下面是使用 msgbox 小部件的例子：

```
$ dialog --title Testing --msgbox "This is a test" 10 20
```

在输入此命令后，消息框将出现在您正在使用的终端模拟器会话的屏幕上。如图 15-2 所示。

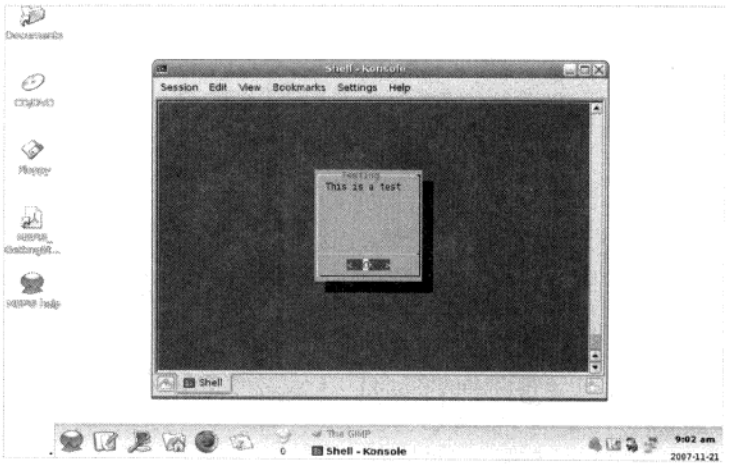



图 15-2 在 dialog 命令中使用 msgbox 小部件

如果您的终端模拟器支持鼠标，可以单击 OK 按钮。也可以使用键盘命令来模拟单击，只需按 Enter 键即可。

### 2. yesno 小部件

yesno 小部件比 msgbox 小部件更进了一步。它允许用户回答显示在窗口中的 yes/no 问题。它会在窗口底部生成两个按钮，分别为 Yes 和 No。用户可以使用鼠标、tab 键或键盘方向键在两者之间切换。要选择按钮，用户可以按空格键或 Enter 键。

下面是使用 yesno 小部件的一个例子：

```
$ dialog --title "Please answer" --yesno "Is this thing on?" 10 20
$ echo $?
1
$
```

它所产生的小部件如图 15-3 所示。

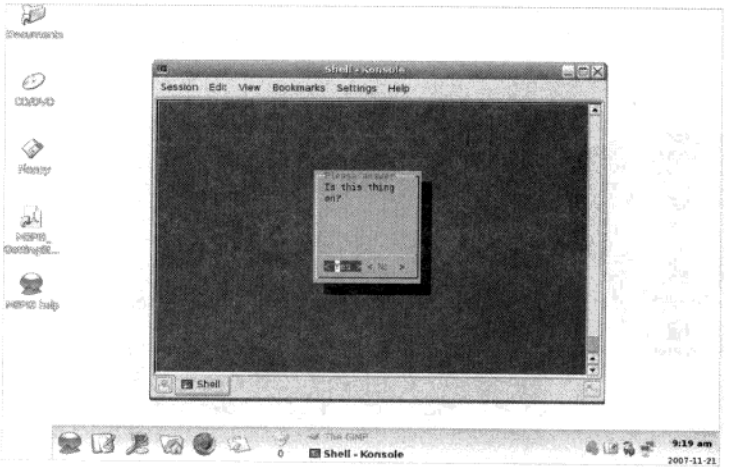



图 15-3 在 dialog 命令中使用 yesno 小部件

dialog 命令的退出状态根据用户选择的按钮设置。如果选择的是 No 按钮，退出状态为 1。

### 3. 输入框小部件

inputbox 小部件为用户输入文本字符串提供了一个简单的文本框。dialog 命令将文本字符串的值发送到 STDERR。必须重定向此值来获取答案。图 15-4 显示了 inputbo 小部件的外观。

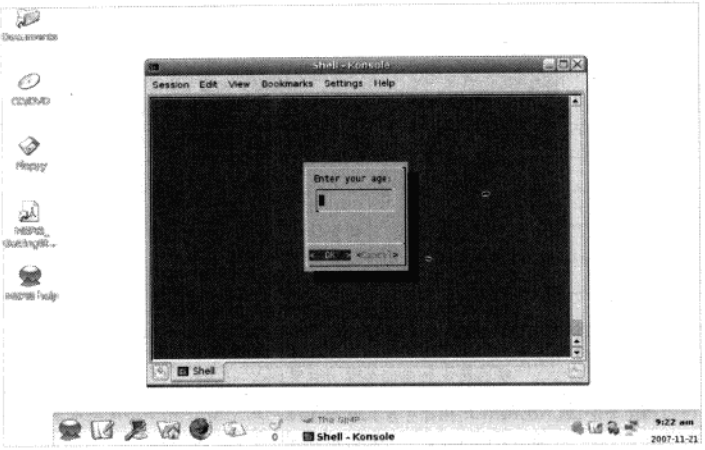



图 15-4　inputbox 小部件

如图 15-4 所示，inputbox 提供了两个按钮，OK 和 Cancel。如果选择 Cancel 按钮，则该命令的退出状态为 1，否则退出状态是 0：

```
$ dialog --inputbox "Enter your age:" 10 20 2>age.txt
$ echo $?
0
$ cat age.txt
12$
```

您会注意到，使用 cat 命令显示文本文件的内容时，值的后面没有换行符。这使您可以轻松地将文件内容重定向到 shell 脚本中的变量，以提取用户输入的字符串。

### 4. 文本框小部件

textbox 小部件是在一个窗口内显示大量信息的好方法。它可以提供一个滚动的窗口，此窗口包含参数指定的文件的文本。

```
$ dialog --textbox /etc/passwd 15 45
```

/etc/passwd 文件的内容将在滚动的文本窗口内显示，如图 15-5 所示。

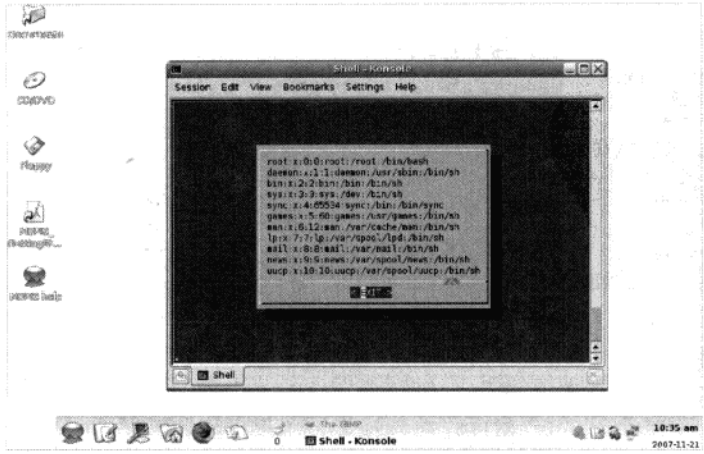



图 15-5　文本框小部件

可以使用方向键在文本文件中上下左右移动。窗口的最后一行显示当前所看的内容在文本中的百分比位置。文本框中仅仅有一个 Exit 按钮供选择来退出此小部件。

**5. 菜单小部件**

菜单小部件允许创建在本章前面创建的文本菜单的窗口版本。所需要做的只是为每个菜单项提供一个选择标签和文本：

```
$ dialog --menu "Sys Admin Menu" 20 30 10 1 "Display disk space"
2 "Display users" 3 "Display memory usage" 4 "Exit" 2>text.txt
```

第一个参数定义菜单的标题。后两个参数定义菜单窗口的高度和宽度，而第三个参数定义同时出现在窗口中的菜单项数。如果有更多的菜单项，可以使用方向键滚动。

在这些参数之后，必须添加菜单项目对。第一个元素是用于选择该菜单项的标签。对于每个菜单项，这个标签应该是唯一的，并且能够通过敲击键盘上合适的键来选择。第二个元素是在菜单中使用的文本。图 15-6 显示了本示例命令生成的菜单。

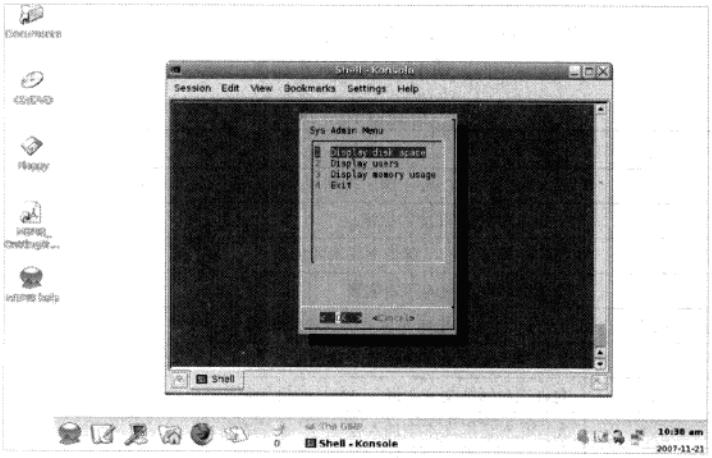



图 15-6 带菜单选项的菜单小部件

如果用户通过按该标签的合适的键选择菜单项目，该菜单项将突出显示但不被选中。只有通过鼠标或 Enter 键选择 OK 按钮之后，选择操作才真正生效。dialog 命令将选中的菜单项文本发送给 STDERR，您可以根据需要重定向。

**6. fselect 小部件**

dialog 命令提供了一些有趣的内置小部件。使用文件名时，fselect 小部件极其方便。无需强制用户输入文件名，而是利用 fselect 小部件浏览文件从而选择文件，如图 15-7 所示。

fselect 小部件的格式如下：

```
$ dialog -title "Select a file" --fselect $HOME/ 10 50 2>file.txt
```

fselect 后的第一个参数是窗口所使用的起始位置。Fselect 小部件窗口包括三部分：左侧的目录列表；右侧的文件列表，用以显示选定目录中的所有文件；以及包含当前选定文件或目录的简单文本框。可以在文本框中手动输入文件名，或者使用目录和文件列表选择一个文件。

## 15.3.2 dialog 选项

除标准小部件之外，还可以在 dialog 命令中自定义许多不同的选项。之前您已经了解了--title 参数的作用。它允许为小部件设置一个标题，显示在窗口顶部。

还有其他许多可用的选项允许您完全自定义窗口的外观和行为。表 15-4 列出了 dialog 命令可用的选项。

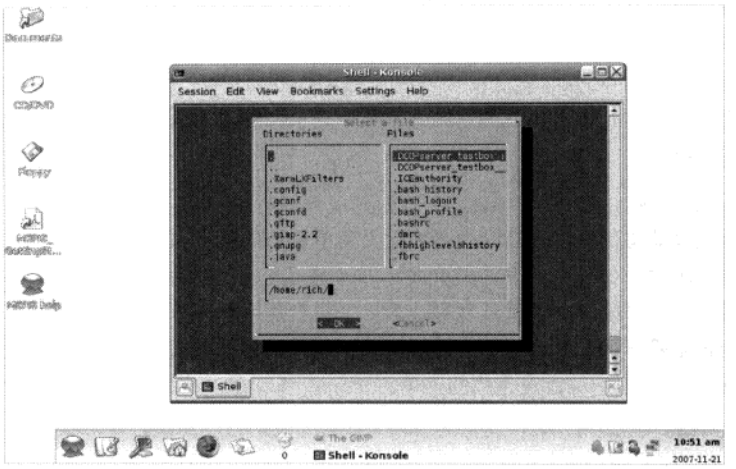



图 15-7　fselect 小部件

表 15-4　　dialog 命令选项

| 选项 | 描述 |
|---|---|
| --add-widged | 除非按 Esc 或 Cancel 键，否则继续进入下一个对话框 |
| --aspect *ratio* | 指定窗口宽度/高度的纵横比 |
| --backtitle *title* | 指定在屏幕顶部、背景上显示的标题 |
| --begin *x y* | 指定窗口左上角的起始位置 |
| --cancel-label *label* | 为 Cancel 按钮指定一个可供选择的标签 |
| --clear | 使用对话框默认背景色清除显示 |
| --colors | 允许在对话框文本中嵌入 ANSI 颜色代码 |
| --cr-wrap | 允许在对话框文本中加入换行符，强制换行 |
| --create-rc *file* | 转储样本配置文件至指定文件 |
| --defaultno | 将 yes/no 对话框的默认值设置为 NO |
| --default-item *string* | 设置检查列表、表单或菜单对话框的默认项目 |
| --exit-label *label* | 为 Exit 按钮指定可供选择的标签 |
| --extra-button | 在 OK 和 Cancel 按钮之间，显示一个其他按钮 |
| --extra-label *label* | 为 Extra 按钮指定可供选择的标签 |
| --help | 显示 dialog 命令的帮助信息 |
| --help-button | 在 OK 和 Cancel 按扭之后显示 Help 按钮 |
| --help-label label | 为 Help 按钮指定可供选择的标签 |
| --help-status | 在 Help 按钮中的帮助信息被选中后，记录下检查列表、单选框列表或表单信息 |
| --ignore | 忽略 dialog 不能识别的选项 |
| --input-fd *fd* | 除 STDIN 之外，指定其他可供选择的文件描述符 |
| --insecure | 更改 password 小部件以在输入时以星号显示 |
| --item-help | 在屏幕的底部，为检查列表、单选框列表、菜单中的每一个标签添加该标签的帮助 |
| --keep-window | 不要清除屏幕上的旧小部件 |
| --max-input *size* | 指定输入字符串的最大规模。默认为 2048 |
| --nocancel | 禁止 Cancel 按钮 |
| --no-collapse | 在对话框文本中，不要将制表换行符转换为空格 |
| --no-kill | 将 tailboxbg 对话框放在背景中，并禁止该进程的 SIGHUP |

续表

| 选项 | 描述 |
| --- | --- |
| --no-label *label* | 为 No 按钮指定一个可供选择的标签 |
| --no-shadow | 不要为对话框窗口显示阴影 |
| --ok-label *label* | 为 OK 按钮指定一个可供选择的标签 |
| --output-fd *fd* | 除 STDERR 之外，指定可供选择的输出文件描述符 |
| --print-maxsize | 打印对话框窗口允许输出的最大尺寸 |
| --print-size | 将每个对话框窗口的尺寸打印至输出 |
| --print-version | 将对话框版本打印至输出 |
| --separator-output | 无需引用，将 checklist 小部件的结果一次一行的输出 |
| --separator *string* | 指定一个字符串来隔离每个小部件的输出 |
| --separator-widget *string* | 指定一个字符串来隔离每个小部件的输出 |
| --shadow | 绘制阴影至每一个窗口的右下方 |
| --single-quoted | 需要时对检查列表输出使用单引用 |
| --sleep *sec* | 在处理对话窗口后延迟指定的秒数 |
| --stderr | 将输出发送至 STDERR（这是默认行为） |
| --stdout | 将输出发送至 STDOUT |
| --tab-correct | 将换行符转换为空格 |
| --tab-len *n* | 指定一个制表符所使用的空格数（默认值是 8） |
| --timeout *sec* | 指定在没有用户输入的情况下，用错误代码退出所需的秒数 |
| --title *title* | 指定对话框窗口的名称 |
| --trim | 删除对话框文本中字符串首部的空格和换行符 |
| --visit-*items* | 为了把项目列表包含在内，而修改对话框窗口中的制表符 |
| --yes-label *label* | 为 Yes 按钮指定一个可供选择的标签 |

--backtitle 选项是创建贯穿脚本的通用菜单标题的一种十分方便的方法。如果为每个对话框窗口指定，该选项将贯穿应用程序始终，为脚本创建专业级的外观。

正如表 15-4 所示，可以改写对话框窗口中任意按钮的标签。此功能可以使您创建所需要的任何窗口环境。

### 15.3.3 在脚本中使用 dialog 命令

在脚本中使用 dialog 命令是一件非常容易的事。只需记住两点：

- 如果 Cancel 或 No 按钮可用，请检查 dialog 命令的退出状态。
- 重定向 STDERR 以获取输出值。

如果遵循以上两条规则，就可以立即获得专业外观的交互式脚本。下面是一个使用对话框小部件重现本章前面创建的系统管理菜单的例子：

```
$ cat menu3
#!/bin/bash
# using dialog to create a menu

temp='mktemp -t test.XXXXXX'
temp2='mktemp -t test2.XXXXXX'

function diskspace {
   df -k > $temp
   dialog --textbox $temp 20 60
}
```

```
function whoseon {
   who > $temp
   dialog --textbox $temp 20 50
}

function memusage {
   cat /proc/meminfo > $temp
   dialog --textbox $temp 20 50
}

while [ 1 ]
do
dialog --menu "Sys Admin Menu" 20 30 10 1 "Display disk space" 2
"Display users" 3 "Display memory usage" 0 "Exit" 2> $temp2
if [ $? -eq 1 ]
then
   break
fi
selection='cat $temp2'

case $selection in
1)
   diskspace ;;
2)
   whoseon ;;
3)
   memusage ;;
0)
   break ;;
*)
   dialog --msgbox "Sorry, invalid selection" 10 30
esac
done
rm -f $temp 2> /dev/null
rm -f $temp2 2> /dev/null
$
```

该脚本使用 `while` 循环（值恒为 **true**）创建显示菜单对话框的无限循环。这意味着，在每一次功能实现后，该脚本都会返回到显示菜单。对话菜单如图 15-8 所示。

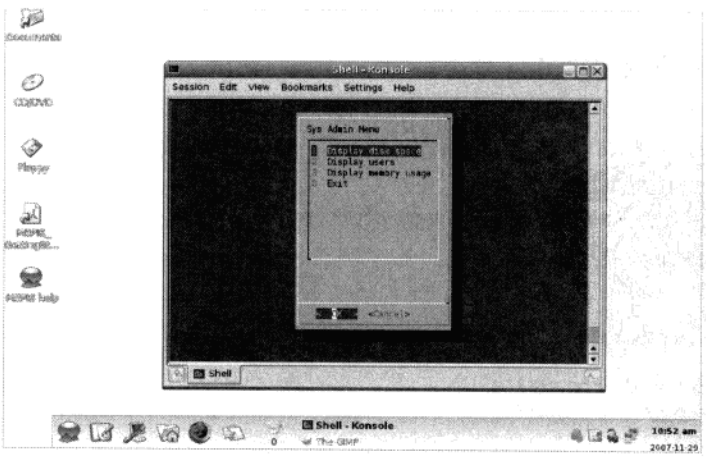



图 15-8　使用 dialog 命令的脚本菜单

菜单对话框包含一个 `Cancel` 按钮，因此该脚本需要检查 `dialog` 命令的退出状态，以防用户按 **Cancel** 按钮退出。由于是在 `while` 循环中，所以退出就像使用 `break` 命令跳出 `while` 循环一

样简单。

该脚本使用 mktemp 函数命令创建两个临时文件暂存 dialog 命令的数据。第一个文件$temp 用于保存 df 命令的输出，以便其可以显示在文本框对话框中。第二个临时文件$temp2 用于保存在主菜单对话框中选择的值。

## 15.4 获取图形

如果正在为交互式脚本寻找更多的图形，就可以更进一步。无论是 KDE 还是 GNOME 桌面环境（参阅第 1 章）都扩充了 dialog 命令的思想，在各自环境中包含了生成 X Windows 图形小部件的命令。

本节将介绍 kdialog 和 zenity 软件包，它们分别为 KDE 和 GNOME 桌面提供图形化的窗口小部件。

### 15.4.1 KDE 环境

KDE 图形环境默认包括 kdialog 软件包。在 KDE 桌面环境下，kdialog 软件包使用 kdialog 命令生成标准窗口，类似对话类型的小部件。这使您可以直接在 shell 脚本中创建 Windows 质量的用户界面。

**1. kdialog 小部件**

正如 dialog 命令一样，kdialog 命令使用命令行选项指定使用的窗口小部件的类型。kdialog 命令的格式为：

```
kdialog display-options window-options arguments
```

window-option 选项允许您指定使用的窗口小部件的类型。可用的选项如表 15-5 所示。

表 15-5 kdialog 窗口选项

| 选项 | 描述 |
|---|---|
| --checklist title[*tag item status*] | 检查列表菜单，其状态表明是否选中了该项目 |
| --error text | 错误消息框 |
| --inputbox text[*init*] | 输入文本框。可以使用 init 值来指定默认值 |
| --menu title[*tag item*] | 菜单选择框标题和通过标签辨认的项目列表 |
| --msgbox text | 带有指定文本的简单消息框 |
| --password text | 可以隐藏用户输入的密码输入文本框 |
| --radiolist title[*tag item status*] | 单选框列表菜单，其状态表明是否选中了该项目 |
| --separate-output | 以独立行返回检查列表和单选框列表的菜单项目 |
| --sorry text | Sorry 消息框 |
| --textbox file[*width*][*height*] | 显示 file 内容的文本框，可以选择指定宽度和高度 |
| --title *title* | 为 dialog 窗口的 TitleBar 区域指定一个标题 |
| --warningyesno *text* | 带有 Yes/No 按钮的警告信息框 |
| --warningcontinuecancel *text* | 带有 Continue/Cancel 的警告信息框 |
| --warningyesnocancel *text* | 带有 Yes/No/Cancel 的警告信息框 |
| --yesno *text* | 带有 Yes/No 按钮的询问框 |
| --yesnocancel *text* | 带有 Yes/No/Cancel 的询问框 |

正如从表 15-5 中看到的，所有的标准窗口 dialog 类型都有所描述。然而，在使用 kdialog 窗口小部件时，会在 KDE 桌面环境下显示为一个独立的窗口，而不是包含在终端模拟器会话中。

checklist 和 radiolist 小部件允许您定义列表中的单独项目，以及它们是否默认选中：

```
$ kdialog --checklist "Items I need" 1 "Toothbrush" on 2 "Toothpaste"
 off 3 "Hair brush" on 4 "Deodorant" off 5 "Slippers" off
```

最终得到的检查列表窗口如图 15-9 所示。

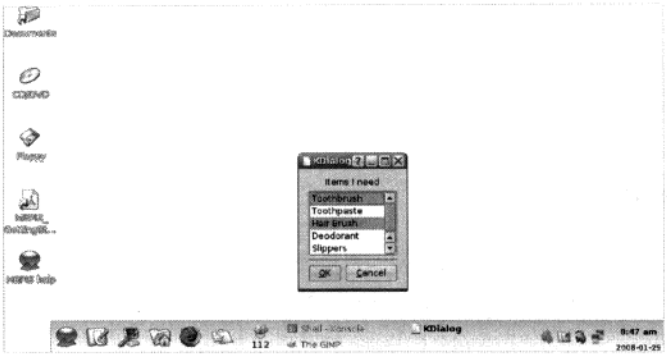



图 15-9　kdialog 检查列表对话框窗口

要选中或者取消选中检查列表中的项目，只需单击它。如果选择 OK 按钮，**kdialog** 将该标签值发送到 STDOUT：

```
$ kdialog --checklist "Items I need" 1 "Toothbrush" on 2 "Toothpaste"
 off 3 "Hair brush" on 4 "Deodorant" off 5 "Slippers" off
"1" "3" "5"
$
```

按 **Enter** 键时，带选项的 **kdialog** 框出现。单击 **OK** 或 **Cancel** 按钮时，kdialog 命令以字符串值的形式返回各个标签到 STDOUT（这些就是在例子中看到的值“1”、“3”、“5”）。脚本必须能够解析这些值，并将它们与原始值进行匹配。

### 2. 使用 kdialog

可以在 **shell** 脚本中使用 kdialog 窗口小部件，与使用对话框小部件的方法类似。最大的区别是，kdialog 窗口小部件使用 STDOUT 而非 STDERR 输出值。

下面的脚本可以将前面创建的系统管理员菜单转换成一个 **KDE** 应用程序：

```
$ cat menu4
#!/bin/bash
# using kdialog to create a menu

temp='mktemp -t temp.XXXXXX'
temp2='mktemp -t temp2.XXXXXX'

function diskspace {
   df -k > $temp
   kdialog --textbox $temp 1000 10
}

function whoseon {
   who > $temp
   kdialog --textbox $temp 500 10
}

function memusage {
   cat /proc/meminfo > $temp
   kdialog --textbox $temp 300 500
}

while [ 1 ]
do
kdialog --menu "Sys Admin Menu" "1" "Display disk space" "2" "Display
users" "3" "Display memory usage" "0" "Exit" > $temp2
if [ $? -eq 1 ]
then
   break
```

```
fi

selection='cat $temp2'

case $selection in
1)
   diskspace ;;
2)
   whoseon ;;
3)
   memusage ;;
0)
   break ;;
*)
   kdialog --msgbox "Sorry, invalid selection"
esac
done
$
```

使用 kdialog 命令和使用 dialog 命令并没有太多的区别。最终得到的菜单如图 15-10 所示。

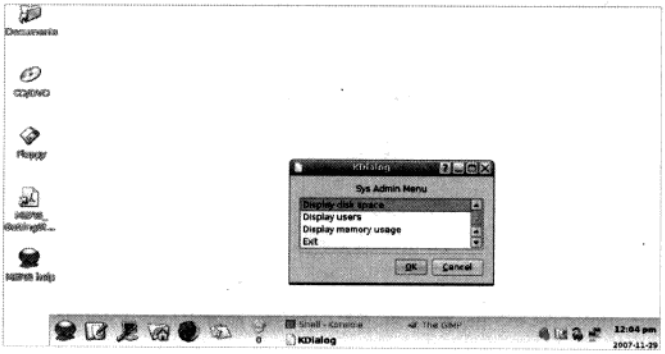



图 15-10 使用 kdialog 的系统管理菜单脚本

现在，脚本已经像一个真正的应用程序了！现在，可以用您自己的交互式脚本去做任何事情，而不受限制。

## 15.4.2 GNOME 环境

GNOME 图形化环境支持两种可以生成标准窗口的流行软件包：

- gdialog；
- zenity。

到目前为止，在大多数 GNOME 桌面 Linux 发布版（如 Ubuntu 和 Fedora）中，zenity 是最常见的软件包。本节介绍 zenity 的功能并说明如何在 shell 脚本中使用它。

### 1. zenity 小部件

正如所预期的，zenity 允许使用命令行选项创建不同的窗口小部件。表 15-6 给出了 zenity 可以创建的不同小部件。

表 15-6 zenity 窗口小部件

| 选项 | 描述 |
| --- | --- |
| --calendar | 显示一完整的月历 |
| --entry | 显示一个文本输入对话框窗口 |
| --error | 显示一个错误信息对话框窗口 |
| --file-selection | 显示完整的路径名和文件名对话框窗口 |

续表

| 选项 | 描述 |
| --- | --- |
| --info | 显示信息对话框窗口 |
| --list | 显示检查列表或单选框列表对话框窗口 |
| --notification | 显示一个通知图标 |
| --progress | 显示一个进度栏对话框窗口 |
| --question | 显示一个 Yes/No 问题对话框窗口 |
| --scale | 显示一个刻度比例对话框窗口 |
| --text-info | 显示一个含有文字的文本框 |
| --warning | 显示一个警告对话框窗口 |

zenity 程序的工作方式与 kdialog 程序和 dialog 程序略有不同。许多类型的小部件要在命令行上使用附加的选项来定义，而不是把它们作为选项的参数。

zenity 程序的确提供了一些相当酷的基本对话框窗口。日历选项可以产生一个完整的月历，如图 15-11 所示。

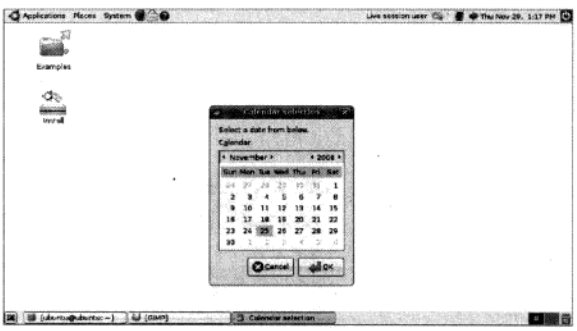


图 15-11　zenity 日历对话框窗口

从日历中选择一个日期，zenity 像 kdialog 一样将该值返回到 STDOUT：

```
$ zenity --calendar
11/25/2008
$
```

zenity 中另一个相当酷的窗口是文件选择选项，如图 15-12 所示。

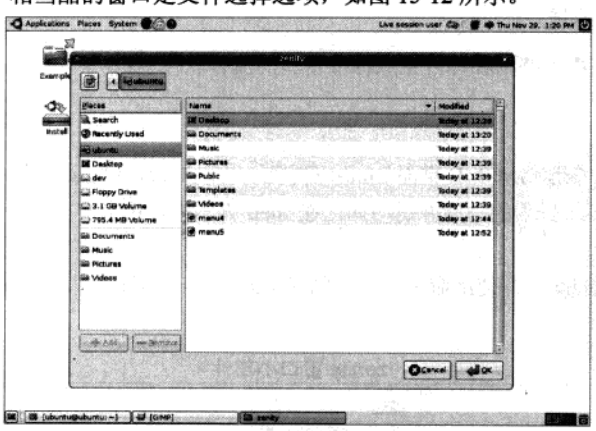


图 15-12　zenity 文件选择对话框窗口

可以使用该对话框窗口浏览系统上的任何目录（只要有权查看该目录）并且选择一个文件。选择文件时，zenity 会返回完整的文件和路径名：

```
$ zenity --file-selection
/home/ubuntu/menu5
$
```

这就是在 shell 脚本世界能够获得的专业外观！

### 2. 在脚本中使用 zenity

正如所预期的，zenity 程序在 shell 脚本中运行得很好。遗憾的是，zenity 不遵守在 dialog 和 kdialog 中使用的选项约定，因此将任何现有的交互式脚本转换为 zenity 都具有挑战性。

在将系统管理菜单从 kdialog 转换为 zenity 时，我发现自己需要对许多小部件定义进行相当多的处理：

```
$cat menu5
#!/bin/bash
# using zenity to create a menu

temp='mktemp -t temp.XXXXXX'
temp2='mktemp -t temp2.XXXXXX'

function diskspace {
   df -k > $temp
   zenity --text-info --title "Disk space" --filename=$temp
--width 750 --height 10
}

function whoseon {
   who > $temp
   zenity --text-info --title "Logged in users" --filename=$temp
--width 500 --height 10
}

function memusage {
   cat /proc/meminfo > $temp
   zenity --text-info --title "Memory usage" --filename=$temp
--width 300 --height 500
}

while [ 1 ]
do
zenity --list --radiolist --title "Sys Admin Menu" --column "Select"
--column "Menu Item" FALSE "Display disk space" FALSE "Display users"
FALSE "Display memory usage" FALSE "Exit" > $temp2
if [ $? -eq 1 ]
then
   break
fi

selection='cat $temp2'
case $selection in
"Display disk space")
   diskspace ;;
"Display users")
   whoseon ;;
"Display memory usage")
   memusage ;;
Exit)
   break ;;
*)
   zenity --info "Sorry, invalid selection"
esac
done
$
```

由于 zenity 不支持菜单对话框窗口，在建立主菜单时我使用了单选框列表类型窗口，如图 15-13 所示。

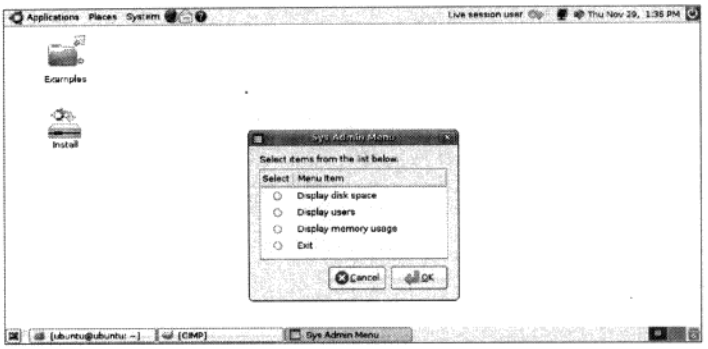



图 15-13　使用 zenity 的系统管理菜单

单选框列表使用两列，每一列都有一个列标题。第一列是可供选择的单选按钮，第二列是项目文本。单选框列表的项目也没有使用标签。在选择一个项目时，该项目的整个文本将被返回到 STDOUT。这使得 case 命令更有趣。必须在 case 选项中使用来自项目的整个文本。如果文本中有空格，就需要是使用引号将该文本引起来。

使用 zenity 软件包，可以在 GNOME 桌面上，为交互式 shell 脚本增加 Windows 感觉。

## 15.5　小结

交互式 shell 脚本曾一度被认为是单调乏味的。使用大多数 Linux 系统上可用的几种不同技术和工具可以改变这一情况。首先，可以使用 case 命令和 shell 脚本函数创建交互式脚本菜单系统。

case 命令允许使用标准的 echo 命令绘制一个菜单，使用 read 命令读取用户的响应。然后，case 命令根据输入的值选择合适的 shell 脚本函数。

使用 ANSI 转义控制码设置菜单文字的颜色，并控制菜单文字的特征，比如使文本闪烁和黑体，可以使文本方式的菜单看起来更有生机。常常是仅仅改变菜单的颜色就会使脚本更赏心悦目。

dialog 程序提供了一些预制的文本框小部件，可以在基于文本的终端模拟器上创建类 Windows 对象。可以使用 dialog 程序创建显示文本、输入文本以及选择文件和日期的对话框。这会使 shell 脚本更生动。

如果您正在图形化的 X Windows 环境中运行 shell 脚本，那么在交互式脚本中甚至可以使用更多的工具。对于 KDE 桌面而言，有 kdialog 程序。该程序提供简单命令，用于为所有基本的窗口函数创建窗口小部件。对于 GNOME 桌面而言，有 gdialog 和 zenity 程序。上述程序均提供与 GNOME 桌面兼容的、类似真正的 Windows 应用程序的窗口小部件。

下一章将深入介绍文本数据文件的编辑和处理。通常情况下，shell 脚本的最大用途涉及解析和显示文本文件（如日志和错误文件）中的数据。Linux 环境包括两个非常有用的工具：sed 和 gawk，用于在 shell 脚本中处理文本数据。下一章将介绍这些工具，并讨论它们的基本使用方法。

# 第 16 章
# sed 和 gawk 介绍

CHAPTER 16

本章内容：

- 使用文本文件；
- 探索 sed；
- 探索 gawk。

到目前为止，shell 脚本的最常用功能之一是处理文本文件。在检查日志文件、读取配置文件以及处理数据元素之间，shell 脚本可以把处理文本文件中包含的所有类型的数据这样的普通任务自动化。然而，只使用 shell 脚本命令处理文本文件的内容却有些麻烦。如果要在 shell 脚本中进行任何类型的数据处理，就要熟悉 Linux 中现有的 sed 和 gawk 工具。这些工具可以极大地简化需要完成的任何数据处理任务。

## 16.1 文本处理

第 7 章介绍了在 Linux 环境中如何使用现有的不同编辑器程序编辑文本文件。这些编辑器允许使用简单的命令或鼠标单击方便地处理文本文件中包含的文本。

有时，您会发现自己需要动态地处理文本文件中的文本，而不必使用完备的交互式文本编辑器。在这些情况下，拥有一个能够方便地格式化、插入、修改或删除文本元素的简单命令行编辑器就很有用。

Linux 系统有两个常用工具可以完成这一任务。本节将讨论 Linux 环境下最常用的两个命令行编辑器：sed 和 gawk。

### 16.1.1 sed 编辑器

sed 编辑器称为**流编辑器**（stream editor），与普通的交互式文本编辑器相对应。在交互式文本编辑器（如 vim）中，可以交互地使用键盘命令插入、删除、或者替换数据中的文本。流编辑器将根据在编辑器处理数据之前事先提供的规则集编辑数据流。

sed 编辑器可以根据输入命令行的命令或者存储在命令文本文件中的命令处理数据。它每次从输入读取一行数据，将该数据与所提供的编辑器命令进行匹配，根据命令修改数据流中的数据，然后将新数据输出到 STDOUT。在流编辑器将全部命令和一行数据匹配完之后，它读取下一行数据，并重复上述过程。处理完数据流中的全部数据行之后，流编辑器停止。

由于是按顺序逐行应用命令，所以 sed 编辑器进行编辑仅需浏览数据流一次。这使得 sed 编辑器比交互式编辑器要快很多，因而可以动态地快速修改文件中的数据。

使用 sed 命令的格式是：

```
sed options script file
```

options 参数允许自定义 sed 命令的行为，它包含的选项如表 16-1 所示。

表 16-1 sed 命令选项

| 选项 | 描述 |
| --- | --- |
| -e *script* | 将脚本中指定的命令添加到处理输入时执行的命令中 |
| -f *file* | 将文件中指定的命令添加到处理输入时执行的命令中 |
| -n | 不需要为每个命令产生输出，但要等待打印命令 |

script 参数指定要应用于流数据的单个命令。如果需要多个命令，就必须使用-e 选项在命令行中指定，或者使用-f 选项在单独的文件中指定。处理数据可用的命令有很多。本章后面将介绍 sed 编辑器使用的一些基本命令，然后在第 18 章中介绍一些更高级的命令。

### 1. 在命令行中定义编辑器命令

默认情况下，sed 编辑器将指定的命令应用于 STDIN 输入流。这就允许将数据直接管道传送给 sed 编辑器处理。下面是一个例子，说明如何实现这一功能：

```
$ echo "This is a test" | sed 's/test/big test/'
This is a big test
$
```

该例在 sed 编辑器中使用了 s 命令。s 命令用第二个文本字符串替换第一个两个斜杠之间指定的文本字符串模式。在本例中，我用单词 big test 替换了单词 test。

运行该例时，它应该立刻显示结果。这就是使用 sed 编辑器的威力，仅仅使用大约与某些交互式编辑器启动相同的时间，就可以对数据进行多处修改。

当然，这个简单的例子只编辑了一行数据。在编辑下面的完整数据文件时，应该能够得到相同的速度：

```
$ cat data1
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.
$ sed 's/dog/cat/' data1
The quick brown fox jumps over the lazy cat.
The quick brown fox jumps over the lazy cat.
The quick brown fox jumps over the lazy cat.
The quick brown fox jumps over the lazy cat.
$
```

sed 命令执行与返回数据几乎同时进行。它在处理每一行数据的同时就显示结果。在 sed 编辑器结束处理整个文件之前，就可以看到结果。

注意，sed 编辑器并不修改文本文件中的数据。它只是将修改后的文本发送到 STDOUT。如果您查看文本文件，它包含的仍然是原数据：

```
$ cat data1
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.
$
```

### 2. 在命令行中使用多个编辑器命令

要从 sed 命令行执行多个命令，只需要使用-e 选项：

```
$ sed -e 's/brown/green/; s/dog/cat/' data1
```

```
The quick green fox jumps over the lazy cat.
The quick green fox jumps over the lazy cat.
The quick green fox jumps over the lazy cat.
The quick green fox jumps over the lazy cat.
$
```

两个命令将同时应用于文件中的每一行数据。命令必须用分号隔开，且在命令结尾和分号之间不能有任何空格。

在 bash shell 中，可以使用次提示符，而不是分号分隔命令。只需输入前单引号打开脚本，bash 将继续提示您输入更多的命令，直到输入后单引号：

```
$ sed -e '
> s/brown/green/
> s/fox/elephant/
> s/dog/cat/' data1
The quick green elephant jumps over the lazy cat.
The quick green elephant jumps over the lazy cat.
The quick green elephant jumps over the lazy cat.
The quick green elephant jumps over the lazy cat.
$
```

必须记住：要在后单引号出现的同一行上完成该命令，因为 bash 检测到后引号之后，就处理命令。一旦开始，sed 命令就会将指定的每一个命令应用于文本文件中的每一行数据。

**3. 从文件读取编辑器命令**

最后，如果有太多的 sed 命令要处理，那么将它们保存在一个独立的文件中，然后在 sed 命令中使用-f 选项指定文件通常更方便：

```
$ cat script1
s/brown/green/
s/fox/elephant/
s/dog/cat/
$ sed -f script1 data1
The quick green elephant jumps over the lazy cat.
The quick green elephant jumps over the lazy cat.
The quick green elephant jumps over the lazy cat.
The quick green elephant jumps over the lazy cat.
$
```

在这种情况下，不需要在每个命令后放一个分号。sed 编辑器知道一行包含单独一个命令。同在命令行上输入命令完全一样，sed 编辑器从指定的文件中读取命令并将它们应用于数据文件中的每一行。

我们将在“sed 编辑器基础知识”一节讨论其他能够方便地处理数据的 sed 编辑器命令。在此之前，让我们首先快速了解一下另一个 Linux 数据编辑器。

## 16.1.2 gawk 程序

尽管 sed 编辑器是动态修改文本文件的便利工具，但它也有自己的局限性。通常，您需要更高级的工具来处理文件中的数据，一个能够提供类似于编程环境的工具，它允许修改和重新组织文件中的数据。这正是 gawk 的特长。

gawk 程序是 Unix 中原 awk 程序的 GNU 版本。awk 程序在流编辑方面比 sed 编辑器更先进的是：它提供了一种编程语言而不仅仅是编辑器命令行。在编程语言内部，可以：

- 定义要保存数据的变量。
- 使用算术和字符串操作符对数据进行运算。
- 使用结构化编程概念，例如 if-then 语句和循环，将逻辑添加到数据处理过程。
- 通过从数据文件内抽取数据元素以及按照其他顺序或格式对它们重定位，生成带格式的报告。

gawk 程序的报告生成能力常用于从大文本文件中抽取数据元素，并将其格式化为易读的报告。这方面的一个完美示例是格式化日志文件。试图浏览日志文件中的错误行可能很难。gawk 程序允许您从

日志文件中仅筛选要查看的数据元素，然后用一种更便于阅读主要数据的方式格式化数据。

### 1. gawk 命令格式

gawk 程序的基本格式是：

```
gawk options program file
```

表 16-2 显示了 gawk 程序可用的选项。

表 16-2　　gawk 选项

| 选项 | 描述 |
| --- | --- |
| -F *fs* | 指定描绘一行中数据字段的文件分隔符 |
| -f *file* | 指定读取程序的文件名 |
| -v *var=value* | 定义 gawk 程序中使用的变量和默认值 |
| -mf *N* | 指定数据文件中要处理的字段的最大数目 |
| -mr *N* | 指定数据文件中的最大记录大小 |
| -W *keyword* | 指定 gawk 的兼容模式或警告级别 |

命令行选项提供了在 gawk 程序中自定义功能的简单方法。在介绍使用 gawk 时，我们将详细研究这些选项。

gawk 的威力在程序脚本中。可以编写脚本读取一个文本行中的数据，然后处理并显示这些数据，以创建任意类型的输出报告。

### 2. 自命令行读取程序脚本

gawk 程序脚本由左大括号和右大括号定义。脚本命令必须放置在两个大括号之间。由于 gawk 命令行假定脚本是单文本字符串，所以必须将脚本包括在单引号内。下面是在命令行上指定的一个简单的 gawk 程序脚本：

```
$ gawk '{print "Hello John!"}'
```

该程序脚本定义了一个命令，print 命令。print 命令执行打印功能，将文本输出到 STDOUT。如果试着执行本命令，您可能会有点失望，因为不会立刻发生任何事。因为在命令行中没有定义文件名，所以 gawk 程序要从 STDIN 获取数据。在运行本程序时，它只能等待通过 STDIN 输入的文本。

如果您输入一行文本，按 Enter 键，gawk 将通过该程序脚本处理文本：

```
$ gawk '{print "Hello World!"}'
This is a test
Hello John!
hello
Hello John!
This is another test
Hello John!

$
```

同 sed 编辑器完全一样，gawk 程序对数据流中可用的每一行文本执行程序脚本。由于本程序脚本设定为显示固定的文本字符串，因此无论在数据流中输入什么，都将得到相同的输出。

要结束 gawk 程序，必须信号说明数据流已经结束。bash shell 提供了生成 End-of-File（EOF）字符的组合键。在 bash 中，Ctrl+D 组合键生成 EOF 字符。使用这一组合键结束 gawk 程序，返回命令行界面提示符。

### 3. 使用数据字段变量

如前所述，gawk 的主要功能之一是其处理文本文件中数据的能力。它通过自动将变量分配给每行中的每个数据元素实现这一功能。默认情况下，gawk 将下面的变量分配给在文本行中检测到的每个数据字段：

- $0 表示整行文本；

- $1 表示文本行中的第一个数据字段；
- $2 表示文本行中的第二个数据字段；
- $n 表示文本行中的第 n 个数据字段。

各数据字段依据文本行中的**字段分隔符**（field separation character）确定。gawk 读取一行文本时，使用定义的字段分隔符描述各数据字段。gawk 的默认字段分隔符是任意空白字符（如制表符或空格符）。

为说明这一点，下面给出了一个 gawk 程序读取文本文件且仅显示第一个数据字段的例子：

```
$ cat data3
One line of test text.
Two lines of test text.
Three lines of test text.
$ gawk '{print $1}' data3
One
Two
Three
$
```

该程序使用$1 字段变量仅显示每个文本行的第一个数据字段。

如果您正在读取使用不同字段分隔符的文件，可以使用-F 选项指定：

```
$ gawk -F: '{print $1}' /etc/passwd
root
bin
daemon
adm
lp
sync
shutdown
halt
...
```

这个小程序能显示系统上密码文件中的第一个数据字段。因为/etc/passwd 文件使用冒号分隔数据字段，因此如果您要分隔每个数据元素，则必须在 gawk 选项中将冒号指定为字段分隔符。

#### 4. 在程序脚本中使用多个命令

如果只能执行一个命令，这样的编程语言不是特别有用。gawk 编程语言允许您将命令组合成普通程序。要在命令行指定的程序脚本中使用多个命令，只需在各命令之间加一个分号：

```
$ echo "My name is Rich" | gawk '{$4="Dave"; print $0}'
My name is Dave
$
```

第一个命令为$4 字段变量指定一个值。然后，第二个命令打印整个数据字段。注意，gawk 程序的输出已经用新值替换了原文本中的第四个数据字段。

也可以使用次提示符每次输入一行程序脚本命令：

```
$ gawk '{
> $4="testing"
> print $0 }'
This is not a good test.
This is not testing good test.
$
```

在开放单引号之后，bash shell 提供次提示符提示您输入更多数据。可以每次在每行添加一个命令，直到输入后单引号。要退出该程序，只需按 Ctrl+D 组合键发送数据结束信号即可。

#### 5. 从文件读取程序

同 sed 编辑器一样，gawk 编辑器允许您将程序保存在文件中并在命令行引用它们：

```
$ cat script2
{ print $5 "'s userid is " $1 }
$ gawk -F: -f script2 /etc/passwd
```

```
root's userid is root
bin's userid is bin
PostgreSQL Server's userid is postgres
FTP User's userid is ftp
GDM User's userid is gdm
HTDIG User's userid is htdig
Dhcpd User's userid is dhcpd
Bind User's userid is named
NSCD Daemon's userid is nscd
X Font Server's userid is xfs
MySQL server's userid is mysql
Rich's userid is rich
test account's userid is testing
postfix's userid is postfix
$
```

script2程序脚本再次使用print命令打印/etc/passwd文件的注释数据字段(字段变量$5)、一条简短的文本消息，然后是userid数据字段（字段变量$1)。

可以在程序文件中指定多个命令。要达到这一目的，只需在每行单独放置一个命令。不需要使用分号：

```
$ cat script3
{
text="'s userid is "
print $5 text $1
}
$ awk -F: -f script3 /etc/passwd | more
root's userid is root
bin's userid is bin
PostgreSQL Server's userid is postgres
FTP User's userid is ftp
GDM User's userid is gdm
HTDIG User's userid is htdig
Dhcpd User's userid is dhcpd
Bind User's userid is named
NSCD Daemon's userid is nscd
X Font Server's userid is xfs
MySQL server's userid is mysql
Rich's userid is rich
test account's userid is testing
postfix's userid is postfix
$
```

script3程序脚本定义了一个变量存储print命令中使用的文本字符串。注意，引用某个变量的值时，gawk程序不像shell脚本那样使用美元符号。

**6. 在处理数据之前运行脚本**

gawk程序还允许指定运行脚本的时间。默认情况下，gawk从输入读取一行文本，然后执行程序脚本处理文本行中的数据。但有时，可能需要在处理数据之前运行脚本，例如创建报告的标题部分。为达到这一目的，要使用BEGIN关键字。这样可以强制gawk在读取数据之前，执行BEGIN关键字之后指定的程序脚本：

```
$ gawk 'BEGIN {print "Hello World!"}'
Hello World!
$
```

这次，print命令将在读取数据之前显示文本。不过，在显示该文本之后，它将迅速退出，不等待数据输入。

原因是BEGIN关键字仅在处理任何数据之前应用指定的脚本。如果要用正常程序脚本处理数据，则必须使用另一脚本片段定义程序：

```
$ gawk 'BEGIN {print "Hello World!"} {print $0}'
```

```
Hello World!
This is a test
This is a test
This is another test
This is another test
This is the last test
This is the last test

$
```

现在，在 gawk 执行 BEGIN 脚本之后，它会使用第二段脚本处理出现的任何数据。要退出该程序，只需按 Ctrl+D 组合键结束数据。

这样做时要谨慎。注意，这两段脚本仍然被认为是 gawk 命令行上的一个文本字符串。因此需要适当地放置单引号。

**7. 在处理数据之后运行脚本**

与 BEGIN 关键字类似，END 关键字允许指定在读取数据之后 gawk 执行的程序脚本：

```
$ gawk 'BEGIN {print "Hello World!"} {print $0} END {print
    "byebye"}'
Hello World!
This is a test
This is a test
This is another test.
This is another test.
byebye
$
```

这次，在按 Ctrl+D 组合键结束数据之后，gawk 程序执行 END 脚本中的命令。这是在处理完所有正常的数据之后，在报告中添加页脚数据的一项优秀技术。

可以将所有这些元素一起集成到一个小小的程序脚本文件中，以根据一个简单的数据文件创建一份完整的报告：

```
$ cat script4
BEGIN {
print "The latest list of users and shells"
print " Userid        Shell"
print "--------      -------"
FS=":"
}

{
print $1 "       " $7
}

END {
print "This concludes the listing"
}
$
```

本脚本使用 BEGIN 脚本创建报告的标题部分。它还定义了一个特殊的变量 FS。这也是定义字段分隔符的另一种方法。这样，您就不必依赖使用该脚本的人在命令行选项中定义字段分隔符。

下面是运行该 gawk 脚本后，略加删减的输出：

```
$ gawk -f script4 /etc/passwd
The latest list of users and shells
 Userid        Shell
--------      -------
root      /bin/bash
sync      /bin/sync
shutdown      /sbin/shutdown
halt      /sbin/halt
mysql     /bin/bash
```

```
rich      /bin/bash
test2     /bin/csh
test      /bin/bash
This concludes the listing
$
```

正如预期的一样，BEGIN脚本创建了标题文本，程序脚本处理来自指定数据文件（/etc/passwd文件）的信息，END脚本生成页脚文本。

这让您品尝了一下使用简单的gawk脚本的滋味。第19章将介绍gawk脚本可用的一些更基本的编程原理，以及一些可以在gwak脚本中使用的更先进的程序设计理念，以从含义最隐晦的数据文件创建有专业外观的报告。

## 16.2 sed编辑器基础知识

成功使用sed编辑器的关键是要了解其大量的命令和格式，这些命令和格式有助于自定义文本编辑。本节将介绍一些基本的命令和功能，您可以将它们集成在自己的脚本中，由此开始使用sed编辑器。

### 16.2.1 更多替换选项

您已经看到如何使用s命令用新文本替换某一行中的文本。然而，还有其他几个选项可用于替换命令，从而使您的工作更轻松。

**1. 替换标记**

对于替换命令替换文本字符串中匹配模式的方法，有一个警告。请看下面的例子：

```
$ cat data4
This is a test of the test script.
This is the second test of the test script.
$ sed 's/test/trial/' data4
This is a trial of the test script.
This is the second trial of the test script.
$
```

替换命令在替换多个文本行中的文本时效果不错，但默认情况下仅替换各行中首次出现的文本。要使替换命令继续替换之后出现的文本，则必须使用**替换标记**（substitution flag）。替换标记要放在替换命令字符串之后：

```
s/pattern/replacement/flags
```

可用的替换标记有4种：

- 数字：表示新文本替换的模式。
- g：表示用新文本替换现有文本的全部实例。
- p：表示打印原始行的内容。
- w file：将替换的结果写入文件。

在第一种类型的替换中，可以指定sed编辑器用新文本替换匹配模式的哪一个实例：

```
$ sed 's/test/trial/2' data4
This is a test of the trial script.
This is the second test of the trial script.
$
```

指定2作为替换标记的结果是，sed编辑器仅替换每一行中第二次出现的模式。g替换标记能够替换文本中模式的每一个实例：

```
$ sed 's/test/trial/g' data4
This is a trial of the trial script.
This is the second trial of the trial script.
$
```

p 替换标记会打印包含替换命令中匹配模式的那一行。经常和-n sed 选项一起使用：

```
$ cat data5
This is a test line.
This is a different line.
$ sed -n 's/test/trial/p' data5
This is a trial line.
$
```

-n 选项禁止 sed 编辑器的输出。然而，p 替换标记会输出所有已修改的行。二者结合使用仅生成替换命令已更改的那些行的输出。

w 替换标记生成相同的输出，但会将输出保存到指定的文件中：

```
$ sed 's/test/trial/w test' data5
This is a trial line.
This is a different line.
$ cat test
This is a trial line.
$
```

sed 编辑器的正常输出出现在 STDOUT 中，只有包含匹配模式的那些行保存在指定输出文件中。

**2. 替换字符**

有时可能会在文本字符串中遇到不容易在替换模式中使用的字符。在 Linux 环境下，最常见的一个例子是正斜杠。

替换某个文件中的路径名可能很难。例如，如果要用 Cshell 替换/etc/passwd 文件中 bash shell，必须这样做：

```
$ sed 's/\/bin\/bash/\/bin\/csh/' /etc/passwd
```

因为正斜杠要用作字符串定界符，所以如果正斜杠出现在模式文本中，必须使用反斜杠使其转义。这常常会导致混淆和错误。

要解决这一问题，sed 编辑器允许为替换命令中的字符串定界符选择一个不同的字符：

```
$ sed 's!/bin/bash!/bin/csh!' /etc/passwd
```

在本例中，感叹号被用作字符串定界符，从而使阅读和理解路径名更容易。

## 16.2.2 使用地址

默认情况下，在 sed 编辑器中使用的命令应用于所有文本数据行。如果仅想将某个命令应用于某一特定的文本数据行或一组文本数据行，则必须使用**行寻址**（line addressing）。

在 sed 编辑器中，有两种行寻址形式：

- 行的数值范围；
- 筛选行的文本模式。

两种形式使用相同的格式指定地址：

```
[address]command
```

也可以将多个命令组合在一起，应用于一个特定的地址：

```
address {
    command1
    command2
    command3
}
```

sed 编辑器将指定的每一个命令仅应用于与指定地址匹配的行。

本节将介绍在 sed 编辑器脚本中使用这些寻址技术的方法。

**1. 数字式行寻址**

使用数字式行寻址时，可以使用行在文本流中的位置引用行。sed 编辑器指定文本流中的首行行号为 1，每一个新行的行号顺延。

在命令中指定的地址可以是单个行号，也可以是由起始行号、逗号和结束行号指定的行范围。下

面是一个例子，它指定了 sed 命令将应用的那一行的行号：

```
$ sed '2s/dog/cat/' data1
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy cat
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog
$
```

sed 编辑器仅修改指定地址第二行中的文本。下面是另一个例子，这次使用行地址范围：

```
$ sed '2,3s/dog/cat/' data1
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy cat
The quick brown fox jumps over the lazy cat
The quick brown fox jumps over the lazy dog
$
```

如果要将命令应用于文本内从某一点开始直到文本结束的一组文本行，可以使用特殊地址——美元符号：

```
$ sed '2,$s/dog/cat/' data1
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy cat
The quick brown fox jumps over the lazy cat
The quick brown fox jumps over the lazy cat
$
```

因为您可能不知道文本中到底有多少行数据，所以使用美元符号通常很方便。

**2. 使用文本模式筛选器**

限制某个命令应用于哪些行的另一种方法稍微有点复杂。sed 编辑器允许指定为该命令筛选文本行所使用的文本模式。

命令格式是：

```
/pattern/command
```

必须用正斜杠标记指定的 pattern。sed 编辑器仅将该命令应用于包含指定文本模式的行。

例如，如果只想为用户 rich 更改默认的 shell，可以使用 sed 命令：

```
$ sed '/rich/s/bash/csh/' /etc/passwd
rich:x:500:500:Rich Blum:/home/rich:/bin/csh
barbara:x:501:501:Barbara:/home/barbara:/bin/bash
katie:x:502:502:Katie:/home/katie:/bin/bash
jessica:x:503:503:Jessica:/home/jessica:/bin/bash
test:x:504:504:Ima test:/home/test:/bin/bash
$
```

sed 编辑器仅将这个命令应用于与文本模式匹配的那一行。与在 userid 例子中一样，虽然使用固定的文本模式可能对于筛选特定值很有用，但所完成的功能有点受限。sed 编辑器允许使用**正则表达式**（regular expressions）功能创建复杂的文本模式。

正则表达式允许创建高级文本模式匹配公式来匹配各种类型的数据。这些公式可以组合一系列通配符、特殊字符以及固定的文本符号，形成能够匹配几乎所有文本情况的简明模式。正则表达式是 shell 脚本编程非常复杂的部分之一，将在第 17 章详细介绍。

**3. 组合命令**

如果需要在单独一行上执行多个命令，请使用大括号将命令组合在一起。sed 编辑器将处理地址行上列出的所有命令：

```
$ sed '2{
> s/fox/elephant/
> s/dog/cat/
> }' data1
The quick brown fox jumps over the lazy dog.
The quick brown elephant jumps over the lazy cat.
The quick brown fox jumps over the lazy dog.
```

```
The quick brown fox jumps over the lazy dog.
$
```

两个命令都对该地址进行了处理。当然，也可以在命令组合之前指定地址范围：

```
$ sed '3,${
> s/brown/green/
> s/lazy/active/
> }' data1
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.
The quick green fox jumps over the active dog.
The quick green fox jumps over the active dog.
$
```

sed 编辑器将全部命令应用于地址范围内的所有文本行。

### 16.2.3 删除行

文本替换命令不是 sed 编辑器可用的唯一命令。如果需要删除文本流中特定的文本行，还有一个删除命令。

删除命令 d 完成的功能即为删除。它将删除与所提供的寻址模式一致的所有文本行。使用删除命令要小心，因为如果忘记包含寻址模式，所有文本行都将会从文本流中删除：

```
$ cat data1
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog
$ sed 'd' data1
$
```

显然，删除命令与指定的地址一起使用最有用。这样可以从数据流中删除特定的文本行，可以按行号：

```
$ sed '3d' data6
This is line number 1.
This is line number 2.
This is line number 4.
$
```

或者特定的文本行范围：

```
$ sed '2,3d' data6
This is line number 1.
This is line number 4.
$
```

或者使用特殊的文件结束符：

```
$ sed '3,$d' data6
This is line number 1.
This is line number 2.
$
```

sed 编辑器的模式匹配功能也适用于删除命令：

```
$ sed '/number 1/d' data6
This is line number 2.
This is line number 3.
This is line number 4.
$
```

sed 编辑器会删除包含与指定模式匹配的文本的行。

注意：记住，sed 编辑器不会处理原始文件。所删除的所有文本行仅从 sed 编辑器输出中删除。原始文件仍然包含这些“已删除的”行。

还可以使用两个文本模式删除若干行，但这样做时要小心。指定的第一个模式将“打开”行删除，而第二个模式将“关闭”行删除。sed 编辑器将删除指定的这两行（包括指定的行）之间的所有文本行：

```
$ sed '/1/,/3/d' data6
This is line number 4.
$
```

我曾提到，这样做时必须小心，因为只要 sed 编辑器在数据流中检测到开始模式，删除功能就会随时打开。这可能会产生意想不到的结果：

```
$ cat data7
This is line number 1.
This is line number 2.
This is line number 3.
This is line number 4.
This is line number 1 again.
This is text you want to keep.
This is the last line in the file.
$ sed '/1/,/3/d' data7
This is line number 4.
$
```

包含数字 1 的行第二次出现再次触发了删除命令，删除了数据流中余下的文本行，因为它没有识别出停止模式。当然，如果指定文本中没有出现的模式，则会发生另一个显而易见的问题：

```
$ sed '/1/,/5/d' data7
$
```

因为删除功能在第 1 个模式匹配时已经打开，但没有发现匹配的结束模式，因此删除了整个数据流。

### 16.2.4 插入和附加文本

正如所期望的，同任何其他的编辑器一样，sed 编辑器允许向数据流插入和附加文本。两个操作之间的差别很容易混淆：

- 插入命令（i）在指定行之前添加新的一行；
- 附加命令（a）在指定行之后添加新的一行。

关于这两个命令，容易混淆的是它们的格式。不能在单命令行上使用这两个命令。必须单独指定要插入或附加的行。实现这一功能的格式为：

```
sed '[address]command\
new line'
```

new line 中的文本出现在 sed 编辑器输出中的指定位置。记住，使用插入命令时，文本出现在该数据流文本之前：

```
$ echo "testing" | sed 'i\
> This is a test'
This is a test
testing
$
```

然而，在使用附加命令时，在数据流文本之后出现文本：

```
$ echo "testing" | sed 'a\
> This is a test'
testing
This is a test
$
```

在命令行界面提示符下使用 sed 编辑器时，将得到次提示符以输入新的行数据。必须在这一行上完成 sed 编辑器命令，因为一旦您输入结束单引号，bash shell 就会处理该命令。

这一点非常适合在数据流文本前或后添加文本，但在数据流内部添加文本又该怎么样呢？

为在数据流行内部插入或附加数据，必须使用寻址告诉 sed 编辑器想要数据出现的地方。使用这些命令时只可以指定一行地址。可以与某一数字行号或文本模式匹配，但不能使用地址范围（这是有道理的，因为只能在某一行而不是某一范围前或后插入或附加）。

下面是一个例子，在数据流的第三行之前插入新的一行：

```
$ sed '3i\
> This is an inserted line.' data6
This is line number 1.
This is line number 2.
```

```
This is an inserted line.
This is line number 3.
This is line number 4.
$
```

下面是一个例子，在数据流的第三行之后插入新的一行：

```
$ sed '3a\
>This is an inserted line.' data6
This is line number 1.
This is line number 2.
This is line number 3.
This is an inserted line.
This is line number 4.
$
```

它与插入命令使用相同的过程；仅仅是将新的一行放在指定行号之后。如果数据流有多行，而您要在数据流末尾新附加一行文本，只需使用美元符号，它表示数据的最后一行：

```
$ sed '$a\
> This is a new line of text.' data6
This is line number 1.
This is line number 2.
This is line number 3.
This is line number 4.
This is a new line of text.
$
```

要在数据流最前面添加新的一行，可以利用相同的思路。在第一行之前插入新的一行即可。

要插入或附加多行文本，必须在每一新文本行之前使用一个反斜杠，直到到达要插入或附加文本的最后一行：

```
$ sed '1i\
> This is one line of new text.\
> This is another line of new text.' data6
This is one line of new text.
This is another line of new text.
This is line number 1.
This is line number 2.
This is line number 3.
This is line number 4.
$
```

指定的两行都添加到数据流中。

### 16.2.5 更改行

更改行命令允许更改数据流中整行文本的内容。其工作方式与插入和附加命令相同，因此您必须独立于 sed 命令的其他部分指定新行：

```
$ sed '3c\
> This is a changed line of text.' data6
This is line number 1.
This is line number 2.
This is a changed line of text.
This is line number 4.
$
```

在本例中，sed 编辑器更改了第三行中的文本。指定该地址也可以使用文本模式：

```
$ sed '/number 3/c\
> This is a changed line of text.' data6
This is line number 1.
This is line number 2.
This is a changed line of text.
This is line number 4.
$
```

文本模式更改命令将更改与文本模式匹配的所有文本行。

```
$ sed '/number 1/c\
> This is a changed line of text.' data7
```

```
This is a changed line of text.
This is line number 2.
This is line number 3.
This is line number 4.
This is a changed line of text.
This is yet another line.
This is the last line in the file.
$
```

在更改命令中，可以使用地址范围，但其结果可能不是预期结果：

```
$ sed '2,3c\
> This is a new line of text.' data6
This is line number 1.
This is a new line of text.
This is line number 4.
$
```

sed 编辑器没有用该文本更改两行，而是使用单一文本行替换了两行文本。

### 16.2.6　变换命令

变换命令（y）是唯一对单个字符进行操作的 sed 编辑器命令。变换命令使用格式：

```
[address]y/inchars/outchars/
```

变换命令将 inchars 和 outchars 的值进行一对一映射。将 inchars 中的第一个字符转换为 outchars 中的第一个字符。将 inchars 中的第二个字符转换为 outchars 中的第二个字符。这样的映射一直持续直到完成指定的字符长度。如果 inchars 和 outchars 的长度不同，sed 编辑器将生成错误消息。

使用变换命令的一个简单示例如下：

```
$ sed 'y/123/789/' data7
This is line number 7.
This is line number 8.
This is line number 9.
This is line number 4.
This is line number 7 again.
This is yet another line.
This is the last line in the file.
$
```

输出显示，在 inchars 模式中指定的每一个字符实例已经被 outchars 模式中同一位置上的字符所代替。

变换命令是全局命令，也就是说，它自动对在文本行中发现的任意字符进行变换，而不管其是第几次出现。

```
$ echo "This 1 is a test of 1 try." | sed 'y/123/456/'
This 4 is a test of 4 try.
$
```

sed 编辑器对文本行中匹配字符 1 的两个实例都进行了变换。不能限定对字符特定的出现次数进行变换。

### 16.2.7　打印命令温习

“更多替换选项”一节介绍了替换命令如何使用 p 标记显示 sed 编辑器更改的行。还有 3 个命令也可以用于打印来自数据流的信息：

- 打印文本行的小写 p 命令；
- 打印行号的等号（=）命令；
- 列出行的 l（小写 L）命令。

下面几节介绍 sed 编辑器中的这 3 个打印命令。

### 1. 打印行

与替换命令中的 p 标记类似，p 命令打印 sed 编辑器输出中的一行。单独使用 P 命令没有什么效果：

```
$ echo "this is a test" | sed 'p'
this is a test
this is a test
$
```

它所做的只是打印您已知的文本。打印命令最常见的用途是打印包含与文本模式匹配的文本行：

```
$ sed -n '/number 3/p' data6
This is line number 3.
$
```

在命令行上使用-n 选项可以禁止所有其他的行，而仅打印包含匹配文本模式的行。

也可以使用它作为打印数据流中部分行的快速方法：

```
$ sed -n '2,3p' data6
This is line number 2.
This is line number 3.
$
```

打印命令的另一个用途是在改变某行之前查看该行，如利用替换或更改命令。可以创建脚本在一行更改之前显示该行：

```
$ sed -n '/3/{
p
s/line/test/p
}' data6
This is line number 3.
This is test number 3.
$
```

这一 sed 编辑器命令搜索包含数字 3 的行，然后执行两个命令。首先，脚本使用 p 命令打印该行的原始版本，然后使用 s 命令替换文本，并使用 p 标记打印最终得到的文本。输出显示了原来的行文本和新的行文本。

### 2. 打印行号

等号命令打印数据流内当前行的行号。行号使用数据流中的换行符确定。每次换行符出现在数据流中时，sed 编辑器认为它结束一行文本。

等号命令的基本用途不足为奇：

```
$ sed '=' data1
1
The quick brown fox jumps over the lazy dog.
2
The quick brown fox jumps over the lazy dog.
3
The quick brown fox jumps over the lazy dog.
4
The quick brown fox jumps over the lazy dog.
$
```

sed 编辑器在实际文本行之前打印行号。如果要在数据流中搜索特定的文本模式，等号命令就能够派上用场：

```
$ sed -n '/number 4/{
=
p
}' data6
4
This is line number 4.
$
```

使用-n 选项可以使 sed 编辑器仅显示包含匹配文本模式的那一行的行号和文本。

### 3. 列出行

列表命令（l）允许打印数据流中的文本和不可打印的 ASCII 字符。所有不可打印的字符或者使用反

斜杠加上八进制值表示，或者使用常用的不可打印字符的标准C样式术语表示，比如用\t表示制表符：

```
$ cat data8
This    line    contains          tabs.
$ sed -n 'l' data8
This\tline\tcontains\ttabs.$
$
```

制表符的位置用\t表示。行尾的美元符号表示换行符。如果数据流中包含转义字符，列表命令将使用八进制编码显示它：

```
$ cat data9
This line contains an escape character
$ sed -n 'l' data9
This line contains an escape character \033[44m$
$
```

data9文本文件包含一个改变显示颜色的转义控制符（参阅第15章）。使用cat命令显示文本文件时，看不到转义控制符；它只是改变显示的颜色。

然而，使用列表命令可以显示所使用的转义控制代码。\033是Esc键的ASCII码的八进制值。

### 16.2.8 将文件用于sed

替换命令包含允许您处理文件的标记。也有正规的sed编辑器命令允许您处理文件而不必替换文本。

#### 1. 写文件

w命令用于将文本行写入文件。w命令的格式为：

```
[address]w filename
```

*filename*可以指定为相对或绝对路径，但在任意一种情况下，运行sed编辑器的用户必须对该文件有写权限。地址可以是sed中使用的任意类型的寻址方法，例如单一行号、文本模式或者行号或文本模式范围。

下面是一个例子，仅将数据流的前两行写入文本文件：

```
$ sed '1,2w test' data6
This is line number 1.
This is line number 2.
This is line number 3.
This is line number 4.
$ cat test
This is line number 1.
This is line number 2.
$
```

当然，如果不想在STDOUT上显示这些行，可以将-n选项应用于该sed命令。

如果需要基于常见文本值从主文件创建一个数据文件，这是一个很好的工具。例如在邮件列表中使用此工具：

```
$ cat data10
Blum, Katie          Chicago, IL
Mullen, Riley      West Lafayette, IN
Snell, Haley       Ft. Wayne, IN
Woenker, Matthew        Springfield, IL
Wisecarver, Emma        Grant Park, IL
$ sed -n '/IN/w INcustomers' data10
$ cat INcustomers
Mullen, Riley      West Lafayette, IN
Snell, Haley       Ft. Wayne, IN
$
```

sed编辑器仅将包含文本模式的数据行写入目标文件中。

#### 2. 从文件读取数据

您已经知道从sed命令行向数据流插入数据和附加文本的方法。读命令（r）允许您插入包含在

独立文件中的数据。

读命令的格式为：

```
[address]r filename
```

filename 参数可以为包含数据的文件指定相对或绝对路径。对于读命令，不能使用地址范围。只能指定单一的行号或文本模式地址。sed 编辑器在该地址之后插入文件中的文本。

```
$ cat data11
This is an added line.
This is the second added line.
$ sed '3r data11' data6
This is line number 1.
This is line number 2.
This is line number 3.
This is an added line.
This is the second added line.
This is line number 4.
$
```

sed 编辑器将数据文件中的所有文本行插入数据流。使用文本模式地址时，工作方式相同：

```
$ sed '/number 2/r data11' data6
This is line number 1.
This is line number 2.
This is an added line.
This is the second added line.
This is line number 3.
This is line number 4.
$
```

如果要将文本添加到数据流末尾，只需使用美元地址符号：

```
$ sed '$r data11' data6
This is line number 1.
This is line number 2.
This is line number 3.
This is line number 4.
This is an added line.
This is the second added line.
$
```

读命令的一个很好的用途是和删除命令一起使用，用另一个文件中的数据替代一个文件中的占位符。例如，假设有一封套用信函保存在某个文本文件中，如下所示：

```
$ cat letter
Would the following people:
LIST
please report to the office.
$
```

套用信函使用类占位符 LIST 代替人员列表。要在占位符后插入人员列表，只需使用读命令。不过，这样仍会将占位符文本留在输出中。要删除占位符，只需使用删除命令。结果如下所示：

```
$ sed '/LIST/{
> r data10
> d
> }' letter
Would the following people:
Blum, Rich          Chicago, IL
Mullen, Riley       West Lafayette, IN
Snell, Haley        Ft. Wayne, IN
Woenker, Matthew        Springfield, IL
Wisecarver, Emma        Grant Park, IL
please report to the office.
$
```

现在，占位符文本已经用数据文件中的姓名列表替换了。

## 16.3 小结

尽管shell脚本可以独立完成许多工作，但仅使用shell脚本很难处理数据。Linux提供了两个便利的实用程序来辅助处理文本数据。sed 编辑器是能够在读取数据时动态处理数据的流编辑器。必须为流编辑器提供一系列要应用于数据的编辑命令。

gawk程序是来自GNU组织的一个实用程序，它能够模拟和扩展Unix awk程序的功能。awk程序包含内置编程语言，能够编写处理数据的脚本。可以使用gawk程序从大的数据文件中抽取数据元素，并用想要的格式输出。这使得处理大的日志文件，以及由数据文件创建自定义报告变得很容易。

sed和gawk程序的至关重要的一点是知道如何使用正则表达式。正则表达式是创建自定义筛选器抽取和处理文本文件中的数据的关键。下一章将深入讨论常被误解的正则表达式，说明如何构建正则表达式以处理各种类型的数据。

# 第 17 章
# 正则表达式

在 shell 脚本中成功地使用 sed 编辑器和 gawk 程序的关键是自如地使用正则表达式。这不是一件容易做到的事情，因为设法从大批数据中筛选特定的数据可能（并且通常的确是）很复杂。本章讨论如何在 sed 编辑器和 gawk 程序中创建正则表达式，以筛选自己所需的数据。

## 17.1 正则表达式是什么

理解正则表达式的第一步是准确地定义正则表达式。这一节将介绍什么是正则表达式，以及 Linux 如何使用正则表达式。

### 17.1.1 正则表达式的定义

正则表达式是您定义的、Linux 实用程序用来筛选文本的模式模板。Linux 实用程序（如 sed 编辑器或 gawk 程序）在输入数据时，将正则表达式模式和数据进行匹配。如果数据与模式一致，它接受处理。如果数据与模式不一致，它就拒绝，如图 17-1 所示。

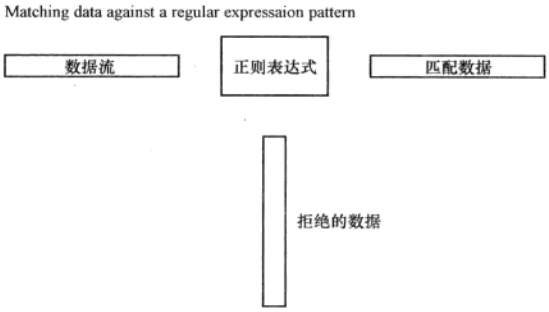



图 17-1 匹配数据和正则表达式模式

正则表达式模式利用通配符表示数据流中的一个或多个字符。在 Linux 中，有许多场合可以指定通配符来表示您不了解的数据。本书曾介绍过一个在 Linux `ls` 命令中使用通配符列出文件和目录的例子（参阅第 3 章）。

星号通配符只允许列出与某一准则匹配的文件。例如：

17
CHAPTER

本章内容：

- 定义正则表达式；
- 基础知识；
- 扩展模式；
- 创建表达式。

```
$ ls -al da*
-rw-r--r--    1 rich    rich           45 Nov 26 12:42 data
-rw-r--r--    1 rich    rich           25 Dec  4 12:40 data.tst
-rw-r--r--    1 rich    rich          180 Nov 26 12:42 data1
-rw-r--r--    1 rich    rich           45 Nov 26 12:44 data2
-rw-r--r--    1 rich    rich           73 Nov 27 12:31 data3
-rw-r--r--    1 rich    rich           79 Nov 28 14:01 data4
-rw-r--r--    1 rich    rich          187 Dec  4 09:45 datatest
$
```

da*参数指示 ls 命令仅列出文件名以 da 开始的文件。在文件名中，da 之后可以有任意多个字符（包括一个也没有）。ls 命令读取该目录中所有文件的信息，但只显示与通配符匹配的文件。

正则表达式通配符模式的工作方式类似。正则表达式模式包括文本和/或特殊字符，可以定义 sed 编辑器和 gawk 程序匹配数据时要遵循的模板。在正则表达式中，可以使用不同的特殊字符定义特定的模式筛选数据。

### 17.1.2　正则表达式的类型

使用正则表达式最大的问题是不是仅有一组正则表达式。在 Linux 领域，几个不同的应用程序要使用不同类型的正则表达式。这些不同的应用程序包括编程语言（Java、Perl 以及 Python）、Linux 实用程序（如 sed 编辑器、gawk 程序、grep 实用程序），以及主流应用程序（如 MySQL 和 PostgreSQL 数据库服务器程序）。

正则表达式可以使用**正则表达式引擎**（regular expression engine）实现。正则表达式引擎是解释正则表达式模式并使用这些模式匹配文本的基础软件。

在 Linux 世界，常用的正则表达式引擎有两种：

- POSIX 基本正则表达式（BRE）引擎；
- POSIX 扩展正则表达式（BRE）引擎。

大多数 Linux 实用程序至少要符合 POSIX BRE 引擎规范，能够识别其定义的所有模式符号。遗憾的是，有些实用程序（如 sed 编辑器）只符合 BRE 引擎规范的一个子集。这主要是由于速度约束，因为 sed 编辑器要尽可能快地处理数据流中的文本。

POSIX BRE 引擎常常可以在依靠正则表达式筛选文本的编程语言中发现。POSIX BRE 引擎可以为通用模式（例如匹配数字、单词以及字母数字字符）提供高级模式符号以及特殊符号。gawk 程序使用 BRE 引擎处理正则表达式模式。

由于实现正则表达式有太多不同的方法，所以很难提出对所有可能的正则表达式的单一的、简明的描述。下面几节将讨论最常见的正则表达式，并说明如何在 sed 编辑器和 gawk 程序中使用这些正则表达式。

## 17.2　定义 BRE 模式

最基本的 BRE 模式是匹配数据流中的文本字符。本节将演示如何在正则表达式模式中定义文本以及预期得到的结果。

### 17.2.1　纯文本

第 16 章演示了在 sed 编辑器和 gawk 程序中如何使用标准的文本字符串筛选数据。下面是一个例子，以唤醒您的记忆：

```
$ echo "This is a test" | sed -n '/test/p'
This is a test
$ echo "This is a test" | sed -n '/trial/p'
```

```
$
$ echo "This is a test" | gawk '/test/{print $0}'
This is a test
$ echo "This is a test" | gawk '/trial/{print $0}'
$
```

第一个模式定义了一个单词 test。sed 编辑器和 gawk 程序脚本各自使用它们自己的打印命令打印与该正则表达式匹配的所有行。由于 echo 语句在其字符串中包含单词 test，所以该数据流文本与定义的正则表达式模式匹配，因而 sed 编辑器显示该行。

第二个模式再一次定义了一个单词，这次是单词 trial。由于 echo 语句文本字符串没有包含该单词，所以正则表达式模式不匹配，因而 sed 编辑器和 gawk 程序都不打印该行。

您可能已经注意到，正则表达式并不关心模式出现在数据流中的位置。模式出现的次数也无关紧要。一旦正则表达式能够在文本字符串的任何位置匹配该模式，它就将字符串传递给使用它的 Linux 实用程序。

关键是匹配正则表达式模式与数据流文本。重要的是要记住正则表达式在匹配模式时异常挑剔。要记住的第一条规则是正则表达式模式区分大小写。这就是说它们只匹配字符大小写正确的模式：

```
$ echo "This is a test" | sed -n '/this/p'
$
$ echo "This is a test" | sed -n '/This/p'
This is a test
$
```

第一次尝试未能匹配，因为单词 this 在该文本字符串中不是全部小写字母。而第二次尝试，由于模式中使用了大写字母，所以刚刚合适。

在正则表达式中，不必局限于完整的单词。如果所定义的文本出现在数据流的任意位置，正则表达式都将匹配：

```
$ echo "The books are expensive" | sed -n '/book/p'
The books are expensive
$
```

尽管数据流中的文本是 books，但数据流中的数据包含正则表达式 book，所以正则表达式模式与数据匹配。当然，如果反过来尝试一下，正则表达式将失败：

```
$ echo "The book is expensive" | sed -n '/books/p'
$
```

全部正则表达式文本没有出现在数据流中，因此匹配失败，sed 编辑器不会显示该文本。

在正则表达式中，也不必局限于单个单词。也可以在文本字符串中包含空格和数字：

```
$ echo "This is line number 1" | sed -n '/ber 1/p'
This is line number 1
$
```

空格将像正则表达式中的其他任何字符一样处理：

```
$ echo "This is line number1" | sed -n '/ber 1/p'
$
```

如果在正则表达式中定义一个空格，它必须出现在数据流中。甚至可以创建一个正则表达式模式来匹配多个连续的空格：

```
$ cat data1
This is a normal line of text.
This is a line with too many spaces.
$ sed -n '/ /p' data1
This is a line with too many spaces.
$
```

单词之间有两个空格那一行与正则表达式模式匹配。这是捕捉文本文件中间距问题的一种重要方法！

### 17.2.2 特殊字符

在正则表达式模式中使用文本字符串时，有一个问题需要注意。在正则表达式中定义文本字符时有几个例外。正则表达式模式赋予了几个字符特殊的含义。如果在文本模式中试图使用这些字符，就

可能得不到预期的结果。

正则表达式认可的特殊字符有：

```
.*[]^${}\+?|()
```

随着本章内容的展开，很快就可以了解这些特殊字符在正则表达式中的作用。现在，只需记住，不要在文本模式中单独使用这些字符。

如果要使用这些特殊字符中的一个作为文本字符，需要**转义**（escape）它。转义特殊字符时，需要在特殊字符的前面添加一个特殊字符，向正则表达式引擎说明：它应该将下一个字符解释为普通文本字符。实现这一功能的特殊字符是反斜杠字符（\）。

例如，如果要在文本中搜索美元符号，需要在其前面放置一个反斜杠字符：

```
$ cat data2
The cost is $4.00
$ sed -n '/\$/p' data2
The cost is $4.00
$
```

由于反斜杠是一个特殊字符，所以如果在正则表达式模式中需要使用反斜杠，也需要对其转义，结果是双反斜杠：

```
$ echo "\ is a special character" | sed -n '/\\/p'
\ is a special character
$
```

最后，虽然正斜杠不是正则表达式的特殊字符，但如果在 sed 编辑器或 gawk 程序的正则表达式模式中使用它，仍会出错：

```
$ echo "3 / 2" | sed -n '///p'
sed: -e expression #1, char 2: No previous regular expression
$
```

要使用正斜杠，也需要对它转义：

```
$ echo "3 / 2" | sed -n '/\//p'
3 / 2
$
```

现在，sed 编辑器可以正确地解释该正则表达式模式，并且一切正常。

### 17.2.3 定位符

如“纯文本”一节所示，默认情况下，在指定正则表达式模式时，只要模式出现在数据流之中，就匹配。有两个特殊字符可以用来将模式定位到数据流行的开头或结尾。

#### 1. 从头开始

脱字符（^）定义从数据流中文本行开头开始的模式。如果该模式位于文本行的其他任意位置，正则表达式失败。

要使用脱字符，需将它放在正则表达式指定的模式之前：

```
$ echo "The book store" | sed -n '/^book/p'
$
$ echo "Books are great" | sed -n '/^Book/p'
Books are great
$
```

脱字符定位符在各个新数据行（由换行符确定）的开头检查模式：

```
$ cat data3
This is a test line.
this is another test line.
A line that tests this feature.
Yet more testing of this
$ sed -n '/^this/p' data3
this is another test line.
$
```

只要该模式出现在新行的开始位置，脱字符定位符就会捕获它。

如果将脱字符放在模式的其他位置，它就充当普通字符而不再作为特殊字符：

```
$ echo "This ^ is a test" | sed -n '/s ^/p'
This ^ is a test
$
```

由于脱字符在正则表达式的最后列出，所以 sed 编辑器将它用作普通字符来匹配文本。

Note 注意：如果要指定仅使用脱字符的正则表达式模式，不必使用反斜杠转义。但是，如果要指定首先是脱字符，然后是其他文本的模式，那么在脱字符前必须使用转义符。

**2. 查找结尾**

与在一行开始查找模式相反的是在行尾查找它。美元符号($)特殊字符定义结尾定位。在文本模式之后添加这个特殊字符表示数据行必须以此文本模式结束：

```
$ echo "This is a good book" | sed -n '/book$/p'
This is a good book
$ echo "This book is good" | sed -n '/book$/p'
$
```

结尾文本模式的问题是对自己要查找的内容必须仔细：

```
$ echo "There are a lot of good books" | sed -n '/book$/p'
$
```

使行末的单词 book 变为复数意味着它与正则表达式模式不再匹配，即使 book 在数据流中。为使模式匹配，该文本模式必须是该行上最后的内容。

**3. 联合定位**

两种常见情况是要在同一行将开始和结尾定位符相结合。第一种情况，假设要查找仅包含某一特定文本模式的数据行：

```
$ cat data4
this is a test of using both anchors
I said this is a test
this is a test
I'm sure this is a test.
$ sed -n '/^this is a test$/p' data4
this is a test
$
```

sed 编辑器会忽略除包含特定文本之外的行。

第二种情况起初看起来可能有点奇怪，但却极其有用。将这两个定位符在没有文本的模式中结合在一起，可以筛选数据流中的空行。请看下面的例子：

```
$ cat data5
This is one test line.

This is another test line.
$ sed '/^$/d' data5
This is one test line.
This is another test line.
$
```

所定义的正则表达式模式查找行头和行尾之间没有任何内容的行。由于空行在两个换行符之间没有包含任何文本，所以与该正则表达式模式匹配。sed 编辑器使用删除命令删除与此正则表达式匹配的行，因此会移除文本中的所有空行。这是移除文档中空行的一种有效方法。

## 17.2.4 点字符

点特殊字符用于匹配除换行符之外的任何单个字符。但点字符必须匹配一个字符；如果在圆点位置没有字符，那么模式匹配失败。

下面是几个在正则表达式模式中使用点字符的例子：

```
$ cat data6
```

```
This is a test of a line.
The cat is sleeping.
That is a very nice hat.
This test is at line four.
at ten o'clock we'll go home.
$ sed -n '/.at/p' data6
The cat is sleeping.
That is a very nice hat.
This test is at line four.
$
```

您应该能够理解第一行失败，而第二行和第三行通过的原因。第四行需要一些技巧。注意，我们要匹配 at，但在其前面没有字符来匹配点字符。但其实是有！在正则表达式中，空格作为字符计，因此 at 前面的空格匹配该模式。第五行通过将 at 放在该行的最前面，进一步证明了这一点，它未能匹配该模式。

### 17.2.5　字符类

点特殊字符适合匹配某一字符位置上的任意字符，但如果要限制匹配的字符又怎么样呢？这在正则表达式中称为**字符类**（character class）。

可以定义一类字符来匹配文本模式中的某一位置。如果该字符类中的字符之一在数据流中，它就和模式匹配。

为定义字符类，要使用方括号。应该将要包括在该类中的所有字符用方括号括起来。然后在模式中使用整个字符类，就像任意的其他通配符一样。一开始，可能要稍微花些时间习惯一下，但一旦理解就可以得到一些相当好的结果。

下面是一个例子，说明创建字符类的方法：

```
$ sed -n '/[ch]at/p' data6
The cat is sleeping.
That is a very nice hat.
$
```

虽然与点特殊字符例子使用相同的数据文件，但我们得到了不同的结果。这次，我们设法筛选出了仅包含单词 at 的那一行。与这一模式匹配的单词仅有 cat 和 hat。同时还注意到，以 at 开始的那一行仍不匹配。必须有字符类中的一个字符匹配适当的位置。

如果不能确定某个字符的大小写，字符类就可以派上用场：

```
$ echo "Yes" | sed -n '/[Yy]es/p'
Yes
$ echo "yes" | sed -n '/[Yy]es/p'
yes
$
```

在单个表达式中可以使用多个字符类：

```
$ echo "Yes" | sed -n '/[Yy][Ee][Ss]/p'
Yes
$ echo "yEs" | sed -n '/[Yy][Ee][Ss]/p'
yEs
$ echo "yeS" | sed -n '/[Yy][Ee][Ss]/p'
yeS
$
```

正则表达式使用 3 个字符类就能包括全部 3 个字符位置的大小写。

字符类不必只包含字母；也可以在其中使用数字：

```
$ cat data7
This line doesn't contain a number.
This line has 1 number on it.
This line a number 2 on it.
This line has a number 4 on it.
$ sed -n '/[0123]/p' data7
This line has 1 number on it.
```

```
This line a number 2 on it.
$
```

正则表达式模式可以匹配包含数字 0、1、2 或 3 的所有行。忽略其他所有数字，与其中没有数字的行一样。

可以组合字符类来检查适当格式化的数字，如电话号码和邮政编码等。但是，在尝试匹配特定格式时，必须小心。

下面是匹配邮政编码时出错的一个例子：

```
$ cat data8
60633
46201
223001
4353
22203
$ sed -n '
>/[0123456789][0123456789][0123456789][0123456789][0123456789]/p
>' data8
60633
46201
223001
22203
$
```

它可能没有产生预期的结果。它在筛选太短不能作为邮政编码的数字方面做的很好，因为最后一个字符类没有与其匹配的字符。然而，即使我们只定义了 5 个字符类，它仍然通过了那个 6 位数字。

记住，正则表达式模式可以出现在数据流文本的任何地方。除匹配的模式字符之外，可能总是有其他的字符。如果要确保仅匹配 5 个数字，就要设法用空格描述或者如下面的例子所示，说明它们应该在一行的开头和结尾：

```
$ sed -n '
> /^[0123456789][0123456789][0123456789][0123456789][0123456789]$/p
> ' data8
60633
46201
22203
$
```

现在好很多了！稍后，我们将讨论进一步简化它的方法。

字符类的一种极其常见的用途是解析可能拼写错误的单词，例如在用户表单中输入的数据。可以毫不费力地创建能够接受数据中常见误拼的词的正则表达式：

```
$ cat data9
I need to have some maintenence done on my car.
I'll pay that in a seperate invoice.
After I pay for the maintenance my car will be as good as new.
$ sed -n '
/maint[ea]n[ae]nce/p
/sep[ea]r[ea]te/p
' data9
I need to have some maintenence done on my car.
I'll pay that in a seperate invoice.
After I pay for the maintenance my car will be as good as new.
$
```

这个例子中的两个 sed 打印命令利用正则表达式字符类帮助捕捉文本中拼错的单词 maintenance 和 separate。相同的正则表达式模式还可以匹配正确拼写的 maintenance 实例。

### 17.2.6 否定字符类

在正则表达式模式中，也可以颠倒字符类的作用。不是查找字符类中包含的字符，而是查找不在该字符类中的字符。为此，只需在字符类范围的开头添加脱字符：

```
$ sed -n '/[^ch]at/p' data6
This test is at line two.
$
```

通过否定字符类，正则表达式模式可以匹配除 c、h 和文本模式之外的任意字符。由于空格字符属于这一类别，所以它通过了模式匹配。但是，即使使用否定，字符类仍必须匹配一个字符，因此 at 位于行首的那一行仍与模式不匹配。

### 17.2.7 使用范围

您可能已经注意到，在本章前面介绍邮政编码示例时，有些麻烦，必须在每个字符类中列出全部可能的数字。幸运的是，可以使用捷径避免这样做。

通过使用短划线符号可以在字符类中使用一系列字符范围。只需指定范围内的第一个字符、短划线，然后是范围内的最后一个字符。根据 Linux 系统使用的字符集（参阅第 2 章），正则表达式包括指定字符范围内的所有字符。

现在，可以通过指定数字的范围简化邮政编码示例：

```
$ sed -n '/^[0-9][0-9][0-9][0-9][0-9]$/p' data8
60633
46201
45902
$
```

这样可以节省许多敲击键盘的动作。每个字符类都将匹配 0～9 之间的任意一个数字。如果数据中任何位置存在一个字母，匹配将失败：

```
$ echo "a8392" | sed -n '/^[0-9][0-9][0-9][0-9][0-9]$/p'
$
$ echo "1839a" | sed -n '/^[0-9][0-9][0-9][0-9][0-9]$/p'
$
$ echo "18a92" | sed -n '/^[0-9][0-9][0-9][0-9][0-9]$/p'
$
```

同样的方法也适用于字母：

```
$ sed -n '/[c-h]at/p' data6
The cat is sleeping.
That is a very nice hat.
$
```

新模式[c-h]at 会匹配首字母在 c 和 h 之间的单词。在这种情况下，单词 at 所在的行不能与模式匹配。

还可以在单个字符类中指定多个非连续的范围：

```
$ sed -n '/[a-ch-m]at/p' data6
The cat is sleeping.
That is a very nice hat.
$
```

字符类允许在 at 文本之前出现 a 到 c、h 到 m 范围内的字母。这一范围将拒绝 d 和 q 之间的所有字母：

```
$ echo "I'm getting too fat." | sed -n '/[a-ch-m]at/p'
$
```

该模式拒绝了 fat 文本，因为它不在指定的范围内。

### 17.2.8 特殊字符类

除定义自己的字符类之外，BRE 还包含特殊字符类，可以用于匹配特定类型的字符。表 17-1 给出了可以使用的 BRE 特殊字符。

表 17-1 BRE 特殊字符类

| 类 | 描述 |
| --- | --- |
| [[:alpha:]] | 匹配任意字母字符，大写或者小写 |
| [[:alnum:]] | 匹配任意字母数字字符，0～9、A～Z 或 a～z |

续表

| 类 | 描述 |
| --- | --- |
| [[:blank:]] | 匹配空格或者制表符字符 |
| [[:digit:]] | 匹配 0～9 之间的数字 |
| [[:lower:]] | 匹配任意小写字母字符，即 a～z |
| [[:print:]] | 匹配任意可打印字符 |
| [[:punct:]] | 匹配标点符号 |
| [[:space:]] | 匹配任意空白字符：空格、制表符、NL、FF、VT、CR |
| [[:upper:]] | 匹配任意大写字母字符，即 A～Z |

在正则表达式中，完全可以像使用普通字符类一样使用特殊字符类：

```
$ echo "abc" | sed -n '/[[:digit:]]/p'
$
$ echo "abc" | sed -n '/[[:alpha:]]/p'
abc
$ echo "abc123" | sed -n '/[[:digit:]]/p'
abc123
$ echo "This is, a test" | sed -n '/[[:punct:]]/p'
This is, a test
$ echo "This is a test" | sed -n '/[[:punct:]]/p'
$
```

使用特殊字符类是定义范围的一种简便方法。不必使用范围[0-9]，而只需使用[[:digit:]]。

### 17.2.9 星号

在某个字符之后加一个星号表示该字符必须在匹配模式的文本中不出现或出现多次：

```
$ echo "ik" | sed -n '/ie*k/p'
ik
$ echo "iek" | sed -n '/ie*k/p'
iek
$ echo "ieek" | sed -n '/ie*k/p'
ieek
$ echo "ieeek" | sed -n '/ie*k/p'
ieeek
$ echo "ieeeek" | sed -n '/ie*k/p'
ieeeek
$
```

该模式符号常用于处理语言拼写中常见的错误拼写或变化的单词。例如，如果要编写可能用于美式或英式英语的脚本，可能这样写：

```
$ echo "I'm getting a color TV" | sed -n '/colou*r/p'
I'm getting a color TV
$ echo "I'm getting a colour TV" | sed -n '/colou*r/p'
I'm getting a colour TV
$
```

该模式中的 u*表示字母 u 可以出现也可以不出现在匹配模式的文本中。类似地，如果知道某个单词常常被拼错，也可以使用星号来包含它：

```
$ echo "I ate a potatoe with my lunch." | sed -n '/potatoe*/p'
I ate a potatoe with my lunch.
$ echo "I ate a potato with my lunch." | sed -n '/potatoe*/p'
I ate a potato with my lunch.
$
```

在可能多余的字母之后加一个星号允许您接受拼错的单词。另一个方便的功能是将点特殊符号与星号特殊符号相结合。这样的组合可以提供匹配许多任意字符的模式。常用于数据流中两个可能彼此相邻出现也可能彼此不相邻出现的文本字符串之间：

```
$ echo "this is a regular pattern expression" | sed -n '
```

```
> /regular.*expression/p'
this is a regular pattern expression
$
```

使用这个模式，可以方便地搜索出现在数据流中一行文本上任意位置的多个单词。

还可以将星号应用于字符类。允许指定在文本中多次出现的一组或某一范围的字符：

```
$ echo "bt" | sed -n '/b[ae]*t/p'
bt
$ echo "bat" | sed -n '/b[ae]*t/p'
bat
$ echo "bet" | sed -n '/b[ae]*t/p'
bet
$ echo "btt" | sed -n '/b[ae]*t/p'
$
$ echo "baat" | sed -n '/b[ae]*t/p'
baat
$ echo "baaeeet" | sed -n '/b[ae]*t/p'
baaeeet
$ echo "baeeaeeat" | sed -n '/b[ae]*t/p'
baeeaeeat
$ echo "baabeeet" | sed -n '/b[ae]*t/p'
$
```

只要 a 和 e 字符的任意组合出现在字符 b 和 t 之间（包括完全不出现），模式就匹配。如果所定义的字符类之外的其他任意字符出现，则模式匹配失败。

## 17.3 扩展的正则表达式

POSIX ERE 模式包括某些 Linux 应用程序和工作使用的其他几个符号。gawk 程序能够识别 ERE 模式，但是 sed 编辑器不能。

警告：重要的是要记住，sed 编辑器和 gawk 程序的正则表达式引擎存在差异。gawk 程序可以使用大部分扩展正则表达式模式的符号，并且具有 sed 编辑器没有的其他一些筛选能力。但是，也正是由于这一点，它处理数据流的速度通常较慢。

本节论述可以在 gawk 程序脚本中使用的常见 ERE 模式符号。

### 17.3.1 问号

问号与星号相似，但略有变化。问号表示其前面的字符可以不出现或出现一次，但只能是这样。它不匹配重复出现的字符：

```
$ echo "bt" | gawk '/be?t/{print $0}'
bt
$ echo "bet" | gawk '/be?t/{print $0}'
bet
$ echo "beet" | gawk '/be?t/{print $0}'
$
$ echo "beeet" | gawk '/be?t/{print $0}'
$
```

如果字符 e 不出现在文本中，或者仅在文本中出现一次，则模式就匹配。

正如星号一样，可以将问号和字符类连用：

```
$ echo "bt" | gawk '/b[ae]?t/{print $0}'
bt
$ echo "bat" | gawk '/b[ae]?t/{print $0}'
bat
$ echo "bot" | gawk '/b[ae]?t/{print $0}'
$
$ echo "bet" | gawk '/b[ae]?t/{print $0}'
bet
```

```
$ echo "baet" | gawk '/b[ae]?t/{print $0}'
$
$ echo "beat" | gawk '/b[ae]?t/{print $0}'
$
$ echo "beet" | gawk '/b[ae]?t/{print $0}'
$
```

如果该字符类中字符没有出现或者其中一个字符出现，模式匹配通过。然而，如果两个字符同时出现，或者一个字符出现两次，则模式匹配失败。

### 17.3.2 加号

加号是与星号相似，但与问号不同的另一个模式符号。加号表示其前面的字符可以出现一次或多次，但必须出现至少一次。如果该字符不存在，则模式不匹配：

```
$ echo "beeet" | gawk '/be+t/{print $0}'
beeet
$ echo "beet" | gawk '/be+t/{print $0}'
beet
$ echo "bet" | gawk '/be+t/{print $0}'
bet
$ echo "bt" | gawk '/be+t/{print $0}'
$
```

如果字符 e 不存在，则模式匹配失败。加号也适用于字符类，与星号和问号的作用方式相同：

```
$ echo "bt" | gawk '/b[ae]+t/{print $0}'
$
$ echo "bat" | gawk '/b[ae]+t/{print $0}'
bat
$ echo "bet" | gawk '/b[ae]+t/{print $0}'
bet
$ echo "beat" | gawk '/b[ae]+t/{print $0}'
beat
$ echo "beet" | gawk '/b[ae]+t/{print $0}'
beet
$ echo "beeat" | gawk '/b[ae]+t/{print $0}'
beeat
$
```

这次，如果字符类中定义的任一字符出现，该文本就与指定的模式匹配。

### 17.3.3 使用大括号

在 ERE 中可以使用大括号指定对可重复的正则表达式的限制。这通常称为**间隔**（interval）。可以用两种格式表示间隔：

- *m*：该正则表达式正好出现 m 次；
- *m*,*n*：该正则表达式出现最少 m 次，最多 n 次；

这一功能允许您调整某一字符（或字符类）在模式中出现的精确次数。

**警告：默认情况下，gawk 程序不能识别正则表达式间隔。必须指定--re-interval 命令行选项，以便 gawk 程序识别正则表达式间隔。**

下面是使用值为 1 的简单间隔的例子：

```
$ echo "bt" | gawk --re-interval '/be{1}t/{print $0}'
$
$ echo "bet" | gawk --re-interval '/be{1}t/{print $0}'
bet
$ echo "beet" | gawk --re-interval '/be{1}t/{print $0}'
$
```

通过指定间隔 1，可以将该字符在要匹配模式的字符串中存在的次数限定为 1。如果该字符出现多次，则该模式匹配失败。

需要指定上下限时，大括号可以派上用场：

```
$ echo "bt" | gawk --re-interval '/be{1,2}t/{print $0}'
$
$ echo "bet" | gawk --re-interval '/be{1,2}t/{print $0}'
bet
$ echo "beet" | gawk --re-interval '/be{1,2}t/{print $0}'
beet
$ echo "beeet" | gawk --re-interval '/be{1,2}t/{print $0}'
$
```

在本例中，字符 e 可以出现一次或两次，模式匹配通过；否则模式匹配失败。

间隔模式匹配也适用于字符类：

```
$ echo "bt" | gawk --re-interval '/b[ae]{1,2}t/{print $0}'
$
$ echo "bat" | gawk --re-interval '/b[ae]{1,2}t/{print $0}'
bat
$ echo "bet" | gawk --re-interval '/b[ae]{1,2}t/{print $0}'
bet
$ echo "beat" | gawk --re-interval '/b[ae]{1,2}t/{print $0}'
beat
$ echo "beet" | gawk --re-interval '/b[ae]{1,2}t/{print $0}'
beet
$ echo "beeat" | gawk --re-interval '/b[ae]{1,2}t/{print $0}'
$
$ echo "baeet" | gawk --re-interval '/b[ae]{1,2}t/{print $0}'
$
$ echo "baeaet" | gawk --re-interval '/b[ae]{1,2}t/{print $0}'
$
```

如果在此文本模式中正好有一个或两个 a 或 e，则此正则表达式模式匹配，但如果存在更多的任意组合，模式匹配就会失败。

### 17.3.4 管道符号

管道符号允许您用逻辑 OR 公式指定正则表达式检查数据流时使用的两个或多个模式。如果任何一个模式与数据流文本匹配，该文本通过。如果没有一个模式匹配，数据流文本失败。

使用管道符号的格式为：

```
expr1|expr2|...
```

下面是一个例子：

```
$ echo "The cat is asleep" | gawk '/cat|dog/{print $0}'
The cat is asleep
$ echo "The dog is asleep" | gawk '/cat|dog/{print $0}'
The dog is asleep
$ echo "The sheep is asleep" | gawk '/cat|dog/{print $0}'
$
```

该例子要在数据流中查找正则表达式 cat 或 dog。正则表达式和管道符号之间不能有任何空格，否则将把空格添加到正则表达式模式中。

管道符号两边的任何正则表达式都可以使用任意正则表达式模式（包括字符类）来定义文本：

```
$ echo "He has a hat." | gawk '/[ch]at|dog/{print $0}'
He has a hat.
$
```

该例子会匹配数据流文本中的 cat、hat 或 dog。

### 17.3.5 将表达式分组

正则表达式模式也可以使用圆括号分组。在将正则表达式模式分组时，一个组合将作为一个标准字符处理。可以像将特殊字符应用于正常字符一样，将特殊字符应用于组合。例如：

```
$ echo "Sat" | gawk '/Sat(urday)?/{print $0}'
```

```
Sat
$ echo "Saturday" | gawk '/Sat(urday)?/{print $0}'
Saturday
$
```

星期的结尾部分组合以及问号允许该模式匹配完整的星期名或缩写名。

将分组和管道符号结合使用以创建可能的模式匹配组很常见：

```
$ echo "cat" | gawk '/(c|b)a(b|t)/{print $0}'
cat
$ echo "cab" | gawk '/(c|b)a(b|t)/{print $0}'
cab
$ echo "bat" | gawk '/(c|b)a(b|t)/{print $0}'
bat
$ echo "bab" | gawk '/(c|b)a(b|t)/{print $0}'
bab
$ echo "tab" | gawk '/(c|b)a(b|t)/{print $0}'
$
$ echo "tac" | gawk '/(c|b)a(b|t)/{print $0}'
$
```

模式(c|b)a(b|t)能够匹配第一组中字母的任意组合以及第二组中字母的任意组合。

## 17.4 正则表达式实战

现在，您已经看到了正则表达式模式的规则和使用正则表达式的几个简单示例，是时候实践这些知识了。下面几节将讨论一些 shell 脚本中常见的正则表达式示例。

### 17.4.1 计算目录文件

作为开始，让我们首先看一个这样的脚本，它可以对 PATH 环境变量定义的目录中存在的可执行文件进行计数。为此，需要将 PATH 变量解析为单独的目录名。第 5 章介绍了如何显示 PATH 环境变量：

```
$ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin:/usr/games:/usr/java/
j2sdk1.4.1_01/bin
$
```

根据应用程序位于 Linux 系统上的位置，PATH 环境变量也不同。关键是要识别 PATH 中用冒号分隔的每一个目录。为得到可以在脚本中使用的目录清单，必须用空格替代每一个冒号。现在，您知道，sed 编辑器使用一个简单的正则表达式就可以做到这一点：

```
$ echo $PATH | sed 's/:/ /g'
/usr/local/bin /bin /usr/bin /usr/X11R6/bin /usr/games /usr/java/
j2sdk1.4.1_01/bin
$
```

一旦将目录分开，就可以在标准 for 语句（参阅第 10 章）中使用它们，以循环处理每一个目录：

```
mypath='echo $PATH | sed 's/:/ /g''
for directory in $mypath
do
...
done
```

一旦拥有各个目录，就可以使用 ls 命令列出各个目录内的所有文件，使用另外一个 for 语句循环处理各个文件，每个文件计数器加 1。

该脚本的最终版如下所示：

```
$ cat countfiles
#!/bin/bash
# count number of files in your PATH
mypath='echo $PATH | sed 's/:/ /g''
count=0
for directory in $mypath
```

```
do
   check='ls $directory'
   for item in $check
   do
         count=$[ $count + 1 ]
   done
   echo "$directory - $count"
   count=0
done
$ ./countfiles
/usr/local/bin - 79
/bin - 86
/usr/bin - 1502
/usr/X11R6/bin - 175
/usr/games - 2
/usr/java/j2sdk1.4.1_01/bin - 27
$
```

### 17.4.2 验证电话号码

上一个例子说明了如何将简单的正则表达式与 sed 结合来替换数据流中的字符，以处理数据。通常，可以使用正则表达式验证数据，以保证该数据在脚本中的格式正确。

常见的数据验证应用程序是核对电话号码的程序。一般数据项表单要求电话号码，而用户通常不能输入格式正确的电话号码。在美国，显示电话号码有几种常用的方法：

```
(123)456-7890
(123) 456-7890
123-456-7890
123.456.7890
```

这样，用户在表单中输入他们的电话号码的方法就有 4 种。正则表达式必须足够强大，能够处理任一种情形。

在构建正则表达式时，最好是从左边开始，构建匹配可能遇到的全部字符的模式。在本例中，第一件事是电话号码中可能有也可能没有一个左圆括号。这个字符可以使用下面的模式来匹配：

```
^\(?
```

使用脱字符指出数据开头。由于左圆括号是一个特殊字符，所以必须将它转义为普通字符使用。问号表示左圆括号可以出现、也可以不出现在要匹配的数据中。

接下来是 3 位数字的地区代码。在美国，地区代码以数字 2 开始（没有使用数字 0 或 1 的地区代码），最大可以到 9。为匹配地区代码，可以使用模式：

```
[2-9][0-9]{2}
```

这就要求第一个字符是 2～9 之间的一个数字，后跟任意两个数字。在地区代码之后，结束右括号可以存在也可以不存在：

```
\)?
```

在地区代码之后可以是一个空格、没有空格、短划线或圆点。可以使用字符组合以及管道符号组合它们：

```
(| |-|\.)
```

第一个管道符号立即出现在左圆括号之后，以匹配没有空格的情况。对于圆点，必须使用转义字符；否则，就会取其匹配任意字符的特殊含义。

再下来是 3 位数字的交换号码。对其没有特殊要求：

```
[0-9]{3}
```

在电话交换码之后，必须匹配一个空格、破折号或点字符（这次不必考虑没有空格的情况，因为在电话交换码和其余的数字之间必须存在至少一个空格）：

```
( |-|\.)
```

然后，为结束整个验证过程，必须匹配字符串尾部、4 位数字的本地电话扩展码：

```
[0-9]{4}$
```

将整个模式放在一起，得到：

```
^\(?[2-9][0-9]{2}\)?(| |-|\.)[0-9]{3}( |-|\.)[0-9]{4}$
```

在 gawk 程序中可以使用这个正则表达式模式筛选非法的电话号码。现在，唯一要做的是在 gawk 程序中使用此正则表达式创建一个简单的脚本，然后通过该脚本筛选自己的电话号码列表。记住，在 gawk 程序中使用正则表达式间隔时，必须使用--re-interval 命令行选项，否则就得不到正确的结果。

下面给出了该脚本：

```
$ cat isphone
#!/bin/bash
# script to filter out bad phone numbers
gawk --re-interval '/^\(?[2-9][0-9]{2}\)?(| |-|\.)
[0-9]{3}( |-|\.)[0-9]{4}/{print $0}'
$
```

尽管在此清单中无法确定，但 gawk 命令在 shell 脚本的一行上。然后，可以将电话号码重定向到该脚本进行处理：

```
$ echo "317-555-1234" | ./isphone
317-555-1234
$ echo "000-555-1234" | ./isphone
$
```

或者可以将整个电话号码文件重定向以筛选有效的号码：

```
$ cat phonelist
000-000-0000
123-456-7890
212-555-1234
(317)555-1234
(202) 555-9876
33523
1234567890
234.123.4567
$ cat phonelist | ./isphone
212-555-1234
(317)555-1234
(202) 555-9876
234.123.4567
$
```

仅与正则表达式模式匹配的有效电话号码出现。

### 17.4.3 解析电子邮件地址

现在，电子邮件地址已经成为至关重要的通信形式。由于构建电子邮件地址的方法有无数种，因此设法验证邮件地址已经成为脚本设计人员必须面对的挑战。电子邮件地址的基本形式为：

```
username@hostname
```

*username* 的值可以使用任意字母数字，以及下面几个特殊字符：

- 圆点；
- 短划线；
- 加号；
- 下划线。

在有效的电子邮件用户 ID 中，这些字符可以任意形式的组合出现。电子邮件地址的 hostname 部分由一个或多个域名和服务器名组成。服务器和域名也必须遵守严格的命名规则，仅允许字母数字字符以及下面的特殊字符：

- 圆点；

✦ 下划线。

服务器和域名之间分别用圆点隔开，且先指定服务器名，然后指定子域名，最后是没有后缀圆点的顶级域名。

过去顶级域名的数目相当少，因而正则表达式模式设计人员试图将它们全部添加到模式中来验证。遗憾的是，随着 Internet 的增长，可能的顶级域也在增长。这种方法已经不再是可行的解决方案。

让我们从左边开始设计该正则表达式模式。我们知道，用户名中可以有多个合法字符。这应该非常简单：

```
^([a-zA-Z0-9_\-\.\+]+)@
```

这样的组合可以指定用户名中允许的字符，加号表示要存在至少一个字符。下一个字符显然应该是@，没有任何疑问。

主机名模式使用相同的方法匹配服务器名和子域名：

```
([a-zA-Z0-9_\-\.]+)
```

这个模式可以匹配文本：

```
server
server.subdomain
server.subdomain.subdomain
```

对于顶级域名有特殊的规则。顶级域名只能是字母字符，并且不能少于两个字符（在国家码部分使用）且长度不能超过 5 个字符。用于顶级域名的正则表达式为：

```
\.([a-zA-Z]{2,5})$
```

将整个模式组合在一起得到：

```
^([a-zA-Z0-9_\-\.\+]+)@([a-zA-Z0-9_\-\.]+)\.([a-zA-Z]{2,5})$
```

该模式可以筛选数据列表中格式不符合要求的电子邮件地址。现在，可以编写脚本实现这个正则表达式：

```
$ echo "rich@here.now" | ./isemail
rich@here.now
$ echo "rich@here.now." | ./isemail
$
$ echo "rich@here.n" | ./isemail
$
$ echo "rich@here-now" | ./isemail
$
$ echo "rich.blum@here.now" | ./isemail
rich.blum@here.now
$ echo "rich blum@here.now" | ./isemail
rich blum@here.now
$ echo "rich/blum@here.now" | ./isemail
$
$ echo "rich#blum@here.now" | ./isemail
$
$ echo "rich*blum@here.now" | ./isemail
$
```

## 17.5 小结

如果要在 shell 脚本中处理数据文件，就要熟悉正则表达式。正则表达式可以在 Linux 实用程序、编程语言以及使用正则表达式引擎的应用程序中实现。在 Linux 领域，有许多不同的正则表达式引擎可以应用。两个最流行的正则表达式引擎是 POSIX 基本正则表达式（BRE）引擎和 POSIX 扩展正则表达式（ERE）引擎。sed 编辑器主要与 BRE 引擎一致，而 gawk 程序可以利用 ERE 引擎的大多数功能。

正则表达式定义用于筛选数据流中文本的模式模板。该模式由标准文本字符和特殊字符的结合组成。正则表达式引擎使用特殊字符匹配一个或多个字符序列，类似于其他应用程序中的通配符的作用。

通过将字符和特殊字符相结合，可以定义匹配大多数数据类型的模式。然后可以使用 sed 编辑器或者 gawk 程序从较大的数据流中筛选特定的数据，或者验证由数据输入应用程序接收的数据。

下一章将深入介绍使用 sed 编辑器进行高级文本处理的方法。在 sed 编辑器中，现有许多先进功能，使其对于处理大的数据流以及筛选所需要的内容非常有用。

CHAPTER 18

# 第 18 章
# 高级 sed 编程

本章内容：

- 进一步了解 sed；
- 处理多行文本；
- 缓存行；
- 使用标签分支；
- 在脚本中使用 sed。

第 16 章展示了如何使用 sed 编辑器的基本特性在数据流中处理文本。基本 sed 编辑器命令已经能够满足文本编辑的日常需要。本章将介绍 sed 编辑器提供的一些更高级的功能。这些功能也许不常用，但最好有所了解，以备将来不时之需。

## 18.1 多行命令

使用基本 sed 编辑器命令时，您可能已经注意到了它的一个限制。所有的 sed 编辑器命令都只在单行数据上执行功能。由于 sed 编辑器读取数据流，因此它以换行符的出现为依据将数据分割为多行。sed 编辑器每次处理一行数据，在一个文本行上处理已经定义的脚本命令，然后移动到下一行重复执行的过程。

但有时需要对多行数据执行操作，特别是需要查找和替换短语时。

例如，如果在数据中查找短语“Linux 系统管理员组”，则这个短语很有可能跨行。如果使用常规的 sed 编辑器命令处理文本，则不可能探测到短语如何跨行。

幸运的是，sed 编辑器的设计者考虑到了这种情况并开发了一种解决方案。sed 编辑器包括 3 个特殊的命令，用于处理多行文本：

- N：在数据流中添加下一行以创建用于处理的多行组。
- D：删除多行组中的单个行。
- P：打印多行组中的单个行。

下面几节将更加详细地介绍这些多行命令，并演示如何在脚本中使用这些命令。

### 18.1.1 next 命令

在介绍多行 next 命令之前，首先需要了解一下 next 命令单行版本的工作方式。了解了该命令的工作方式后，再理解 next 命令多行版本的工作方式就容易多了。

#### 1. 单行 next 命令

小写 n 命令告诉 sed 编辑器移动到数据流中文本的下一行，而不是回到命令的最开始。请记住，通常 sed 编辑器在单个行上

处理完所有定义的命令后才移动到数据流文本的下一行。单行 next 命令改变了这个流程。

这听起来有些复杂，而有时事实确实如此。请先看看下面的简单示例，了解它的工作方式：

```
$ cat data1
This is the header line.

This is a data line.

This is the last line.
$ sed '/header/{
> n
> d
> }' data1
This is the header line.
This is a data line.

This is the last line.
$
```

在本例中，有一个五行的数据文件，其中有两个空行。我们的目标是删除开始行后面的空行，但保留最后一行前面的空行。如果编辑的 sed 脚本只是删除空行，则两个空行都将被删除：

```
$ sed '/^$/d' data1
This is the header line.
This is a data line.
This is the last line.
$
```

由于要删除的是一个空行，因此没有用于搜索的文本以唯一标识该行。解决办法是使用 n 命令。示例脚本查找要删除的空行前面的唯一行，即开始行。识别出开始行后，使用 n 命令将 sed 编辑器移动到文本的下一行。

此时，sed 编辑器继续执行命令列表，即使用 d 命令删除行。当 sed 编辑器到达命令脚本的末尾时，从数据流读取下一行文本并从命令脚本的开头执行命令。

**2. 组合多行文本**

现在，我们了解了单行 next 命令，可以进一步了解其多行版本了。单行 next 命令移动数据流文本的下一行进入 sed 编辑器的处理空间（称为**模式空间**）。next 命令的多行版本（使用大写 N）将文本的下一行添加到已经存在于模式空间的文本中。

其效果相当于将数据流中的两个文本行合并，添加到同一模式空间中。虽然文本行仍然由换行符分隔，但 sed 编辑器已经将它们作为一个文本行处理了。

下面将演示 N 命令的操作方式：

```
$ cat data2
This is the header line.
This is the first data line.
This is the second data line.
This is the last line.
$ sed '/first/{
> N
> s/\n/ /
> }' data2
This is the header line.
This is the first data line. This is the second data line.
This is the last line.
$
```

sed 编辑器搜索包含单词 first 的文本行。当找到该行时，使用 N 命令将该行与下一行合并，然后使用替换命令（s）将换行符替换为一个空格。结果是 sed 编辑器的输出将文本文件的两行显示为一行。

这特别适合用于搜索数据文件中跨行短语的情况。如下面的示例所示：

```
$ cat data3
The first meeting of the Linux System
Administrator's group will be held on Tuesday.
All System Administrators should attend this meeting.
Thank you for your attendance.
$ sed 's/System Administrator/Desktop User/' data3
The first meeting of the Linux System
Administrator's group will be held on Tuesday.
All Desktop Users should attend this meeting.
Thank you for your attendance.
$
```

替换命令在文本文件中查找两个单词组成的短语 System Administrator。在短语出现的单个行上，命令可以正常执行，替换命令可以替换文本。但如果短语跨行出现，则替换命令无法识别匹配模式。

N 命令可以帮助解决此问题：

```
$ sed '
> N
> s/System.Administrator/Desktop User/
> ' data3
The first meeting of the Linux Desktop User's group will be held
on Tuesday.
All Desktop Users should attend this meeting.
Thank you for your attendance.
$
```

通过使用 N 命令将下一行合并到找到第一个单词的行，可以探测到跨行的短语。

请注意，替换命令使用了通配符以匹配空格和换行符。然而，在匹配到换行符时，它被从字符串中删除了，导致两行合并为一行。这可能不是我们想要的结果。

要解决这个问题，可以在 sed 编辑器脚本中使用两个替换命令，一个匹配跨行出现的内容，一个匹配单行出现的内容：

```
$ sed '
> N
> s/System\nAdministrator/Desktop\nUser/
> s/System Administrator/Desktop User/
> ' data3
The first meeting of the Linux Desktop
User's group will be held on Tuesday.
All Desktop Users should attend this meeting.
Thank you for your attendance.
$
```

第一个替换命令专门查找两个要查找的单词之间的换行符，并将其包括在替换内容中。这允许在新文本的同样位置添加换行符。

但是，此脚本中还存在一个微妙的问题。脚本总是在执行 sed 编辑器命令之前将下一行文本读入到模式空间。在到达文本的最后一行时，将没有下一行可以读入，所以 N 命令导致 sed 编辑器停止。如果匹配的文本位于数据流的最后一行，则命令不能找到匹配的数据：

```
$ cat data4
The first meeting of the Linux System
Administrator's group will be held on Tuesday.
All System Administrators should attend this meeting.
$ sed '
> N
> s/System\nAdministrator/Desktop\nUser/
> s/System Administrator/Desktop User/
> ' data4
The first meeting of the Linux Desktop
User's group will be held on Tuesday.
All System Administrators should attend this meeting.
$
```

由于 System Administrator 文本出现在数据流的最后一行，N 命令将它错过了，因为没有下一行来合并到匹配空间中。通过将单行命令移动到 N 命令之前，并使多行命令出现在 N 命令之后，可以轻松解决这个问题，如下所示：

```
$ sed '
> s/System Administrator/Desktop User/
> N
> s/System\nAdministrator/Desktop\nUser/
> ' data4
The first meeting of the Linux Desktop
User's group will be held on Tuesday.
All Desktop Users should attend this meeting.
$
```

现在，查找短语的单行替换命令可以处理数据流的最后一行，并且多行替换命令涵盖了数据流中间的情况。

### 18.1.2 多行删除命令

第 16 章介绍了单行删除命令（d）。sed 编辑器使用它来删除模式空间中的当前行。然而，在使用 N 命令时，使用单行删除命令必须小心。

```
$ sed '
> N
> /System\nAdministrator/d
> ' data4
All System Administrators should attend this meeting.
$
```

删除命令在单独的行上查找单词 System 和 Administrator，然后在模式空间中删除两个行。这可能不是我们想象的那样。

sed 编辑器提供了多行删除命令（D），它只删除模式空间中的第一行。它将删除直至换行符的所有字符（包括换行符）。

```
$ sed '
> N
> /System\nAdministrator/D
> ' data3
Administrator's group will be held on Tuesday.
All System Administrators should attend this meeting.
$
```

由 N 命令添加到模式空间的文本的第二行原封不动。这非常便于删除在找到数据字符串的行之前出现的文本行。

下面是删除数据流中的第一行之前的空行的示例：

```
$ cat data5
This is the header line.
This is a data line.
This is the last line.
$ sed '/^$/{
> N
> /header/D
> }' data5
This is the header line.
This is a data line.

This is the last line.
$
```

sed 编辑器脚本查找空行，然后使用 N 命令将下一行文本添加到模式空间。如果模式空间的新内容包含单词 header，则 D 命令删除模式空间中的第一行。如果不结合使用 N 命令和 D 命令，则无法在不删除其他空行的前提下删除第一个空行。

### 18.1.3 多行打印命令

现在，您可能已经理解了命令的单行版本和多行版本之间的区别。多行打印命令（P）的原理相同。它只打印多行模式空间中的第一行。这包括直至模式空间中换行字符的所有字符。其用法类似于用单行 p 命令显示文本，并使用-n 选项抑制脚本输出。

```
$ sed -n '
> N
> /System\nAdministrator/P
> ' data3
The first meeting of the Linux System
$
```

当发生多行匹配时，P 命令只打印模式空间中的第一行。

多行 P 命令真正的威力在于将它与 N 和 D 多行命令结合使用。

D 命令有一个特有的功能，它能强制 sed 编辑器返回到脚本的开始处，在相同的模式空间（不从数据流中读取新文本行）上重复执行命令。通过在命令脚本中包含 N 命令，可以有效地一步跨过模式空间，一起匹配多个行。

接下来，通过使用 P 命令可以打印第一行，然后使用 D 命令删除第一行并循环回到脚本的开始处。一旦回到脚本的开始处，N 命令读取文本的下一行，然后再次执行全部流程。循环将持续，直至到达数据流的结尾。

## 18.2 保留空间

**模式空间**（pattern space）是一个活动的缓冲区，它在 sed 编辑器处理命令时保留被检查的文本。然而，它不是存储文本的唯一可用空间。

sed 编辑器利用另一个称为**保留空间**（hold space）的缓冲区。在处理模式空间中的其他行时，可以使用保留空间暂时保留文本行。有 5 个命令与使用保留空间有关，如表 18-1 所示。

表 18-1 sed 编辑器保留空间命令

| 命令 | 描述 | 命令 | 描述 |
|---|---|---|---|
| h | 将模式空间复制到保留空间 | G | 将保留空间追加到模式空间 |
| H | 将模式空间追加到保留空间 | x | 将模式空间和保留空间的内容交换 |
| g | 将保留空间复制到模式空间 | | |

这些命令可以用于将文本从模式空间复制到保留空间，其目的是腾空模式空间以载入其他字符串进行处理。

通常，使用 h 或 H 命令移动字符串到保留空间后，最终都会使用 g、G 或 x 命令将保存的字符串移回到模式空间（否则也不必费心将它们保存起来了）。

在使用两个缓冲区时，判断文本行位于哪个缓冲区容易产生混淆。下面的简短示例将演示如何使用 h 和 g 命令在 sed 编辑器缓冲区之间来回移动数据：

```
$ cat data2
This is the header line.
This is the first data line.
This is the second data line.
This is the last line.
$ sed -n '/first/{
> h
> p
> n
> p
```

```
> g
> p
> }' data2
This is the first data line.
This is the second data line.
This is the first data line.
$
```

让我们一步一步来查看此示例：

（1）sed 脚本在地址中使用正则表达式来筛选包含单词 first 的行。

（2）当包含单词 first 的行出现时，h 命令将行放到保留空间中。

（3）p 命令打印模式空间的内容，即仍然是第一行数据。

（4）n 命令检索数据流中的下一行，将其放到模式空间。

（5）p 命令打印模式空间的内容，现在是第二行数据。

（6）g 命令将保留空间的内容（第一行数据）放回到模式空间，替换当前文本。

（7）p 命令打印模式空间的内容，它现在回到第一行数据。

通过使用保留空间来回挪动文本行，可以在输出中强制第一行数据出现在第二行数据之后。如果去掉第一个 p 命令，则可以以相反的顺序输出两行。

```
$ sed -n '/first/{
> h
> n
> p
> g
> p
> }' data2
This is the second data line.
This is the first data line.
$
```

这是实际应用的开始。可以使用此技术创建 sed 脚本以反转整个文本文件行！然而，要实现这个操作，需要了解 sed 编辑器的否定功能，下一节将介绍此功能。

## 18.3 否定命令

在第 16 章中，我们提到 sed 编辑器将命令应用于数据流中的每个文本行，或者应用于通过单个地址或地址范围指定的特定行。也可以配置命令使其不应用到数据流中的特定地址或地址范围。

感叹号命令（!）用于否定命令。它的意思是在通常命令被激活的地方不激活命令。下面是一个演示此功能的示例：

```
$ sed -n '/header/!p' data2
This is the first data line.
This is the second data line.
This is the last line.
$
```

常规的 p 命令只打印出 data2 文件中包含单词 header 的行。添加感叹号后，打印出文件中除包含在地址中引用的文本以外的所有行。

有几种感叹号常用用法。回到本章前面，“next 命令”一节展示了一种情况，sed 编辑器不能对数据流中文本的最后一行执行操作，因为最后一行没有下一行。可以使用感叹号修复这个问题：

```
$ sed '{
$!N
s/System.Administrator/Desktop User/
}' data4
The first meeting of the Linux Desktop User's group will be held on
Tuesday.
```

```
All Desktop Users should attend this meeting.
$
```

此示例展示了将感叹号与 N 命令一起使用，同时用美元符号指定特殊的地址。美元符号表示数据流的最后一行，所以当 sed 编辑器到达最后一行时不会执行 N 命令。对于所有其他的行，则执行命令。

此技术可用于反转数据流中文本行的顺序。要反转文本行出现在文本流中的顺序（首先显示最后一行，最后显示第一行），需要使用保留空间做一些工作。

需要使用的工作模式类似于下面的流程：

（1）将一行放到保留空间中。

（2）将文本的下一行放到模式空间中。

（3）将保留空间追加到模式空间。

（4）将模式空间放到保留空间。

（5）重复第 2 步到第 4 步，直至将所有行以相反的顺序放到保留空间。

（6）检索行并打印它们。

图 18-1 展示了这个过程。

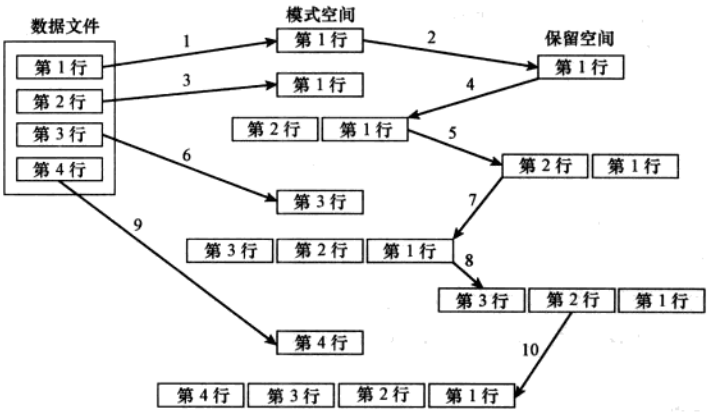



图 18-1　使用保留空间反转文本文件的顺序

使用这个技术时，您不希望在运行时打印行。这意味着需要使用 sed 的-n 命令行选项。下一个需要决定的是如何将保留空间的文本追加到模式空间的文本后面。这是使用 G 命令完成的。唯一的问题就是，您不希望将保留空间追加到处理文本的第一行。可以使用感叹号命令轻易解决这个问题：

```
1!G
```

下一步是将新的模式空间（追加了相反顺序行的文本行）放到保留空间。这也非常简单，只需使用 h 命令。

在模式空间中以相反的顺序获取整个数据流时，需要做的只是打印出来。当到达数据流的最后一行时，即在模式空间中获取了整个数据流。要打印结果，只需使用下面的命令：

```
$p
```

以上就是创建反转行 sed 编辑器脚本所要使用的一些命令行。现在，让我们来试着运行一下：

```
$ cat data2
This is the header line.
This is the first data line.
This is the second data line.
This is the last line.
$ sed -n '{
1!G
h
$p
```

```
}' data2
This is the last line.
This is the second data line.
This is the first data line.
This is the header line.
$
```

sed 编辑器脚本按照预期的方式运行了。脚本的输出将文本文件中的原始行反转。这说明了在 sed 编辑器中使用保留空间是非常有用的。它为操作脚本输出中行的顺序提供了一种简单方法。

注意：也许您正在想，有一个 Linux 命令也可以执行反转文本文件的功能。tac 命令可以以相反的顺序显示文本文件。您可能已经注意到这个命令名称的巧妙之处，因为它正是执行与 cat 命令相反的功能。

## 18.4 更改命令流

通常，sed 编辑器从脚本的开头开始执行命令，并一直执行到脚本的结尾（D 命令例外，它强制 sed 编辑器返回到脚本的开头而不读取新的文本行）。sed 编辑器提供了改变脚本命令流的方法，产生一种类似于结构化编程环境的效果。

### 18.4.1 分支

在上一节中，我们学习了如何使用感叹号命令在文本行上否定命令的执行。sed 编辑器提供了一种否定整个命令小节（基于地址、地址模式或地址范围）的执行的方法。这允许我们只在数据流的特定子集上执行一组命令。

分支命令的格式为：

```
[address]b [label]
```

*adderss* 参数决定哪个行或哪些行激发分支命令。label 参数定义于何处分支。如果 label 参数不存在，则分支命令将继续执行到脚本的结尾。

```
$ sed '{
> 2,3b
> s/This is/Is this/
> s/line./test?/
> }' data2
Is this the header test?
This is the first data line.
This is the second data line.
Is this the last test?
$
```

分支命令跳过数据流中的第二行和第三行的替换命令。

除了到达脚本的最后，还可以定义一个想要分支命令转到的标签。标签以冒号开头，最长为 7 个字符：

```
:label2
```

要指定标签，只需要将它添加到 b 命令的后面。使用标签可以跳过与分支地址匹配的命令，而仍然执行脚本中的其他命令：

```
$ sed '{
> /first/b jump1
> s/ is/ might be/
> s/line/test/
> :jump1
> s/data/text/
> }' data2
This might be the header test.
This is the first text line.
This might be the second text test.
```

```
This might be the last test.
$
```

分支命令指示，如果行中出现匹配文本 first，则程序应该跳到脚本中以 jump1 标记的行。如果分支命令模式不匹配，则 sed 编辑器继续执行脚本中的命令，包括分支标签之后的命令。(也就是说，在不匹配分支模式的行上继续执行所有 3 个替换命令)。

如果行匹配分支模式，则 sed 编辑器分支到分支标签行。因此，只执行最后一个替换命令。

该示例展示了分支到 sed 脚本后面标签的情况。还可以分支到出现在脚本前面的标签，这样就创建了一个循环效果：

```
$ echo "This, is, a, test, to, remove, commas." | sed -n '{
> :start
> s/,//1p
> b start
> }'
This is, a, test, to, remove, commas.
This is a, test, to, remove, commas.
This is a test, to, remove, commas.
This is a test to, remove, commas.
This is a test to remove, commas.
This is a test to remove commas.
```

脚本的每一次递归都从文本字符串删除第一个出现的逗号，然后打印该字符串。该脚本产生了一个难题：它永远不会结束。这种情况创建了一个无休止的循环，它搜索逗号，直至通过 Ctrl+C 组合键手动发出停止信号。

要防止这个问题发生，应该为分支命令指定供其查找的地址模式。如果模式没有出现，则分支应该停止：

```
$ echo "This, is, a, test, to, remove, commas." | sed -n '{
:start
s/,//1p
/,/b start
}'
This is, a, test, to, remove, commas.
This is a, test, to, remove, commas.
This is a test, to, remove, commas.
This is a test to, remove, commas.
This is a test to remove, commas.
This is a test to remove commas.
$
```

现在，分支命令只在行中包含逗号时执行分支。最后一个逗号被删除后，分支命令将不会再执行，允许脚本适当地终止。

### 18.4.2　测试

与分支命令类似，测试命令（t）也用于更改 sed 编辑器脚本的流程。测试命令不是基于地址跳转到标签，而是基于替换命令的结果跳转到标签。

如果替换命令成功匹配并替换了一个模式，则测试命令分支到指定的标签。如果替换命令不匹配指定的模式，则测试命令不分支。

测试命令使用与分支命令相同的格式：

```
[address]t [label]
```

与分支命令类似，如果不指定标签，则测试成功时 sed 将分支到脚本的最后。

测试命令为在数据流的文本上执行基本的 if-then 语句提供了一种简单的方式。例如，如果在另一个替换成功时不需要再执行替换，则测试命令可以帮忙：

```
$ sed '{
> s/first/starting/
```

```
> t
> s/line/test/
> }' data2
This is the header test.
This is the starting data line.
This is the second data test.
This is the last test.
$
```

第一个替换命令查找模式文本 first。如果行上能够匹配模式，则替换该文本，而测试命令跳过后续的替换命令。如果第一个替换命令未能匹配，则执行第二个替换命令。

使用测试命令可以改进分支命令循环：

```
$ echo "This, is, a, test, to, remove, commas." | sed -n '{
:start
s/,//1p
t start
}'
This is, a, test, to, remove, commas.
This is a, test, to, remove, commas.
This is a test, to, remove, commas.
This is a test to, remove, commas.
This is a test to remove, commas.
This is a test to remove commas.
$
```

当没有可替换的内容时，测试命令将不再分支并继续执行脚本剩下的部分。

## 18.5 模式替换

我们已经学习了如何在 sed 命令中使用模式替换数据流中的文本。然而，问题是使用通配符时并不能简单地知道什么样的文本可以和模式匹配得上。

例如，想要给一行中匹配上的单词加上双引号。如果只在模式中查找一个单词，那么这非常容易：

```
$ echo "The cat sleeps in his hat." | sed 's/cat/"cat"/'
The "cat" sleeps in his hat.
$
```

但是，如果在模式中使用通配符以匹配多个单词会产生如下结果：

```
$ echo "The cat sleeps in his hat." | sed 's/.at/".at"/g'
The ".at" sleeps in his ".at".
$
```

看来这行不通。替换字符串使用句点通配符匹配任何以 at 结尾的单词。不幸的是，替换字符串不能以通配符的值替换匹配的单词。

### 18.5.1 与号

sed 编辑器为此提供了一种解决方法。与号（&）用于表示替换命令中的匹配模式。无论什么文本匹配上定义的模式，都可以使用与号在替换模式中调用它。这允许处理与定义的模式匹配的任何单词：

```
$ echo "The cat sleeps in his hat." | sed 's/.at/"&"/g'
The "cat" sleeps in his "hat".
$
```

当模式匹配 cat，cat 出现在替换的单词中。当它匹配 hat 时，hat 出现在替换的单词中。

### 18.5.2 替换个别单词

与号检索与在替换命令中指定的模式匹配的整个字符串。有时可能只想检索字符串的子集。这当然也没有问题，但需要一些小技巧。

sed 编辑器使用圆括号定义替换模式中的子字符串元素。然后在替换模式中使用特定的符号来引

用子字符串元素。替换字符由反斜杠和数字组成。数字表示子字符串元素的位置。sed 编辑器将第一个元素分配为字符\1，第二个元素分配成字符\2，依次类推。

警告：在替换命令中使用圆括号时，必须使用转义字符来表示它们是分组字符，而不是普通的圆括号。这不利于对其他特殊字符的转义。

下面是使用此功能的一个 sed 编辑器脚本的示例：

```
$ echo "The System Administrator manual" | sed '
> s/\(System\) Administrator/\1 User/'
The System User manual
$
```

此替换命令对单词 System 使用了一组圆括号，将其指定为子字符串元素。然后在替换模式中使用\1 调用第一个指定的元素。这没有什么特别之处，但在使用通配符时确实非常管用。

如果需要以单个单词（短语的子集）替换一个短语，而子字符串又正好使用了通配符，则最好使用子字符串元素：

```
$ echo "That furry cat is pretty" | sed 's/furry \(.at\)/\1/'
That cat is pretty
$ echo "That furry hat is pretty" | sed 's/furry \(.at\)/\1/'
That hat is pretty
$
```

在这种情况下，不能使用与号，因为它将替换整个匹配的模式。子字符串元素提供了答案，它允许将模式的某个部分用作替换模式。

在需要将文本插入到两个或多个子字符串元素时，这一功能特别有用。下面是使用子字符串元素在长数字中插入逗号的示例脚本：

```
$ echo "1234567" | sed '{
> :start
> s/\(.*[0-9]\)\([0-9]\{3\}\)/\1,\2/
> t start
> }'
1,234,567
$
```

脚本将匹配模式分隔为两个元素：

```
.*[0-9]
[0-9]{3}
```

此模式查找两个子字符串。第一个字符串是任意个字符，以数字结尾。第二个子字符串是一系列 3 个数字（请参阅第 17 章，了解如何在正则表达式中使用括号）。如果在文本中找到该模式，则替换文本在两个元素之间放置一个逗号，每个元素是根据元素的位置进行区分的。脚本使用测试命令在数字中递归，直至放置好所有的逗号。

## 18.6　在脚本中使用 sed

现在，我们学习了 sed 编辑器的各个方面，是时候加以综合并在 shell 脚本中使用它们了。本节将介绍一些在 bash shell 脚本中使用 sed 编辑器时应该知道的功能。

### 18.6.1　使用包装器

您可能已经注意到，尝试运行 sed 编辑器脚本可能是件麻烦事，特别是脚本很长时。与其在每次需要时重新输入整个脚本，不如将 sed 编辑器命令放到 shell 脚本**包装器**（wrapper）中。包装器就像是 sed 编辑器脚本和命令行之间的媒介。

在 shell 脚本中时，可以在 sed 编辑器脚本中使用常规的 shell 变量和参数。下面的示例展示了将命令行参数变量用作 sed 脚本输入：

```
$ cat reverse
```

```
#!/bin/bash
# shell wrapper for sed editor script to reverse lines

sed -n '{
1!G
h
$p
}' "$1"
$
```

shell 脚本的名称是 reverse，它使用 sed 编辑器脚本反转数据流中的文本行。它使用$1 shell 参数从命令行检索第一个参数，它应该是要反转文本的名称：

```
$ ./reverse data2
This is the last line.
This is the second data line.
This is the first data line.
This is the header line.
$
```

现在，您可以轻松地对任何文件使用 sed 编辑器脚本，而不需要不断地重新输入整个命令行了。

### 18.6.2 重定向 sed 输出

默认情况下，sed 编辑器将脚本的结果输出到 STDOUT。可以在 shell 脚本中使用所有标准方法来重定向 sed 编辑器的输出。

可以使用反引号将 sed 编辑器命令的输出重定向到一个变量，以便在脚本的后面使用。下面的示例展示了使用 sed 脚本为数字计算的结果添加逗号：

```
$ cat fact
#!/bin/bash
#add commas to numbers in factorial answer

factorial=1
counter=1
number=$1

while [ $counter -le $number ]
do
   factorial=$[ $factorial * $counter ]
   counter=$[ $counter + 1 ]
done

result='echo $factorial | sed '{
:start
s/\(.*[0-9]\)\([0-9]\{3\}\)/\1,\2/
t start
}''

echo "The result is $result"
$ ./fact 20
The result is 2,432,902,008,176,640,000
$
```

在使用常规的阶乘计算脚本后，该脚本的结果被用作 sed 编辑器脚本的输入，以添加逗号。然后在 echo 语句中使用该值产生出结果。

## 18.7 创建 sed 工具

从本章前面介绍的几个简短的例子可以看出，使用 sed 编辑器可以执行许多非常酷的格式化数据任务。本节将展示几个便利和广泛使用的 sed 编辑器脚本，它们都用于执行常规的数据处理功能。

### 18.7.1 双倍行距

首先，我们先看一个在文本文件的各行之间添加空行的例子：

```
$ sed 'G' data2
This is the header line.

This is the first data line.

This is the second data line.

This is the last line.

$
```

这非常简单！其技巧在于保留空间的默认值。请记住，G 命令只是单纯地将保留空间的内容追加到模式空间的内容。在启动 sed 编辑器时，保留空间包含一个空行。通过将其追加到现有的行，可以在现有行后创建一个空行。

您可能已经注意到，此脚本还为数据流中的最后一行添加空行，在文件的结尾产生了一个空行。如果想去掉这个空行，可以使用否定符号和末行符号来确保脚本不会在数据流的末行之后添加空行：

```
$ sed '$!G' data2
This is the header line.

This is the first data line.

This is the second data line.

This is the last line.
$
```

现在的效果好多了。只要不是末行，那么 G 命令就追加保留空间的内容。当 sed 编辑器到达最后一行时，则跳过 G 命令。

### 18.7.2 对可能有空行的文件使用双倍行距

有时需要更进一步地使用双倍行距，例如文件文件已经具有一些空行，而想让所有的行都使用双倍行距。如果使用前面的脚本，则有些地方的行距就会过大，即已经具有空行的地方会产生两个空行：

```
$ cat data6
This is line one.
This is line two.

This is line three.
This is line four.
$ sed '$!G' data6
This is line one.

This is line two.


This is line three.

This is line four.
$
```

现在，原来有一个空行的位置有了 3 个空行。解决的办法是先从数据流中删除空行，然后使用 G 命令在所有行后插入新的空行。要删除现有空行，只需要使用 d 命令和能够匹配上空行的模式：

```
/^$/d
```

此模式使用行的开始标记（脱字符号）和结束标记（美元符号）。将其添加到脚本则可生成希望的结果：

```
$ sed '/^$/d;$!G' data6
This is line one.
```

```
This is line two.

This is line three.

This is line four.
$
```

太完美了！

### 18.7.3 对文件中的行记数

第 16 章展示了如何使用等号显示数据流中的行数：

```
$ sed '=' data2
1
This is the header line.
2
This is the first data line.
3
This is the second data line.
4
This is the last line.
$
```

这读起来有一些费劲，因为行号位于数据流中实际行的上面。更好的解决办法是将行号置于文本的同一行上。

现在，我们已经学习了如何使用 N 命令对行进行组合，在 sed 编辑器脚本中利用这一信息应该不会太难。其中的技巧在于不能在同一脚本中组合两个命令。

得到等于号命令的输出结果后，可以通过管道将其输出到另一个使用 N 命令结合两个行的 sed 编辑器脚本。还需要使用替换命令以空格或制表符替换换行符。如下面的示例所示：

```
$ sed '=' data2 | sed 'N; s/\n/ /'
1 This is the header line.
2 This is the first data line.
3 This is the second data line.
4 This is the last line.
$
```

现在看起来好多了。在需要查看错误消息的行号时，这个小工具用起来非常得心应手。

### 18.7.4 打印最后几行

目前为止，我们学习了如何使用 p 命令打印数据流中的所有行或与特定模式匹配的行。如果需要使用一个很长的列表的最后几行（如日志文件），又该如何呢？

美元符号表示数据流的最后一行，所以要显示最后一行非常容易：

```
$ sed -n '$p' data2
This is the last line.
$
```

现在的问题是，如何使用美元符号显示数据流中的最后几行？答案是创建**滚动窗口**（rolling window）。

滚动窗口是一种用于检查模式空间中的文本行块的常用方法，通常与 N 命令组合使用。N 命令将文本的下一行追加到已经存在于模式空间中的文本后面。假如在模式空间中有一个 10 行的文本块，可以通过美元符号判断是否处于数据流的最末。如果不是最末，则可以继续将更多的行添加到模式空间，但删除原始的行（使用 D 命令，删除模式空间的第一行）。

通过 N 命令和 D 命令的循环，可以向模式空间中的行块添加新行，同时删除旧行。分支命令非常适合用于这个循环。要结束循环，只需找出最后一行并使用 q 命令退出。

下面是最终的 sed 编辑器脚本：

```
$ sed '{
```

```
> :start
> $q
> N
> 11,$D
> b start
> }' /etc/passwd
mysql:x:415:416:MySQL server:/var/lib/mysql:/bin/bash
rich:x:501:501:Rich:/home/rich:/bin/bash
katie:x:502:506:Katie:/home/katie:/bin/bash
jessica:x:503:507:Jessica:/home/jessica:/bin/bash
testy:x:504:504:Test account:/home/testy:/bin/csh
barbara:x:416:417:Barbara:/home/barbara/:/bin/bash
ian:x:505:508:Ian:/home/ian:/bin/bash
emma:x:506:509:Emma:/home/emma:/bin/bash
bryce:x:507:510:Bryce:/home/bryce:/bin/bash
test:x:508:511::/home/test:/bin/bash
$
```

脚本首先查看行是否是数据流中的最后一行。如果是，则退出命令将停止循环。N 命令将下一行追加到模式空间当前的行。如果当前行是第 10 行后面的行，`11,$D` 命令删除模式空间中的第一行。这创造了一种滚动窗口的效果。

### 18.7.5　删除行

sed 编辑器另一个有用的工具是删除数据流中不需要的空行。从数据流中删除所有空行非常容易，但有选择地删除空行就需要技巧了。本节将展示一些简单的 sed 编辑器脚本，可以使用这些脚本帮助从数据中删除不需要的空行。

#### 1. 删除连续空行

数据文件中空行泛滥是一件令人讨厌的事。通常一个数据文件中有许多空行，而有时数据行丢失并产生许多空行（如前面讲到的使用双倍行距的示例）。

删除连续空行的最简单的方法是使用地址范围查看数据流。第 16 章展示了如何在地址中使用范围，包括如何在地址范围中结合一些模式。sed 编辑器对与指定的地址范围匹配的所有行执行命令。

删除连续空行的关键是创建一个地址范围，该范围包括一个非空行和一个空行。如果 sed 编辑器遇到这个范围，则不会删除行。然而，对于不匹配该范围的行（行中的两个或多个空行），则删除这些行。

如下面的脚本所示：

```
/./,/^$/!d
```

范围是`/./`到`/^$/`。范围中的起始地址匹配任何包含至少一个字符的行。范围中的结束地址匹配一个空行。位于此范围的行不会被删除。

这个脚本的操作示例如下：

```
$ cat data6
This is the first line.


This is the second line.

This is the third line.


This is the fourth line.
$ sed '/./,/^$/!d' data6
This is the first line.

This is the second line.
```

```
This is the third line.

This is the fourth line.
$
```

无论文件中数据行之间有多少空行，输出仅在各行之间保留一个空行。

**2. 删除开头的空行**

另一件麻烦的事是数据文件在开始处包含多个空行。通常，在将数据从文本文件导入到数据库中时，空行将创建许多 **null** 条目，任何使用这些数据的计算都是白费力气。

从数据流的开头删除空行并不是一个困难任务。下面的脚本可完成此功能：

```
/./,$!d
```

此脚本使用地址范围确定要删除的行。地址范围以包含一个字符的行开始，一直到数据流的结尾结束。此范围内的任何行都不会从输出中删除。这意味着包含一个字符的第一行之前的任何行都将被删除。

这个简单脚本的操作示例如下：

```
$ cat data7


This is the first line.

This is the second line.
$ sed '/./,$!d' data7
This is the first line.

This is the second line.
$
```

测试文件在数据行前包含两个空行。脚本成功地删除了开始处的两个空行，而保留了数据间的空行。

**3. 删除结尾的空行**

不幸的是，删除结尾的空行不像删除开头处的空行那样容易。正如打印出数据流的结尾一样，删除数据流结尾的空行需要一些技巧并使用循环。

在开始讨论之前，让我们来看看脚本的样子：

```
sed '{
:start
/^\n*$/{$d; N; b start }
}'
```

乍看起来可能有些奇怪。请注意，常规脚本的大括号之间还有大括号。这允许在全部的命令脚本之中对命令进行组合。组合的命令应用于特定的地址模式。地址模式匹配任何只包含换行符的行。当找到一行时，如果是最后一行，则删除命令将其删除。如果不是最后一行，则 N 命令将追加其下一行，然后分支命令循环回到开始处。

这个脚本的操作示例如下：

```
$ cat data8
This is the first line.
This is the second line.


$ sed '{
:start
/^\n*$/{$d ; N; b start }
}' data8
This is the first line.
This is the second line.
$
```

脚本成功地删除了文本文件最后的空行。

### 18.7.6　删除 HTML 标记

现如今，从 Web 站点下载文本以保存或用作应用程序的数据已经是再寻常不过的事情了。然而，有时在从 Web 站点下载文本时还会将用于格式化数据的 HTML 标记也一同下载下来。如果想要的只是数据，那么这就成问题了。

标准的 HTML Web 页包含若干种不同类型的 HTML 标记，标识格式化特性以适当地显示页面信息。下面是一个 HTML 文件的示例：

```
$ cat data9
<html>
<head>
<title>This is the page title</title>
</head>
<body>
<p>
This is the <b>first</b> line in the Web page. This should provide
some <i>useful</i> information for us to use in our shell script.
</body>
</html>
$
```

HTML 标记通过小于号和大于号标识。大多数 HTML 标记都成对出现。一个标记表示格式化过程的开始（例如，用于加粗），而另一个标记表示格式化过程的结束（例如，表示结束加粗）。

如果不加以小心，则删除 HTML 标记时会造成一定的后果。表面上看，您可能认为删除 HTML 标记就是简单地查找以小于号开始，以大于号结束，其中包含数据的文本：

```
s/<.*>//g
```

不幸的是，这个命令将产生一些不希望的结果：

```
$ sed 's/<.*>//g' data9


This is the line in the Web page. This should provide
some information for us to use in our shell script.

$
```

请注意，标题文本不见了，加粗和斜体的文本也不见了。sed 编辑器从字面上解析脚本，认为要删除任何小于号和大于号之间的文本，也包括其他的小于号和大于号！每个被 HTML 标记包裹的文本（如first），sed 脚本都删除整个文本。

这个问题的解决方法是让 sed 编辑器忽略原始标记之间嵌入的大于号。要完成此操作，可以创建一个字符类，让它否定大于号。将脚本更改为：

```
s/<[^>]*>//g
```

脚本可以正常工作，即显示需要从 Web 页 HTML 代码查看的数据：

```
$ sed 's/<[^>]*>//g' data9

This is the page title


This is the first line in the Web page. This should provide
some useful information for us to use in our shell script.

$
```

现在好多了。要使输出更加简洁，可以添加一个删除命令来删除多余的空行：

```
$ sed 's/<[^>]*>//g;/^$/d' data9
This is the page title
This is the first line in the Web page. This should provide
```

```
some useful information for us to use in our shell script.
$
```

现在的结果更加紧凑了：只有需要查看的数据。

## 18.8 小结

sed 编辑器提供了许多允许使用跨多行的文本模式的高级功能。本章展示了如何使用 next 命令检索数据流中的下一行并将其放入模式空间。在模式空间中，可以执行复杂的替换命令，将跨多行文本的短语替换掉。

多行删除命令允许在模式空间包含两行或多行时删除第一行。这是一种在数据流中递归多行的简便方法。类似地，多行打印命令允许在模式空间包含两个或多个文本行时只打印第一行。结合使用多行命令允许在数据流中递归和创建多行替换系统。

接下来，本章讨论了保留空间。保留空间允许在处理多个文本行时保留一行文本。可以在任何时间重新调用保留空间的内容，用于替换模式空间的文本或者将保留空间的内容追加到模式空间。使用保留空间允许在数据流中排序，反转文本行在数据中出现的顺序。

本章还讨论了 sed 编辑器流控制命令。分支命令提供了一种更改脚本中常规的 sed 编辑器命令流、创建循环或在特定条件下跳过命令的方法。测试命令为 sed 编辑器命令脚本提供了一种类似于 if-then 的语句。测试命令只在替换命令成功替换行中的文本的条件下执行分支。

本章的最后讨论了如何在 shell 脚本中使用 sed 脚本。对于大型的 sed 脚本，常用的方法是将脚本放到 shell 包装器中。在 sed 脚本中使用命令行参数变量以传递 shell 命令行值。这为直接从命令行或其他 shell 脚本使用 sed 编辑器脚本提供了一种简便方法。

下一章将深入探索 gawk 世界。Gawk 程序支持高级编程语言的许多特性。使用 gawk 可以创建一些很好的数据操作和报告程序。另外，下一章还将介绍不同的编程特性，以及如何使用这些特性从简单的数据生成很好的报表。

CHAPTER 19

# 第 19 章
# 高级 gawk 编程

本章内容：

- 深入了解 gawk；
- 在 gawk 中使用变量；
- 结构化命令；
- 格式化输出；
- 函数。

第 16 章介绍了 gawk 程序并演示了它的基本使用方法，即从原数据文件创建格式化的报告。本章将深入研究如何自定义 gawk 以生成报告。gawk 程序是一种成熟的编程语言，它提供了编写高级程序进行数据处理所需要的功能。在本章中，我们将了解如何使用 gawk 编程语言编写程序，以处理可能遇到的任何数据格式化任务。

## 19.1 使用变量

任何编程语言的一项重要功能就是使用变量保存并重新调用值。gawk 编程语言支持两种不同类型的变量：

- 内置变量；
- 用户定义的变量。

在 gawk 中有许多内置变量可用。内置变量包含在处理数据文件中的数据字段和记录中使用的信息。还可以在 gawk 程序中创建自己的变量。下面几节将介绍如何在 gawk 程序中使用变量。

### 19.1.1 内置变量

gawk 程序使用内置变量引用程序数据内的特定功能。本节将描述在 gawk 程序中可以使用的内置变量以及这些内置变量的用法。

**1. 字段和记录分隔符变量**

第 16 章描述了 gawk 中的一种内置变量，数据字段变量。数据字段变量允许使用美元符号和数据字段在记录中的数字位置引用数据记录中的单个数据字段。因此，要引用记录中的第一个数据字段，则可以使用`$1`变量。要引用第二个数据字段，则使用`$2`变量，依次类推。

数据字段由字段分隔符号描述。默认情况下，字段分隔符号是一个空白符号，如空格或制表符。第 16 章展示了如何从命令行上使用`-F`命令行参数或在 gawk 程序中使用特定的`FS`内置变量更改字段分隔符号。

`FS`内置变量属于一组内置变量，它们用于控制 gawk 在输入数据和输出数据中处理字段和记录的方式。表 19-1 列出了这个组中包含的内置变量。

表 19-1 gawk 数据字段和记录变量

| 变量 | 描述 |
| --- | --- |
| FIELDWIDTHS | 以空格分隔的数字列表，用空格定义每个数据字段的精确宽度 |
| FS | 输入字段分隔符号 |
| RS | 输入记录分隔符号 |
| OFS | 输出字段分隔符号 |
| ORS | 输出记录分隔符号 |

FS 和 OFS 变量定义 gawk 程序在数据流中处理数据字段的方式。我们已经学习了如何使用 FS 变量定义字符以分隔记录中的数据字段。OFS 变量执行相同的功能，但适用 print 命令的输出。

默认情况下，gawk 将 OFS 变量设置为空格，所以在使用下面的命令时：

```
print $1,$2,$3
```

将看到如下结果：

```
field1 field2 field3
```

示例如下所示：

```
$ cat data1
data11,data12,data13,data14,data15
data21,data22,data23,data24,data25
data31,data32,data33,data34,data35
$ gawk 'BEGIN{FS=","} {print $1,$2,$3}' data1
data11 data12 data13
data21 data22 data23
data31 data32 data33
$
```

print 命令自动将 OFS 变量的值放到输出的每个数据字段之间。通过设置 OFS 变量，可以使用任何字符串分隔输出中的数据字段：

```
$ gawk 'BEGIN{FS=","; OFS="-"} {print $1,$2,$3}' data1
data11-data12-data13
data21-data22-data23
data31-data32-data33
$ gawk 'BEGIN{FS=","; OFS="--"} {print $1,$2,$3}' data1
data11--data12--data13
data21--data22--data23
data31--data32--data33
$ gawk 'BEGIN{FS=","; OFS="<-->"} {print $1,$2,$3}' data1
data11<-->data12<-->data13
data21<-->data22<-->data23
data31<-->data32<-->data33
$
```

FIELDWIDTHS 变量允许不使用字段分隔符读取记录。在某些应用程序中，数据不使用字段分隔符号，而是被放置在记录的特定的列中。在这种情况下，必须设置 FIELDWIDTHS 变量以匹配记录中的数据布局。

设置 FIELDWIDTHS 变量后，gawk 忽略 FS 并基于提供的字段宽度计算数据字段。下面的示例就使用了字段宽度而不是字段分隔符号：

```
$ cat data1b
1005.3247596.37
115-2.349194.00
05810.1298100.1
$ gawk 'BEGIN{FIELDWIDTHS="3 5 2 5"}{print $1,$2,$3,$4}' data1b
100 5.324 75 96.37
115 -2.34 91 94.00
058 10.12 98 100.1
$
```

FIELDWIDTHS 变量定义了 4 个数据字段，gawk 相应地解析数据记录。每个记录中的数字字符串由定义的宽度值进行分隔。

警告：请千万记住，设置FIELDWIDTHS变量后，这些值必须是常量。此方法不支持长度为变量的数据字段。

RS和ORS变量定义gawk程序处理数据流中记录的方式。默认情况下，gawk将RS和ORS变量设置为换行符。默认RS变量值表示输入数据流中文本的每个新行为一条新的记录。

有时，数据字段分布在数据流的多行上。典型的例子是包含地址和电话号码的数据，它们分别在单独的行上：

```
Riley Mullen
123 Main Street
Chicago, IL 60601
(312)555-1234
```

如果使用默认的FS和RS变量值读取数据，则gawk会将每行作为单独的记录处理，并将记录中的每个空格作为字段分隔符。这不是我们想要的结果。

要解决这个问题，请将RS变量设置为空字符串，然后在数据流中的数据记录之间保留一个空行。gawk程序会将每个空行解析为记录的分隔符。

如下面的示例所示：

```
$ cat data2
Riley Mullen
123 Main Street
Chicago, IL 60601
(312)555-1234

Frank Williams
456 Oak Street
Indianapolis, IN 46201
(317)555-9876

Haley Snell
4231 Elm Street
Detroit, MI 48201
(313)555-4938
$ gawk 'BEGIN{FS="\n"; RS=""} {print $1,$4}' data2
Riley Mullen (312)555-1234
Frank Williams (317)555-9876
Haley Snell (313)555-4938
$
```

太完美了，gawk程序将文件中的每一行当作一个数据字段并将空行当作记录分隔符。

### 2. 数据变量

除了字段和记录分隔符变量以外，gawk还提供了一些其他的内置变量，这些变量有助于了解数据处理进行的状况和从shell环境中提取信息。表19-2展示了gawk中的其他内置变量。

表19-2 更多gawk内置变量

| 变量 | 描述 |
|---|---|
| ARGC | 出现的命令行参数的个数 |
| ARGIND | 当前正在处理的文件在ARGV中的索引 |
| ARGV | 命令行参数数组 |
| CONVFMT | 数字的转换格式（参见printf语句）。默认值为%.6g |
| ENVIRON | 当前shell环境变量及其值的关联数组 |
| ERRNO | 当读取或关闭输入文件时发生错误时的系统错误 |
| FILENAME | 用于输入到gawk程序的数据文件的文件名 |
| FNR | 数据文件的当前记录号 |
| IGNORECASE | 如果设置为非0，则忽略gawk命令中使用的字符串的大小写 |
| NF | 数据文件中数据字段的个数 |
| NR | 已处理的输入记录的个数 |

续表

| 变量 | 描述 |
| --- | --- |
| OFMT | 显示数字的输出格式。默认为%.6g |
| RLENGTH | 匹配函数中匹配上的子字符串的长度 |
| RSTART | 匹配函数中匹配上的子字符串的开始索引 |

在进行 shell 脚本编程时可能已经接触了其中一些变量。ARGC 和 ARGV 变量用于从 shell 检索命令行参数的编号及其值。然而这需要一定的技巧，因为 gawk 不认为程序脚本是命令行参数的一部分。

```
$ gawk 'BEGIN{print ARGC,ARGV[1]}' data1
2 data1
$
```

ARGC 变量表示命令行上有两个参数。这包括 gawk 命令和 data1 参数（注意，程序脚本不是参数）。ARGV 数组以索引 0 开始，它表示命令。第一个数组值是 gawk 命令后的第一个命令行参数。

注意：请注意，与 shell 变量不同，在脚本中引用 gawk 变量时，不需要在变量名称前加美元符号。

ENVIRON 变量可能看起来有些奇怪。它使用**关联数组**（associative array）检索 shell 环境变量。关联数组使用文本而不是使用数值作为数组的索引值。

数组索引中的文本是 shell 环境变量。数组的值就是 shell 环境变量的值。如下面的示例所示：

```
$ gawk '
> BEGIN{
> print ENVIRON["HOME"]
> print ENVIRON["PATH"]
> }'
/home/rich
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin
$
```

ENVIRON["HOME"]变量从 shell 检索 HOME 环境变量的值。类似地，ENVIRON["PATH"]变量检索 PATH 环境变量的值。可以使用这些技术从 shell 检索任何环境变量值以用于自己的 gawk 程序。

FNR、NF 和 NR 变量在 gawk 程序中用于跟踪数据字段和记录。有时，您可能不知道记录中具体有多少个数据字段。NF 变量允许指出记录中的最后一个数据字段而不需要知道它的位置：

```
$ gawk 'BEGIN{FS=":"; OFS=":"} {print $1,$NF}' /etc/passwd
rich:/bin/bash
testy:/bin/csh
mark:/bin/bash
dan:/bin/bash
mike:/bin/bash
test:/bin/bash
$
```

NF 变量包含数据文件中最后一个数据字段的数值。通过在前面加美元符号，可以将其作为数据字段变量使用。

FNR 和 NR 变量相类似，只是略有不同。FNR 变量包含当前数据文件中处理的记录数。NR 变量包含已经处理的记录的总数。让我们看几个例子来了解它们的不同之处：

```
$ gawk 'BEGIN{FS=","}{print $1,"FNR="FNR}' data1 data1
data11 FNR=1
data21 FNR=2
data31 FNR=3
data11 FNR=1
data21 FNR=2
data31 FNR=3
$
```

在本例中，gawk 程序命令行定义了两个输入文件（它两次都指定了同一输入文件）。脚本打印第一个数据字段的值和 FNR 变量的当前值。请注意，FNR 变量在 gawk 程序处理第二个数据文件时被重

新设置为 1。

现在，让我们添加 NR 变量，看看结果是什么：

```
$ gawk '
> BEGIN {FS=","}
> {print $1,"FNR="FNR,"NR="NR}
> END{print "There were",NR,"records processed"}' data1 data1
data11 FNR=1 NR=1
data21 FNR=2 NR=2
data31 FNR=3 NR=3
data11 FNR=1 NR=4
data21 FNR=2 NR=5
data31 FNR=3 NR=6
There were 6 records processed
$
```

FNR 变量值在 gawk 处理第二个数据文件时被重置，但 NR 变量在处理第二个数据文件时仍然继续增加。也就是说，如果只使用一个数据文件作为输入，则 FNR 和 NR 值将是相同的。如果使用多个数据文件作为输入，则 FNR 值在每次处理新的数据文件时重置，而 NR 值对所有数据文件进行累积计数。

注意：您可能已经注意到了，在使用 gawk 时，通常 gawk 脚本会变得比其他 shell 脚本大。在本章的示例中，为简单起见，我都是直接从命令行运行 gawk 脚本，使用了 shell 的多行功能。在 shell 脚本中使用 gawk 时，应该在不同的行上放置不同 gawk 命令。这可以让脚本更容易阅读和跟踪，而不是全部挤在 shell 脚本的一行上。

## 19.1.2　用户定义的变量

与其他高级编程语言类似，gawk 允许用户定义自己的变量以便在程序代码中使用。gawk 用户自定义变量的名称可以是多个字母、数字和下划线，但不能以数字开头。还需要记住的是，gawk 变量名称是区分大小写的。

### 1. 在脚本中赋值变量

在 gawk 程序中赋值变量的方法类似于 shell 脚本，即使用**赋值语句**（assignment statement）：

```
$ gawk '
> BEGIN{
> testing="This is a test"
> print testing
> }'
This is a test
$
```

print 语句的输出是 testing 变量的当前值。像 shell 脚本变量一样，gawk 变量可以保存数字或文本值：

```
$ gawk '
> BEGIN{
> testing="This is a test"
> print testing
> testing=45
> print testing
> }'
This is a test
45
$
```

在本例中，testing 变量的值从文本变换为数值。

赋值语句还可以包含数学算法以处理数值：

```
$ gawk 'BEGIN{x=4; x= x * 2 + 3; print x}'
11
$
```

如上面的示例所示，gawk 编程语言包括了处理数值的标准数学运算符。还可以包含求余的符号(%)

和求幂的符号（使用^或**）。

**2. 在命令行中赋值变量**

还可以使用 gawk 命令行为 gawk 程序变量赋值。这允许您在常规代码以外设置值、实时更改值。下面的示例使用命令行变量显示文件中的特定数据字段：

```
$ cat script1
BEGIN{FS=","}
{print $n}
$ gawk -f script1 n=2 data1
data12
data22
data32
$ gawk -f script1 n=3 data1
data13
data23
data33
$
```

此功能可以更改脚本的行为而不必更改实际的脚本代码。第一个示例展示了文件中的第二个数据字段，而第二个示例展示了第三个数据字段，只是简单地在命令行上设置 n 变量的值。

使用命令行参数定义变量的值存在一个问题。在设置变量时，值不能在代码的 BEGIN 部分使用：

```
$ cat script2
BEGIN{print "The starting value is",n; FS=","}
{print $n}
$ gawk -f script2 n=3 data1
The starting value is
data13
data23
data33
$
```

可以通过在命令行参数中使用-v 来解决。它也允许指定在代码的 BEGIN 部分之前设置的变量。-v 命令行参数必须放置在命令行上的脚本代码之前：

```
$ gawk -v n=3 -f script2 data1
The starting value is 3
data13
data23
data33
$
```

现在 n 变量包括命令行上设置的，在代码的 BEGIN 部分使用的值。

## 19.2 使用数组

许多编程语言都提供数组在一个变量中保存多个值。gawk 编程语言使用**关联数组**（associative arrays）提供这一功能。

关联数组不同于数字数组，它的索引值可以是任何文本字符串，不需要使用一系列的数字来标识数组中包含的数据元素。关联数组由引用各个值的混杂的字符串构成。每个索引字符串必须唯一，且唯一标识分配给它的数据元素。

下面几节将演示在 gawk 程序中使用关联数组的方法。

### 19.2.1 定义数组变量

可以使用标准的赋值语句定义数组变量。数组变量的赋值格式为：

```
var[index] = element
```

其中，var 是变量的名称，index 是关联数组的索引值，而 element 是数据元素值。下面是 gawk 中的一些数组变量的示例：

```
capital["Illinois"] = "Springfield"
capital["Indiana"] = "Indianapolis"
capital["Ohio"] = "Columbus"
```

在引用数组变量时，必须包括索引值才能检索相应的数据元素值：

```
$ gawk 'BEGIN{
> capital["Illinois"] = "Springfield"
> print capital["Illinois"]
> }'
Springfield
$
```

在引用数组变量时，数据元素值出现了。这也适用数字数据元素值：

```
$ gawk 'BEGIN{
> var[1] = 34
> var[2] = 3
> total = var[1] + var[2]
> print total
> }'
37
$
```

从这个示例可以看出，在gawk程序中可以像任何其他变量一样使用数组变量。

### 19.2.2 在数组变量中递归

使用关联数组变量的一个问题是，我们可能不知道索引值是什么。与数字数组不同，它使用连续的数字作为索引值，而关联数组的索引可以是任何值。

如果需要在gawk的关联数组中递归，可以使用特殊格式的for语句：

```
for (var in array)
{
   statements
}
```

for语句在各语句中循环，每次都向变量var分配array关联数组中的下一个索引值。要点是要记住变量是索引的值，而不是数据元素值。通过将变量用作数组索引，可以轻松地提取数据元素值：

```
$ gawk 'BEGIN{
> var["a"] = 1
> var["g"] = 2
> var["m"] = 3
> var["u"] = 4
> for (test in var)
> {
>    print "Index:",test," - Value:",var[test]
> }
> }'
Index: u - Value: 4
Index: m - Value: 3
Index: a - Value: 1
Index: g - Value: 2
$
```

请注意，索引值不是以一定的顺序返回的，但它们引用了正确的数据元素值。重要的是要知道，不能指望以相同的顺序返回值，但索引和数据值是匹配的。

### 19.2.3 删除数组变量

从关联数组删除数组索引需要一个特殊的命令：

```
delete array[index]
```

delete命令从数组删除关联索引值和关联的数组元素值：

```
$ gawk 'BEGIN{
```

```
> var["a"] = 1
> var["g"] = 2
> for (test in var)
> {
>    print "Index:",test," - Value:",var[test]
> }
> delete var["g"]
> print "---"
> for (test in var)
>    print "Index:",test," - Value:",var[test]
> }'
Index: a - Value: 1
Index: g - Value: 2
---
Index: a - Value: 1
$
```

从关联数组删除索引值后，就不能再检索它了。

## 19.3 使用模式

gawk 程序支持多种类型的模式匹配以过滤数据记录，这非常类似于 sed 编辑器的处理方式。第 16 章已经以实例介绍了两种特殊的模式。BEGIN 和 END 关键字是特殊的模式，它们分别在读取数据流数据之前和之后执行语句。类似地，可以创建其他模式以便在匹配数据出现在数据流中时执行语句。

本节将演示如何在 gawk 脚本中使用匹配模式以限制程序脚本应用到特定的记录。

### 19.3.1 正则表达式

第 17 章展示了如何将正则表达式用作匹配模式。可以使用**基本正则表达式**（Basic Regular Expression，BRE），或者**扩展的正则表达式**（Extended Regular Expression，ERE）来过滤程序脚本要应用到的数据流中的行。

在使用正则表达式时，正则表达式必须出现在程序脚本的左括号之前：

```
$ gawk 'BEGIN{FS=","} /11/{print $1}' data1
data11
$
```

正则表达式/11/匹配任何数据字段包含字符串 11 的记录。gawk 程序将定义的正则表达式从记录中的所有数据字段进行匹配，包括字段分隔符：

```
$ gawk 'BEGIN{FS=","} /,d/{print $1}' data1
data11
data21
data31
$
```

这个示例在正则表达式中匹配用作字段分隔符的逗号。这并不总是好的做法，它会导致匹配特定于一个数据字段的数据可能出现在另一个数据字段中。如果需要让正则表达式匹配特定的数据，则应该使用匹配操作符。

### 19.3.2 匹配操作符

**匹配操作符**（matching operator）允许限制正则表达式匹配记录中的特定数据字段。匹配操作符即波浪号（～）。需要指定匹配操作符，数据字段变量，还有要匹配的正则表达式：

```
$1 ~ /^data/
```

$1 变量表示记录中的第一个数据字段。此表达式可过滤出第一个数据字段以文本 data 开头的记录。下面是在 gawk 程序脚本中使用它的示例：

```
$ gawk 'BEGIN{FS=","} $2 ~ /^data2/{print $0}' data1
```

```
data21,data22,data23,data24,data25
$
```

匹配操作符将第二个数据字段与正则表达式/&data2/进行比较，/&data2/表示以文本 data2 开头的字符串。

这是一种在gawk程序脚本中常用的，也是非常好用的方法，用于在数据文件中搜索特定的数据元素：

```
$ gawk -F: '$1 ~ /rich/{print $1,$NF}' /etc/passwd
rich /bin/bash
$
```

此示例在第一个数据字段中搜索文本 rich。当在记录中找到模式时，打印出记录的第一个和最后一个数据字段。

还可以通过!符号否定正则表达式的匹配：

```
$1 ! ~ /expression/
```

如果没有在记录中找到正则表达式，则程序脚本将应用到记录数据上：

```
$ gawk 'BEGIN{FS=","} $2 ! ~ /^data2/{print $1}' data1
data11
data31
$
```

在本例中，gawk程序脚本打印记录的第二个数据字段不以 data2 开头的第一个数据字段。

### 19.3.3 数学表达式

除了正则表达式，还可以在匹配模式中使用数学表达式。当数据字段中包含数值时，使用这一功能将非常得心应手。例如，如果想显示所有属于根用户组（组号为0）的系统用户，则可以使用下面的脚本：

```
$ gawk -F: '$4 == 0{print $1}' /etc/passwd
root
sync
shutdown
halt
operator
$
```

脚本查看第4个数字字段包含值0的记录。在我的Linux系统上，有5个属于根用户组的用户账户。

可以使用常规的数学比较表达式：

- x == y：值x等于y。
- x <= y：值x小于等于y。
- x < y：值x小于y。
- x >= y：值x大于等于y。
- x > y：值x大于y。

还可以使用带文本数据的表达式，但使用时必须小心。不同于正则表达式，表达式是精确匹配的。数据必须与模式精确匹配：

```
$ gawk -F, '$1 == "data"{print $1}' data1
$
$ gawk -F, '$1 == "data11"{print $1}' data1
data11
$
```

第一个测试不匹配任何记录，因为第一个数据字段值 data 不存在于任何记录。第二个测试匹配值为 data11 的一条记录。

## 19.4 结构化命令

gawk编程语言支持一般的结构化编程命令。本节将介绍这些命令并演示如何在gawk编程环境中使用这些命令。

### 19.4.1 if 语句

gawk 编程语言支持 if 语句的标准 if-then-else 格式。必须为 if 语句定义一个条件以进行评估，条件使用圆括号括起来。如果条件的值为 TRUE，则执行紧跟在 if 语句后面的语句。如果条件值为 FALSE，则跳过语句。其格式为：

```
if (condition)
    statement1
```

或者如下所示，放在同一行上：

```
if (condition) statement1
```

下面的简单示例展示了这个格式：

```
$ cat data4
10
5
13
50
34
$ gawk '{if ($1 > 20) print $1}' data4
50
34
$
```

不是很复杂。如果需要在 if 语句中执行多个语句，则必须用大括号括起来：

```
$ gawk '{
> if ($1 > 20)
> {
>   x = $1 * 2
>   print x
> }
> }' data4
100
68
$
```

请不要将 if 语句的括号与程序脚本的开始和结束括号混淆。gawk 程序可以检测到丢失的括号并在您弄乱时报告错误消息：

```
$ gawk '{
> if ($1 > 20)
> {
>   x = $1 * 2
>   print x
> }' data4
gawk: cmd. line:7: (END OF FILE)
gawk: cmd. line:7: parse error
$
```

gawk 的 if 语句还支持 else 分句，允许在 if 语句失败时执行一个或多个语句。下面是使用 else 分句的示例：

```
$ gawk '{
> if ($1 > 20)
> {
>    x = $1 * 2
>    print x
> } else
> {
>    x = $1 / 2
>    print x
> }}' data4
5
2.5
6.5
```

```
100
68
$
```

可以在单个行上使用 else 分句，但必须在 if 语句部分后使用分号：

```
if (condition) statement1; else statement2
```

下面是使用单行格式的同一示例：

```
$ gawk '{if ($1 > 20) print $1 * 2; else print $1 / 2}' data4
5
2.5
6.5
100
68
$
```

其格式更加紧凑，但也更难阅读。

### 19.4.2 while 语句

while 语句为 gawk 程序提供了基本的循环功能。while 语句的格式为：

```
while (condition)
{
   statements
}
```

while 循环允许在数据集中递归，检查停止递归的条件。当每个记录中有多个数据值，且必须在计算中使用这些值时，while 语句非常有用：

```
$ cat data5
130 120 135
160 113 140
145 170 215
$ gawk '{
> total = 0
> i = 1
> while (i < 4)
> {
>    total += $i
>    i++
> }
> avg = total / 3
> print "Average:",avg
> }' data5
Average: 128.333
Average: 137.667
Average: 176.667
$
```

while 语句在记录中的数据字段中递归，将值加到 total 变量，然后增加计数变量 i。当计数器的值等于 4 时，while 的条件变成 FALSE，循环终止，进入到脚本的下一语句。该语句计算平均值，然后打印平均值。数据文件中的每条记录都重复执行这个过程。

gawk 编程语言支持在 while 循环中使用 break 和 continue 语句，从而跳出循环：

```
$ gawk '{
> total = 0
> i = 1
> while (i < 4)
> {
>    total += $i
>    if (i == 2)
>       break
>    i++
> }
> avg = total / 2
```

```
> print "The average of the first two data elements is:",avg
> }' data5
The average of the first two data elements is: 125
The average of the first two data elements is: 136.5
The average of the first two data elements is: 157.5
$
```

break 语句用于在 i 变量的值为 2 时跳出 while 循环。

### 19.4.3 do-while 语句

do-while 语句类似于 while 语句，但是在检查条件之前执行语句。do-while 语句的格式为：

```
do
{
   statements
} while (condition)
```

此格式保证在评估条件之前语句至少被执行一次。这在需要评估条件之前执行语句时非常有用：

```
$ gawk '{
> total = 0
> i = 1
> do
> {
>     total += $i
>     i++
> } while (total < 150)
> print total }' data5
250
160
315
$
```

脚本从每条记录读取数据字段，然后计算总和，直至和的值达到 150。如果第一个数据字段超过 150（如第二个记录），则脚本可以至少在评估条件之前读取第一个数据字段。

### 19.4.4 for 语句

for 语句是许多编程语言常用的循环方法。gawk 编程语言支持 C 语言形式的 for 循环：

```
for( variable assignment; condition; iteration process)
```

这可以通过将若干个功能组合在一个语句中来简化循环：

```
$ gawk '{
> total = 0
> for (i = 1; i < 4; i++)
> {
>     total += $i
> }
> avg = total / 3
> print "Average:",avg
> }' data5
Average: 128.333
Average: 137.667
Average: 176.667
$
```

通过在 for 循环中定义递归计数器，不需要担心像使用 while 语句那样来自己增加计数器的值。

## 19.5 格式化打印

您可能已经注意到，print 语句不能够完全控制 gawk 显示数据的方式。而我们能做的只是控制输出字段的分隔符号（OFS）。如果需要创建详细的报告，则通常将数据放到特定的格式和位置中。

解决办法是使用格式化打印命令，即 printf 命令。如果熟悉 C 语言编程，则 gawk 执行 printf

命令的方式与其相同，允许对显示数据的方式进行详细说明。

printf 命令的格式为：

```
printf "format string", var1, var2...
```

*format string* 是格式化输出的关键。它使用文本元素和**格式化说明符**（format specifier）说明了格式化后输出的显示方式。格式化说明符是一种特殊的代码，它指示显示什么样的变量类型及其显示方式。gawk 程序将每个格式化说明符用作命令中列出的每个变量的占位符。第一个格式化说明符匹配列出的第一个变量，第二个格式化说明符匹配列出的第二个变量，依次类推。

格式化说明符使用下面的格式：

```
%[modifier]control-letter
```

其中，*control-letter* 是表示要显示的数据值类型的字符代码，而 *modifier* 定义可选的格式化功能。

表 19-3 列出了可以在格式化说明符中使用的控制字。

因此，如果需要显示字符串变量，则可以使用格式化说明符%s。如果需要显示整数，则使用%d 或%i（%d 是 C 语言样式的十进制数）。如果想以科学计数法显示一个较大的数，则需要使用%e 格式化说明符：

```
$ gawk 'BEGIN{
> x = 10 * 100
> printf "The answer is: %e\n", x
> }'
The answer is: 1.000000e+03
$
```

**表 19-3　　格式化说明符控制字**

| 控制字 | 描述 | 控制字 | 描述 |
|---|---|---|---|
| c | 将数据显示为 ASCII 字符 | g | 以科学计数法和浮点数的较短者显示数字 |
| d | 显示整数值 | o | 显示八进制数值 |
| i | 显示整数值（与 d 相同） | s | 显示文本字符串 |
| e | 用科学计数法显示数字 | x | 显示十六进制数值 |
| f | 显示浮点数值 | X | 显示十六进制数值，但对 A 到 F 使用大写字母 |

除了控制字以外，还可以使用 3 种修饰符更多地控制输出。

- width：指定输出字段最小宽度的数字值。如果输出比较短，printf 用空格填补空白，使文本右对齐。如果输出的宽度超过指定的宽度，则使用该值覆盖宽度。
- prec：指定浮点数中十进制数小数点右侧位数的数值，或者文本字符串中显示的最大字符数。
- -（减号）：减号表示在将数据放到格式化空间时不使用右对齐，而是使用左对齐。

在使用 printf 语句时，可以完全控制输出的格式。例如，在“内置变量”一节中，我们使用了 print 命令打印记录中要显示的数据字段：

```
$ gawk 'BEGIN{FS="\n"; RS=""} {print $1,$4}' data2
Riley Mullen (312)555-1234
Frank Williams (317)555-9876
Haley Snell (313)555-4938
$
```

可以使用 printf 命令帮助格式化输出以使其更加美观。首先，让我们只将 print 命令替换为 printf 命令，看看是什么效果：

```
$ gawk 'BEGIN{FS="\n"; RS=""} {printf "%s %s\n", $1, $4}' data 2
Riley Mullen (312)555-1234
Frank Williams (317)555-9876
Haley Snell (313)555-4938
$
```

输出的结果与 print 命令的结果相同。printf 命令使用%s 格式化说明符作为两个字符串值的占位符。

请注意，必须手动在 printf 命令的结尾添加换行符以强制换行。如果不强制换行，则 printf

命令将继续使用同一行打印后面的内容。

这在需要将内容打印到同一行上时非常有用，但是要使用单独的 printf 命令：

```
$ gawk 'BEGIN{FS=","} {printf "%s ", $1} END{printf "\n"}' data1
data11 data21 data31
$
```

每个 printf 的输出都出现在同一行上。要终止行，END 的部分应打印换行符。

接下来，让我们使用修饰符格式化第一个字符串的值：

```
$ gawk 'BEGIN{FS="\n"; RS=""} {printf "%16s %s\n", $1, $4}' data2
    Riley Mullen (312)555-1234
  Frank Williams (317)555-9876
     Haley Snell (313)555-4938
$
```

通过添加 16 修饰符值，我们可以强制第一个字符串占 16 个字符。默认情况下，printf 命令使用右对齐将数据放到格式化的空间中。要使用左对齐，只需要向修饰符添加一个减号：

```
$ gawk 'BEGIN{FS="\n"; RS=""} {printf "%-16s %s\n", $1, $4}' data2
Riley Mullen      (312)555-1234
Frank Williams    (317)555-9876
Haley Snell       (313)555-4938
$
```

现在输出看起来更加专业了！

printf 命令在处理浮点数值时也非常便利。通过指定变量的格式，可以使输出看起来更加统一：

```
$ gawk '{
> total = 0
> for (i = 1; i < 4; i++)
> {
>     total += $i
> }
> avg = total / 3
> printf "Average: %5.1f\n",avg
> }' data5
Average: 128.3
Average: 137.7
Average: 176.7
$
```

通过使用%5.1f 格式化描述符，可以强制 printf 命令浮点值保留小数点后一位。

## 19.6 内置函数

gawk 编程语言提供了许多执行常规数学计算、字符串计算，甚至是时间计算的内置函数。可以在您的 gawk 程序中使用这些函数帮助缩减脚本中的代码量。本节将介绍 gawk 中各种不同的内置函数。

### 19.6.1 数学函数

如果使用过任何类型的语言编写过程序，则一定对在代码中使用内置函数执行常规的数学计算有所了解。gawk 编程语言当然也不会让那些想使用高级数学功能的人失望。

表 19-4 展示了 gawk 中的内置数学函数。

表 19-4 gawk 数学函数

| 函数 | 描述 | 函数 | 描述 |
|---|---|---|---|
| atan2(x, y) | x/y 的反正切，x 和 y 以弧度表示 | rand() | 大于 0 小于 1 的随机浮点值 |
| cos(x) | x 的余弦，x 以弧度表示 | sin(x) | x 的正弦，x 以弧度表示 |
| exp(x) | x 的指数 | sqrt(x) | x 的平方根 |
| int(x) | x 的整数部分，截止到 0 | srand(x) | 用指定值计算的随机数 |
| log(x) | x 的自然对数 | | |

虽然没有列出完全的数学函数，但 gawk 确实提供了一些用于标准数学处理的基本元素。int() 函数计算一个值的整数部分，但不会四舍五入。它与其他编程语言的取整函数的功能类似。它产生 0 和值之间最接近整数的值。

这表示，5.6 的 int() 函数值将返回 5，而-5.6 的 int() 函数值将返回-5。

rand() 函数用于创建随机数，但需要使用一些技巧来获取有意义的值。rand() 函数返回一个随机数，但只返回 0 到 1 之间（不包括 0 和 1）的随机数。要取得更大的数值，需要放大返回的值。

常用的做法是使用 rand() 函数和 int() 函数生成一个算法来产生更大的整数：

```
x = int(10 * rand())
```

这将返回 0～9（包括 0 和 9）之间的随机整数。只需要为应用程序将 10 替换为更高的限制值，就可以得到更大的整数。

有一些数学函数需要小心使用，因为 gawk 编程语言只能处理有限范围的数字值。如果超出范围，则会得到一条错误消息：

```
$ gawk 'BEGIN{x=exp(100); print x}'
2.68812e+43
$ gawk 'BEGIN{x=exp(1000); print x}'
gawk: cmd. line:1: warning: exp argument 1000 is out of range
inf
$
```

第一个示例计算 100 的指数，这是一个非常大的数，但是在系统的范围之内。第二个示例试图计算 1000 的指数，它超出了系统的数值范围限制并产生了一条错误消息。

除了标准的数学函数，gawk 还提供了一些函数用于数据的位操作处理：

- and(*v1*, *v2*)：执行 *v1* 和 *v2* 的与操作。
- compl(*val*)：执行对 *val* 的补位。
- lshift(*val*, count)：将值 *val* 向左移动 count 位。
- or(*v1*, *v2*)：执行 *v1* 和 *v2* 的或操作。
- rshift(*val*, count)：将值 *val* 向右移动 count 位。
- xor(*v1*, *v2*)：执行 *v1* 和 *v2* 的异或操作。

在数据中使用二进制数时，位操作函数非常有用。

## 19.6.2 字符串函数

gawk 编程语言还提供了一些用于处理字符串值的函数，如表 19-5 所示。

**表 19-5 gawk 字符串函数**

| 函数 | 描述 |
|---|---|
| asort(*s* [,*d*]) | 基于数据元素值对数组 *s* 进行排序。索引值被替换为表示新排序顺序的一系列数字。另外，可以指定将新排序后的数组存放在 d 数组中 |
| asorti(*s* [,*d*]) | 基于索引值对数组 *s* 进行排序。产生的数组将索引值包含为数据元素值，并带有表示排序顺序的一系列数字。另外，可以指定将新排序后的数组存放在 *d* 数组中 |
| gensub(*r*, *s*, *h* [, *t*]) | 搜索变量 *$0*，或目标字符串 t(如果指定)以匹配正则表达式 *r*。如果 *h* 是以 *g* 或 *G* 开头的字符串，则以 *s* 替换匹配的文本。如果 *h* 是一个数字，则表示用 *r* 进行替换的次数 |
| gsub(*r*, *s* [,*t*]) | 搜索变量 *$0*，或目标字符串 *t*(如果指定)以匹配正则表达式 *r*。如果找到，则全局替换字符串 *s* |
| index(*s*, *t*) | 返回字符串 *s* 中字符串 *t* 的索引，如果找不到，则返回 *0* |
| length([*s*]) | 返回字符串 *s* 的长度，如果未指定，则返回 *$0* 的长度 |
| match(*s*, *r* [,*a*]) | 在具有正则表达式 *r* 的地方返回字符串 *s* 的索引。如果指定数组 *a*，则包含 *s* 与正则表达式匹配的部分 |
| split(*s*, *a* [,*r*]) | 使用 *FS* 字符或正则表达式(如果已指定)将 *s* 分为数组 *a*。返回字段的个数 |

续表

| 函数 | 描述 |
| --- | --- |
| sprintf(*format*,*variables*) | 返回一个字符串，该字符串与使用指定的 *format* 和 *variables* 的 printf 的输出类似 |
| sub(*r*, *s* [,*t*]) | 搜索变量 *$0* 或目标字符串 *t*，以匹配正则表达式 *r*。如果找到，则将第一个找到的目标替换为字符串 *s* |
| substr(*s*, *i* [,*n*]) | 返回子字符串 *s* 的从索引 *i* 开始的第 *n* 个字符。如果未指定 n，则使用其余的 *s* |
| tolower(*s*) | 将 *s* 中的所有字符转换为小写 |
| toupper(*s*) | 将 *s* 中的所有字符转换为大写 |

有一些字符串函数从名称上就可以知道它的用途：

```
$ gawk 'BEGIN{x = "testing"; print toupper(x); print length(x) }'
TESTING
7
$
```

然而，有一些字符串函数却非常复杂。asort 和 asorti 函数是 gawk 函数的新成员，允许基于数据元素值（asort）或索引值（asorti）对数组变量进行排序。下面的示例使用了 asort：

```
$ gawk 'BEGIN{
> var["a"] = 1
> var["g"] = 2
> var["m"] = 3
> var["u"] = 4
> asort(var, test)
> for (i in test)
>     print "Index:",i," - value:",test[i]
> }'
Index: 4 - value: 4
Index: 1 - value: 1
Index: 2 - value: 2
Index: 3 - value: 3
$
```

新的数组 test 包含原始数组新排序的数据元素，但现在的索引值更改为表示适当顺序的数字值。

split() 函数特别适合将数据字段放到数组中以供后续处理：

```
$ gawk 'BEGIN{ FS=","}{
> split($0, var)
> print var[1], var[5]
> }' data1
data11 data15
data21 data25
data31 data35
$
```

新的数组使用一系列数字作为数组的索引，索引值从第一个数据字段的 1 开始。

### 19.6.3 时间函数

gawk 编程语言包含一些函数来帮助处理时间值，如表 19-6 所示。

表 19-6 gawk 时间函数

| 函数 | 描述 |
| --- | --- |
| mktime(*datespec*) | 将以 YYYY MM DD HH MM SS 格式显示的日期[DST]，转换为时间戳值 |
| strftime(format[, *timestamp*]) | 将日时间戳的当前时间，或时间戳（如果提供），使用 date() shell 函数格式转换为格式化的日和日期 |
| systime() | 返回当前日时间的时间戳 |

时间函数经常用于处理需要进行比较的包含日期的日志文件。通过将日期的文本表示转换为纪元时间（从 1970 年 1 月 1 日开始计算的秒数），可以轻松实现对日期的比较。

下面是在gawk程序中使用时间函数的示例：

```
$ gawk 'BEGIN{
> date = systime()
> day = strftime("%A, %B %d, %Y", date)
> print day
> }'
Friday, December 28, 2007
$
```

此示例使用systime()函数从系统检索当前的纪元时间戳，然后使用strftime()函数将其转换为可供阅读的使用date shell命令的日期格式字符的格式。

## 19.7 用户定义的函数

用户不仅可以使用gawk提供的内置函数，还可以创建自己的函数，以便在gawk程序中使用。本节将介绍如何在gawk程序中使用自己定义的函数。

### 19.7.1 定义函数

要定义自己的函数，必须使用function关键字：

```
function name([variables])
{
    statements
}
```

函数名必须唯一标识函数。可以在调用gawk程序时将一个或多个变量传递给函数：

```
function printthird()
{
    print $3
}
```

此函数将打印记录中的第三个数据字段。

函数还可以使用return语句返回值：

```
return value
```

*value*可以是变量，或计算值的公式：

```
function myrand(limit)
{
    return int(limit * rand())
}
```

在gawk程序中可以将函数返回的值分配给变量：

```
x = myrand(100)
```

变量将包含从函数返回的值。

### 19.7.2 使用自己的函数

在定义函数时，它必须单独出现在定义的任何代码部分（包括BEGIN部分）之前。乍看起来可能有些奇怪，但这可以帮助函数代码独立于gawk程序的其余部分：

```
$ gawk '
> function myprint()
> {
>     printf "%-16s - %s\n", $1, $4
> }
> BEGIN{FS="\n"; RS=""}
> {
>     myprint()
> }' data2
Riley Mullen       - (312)555-1234
Frank Williams     - (317)555-9876
```

```
Haley Snell        - (313)555-4938
$
```

函数定义了 myprint()函数，它格式化记录中的第一个和第四个数据字段以进行打印。然后，gawk 程序使用该函数显示数据文件中的数据。

定义函数后，可以在程序代码中任意多次地使用。在使用较长的算法时，这可以节省许多工作。

### 19.7.3 创建函数库

显然，在每次需要时重新编写 gawk 函数不是一种很好的做法。因此，gawk 提供了一种将函数合并到单个库文件中的方法，可以将此文件用于所有的 gawk 编程工作。

首先，需要创建包含所有 gawk 函数的文件：

```
$ cat funclib
function myprint()
{
   printf "%-16s - %s\n", $1, $4
}

function myrand(limit)
{
   return int(limit * rand())
}

function printthird()
{
   print $3
}
$
```

funclib 文件包含 3 个函数定义。要使用它们，需要使用-f 命令行参数。不幸的是，不能将-f 命令行参数与内嵌的 gawk 脚本结合使用，但可以在同一命令行上使用多个-f 参数。

因此，要使用库，只需要创建包含 gawk 程序的文件，然后在命令行上指定库文件和程序文件：

```
$ cat script4
BEGIN{ FS="\n"; RS=""}
{
     myprint()
}
[rich@test2 ch19]$ gawk -f funclib -f script4 data2
Riley Mullen       - (312)555-1234
Frank Williams     - (317)555-9876
Haley Snell        - (313)555-4938
$
```

现在，需要做的只是在每次需要使用库中定义的函数时将 funclib 文件添加到 gawk 命令行。

## 19.8 小结

在本章中，我们学习了更多有关 gawk 编程语言的高级功能。每种编程语言都需要使用变量，gawk 也不例外。gawk 编程语言包含一些内置变量，可以用它们引用特定的数据字段值、检索关于数据文件中处理的数据字段和记录数目的信息。还可以在脚本中创建自己的变量。

gawk 编程语言还提供了许多编程语言的标准结构化命令。使用 if-then 逻辑、while 循环、do-while 循环和 for 循环，可以轻松创建富有想象力的程序。这些命令允许用户更改 gawk 程序脚本流，以便在数据字段值中递归，从而创建详细的数据报告。

printf 命令非常有利于自定义报告的输出。可以为显示来自 gawk 程序脚本的数据指定精确的格式。可以轻松创建格式化的报告，将数据元素放到精确的位置。

最后，本章讨论了 gawk 编程语言的多种内置函数，并展示了如何创建用户自己的函数。gawk 程序包含许多有用的函数用于处理数学功能，如标准平方根、对数和三角函数等。还有许多与字符串相关的函数，使用这些函数从长字符串中提取子字符串不费吹灰之力。

您可以自由地在 gawk 程序中使用内置函数。如果工作的应用程序使用许多特殊的算法，则可以创建自己的函数来处理这些算法，然后在代码中使用这些自己创建的函数。还可以建立库文件，库文件包含所有在 gawk 程序中使用的函数，从而在编程过程中节省时间和精力。

本书的下一部分将介绍另一主题，介绍在 Linux shell 脚本编写过程中可能使用到的其他 shell 环境。bash shell 是 Linux 中最常用的 shell，但不是唯一的 shell。下一部分有助于您了解其他 shell 以及它们与 bash shell 的区别。

PART

4

# 可选的 Linux shell

本部分内容：

第 20 章
ash shell

第 21 章
tcsh shell

第 22 章
Korn shell

第 23 章
zsh shell

# 第 20 章
# ash shell

本章内容：

- ash shell 简介；
- ash shell 的组成部分；
- dash shell；
- 脚本差异。

前面已经学习了标准的 Linux bash shell，知道了使用标准 bash shell 能够做些什么，现在开始讨论 Linux 世界中可以使用的一些其他 shell。ash shell 是一种低预算 shell，体型娇小，提供基本特性。尤其适合低内存应用，例如嵌入式 Linux 系统。本章将介绍 ash shell 环境，以及在 ash shell 环境中处理脚本需要了解的知识。

## 20.1 什么是 ash shell

要理解 ash shell，首先要了解所使用的 ash shell 版本。ash shell 刚产生时，是对 UNIX 系统上的原始 Bourne shell（参阅第 1 章）的简单复制。Kenneth Almquist 创造了一个用于 UNIX 系统的小型 Bourn shell 版本，称为 Almquist shell，后来缩写为 ash。这个 ash shell 的原始版本极其小，运行很快，但不具有高级功能，如命令行编辑或历史记录特性，所以难于作为交互式 shell 使用。

NetBSD Unix 操作系统曾采用 ash shell，而且至今仍将其作为默认 shell 使用。NetBSD 开发人员对 ash shell 进行定制，添加一些特性，使其更接近 Bourne shell。新特性包括使用 emacs 和 vi 编辑器命令的命令行编辑，重新调用先前输入的命令的历史命令。这一版本的 ash shell 还被 FreeBSD 操作系统作为默认登录 shell 使用。

Debian Linux 发行版拥有自己的 ash shell（称为 Debian ash 或 dash），并包含在该 Linux 系统中。dash 中大部分都是复制的 NetBSD 版的 ash shell 特性，提供高级的命令行编辑功能。

但是，实际上 dash shell 并不是许多基于 Debian 的 Linux 发行版的默认 shell。由于 bash shell 在 Linux 中的盛行，大多数基于 Debian 的 Linux 发行版使用 bash shell 作为常规的登录 shell，而只将 dash shell 作为安装发行版文件的安装脚本的快速启动 shell。

流行的 Ubuntu 发行版是个例外。这通常使 shell 脚本编程人员感到混乱，并使在 Linux 环境中运行 shell 脚本时产生大量问题。Ubuntu Linux 发行版使用 bash shell 作为默认的交互式 shell，但使用 dash shell 作为默认的`/bin/sh` shell。这个“特性”很让 shell 脚本编程人员感到混乱。

如第 8 章所述，每个 shell 脚本都必须在第一行声明脚本使用的 shell。在 bash shell 脚本中，使用的是：

```
#!/bin/bash
```

这告诉 shell 使用位于/bin/bash 下的 shell 程序来执行脚本。在 Unix 世界中，默认的 shell 通常是/bin/sh。许多熟悉 Unix 环境的 shell 脚本编程人员都会将其复制到 Linux shell 脚本中：

```
#!/bin/sh
```

在大多数 Linux 发行版中，/bin/sh 文件都是指向/bin/bash shell 程序的一个软链接（参阅第 3 章）。这样，不必进行修改就能够很容易地将针对 Unix Bourne shell 设计的 shell 脚本移到 Linux 环境下使用。

不幸的是，Ubuntu Linux 发行版将/bin/sh 文件链接到/bin/dash shell 程序。因为 dash shell 只包含原始 Bourne shell 中的一部分命令，所以这样可能（经常）会造成一些 shell 脚本无法正常工作。

本章后面的部分将介绍在 Linux shell 脚本中可能会遇到的各种版本的 ash shell，并介绍在需要使用 ash 或 dash shell 执行 shell 脚本时需要注意的一些事项。

## 20.2　原始 ash shell

在讨论更新更高级的 ash 和 dash shell 之前，最好先了解原始 ash shell。虽然在桌面或服务器 Linux 发行版中可能不会遇到原始 ash shell，但一些嵌入式 Linux 发行版至今仍然在使用 ash shell，所以最好了解这种 shell 的局限性。

### 20.2.1　原始 ash 命令行参数

原始 ash shell 使用命令行参数来控制 shell 的行为。表 20-1 列出了原始 ash shell 中可用的命令行参数。

表 20-1　　原始 ash shell 命令行参数

| 参数 | 描述 |
|---|---|
| -c | 从命令行执行指定的 shell 命令 |
| -e | 执行命令后返回非零值时退出 |
| -f | 关闭文件名生成功能 |
| -I | 忽略文件结束符 |
| -i | 强制交互式 shell |
| -j | 启用 Berkeley 作业控制（使用-i 参数时的默认值） |
| -n | 读取命令，但不执行命令 |
| -s | 从 STDIN 读取命令 |
| -x | 执行命令前先显示命令 |
| -z | 允许文件名生成功能生成零文件 |

可以按任意顺序指定命令行参数，在单短划线后面可以指定一个参数，也可以指定多个参数。

-e 命令行选项可以强制 ash shell 在命令生成错误时立即退出。在调试 shell 脚本时，使用这个特性很方便。

```
$ cat test1
#!/bin/ash -e
# exiting on an error
cp /home/badfile test
echo "Does this display?"
$ ./test1
cp: /home/badfile: No such file or directory
$
```

上面的 cp 命令试图复制一个不存在的文件，所以会生成错误。当 ash shell 检测到该错误时，会立即退出，停止脚本。

-f 命令行选项可以关闭 shell 中的通配符。shell 会将标准通配符视为普通字符处理：

```
$ ash -f
$ ls -al *.txt
ls: *.txt: No such file or directory
$
```

这个例子中，shell 查找一个名为*.txt 的文件，而不是匹配任意以.txt 结束的文件。

-n 命令行选项不用在交互式 shell 中，但如果在 shell 脚本中使用-n 命令行选项，则 ash shell 将扫描脚本，但不执行脚本。下面是一个例子：

```
$ cat test2
#!/bin/ash -n
# test run a script

echo "This is a test"
echo "This is a bad line.
$ ./test2
./test2: 5:Syntax error: Unterminated quoted string
$
```

shell 不会执行正确的 echo 语句，因为错误的 echo 语句中包含一个错误。但是 shell 会指出错误的 echo 语句行中存在一个错误，所以至少可以知道产生问题的地方（错误消息甚至可以指出错误所在的行号）。这个方法可以快速的解决 ash shell 脚本中的编码问题，而不必真正一行行的执行整个脚本。

### 20.2.2 原始 ash 内置命令

因为原始 ash shell 的设计初衷就是小型 shell，所以它只包含一部分 Bourne shell 内置命令。因此关键就在于了解可以使用哪些命令，不可以使用哪些命令。表 20-2 给出了原始 ash shell 中可以使用的内置命令。

表 20-2 原始 ash shell 内置命令

| 命令 | 描述 |
|---|---|
| bg | 在后台模式下继续指定的作业 |
| bltin | 执行指定的内置命令 |
| cd | 改变为指定的目录，如果未指定路径，则切换到$HOME |
| eval | 将提供的字符串解析成一个 shell 命令并执行 |
| exec | 将 shell 进程替换为指定的程序 |
| exit | 终止 shell 进程 |
| export | 导出指定的变量和变量值，以便 shell 子进程可以使用它们 |
| fg | 以前台模式继续指定的作业 |
| getopts | 获取 shell 命令行中指定的选项和值 |
| hash | 显示、删除或使用命令哈希表中的内容 |
| jobid | 显示提供的作业中包含的进程的进程 ID（PID） |
| jobs | 显示当前 shell 进程的后台子进程 |
| lc | 重新执行上一个执行过的命令，或定义一个函数名来执行上一个执行过的命令 |
| pwd | 显示当前的工作目录 |
| read | 读取 STDIN 中的一行数据，将它赋值给一个变量 |
| readonly | 读取 STDIN 中的一行数据，将它赋值给一个不可以修改的变量 |
| set | 列出 shell 变量的值，设置（或清除）选项标志，设置 shell 的位置参数值 |
| setvar | 将指定的值赋给指定的变量 |
| shift | 按指定次数移动位置参数 |
| trap | 接收到指定信号时，解析并执行指定的命令 |
| umask | 设置创建文件和目录时使用的默认权限 |
| unset | 撤销对指定变量或选项的设置 |
| wait | 等待指定作业完成并返回退出状态 |

从表 20-2 中可以看到，ash shell 包含一些已经熟知的 bash 命令，但还缺少很多命令。原始 ash shell

的命令环境中没有提供许多警示铃声功能。它只为程序执行提供一个最简单的环境。当然，正因为这样，它才在低内存操作系统（如嵌入式 Linux 设备）中如此流行。

接下来介绍没有在 bash shell 中学习过的一些基本 ash shell 内置命令。

### 1. bltin 命令

当想要执行一个内置命令，但却存在其他的同名命令时，可以使用 bltin 命令。执行 shell 脚本时，如果有多个同名脚本，只要指定完整路径名称就可以选择正确的脚本。但对于内置命令时，这种方法行不通。

要解决这个问题，ash shell 提供了 bltin 命令（在 bash shell 中名为 builtin）。这个命令可以强制 ash shell 根据指定的名称查找内置命令而不是查找 shell 脚本。

### 2. hash 命令

ash shell 维护一个命令表，表中包含在 shell 进程中输入的所有命令。ash shell 使用这个表（称为**哈希表**）可以快速得到最近使用过的命令的位置，而不必遍历 PATH 环境变量中的所有目录来进行查找。

使用不带任何参数的 hash 命令就可以显示当前的哈希表：

```
$ hash
builtin hash
builtin pwd
/bin/ls
$
```

哈希表中包含命令，起始为最近使用的命令。从示例中可以看到，表中还保存了执行过的内置 shell 命令。

如果使用 cd 命令切换目录，shell 会在哈希表中所有使用路径名称的命令后面放置一个星号：

```
$ cd
$ hash
builtin hash
builtin pwd
builtin cd
/bin/ls*
$
```

星号表示上一次执行该命令后切换了目录，所以路径名称不再可用。

使用-r 参数可以从哈希表中删除某项：

```
$ hash -r /bin/ls
$ hash
builtin hash
builtin pwd
builtin cd
$
```

有时会遇到命令位置已经被更改，但 shell 尚未发觉，仍然试图使用哈希表中指定的位置执行该命令的问题。此时，可以通过删除哈希表中该命令的条目来强制 shell 重新查找该命令。

-v 参数告诉 hash 命令显示找到指定命令的完整路径：

```
$ hash -v cd
builtin cd
$ hash -v ls
/bin/ls
$ hash -v fortune
/usr/games/fortune
$
```

该例还说明，如果命令是内置 shell 命令，那么 hash 命令也会将其标识为内置命令。

### 3. lc 命令

lc 命令是一个很灵巧的工具，它可以减少命令行中的一些输入。它表示**前一个命令**（last command），

它的作用是执行在命令行中执行过的前一个命令，并包含所有命令行参数。下面是一些使用 lc 命令的示例：

```
$ pwd
/home/rich/test
$ lc
/home/rich/test
$ ./test1
Your HOME directory exists
$ lc
Your HOME directory exists
$ ls -al
total 148
drwxr-xr-x     2 rich   rich      4096 Jan 7 08:15 .
drwxr-xr-x    17 rich   rich      4096 Jan 3 10:03 ..
-rwxr--r--     1 rich   rich        97 Jan 3 10:04 test1
-rwxr--r--     1 rich   rich        84 Jan 7 08:01 test2
$ lc
total 148
drwxr-xr-x     2 rich   rich      4096 Jan 7 08:15 .
drwxr-xr-x    17 rich   rich      4096 Jan 3 10:03 ..
-rwxr--r--     1 rich   rich        97 Jan 3 10:04 test1
-rwxr--r--     1 rich   rich        84 Jan 7 08:01 test2
$
```

无论前一个命令是什么类型的命令，内置命令、应用程序还是 shell 脚本，lc 命令都会执行。从例子中可以看到，lc 命令还会复制提供给原命令的所有命令行参数。

### 20.2.3 ash shell 文件

与 bash shell 一样，ash shell 使用默认文件来控制它的启动环境。在默认文件中可以定义交互式模式下使用 ash shell 时的 shell 环境变量和设置。

ash shell 使用的文件有 3 个：

- /etc/profile 文件；
- $HOME/.profile 文件；
- ENV 环境变量中定义的文件。

如果 ash shell 检测到自己是作为登录 shell 使用，那么它首先执行/etc/profile 文件中的内容。该文件用于设置系统范围的环境变量的定义，这些环境变量可以被所有交互用户使用。

接下来，ash shell 搜索用户 HOME 目录下的内容，查找名为.profile 的文件。如果找到该文件，ash shell 执行该文件。最后，ash shell 检查环境变量是否已经设置完毕（检查可以在 ash shell 运行前，也可以作为登录脚本的一部分来进行）。如果已经设置了变量，ash shell 尝试执行这个变量指定的文件名的内容。

## 20.3 dash shell

原始 ash shell 为对原始 Bourne shell 特性进行仿效开辟了很好的开端，但还需要增加很多功能才能成为真正的交互式 shell。NetBSD Unix 克隆开发人员在原始 ash shell 的基础上进行修改，使其可以在 NetBSD Unix 上所用。随后 Debian Linux 发行版开发人员在 NetBSD ash shell 的基础上进行修改，使其可以在 Debian Linux 上运行。本节将介绍 Debian 版本的 ash shell，称为 dash。

### 20.3.1 dash 命令行参数

dash shell 使用命令行参数控制其行为。表 20-3 列出了命令行参数和各参数的功能说明。

表 20-3　　dash 命令行参数

| 参数 | 描述 |
|---|---|
| -a | 导出赋予 shell 的所有变量 |
| -c | 从指定的命令字符串中读取命令 |
| -e | 非交互模式下，当未测试的命令失败时立即退出 |
| -f | 显示路径名称通配符 |
| -n | 非交互模式下，读取命令但不执行命令 |
| -u | 调用未设置的变量时，向 STDERR 写入错误消息 |
| -v | 将读取的输入写入 STDERR |
| -x | 将执行过的每个命令写入 STDERR |
| -I | 在交互模式下，忽略输入中的 EOF 符 |
| -i | 强制 shell 以交互模式运行 |
| -m | 启用作业控制（交互模式下默认是启用的） |
| -s | 从 STDIN 读取命令（未给出文件参数时的默认行为） |
| -E | 启用 emacs 命令行编辑器 |
| -V | 启用 vi 命令行编辑器 |

其中只有几个命令行参数是 Debian 新增到原始 ash shell 命令行参数列表中的。-E 和-V 命令行参数能够启用 dash shell 的特殊命令行编辑特性。

-E 命令行参数允许用户使用 emacs 编辑器命令来编辑命令行文本（参阅第 7 章）。利用 Ctrl 和 Meta 组合键，可以使用所有 emacs 命令来操作某行中的文本。

-V 命令行参数允许用户使用 vi 编辑器命令来编辑命令行文本（参阅第 7 章）。通过使用 Esc 键，该特性能够实现命令行上的常规模式和 vi 编辑模式间的切换。在 vi 编辑器模式下，可以使用所有标准 vi 编辑器命令（例如 x 可以删除一个字符，i 可以插入文本）。完成对命令行的编辑后，必须再次按 Esc 键来退出 vi 编辑器模式。

## 20.3.2　dash 环境变量

在 dash shell 中，有很多用于跟踪信息的默认环境变量，还可以创建自定义环境变量。本节介绍环境变量和 dash 如何使用这些环境变量。

### 1. 默认的环境变量

表 20-4 列出了默认的 dash 环境变量，并给出这些环境命令的使用说明。

表 20-4　　dash shell 环境变量

| 变量 | 描述 |
|---|---|
| CDPATH | cd 命令使用的搜索路径 |
| HISTSIZE | 历史文件中存储的行数 |
| HOME | 用户的默认登录目录 |
| IFS | 输入字段分隔符。默认值为空格、制表符和换行符 |
| MAIL | 用户的邮箱文件名 |
| MAILCHECK | 检查邮箱文件中新邮件的频率 |
| MAILPATH | 由冒号分隔的多个邮箱文件名列表。如果设置该变量，则设置的值会覆盖 MAIL 环境变量 |
| OLDPWD | 先前的工作目录值 |
| PATH | 默认的可执行文件搜索路径 |
| PPID | 当前 shell 的父进程 ID |
| PS1 | shell 命令行界面的主提示 |
| PS2 | shell 命令行界面的次提示 |
| PS4 | 启用执行轨迹时，打印在每一行前面的字符 |
| PWD | 当前的工作目录值 |
| TERM | shell 的默认终端设置 |

您可能已经注意到，dash 环境变量与 bash 中使用的环境变量（参阅第 5 章）非常相似。这并不是偶然的。记住，dash 和 bash shell 都是 Bourne shell 的扩展，因此它们都包含了许多 Bourne shell 特性。但是，由于以简单为目的，dash shell 包含的环境变量比 bash shell 少很多。在 ash shell 环境下创建 shell 脚本时，需要考虑这一点。

dash shell 使用 `set` 命令显示环境变量：

```
$set
COLORTERM=''
DESKTOP_SESSION='default'
DISPLAY=':0.0'
DM_CONTROL='/var/run/xdmctl'
GS_LIB='/home/atest/.fonts'
HOME='/home/atest'
IFS='
'
KDEROOTHOME='/root/.kde'
KDE_FULL_SESSION='true'
KDE_MULTIHEAD='false'
KONSOLE_DCOP='DCOPRef(konsole-5293,konsole)'
KONSOLE_DCOP_SESSION='DCOPRef(konsole-5293,session-1)'
LANG='en_US'
LANGUAGE='en'
LC_ALL='en_US'
LOGNAME='atest'
OPTIND='1'
PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'
PPID='5293'
PS1='$ '
PS2='> '
PS4='+ '
PWD='/home/atest'
SESSION_MANAGER='local/testbox:/tmp/.ICE-unix/5051'
SHELL='/bin/dash'
SHLVL='1'
TERM='xterm'
USER='atest'
XCURSOR_THEME='default'
_='ash'
$
```

由于登录时不同的 Linux 发行版会分配不同的默认环境变量，默认的 dash shell 环境变量很可能会不同。

**2. 位置参数**

除了默认的环境变量，dash shell 还能够将特殊变量赋值给命令行中定义的参数。下面是 dash shell 中可以使用的位置参数。

- `$0`：shell 的名称；
- `$n`：第 n 个参数；
- `$*`：该值包含所有参数内容，由 IFS 环境变量中的第一个字符分隔，若未定义 IFS 则使用空格分隔；
- `$@`：扩展为多个参数，包括所有命令行参数；
- `$#`：位置参数的数量；
- `$?`：最近命令的退出状态；
- `$-`：当前选项标志；
- `$$`：当前 shell 的进程 ID（PID）；
- `$!`：最近后台命令的进程 ID（PID）。

所有 dash 位置参数都是仿效 bash shell 中的相同位置参数：

```
$ cat test3
#!/bin/dash
# testing positional parameters
number=$#
pid=$$
third=$3
echo "Passed $number parameters"
echo "The third parameter is $third"
echo "The PID is $pid"
$ ./test3 one two three
Passed 3 parameters
The third parameter is three
The PID is 10095
$
```

可以像在 bash shell 中一样，在 shell 脚本中使用每个位置参数。

**3. 用户定义环境变量**

dash shell 还允许设置自定义环境变量。与 bash 一样，可以使用赋值语句在命令行中定义新环境变量：

```
$ testing=10
$ echo $testing
10
$
```

默认情况下，环境变量只在定义它们的 shell 会话中可见。要想使环境变量在子 shell 或子进程中也可见，必须使用 export 命令：

```
$ testing=10 ; export testing
$ dash
$ echo $testing
10
$
```

如果没有 export 命令，用户定义环境变量只能在当前 shell 或进程中可见。

警告：dash 变量与 bash 变量有一个很大的不同。dash shell 不支持变量数组。这个小小的特性可能会给高级 shell 脚本编程人员带来各种问题。

## 20.3.3　dash 内置命令

与 bash shell 一样，dash shell 包含一组可识别的内置命令。可以直接在命令行界面中使用这些命令，也可以把它们结合到 shell 脚本中。表 20-5 列出了 dash shell 内置命令。

表 20-5　　dash shell 内置命令

| 命令 | 描述 |
|---|---|
| alias | 创建一个别名来表示一个文本字符串 |
| bg | 以后台模式继续指定的作业 |
| cd | 改变为指定的目录 |
| echo | 显示文本字符串和环境变量 |
| eval | 使用空格连接所有参数 |
| exec | 用指定的命令替换 shell 进程 |
| exit | 终止 shell 进程 |
| export | 导出指定的环境变量，供子 shell 使用 |
| fc | 列出、编辑或重新执行命令行中输入的上一个命令 |
| fg | 以前台模式继续指定作业 |
| getopts | 从参数列表中获取选项和参数 |
| hash | 维护和检索一个近期命令及其位置的哈希表 |
| pwd | 显示当前工作目录值 |

续表

| 命令 | 描述 |
|---|---|
| read | 从 STDIN 读取一行，将读取的值赋值给一个变量 |
| readonly | 从 STDIN 读取一行，并赋值给一个不可变的变量 |
| printf | 使用格式化字符串显示文本和变量 |
| set | 列出或设置选项标志和环境变量 |
| shift | 将位置参数移动指定的次数 |
| test | 求表达式的值，如果值为真返回 0，如果值为假返回 1 |
| times | 显示当前 shell 和所有 shell 进程的所积累的用户和系统次数 |
| trap | shell 收到指定信号时，解析并执行一个动作 |
| type | 解析指定名称的类型，并显示结果（别名、内置、命令或关键字） |
| ulimit | 查询或设置进程限制 |
| umask | 设置默认的文件和目录权限值 |
| unalias | 删除指定的别名 |
| unset | 删除并撤消导出指定变量或选项标志 |
| wait | 等待指定作业的完成并返回退出状态 |

您可能已经认出了所有在 bash shell 中包含的那些内置命令。dash shell 支持的很多内置命令和 bash shell 相同。注意，dash shell 中没有关于命令历史文件和目录堆栈的命令。dash shell 不支持这些特性。

接下来的两节将介绍 dash shell 中存在，但 ash shell 中不存在的的一些命令。

### 1. printf 命令

echo 命令不是显示命令行中或 shell 脚本中数据的唯一方法。第 10 章介绍过如何在 gawk 程序中使用 C 编程语言的 printf 命令。

dash shell 也结合了 printf 内置命令，可以自定义输出。printf 命令使用特殊的格式标签，对输出中数据的显示格式和位置提供更强的控制。

printf 命令的格式如下：

```
printf "format string" var1 var2 ...
```

注意，dash 版中的 printf 命令在格式字符串和变量间不使用逗号。与 gawk 中相同，格式字符串是由 3 个元素组成的字符串值：

- ✦ 标准文本字符；
- ✦ 格式规范；
- ✦ 字符转义序列。

printf 命令使用单独的**格式规范**（format specification）来显示命令行中的每个参数。格式规范定义数据的显示类型和显示方式。dash 的 printf 命令使用与 gawk printf 命令相同的格式规范（参阅第 19 章）。

下面是一个在 dash shell 脚本中使用 printf 命令的例子：

```
$ cat test4
#!/bin/dash
# testing the printf command

factorial=1
value=1
while [ $value -le 10 ]
do
   count=1
   while [ $count -le $value ]
   do
      factorial=$(( $factorial * $count ))
      count=$(( $count + 1 ))
   done
```

```
    printf "The factorial of %d is %6.2e\n" $value $factorial
    factorial=1
    value=$(( $value + 1 ))
done
```

上面的 printf 命令允许格式化两个变量的输出，本例使用科学记数法显示阶乘的值：

```
$ ./test4
The factorial of 1 is 1.00e+00
The factorial of 2 is 2.00e+00
The factorial of 3 is 6.00e+00
The factorial of 4 is 2.40e+01
The factorial of 5 is 1.20e+02
The factorial of 6 is 7.20e+02
The factorial of 7 is 5.04e+03
The factorial of 8 is 4.03e+04
The factorial of 9 is 3.63e+05
The factorial of 10 is 3.63e+06
$
```

格式%6.2e 表示使用六位科学记数值，其中小数点后面有两位。

### 2. ulimit 命令

ulimit 命令是 dash shell 中很有用的一个命令。它可以限制系统资源，以避免在 shell 中过度使用。如果 shell 中运行了 ulimit 命令，那么就会对该 shell 实施限制。

dash shell 中可以使用下面两种类型的限制：

- 强限制，只有根用户账户可以重置。
- 弱限制，shell 的所有者用户账户可以对其进行重置。

如果 shell 违反了强限制，就会产生错误，阻塞操作。这个工具对于防止系统用户过度使用系统资源非常有效。

ulimit 命令使用一些命令行参数来定义系统资源限制。如表 20-6 所示。

表 20-6　　ulimit 命令行参数

| 参数 | 描述 |
|---|---|
| -a | 当前起作用的限制 |
| -c | 核心存储的最大大小（单位为 KB） |
| -d | 进程数据段的最大大小（单位为 KB） |
| -f | 文件块的最大大小（单位为 512 字节） |
| -l | 可以锁定的最大内存量（单位为 KB） |
| -m | 可以使用的最大物理内存量（单位为 KB） |
| -n | 进程可以同时打开的文件数 |
| -p | 用户可以同时激活的进程数 |
| -s | 进程的最大堆栈大小 |
| -t | CPU 时间限制 |
| -H | 显示或设置强限制 |
| -S | 显示或设置弱限制 |

要显示当前设置的所有限制，使用-a 参数：

```
$ ulimit -a
time(seconds)          unlimited
file(blocks)           unlimited
data(kbytes)           unlimited
stack(kbytes)          8192
coredump(blocks)       0
memory(kbytes)         unlimited
locked memory(kbytes)  unlimited
process                unlimited
nofiles                1024
```

```
vmemory(kbytes)        unlimited
locks                  unlimited
$
```

要设置某一限制，只要将表示限制数量的数字加在正确的参数后面即可：

```
$ ulimit -f 2
$
```

上面的例子对进程中新文件可以包含的块数设置了一个弱限制。如果要创建大于限制的文件，shell 将会阻塞操作，并显示警告：

```
$ cat data1 >> data2
File size limit exceeded
$
```

如果想要为每个用户设置登录 shell 限制，可以在/etc/profile 文件中使用 ulimit 命令，dash shell 会在每次登录时执行这个文件。这是个限制用户使用系统资源的好办法。

## 20.4 在 dash 中编写脚本

不幸的是，dash shell 并不识别 bash shell 的所有编写脚本特性。将针对 bash 环境编写的 shell 脚本放到 ash 或 dash shell 环境中执行时经常会出错。本节将介绍为了使 shell 脚本能够在 ash 或 dash shell 环境中正确执行需要注意的一些区别。

### 20.4.1 创建 ash 和 dash 脚本

您现在可能已经猜到为 ash shell 创建脚本同为 bash shell 创建脚本非常相似。必须在脚本中指定想要使用的 shell 类型，来保证脚本使用正确的 shell 执行。

在 shell 脚本的第一行中指定想要使用的 shell：

```
#!/bin/ash
```

或

```
#!/bin/dash
```

在这一行中还可以指定一个 shell 命令行参数，如前面“原始 ash shell”一节的例子中指定了-e 命令行参数。

### 20.4.2 失效的特性

不幸的是，由于 ash 和 dash shell 只是 Bourne shell 特性的子集，所以 bash shell 脚本中的一些特性在 ash 或 dash shell 中将会失效。这些特性通常称为 bashisms(bash 专有特性)。本节将对可以在 bash shell 中使用但在 ash 或 dash shell 环境中将会失效的 bash shell 特性进行简要概括。

**1. 数学运算**

第 8 章介绍过 3 种在 bash shell 脚本中表示数学运算的方法。

- 使用 expr 命令：expr operation;
- 使用方括号：$[ operation ];
- 使用双圆括号：$(( operation ))。

ash shell 和 dash shell 支持 expr 命令和双圆括号方式，但不支持方括号方法。如果脚本中有大量的数学运算使用了方括号，就会产生错误：

```
$ cat test5
#!/bin/dash
# testing mathematical operations

value1=10
value2=15
```

```
value3=$[ $value1 * $value2 ]
echo "The answer is $value3"
$ ./test5
./test5: 7: value1: not found
The answer is
$
```

在 ash shell 或 dash shell 脚本中处理数学运算的正确格式是使用双圆括号方法：

```
$ cat test5b
#!/bin/dash
# testing mathematical operations

value1=10
value2=15

value3=$(( $value1 * $value2 ))
echo "The answer is $value3"
$ ./test5b
The answer is 150
$
```

现在 shell 可以正确的进行计算处理了。

### 2. test 命令

dash shell 支持 test 命令，但必须谨慎使用。bash shell 版的 test 命令和 dash shell 版的 test 命令略有不同。

bash shell 的 test 命令允许使用双等于号（==）来测试两个字符串是否相等。这对于已经熟悉在其他编程语言中使用这种格式的编程人员来说是个好事：

```
$ cat test6
#!/bin/bash
# testing the == comparison

test1=abcdef
test2=abcdef

if [ $test1 == $test2 ]
then
   echo "They're the same!"
else
   echo "They're different"
fi
$ ./test6
They're the same!
$
```

非常简单。但是，如果在 ash 或 dash shell 环境中执行这段脚本，就不会得到想要的结果：

```
$ ./test6
[: ==: unexpected operator
They're different
$
```

ash 和 dash shell 中的 test 命令不能识别= =符号以进行文本比较。它只能识别=符号。如果将文本比较符号改成单等于号，在 ash、bash 和 dash shell 环境中都会顺利执行：

```
$ cat test7
#!/bin/dash
# testing the = comparison

test1=abcdef
test2=abcdef

if [ $test1 = $test2 ]
then
   echo "They're the same!"
```

```
else
   echo "They're different"
fi
$ ./test7
They're the same!
$
```

就这个小小的 bash 专有特性，有可能是造成 shell 编程人员数小时停止不前的症结所在！

### 3. echo 语句选项

简单的 echo 语句也是阻碍 dash shell 编程人员的另一个祸源。在 dash 和 bash shell 中它的行为不同。在 bash shell 中，如果要在输出中显示特殊字符，必须使用-e 命令行参数：

```
echo -e "This line contains\ta special character"
```

上面的 echo 语句包含特殊字符\t，表示一个制表符。如果不使用-e 命令行参数，bash 版的 echo 语句将忽略该特殊字符。下面 bash shell 中的例子说明了这一点：

```
$ cat test8
#!/bin/bash
# testing echo commands

echo "This is a normal test"
echo "This test uses a\tspecial character"
echo -e "This test uses a\tspecial character"
echo -n "Does this work: "
read test
echo "This is the end of the test"
$ ./test8
This is a normal test
This test uses a\tspecial character
This test uses a special character
Does this work: N
This is the end of the test
$
```

未使用-e 命令行参数的 echo 语句只将\t 字符作为普通文本显示。所以要想显示为制表符，必须使用-e 命令行参数。

在 ash shell 和 dash shell 中，情况是不一样的。ash 和 dash shell 中的 echo 语句自动识别和显示特殊字符。因此，没有-e 命令行参数。如果要在 ash 或 dash 环境中执行同一脚本，会得到下面的输出：

```
$ ./test8
This is a simple test
This line uses a        special character
-e This line uses a     special character
Does this work: N
This is the end of the test
$
```

从输出中可以看到，dash shell 版的 echo 命令能够正确识别不包含-e 命令行参数的行中的特殊字符，但对于包含-e 命令行参数的行，就会产生混淆，将-e 作为普通文本显示。



警告：-n 命令行参数也是不可靠的。bash 中支持-n 命令行参数，用来抑制行尾的换行符，但所有版本的 ash shell 都不支持-n。dash shell 的 echo 语句支持-n 参数，但不是所有版本的 ash shell 都支持。

不幸的是，没有任何简单的方法能够解决这一问题。如果必须编写既可以在 bash 又可以在 ash 环境中执行的脚本，最好的方案就是使用 printf 命令来显示文本。这个命令在两种 shell 环境中的工作方式相同，也可以很好的显示特殊字符。

### 4. 函数命令

第 14 章中介绍过如何在 shell 脚本中自定义函数。bash shell 支持两种函数定义方法。第一种方法是使用函数语句：

```
function name {
    commands
}
```

name 属性定义赋予函数的唯一名称。脚本中定义的每个函数的名称都必须是唯一的。

commands 是组成函数的一个或多个 bash shell 命令。调用这个函数时，bash shell 按函数中命令的出现顺序执行命令，就同执行普通脚本一样。

在 bash shell 脚本中定义函数的第二种格式更接近于其他编程语言中的函数定义格式：

```
name() {
commands
}
```

函数名后面的空圆括号表示这是个函数定义。这种格式中采用的命名规则和原始 shell 脚本函数格式相同。

ash shell 和 dash shell 不支持第一种函数定义方法（它们都不支持函数语句）。在 ash 和 dash shell 中，必须使用带圆括号的函数名来定义函数。如果要在 ash 或 dash shell 中执行如下所示的为 bash 编写的函数，会得到错误消息：

```
$ cat test9
#!/bin/dash
# testing functions

function func1() {
   echo "This is an example of a function"
}

count=1
while [ $count -le 5 ]
do
   func1
   count=$(( $count + 1 ))
done
echo "This is the end of the loop"
func1
echo "This is the end of the script"
$ ./test9
./test9: 4: Syntax error: "(" unexpected
$
$
```

dash shell 不是将函数代码赋予函数，而是执行函数定义中的代码，然后抱怨 shell 脚本的格式。

如果编写的 shell 脚本可能会用在 ash 或 dash 环境中，就必须使用第二种方法定义函数：

```
$ cat test10
#!/bin/dash
# testing functions

func1() {
   echo "This is an example of a function"
}

count=1
while [ $count -le 5 ]
do
   func1
   count=$(( $count + 1 ))
done
echo "This is the end of the loop"
func1
echo "This is the end of the script"
$ ./test10
This is an example of a function
```

```
This is an example of a function
This is an example of a function
This is an example of a function
This is an example of a function
This is the end of the loop
This is an example of a function
This is the end of the script
$
```

现在 dash shell 可以正确识别脚本中定义的函数，可以在脚本中使用函数了。

## 20.5 小结

本章讨论了 ash shell 环境中的有关事项。ash shell 不是 Linux 发行版中的常用 shell，但 NetBSD 和 FreeBSD Unix 发行版都使用它。ash shell 的一个派生 shell 是 dash shell，应用在许多基于 Debian 的 Linux 发行版中，尽管不是登录 shell。

ash shell 和 dash shell 是 Bourne shell 的小型版本，所以它们只支持 bash shell 的部分特性。如果编写可能会用在 ash 或 dash shell 环境中的 shell 脚本，需要考虑到这一点。

下一章将讨论可能会遇到另一种 Linux shell 环境。C shell 在一些 Unix 环境中很流行，在 Linux 世界中它以 tcsh 的形式出现。下一章将学习 tcsh shell 与 bash shell 的区别，以及使脚本能在 tcsh shell 环境中运行需要了解的知识。

CHAPTER 21

# 第21章 tcsh shell

本章内容：

- tcsh shell的历史；
- tcsh shell的组成部分；
- 使用tcsh shell。

Linux世界中另一种流行的shell是C shell。C shell将C编程语言中的特性合并到shell脚本中。最流行的开源C shell是tcsh shell。本章将讨论tcsh shell，说明tcsh shell的特性并介绍如何编写tcsh shell脚本。

## 21.1 什么是tcsh shell

C shell是加利福尼亚伯克利大学的Bill Joy开发的，用来替换AT&T实验室创建的原始Unix shell（在Bourne shell出现之前）。伯克利的开发人员设计并创建了一个Unix系统同AT&T Unix竞争，他们选择C shell作为默认shell。这个版本的Unix通常被称为Berkeley Software Distribution（或BSD Unix）。C shell的目的是提供一个适合C编程人员的命令行和脚本编写环境。

20世纪70年代后期Ken Greer对C shell进行扩展，添加了TENEX操作系统中的命令行编程特性。这就是tcsh名字的由来。tcsh shell已经成为许多以BSD Unix为基础的Unix系统（包括早期版本的Mac OS X操作系统；现在较新版本使用bash shell作为默认shell）的流行shell。

虽然bash shell几乎已成为所有Linux发行版的默认shell，但tcsh shell也是一种可选的shell，如果需要可以直接通过源代码很容易的安装tcsh shell。如果要将BSD Unix环境下的shell脚本迁移到Linux环境，使用tcsh shell是个不错的选择。

因为C shell是在Bourne shell之前创建的，所以二者之间有很大差异。又因为bash是基于Bourne shell的，所以tcsh和bash shell间的区别非常大。编写可以在两种shell环境中完全通用的脚本几乎是不可能的，但对于最简单的应用还是可以实现的。

接下来的各节将介绍tcsh shell的组件和特性，并介绍如何编写tcsh环境下的shell脚本。

## 21.2 tcash shell的组成部分

与bash shell一样，tcsh shell使用命令行参数、默认文件、

环境变量和内置命令来定义 shell 操作环境。本节将分别介绍 tcsh shell 的这些特性。

## 21.2.1 tcsh 命令行参数

tcsh shell 提供了几个命令行参数，使用这些参数可以自定义 shell 操作环境。这些命令行参数定义交互环境中可用的 shell 特性和 shell 的行为。表 21-1 列出了 tcsh shell 可用的命令行参数。

表 21-1 tcsh shell 命令行参数

| 参数 | 描述 |
|---|---|
| -b | 暂停选项参数，强制将剩余的所有参数视为非选项参数 |
| -c | 从指定参数中读取命令 |
| -d | shell 从文件$HOME/.cshdirs 中载入目录栈 |
| -e | 如果命令非正常终止或者以非 0 退出状态终止，那么 shell 退出 |
| -f | shell 不处理$HOME/.tcshrc 文件 |
| -i | 强制交互式 shell 会话 |
| -l | 指定某 shell 用作登录 shell |
| -m | 强制 shell 执行$HOME/.tcshrc 文件，即使该文件不属于有效用户 |
| -n | shell 解析命令但不执行命令 |
| -q | shell 接收 SIGQUIT 信号，并关闭作业控制 |
| -s | 从 STDIN 读取 shell 命令 |
| -t | shell 读取并执行输入的单行 |
| -v | 设置 shell 变量 verbose，使得使用历史置换时显示命令输入 |
| -V | 在执行$HOME/.tcshrc 文件前设置 shell 变量 verbose |
| -x | 设置 shell 变量 echo，使得在执行命令前即刻显示命令 |
| -X | 在执行$HOME/.tcshrc 文件前设置 shell 变量 echo |
| --help | 在 STDOUT 中显示帮助消息并退出 |
| --version | 显示 tcsh shell 的版本信息 |

tcsh shell 使用的一些命令行参数与 bash shell 相同，但还有一些是独有的。

在解决 tcsh shell 中的一些问题时，-v 和-V 参数会很好用。它们可以使 shell 在执行命令时显示命令。

-V 参数使得 shell 在启动时就开始显示命令，包括登录和启动 shell 文件（参阅“tcsh 文件”一节）。而-v 参数在启动和登录文件完成之后才显示命令。

对于命令行这不是非常有用，但处理 shell 脚本时非常酷：

```
$ cat test1
#!/bin/tcsh -v
# test run a script

set testing=10
echo "This is a test"
echo "The value of testing is $testing"
$ ./test1
set testing=10
echo "This is a test"
This is a test
echo "The value of testing is $testing"
The value of testing is 10
$
```

在脚本中定义 tcsh shell 时使用-v 参数可以使 shell 在执行语句前显示每个语句。

## 21.2.2 tcsh 文件

tcsh shell 自动搜索一些默认文件来设置环境变量并执行命令。如果存在某一默认文件，tcsh shell 会根据具体情况执行这一文件。tcsh shell 可以处理 3 种默认文件：

- 登录文件；
- shell 启动文件；
- 退出文件。

本节将探讨可能为 tcsh shell 使用的各种文件。

### 21.2.3　tcsh 登录文件

如果使用 tcsh shell 作为登录 shell，它首先搜索并执行以下文件：

- /etc/csh.login；
- /etc/csh/login。

注意，这两个文件的文件名中都使用了 csh。因为设计 tcsh shell 的目的是为了替代原始 C shell，所以将它设计为能够处理 C shell 可以处理的所有默认文件。这使得可以实现从 C shell 环境向 tcsh shell 环境进行迁移。

对于所有使用 tcsh shell 作为登录 shell 的用户将会执行这两个文件。这两个文件中应当包含为所有用户而设置的系统范围环境变量，和所有用户都可以使用的可执行程序（例如设置默认的文件和目录权限的 umask 命令）。如果使用的 Linux 发行版支持 tcsh shell，那么极有可能已经存在这两个设置登录系统环境变量的文件之一。

执行完这两个文件的任意一个后，tcsh shell 搜索每个用户的$HOME 目录下的一个文件：

```
$HOME/.login
```

该文件中可以包含用户特定的环境变量设置，以及用户登录系统时在命令行界面提示出现前要执行的可执行命令。

$HOME/.login 文件中的环境变量设置将覆盖/etc/csh.login 文件中的系统范围设置。可以为每个文件添加一个简单的 echo 语句，然后登录，来测试 tcsh shell 执行这些文件的顺序。

```
$ telnet localhost
Trying 127.0.0.1...
Connected to localhost.localdomain.
Escape character is '^]'.
Welcome to testbox
login: test
Password:
Last login: Fri Jan 11 18:54:41 from localhost.localdomain
This is the /etc/csh.login file
This is the .login file
$
```

从这个例子中可以看到 tcsh shell 先执行/etc/csh.log 文件。

### 21.2.4　shell 启动文件

如果不使用 tcsh shell 作为登录 shell，而只作为交互式 shell，那么当 shell 启动时必须处理不同的文件集。

默认的系统范围启动文件为：

```
*/etc/csh.cshrc
```

执行这个文件（如果存在的话）后，tcsh shell 遍历用户的$HOME 目录，搜索以下文件中的一个：

- $HOME/.cshrc；
- $HOME/.tcshrc。

如果两个文件都存在，将会只执行.tcshrc 文件。可以通过 echo 语句执行简单测试来测试 tcsh shell 执行这些文件的顺序：

```
$ tcsh
This is the /etc/csh.cshrc file
This is the /home/test/.tcshrc file
$
```

当启动一个新 tcsh shell 时，首先执行/etc/csh.cshrc 文件，然后执行$HOME 目录下的.tcshrc

文件。

如果 shell 脚本使用 tcsh shell，也会执行 shell 启动文件：

```
$ cat test2
#!/bin/tcsh
# test run a script
echo "This is a test"
$ ./test2
This is the /etc/csh/cshrc file
This is the /home/test/.tcshrc file
This is a test
$
```

注意在出现 shell 脚本的输出前，可以看到 tcsh shell 启动文件中的 echo 语句。

启动文件不会替换登录文件，而是对登录文件进行增补。如果两种文件都设置了的话，那么当使用新登录 shell 登录时，不仅执行登录 shell 文件，还执行启动 shell 文件：

```
$ telnet localhost
Trying 127.0.0.1...
Connected to localhost.localdomain.
Escape character is '^]'.
Welcome to testbox
login: test
Password:
Last login: Fri Jan 11 18:55:26 from localhost.localdomain
This is the /etc/csh.cshrc file
This is the /etc/csh.login file
This is the /home/test/.tcshrc file
This is the .login file
$
```

注意 shell 执行默认文件的顺序：

（1）所有用户的启动文件。

（2）所有用户的登录文件。

（3）私有启动文件。

（4）私有登录文件。

如果要设置环境变量，那么记住这个顺序非常重要，因为如果这些文件中重复设置了同一个环境变量的值，只有最后一个值有效。

### 21.2.5 退出文件

除了登录和启动文件，tcsh shell 还具有在退出交互式或登录 shell 会话时执行文件中的命令的能力。这些文件可能位于以下位置：

- /etc/csh.logout；
- $HOME/.logout。

现在您可能已经发现，系统上的每个用户在退出时都执行/etc/csh.logout 文件中的命令，同时每个用户都有一个唯一的$HOME/.logout 文件。tcsh shell 将在退出当前 shell 前执行这些文件中的所有命令。

这不包括只从交互式 shell 退出的情况。因此，如果从现有 shell 会话中启动另一个 tcsh shell，那么退出这个 shell 时 tcsh shell 将不会执行退出文件。

要测试该特性，不能只使用 echo 语句显示文本，因为 shell 将很可能快速结束，人眼很难看到。但是，可以将输出重定向到文件，然后在退出后查看文件。

下面是我退出 tcsh shell 会话后的重定向输出文件的内容：

```
$ cat logout.txt
This is the /etc/csh.logout file
This is the /home/test/.logout file
$
```

tcsh shell 先执行/etc/csh.logout 文件中的命令，然后执行$HOME/.logout 文件中的命令。

### 21.2.6 tcsh 环境变量

tcsh shell 环境变量可能有点混淆。Bourne shell（及其派生的 bash，ash 和 dash）只使用一种环境变量来存储系统信息。tcsh shell 包含两种环境变量：

- ✦ shell 变量；
- ✦ 系统环境变量。

系统环境变量由大写字符串组成，提供标准系统信息。tcsh shell 变量是小写变量，对 shell 具有特殊含义。

### 21.2.7 shell 变量

表 21-2 给出了 tcsh 使用的设置 shell 中行为特性的 shell 变量。

表 21-2 tcsh 特有的 shell 变量

| 变量 | 描述 |
|---|---|
| addsuffix | 如果设置（默认设置），那么文件名完成会在目录的结尾添加一个正斜杠（/），在普通文件名结尾添加空格 |
| afsuser | 如果设置，则 autologout 使用该值替代本地用户名进行 Kerberos 识别 |
| ampm | 如果设置，则将以 12 时 AM/PM 格式显示时间 |
| argv | 提供给 shell 的参数 |
| autocorrect | shell 在每个命令前自动调用单词拼写编辑器 |
| autoexpand | 如果设置，则 shell 在每个命令前自动调用历史扩展编辑器 |
| autolist | 如果设置，则 shell 在不明确的命令后面列出可能的命令 |
| autologout | 设置在自动退出前休止状态的分钟数。第二个参数为可选参数，指定锁定终端前休止的分钟数 |
| backslash_quote | 如果设置，则 shell 总是使用反斜杠引导特殊字符（反斜杠，单引号和双引号） |
| catalog | 消息目录的文件名 |
| cdpath | cd 命令将要搜索的子目录的目录列表 |
| color | 如果设置，则为某些命令启用彩色显示 |
| colorcat | 如果设置，则启用消息颜色转义序列 |
| command | 如果设置，则使用-c 命令行参数将命令传递给 shell |
| complete | 控制命令完成。如果设置为 enhance，则完成忽略大小写和考虑特殊字符。如果设置为 igncase，则完成只忽略大小写 |
| continue | 如果设置给某命令列表，则 shell 将继续该命令列表而不启动新列表 |
| continue_args | 同 continue，但 shell 传递参数 |
| correct | 如果设置为 cmd，会自动纠正命令的拼写。如果设置为 complete，会自动完成命令。如果设置为 all，会纠正整个命令行 |
| csubstnonl | 如果设置，则将命令中的换行符和回车替换为空格 |
| cwd | 当前目录的完整路径 |
| dextract | 如果设置，则 pushd 命令从目录栈中抽取第 n 个记录，而不将该记录翻转至顶部 |
| dirsfile | dirs 命令搜索历史文件的默认位置 |
| dirstack | 目录堆栈上包含所有目录的数组 |
| dspmbyte | 控制多字节字符集的编辑 |
| dunique | 如果设置，则 pushd 命令移除堆栈中指定目录名称的所有实例 |
| echo | 如果设置，则 shell 在执行前显示每个命令 |
| echo_style | 定义 echo 命令的风格（bsd、sysv、两者或没有） |
| edit | 如果设置，则使用命令行编辑器 |
| ellipsis | 如果设置，则提示中的%c、%和%C 提示序列表示使用…（省略号）跳过的目录 |
| fignore | 列出完成忽略的文件名称后缀 |
| filec | 如果没有设置 edit 变量，则使用传统的 csh shell 命令完成 |
| gid | 用户的读取组 ID |

续表

| 变量 | 描述 |
| --- | --- |
| group | 用户的组名称 |
| histchars | 决定历史置换中使用的字符的字符串值 |
| histdup | 控制对历史文件中重复记录的处理（all、prev、erase） |
| histfile | 历史文件的默认位置 |
| histlit | 如果设置，则在历史列表中的内置命令、编辑器命令和 savehist 使用行的非扩展形式 |
| history | 第一个记录表示存储的历史项目的数量。可选的第二个记录表示历史的打印格式 |
| home | 用户的主目录 |
| ignoreeof | 如果设置为空字符串或 0，则按 Ctrl-D 组合键时不会退出 shell |
| implicitcd | 如果设置，则在命令行中输入一个目录名称的效果就像是使用 cd 命令后接目录名称一样 |
| inputmode | 在每行的开始将编辑器设置为插入模式或重写模式 |
| killdup | 控制删除环中的重复记录。可以设置为 all、prev 或 erase |
| killring | 保存在内存中的删除的字符串数量（默认设置为 30） |
| listflags | 添加到 ls-F 命令的标志列表（可以是 x、a 或 A） |
| listjobs | 如果设置，则当某个作业挂起时，列出所有作业。如果设置为 long，则使用长格式 |
| listlinks | 如果设置，则 ls-F 命令显示所有符号链接指向的文件的类型 |
| listmax | list-choices 编辑器命令列出的项目的最大数量 |
| listmaxrows | list-choices 编辑器命令列出的行的最大数量 |
| loginsh | 如果 shell 为登录 shell，则 shell 会设置该变量 |
| logout | shell 设置其值为 normal、automatic 或 hangup |
| mail | 检验新邮件时用到的文件名或目录（由空格分隔） |
| matchbeep | 设置完成匹配时产生蜂鸣器鸣叫的条件，设置为 never、nomatch、ambiguous 或 notunique |
| nobeep | 禁用蜂鸣器 |
| noclobber | 设置该变量，可以防止因重定向而自动重写文件 |
| noding | 如果设置，则当小时改变时，不在提示时间中显示 DING! |
| noglob | 如果设置，则禁用文件名替换和目录栈替换 |
| nokanji | 如果设置而且 shell 支持 Kanji，则会禁用 Kanji，使得可以使用元键 |
| nonomatch | 如果设置，则当文件名替换或目录栈替换不匹配时不生成错误 |
| nostat | 在完成操作期间，不包含在 stat 命令中的目录列表 |
| notify | 如果设置，则 shell 异步通知作业完成 |
| oid | 用户的真实组织 ID |
| owd | 旧工作目录 |
| path | 用于搜索可执行命令的目录列表 |
| printexitvalue | 如果在交互式 shell 中设置，则 shell 显示非零退出状态 |
| prompt | 打印在每个命令行前面的字符串 |
| prompt2 | 用于命令完成的次命令行提示 |
| prompt3 | 用于自动拼写校正的字符串 |
| promptchars | 两个字符的字符串。如果设置，则第一个字符用于普通用户提示，第二个字符用于根用户提示 |
| pushdtohome | 如果设置，则 pushd 命令默认推到$HOME 目录 |
| pushdsilent | 如果设置，则 pushd 和 popd 命令不打印目录栈 |
| recexact | 如果设置，则完成以精确匹配结束，即使可能会产生较长的匹配 |
| recognize_only_executables | 如果设置，则命令清单中只显示路径下的可执行文件 |
| rmstar | 如果设置，则在执行 rm*命令前会提示用户 |
| rprompt | 在屏幕左侧显示提示时，显示在屏幕右侧的字符串 |
| savedirs | 如果设置，则 shell 在退出前执行一个 dirs -S 命令，保存当前的 dirs 列表 |
| savehist | 如果设置，则 shell 在退出前执行一个 history -S 命令，保存当前的历史列表 |
| sched | sched 命令显示预定事件的格式 |
| shell | shell 程序的文件名 |

续表

| 变量 | 描述 |
|---|---|
| shlvl | 嵌套 shell 数量 |
| status | 最后一个命令返回的退出状态 |
| symlinks | 控制符号链接方式。可以设置为 chase、ignore 或 expand |
| tcsh | tcsh shell 的版本号 |
| term | 终端类型 |
| time | 如果设置一个数字，则当命令的执行时间超过指定的 CPU 秒数时，自动执行 time 命令 |
| tperiod | 周期性执行的特定别名的时间周期，单位为分钟 |
| tty | shell 附加到的当前 TTY 设备的名称 |
| uid | 用户的真实用户 ID |
| user | 用户的登录名 |
| verbose | 如果设置，则 shell 在历史置换后显示每个命令 |
| version | 显示 tcsh shell 版本 |
| visiblebell | 如果设置，则使用闪屏代替蜂鸣器鸣叫 |
| watch | 设置用户/终端对列表，供登录和退出使用（可以使用任意关键字）。报告用户或终端是否被使用 |
| who | 用于查看消息的格式字符串 |
| wordchars | 编辑器命令将其视为单词组成部分的非文字和数字字符列表 |

tcsh shell 的可用 shell 变量很多。这些变量展示了 tcsh shell 的强大功能。可以通过设置 shell 变量来自定义许多不同的 shell 功能。

要显示 shell 中当前设置的 shell 变量，使用 set 命令：

```
$ set
_               clear

addsuffix
argv      ()
autologout          60
cwd       /home/test
dirstack            /home/test
echo_style          both
edit
file      /etc/sysconfig/i18n
gid       511
group     test
history   1000
home      /home/test
loginsh
owd
path      (/usr/local/bin /bin /usr/bin /usr/X11R6/bin)
prompt    [%n@%m %c]$
prompt2   %R?
prompt3   CORRECT>%R (y|n|e|a)?
shell     /bin/tcsh
shlvl     1
sourced   1
status    0
tcsh      6.10.00
term      xterm
testing   10
tty       pts/0
uid       508
user      test
version tcsh 6.10.00 (Astron) 2000-11-19 (i386-intel-linux) options
8b,nls,dl,al,rh,color
$
```

注意，tcsh set 命令的输出与在 bash shell 中列出变量时的输出不同。tcsh shell 在左侧显示 shell 变量，在右侧显示当前值。

这些命令中有许多在 bash 中都有相应的命令。下面是需要注意的两点区别：

- tcsh shell 使用 shell 变量 prompt（连同 prompt2 和 prompt3）而不是 PS1、PS2 和 PS3 环境变量。
- shell 变量 path 包含一个索引目录列表，而不是一个以冒号分隔各个值的字符串。

要显示或使用当前 shell 变量值，必须在变量前加上一个美元符：

```
$ echo $path
/usr/local/bin /bin /usr/bin /usr/X11R6/bin
$
```

索引字符串使用数组格式来包含每个 shell 变量值。可以使用数组格式来访问每个元素：

```
$ echo $path[1]
/usr/local/bin
$ echo $path[2]
/bin
$ echo $path[3]
/usr/bin
$
```

注意数组的起始索引位置是 1。

记住 shell 变量全部使用小写字母非常重要，而环境变量全部使用大写字母。混淆它们会产生问题。

### 21.2.8 环境变量

与 Bourne shell 类似，tcsh shell 还维护一个环境变量集。表 21-3 列出这些环境变量。

表 21-3 tcsh shell 环境变量

| 变量 | 描述 |
|---|---|
| AFSUSER | 相当于 shell 变量 afsuser |
| COLUMNS | 终端上可用的列数 |
| DISPLAY | 图形窗口的 X Windows 服务器的指针 |
| EDITOR | 默认编辑器的路径名 |
| GROUP | 相当于 shell 变量 group |
| HOME | 相当于 shell 变量 home |
| HOST | 运行 shell 进程的系统名 |
| HOSTTYPE | 运行 shell 进程的机器类型 |
| HPATH | 编辑器命令 run-help 查找命令文档使用的以冒号分隔的目录列表 |
| LANG | 指定首选字符环境 |
| LC_CTYPE | 如果设置，则只改变 ctype 字符处理 |
| LINES | 终端上的行数 |
| LS_COLORS | 同 ls 命令一起使用的以冒号分隔的文件的类型和颜色分配列表 |
| MACHTYPE | 微处理器类型或系统机器型号 |
| OSTYPE | 系统的操作系统 |
| PATH | 用于放置可执行文件的以冒号分隔的目录列表 |
| PWD | 相当于 shell 变量 cwd，但只在实际目录修改后进行更新 |
| REMOTEHOST | 如果当前用户是远程登录的，则为远程主机的 IP 地址 |
| SHLVL | 嵌套 shell 的数目（相当于 shell 变量 shlvl） |
| SYSTYPE | 当前系统类型 |
| TERM | 终端类型（相当于 shell 变量 term） |
| TERMCAP | 终端功能字符串 |
| USER | 相当于 shell 变量 user |

续表

| 变量 | 描述 |
| --- | --- |
| VENDOR | 系统处理器的厂商 |
| VISUAL | 全屏编辑器的路径名 |

许多 tcsh 环境变量和同名的 shell 变量重复。这是为了提供对 Bourne shell 环境的兼容（注意环境变量 PATH 包含的内容，PATH 环境变量的格式和 Bourne shell 环境变量 PATH 的格式相同）。

但是，二者并不链接在一起。如果修改其中一个，另一个不会改变。因此，如果设置了 shell 变量（如 path）而没有设置相应的环境变量 PATH，就可能造成各种各样的问题。

要显示当前环境变量，必须使用 setenv 命令：

```
$ setenv
USER=test
LOGNAME=test
HOME=/home/test
PATH=/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin:/usr/X11R6/bin
MAIL=/var/mail/test
SHELL=/bin/tcsh
TERM=xterm
HOSTTYPE=i386-linux
VENDOR=intel
OSTYPE=linux
MACHTYPE=i386
SHLVL=1
PWD=/home/test
GROUP=test
HOST=testbox
INPUTRC=/etc/inputrc
LESS=-MM
LESSKEY=/etc/.less
BSNUM=14
BACKSPACE=
HOSTNAME=testbox
LS_COLORS=no=00:fi=00:di=01;34:ln=01;36:pi=40;33:so=01;35:bd=40;33;01
:cd=40;33;01:or=01;05;37;41:mi=01;05;37;41:ex=01;32:*.cmd=01;32:*.exe
=01;32:*.com=01;32:*.btm=01;32:*.bat=01;32:*.tar=01;31:*.tgz=01;31:*.
tbz2=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lzh=01;31:*.lha=01;3
1:*.zip=01;31:*.z=01;31:*.Z=01;31:*.gz=01;31:*.bz2=01;31:*.bz=01;31:*
.tz=01;31:*.rpm=01;31:*.jpg=01;35:*.jpeg=01;35:*.gif=01;35:*.bmp=01;3
5:*.xbm=01;35:*.xpm=01;35:*.png=01;35:*.tif=01;35:*.tiff=01;35:
LC_CTYPE=en_US
LC_MONETARY=en_US
LANGUAGE=en_US:en
LC_TIME=en_US
LC_NUMERIC=en_US
LC_COLLATE=en_US
LC_MESSAGES=en_US
LANG=en
TMPDIR=/home/test/tmp
TMP=/home/test/tmp
$
```

注意，setenv 命令使用和 bash shell 的 set 命令相同的输出格式。

要显示或使用环境变量值，也是必须包含美元符：

```
$ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin
$
```

虽然 tcsh 环境变量看起来和作用上与 bash shell 相应的变量类似，实际上有很大的不同。下一节将介绍在 tcsh shell 中设置环境变量和 shell 变量需要做的事情。

### 21.2.9 在 tcsh 中设置变量

在 tcsh 中设置变量较为复杂。如果不是对所有规则都很熟悉，则根据要使用的数据值的不同，设置变量可能成为一个令人沮丧的过程。

### 21.2.10 使用 set 命令

对于 shell 变量，可以使用 set 命令设置 shell 变量值。对于简单的一个词的数据值，使用下面的格式：

```
set variable=value
```

非常简单：

```
$ set testing=100
$ echo $testing
100
$ set test2=test
$ echo $test2
test
$
```

如果需要使用索引数组值则会比较复杂。如果索引值已经存在，很容易就可以替换为新值：

```
$ echo $path
/usr/local/bin /bin /usr/bin /usr/X11R6/bin
$ set path[4]=/home/test
$ echo $path
/usr/local/bin /bin /usr/bin /home/test
$
```

但是，不能使用这一技术添加新元素：

```
$ set path[5]=/home/test
set: Subscript out of range.
$
```

而是必须使用圆括号来表示新数组：使用$path 将现有数组元素插入到新数组中，然后将新数组元素添加到最后：

```
$ set path=($path /home/test2)
$ echo $path
/usr/local/bin /bin /usr/bin /home/test /home/test2
$ echo $path[5]
/home/test2
$
```

现在新数组值中就包含新增的元素。

最后，tcsh shell 使用 at 符（@）作为 set 命令的别名。at 符的使用方法和 set 命令的使用方法相同。

```
$ @ testing = 100
$ echo $testing
100
$
```

必须注意在 at 符后面一定要放置一个空格，否则会产生错误消息。

### 21.2.11 使用 setenv 命令

使用 setenv 命令设置 tcsh shell 中的环境变量值。但是 setenv 命令的格式和 set 命令的格式略有区别：

```
setenv variable value
```

注意，不使用等号进行赋值：

```
$ setenv TESTING 10
$ echo $TESTING
10
$ setenv TEST2 test
```

```
$ echo $TEST2
test
$
```

setenv 命令的奇怪之处是向值列表中新增一个值（如 PATH 环境变量）时，如果要像在 bash shell 中那样处理，就会得到错误消息：

```
$ setenv PATH $PATH:/home/test
Bad : modifier in $ (/).
$
```

tcsh shell 将冒号解释为 shell 变量的一部分。要向 PATH 环境变量中添加元素，需要在环境变量名称两边使用大括号：

```
$ setenv PATH ${PATH}:/home/test
$ echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin:/home/test
$
```

现在新元素就正确的添加到了 PATH 环境变量中。

## 21.2.12　tcsh 内置命令

tcsh shell 包含许多内置命令，在命令行和 shell 脚本中都可以使用这些内置命令。表 21-4 介绍了可用的命令。

表 21-4　　tcsh shell 内置命令

| 命令 | 描述 |
|---|---|
| @ | 显示或设置 shell 变量 |
| alias | 创建命令别名 |
| alloc | 显示动态内存状态 |
| bg | 将当前或指定作业置于后台模式执行 |
| bindkey | 显示或设置编辑器命令与键盘按键的映射关系 |
| builtins | 显示所有内置命令 |
| cd | 改变工作目录为主目录或指定目录 |
| chdir | 同 cd 命令 |
| complete | 显示或管理命令完成字符串 |
| dirs | 显示或保存当前目录栈 |
| echo | 在 STDOUT 中显示字符串 |
| echotc | 在 STDOUT 中执行终端命令 |
| eval | 将指定的字符串作为输入，让 shell 来执行 |
| exec | 执行指定的命令来替代当前 shell |
| exit | 终止当前 shell 进程 |
| fg | 以前台模式继续执行指定作业 |
| filetest | 对指定的文件进行指定的测试 |
| glob | 类似 echo，但不识别转义符，而且词与词之间通过 null 字符进行分界 |
| hashstat | 显示命令行哈希表元素使用情况的统计信息 |
| history | 显示和管理命令历史文件 |
| hup | 执行指定的命令，而且如果接收到一个挂起信号则退出 |
| jobs | 显示活动作业 |
| kill | 发送指定信号到指定的进程 ID（PID） |
| limit | 限制指定的资源，使使用的资源不能超过指定的资源（CPU 时间、文件空间、内存大小等） |
| log | 显示 shell 变量 watch 的内容，并报告每个用户 |
| login | 终止 shell，并将其替换为/bin/login 进程 |
| logout | 终止 shell |
| ls-F | 使用 ls 命令与-F 参数显示文件，通过特殊字符标识文件类型 |
| nice | 设置 shell 或指定命令的优先级策略 |

续表

| 命令 | 描述 |
|---|---|
| nohup | 执行命令，忽略挂起信号 |
| notify | 当指定作业的状态改变时通知用户 |
| onintr | 收到中断时设置 shell 动作 |
| popd | 目录栈出栈并返回到新的顶端目录 |
| printenv | 显示环境变量的名称和值 |
| pushd | 交换目录堆栈最上面两个元素 |
| rehash | 重新计算命令哈希表 |
| repeat | 按指定的次数执行指定命令 |
| sched | 显示或管理预定事件列表 |
| set | 显示或管理 shell 变量 |
| setenv | 显示或管理环境变量 |
| settc | 将指定的终端能力特性设置为指定值 |
| setty | 设置不允许改变的 TTY 模式 |
| shift | 将位置参数移动一个位置 |
| source | 读取并执行指定文件中的命令 |
| stop | 停止在后台模式下执行的指定作业或进程 |
| suspend | 使用 SIGSTOP 信号立即停止 shell 进程 |
| telltc | 显示所有终端功能值 |
| time | 执行指定的命令并显示一个时间总和 |
| umask | 为新文件或目录设置默认权限 |
| unalias | 删除与指定模式相匹配的所有别名 |
| uncomplete | 删除与指定模式相匹配的所有完成 |
| unhash | 关闭使用哈希表来查找可执行程序 |
| unlimit | 删除某一资源的资源约束 |
| unset | 删除变量名与指定模式相匹配的所有 shell 变量 |
| unsetenv | 删除变量名与指定模式相匹配的所有环境变量 |
| wait | 等待所有后台作业结束 |
| where | 显示指定命令的所有实例，包括别名 |
| which | 匹配指定模式后，显示 shell 要执行的命令 |

tcsh shell 提供了很多内置命令。这些命令中有许多在 bash 中都有相应的命令。tcsh shell 提供的有一些命令在 bash shell 中是没有的。

alloc 命令是检查 Linux 系统内存状态的简单方法：

```
$ alloc
tcsh current memory allocation:
free:      0   83   93    5    13    6    1    1    0    0    1    1
0    0    0    0    0    0    0    0    0    0    0    0    0    0
0    0    0
used:      0  173  611  347   35  834    7    7    5    1    1    1
0    0    0    0    0    0    0    0    0    0    0    0    0    0
0    0    0
       Total in use: 312176, total free: 33936
       Allocated memory from 0x80c31e0 to 0x8117c00. Real top at
0x8117c00
$
```

alloc 命令的输出有些神秘。两个数字集合列出了内存块清单，按块大小递增的方式排列，首先是 8 字节块，然后是 16 字节块，以此类推，直到 2MB 内存块。因此，在本例中，有 83 块 16 字节的块空闲，93 个 32 字节的块空闲，5 个 64 字节的块空闲等，最后有一个 16KB 的块空闲。同样的形式还用来显示内存块的使用情况。结果总计显示在清单末尾。

ls-F 命令是另一个只在 tcsh shell 中存在的命令。它生成的输出和使用 ls 命令加上-F 参数的输

出相同（-F 参数在每个条目旁边显示一个指示符，表示可执行文件、链接文件和目录）。因为它是内置命令，所以与标准的 ls 命令相比，它可以更快的生成结果。

bindkey 命令是 tcsh shell 中又一个有趣的内置命令。它允许将组合键与任意键序列或 shell 命令进行绑定。默认情况下，tcsh shell 将许多组合键和编辑器命令进行了绑定，可以使用简单的编辑器组合键轻松编辑命令行输入。

在命令行中输入不带参数的 bindkey 命令，可以列出当前的所有键绑定信息：

```
$ bindkey
Standard key bindings
"^@"          -> set-mark-command
"^A"          -> beginning-of-line
"^B"          -> backward-char
"^C"          -> tty-sigintr
"^D"          -> delete-char-or-list-or-eof
"^E"          -> end-of-line
```

这只是 tcsh shell 中当前键绑定列表的一部分。整个列表非常长。

要定义新的键绑定，列出需要的键组合和命令或要赋值的字符串：

```
$ bindkey -c ^G ls-F
$
test1~*  test2*        tmp/
logout.txt test2~*     test.txt
$
```

现在每次按下 Ctrl-G 组合键就会得的 ls-F 的输出结果！

## 21.3　在 tcsh 中编写脚本

在 tcsh shell 中编写 shell 脚本并不比在 bash shell 中编写脚本困难，只是略有不同。需要适应一种略有不同的方法来执行许多已经习惯了的 bash 标准语句。

### 21.3.1　处理变量

您已经看到，处理变量时需要使用 set 命令。除此之外，可以像在 bash shell 中一样使用变量。记住在想要引用变量值时在变量前面加上美元符。

### 21.3.2　数组变量

tcsh shell 也支持使用索引值的一维数组变量：

```
$ set myarray = (one two three)
$ echo $myarray[1]
one
$ echo $myarray[2]
two
$ echo $myarray[3]
three
$
```

要创建新数组，将每个元素放到圆括号中，用空格分隔。数组中的第一个元素必须作为项目 1 被引用。

### 21.3.3　处理数学运算

在 bash shell 中，可以使用 expr 命令，双圆括号或方括号通过变量来处理数学运算。在 tcsh shell 中，只需要使用圆括号就可以处理所有数学函数。但是，只能使用 set 命令的别名 at 符执行数学运算。

```
$ set test1 = 10
$ set test2 = 15
$ @ test3 = ( $test1 * $test2 )
$ echo $test3
```

```
150
$
```

唯一一个必须使用 at 符的语句是执行数学运算的语句。只进行赋值操作的语句可以使用 set 命令也可以使用 at 符别名。

### 21.3.4 结构化命令

tcsh shell 支持以下结构化命令：

- if-then-else；
- foreach；
- while；
- switch。

下面几节将分别介绍如何在 shell 脚本中使用每种结构化命令。

### 21.3.5 if 语句

tcsh 的 if 语句有多种格式。第一种格式如下：

```
if (expression) command
```

如果圆括号中定义的 expression 的值为 TRUE，shell 将执行行中列出的 command：

```
$ cat test3
#!/bin/tcsh
# simple if statement test

set testing = 10

if ($testing == 10) echo "it worked"
echo "This is the end of the test"
$ ./test3
it worked
This is the end of the test
$
```

如果表达式为 TRUE，if 语句还可以和 then 子句一同使用来执行多个语句：

```
if (expression) then
    statements
endif
```

与 bash 的 if 语句类似，如果表达式为 FALSE，tcsh 的 if 语句可以和 else 子句联合起来执行备选语句：

```
if (expression) then
    statements
else
    other statements
endif
```

最后，可以在 else 行中再使用一个 if 语句将多个 if 语句链接在一起：

```
if (expression1) then
    statement1
else if (expression2) then
   statement2
endif
```

对于数学和字符串比较，if 语句中的表达式都使用标准 C 风格的符号，==、!=、<、>、<=、>+、&&（逻辑 AND）和||（逻辑 OR）。除了数学符号，tcsh 的 if 语句还支持两个字符串符号：

- =~匹配字符串模式。
- !~不匹配字符串模式。

下面是使用字符串表达式的例子：

```
$ cat test4
```

```
#!/bin/tcsh
# testing a string comparison

set testing=testing

if ( $testing =~ test* ) echo "it matched test*"
if ( $testing !~ test ) echo "it didn't match test"
$ ./test4
it matched test*
it didn't match test
$
```

还有另一个表达式符号可能会很好用。$?符号可以测试某个变量是否被设置：

```
$ cat test5
#!/bin/tcsh
# using the $? symbol

set test1 = 10

if ($?test1) echo "The test1 variable has been set"
if ($?test2) echo "The test2 variable has been set"
$ ./test5
The test1 variable has been set
$
```

tcsh shell 还包含特殊的测试运算符来测试文件和目录。这些测试形式如下：

```
-op file
```

其中 op 是一个单字符运算，定义要测试的文件操作的类型。

表 21-5 给出了 tcsh 中可以使用的文件测试运算符。

**表 21-5　　tcsh 文件测试操作符**

| 操作 | 描述 | 操作 | 描述 |
|---|---|---|---|
| r | 读访问 | z | 大小为 0 |
| w | 写访问 | s | 大小非 0 |
| x | 可执行 | f | 无格式文件 |
| X | 包含在路径或内置命令中 | d | 目录 |
| e | 文件存在 | l | 符号链接 |
| o | 文件的所有者 | | |

这样为测试 shell 脚本中文件的状态提供了一种简单的方法：

```
$ cat test6
#!/bin/tcsh
# testing for the $HOME directory and a file

if ( -d $HOME ) then
   echo "The HOME directory exists"
endif
if ( -f $HOME/test) then
   echo "The test file exists"
else
   echo "The test file doesn't exist"
endif
$ ./test6
The HOME directory exists
The test file doesn't exist
$ touch $HOME/test
$ ./test6
The HOME directory exists
The test file exists
$
```

使用文件测试运算符，可以在脚本使用文件前就很容易的了解文件状态。

### 21.3.6 foreach 语句

tcsh shell 不使用 for 语句，但它包含一个 foreach 语句来对数组变量或列表中的值进行迭代。foreach 语句的格式如下：

```
foreach var1 (wordlist)
    statements
end
```

参数 wordlist 可以是一个数组变量，或是以空格分隔的值列表：

```
$ cat test7
#!/bin/tcsh
# testing the foreach statement

echo "The directories in your PATH are:"
foreach dir ($path)
   echo $dir
end
$ ./test7
The directories in your PATH are:
/usr/local/bin
/bin
/usr/bin
/usr/X11R6/bin
$
```

这是迭代数组变量的很不错的方法，尤其在不知道数组中元素个数的情况下。

### 21.3.7 while 语句

tcsh 中的 while 语句允许一直循环到指定的表达式的值等于 FALSE 时结束。while 命令的格式为：

```
while (expression)
    statements
end
```

while 语句中使用的表达式和 if 语句中的表达式格式相同，使用 C 风格的数学比较符号，以及特殊的=~和!~字符串比较符：

```
$ cat test8
#!/bin/tcsh
# determining the factorial of a value

set value=$1
set factorial = 1
set counter = 1
while ( $counter <= $value )
   @ factorial = ( $factorial * $counter )
   @ counter += 1
end
echo "The factorial of $value is $factorial"
$ ./test8 5
The factorial of 5 is 120
$
```

while 语句会不断循环这些语句，直到表达式的值为 FALSE。

### 21.3.8 switch 命令

tcsh shell 中的 switch 命令允许基于字符串匹配值执行多个语句。switch 语句的格式为：

```
switch (string)
case val1:
    statements
```

```
case val2:
    statements
default:
    statements
endsw
```

shell 比较 string 值和每个 case 语句中给出的值。如果某个 case 语句中的值和 string 相匹配，那么 shell 将执行该 case 语句中包含的所有语句，以及后面所有 case 语句中的语句。

如果希望 shell 只执行一个 case 区中的语句，可以在 case 区的结尾使用 breaksw 命令。这样 shell 就会跳到 end 语句。如果没有任何 case 值匹配，会执行 default 区中的语句。

下面是在 shell 脚本中使用 switch 语句的一个例子：

```
$ cat test9
#!/bin/tcsh
# reading input and using the switch statement

echo -n "Please enter a word: "
set input = $<
switch ($input)
case test:
   echo "This is a test"
   breaksw
case hello:
   echo "Hello world!"
   breaksw
case exit:
   echo "Thank you for playing"
   breaksw
default:
   echo "Sorry, I don't understand that command"
endsw
$ ./test9
Please enter a word: test
This is a test
$ ./test9
Please enter a word: testing
Sorry, I don't understand that command
$ ./test9
Please enter a word: exit
Thank you for playing
$
```

因为 tcsh shell 不像 bash shell 那样包含 read 语句，所以必须直接从 STDIN 上读取输入。可以使用$<符号来进行读取：

```
set input = $<
```

获取输入之后，switch 语句将读取的文本和每个 case 语句进行比较。如果没有匹配值，使用 default 语句（如果提供的话）。

注意：可能会感到不可思议，但 tcsh shell 的脚本中不支持函数。在将 bash shell 脚本转到 tcsh 环境下时，这是个巨大的障碍。

## 21.4 小结

本章讨论了 C shell 中的特性。虽然 C shell 在 Linux 中并不是特别流行，但很可能会遇到它，所以最好了解它和 bash shell 有哪些区别。

最流行的 C shell 实现是 tcsh shell。虽然 tcsh shell 通常不是 Linux 的默认 shell，但它通常是有效的安装文件。这就允许编写和测试使用 C shell 的 Unix 环境下使用的脚本。

C shell 使用两类变量，shell 变量和环境变量。shell 变量是小写的，包含系统信息和 C shell 中常用

的设置。环境变量大多数用于兼容 Bourne 和 bash shell 脚本，并且维护简单的值，如路径和 shell 名称。

要设置 shell 变量，必须使用 `set` 语句。C shell 允许使用赋值数学运算给变量，但必须使用 `set` 语句的别名 at 符（`@`）。使用这个符号就可以使用标准数学符号和圆括号来编写数学等式。

要设置环境变量，必须使用 `setenv` 语句。

C shell 提供大多数 bash shell 中使用的标准结构命令。不幸的是，它们使用的格式稍有不同，所以需要重写 bash shell 脚本才能使这些脚本在 C shell 环境中正确执行。

下一章将研究另一个流行的 Unix shell 在 Linux 世界的相应版本。Korn shell 在 Sun Unix 环境中很流行，当必须将 shell 脚本从 Sun 服务器上向 Linux 服务器上迁移时，会经常用在 Linux 中。

CHAPTER 22

# 第 22 章 Korn shell

本章内容：

- Korn shell；
- ksh93 shell 的组成部分；
- 在 ksh93 shell 中编写脚本。

探讨各种 Linux shell 的过程中，毫无疑问早晚都会遇到 Korn shell。Korn shell 在 Unix 世界中很流行，但在 Linux 世界中并不同样流行。Korn shell 提供了 Bourne shell 和 C shell 世界的混合特性。本章将讨论 Korn shell 特性，介绍 Korn shell 最常用的版本 ksh93 shell。

## 22.1 Korn shell 的历史

原始 Korn shell 是 David Korn 于 20 世纪 80 年代在 AT&T 贝尔实验室工作期间开发的。David 开发 Korn shell（其名字的由来显而易见）目的是使其成为下一代编程 shell，结合了最佳的 Bourne shell 特性和最佳的 C shell 特性。Korn shell 很快成为众所周知的程序员 shell。它支持 Bourne shell 和 C shell 缺少的高级编程特性，包括关联数组与浮点计算。

在 2000 年以前，原始 Korn shell 一直是由 AT&T 持有的私有 shell。此后它才成为开源软件。

原始 Korn shell 分两种不同的路线：

- ksh88；
- ksh93。

大多数 Korn shell 实现（包括 Linux 发行版中的那些 Korn shell）使用 ksh93 shell。Sun Solaris 除外。Sun 使用改良版的 ksh88 shell，与 ksh93 shell 有些不同。因为 Linux 系统上使用的 Korn shell 使用 ksh93 shell，所以本章将介绍 ksh93 shell。

注意：某些 Linux 发行版和 Unix 系统（如 IBM 的 AIX 操作系统）也将 ksh93 shell 称为增强型 Korn shell。

在 Linux 环境中还可能会遇到另一种 Korn shell。当 Korn shell 还是私有软件时，开发了一种公共的 Korn shell（称为 pdksh shell）。pdksh shell 和 ksh88 shell 具有很多相同的特性，但它缺少 ksh93 shell 的高级数学特性。

## 22.2 ksh93 shell 的组成部分

因为 ksh93 shell 是基于 Bourne shell 和 C shell 的，所以 ksh93

shell 与 bash shell 和 tcsh shell 有很多相似之处。本节将介绍 ksh93 shell 中特有的特性，包括命令行参数、环境变量和内置命令。

## 22.2.1 命令行参数

和其他 shell 一样，Linux ksh93 shell 使用命令行参数来定义其作为交互式 shell 和脚本 shell 的行为。表 22-1 给出了启动新 ksh93 shell 会话时可能使用到的命令行参数。

表 22-1 ksh93 shell 命令行参数

| 参数 | 描述 |
|---|---|
| -a | 自动导出 shell 变量 |
| -A | 将命令行参数值赋给指定的数组变量 |
| -b | 作业状态改变后立即显示作业完成消息 |
| -B | 启用大括号格式的字段生成 |
| -c | 读取第一个参数中的命令 |
| -C | 防止重定向时重写已有文件 |
| -D | 不执行命令。显示由美元符引导的双引号引起来的字符串 |
| -e | 如果命令返回非零退出状态，则退出 shell |
| -f | 关闭文件名生成 |
| -G | 使通配符**同文件和目录进行匹配 |
| -h | 对每个命令进行别名跟踪 |
| -i | 交互式 shell。所有输出都发送到终端 |
| -m | 后台作业以独立的进程组执行 |
| -n | 读取命令并检查错误，但不执行命令 |
| -o | 设置默认的 shell 选项 |
| -p | 不处理$HOME/.profile 文件 |
| -P | 启动 shell 作为 profile shell |
| -r | 启动 shell 作为受限 shell |
| -R | 生成交叉引用数据库 |
| -s | 将所有输出发送到文件描述符 2 |
| -v | 读取时显示命令 |
| -x | 执行时显示命令及其参数 |

ksh93 shell 命令行参数与 bash shell 的非常相似。除了命令行参数，还可以使用-o 命令行参数自定义 shell 行为。使用-o 参数时可以设置以下一些 shell 选项。

- allexport：自动导出变量。
- errexit：如果命令返回非零退出状态，则退出 shell。
- bgnice：以较低优先级执行所有后台作业。
- bracexpand：启用大括号格式的字段生成。
- emacs：使用 emacs 编辑器模式进行命令行编辑。
- ignoreeof：遇到 EOF 符，不退出 shell。
- markdirs：文件生成产生的所有目录名称都以斜杠结束。
- monitor：后台作业以单独的进程组执行。
- noclobber：防止重定向重写文件。
- noexec：读取命令并显示错误，但不执行命令。
- noglob：禁用文件名生成。
- nolog：不在历史日志中保存函数。
- notify：作业状态改变时，显示作业的新状态。

- nounset：将 unset 参数视为错误。
- pipefail：直到所有组件都完成管道才完成。
- privileged：关闭处理$HOME/.profile 文件。
- verbose：读取命令时显示命令。
- trackall：对每个命令进行别名跟踪。
- vi：使用 vi 编辑器模式进行命令行编辑。
- viraw：按字符在 vi 编辑器模式下的输入处理每个字符。
- xtrace：执行命令时显示命令。

可以在命令行中设置多个命令行选项，来一次设置多个 shell 特性。

### 22.2.2　默认文件

使用 ksh93 shell 作为登录 shell 时，从下面两个文件中读取命令：

- /etc/profile；
- $HOME/.profile。

需要将所有用户都要执行的命令存储在/etc/profile 文件中，而将每个用户私有的命令存储在正确的$HOME/.profile 文件中。

### 22.2.3　环境变量

ksh93 shell 使用环境变量来设置 shell 环境的特性和提供 shell 环境的信息。表 22-2 给出了 ksh93 shell 中使用的默认环境变量。

表 22-2　　ksh93 环境变量

| 变量 | 描述 |
|---|---|
| CDPATH | cd 命令的搜索路径 |
| COLUMNS | 编辑模式的终端宽度，以字符为单位 |
| EDITOR | 命令行编辑的默认编辑器 |
| ENV | 定义一个文件来指定参数扩展、命令置换和算术置换（通常定义为$HOME/.kshrc） |
| FIGNORE | 设置进行文件名匹配时要忽略的文件名 |
| FPATH | 函数的搜索路径 |
| HISTCMD | 历史文件中当前命令的数量 |
| HISTEDIT | 历史命令的默认编辑器 |
| HISTFILE | 存储命令历史的文件 |
| HOME | 设置的默认用户登录目录 |
| IFS | 内在的字段分隔符，默认设置为空格、制表符和换行符 |
| LANG | 函数使用的本地目录，不会被环境变量 LC_覆盖 |
| LC_ALL | 覆盖 LANG 和所有 LC_环境值设置 |
| LC_COLLATE | 用于校对的语言 |
| LC_CTYPE | 处理字符集使用的语言 |
| LC_NUMERIC | 十进制小数点字符使用的语言 |
| LINES | 终端的可用行数 |
| LINENO | 正在处理的脚本中的当前行 |
| MAIL | 检查新邮件用到的邮件文件名称 |
| MAILCHECK | 检查 MAIL 指定的邮件文件的时间间隔（以秒为单位） |
| MAILPATH | 检查 MAIL 指定的邮件文件时用到的冒号分隔的目录列表 |
| OLDPWD | 前一个工作目录 |
| PATH | 用于搜索可执行命令的冒号分隔的目录列表 |
| PPID | 父进程的进程 ID |

续表

| 变量 | 描述 |
| --- | --- |
| PS1 | 主命令行提示字符串（默认为$） |
| PS2 | 次命令行提示字符串（默认为>） |
| PS3 | 选择循环提示（默认为#?） |
| PS4 | 进行参数求值、命令置换和算术置换时使用的提示（默认为+） |
| PWD | 当前工作目录 |
| RANDOM | 每次访问时会生成一个新的随机整数值 |
| SECONDS | shell 启动后的秒数 |
| SHELL | 当前 shell 的路径名 |
| TIMEFORMAT | 指定显示时间的格式字符串 |
| TMOUT | 如果该值不为零，则表示读取命令的默认等待时间 |
| VISUAL | 允许启用 emacs、gmac 或 vi 编辑器 |

ksh93 环境变量使用的格式与 bash 环境变量格式相同，系统环境变量都使用大写字符。使用 set 命令显示当前的环境变量设置：

```
$ set
_=/bin/ksh
COLORTERM=''
DBUS_SESSION_BUS_ADDRESS=unix:abstract=/tmp/dbus-
J1Namp8Rkt,guid=a1ed8c4706b2db3ebaa39969fed7c100
DESKTOP_SESSION=kde
DISPLAY=:0.0
DM_CONTROL=/var/run/xdmctl
ENV='$HOME/.kshrc'
FCEDIT=/bin/ed
GS_LIB=/home/rich/.fonts
GTK2_RC_FILES=/home/rich/.gtkrc-2.0
GTK_RC_FILES=/etc/gtk/gtkrc:/home/rich/.gtkrc:/home/rich/.kde/share/
config/gtkrc
HISTCMD=35
HOME=/home/rich
IFS=$' \t\n'
KDEDIR=/usr
KDE_FULL_SESSION=true
KDE_MULTIHEAD=false
KDEROOTHOME=/root/.kde
KONSOLE_DCOP='DCOPRef(konsole-4373,konsole)'
KONSOLE_DCOP_SESSION='DCOPRef(konsole-4373,session-1)'
LANG=en_US
LANGUAGE=en
LC_ALL=en_US
LINENO=1
LOGNAME=rich
MAILCHECK=600
OLDPWD=/home/rich
OPTIND=1
PATH=/bin:/sbin:/usr/bin:/usr/sbin:/usr/X11R6/bin:/usr/local/bin:/usr
/local/sbin
PPID=5619
PS1='$ '
PS2='> '
PS3='#? '
PS4='+ '
PWD=/home/rich/test
QTDIR=/usr/share/qt3
RANDOM=17445
```

```
SECONDS=1.291
SESSION_MANAGER=local/testbox:/tmp/.ICE-unix/4315
SHELL=/bin/ksh
SHLVL=3
SSH_AGENT_PID=4270
SSH_AUTH_SOCK=/tmp/ssh-HcqYkH4172/agent.4172
TERM=xterm
TMOUT=0
USER=rich
WINDOWID=35651589
XCURSOR_THEME=default
XDM_MANAGED=/var/run/xdmctl/xdmctl-
:0,maysd,mayfn,sched,rsvd,method=classic
$
```

在 ksh93 shell 中还可以创建自定义环境变量。通常自定义环境变量使用小写字符，以避免同系统环境变量混淆。

在 ksh93 shell 中设置环境变量，使用赋值运算符：

```
$testing=10 ; export testing
$ ksh
$ echo $testing
10
$
```

ksh93 shell 使用 `export` 命令将环境变量导出到子 shell。

后面几节将介绍一些有趣的 ksh93 环境变量特性。

### 1. 随机数

ksh93 shell 使用 RANDOM 变量可以很容易地在脚本中生成随机数。每次引用 RANDOM 变量时，它都会生成 0～32767 之间的一个随机整数。

```
$ echo $RANDOM
13452
$ echo $RANDOM
12498
$ echo $RANDOM
31104
$
```

可以赋给 RANDOM 变量一个整数值作为随机数生成器的种子。不同的种子值会生成不同的随机数序列。通常使用变量$$作为种子值。这个特殊变量表示 shell 的进程 ID（PID），shell 的每个运行实例的 PID 是唯一的。

### 2. 定义变量类型

ksh93 shell 中有一个 bash shell 中没有的特性，即 `typeset` 命令。这个命令允许设置变量和函数的属性。其中可以设置的一个属性是变量使用的数据类型。表 22-3 列出了 `typeset` 命令可以使用的参数。

表 22-3　typeset 命令参数

| 参数 | 描述 | 参数 | 描述 |
|---|---|---|---|
| -A | 设置变量为关联数组 | -L | 删除前面的空格并使文本左对齐 |
| -b | 变量可以包含任意数目的数据字节 | -n | 声明变量为其他变量的引用 |
| -E*n* | 设置变量为双精度浮点数，n 位有效数字 | -r | 设置变量为只读 |
| -F*n* | 设置变量为双精度浮点数，n 位小数 | -R | 在前面添加空格使文本右对齐 |
| -f | 为函数命名 | -u | 将所有小写字符转成大写 |
| -i*n* | 设置变量为整数值，进制数为 n | -x | 标记用于自动导出的变量 |
| -l | 将所有大写字符转成小写 | -Z | 右对齐，前面补零 |

`typeset` 命令为在不同的进制之间进行数字转换提供了很好的方法：

```
$ typeset -i2 a
$ typeset -i8 b
$ typeset -i16 c
$ a=100
$ b=100
$ c=100
$ echo $a $b $c
2#1100100 8#144 16#64
$
```

每个变量都使用不同的进制进行显示（显示值时进制显示在英镑符前面）。

-l 和-u 参数在转换字符串方面作用很大：

```
$ typeset -u testing
$ testing="This is a test"
$ echo $testing
THIS IS A TEST
$
```

没有比这更简单的了！

使用-n 参数可以使一个变量引用另一个单独变量的值。这会带来混乱，所以要谨慎使用。如下面的例子所示：

```
$ ntest1=100
$ typeset -n ntest1=ntest2
$ ntest2=1
$ echo $ntest1
1
$ ntest1=100
$ echo $ntest2
100
$
```

typeset 命令强制变量 ntest1 和 ntest2 引用同一个内存地址。无论赋给其中一个变量什么值，另一个变量都会引用这个值。当将 1 赋值给 ntest2 变量时，会替换 ntest1 的现有值。当显示变量 ntest1 的值时，显示的值和赋给 ntest2 的值相同。同样给变量 ntest1 赋值时 ntest2 也适用该值。

### 3. 使用数组

与 bash shell 一样，ksh93 支持使用数字数组变量：

```
$ mytest=(one two three four five)
$ echo $mytest
one
$ echo ${mytest[2]}
three
$ echo ${mytest[*]}
one two three four five
$
```

ksh93 shell 还支持关联数组。关联数组允许使用字符串作为索引值，来替代数字。在定义关联数组中的值之前，必须声明将要使用的变量。使用 typeset 命令进行声明：

```
$ typeset -A test2
$
```

声明变量为关联数组后，就可以使用下面的格式对键和值进行赋值：

```
variable=( [key1]=value1 [key2]=value2...)
```

键可以是任意长度的字符串，值可以是数字也可以是文本字符串：

```
$ test2=( [fruit]=banana [vegetable]=carrot [age]=18)
$
```

定义了关联数组中的元素后，可以根据键进行访问：

```
$ echo ${test2[fruit]}
banana
$ echo ${test2[age]}
```

```
18
$
```

使用关联数组的一个问题是，不能像数字数组那样循环遍历索引数字，无法获知存在的键值。为了解决这个问题，ksh93 shell 提供了一个特殊格式来显示关联数组的键值：

```
$ echo ${!test2[@]}
fruit age vegetable
$
```

注意，打印的键的顺序和它们定义的顺序不同。这一点非常重要，必须牢记。不能保证 ksh93 按定义的顺序存储关联数组值。

虽然这看起来很混乱，但没什么大不了的。下面显示出所有的元素值，就会明白了：

```
$ echo ${test2[*]}
banana 18 carrot
$
```

显示元素值时，它们的显示顺序和键的显示顺序相同。这样就可以很容易将键和值进行匹配。

关联数组提供了一种在 shell 脚本中匹配字符串键和值的方法，它是个很强大的工具。

#### 4. 复合变量

ksh93 shell 还支持另一种变量，称为**复合变量**（compound variable）。复合变量和关联数组类似，但引用值的形式稍有不同。

定义复合变量的格式为：

```
variable=( subscript1=value1 subscript2=value2 ...)
```

其中下标（subscript）是一个文本值，与关联数组中的键类似。值可以是任意文本或数字：

```
$ test3=( fruit=banana vegetable=carrot age=18)
$
```

赋值下标后，就可以通过下面的格式引用单个下标值：

```
variable.subscript
```

示例如下：

```
$ echo ${test3.fruit}
banana
$ echo ${test3.age}
18
$
```

每个下标值都可以通过引用变量名加下标名来获取。

#### 5. 特殊的变量下标

ksh93 shell 中可以使用一些特殊的下标。这些下标定义当前的 shell 信息，当前 shell 中运行的进程的当前信息。表 22-4 列出了特殊的 ksh93 shell 下标变量。

表 22-4　　ksh93 下标变量

| 变量 | 描述 |
| --- | --- |
| .sh.command | 将要执行的当前命令行 |
| .sh.edchar | 为上一个 KEYBD 捕获输入的键盘字符值 |
| .sh.edcol | 上一个 KEYBD 捕获处的光标字符位置 |
| .sh.edmode | 当在 vi 编辑器模式下处理按键时设置为 ESC，其他情况设置为空 |
| .sh.edtext | 处理上一个 KEYBD 捕获时键盘缓冲区中的字符 |
| .sh.file | 包含当前命令的文件路径名 |
| .sh.fun | 当前正在执行的函数名 |
| .sh.match | 存储最近匹配和 subpattern 匹配的索引数组 |
| .sh.name | 执行某个规程函数时的变量名 |
| .sh.subscript | 设置为执行某个规程函数时变量的下标名称 |
| .sh.subshell | 当前子 shell 和命令替换的深度 |

续表

| 变量 | 描述 |
| --- | --- |
| .sh.value | 执行某个规程函数时的变量值 |
| .sh.version | shell 的版本 |

因为下标变量的名称中使用句点，所以在引用下标变量时必须使用大括号将它们括起来：

```
$ echo ${.sh.version}
Version M 1993-12-28 r-
$
```

规程函数中使用的下标变量将在后面的“规程函数”一节中讨论。

### 22.2.4 内置命令

ksh93 内置命令看起来和 bash 命令很相似，因为它们都是基于 Bourne shell 的。表 22-5 给出了 ksh93 内置命令。

表 22-5 ksh93 的内置命令

| 命令 | 描述 |
| --- | --- |
| alias | 为命令定义别名 |
| bg | 在后台模式启动指定作业 |
| builtin | 显示内置命令 |
| cd | 修改当前工作目录 |
| command | 执行指定的命令 |
| disown | shell 终止时，shell 不发送 HUP 信号给指定的作业 |
| echo | 显示文本字符串和变量 |
| eval | 将指定的参数读取到 shell 中并执行 |
| exec | 执行指定的命令来替代 shell，但不新建进程 |
| exit | 使 shell 以指定的退出状态退出 |
| export | 将指定的环境变量导出到所有子 shell |
| false | 退出状态为 1 时退出 |
| fg | 以前台模式启动指定的作业 |
| getconf | 显示 shell 配置参数的当前值 |
| getopts | 每次调用时，获取选项字符串中的下一个选项 |
| hist | 显示和编辑历史文件 |
| jobs | 显示指定作业或所有活动作业的信息 |
| kill | 发送指定的信号（如果未指定则发送 TERM）给指定的进程 ID |
| let | 计算指定的数学运算。如果表达式的值不等于零返回 0，否则返回 1 |
| newgrp | 作用相当于/bin/newgrp |
| print | 在 STDOUT 上显示参数 |
| printf | 使用 C 风格的格式字符串显示参数 |
| pwd | 显示当前工作目录 |
| read | 读取 STDIN 上的输入，并存储到变量中 |
| readonly | 读取 STDIN 上的输入，并存储到不可修改的变量中 |
| set | 设置定义在命令行参数中的 shell 特性 |
| shift | 重命名位置参数，使参数值左移一个位置 |
| sleep | 挂起执行，挂起时间长度为指定的秒数 |
| trap | 捕获指定信号，并执行指定命令 |
| true | 以 0 退出状态退出 |
| typeset | 设置 shell 变量和函数的属性和值 |

续表

| 命令 | 描述 |
|---|---|
| ulimit | 显示或设置资源限制 |
| umask | 显示或设置新建文件和目录的默认权限 |
| unalias | 删除定义的别名 |
| Unset | 删除指定变量的值 |
| wait | 等待，直到指定作业完成并报告其退出状态 |
| whence | 显示对指定的命令的解释方式（内置或工具位置） |

可以看到，ksh93 shell 包含很多内置命令，从而使用它编写脚本时很方便。可以看到其中大多数命令在 bash shell 中出现过。

ksh93 shell 使用 echo、print 和 printf 命令显示数据和文本。这 3 个命令的区别有时会被混淆。

ksh93 shell 的 echo 命令有些不可靠。大多数情况下，它和 bash shell 中的 echo 作用相同，但不能保证。ksh93 shell 中最好只使用最基础的 echo 命令用法。而更高级的格式打印，需要使用 print 或 printf 命令。

print 命令的格式为：

```
print [ -Renprs ] [ -u unit ][ -f format ] [ arg ...]
```

命令行参数包括以下几种。

- -R：打印所有后续选项和参数，而不打印-n。
- -e：启用转义符（如\n 和\t）。这是默认行为。
- -n：行尾不添加换行符。
- -p：将参数写入衍生进程的管道上，而不是 STDOUT。
- -r：关闭格式字符串中的转义符。
- -s：将参数写入历史文件中，而不是 STDOUT。
- -u：将输出写入指定文件描述符，而不是 STDOUT。
- -f：使用指定的 C 风格 printf 格式字符串显示参数。

可以使用命令行参数或 C 风格 printf 格式字符串来自定义 print 命令的输出（请参阅第 19 章）。默认的 print 命令只输出变量：

```
$ test=10
$ test2=15
$ print $test $test2
10 15
$
```

print 命令后面还可以使用同 echo 命令一样的格式：

```
$ value1=10
$ value2=15
$ print "The first value is $value, while the second is $value2"
The first value is 10, while the second is 15
$
```

print 命令还允许使用 printf 格式字符串来以格式文本字符串的风格显示变量，很像 printf 命令：

```
$ test=testing
$ test2=10
$ print -f "This %s value is %d\n" $test $test2
This testing value is 10
$
```

%s 和%d 控制代码决定 print 命令中列出的相关变量的输出的格式。

默认的 printf 命令只使用 C 风格 printf 格式显示参数。printf 命令的格式为：

```
printf format [ arg ...]
```

参数 format 是一个文本字符串，使用标准符号来表示输出中使用的具体的数据类型和格式（请

参阅第 19 章)。每个数据符号匹配命令行中提供的一个参数：

```
$ printf "This %s value is %d\n" $test $test2
This testing value is 10
$
```

printf 命令提供一些 print 命令没有提供的格式功能：

- 重复使用控制代码直到显示完列出的所有变量。
- 使用 date 命令的格式打印时间和日期值。

如果在 printf 命令行中提供的变量数多于格式字符串中的控制码数，printf 命令会自动返回到第一个控制码对剩下的变量重新应用控制码。如果要打印表中的值，这种方法很有效。只需要定义第一行的控制码，然后在同一行中列出所有变量：

```
$ test1=10
$ test2=15
$ test3=20
$ test4=25
$ printf "%d - %d\n" $test1 $test2 $test3 $test4
10 - 15
20 - 25
$
```

使用 printf 命令还可以很容易的利用 date 命令格式将日期和时间值重新格式化为任意格式：

```
$ value1='1/31/08'
$ print $value1
1/31/08
$ printf "%(%Y)T\n" $value1
2008
$ printf "%(%A, %B %d, %Y)T\n" $value1
Thursday, January 31, 2008
$ printf "%(%F)T\n" $value1
Thu Jan 31 12:42:11 EST 2008
$
```

使用%T 格式控制码的效果太令人吃惊了！

## 22.3 在 ksh93 shell 中编写脚本

大多数情况下，在 ksh93 shell 中编写代码和在 bash shell 中编写代码区别不大。大多数 bash shell 脚本可以很好地在 ksh93 环境下工作，而不能很好工作的那些脚本也只需稍作修改。

如果要编写专门的 ksh93 shell 脚本，可以使用一些方法使脚本编写更容易。本节将介绍 ksh93 shell 和 bash shell 的主要区别。

### 22.3.1 数学运算

ksh93 shell 的一个很棒的特性是可以轻松的使用数学运算。本节将介绍 ksh93 shell 中用于处理数字和执行数学运算的特性。

**1. 执行计算**

执行数学计算的方法有两种：

- let 命令；
- 双圆括号。

使用 let 命令时可以将数学运算的结果直接赋值给一个变量：

```
$ let value1=4*5
$ echo $value1
20
$
```

使用这个格式，在等于号和等式元素之间不能有空格。但可以将等式包含在双引号中，双引号里面可以包含空格：

```
$ let value1=" 4 * 3 "
$ echo $value1
12
$
```

ksh93 shell 的数学能力中的最有用的特性可能是它可以处理浮点数，而 bash shell 不能处理浮点数：

```
$ let value1=" 10.5 * 0.5 "
$ echo $value1
5.25
$
```

正是这个特性使 ksh93 shell 成为许多 shell 编程人员的宠儿。

如果不使用 `let` 命令，还可以使用双圆括号方法来定义数学运算：

```
$ value1=$(( 4 * 3 ))
$ echo value1
12
$
```

在圆括号中可以执行所需的任意数学运算，使用数字和变量：

```
$ value1=10.25
$ value2=$(( $value1 / 10 ))
$ echo $value2
1.025
$
```

如果要执行无效的运算，ksh93 shell 将通过错误消息报告无效运算：

```
$ value1=0
$ value2=$(( 10.25 / $value1 ))
ksh93: 10.25 / 0 : divide by zero
$
```

除了错误信息，ksh93 shell 还返回一个非零退出状态，表示运算失败。

**2. 数学函数**

ksh93 shell 包含一些内置数学函数，可以在 shell 脚本中使用。ksh93 shell 中内置的数学库包括以下函数。

- `abs(x)`：x 的绝对值。
- `acos(x)`：x 的反余弦值（单位为弧度）。
- `asin(x)`：x 的反正弦值（单位为弧度）。
- `atan(x)`：x 的反正切值（单位为弧度）。
- `atan2(y,x)`：y/x 的反正切值。
- `cos(x)`：x 的余弦值（单位为弧度）。
- `cosh(x)`：x 的双曲余弦值（单位为弧度）。
- `exp(x)`：x 的以 e 为底的指数值。
- `floor(x)`：比 x 小的最大整数值。
- `fmod(x,y)`：x/y 后的浮点余数。
- `hypot(x,y)`：以长度 x 和 y 为直角边的三角形的斜边长度。
- `int(x)`：返回 0 和 x 之间最接近 x 的整数。
- `log(x)`：x 的自然对数值。
- `pow(b,e)`：b 的 e 次幂。
- `sin(x)`：x 的正弦值（单位为弧度）。
- `sinh(x)`：x 的双曲正弦值（单位为弧度）。
- `sqrt(x)`：x 的平方根。

- tan(x)：x 的正切值（单位为弧度）。
- tanh(x)：x 的双曲正切值（单位为弧度）。

有许多函数可用！使用它们只需在 shell 脚本代码中包含函数名和参数：

```
$ cat test1
#!/bin/ksh93
# testing the ksh93 math functions

value1=9
value2=$(( sqt($value1) ))
value3=$(( int(10.52))
value4=$(( sin(45) ))
print "The square root of $value1 is $value2"
print "The integer part of 10.52 is $value3"
print "The sine of 45 is $value4"
$ ./test1
The square root of 9 is 3
The integer part of 10.52 is 10
The sine of 45 is 0.850903524534
$
```

从上面的例子可以看出，ksh93 的数学函数还可以处理浮点数。

## 22.3.2 结构化命令

ksh93 shell 实现了 bash shell 中的所有结构化命令：

- if-then；
- for 循环（包括 c 风格）；
- while 循环；
- until 循环；
- select 语句；
- case 语句。

所有这些结构化命令都使用与 Bourne shell 一样的格式（故与 bash shell 和 ash shell 也一样），因此这里就不详细介绍了。但是，ksh93 shell 提供了更简单的方法在 if-then 语句和 while/until 循环中指定数学运算。下面几节将介绍这些特性。

### 1. if-then-else 语句

if-then 语句使用与 bash shell 相同的格式（请参阅第 9 章）。if-then 语句的基本格式为：

```
if command then
    statements1
else
    statements2
fi
```

与 bash shell 相同，ksh93 shell 的 if-then 语句执行提供的命令，如果返回零退出状态，则继续执行第一个语句集。否则，执行第二个语句集。

ksh93 shell 还支持 elif 语句，允许将 if-then-else 串联起来：

```
if command1 then
   statements1
elif command2 then
   statements2
fi
```

ksh93 的 if-then 语句支持 test 命令（请参阅第 9 章），但不能识别使用单方括号的 test 命令。相反，ksh93 shell 支持使用双圆括号的数学比较，双方括号的非数学比较。数学比较例子如下：

```
$ cat test2
#!/bin/ksh93
```

```
# testing the test command
value1=10
value2=20
if (( $value1 < $value2 )) then
   print "The value $value1 is less than $value2"
else
   print "The value $value1 is greater than $value2"
fi
$ ./test2
The value 19 is less than 20
$
```

可以看到，这与 tcsh shell 类似，在比较数字值时，ksh93 shell 支持标准的数学运算符。

注意：ksh93 还支持 bash 风格的数字比较符（例如-gt，-lt 和-eq），但它们被认为过时了，在以后的 Korn shell 版本中可能会去掉。

ksh93 shell 使用与 bash shell 相同的文件和目录测试，如对于目录使用-d，文件使用-f，可读文件使用-r，可执行文件使用-x。下面是使用目录比较符的例子：

```
$ cat test3
#!/bin/ksh93
# testing the non-math comparison

if [[ -d $HOME ]] && [[ -f $HOME/testing ]] then
   print "The file exists."
else
   print "Sorry, the file doesn't exist."
fi
$ ./test3
Sorry, the file doesn't exist.
$ touch $HOME/testing
$ ./test3
The file exists.
$
```

从该例可以看出，ksh93 shell 支持使用逻辑 AND（&&）符号。还支持使用逻辑 OR（||）。

### 2. while 和 until 循环

while 和 unitl 循环需要一个条件来决定循环的结束时刻。ksh93 shell 允许使用和 if-then 相同的数学比较符来设置条件：

```
$ cat test5
#!/bin/ksh93
# calculating factorials the ksh93 way

value=1
while (( $value <= 10 ))
do
   factorial=1
   counter=1
   while (( $counter <= $value ))
   do
      factorial=$(( $factorial * $counter ))
      counter=$(( $counter + 1 ))
   done
   print "The factorial of $value is $factorial"
   value=$(( $value + 1 ))
done
$ ./test5
The factorial of 1 is 1
The factorial of 2 is 2
The factorial of 3 is 6
The factorial of 4 is 24
The factorial of 5 is 120
```

```
The factorial of 6 is 720
The factorial of 7 is 5040
The factorial of 8 is 40320
The factorial of 9 is 362880
The factorial of 10 is 3628800
$
```

shell 脚本中 while 语句按照预期执行。

### 22.3.3 命令重定向

在 ksh93 shell 中，不能使用反引号将命令的输出重定向到一个变量。而是必须使用下面的格式：

```
variable=$( command )
```

这个格式将命令的输出赋值给一个变量：

```
$ value1=$( date )
$ echo $value1
Mon Jan 14 19:23:14 EST 2008
$
```

当需要从命令的输出中抽取数据时，这个特性在 shell 脚本中非常有用（请参阅第 27 章获取更多信息）。

### 22.3.4 规程函数

与命令重定向相关的 ksh93 shell 特性是**规程**（discipline）函数。规程函数是对一个变量进行操作的函数。有 3 种基本规程函数。

- ✦ get：获取函数变量的值。
- ✦ set：设置函数变量的值。
- ✦ unset：重置函数变量的值。

定义规程函数与定义常规函数的方法类似（ksh93 shell 使用 function 命令，与 bash shell 的方式类似）。规程函数的关键是返回结果的方式。

如果要对变量使用 get 类型的规程函数，而不使用 return 命令，那么规程函数使用特殊的 .sh.value 变量。这样就可以像使用变量一样使用函数，每次引用规程函数名称时，都会返回存储在 .sh.value 变量中的值。

下面是一个定义和使用规程函数的例子：

```
$ function date.get {
> .sh.value=$( date +'%A, %B %d, %Y' )
> }
$
```

现在每当获取 date 变量值时，date 变量将返回格式化的日期。

```
$ echo $date
Monday, January 14, 2008
$
```

使用 set 类型的规程函数来自动改变赋值语句中变量值的设置方式：

```
$ function sqrt.set {
> .sh.value=$(( sqrt(${.sh.value}) ))
}
$
```

现在无论何时对 sqrt 变量赋值，规程函数都会自动计算这个值的平方根，并将结果赋值给变量：

```
$ sqrt=9
$ echo $sqrt
3
$ sqrt=16
$ echo $sqrt
4
$
```

这是创建简单函数的好方法，在交互式 shell 和 shell 脚本中都是如此。

注意：也许您会奇怪，ksh93 shell 也支持常规函数。它们使用与 bash shell 中函数相同的格式，或者使用 function 命令，或者使用双圆括号来定义函数名称。

## 22.4　小结

本章介绍了 Korn shell 并讨论了一种最流行的 Korn shell 实现。ksh93 shell 是原始 Korn shell 的一个高级版本，提供了许多有趣的特性。

ksh93 shell 允许设置变量类型，定义变量可以存储的数据类型以及数据处理方式。通过设置变量类型，可以修改 shell 显示数值的方式（如按进制显示），还可以修改文本存储的方式（如全部大写或全部小写）。

在 ksh93 shell 中可以创建数字和关联数组变量。关联数组允许向数据值分配字符串键。要调用数据值，只要引用数组中的字符串键就可以。复合变量的使用与此类似，可以将多个值赋给一个变量，并使用数组下标引用这些值。

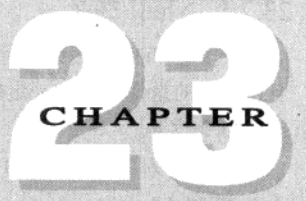
ksh93 shell 还提供大量的内置命令。一个便捷的命令是 printf 命令，使用这个命令可以利用 C 风格的 printf 格式化特性来格式化输出。

ksh93 shell 的一个突出特点是支持数学特性。ksh93 shell 能够执行数学运算，使用标准 C 风格数学符号，甚至包括许多高级的内置数学函数，例如平方根，绝对值和标准三角函数。对于许多编程人员来说，ksh93 shell 的卖点就是它对浮点数的完全支持。在 shell 脚本中可以像处理整数值那样直接处理浮点值。

最后，本章讨论 ksh93 shell 特有的一个特性，即规程函数。规程函数允许修改变量设置的方式，以及引用函数时返回值的方式。通过覆盖默认行为，可以将任意类型的函数赋值给一个变量。当引用变量时，ksh93 shell 就会自动执行这个函数并在函数执行完成后赋予正确的值。

下一章将讨论 Linux shell 旅程中的最后一种 shell，即 Z shell（zsh）。Z shell 是一种最新的 shell，其中包含了很多高级特性。

# 第 23 章
# zsh shell




作为讨论各种 Linux shell 的最后一章，本章将着眼于最新的 shell，即 zsh shell。zsh shell 提供一些神奇的特性，并将 shell 未来发展的门槛提高到相当高的高度。本章将讨论 zsh shell 如此独特的原因，并介绍可以在 zsh shell 脚本中使用的各种特性。

**本章内容：**

- zsh shell 介绍；
- zsh shell 的组成部分；
- 编写 zsh 脚本。

## 23.1 zsh shell 的历史

Z shell（称为 zsh）是 Paul Falstad 开发的开源 Unix shell。它是集合了 Bourne、bash、ash 和 tcsh 等 shell 的特性，并加入许多特有的特性，以编程人员使用为目的而创建的一种成熟的高级 shell。

其中一些特性使得 zsh shell 独具特色：

- 改良的 shell 选项控制；
- shell 兼容性模式；
- 可载入模块。

这些特性中，可载入模块特性在 shell 设计中思想最先进。在前面介绍 shell 的各章中可以看到，每种 shell 都包含一个不需要外部工具程序就可以使用的内置命令集。内置命令的好处是提高执行速度。shell 不必像在运行工具程序前必须将工具程序载入到内存中那样载入内置命令，内置命令早已经存在于 shell 内存当中，时刻准备着。

zsh shell 提供一个核心内置命令集，还提供添加附加**命令模块**（command modules）的功能。每个命令模块都提供一个针对特殊环境的附加内置命令集，如网络支持和高级数学函数。用户可以根据具体情况，只添加需要的模块。

这个特性既能够在需要小体积 shell 和较少命令的情况下很好的限制 zsh shell 的大小，又能够在需要较快的执行速度的情况下扩充可用内置命令的数目。

## 23.2 zsh shell 的组成部分

本节将介绍 zsh shell 的基础知识，讨论可用的（或可以通

过安装模块添加的）内置命令，以及 zsh shell 使用的命令行参数和环境变量。

### 23.2.1　shell 选项

大多数 shell 使用命令行参数来定义 shell 的行为。zsh shell 使用一些命令行参数定义 shell 的操作，但它更多的是使用**选项**来定制 shell 的行为。shell 选项可以通过命令行设置，也可以在 shell 内部使用 `set` 命令设置。

表 23-1 列出了 zsh shell 可以使用的命令行参数。

**表 23-1　　zsh shell 命令行参数**

| 参数 | 描述 | 参数 | 描述 |
|---|---|---|---|
| `-c` | 只执行指定的命令并退出 | `-s` | 强制 shell 从 `STDIN` 读取命令 |
| `-i` | 作为交互式 shell 启动，提供一个命令行界面提示 | `-o` | 指定命令行选项 |

虽然这些命令行参数看起来很少，但其中参数`-o` 很容易让人误解。使用参数`-o` 可以对定义 shell 内部特性的 shell 选项进行设置。zsh shell 是目前自定义功能最强的 shell。shell 环境的许多特性都可以改变。可以将选项划分为以下几类。

- **改变目录**：控制 `cd` 和 `dirs` 命令如何处理目录改变的选项。
- **完成**：控制命令完成特性的选项。
- **扩充和缩小**：控制命令中文件扩展的选项。
- **历史**：控制命令历史调用的选项。
- **初始化**：控制启动时 shell 如何处理变量和启动文件的选项。
- **输入/输出**：控制命令处理的选项。
- **作业控制**：指示 shell 如何处理和启动作业的选项。
- **提示**：定义 shell 如何处理命令行提示的选项。
- **脚本和函数**：控制 shell 如何处理 shell 脚本和定义 shell 函数的选项。
- **shell 模拟**：允许设置 zsh shell 模拟其他类型 shell 的行为的选项。
- **shell 状态**：定义 shell 的启动类型的选项。
- **zle（zsh line editor）**：控制 zsh 行编辑器特性的选项。
- **选项别名**：可用作其他选项名别名的特殊选项。

通过这些 shell 选项分类，可以想象一下 zsh shell 实际上支持多少选项。下面几节将介绍定制 zsh shell 环境时可以使用的一些 zsh shell 选项。

#### 1. shell 状态

定义 shell 启动类型的 zsh shell 选项有六个。

- interactive (`-i`)：为输入内置命令和程序名提供一个命令行界面提示。
- login (`-l`)：zsh shell 的默认类型，处理 zsh shell 启动文件，并提供一个命令行界面提示。
- privileged (`-p`)：用户的有效用户 ID（UID）与实际的 UID 不同（即用户成为根用户）时，使用的默认选项。该选项关闭用户启动文件。
- restricted (`-r`)：将用户限制在指定的 shell 目录结构中。
- shin_stdin (`-s`)：从 `STDIN` 读取命令。
- single_command (`-t`)：执行来自 `STDIN` 的单个命令并退出。

shell 状态定义 shell 启动时是否带有命令行界面提示，以及用户在 shell 内的访问范围。

#### 2. shell 模拟

使用 shell 模拟选项可以对 zsh shell 进行定制，使其模拟 csh 或 ksh shell 的执行方式。选项包括以下几种。

- **bsd_echo**：使 `echo` 语句与 C shell 的 echo 命令兼容。

- **csh_junkie_history**：使用无说明符的 `history` 命令引用上一个命令。
- **csh_junkie_loops**：允许 `while` 和 `for` 循环中使用 C shell 中的 `end` 而不是 `do` 和 `done`。
- **csh_junkie_quotes**：将单引号和双引号的使用规则改为与 C shell 匹配。
- **csh_nullcmd**：执行没有命令的重定向时，不使用 `NULLCMD` 和 `READNULLCMD` 变量的值。
- **ksh_arrays**：使用 Korn 风格的数组，数字数组起始索引为 0，使用大括号引用数组元素。
- **ksh_autoload**：模仿 Korn shell 的自动载入函数特性。
- **ksh_option_print**：模仿 Korn shell 打印选项的方式。
- **ksh_typeset**：改变处理 `typeset` 命令参数的方式。
- **posix_builtins**：使用 `builtin` 命令执行内置命令。
- **sh_file_expansion**：执行文件名扩展先于其他扩展。
- **sh_nullcmd**：执行重定向时不使用 `NULLCMD` 和 `READNULLCMD` 变量。
- **sh_option_letters**：按照 Korn shell 的方式解释单字母 shell 命令行选项。
- **sh_word_split**：对未引起来的参数扩充执行字段分割。
- **traps_async**：等待程序退出的同时，处理信号并立即执行捕获。

由于提供了多个选项，可以根据需要只选择需要在 zsh shell 中模拟的那些 csh shell 或 ksh shell 特性，而不必全部模拟。

**3. 初始化**

下面是控制 shell 启动特性的选项。

- **all_export**：将所有参数和变量都自动导出到子 shell 进程。
- **global_export**：导出到环境中的参数不对局部函数起作用。
- **global_rcs**：如果不设置，则 zsh shell 不运行全局的启动文件，但仍然运行本地启动文件。
- **rcs**：如果不设置，则 zsh shell 运行 `/etc/zshenv` 启动文件，但不运行其他文件。

使用初始化选项可以指定在 shell 环境中运行哪些 zsh shell 启动文件。也可以在启动文件中设置这些值来限制 shell 执行的文件。

**4. 脚本和函数**

通过脚本和函数选项可以定制 zsh shell 中的 shell 脚本编写环境。对于设置 shell 中函数的执行方式这很方便。

- **c_bases**：以 C 格式（`0xdddd`）显示十六进制数，而不使用 shell 格式（`16#dddd`）。
- **err_exit**：如果命令以非零退出状态退出，则在 ZERR 捕获中执行命令并退出。
- **err_return**：如果命令以非零退出状态退出，则立即从嵌套的函数返回。
- **eval_lineno**：如果设置，则将使用 `eval` 内置命令求值的表达式的行数同其他 shell 环境分开，进行独立跟踪。
- **exec**：执行命令。如果没有设置该选项，则读取命令并报告错误，但不执行命令。
- **function_argzero**：将 `$0` 设置为函数或脚本的名称。
- **local_options**：如果设置，则当一个 shell 函数返回时，恢复执行该函数前设置的所有选项。
- **local_traps**：如果设置，则当在函数中设置了一个信号捕获时，在函数退出时恢复捕获原来的状态。
- **multios**：当尝试进行多重定向时执行明确的 tee 或 cat。
- **octal_zeroes**：将所有以 0 开头的整数字符串视为八进制数。
- **typeset_silent**：如果不设置，则使用 `typeset` 加参数名来显示当前参数值。
- **verbose**：shell 读取输入行时显示 shell 输入行。
- **xtrace**：shell 执行命令时显示命令及其参数。

zsh shell 允许定制多种退出 shell 中定义的函数时执行的特性。

## 23.2.2　zsh shell 文件

zsh shell 在登录和退出时都使用文件来预置 zsh shell 的变量和特性。zsh shell 自动搜索一些默认文件来设置环境变量和执行命令。如果存在某个默认文件，zsh shell 会根据具体情况执行文件。zsh shell 能够处理 4 种类型的默认文件：

- shell 启动文件；
- 登录文件；
- 交互式文件；
- 退出文件。

本节将介绍可能在 zsh shell 中使用到的各种文件。

### 1. shell 启动文件

当启动一个新 zsh shell 作为登录 shell 时（或者通过登录系统，或者通过运行 zsh shell 程序），zsh shell 搜索两个文件中的命令：

默认的系统范围 zsh shell 启动文件：

```
/etc/zshenv
```

执行这个文件后（如果该文件存在的话），zsh shell 继续搜索用户的$HOME 目录查找下面的文件：

```
$HOME/.zshenv
```

可以通过在每个文件中使用 echo 语句执行一个简单的测试来测试 zsh shell 执行这些文件的顺序：

```
% zsh
This is the /etc/zshenv file.
This is the .zshenv file in HOME.
%
```

启动一个新 zsh shell 时，它先执行/etc/zshenv 文件，然后执行$HOME 目录下的.zshenv 文件。

如果 shell 脚本要使用 zsh shell，则 shell 启动文件也执行：

```
% cat test1
#!/bin/zsh
# test run a script
echo "This is a test"
% ./test1
This is the /etc/zshenv file.
This is the .zshenv file in HOME.
This is a test
%
```

注意，在显示 shell 脚本的输出前，可以看到 zsh shell 启动文件中的 echo 语句。

注意：一些 Linux 发行版（如 Debian）在/etc 目录下创建一个 zsh 目录来包含所有 zsh shell 启动文件，而不是将它们直接放在/etc 文件夹中。

### 2. shell 登录文件

如果使用 zsh shell 作为登录 shell，它首先搜索并执行两个文件：

- /etc/zlogin；
- /etc/zprofile。

对于所有使用 zsh shell 作为登录 shell 的用户都会执行这些文件。这两个文件应当包含为所有用户设置的系统范围的环境变量，所有用户都可能会使用的可执行程序（如设置默认文件和目录权限的 umash 命令）。大多数情况下，如果使用的 Linux 发行版支持 zsh shell，那这些在登录时设置系统环境变量的文件已经存在。

执行这些文件后，zsh shell 在用户的$HOME 目录中搜索下面两个文件：

- $HOME/.zlogin；

- $HOME/.zrofile。

这两个文件可以包含用户特定的环境变量设置，和某个用户想要在登录系统时命令行界面提示出现之前执行的可执行命令。

在$HOME/.zprofile 中设置的环境变量设置将覆盖/etc/zprofile 文件中设置的系统范围设置，这与.zlogin 文件类似。可以测试 zsh shell 执行这些文件的顺序，通过在每个文件中添加简单的 echo 语句，然后登录：

```
% zsh -l
This is the /etc/zshenv file.
This is the .zshenv file in HOME.
This is the /etc/zprofie file.
This is the .zprofile file in HOME.
This is the /etc/zlogin file.
This is the .zlogin file in HOME.
%
```

从该例可以看出，zsh shell 先执行 zshenv 文件，然后执行 zprofile 文件，最后执行 zlogin 文件。

**3. 交互式 shell 文件**

如果启动一个交互式 zsh shell 会话，还会涉及以下一些包含启动变量和命令的文件：

- /etc/zshrc；
- $HOME/.zshrc。

启动交互式 shell 时，在启动文件后 zsh shell 立即执行这些文件：

```
% zsh
This is the /etc/zshenv file.
This is the .zshenv file in HOME.
This is the /etc/zshrc file.
This is the .zshrc file in HOME.
%
```

因为登录 shell 也是交互式 shell，所以 zsh shell 也执行这个 zshrc 文件中的内容：

```
% zsh -l
This is the /etc/zshenv file.
This is the .zshenv file in HOME.
This is the /etc/zprofie file.
This is the .zprofile file in HOME.
This is the /etc/zshrc file.
This is the .zshrc file in HOME.
This is the /etc/zlogin file.
This is the .zlogin file in HOME.
%
```

注意 zsh shell 执行这些文件的顺序。zshrc 文件对排在 zprofile 文件之后，但在 zlogin 文件之前。

**4. shell 退出文件**

除了登录文件和启动文件，zsh shell 还具有在退出交互式或登录 shell 会话时执行文件中的命令的能力。这些文件可以位于以下位置：

- /etc/zlogout；
- $HOME/.zlogout。

现在也许可以发现，系统上的每个用户在退出时都执行/etc/zlogout 文件中的命令，而每个用户都拥有各自的$HOME/.zlogout 文件。zsh shell 将在退出当前 shell 前执行这些文件中的命令。

这不包括只从交互式 shell 中退出。因此，如果在现有 shell 会话中启动另一个 zsh shell，那么退出 shell 时，zsh shell 不会执行退出文件。但是，如果启动一个登录 shell，退出时会执行退出文件：

```
% zsh -l
```

```
This is the /etc/zshenv file.
This is the .zshenv file in HOME.
This is the /etc/zprofie file.
This is the .zprofile file in HOME.
This is the /etc/zshrc file.
This is the .zshrc file in HOME.
This is the /etc/zlogin file.
This is the .zlogin file in HOME.
% exit
This is the .zlogout file in HOME.
This is the /etc/zlogout file.
%
```

注意，shell 在执行全局的 zlogout 文件之前执行.zlogout 文件，与执行其他类型的 zsh 文件的顺序相反。

### 23.2.3　环境变量

与其他 shell 一样，zsh 提供标准环境变量来跟踪系统信息和 shell 信息。zsh 使用大写变量名和小写变量名相结合的方式。

zsh shell 为一些环境变量既提供了小写版本也提供了大写版本。这是因为 zsh shell 处理数组的方式，以及为了兼容其他 shell。

zsh shell 使用空格分隔的词列表来定义数组。许多源自 Bourne shell 的 shell 使用冒号来分隔多值变量中的数组元素，如 bash shell 中的 PATH 变量。

为了维护兼容性，zsh shell 提供一个小写的 path 变量，该变量使用 zsh 风格的数组，一个大写的 PATH 变量，使用 Bourne 风格的数组。表 23-2 列出了 zsh shell 中的环境变量。

表 23-2　　zsh 环境变量

| 变量 | 描述 |
|---|---|
| ! | 上一个后台命令的进程 ID |
| # | 命令行中指定的参数的数目 |
| $ | shell 的进程 ID |
| * | 包含命令行位置参数的数组 |
| - | 命令行中设置的选项 |
| argv | 包含命令行位置参数的数组 |
| ARGV0 | 外部命令中 argv[0]的值，通常是命令名称 |
| BAUD | 当前连接的波特率 |
| cdpath | cd 命令中用作搜索路径的目录数组 |
| CDPATH | 同 cdpath，但使用冒号作为字段分隔符 |
| COLUMNS | 终端会话中的列数 |
| CPUTYPE | 系统的微处理器类型或型号 |
| DIRSTACKSIZE | 目录栈的最大大小 |
| EGID | shell 进程的有效组 ID |
| ENV | 如果 shell 作为 Bourne 或 Korn shell 调用，则包含 zprofile 文件的位置 |
| ERRNO | 最近失败的系统调用的错误状态 |
| EUID | shell 进程的有效用户 ID |
| FCEDIT | 命令历史编辑器使用的默认编辑器 |
| fignore | 文件完成期间忽略的文件后缀数组 |
| FIGNORE | 同 fignore，但使用冒号作为分隔符 |
| fpath | 指定函数的搜索路径的目录数组 |
| FPATH | 同 fpath，但使用冒号作为分隔符 |

续表

| 变量 | 描述 |
| --- | --- |
| GID | shell 进程的真正组 ID |
| histchars | 历史命令使用的字符 |
| HISTCMD | 当前历史行号 |
| HISTFILE | 退出交互式 shell 时存储历史的文件 |
| HISTSIZE | 历史文件中允许的最大事件数目 |
| HOME | 用户的默认工作目录 |
| HOST | 当前主机名称 |
| IFS | 内部文件分割符，默认为空格、制表符、换行符和 NUL 字符 |
| KEYTIMEOUT | 读取多字符序列时，shell 等待下一按键的时间（以百分之一秒计算） |
| LANG | 未被 LC_变量覆盖的 locale 类别 |
| LC_ALL | 覆盖所有其他 locale 类别定义的 locale 类别 |
| LC_COLLATE | 用于决定字符校对的 locale 类别 |
| LC_CTYPE | 用于决定字符处理函数的 locale 类别 |
| LC_MESSAGES | 用于显示消息的 locale 类别 |
| LC_NUMERIC | 用于显示十进制小数点和千分隔符字符的 locale 类别 |
| LC_TIME | 用于显示日期和时间的 locale 类别 |
| LINES | 终端会话中的行数 |
| LINENO | 脚本或 shell 函数中当前命令的行数 |
| LISTMAX | 在请求更多匹配前，行编辑器中列出的匹配数目 |
| LOGCHECK | 使用 watch 参数检查不活动状态的间隔秒数 |
| LOGNAME | 与当前 shell 会话相对应的登录名称 |
| MAIL | 如果设置，则 shell 将在指定的文件中检查邮件 |
| MAILCHECK | 邮件检查的间隔秒数 |
| mailpath | 邮件检查的文件名数组 |
| MAILPATH | 同 mailpath，但使用冒号作为分隔符 |
| manpath | zsh shell 不使用的目录数组（与 Bourne shell 中的 MANPATH 对应） |
| MANPATH | 同 manpath，但使用冒号作为分隔符 |
| module_path | zmodload 用来载入新 zsh 模块的目录数组 |
| MODULE_PATH | 同 module_path，但使用冒号作为分隔符 |
| NULLCMD | 如果指定重定向时没有附加命令，则假定为该命令名称。默认值为 cat 命令 |
| path | 搜索命令的目录数组 |
| PATH | 同 path，但使用冒号作为分隔符 |
| POSTEDIT | 退出行编辑器时显示的字符串 |
| PROMPT | shell 使用的主提示字符串 |
| PROMPT2 | 当命令需要更多输入时使用的次提示字符串 |
| PROMPT3 | 选择循环中使用的选择提示 |
| PROMTP4 | 跟踪命令时使用的跟踪提示 |
| prompt | 同 PROMPT |
| PS1 | 同 PROMPT |
| PS2 | 同 PROMPT2 |
| PS3 | 同 PROMPT3 |
| PS4 | 同 PROMPT4 |
| psvar | 一个数组，它的前 9 个值分别用相应的提示字符串引用 |
| PSVAR | 同 psvar，但使用冒号作为分隔符 |
| RANDOM | 随机数生成器，范围为 0～32767 之间的整数 |

续表

| 变量 | 描述 |
|---|---|
| READNULLCMD | 如果指定输入重定向时没有附加命令，则假定为该命令名称。默认值为 more 命令 |
| REPORTTIME | 报告时间统计信息的命令数组 |
| reply | 用于在 shell 脚本和函数之间传送字符串值的保留变量 |
| REPLY | 同 reply，但使用数组值，而不使用字符串值 |
| RPROMPT | 显示在命令行界面右侧的提示 |
| RPS1 | 同 RPROMPT |
| RPROMPT2 | 当命令需要更多输入信息时，显示在命令行界面右侧的提示 |
| RPS2 | 同 RPROMPT2 |
| SAVEHIST | 历史文件中保存的事件的最大数量 |
| SECONDS | shell 创建以来的秒数 |
| SHLVL | 调用的子 shell 层级 |
| SPROMPT | 拼写校正提示 |
| STTY | 用于为 shell 设置 TTY 会话的值 |
| TERM | 用于 shell 会话的终端类型 |
| TIMEFMT | 处理时间请求所使用的格式 |
| TMOUT | 如果没有输入命令，在发出警告前等待的时间（以秒计算） |
| TMPREFIX | 所有临时文件的路径名前缀，默认为/tmp/zsh |
| USERNAME | 与 shell 进程的实际用户 ID 相对应的用户登录名 |
| watch | 报告登录和退出事件的数组 |
| WATCH | 同 watch，但使用冒号作为分隔符 |
| WATCHFMT | 设置 watch 变量情况下，生成的报告的格式 |
| WORDCHARS | 行编辑器视为是单词的组成部分的非字母字符列表 |
| ZBEEP | 发送给显示器的一串字符，代替生成 beep 警告 |
| ZDOTDIR | 搜索 zsh 私有启动文件的目录默认为$HOME 目录 |

从表 23-2 中可以看出，zsh shell 的环境变量中提供一些特有的特性。我喜欢的一个是右提示(RPROMPT)变量。

RPROMPT 变量在命令行界面的右侧显示提示：

```
% RPROMPT='%d'
%                                                    /home/test
% cd test                                           /home/test
% %                                                /home/test/test
```

如果最右边不需要显示文本，则右提示在最右边显示定义的提示(在本例中设置为当前工作路径)。如果 shell 需要显示文本，就不显示右提示。

zsh shell 支持 typeset 命令，该命令可以在变量使用前声明变量的属性。表 23-3 给出了 zsh 的 typeset 命令可以使用的选项。

表 23-3　typeset 命令参数

| 参数 | 描述 |
|---|---|
| -a | 创建数值数组变量 |
| -A | 创建关联数组变量 |
| -E | 创建双精度浮点变量，并使用科学记数法显示 |
| -f | 定义一个函数名，而不是变量 |
| -F | 创建双精度浮点变量，并使用固定小数位数法显示 |
| -h | 创建特殊的隐藏变量 |
| -H | 不显示变量的值 |

续表

| 参数 | 描述 |
| --- | --- |
| -i | 创建整数数据类型变量 |
| -l | 将变量值转成小写 |
| -L | 左对齐，删除变量前面的空格字符 |
| -r | 使指定的变量成为只读变量 |
| -R | 在变量左面添加空格，右对齐 |
| -t | 标记指定的变量 |
| -u | 将变量值改为大写 |
| -U | 只保留数值数组中的第一个重复值 |
| -x | 标记指定的变量以自动导出 |
| -Z | 右对齐，前面补零 |

从 typeset 命令参数中可以看到，zsh shell 支持数值数组和关联数组。如果习惯于使用 bash 和 ksh shell 中的数值数组，那么在 zsh shell 中使用时要多加小心。zsh shell 的起始索引值是 1 而不是 0。

```
% mytest=(one two three four)
% echo ${mytest[1]}
one
% echo ${mytest[2]}
two
%
```

在 zsh shell 创建关联数组也略有不同。必须首先使用 typeset 命令进行声明：

```
% typest -A test
%
```

声明变量为关联数组后，可以定义键和值。这是不同之处。在 zsh shell 中，要交替列出键和值：

```
variable=( key1 value1 key2 value2 ...)
```

因此，向 test 数组中添加数据：

```
$ test=( fruit banana vegetable carrot age 18 )
$ echo ${test[fruit]}
banana
$ echo ${test[age]}
18
$
```

因为这个语法，zsh 关联数组的每个键后必须分配一个值。

### 23.2.4 内置命令

zsh shell 很独特，允许扩展 shell 中的内置命令。从而能够为多个不同应用提供充足的便捷高效的工具。本节将介绍核心内置命令，和截至本书编写时可用的各种模块。

#### 1. 核心内置命令

zsh shell 的核心包括在其他 shell 中已经看到过的基本内置命令。表 23-4 给出了可用的内置命令。

表 23-4 zsh shell 的核心内置命令

| 命令 | 描述 |
| --- | --- |
| alias | 为命令和参数定义别名 |
| autoload | 将 shell 函数预先加载到内存，以获得更快的访问 |
| bg | 以后台模式执行某个作业 |
| bindkey | 为命令绑定组合键 |
| builtin | 执行指定的内置命令，而不是执行同名的可执行文件 |
| bye | 同 exit |
| cd | 改变当前的工作目录 |
| chdir | 改变当前的工作目录 |

续表

| 命令 | 描述 |
| --- | --- |
| command | 将指定的命令视为外部文件来执行，而不是作为函数或内置命令执行 |
| declare | 设置变量的数据类型（同 typeset） |
| dirs | 显示目录栈的内容 |
| disable | 临时禁用指定的哈希表元素 |
| disown | 将指定的作业从作业表中删除 |
| echo | 显示变量和文本 |
| emulate | 设置 zsh 模拟其他 shell，如 Bourne、Korn 或 C shell |
| enable | 启用指定的哈希表元素 |
| eval | 在当前 shell 进程中执行指定命令和参数 |
| exec | 执行指定命令和参数，替换当前 shell 进程 |
| exit | 以指定的退出状态退出 shell；如果没有指定，则使用前一个命令的退出状态 |
| export | 允许子 shell 进程使用指定的环境变量名和值 |
| false | 返回 1 退出状态 |
| fc | 从历史列表中选择一个命令范围 |
| fg | 以前台模式执行指定的作业 |
| float | 将指定的变量设置为浮点类型变量 |
| functions | 设置指定的名称为函数名称 |
| getln | 读取缓存堆栈中的下一个值，将它赋给指定的变量 |
| getopts | 获取命令行参数中的下一个有效选项，赋值给指定的变量 |
| hash | 直接修改命令哈希表中的内容 |
| history | 列出历史文件中包含的命令 |
| integer | 设置指定的变量为整数类型 |
| jobs | 列出指定作业或者赋予 shell 进程的所有作业的相关信息 |
| kill | 发送信号（默认信号为 SIGTERM）给指定的进程或作业 |
| let | 计算某个数学运算的值并将结果赋值给一个变量 |
| limit | 设置或显示资源限制 |
| local | 为指定变量设置数据特性 |
| log | 显示受 watch 影响的当前登录的所有用户 |
| logout | 同 exit，但只有当 shell 作为登录 shell 时有效 |
| popd | 从目录堆栈中删除下一个条目 |
| print | 显示变量和文本 |
| printf | 使用 C 风格格式字符串显示变量和文本 |
| pushd | 修改当前工作目录，将上一个目录存储在目录堆栈中 |
| pushln | 将指定的参数添加到编辑缓存堆栈中 |
| pwd | 显示当前工作目录的完整路径名 |
| read | 使用 IFS 字符读取行并将数据字段赋值给指定的变量 |
| readonly | 将值赋给不可变变量 |
| rehash | 重新生成命令哈希表 |
| set | 为 shell 设置选项或位置参数 |
| setopt | 为 shell 设置选项 |
| shift | 读取并删除第一个位置参数，然后将剩下的参数向左移一个位置 |
| source | 查找指定文件，并将它的内容复制到当前位置 |
| suspend | 挂起 shell 的执行，直到接收到 SIGCONT 信号 |
| test | 如果指定条件为 TRUE，则返回 0 退出状态 |
| times | 为 shell 和 shell 中执行的进程显示累积用户和系统时间 |

续表

| 命令 | 描述 |
| --- | --- |
| trap | 阻止 shell 处理指定的信号，如果收到信号则执行指定的命令 |
| true | 返回 0 退出状态 |
| ttyctl | 锁定或解锁显示 |
| type | 显示 shell 如何解释指定的命令 |
| typeset | 设置或显示变量属性 |
| ulimit | 设置或显示 shell 或 shell 中运行的进程的资源限制 |
| umask | 设置或显示创建文件和目录的默认权限 |
| unalias | 删除指定的命令别名 |
| unfunction | 删除指定的定义函数 |
| unhash | 从哈希表中删除指定的命令 |
| unlimit | 删除指定的资源限制 |
| unset | 删除指定的变量属性 |
| unsetopt | 删除指定的 shell 选项 |
| wait | 等待指定的作业或进程完成 |
| whence | 显示 shell 如何解释指定的命令 |
| where | 若 shell 找到指定命令的路径，则显示指定命令的路径名 |
| which | 使用 csh 风格的输出显示指定命令的路径名 |
| zcompile | 编译指定的函数或脚本，以获得更快的自动加载 |
| zmodload | 对可载入 zsh 模块进行操作 |

zsh shell 在提供内置命令上没有丝毫懈怠。可以到看这些命令中的大多数在 bash 中都有对应。zsh shell 内置命令最重要的特点是模块。

### 2. 添加模块

模块的数量有很多，为 zsh shell 提供了大量的附加内置命令，而且随着编程人员不断的创建新模块，模块数量还在一直增加。表 23-5 给出了截至本书编写时可用的各种模块。

表 23-5 zsh 模块

| 模块 | 描述 |
| --- | --- |
| zsh/cap | POSIX 兼容性命令 |
| zsh/clone | 将某个运行中的 shell 克隆到另一个终端的命令 |
| zsh/compctl | 控制命令完成的命令 |
| zsh/complete | 命令行完成命令 |
| zsh/complist | 命令行完成列表扩充命令 |
| zsh/computil | 命令行完成相关的工具命令 |
| zsh/datetime | 附加的日期和时间命令与变量 |
| zsh/deltochar | 行编辑器函数，功能上与 emacs 相同 |
| zsh/files | 基本文件处理命令 |
| zsh/mapfile | 通过关联数组访问外部文件 |
| zsh/mathfunc | 附加的科学函数 |
| zsh/parameter | 通过关联数组访问命令哈希表 |
| zsh/pcre | 扩充的正则表达式库 |
| zsh/sched | 为执行时控命令调度命令 |
| zsh/net/socket | Unix 域套接字支持 |
| zsh/stat | 访问提供系统统计信息的统计系统 |
| zsh/system | 各种低级系统特性接口 |

续表

| 模块 | 描述 |
| --- | --- |
| zsh/net/tcp | 访问 TCP 套接字 |
| zsh/termcap | 与 termcap 数据库的接口 |
| zsh/terminfo | 与 terminfo 数据库的接口 |
| zsh/zftp | 专门的 FTP 客户端命令 |
| zsh/zle | zshell 行编辑器 |
| zsh/zleparameter | 使用变量访问修改行编辑器 |
| zsh/zprof | 允许为 shell 函数创建配置文件 |
| zsh/zpty | 在虚拟终端上启动一个命令 |
| zsh/zselect | 当文件描述符就绪时中断并返回 |
| zsh/zutil | 各种 shell 工具 |

zsh shell 模块覆盖的领域很广，从提供简单的命令行编辑器功能到高级的网络函数。zsh shell 的主要思想是提供一个最小的基础的 shell 环境，并可以根据需要继续添加模块来完成实际编程工作。

### 3. 查看、添加和删除模块

zmodload 命令是对 zsh 模块的接口。使用这个命令可以查看、添加和删除 zsh shell 会话中的模块。

使用 zmodload 命令，不加任何命令行参数，会显示当前安装在 zsh shell 中的模块：

```
% zmodload
zsh/zutil
zsh/complete
zsh/main
zsh/terminfo
zsh/zle
zsh/parameter
%
```

不同的 zsh shell 实现默认包含不同的模块。要添加新模块，只要在 zmodload 命令行中指定模块名称即可：

```
% zmodload zsh/zftp
%
```

看不到表示模块已加载完成的信息。可以再执行一次 zmodload 命令，新模块就会显示在已安装模块列表中。

载入一个模块后，就可以像使用内置命令那样使用与该模块相关的命令：

```
% zftp open myhost.com rich testing1
Welcome to the myhost FTP server.
% zftp cd test
% zftp dir
01-21-08 11:21PM    120823 test1
01-21-08 11:23PM    118432 test2
% zftp get test1 > test1.txt
% zftp close
%
```

使用 zftp 命令可以直接通过 zsh shell 命令行引入一个完整的 FTP 会话。可以将这些命令结合到 zsh shell 脚本中直接从脚本执行文件传输。

要删除一个已安装模块，使用-u 参数，后面加模块名称：

```
% zmodload -u zsh/zftp
% zftp
zsh: command not found: zftp
%
```

可以通过标准的 zsh shell 启动文件来管理模块（请参阅“zsh shell 文件”一节）。

注意：常见的做法是在$HOME/.zshrc 文件中放置 zmodload 命令，这样所需的函数就会自动载入。

## 23.3 使用 zsh 编写脚本

zsh shell 的主要目的是为 shell 编程人员提供一个高级的编程环境。以此为目标，无疑 zsh shell 提供了许多特性使 shell 编程更容易。

### 23.3.1 数学运算

像期待的那样，zsh shell 能够轻松地执行数学函数。同 ksh93 shell 类似（请参阅第 22 章），zsh shell 提供对浮点数的所有数学运算的完全支持。

**1. 执行计算**

zsh shell 支持和 ksh93 shell 同样的两种处理数学运算的方法：

- let 命令；
- 双圆括号参数。

使用 let 命令时，需要将运算包含在双引号中才允许有空格：

```
% let value1=" 4 * 5.1 / 3.2 "
% echo $value1
6.3749999999999991
%
```

注意，使用浮点数会带来精度问题。为了解决这个问题，通常使用 printf 命令，指定需要的小数精度来正确显示结果：

```
% printf "%6.3f\n" $value1
6.375
%
```

现在就好很多了！

第二种方法是使用双圆括号。这种方法结合了两种定义数学运算的技术：

```
% value1=$(( 4 * 5.1 ))
% (( value2 = 4 * 5.1 ))
% printf "%6.3f\n" $value1 $value2
20.400
20.400
%
```

注意，可以将双圆括号加在运算式两边（由美元符引导）也可以加在整个赋值语句两边。两种方法的结果相同。

如果不使用 typeset 命令在使用变量前声明变量的数据类型，zsh shell 会自动赋予数据类型。当既处理整数又处理浮点数时，这将会非常危险。观察下面的例子：

```
% value=10
% value2=$(( $value1 / 3 ))
% echo $value2
3
%
```

上面的计算结果可能不是所希望的结果。当指定数值但没有指定小数点位置时，zsh shell 会将它们视为整数值，执行整数运算。为了确保结果是浮点数，必须指定数值的小数点位置：

```
% value=10.
% value2=$(( $value1 / 3. ))
% echo $value2
3.3333333333333335
%
Now the result is in the floating point format.
```

**2. 数学函数**

在 zsh shell 中，内置数学函数既丰富又匮乏。默认的 zsh shell 不包含任何特殊的数学函数。但是如果安装 zsh/mathfunc 模块，将获得更多的数学函数，其中很多函数可能都用不到：

```
% value1=$(( sqrt(9) ))
zsh: unknown function: sqrt
% zmodload zsh/mathfunc
% value1=$(( sqrt(9) ))
% echo $value1
3.
%
```

非常简单。现在就拥有一个完整的数学函数库。

注意：要查看 zsh/mathfunc 模块提供的所有数学函数列表，请参阅 zsh 模块的手册。

## 23.3.2　结构化命令

zsh shell 为 shell 脚本提供了常规的结构化命令集：

- ✦ if-then-else 语句；
- ✦ for 循环（包括 C 风格）；
- ✦ while 循环；
- ✦ until 循环；
- ✦ select 语句；
- ✦ case 语句。

对于这些结构化命令，zsh shell 使用和 bash shell 中相同的语法。zsh shell 还包含一个不同的结构化命令，称为 repeat。repeat 命令格式为：

```
repeat param
do
   commands
done
```

参数 param 必须是一个数字或结果值为数值的数学运算。然后 repeat 命令执行该数目次数的指定的命令：

```
% cat test1
#!/bin/zsh
# using the repeat command

value1=$(( 10 / 2 ))
repeat $value1
do
   echo "This is a test"
done
$ ./test1
This is a test
This is a test
This is a test
This is a test
This is a test
%
```

该命令允许基于计算结果重复执行代码。

## 23.3.3　函数

zsh shell 支持创建自定义函数，或者使用 function 命令，或者通过定义带圆括号的函数名：

```
% function functest1 {
> echo "This is the test1 function"
}
```

```
% functest2() {
> echo "This is the test2 function"
}
% functest1
This is the test1 function
% functest2
This is the test2 function
%
```

与 bash shell 函数一样（请参阅第 14 章），可以在 shell 脚本中定义函数，然后使用全局变量或为函数传递参数。下面是使用全局变量的例子：

```
% cat test3
#!/bin/zsh
# testing functions in zsh

dbl() {
   value=$(( $value * 2 ))
   return $value
}
value=10
dbl
echo The answer is $?
% ./test3
The answer is 20
%
```

不是必须将函数放到 shell 脚本中。zsh shell 允许在单独的文件中定义函数，当解析函数名时能够访问那个文件即可。

zsh shell 通过 fpath 环境变量查找函数。可以将函数文件放在该路径下的任意目录中。下面是我的 Linux 工作站上的 fpath 值：

```
% echo $fpath
/usr/local/share/zsh/site-functions
/usr/share/zsh/4.2.5/functions/Completion
/usr/share/zsh/4.2.5/functions/Completion/AIX
/usr/share/zsh/4.2.5/functions/Completion/BSD
/usr/share/zsh/4.2.5/functions/Completion/Base
/usr/share/zsh/4.2.5/functions/Completion/Cygwin
/usr/share/zsh/4.2.5/functions/Completion/Darwin
/usr/share/zsh/4.2.5/functions/Completion/Debian
/usr/share/zsh/4.2.5/functions/Completion/Linux
/usr/share/zsh/4.2.5/functions/Completion/Mandrake
/usr/share/zsh/4.2.5/functions/Completion/Redhat
/usr/share/zsh/4.2.5/functions/Completion/Unix
/usr/share/zsh/4.2.5/functions/Completion/X
/usr/share/zsh/4.2.5/functions/Completion/Zsh
/usr/share/zsh/4.2.5/functions/MIME
/usr/share/zsh/4.2.5/functions/Misc
/usr/share/zsh/4.2.5/functions/Prompts
/usr/share/zsh/4.2.5/functions/TCP
/usr/share/zsh/4.2.5/functions/Zftp
/usr/share/zsh/4.2.5/functions/Zle
%
```

可以看到，zsh shell 可以查找很多位置来解析函数名。在我的系统上，我可以将函数放在/usr/local/share/zsh/site-functions 目录下，zsh shell 将会解析这些函数。

但是，在 zsh shell 解析函数前，必须使用 autoload 命令。该命令将函数载入内存供 shell 访问。下面是一个独立函数的例子：

```
% cat dbl
#!/bin/zsh
# a function to double a value
dbl() {
```

```
    value=$(( $1 * 2 ))
    return $value
}
% cp dbl /usr/local/share/zsh/site-functions
%
```

好的，现在这个函数创建在了一个文件中，存储在 fpath 中的一个目录下。当要使用它时，如果不将它载入内存将会得到错误消息：

```
% dbl 5
zsh: command not found: dbl
% autoload dbl
% dbl 5
% echo $?
10
%
```

这对 shell 脚本同样适用。如果需要使用一个函数，则需要使用 autoload 命令载入函数，才能使用函数：

```
% cat test4
#!/bin/zsh
# testing an external function

autoload dbl

dbl $1
echo The answer is $?
% ./test4 5
The answer is 10
%
```

zsh shell 的另一个有趣的特性是 zcompile 命令。这个命令处理一个函数文件，并为 shell 创建一个“编译后”文件。这同在其他编程语言中使用的编译类型不同。但它将函数转成二进制格式使得 zsh shell 可以更快的将其载入。

运行 zcompile 命令时，会创建一个相应的.zwc 文件。当 autoload 命令在 fpath 中查找这个命令时，将查看.zwc 文件并将这个文件载入，而不是载入文本格式的函数文件。

## 23.4　小结

本章讨论了 zsh shell。zsh shell 为 shell 编程提供了许多高级特性，包括可载入模块中的许多内置命令。zsh shell 提供多种模拟模式，可以很好地模拟其他常用 shell（如 Bourne、csh 和 ksh shell）。这个特性使得可以在 zsh shell 中无缝运行为其他 shell 环境编写的 shell 脚本。

zsh shell 使用可载入模块的概念扩展可用的内置命令。有许多针对 zsh shell 创建的模块，这些模块提供附加特性，如命令行编辑，扩展的数学函数和高级网络支持。使用 zsh shell 的 TCP 模块可以编写复杂的网络脚本，而且不需要其他网络库。

对各种 Linux shell 的介绍至此结束。本书的下一部分将深入探讨在 Linux 环境中可能会遇到的一些特殊脚本应用。下一章介绍如何在脚本中结合使用 Linux 世界的两个最流行的数据库包：MySQL 和 PostgreSQL。

PART

5

# 高级主题

本部分内容：

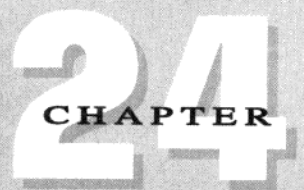
第 24 章
使用数据库

第 25 章
使用 Web

第 26 章
使用电子邮件

第 27 章
管理员使用的 shell 脚本

# 第 24 章
# 使用数据库




shell 脚本要解决的一个问题是持久化数据。可以在 shell 脚本变量中存储所有想要的信息，但是在脚本执行结束后，它们都会消失。很多情况下都希望脚本可以存储数据以便将来使用。过去，要想通过 shell 脚本存储和获取数据需要创建文件，从文件中读取数据，解析数据，然后将数据保存回文件。试图在文件中查找数据意味着必须读取文件中的每条记录来查找需要的数据。现在数据库颇为流行，建立 shell 脚本和专业的开源数据库之间的连接很容易。在 Linux 领域中，MySQL 和 PostgreSQL 是最流行的两个开源数据库。本章将介绍如何在 Linux 系统上运行这些数据库，以及通过命令行进行数据库操作，然后介绍如何使用常规的 bash shell 脚本同每个数据库进行交互。

**本章内容：**

- MySQL 介绍；
- PostgreSQL 介绍；
- 创建数据库对象；
- 编写数据库 shell 脚本。

## 24.1 MySQL 数据库

迄今为止，Linux 环境下最流行的数据库是 MySQL 数据库。它已经成为 Linux-Apache-MySQL-PHP（LAMP）服务器环境的一部分，许多 Internet Web 服务器都使用 LAMP 作为联机存储、博客和应用程序的主机。

本节将介绍如何在 Linux 环境下安装 MySQL 数据库，以及如何创建在 shell 脚本中使用的必要的数据库对象。

### 24.1.1 安装 MySQL

在 Linux 系统上安装软件，通常使用自动软件安装程序。这些安装程序不仅方便从网上下载和安装新的软件，而且还能够自动检查已安装软件包的更新，并安装这些更新。

图 24-1 显示了 Fedora 8 Linux 发行版中的 Add Software 特性。

只要选择 MySQL 选项，Fedora 就会下载并安装完整的 MySQL 服务器和客户端软件。没有比这更简单的了！

openSuse 10.3 Linux 发行版也使用高级的软件管理系统。其中的 Software Management 窗口如图 24-2 所示，在该窗口中可以根据软件类别（MySQL 服务器位于 Database Servers 部分）选择软件包。

同样，选择 MySQL 软件选项下载并安装所需的软件包。

如果使用的 Linux 发行版不支持自动软件加载，或者想要

安装 MySQL 服务器的功能最新最强的版本，可以从 MySQL 直接下载安装文件并手动安装。

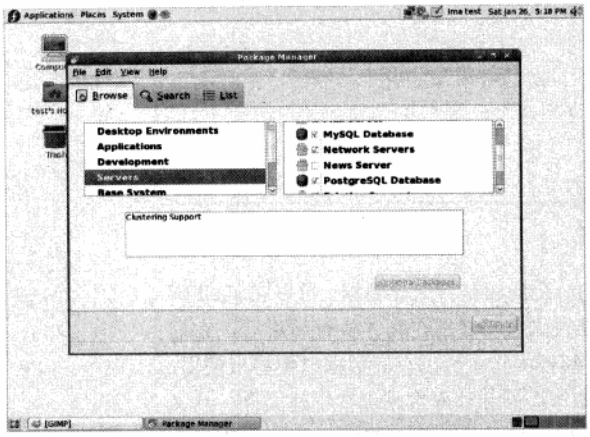



图 24-1　在 Fedora 8 Linux 系统上安装 MySQL

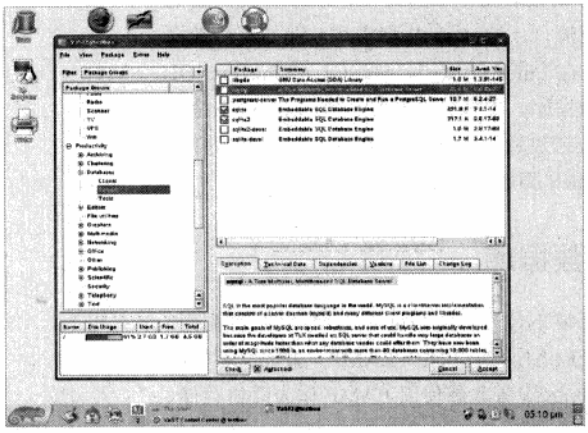


图 24-2　在 openSuse 10.3 Linux 系统上安装 MySQL

下面是下载安装文件的步骤。

（1）打开浏览器窗口，输入 URL：`www.mysql.com`。

（2）单击 Test Drive 部分的 Downloads 链接。

（3）在 MySQL Community Server 部分，选择当前发行版的链接（编写本书时版本为 5.0）。

（4）选择适合某 Linux 平台的的链接。如果存在适合某平台的链接，那么可以选择该链接并使用该平台的安装程序来安装软件（如适合 Debian-based Linux 系统的 `.deb` 包，或适合 Red Hat-based Linux 系统的 `.rpm` 包）。如果没有适合某一平台的链接，那么选择 Linux（*non RPM packages*）选项，下载不带安装包的二进制文件。

（5）选择需要注册的下载链接，或者不需要注册的下载链接。

（6）选择一个镜像位置开始下载。

下载完毕后，系统中会有一个与下面的文件类似的文件（下载文件的版本号会有所不同）：

```
mysql-5.0.45-linux-i686-tar.gz
```

如果还记得第 4 章中的内容，`.tar.gz` 文件扩展名表示该文件是一个使用 gizp 工具进行压缩的 tar 文件。其中包含所有 MySQL 服务器文件，这些文件是先使用 `tar` 命令存档在一个文件中，然后使

用 gzip 压缩工具进行压缩。

接下来需要使用 tar 命令将 tar 文件抽取到一个目录下，以便使用其中的文件。但是，抽取之前还需要其他一些步骤。

（1）创建一个唯一用户和用户组账户负责运行 MySQL 服务器。通常将该账户和用户组命名为 mysql。以 Linux 根用户身份，执行下列命令：

```
# groupadd mysql
# useradd -g mysql mysql
```

（2）选择 MySQL 软件的存储路径。建议路径为/usr/local。使用 tar 命令抽取文件，并创建一个链接，由名为 mysql 的简单路径指向所生成的令人生厌的长路径名：

```
# cp mysql-5.0.45-linux-i686.tar.gz /usr/local
# cd /usr/local
# tar -zxvf mysql-5.0.45-linux-i686.tar.gz
# ln -s mysql-5.0.45-linux-i686 mysql
```

（3）将新 mysql 目录的所有者和用户组更改为所创建的 mysql 用户和用户组账户：

```
# cd mysql
# chown -R mysql .
# chgrp -R mysql .
```

现在 MySQL 服务器和客户端文件位于 Linux 系统中/usr/local/mysql 目录下。

### 24.1.2 完成 MySQL 配置

在安装完 MySQL 软件之后，使用 MySQL 存储数据之前，还要完成一些步骤。本节将介绍 MySQL 服务器在 Linux 系统上运行所需要进行的配置。

#### 1. 初始化数据库文件

对于您的系统，下一步可能需要，也可能不需要。如果安装 MySQL 服务器时使用的是 Linux 发行版的软件安装包，那么它在安装过程中会创建自己的数据库文件，这种情况可以跳过这一部分。

如果是手动下载和安装的 MySQL 二进制包，那么需要创建默认的 MySQL 数据库文件（这些不包含在发行版包中）。以 Linux 根用户身份，输入下面的命令：

```
#/usr/local/mysql/scripts/mysql_install_db --user=mysql
```

在 MySQL 创建完默认数据库后，可以使用下面的命令将除数据库目录外的所有对象的所有者更改回根用户：

```
# cd /usr/local/mysql
# chown -R root .
# chown -R mysql data
```

现在可以启动 MySQL 服务器了。

#### 2. 启动 MySQL 服务器

如果是使用 Linux 发行版的软件安装程序安装的 MySQL，那么它通常会将 MySQL 服务器配置为在下次启动系统时自动启动。可以使用 ps 命令（请参阅第 4 章）进行检查：

```
# ps ax | grep mysql
2084 ?          S      0:00 /bin/sh /usr/bin/mysqld_safe --
datadir=/var/lib/mysql --socket=/var/lib/mysql/mysql.sock --logerror=/
var/log/mysqld.log --pid-file=/var/run/mysqld/mysqld.pid
2141 ?          Sl     0:03 /usr/libexec/mysqld --basedir=/usr --
datadir=/var/lib/mysql --user=mysql --pidfile=/
var/run/mysqld/mysqld.pid --skip-external-locking --
socket=/var/lib/mysql/mysql.sock
8825 pts/1      S+     0:00 grep mysql
#
```

mysqld_safe 脚本用于启动 MySQL 服务器，并且要以 mysql 用户账户的身份进行启动。

如果 MySQL 服务器服务没有运行，则需要启动它。许多 Linux 发行版包含几种服务工具，使用

这些工具可以查看系统启动时启动的服务。在 Fedora 8 和 openSuse 10.3 系统中，该特性都位于 Administrative 部分中。

检查确认 Linux 系统上的 MySQL 服务器服务设置为自动启动。

如果是手动安装的软件，则需要使用 mysqld_safe 脚本启动 MySQL 服务器：

```
#/usr/local/mysql/bin/mysql_safe --user=mysql &
```

这段助手脚本以后台模式启动 MySQL 服务器，以新的 mysql 用户账户身份运行 MySQL 服务器。MySQL 安装还提供一个脚本，用于系统启动时自动启动 MySQL 服务器。第 13 章讨论过如何放置脚本，使它们在 Linux 系统启动时自动启动。可以在发行版的启动脚本中创建一个链接，引用 MySQL 启动脚本，该脚本的位置是：

```
/usr/local/mysql/support-files/mysql.server
```

这段脚本使用 Linux 用户账户 mysql 自动启动 MySQL 服务器。

**3. 保护 MySQL 用户账户安全**

这是 MySQL 服务器安装的最后一步。MySQL 服务器本身会维护用户账户的设置，以控制对数据库对象的访问。默认情况下，MySQL 服务器有一个名为根的超级管理员账户。不幸的是，该账户没有默认密码。为根用户账户修改密码是个非常好的主意。在 Linux 命令行中使用 mysqladmin 命令来修改密码：

```
$ mysqladmin -u root password newpasswd
```

其中 *newpasswd* 表示赋予根用户账户的新密码。

## 24.1.3　MySQL 客户端接口

MySQL 数据库的入口是 mysql 命令行界面程序。本节将介绍 mysql 客户端程序，以及如何使用 mysql 客户端程序同数据库进行交互。

**1. 连接到服务器**

mysql 客户端程序允许用户使用任意用户账户和密码连接到网络中的任意 MySQL 数据库服务器。默认情况下，如果在命令行中输入不带任何参数的 mysql 程序，它将尝试使用 Linux 登录用户名连接在同一个 Linux 系统上运行的 MySQL 服务器。

大多数情况下，这很可能不是您想要的连接数据库的方式。有许多命令行参数不仅可以控制连接到哪个 MySQL 服务器，还可以控制 mysql 接口的行为。表 24-1 列出了可以在 mysql 程序中使用的命令行参数。

表 24-1　　mysql 命令行参数

| 参数 | 描述 |
|---|---|
| -A | 禁用自动重散列 |
| -b | 禁用错误声音警告 |
| -B | 不使用历史文件 |
| -C | 压缩所有在客户端和服务器之间发送的信息 |
| -D | 指定要使用的数据库 |
| -e | 执行指定的语句并退出 |
| -E | 垂直显示查询输出，每行一个数据字段 |
| -f | 如果发生 SQL 错误继续执行 |
| -G | 启用指定的 mysql 命令 |
| -h | 指定 MySQL 服务器的主机名（默认为 localhost） |
| -H | 以 HTML 代码形式显示查询输出 |
| -i | 忽略函数名后面的空格 |
| -N | 结果中不显示列名 |

续表

| 参数 | 描述 |
| --- | --- |
| -o | 忽略除命令行中指定的默认数据库的语句以外的相关语句 |
| -p | 指定用户账户密码 |
| -P | 指定网络连接使用的 TCP 端口号 |
| -q | 不缓存查询结果 |
| -r | 显示列值，但不显示转义 |
| -s | 静音模式 |
| -S | 为本地连接指定套接字 |
| -t | 以表格形式显示输出 |
| -T | 程序退出后显示调试信息、内存和 CPU 统计信息 |
| -u | 指定登录用户账户 |
| -U | 只允许指定键值的 UPDATE 和 DELETE 语句 |
| -v | 冗余模式 |
| -w | 如果建立连接失败，等待并重试 |
| -X | 以 XHTML 代码形式显示查询结果 |

如上所示，有许多命令行选项可用来修改登录 MySQL 服务器的方式。

默认情况下，mysql 客户端程序使用 Linux 登录名来登录 MySQL 服务器。如果没有在 MySQL 中将其配置为一个用户账户，那么需要使用-u 参数来指定登录名：

```
$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 7
Server version: 5.0.45 Source distribution

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql>
```

-p 参数告诉 MySQL 登录时需要用户账户的密码。一旦登录到服务器，就可以开始输入命令。

### 2. mysql 命令

mysql 程序使用两种不同类型的命令：

- ✦ 特殊的 mysql 命令；
- ✦ 标准 SQL 语句。

mysql 程序拥有自己的命令集，使用这些命令能够轻松地控制环境和获取 MySQL 服务器的相关信息。表 24-2 列出了这些命令。

表 24-2 mysql 命令

| 命令 | 快捷命令 | 描述 |
| --- | --- | --- |
| ? | \? | 帮助 |
| clear | \c | 清除命令 |
| connect | \r | 连接数据库和服务器 |
| delimiter | \d | 设置 SQL 语句分隔符 |
| edit | \e | 使用命令行编辑器编辑命令 |
| ego | \G | 向 MySQL 服务器发送命令，并垂直显示结果 |
| exit | \q | 退出 mysql 程序 |
| go | \g | 向 MySQL 服务器发送命令 |
| help | \h | 显示帮助 |
| nopager | \n | 禁用输出分页程序，将输出发送到 STDOUT |

续表

| 命令 | 快捷命令 | 描述 |
|---|---|---|
| note | \t | 不向输出文件发送输出 |
| pager | \P | 为特定程序设置分页程序命令（默认使用 more） |
| print | \p | 打印当前命令 |
| prompt | \R | 更改 mysql 命令提示 |
| quit | \q | 退出 mysql 程序（同 exit） |
| rehash | \# | 重新生成命令完成散列表 |
| source | \. | 执行指定文件中的 SQL 脚本 |
| status | \s | 获取 MySQL 服务器的状态信息 |
| system | \! | 在系统中执行 shell 命令 |
| tee | \T | 将所有输出附加到指定文件中 |
| use | \u | 使用其他数据库 |
| charset | \C | 更改到其他字符集 |
| warnings | \W | 在每个语句后面显示警告 |
| nowarning | \w | 在每个语句后面不显示警告 |

可以在 mysql 命令提示符后面直接使用完整的命令，也可以使用快捷命令：

```
mysql> \s
--------------
mysql Ver 14.12 Distrib 5.0.45, for redhat-linux-gnu (i386) using
readline 5.0

Connection id:          10
Current database:
Current user:           root@localhost
SSL:                    Not in use
Current pager:          stdout
Using outfile:          ''
Using delimiter:        ;
Server version:         5.0.45 Source distribution
Protocol version:       10
Connection:             Localhost via UNIX socket
Server characterset:    latin1
Db characterset:        latin1
Client characterset:    latin1
Conn. characterset:     latin1
UNIX socket:            /var/lib/mysql/mysql.sock
Uptime:                 4 hours 15 min 24 sec

Threads: 1 Questions: 53 Slow queries: 0 Opens: 23 Flush tables:
1 Open tables: 17 Queries per second avg: 0.003
--------------

mysql>
```

mysql 程序实现了所有 MySQL 服务器支持的标准 SQL 命令。后面的“创建 MySQL 数据库对象”一节将对其进行详细介绍。

mysql 程序实现的一个不常用的 SQL 命令是 SHOW 命令。使用该命令可以抽取 MySQL 服务器信息，如创建的数据库和表：

```
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
```

```
| mysql              |
+--------------------+
2 rows in set (0.04 sec)

mysql> USE mysql;
Database changed
mysql> SHOW TABLES;
+---------------------------+
| Tables_in_mysql           |
+---------------------------+
| columns_priv              |
| db                        |
| func                      |
| help_category             |
| help_keyword              |
| help_relation             |
| help_topic                |
| host                      |
| proc                      |
| procs_priv                |
| tables_priv               |
| time_zone                 |
| time_zone_leap_second     |
| time_zone_name            |
| time_zone_transition      |
| time_zone_transition_type |
| user                      |
+---------------------------+
17 rows in set (0.00 sec)

mysql>
```

在本例中，使用 SHOW SQL 命令显示 MySQL 服务器上当前配置的数据库，然后使用 USE SQL 命令连接到一个数据库。mysql 会话每次只能同一个数据库进行连接。

注意，每个命令后面添加了一个分号。分号告知 mysql 程序一个命令结束。如果不使用分号，会提示输入更多数据：

```
mysql> SHOW
    -> DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
+--------------------+
2 rows in set (0.00 sec)

mysql>
```

处理长命令时，使用该特性会非常便利。可以在一行中输入命令的一部分，并按 Enter 键，然后在下一行中继续输入。想要分成多少行输入都可以，直到使用分号表示命令结束。

注意：本章中 SQL 命令都使用大写字母。这已经成为书写 SQL 命令的一种通用方法。但是，mysql 程序允许您指定 SQL 命令使用大写或小写。

### 24.1.4 创建 MySQL 数据库对象

在开始编写同数据库进行交互的 shell 脚本前，需要创建一些数据库对象。至少要创建以下数据对象：

- 用来存储应用数据的唯一数据库；
- 通过脚本访问数据库时使用的唯一用户账户；
- 用来组织数据的一个或多个数据表。

所有这些对象使用 mysql 程序生成。mysql 程序直接和 MySQL 服务器连接，使用 SQL 命令创建和修改每个数据对象。

可以使用 mysql 程序向 MySQL 服务器发送任意类型的 SQL 命令。本节将介绍 shell 脚本中生成基础数据库对象所需的各种 SQL 语句。

**1. 创建数据库**

MySQL 服务器将数据组织到**数据库**中。通常，一个数据库只为某个应用程序存储数据，将这些数据同数据库服务器上其他应用程序的数据分开。为每个 shell 脚本应用程序创建一个单独的数据库有助于消除困惑和数据混乱。

新建数据库的 SQL 语句为：

```
CREATE DATABASE name;
```

这非常简单。当然，必须拥有适当的权限才能在 MySQL 服务器上新建数据库。最简单的方式是以根用户账户登录：

```
$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 15
Server version: 5.0.45 Source distribution

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql> CREATE DATABASE test;
Query OK, 1 row affected (0.00 sec)

mysql>
```

可以使用 SHOW 命令查看新数据库是否创建成功：

```
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| test               |
+--------------------+
3 rows in set (0.01 sec)

mysql>
```

是的，已经成功创建了。现在可以连接到新数据库：

```
mysql> USE test;
Database changed;
mysql> SHOW TABLES;
Empty set (0.00 sec)
mysql>
```

使用 SHOW TABLES 命令可以查看是否创建了表。结果 Empty set 表示还没有可以处理的表。在开始创建表之前，还有一件事情要做。

**2. 创建用户账户**

目前为止，您已经学习了如何使用根管理员账户连接到 MySQL 服务器。该账户拥有对所有 MySQL 服务器对象的控制权（就像 Linux 根账户拥有对 Linux 系统的完全控制一样）。

使用 MySQL 根账户处理常规的应用程序是极其危险的。如果存在安全漏洞并且有人计算出根用户账户的密码，那么您的系统（和数据）就即将大祸临头了。

为防止这种情况发生，应在 MySQL 中创建一个独立的用户账户，该账户只拥有访问某应用程序使用的数据库的权限。这可以通过 GRANT SQL 语句来实现：

```
mysql> GRANT SELECT,INSERT,DELETE,UPDATE ON test.* TO test IDENTIFIED
```

```
by 'test';
Query OK, 0 rows affected (0.35 sec)

mysql>
```

这个命令非常长。下面将介绍它的几部分。

第一部分定义用户账户对数据库拥有的权限。这个语句允许用户账户查询数据库数据（select 权限），插入新数据记录，删除已有数据记录以及更新已有数据记录。

test.*条目定义权限适用于哪些数据库和表。通过下面的格式指定：

```
database.table
```

从该例可以看出，指定数据库和表时可以使用通配符。此格式对 test 数据库中的所有表应用指定的权限。

最后，指定权限适用的用户账户。如果用户账户不存在，grant 命令的好处在于能创建这个用户账户。identified by 部分允许为新用户账户设置默认密码。

可以直接通过 mysql 程序测试新用户账户：

```
$ mysql test -u test -p
Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.0.45 Source distribution

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql>
```

第一个参数指定要使用的默认数据库（test）。还可以看到，-u 参数定义登录用户账户，-p 用来提示输入密码。输入分配给该测试用户账户的密码，就连接到了服务器。

现在已经创建了一个数据库和一个用户账户，接下来就可以开始创建一些数据表。但先看一看其他可以使用的数据库服务器。

## 24.2 PostgreSQL 数据库

PostgreSQL 数据库一开始是作为学术项目出现的，用于演示如何将先进的数据库技术结合到功能数据库服务器中。经过多年的发展，PostgreSQL 已经成为可以在 Linux 环境下使用的最先进的开源数据库服务器之一。

本节将介绍 PostgreSQL 数据库服务器的安装和运行，然后创建在 shell 脚本中使用的用户账户和数据库。

### 24.2.1 安装 PostgreSQL

与 MySQL 一样，可以使用系统的自动软件安装系统来安装 PostgreSQL 数据库服务器包，也可以从 PostgreSQL 网站上下载并手动进行安装。

要使用 Linux 发行版的自动软件安装，按照前面“安装 MySQL”一节中介绍的步骤进行安装。不幸的是，如果必须手动下载 PostgreSQL 服务器软件，所做的工作要稍多于 MySQL。

PostgreSQL 软件仅有的二进制发行版使用 Red Hat RPM 包管理系统。如果 Linux 系统使用 RPM 包管理系统（或像 Debian 发行版那样有一个工具可以将 RPM 包转换为自有格式），那么可以下载并安装该二进制包。

否则，您需要下载 PostgreSQL 的源代码包，在 Linux 系统上手动进行编译。这会是个非常有趣的项目。

注意：编译源代码包需要在 Linux 系统上安装 C 软件开发包。现在这对服务器来说是小菜一碟，但如果您使用的是桌面 Linux 发行版，则可能无法安装 C 软件开发包。请查阅您使用的个人 Linux 发行版关于编译软件的文档，了解编译 C 源代码项目需要的软件包。

获取 PostgreSQL 源代码并在系统上进行安装的步骤如下：

1. 打开浏览器窗口，输入 URL：www.postgresql.org。
2. 选择网页右上角的 Source 链接，链接到最新版本的 PostgreSQL。
3. 选择合适的下载文件格式（bzip2 或 gzip）进行下载。
4. 选择下载镜像站点和方法开始下载。

下载代码后，将代码放在 Linux 系统中的工作目录（如$HOME 目录）下，使用 tar 命令抽取包文件：

```
tar -zxvf postgresql-8.2.6.tar.gz
```

tar 命令在工作目录下创建路径 postgresql-8.2.6（或者其他下载版本号）并抽取源代码文件。现在可以开始进行编译。

按照下面的步骤创建 PostgreSQL 服务器和客户端可执行文件：

（1）切换到包含 PostgreSQL 源代码的新目录。

```
$cd postgresql-8.2.6
```

（2）执行配置脚本：

```
$ ./configure
```

这会检查系统，确认已经拥有编译源代码所需的正确的库。PostgreSQL 需要的库很多。如果提示缺少库，则必须先在 Linux 系统上安装这些库。

（3）执行 gmake 工具来编译源代码：

```
$ gmake
```

（4）更改为根用户，使用 gmake 工具执行安装脚本：

```
$ su
password:
# gmake install
```

这会将 PostgreSQL 二进制文件放置到/usr/local/pgsql 目录。

（5）创建一个名为 postgres 的用户，创建数据库文件目录，使该用户成为该目录的所有者：

```
# adduser postgres
# mkdir /usr/local/postgres/data
# chown postgres /usr/local/postgres/data
```

恭喜您，现在 PostgreSQL 程序在系统上安装好了！在启动 PostgreSQL 前，需要初始化 PostgreSQL 数据库文件。有一个工具程序可以完成这件事，但您必须是用户 postgres 才行：

```
# su postgres
$ /usr/local/postgres/bin/initdb -D /usr/local/postgres/data
```

现在可以启动 PostgreSQL 服务器了。要启动 PostgreSQL 数据库并使它在后台运行，您必须保持 postgres 用户账户的身份：

```
# su postgres
# cd /usr/local/postgres/bin
# ./postgres -D /usr/local/postgres/data >logfile 2>&1 &
```

如果希望在系统启动时自动启动 PostgreSQL 服务器，将这些命令添加到 Linux 发行版的启动脚本中。

登录到 PostgreSQL 服务器和登录到 MySQL 服务器略有区别。是否还记得，MySQL 服务器为有权访问数据库对象的用户维护一个内部数据库。虽然 PostgreSQL 也具有这个功能，但大多数 PostgreSQL 实现（包括默认的源代码安装）使用现有的 Linux 系统用户账户来验证 PostgreSQL 用户。

虽然这可能会使人混淆，但它确实提供一种简洁的方法来控制 PostgreSQL 中的用户账户。只需保证每个 PostgreSQL 用户对应一个有效的 Linux 系统账户，而不必考虑单独的用户账户集。

另一个重要区别是，PostgreSQL 中的管理员账户名为 postgres，而不是 root。因此，只有名

为 postgres 的 Linux 系统账户存在，PostgreSQL 管理员账户才存在。

接下来介绍如何使用 postgres 账户访问 PostgreSQL 服务器。

### 24.2.2 PostgreSQL 命令接口

PostgreSQL 命令行客户端程序名为 psql。该程序提供对 PostgreSQL 服务器上配置的数据库对象的完全访问。本节将介绍 psql 程序，以及如何使用 psql 程序和 PostgreSQL 服务器进行交互。

**1. 连接服务器**

psql 客户端程序提供到 PostgreSQL 服务器的命令行接口。它使用命令行参数控制客户端接口中启用的功能。每个选项都有全称和简称格式。表 24-3 给出了可用的命令行参数。

表 24-3 psql 命令行参数

| 简称 | 全称 | 说明 |
|---|---|---|
| -a | --echo-all | 在输出中显示一个脚本文件中处理的所有命令行 |
| -A | --no-align | 设置输出格式为非对齐模式，数据不以格式化的表格形式显示 |
| -c | --command | 执行指定的 SQL 语句并退出 |
| -d | --dbname | 指定连接的数据库 |
| -e | --echo-queries | 在屏幕上回显所有查询 |
| -E | --echo-hidden | 在屏幕上回显隐藏的 psql 元命令 |
| -f | --file | 执行指定文件中的 SQL 命令并退出 |
| -F | --field-separator | 指定非对齐模式下分隔列数据的字符，默认为逗号 |
| -h | --host | 指定远程 PostgreSQL 服务器的 IP 地址或主机名 |
| -l | --list | 显示服务器上的有效数据库列表并退出 |
| -o | --output | 将查询输出重定向到指定的文件 |
| -p | --post | 指定连接使用的 PostgreSQL 服务器 TCP 端口 |
| -P | --pset | 设置指定的表打印选项为指定值 |
| -q | --quiet | 静音模式，不显示输出消息 |
| -R | --record-separator | 使用指定的字符作为记录分隔符，默认是换行符 |
| -s | --single-step | 每个 SQL 查询后，提示继续或取消 |
| -S | --single-line | 指定 Enter 键定义 SQL 查询的结束，而不是分号 |
| -t | --tuples-only | 在表输出中不显示列头和尾 |
| -T | --table-attr | 在 HTML 模式下，使用指定的 HTML 表格标签 |
| -U | --username | 使用指定的用户名连接 PostgrelSQL 服务器 |
| -v | --variable | 设置指定的变量为指定的值 |
| -V | --version | 显示 psql 版本号并退出 |
| -W | --password | 强制密码提示 |
| -x | --expanded | 启用扩展表输出以显示记录的附加信息 |
| -X | --nopsqlrc | 不处理 psql 启动文件 |
| -? | --help | 显示 pslq 命令行帮助并退出 |

如前一节所述，PostgreSQL 的管理员账户名为 postgres。因为 PostgreSQL 使用 Linux 用户账户来验证用户，所以必须以 Linux 账户 postgres 登录才能以 postgres 用户身份访问 PostgreSQL 服务器。

因为 postgres 用户账户是一个特殊账户，所以不应该给它分配密码。这样没有人可以使用 postgres 账户闯入系统。相反，如果想用 postgres 账户登录，您必须是根用户，然后使用 su 命令更改为 postgres 用户：

```
$ su
```

```
Password:
# su postgres
bash-3.2$ psql
Welcome to psql 8.2.6, the PostgreSQL interactive terminal.

Type: \copyright for distribution terms
      \h for help with SQL commands
      \? for help with psql commands
      \g or terminate with semicolon to execute query
      \q to quit
postgres=#
```

默认的 psql 提示表示已经连接到的数据库。上面提示中的英镑符号表示已经使用管理员用户账户登录成功。现在可以开始输入一些命令同 PostgreSQL 服务器进行交互。

**2. psql 命令**

与 mysql 程序类似，psql 程序使用两种命令类型：

- 标准 SQL 语句；
- PostgreSQL 元命令。

使用 PostgreSQL 元命令可以轻松抽取数据库环境信息，包括设置 psql 会话特性。元命令是通过使用反斜线符号触发的。PostgreSQL 元命令非常多，能够实现许多不同的设置和功能，但实在没有理由现在就为此而担心。最常用的元命令如下所示：

- \l 列出可用的数据库；
- \c 连接到某个数据库；
- \dt 列出某个数据库中的表；
- \du 列出 PostgreSQL 用户；
- \z 列出表权限；
- \?列出所有可用的元命令；
- \h 列出所有可用的 SQL 命令；
- \q 退出数据库。

如果需要查找某个元命令，只要输入\?元命令。就会看到带有说明的所有可用元命令的列表。

要测试元命令，使用\l 元命令列出可用的数据库：

```
postgres=# \l
        List of databases
   Name    | Owner    | Encoding
-----------+----------+----------
 postgres  | postgres | UTF8
 template0 | postgres | UTF8
 template1 | postgres | UTF8
(3 rows)

postgres=#
```

这些是 PostgreSQL 服务器提供的默认数据库。postgres 数据库维护服务器的所有系统数据。template0 和 template1 数据库为新建数据库提供默认的数据库模板。

现在可以开始在 PostgreSQL 中处理数据了。

### 24.2.3　创建 PostgreSQL 数据库对象

本节介绍创建数据库和用户账户以访问数据库的过程。您将会看到 PostgreSQL 中的一些操作和 MySQL 中的非常相似，而另一些则完全不同。

**1. 创建数据库对象**

创建数据库的操作与 MySQL 中的操作相同。记住新建数据库时要以 postgres 管理员账户登录：

```
$ su
Password:
# su postgres
bash-3.2$ psql
Welcome to psql 8.2.6, the PostgreSQL interactive terminal.

postgres=# CREATE DATABASE test;
CREATE DATABASE
postgres=#
```

创建数据库后，使用\l 元命令查看它是否出现在列表中，然后使用\c 元命令连接到该数据库：

```
postgres=# \l
        List of databases
   Name    | Owner    | Encoding
-----------+----------+----------
 postgres  | postgres | UTF8
 template0 | postgres | UTF8
 template1 | postgres | UTF8
 test      | postgres | UTF8
(4 rows)

postgres=# \c test
You are now connected to database "test".
test=#
```

连接到 test 数据库时，psql 提示更改以表示新数据库名称。准备创建数据库对象时，这是个很不错的提示，因为这样您很容易地了解自己在系统中的位置。

注意：PostgreSQL 在数据库中添加有另一个控制层，称为**模式**（schema）。每个数据库可以包含多个模式，每个模式可以包含多个表。这样可以将数据库进行细分，供特殊应用程序或特殊用户使用。

每个数据库默认包含一个称为 public 的模式。如果只有一个应用程序使用数据库，那么只使用 public 模式就可以了。如果希望获得更多高级功能，可以创建新模式。在本例中，我的表只使用了 public 模式。

### 2. 创建用户账户

新建数据库后，下一步是创建一个供 shell 脚本访问的数据库用户账户。如您所见，PostgreSQL 中的用户账户和 MySQL 中的账户非常不同。

PostgreSQL 中的用户账户称为**登录角色**（login role）。PostgreSQL 服务器将登录角色和 Linux 系统用户账户进行匹配。因此，关于创建登录角色，运行访问 PostgreSQL 数据库的 shell 脚本有两种常见的想法：

- ✦ 创建一个特殊的 Linux 账户和一个与之匹配的 PostgreSQL 登录角色来运行所有的 shell 脚本。
- ✦ 为需要运行 shell 脚本来访问数据库的每个 Linux 用户账户创建 PostgreSQL 账户。

在本例中，我将选择第二种方法，创建一个和 Linux 系统账户匹配的 PostgreSQL 账户。这样，我可以直接使用 Linux 用户账户运行 shell 脚本来访问 PostgreSQL 数据库。

首先，必须创建登录角色：

```
test=# CREATE ROLE rich login;
CREATE ROLE
test=#
```

这非常简单。没有登录参数，该角色不能登录到 PostgreSQL 服务器，但可以给它分配权限。这种角色称为**组角色**（group role）。在拥有多个用户和表的大型环境中，组角色非常有用。不必了解不同用户对不同表的具体权限，只要针对表的特定访问类型创建组角色，然后将登录角色分配给适当的组角色即可。

对于简单的 shell 脚本编写，大可不需要担心创建组角色，只要直接给登录角色分配权限即可。我

在这个例子中就是这样做的。

但是，PostgreSQL 处理权限的方式和 MySQL 有些不同。它不允许对数据库中所有的表级对象整体授权，您要为所创建的每个表单独授予权限。尽管这项工作有些痛苦，但它确实有助于强化严格的安全策略。只有先创建表，然后才可以授予权限。这是接下来要介绍的内容。

## 24.3　处理表

现在已经有了可以运行的 MySQL 或 PostgreSQL 服务器，新建了数据库，并创建了访问数据库的用户账户，是时候开始处理数据了！幸运的是，`mysql` 和 `psql` 程序都使用标准 SQL 来创建和管理数据表。本节将介绍在这两个环境中创建表，插入和删除数据，以及查询现有数据所需的 SQL。

### 24.3.1　创建表

MySQL 和 PostgreSQL 服务器都被认为是**关系数据库**（relational database）。在关系数据库中，数据通过数据字段、记录和表进行组织。一个数据字段是一条信息，如员工的姓或工资。记录是相关数据字段的集合，如员工的 ID 号、姓、名、地址和工资。每个记录表示数据字段的一个集合。

表包含所有持有相关数据的记录。这样，您将得到一个名为 `Employees` 的表，里面存储每个员工的记录。

要在数据库中新建一个表，需要使用 `CREATE TABLE` SQL 命令：

```
$ mysql test -u root -p
Enter password:
mysql> CREATE TABLE employees (
    -> empid int not null,
    -> lastname varchar(30),
    -> firstname varchar(30),
    -> salary float,
    -> primary key (empid));
Query OK, 0 rows affected (0.14 sec)

mysql>
```

首先，注意要创建新表需要使用根用户账户登录 MySQL，因为 `test` 用户没有创建表的权限。还要注意，在 `mysql` 程序命令行中指定 `test` 数据库。否则，需要使用 `USE` SQL 命令连接到 `test` 数据库。

警告：创建新表前确认连接到了正确的数据库非常重要。还要确认使用管理员用户账户（MySQL 中是 root 用户，PostgreSQL 中是 postgres）登录来创建表。

表中的每个数据字段都通过数据类型定义。MySQL 和 PostgreSQL 数据库都支持许多不同的数据类型。表 24-4 给出了可能会需要的一些较流行的数据类型。

数据字段 `emid` 还指定了一个**数据约束**（data constraint）。数据约束能够在创建有效记录时限制输入的数据类型。`not null` 数据约束表示每条记录都必须指定一个 `empid` 值。

表 24-4　MySQL 和 PostgreSQL 数据类型

| 数据类型 | 描述 | 数据类型 | 描述 |
|---|---|---|---|
| char | 固定长度的字符串值 | Date | 日期值，格式为 YYYY-MM-DD |
| varchar | 可变长度的字符串值 | Time | 时间值，格式为 HH:mm:ss |
| int | 整数值 | Timestamp | 日期和时间值 |
| float | 浮点值 | Text | 长字符串值 |
| Boolean | 布尔 true/false 值 | BLOB | 大型二进制值，如图像或视频剪辑 |

最后，`primary key` 定义一个数据字段来唯一标识每条记录。这就意味着表中每条数据记录都

必须有一个唯一的 empid 值。

新建表后，可以使用合适的命令来确保表创建成功。在 mysql 中，使用 show table 命令：

```
mysql> show tables;
+----------------+
| Tables_in_test |
+----------------+
| employees      |
+----------------+
1 row in set (0.00 sec)
mysql>
```

在 psql 中，使用\dt 元命令：

```
test=# \dt
        List of relations
 Schema | Name      | Type  | Owner
--------+-----------+-------+----------
 public | employees | table | postgres
(1 row)

test=#
```

是否还记得前面“创建 PostgreSQL 数据库对象”一节中，在 PostgreSQL 中需要以表级分配权限。现在有了一个表，就需要赋予登录角色访问该表的能力：

```
# su postgres
$ psql test
test=# GRANT SELECT,INSERT,DELETE,UPDATE ON public.employees TO rich;
GRANT
test=#
```

指定表的格式必须包含模式名称，默认为 public。而且，记住以 postgres 登录角色执行这个命令。

已经创建了表格，现在可以开始保存一些数据。下一节将介绍关于数据的内容。

### 24.3.2 插入和删除数据

使用 INSERT SQL 命令向表中插入新数据记录。每个 INSERT 命令都必须为要接受记录的 MySQL 或 PostgreSQL 服务器指定数据字段值。

INSERT SQL 命令的格式为：

```
INSERT INTO table VALUES (...)
```

values 后面是通过逗号分隔的每个数据字段的数据值：

```
$ mysql test -u test -p
Enter password:

mysql>
Query OK, 1 row affected (0.35 sec)
```

或者，在 PostgreSQL 中：

```
[rich@testbox ~]$ psql test
Welcome to psql 8.2.6, the PostgreSQL interactive terminal.

test=> INSERT INTO employees VALUES (1, 'Blum', 'Rich', 25000.00);
INSERT 0 1
test=>
```

因为在 MySQL 和 PostgreSQL 中新建的用户账户都拥有插入数据的权限，所以可以使用该账户登录。INSERT 命令将指定的数据值送入表中的数据字段。如果要添加一条与 empid 数据字段值重复的记录，将会提示错误消息：

```
mysql> INSERT INTO employees VALUES (1, 'Blum', 'Barbara', 45000.00);
ERROR 1062 (23000): Duplicate entry '1' for key 1
```

但是，如果将 empid 值更改为唯一值，将一切顺利：

```
mysql> INSERT INTO employees VALUES (2, 'Blum', 'Barbara', 45000.00);
Query OK, 1 row affected (0.00 sec)
```

现在表中应有两条数据记录。

如果需要将数据从表中清除，可以使用 DELETE SQL 命令。但使用时要非常谨慎。

基本的 DELETE 命令的格式为：

```
DELETE FROM table;
```

其中 *table* 指定要删除记录的表。使用这个命令有个小问题：它会删除表中的所有数据。

要指定删除一条记录或一组记录，必须使用 WHERE 子句。WHERE 子句允许创建过滤条件来限制删除哪些记录。像下面这样使用 WHERE 子句：

```
DELETE FROM employees WHERE empid = 2;
```

这样就限制删除所有 empid 值等于 2 的记录。执行这个命令时，mysql 程序返回一条消息表示有多少条记录和过滤条件匹配：

```
mysql> DELETE FROM employees WHERE empid = 2;
Query OK, 1 row affected (0.29 sec)
```

不出所料，只有和过滤条件匹配的一条记录被删除了。

### 24.3.3　查询数据

数据库中存储了全部数据后，就可以开始执行报表来抽取信息。

查询要使用 SQL SELECT 命令。SELECT 命令功能非常强大，但随之而来的就是复杂性。

SELECT 语句的基本格式为：

```
SELECT datafields FROM table
```

datafields 参数是以逗号分隔的希望查询返回的数据字段名称列表。如果想要得到所有数据字段值，可以使用星号作为通配符（*）。

还必须指定查询要搜索的表。要得到有意义的结果，必须使查询数据字段和正确的表匹配。

SELECT 命令默认返回指定表中的所有数据字段：

```
mysql> SELECT * FROM employees;
+-------+----------+-----------+--------+
| empid | lastname | firstname | salary |
+-------+----------+-----------+--------+
|     1 | Blum     | Rich      | 25000  |
|     2 | Blum     | Barbara   | 45000  |
|     3 | Blum     | Katie Jane | 34500 |
|     4 | Blum     | Jessica   | 52340  |
+-------+----------+-----------+--------+
4 rows in set (0.00 sec)
mysql>
```

可以使用一个或多个修饰符来定义数据库服务器如何返回查询结果。下面是可以使用的常用修饰符。

- ✦ WHERE：显示与指定条件相匹配的记录子集；
- ✦ OREDER BY：按指定顺序显示记录；
- ✦ LIMIT：只显示记录的一个子集。

WHERE 子句是 SELECT 命令修饰符中最常用的。它允许指定条件过滤结果集中的数据。下面是一个使用 WHERE 子句的例子：

```
mysql> SELECT * FROM employees WHERE salary > 40000;
+-------+----------+-----------+--------+
| empid | lastname | firstname | salary |
+-------+----------+-----------+--------+
|     2 | Blum     | Barbara   | 45000  |
|     4 | Blum     | Jessica   | 52340  |
+-------+----------+-----------+--------+
```

```
2 rows in set (0.01 sec)
mysql>
```

现在您可以看出在 shell 脚本中添加数据库访问的强大力量了吧！只通过一些 SQL 命令和 mysql 或 psql 程序就可以轻易控制数据管理需求。下一节将介绍如何将这些特性结合到 shell 脚本中。

## 24.4 在脚本中使用数据库

既然已经使数据库运转起来，是时候将注意力转回 shell 脚本世界了。本节将介绍如何使用 shell 脚本同数据库进行交互。

### 24.4.1 连接到数据库

显然，要连接到数据库，需要在 shell 脚本中使用 mysql 或 psql 程序。这个过程不是很复杂，但有一些注意事项。

**1. 查找程序**

第一道障碍是找到 Linux 系统中 mysql 和 psql 命令行客户端程序的位置。Linux 软件安装的一个弊端是通常不同的 Linux 系统将软件包放在不同的位置。

幸运的是，有一个 which 命令。如果要从命令行执行一个命令，which 命令可以告诉 shell 能够在哪里找到这个命令：

```
$ which mysql
/usr/bin/mysql
$ which psql
/usr/bin/psql
$
```

处理这个信息最容易的方法是将它赋给一个环境变量，当想要引用正确的程序时，在 shell 脚本中使用该环境变量：

```
MYSQL='which mysql'
PSQL='which psql'
```

现在$MYSQL 变量指向 mysql 程序的可执行文件，$PSQL 变量指向 psql 程序的可执行文件。

**2. 登录服务器**

找到客户端程序的位置后，可以在脚本中使用它们访问数据库服务器。对于 PostgreSQL 服务器，这很容易：

```
[rich@testbox]$ cat ptest1
#!/bin/bash
# test connecting to the PostgreSQL server

PSQL='which psql'

$PSQL test
[rich@testbox]$ ./ptest1
Welcome to psql 8.2.6, the PostgreSQL interactive terminal.

Type: \copyright for distribution terms
      \h for help with SQL commands
      \? for help with psql commands
      \g or terminate with semicolon to execute query
      \q to quit

test=>
```

由于我从 Linux 用户账户执行脚本，所以只需在 psql 命令行中指定要连接的数据库名称即可。ptest1 脚本连接到 test 数据库，psql 提示表示已位于该数据库中：

如果已经在 MySQL 中为 shell 脚本创建了一个特殊的用户账户，那么需要在 mysql 命令行中指

定该用户账户：

```
$ cat mtest1
#!/bin/bash
# test connecting to the MySQL server

MYSQL='which mysql'

$MYSQL test -u test -p
$ ./mtest1
Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 15
Server version: 5.0.45 Source distribution

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql>
```

这会有效，但对非交互式脚本来说这种方法不是很适合。-p 命令行参数导致 mysql 暂停并询问密码。可以通过在命令行中包含密码来解决这一问题：

```
$MYSQL test -u test -ptest
```

但这并不是个好办法。每个可以访问脚本的用户都将知道数据库的用户账户和密码。

要解决这个问题，可以利用 mysql 程序使用的一个特殊配置文件。mysql 程序使用 $HOME/.my.cnf 文件来读取特殊的启动命令和设置。其中一个设置是用户账户启动 mysql 会话的默认密码。

要在该文件中设置默认密码，只需使用以下代码：

```
$ cat .my.cnf
[client]
password = test
$ chmod 400 .my.cnf
$
```

使用 chmod 命令限制.my.cnf 文件，使得只有设置者可以查看它。现在可以通过命令行来进行测试：

```
$ mysql test -u test
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 15
Server version: 5.0.45 Source distribution

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql>
```

好极了！现在不必在 shell 脚本中的命令行里包含密码了。

### 24.4.2　向服务器发送命令

同服务器建立连接后，就会希望发送命令同数据库进行交互。发送命令有两种方法：

- 发送单个命令并退出；
- 发送多个命令。

要发送单个命令，必须将命令包含在 mysql 或 psql 命令行中，作为其中的一部分。

对于 mysql 命令，使用-e 参数来实现：

```
$ cat mtest2
#!/bin/bash
# send a command to the MySQL server
MYSQL='which mysql'

$MYSQL test -u test -e 'select * from employees'
```

```
$ ./mtest2
+-------+----------+------------+---------+
| empid | lastname | firstname  | salary  |
+-------+----------+------------+---------+
|     1 | Blum     | Rich       | 25000   |
|     2 | Blum     | Barbara    | 45000   |
|     3 | Blum     | Katie Jane | 34500   |
|     4 | Blum     | Jessica    | 52340   |
+-------+----------+------------+---------+
$
```

对于 psql 命令，使用-c 参数实现：

```
$ cat ptest2
#!/bin/bash
# send a command to the PostgreSQL server

PSQL='which psql'

$PSQL test -c 'select * from employees'
$ ./ptest2
empid  | lastname | firstname  | salary
-------+----------+------------+--------
    1  | Blum     | Rich       | 25000
    2  | Blum     | Barbara    | 45000
    3  | Blum     | Katie Jane | 34500
    4  | Blum     | Jessica    | 52340
(4 rows)
$
```

数据库服务器将 SQL 命令的结果返回给 shell 脚本，在 STDOUT 中显示这些结果。

如果需要发送多个 SQL 命令，可以使用文件重定向（请参阅第 12 章）。要在 shell 脚本中重定向命令行，必须定义一个文件结束字符串。文件结束字符串表示重定向数据的开始和结束。

下面是定义一个文件结束字符串的例子，中间包含有数据：

```
$ cat mtest3
#!/bin/bash
# sending multiple commands to MySQL

MYSQL="$(which mysql)"
$MYSQL test -u test <<EOF
show tables;
select * from employees where salary > 40000;
EOF
$ ./mtest3
Tables_in_test
employees
empid    lastname    firstname    salary
2        Blum        Barbara      45000
4        Blum        Jessica      52340
$
```

shell 将所有以 EOF 分隔符结尾的内容重定向到 mysql 命令，mysql 命令会像在提示符中输入这些行一样执行它们。使用这种方法，可以根据需要发送任意数目的命令到 MySQL 服务器。您将会看到在每个命令的输出之间没有分隔。在下一节“格式化数据”中，您将看到如何解决这个问题。

注意：还应该注意当使用重定向的输入方法时，mysql 程序会更改默认的输出风格。由于 mysql 程序检测到输入被重定向，它只返回原始数据，而不是在数据周围创建 ASCII 符号框。这使得抽取单个数据元素将非常方便。

同样的技术对 psql 程序也适用：

```
$ cat ptest3
#!/bin/bash
# sending multiple commands to PostgreSQL
```

```
PSQL="$(which psql)"

$PSQL test <<EOF
\dt
select * from employees where salary > 40000;
EOF
$ ./ptest3
        List of relations
 Schema |    Name    | Type  | Owner
--------+------------+-------+----------
 public | employees | table | postgres
(1 row)

 empid | lastname | firstname | salary
-------+----------+-----------+--------
     2 | Blum     | Barbara   | 45000
     4 | Blum     | Jessica   | 52340
(2 rows)
$
```

psql 程序直接在 STDOUT 中按指定的顺序显示每个命令的输出。

当然，不只限于检索表中的数据。您可以在脚本中使用任意类型的 SQL 命令，如 INSERT 语句：

```
$ cat mtest4
#!/bin/bash
# send data to the table in the MySQL database

MYSQL='which mysql'

if [ $# -ne 4 ]
then
   echo "Usage: mtest4 empid lastname firstname salary"
else
   statement="INSERT INTO employees VALUES ($1, '$2', '$3', $4)"
   $MYSQL test -u test << EOF
   $statement
EOF
   if [ $? -eq 0 ]
   then
      echo Data successfully added
   else
      echo Problem adding data
   fi
fi
$ ./mtest4
Usage: mtest4 empid lastname firstname salary
$ ./mtest4 5 Blum Jasper 100000
Data added successfully
$
$ ./mtest4 5 Blum Jasper 100000
ERROR 1062 (23000) at line 1: Duplicate entry '5' for key 1
Problem adding data
$
```

该例展示了使用该技术的一些事项。当指定文件结束字符串时，该行中必须只能有一个事件，而且该行必须以该字符串开始。如果将 EOF 与 if-then 的其他部分进行匹配，它将失效。

还要注意如何使用特殊的$?变量来测试 mysql 程序的退出状态。这将有助于确定命令是否失败。

只将命令的输出发送到 STDOUT 不是管理和操作数据的最简单方法。下一节将介绍可以使用的一些技巧，来帮助脚本捕获从数据库中获取的数据。

### 24.4.3 格式化数据

mysql 和 psql 命令的标准输出对数据获取的意义并不大。如果需要对获取的数据进行些加工，那么将需要进行些有趣的数据操作。本节将介绍一些帮助脚本捕获从数据库报表中获取的数据的技巧。

**1. 将输出赋值给变量**

尝试捕获数据库数据的第一步是将 mysql 和 psql 命令的输出重定向到环境变量。这样就可以在其他命令中使用输出信息。下面是一个例子：

```
$ cat mtest5
#!/bin/bash
# redirecting SQL output to a variable

MYSQL='which mysql'

dbs='$MYSQL test -u test -Bse 'show databases''
for db in $dbs
do
   echo $db
done
$ ./mtest5
information_schema
test
$
```

该例在 mysql 程序命令行中使用了两个附加参数。-B 参数指定 mysql 程序在批量模式下工作，同时结合了-s（silent）参数，不输出列头和格式化符号。

通过将 mysql 命令的输出重定向到一个变量，这个例子能够单步调试每个返回记录的各个值。

**2. 使用格式化标签**

从前一个例子可以看出，在 mysql 程序命令行中添加-B 和-s 参数可以不输出报头信息，因此得到的只有数据。还可以使用其他一些参数来使工作更容易。

现在生成网页数据很流行。mysql 和 psql 程序都提供一个选项来使用 HTML 格式显示输出。mysql 和 psql 程序都是通过使用-H 命令行参数来实现的：

```
$ psql test -H -c 'select * from employees where empid = 1'
<table border="1">
  <tr>
    <th align="center">empid</th>
    <th align="center">lastname</th>
    <th align="center">firstname</th>
   <th align="center">salary</th>
  </tr>
  <tr valign="top">
    <td align="right">1</td>
    <td align="left">Blum</td>
    <td align="left">Rich</td>
    <td align="right">25000</td>
  </tr>
</table>
<p>(1 row)<br />
</p>
$
```

mysql 程序还支持另一种流行格式，称为可扩展标记语言（Extensible Markup Language，XML）。该语言使用类 HTML 标签标识数据名称和值。

对于 mysql 程序，可以使用-X 命令行参数来实现这个功能：

```
$ mysql test -u test -X -e 'select * from employees where empid = 1'
<?xml version="1.0"?>
```

```
<resultset statement="select * from employees
  ">
 <row>
 <field name="empid">1</field>
 <field name="lastname">Blum</field>
 <field name="firstname">Rich</field>
 <field name="salary">25000</field>
  </row>
</resultset>
$
```

使用 XML，可以很容易地标识数据的每一行和每条记录中的各个数据值。

## 24.5　小结

本章讨论了通过 shell 脚本在数据库中保存，修改和获取数据的能力。可以通过 shell 脚本轻松地访问 MySQL 和 PostgreSQL 数据库服务器。

安装 MySQL 和 PostgreSQL 服务器后，可以使用它们各自的客户端程序通过命令行或 shell 脚本访问服务器。`mysql` 客户端程序提供到 MySQL 服务器的命令行接口。可以通过 shell 脚本发送 SQL 命令及自定义的 MySQL 命令给服务器，然后获取结果。

`psql` 客户端程序按同样的方式操作 PostgreSQL 服务器。有许多可以使用的命令行参数来帮助以正确的方式格式化数据。

两个客户端程序都允许发送单个命令给服务器，或使用输入重定向发送批量命令。程序通常将输出数据从服务器发送到 `STDOUT`，但可以将输出重定向到变量，然后在 shell 脚本中使用该信息。

下一章将讨论 World Wide Web。使 shell 脚本同 Internet 中的网站进行交互是个很麻烦的事情，但一旦掌握后，世界就在您手中，可以从中获取数据。

# 第 25 章
# 使用 Web

CHAPTER 25

本章内容：

- 使用 Lnfx 上网冲浪；
- 研究 cURL；
- zsh 中的客户端/服务器编程。

通常，我们很少把 shell 脚本编程和 Internet 联系在一起。命令行世界似乎与色彩斑斓的、图形化的网络世界无缘。然而，在脚本中有许多易于使用的工具可以用来获取 Web 上的数据内容和其他网络设备。本章将介绍三种 shell 脚本与网络交互的非常受欢迎的方法。

## 25.1 Lynx 程序

与 Internet 的历史一样久，Lynx 程序作为基于文本的浏览器，由 Kansas 大学的学生于 1992 年创建。由于它基于文本，因此 Lynx 程序允许直接从终端会话浏览 Web 站点，将 Web 页面上花哨的图形替换为 HTML 文本标记，从而允许从任何类型的 Linux 终端上网冲浪。Lynx 屏幕示例如图 25-1 所示。

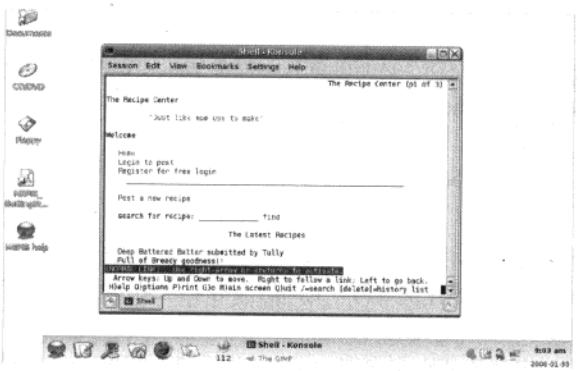



图 25-1 使用 Lynx 查看 Web 页面

Lynx 使用标准键盘在 Web 页面中导航。链接为 Web 页面中突出显示的文本。使用向右方向键可以跟随链接转到下一个 Web 页面。

您可能不知道如何在 shell 脚本中使用图形化的文本程序。Lynx 程序还提供了一种功能，即允许将 Web 页面的文本内容转储到 STDOUT。此功能非常适合于挖掘 Web 页面上包含的数据。本节将介绍如何在 shell 脚本中使用 Lynx 程序从 Web 站点上提取数据。

### 25.1.1　安装 Lynx

虽然 Lynx 程序已经诞生很久了，但它的开发还在进行中。在编写本书时，Lynx 的最新版本是 2.8.6，发布于 2006 年 10 月，并且还在开发新的版本。由于它在 shell 脚本程序员中非常流行，许多 Linux 部署都在默认安装中包括了 Lynx 程序。

如果您的安装没有提供 Lynx 程序，则可以查看发行版的安装包。一般可以从那里找到安装软件包以进行轻松安装。

如果您的发行版未包括 Lynx 程序，或者想获得最新版本，则可以下载源代码并自行编译（前提是 Linux 系统上已经安装了 C 开发环境库）。

注意：Lynx 程序使用 Linux 中的 curses 文本图形库。大多数发行版都默认安装了此库。如果发行版未安装此库，则可以在编译 Lynx 之前参考安装 curses 库的相关发行版说明。

下面是在 Linux 系统上下载、编译和安装 Lynx 的步骤：

（1）Lynx 项目的 Web 页面位于 `lynx.isc.org`。从该页上可以找到最新的版本和开发版本链接。可以选择下载压缩格式（如.`tar.gz` 或 `.zip`）的最新版本。

（2）将下载的文件提取到 Linux 系统上的工作目录（如$HOME 文件夹）：

```
tar -zxvf lynx2.8.6.tar.gz
```

（3）更改到结果工作目录（本例中为 lynx2.8.6）：

```
cd lynx2.8.6
```

（4）执行目录中的配置文件：

```
./configure
```

（5）执行 make 工具以编译源代码并创建可执行文件：

```
make
```

（6）作为根用户，将 lynx 可执行文件复制到 PATH 中的常用目录下，如`/usr/local/bin` 目录。

下一节将描述如何从命令行使用 `lynx` 命令。

### 25.1.2　Lynx 命令行

`lynx` 命令在从远程 Web 站点检索信息方面非常全面。当在浏览器中查看 Web 页面时，只能看到传输到浏览器的部分信息。Web 页面由三种类型的数据元素组成：

- HTTP 头部信息；
- Cookies；
- HTML 内容。

HTTP 头部信息提供连接中发送的数据类型信息，服务器发送数据和连接中使用的安全类型。如果发送特殊类型的数据，如视频或音频剪辑，则服务器将在 HTTP 头部识别出来。Lynx 程序允许查看 Web 页面会话中发送的所有 HTTP 头部信息。

如果您进行过任何类型的 Web 浏览，则一定熟悉 Web 页面的 cookies。Web 站点使用 cookies 保存访问 Web 站点的信息以便将来使用。每个单独的站点可保存信息，但只限于设置的访问信息。`lynx` 命令提供了一些选项来查看 Web 服务器发送的 cookies 和接受或拒绝发送自服务器的特殊 cookies。

Lynx 程序允许以三种格式查看 Web 页面的实际 HTML 内容：

- 使用 curses 图形库在终端会话中显示为文本图形；
- 显示为从 Web 页面转储的原数据的文本文件；
- 显示为从 Web 页面转储的原 HTML 源代码的文本文件。

对于 shell 脚本来说，查看原数据或 HTML 源代码就像一个金矿。一旦从 Web 站点捕获了检索的信息，就可以轻松地提取出各个信息片段。

如您所见，Lynx 程序的功能非常全面。但全面就容易变得复杂，特别是在命令行参数方面。Lynx

程序是 Linux 领域中最复杂的程序之一。

lynx 命令的基本格式是：

```
lynx options URL
```

其中 *URL* 是想要连接到的 HTTP 或 HTTPS 目标，而 *options* 是更改 Lynx 与远程 Web 站点交互行为的一个或多个选项。选项包括 Lynx 需要的任何类型的 Web 交互。表 25-1 展示了可以与 lynx 命令一起使用的所有可用命令行参数。

**表 25-1　　Lynx 命令参数**

| 参数 | 描述 |
|---|---|
| - | 从 STDIN 接收选项和参数 |
| -accept_all_cookies | 如果 Set-Cookie 操作选项设置为 on，则接受 cookies 而不弹出提示。默认设置为 off |
| -anonymous | 对匿名账户加以限制 |
| -assume_charset=*name* | 未指定字符集的默认字符集 |
| -assume_local_charset=*name* | 本地文件的默认字符集 |
| -assume_unrec_charset=*name* | 用于代替未能识别的字符集的默认字符集 |
| -auth=id:pw | 受保护文档的授权信息 |
| -base | 为-source 转储内容酝酿一个请求 URL 注释和 text/HTML 输出的 BASE 标记 |
| -bibhost=*URL* | 本地 bibp 服务器 URL（默认为 http://bibhost/） |
| -book | 将书签页作为 startfile。默认设置为 off |
| -buried_news | 对已埋葬引用的新条目的扫描实行切换。默认设置为 on |
| -cache=*n* | 内存中缓存的文档数 |
| -case | 启动区分大小写的用户搜索。默认设置为 off |
| -center | 对在 HTML<table>标记的中心对齐实行切换。默认设置为 off |
| -cfg=*filename* | 指定除默认 lynx.cfg 文件以外的配置文件 |
| -child | 在 startfile 中以向左箭头退出，并禁用保存到磁盘 |
| -cmd_log=*filename* | 将键盘命令记录到指定文件 |
| -cmd_script=*filename* | 从指定文件读取键盘命令 |
| -connect_timeout=n | 设置连接超时（以秒为单位）。默认为 18000 秒 |
| -cookie_file=*filename* | 指定用于读取 cookies 的文件 |
| -cookie_save_file=*filename* | 指定用于保存 cookies 的文件 |
| -cookies | 对 Set-Cookies 头部操作实行切换。默认设置为 on |
| -core | 对于致命错误强制核心转储实行切换。默认设置为 off |
| -crawl | 通过-traversal，使用-dump 将每页输出到文件，对输出的格式化与-traversal 类似，但输出到 STDOUT |
| -curses_pads | 使用 curses 的填补功能支持左右切换。默认设置为 on |
| -debug_partial | 使用 MessageSecs 延迟显示增量显示阶段。默认设置为 off |
| -delay=n | 在 statusline 消息上设置延迟（以秒为单位）。默认设置为 0.000 |
| -display=display | 为 X Window 程序设置可用显示 |
| -display_charset=*name* | 终端输出的字符集 |
| -dont_wrap_pre | 当设置-dump 和-crawl 时不在<pre>部分换行。在交互会话中标记换行。默认设置为 on |
| -dump | 将第一个 URL 转储到 STDOUT 并退出 |
| -editor=editor | 启用指定编辑器的编辑模式 |
| -emacskeys | 启用类似 emacs 的键移动。默认设置为 off |
| -enable_scrollback | 对回滚键功能实行切换。默认设置为 off |

续表

| 参数 | 描述 |
| --- | --- |
| -error_file=*filename* | 将 HTTP 状态代码写入到指定文件 |
| -exec | 启用本地程序执行 |
| -force_empty_hrefless_a | 强制不带 href 属性的<a>元素为空。默认设置为 off |
| -force_html | 强制将第一个文档作为 HTML 进行解析。默认设置为 off |
| -force_secure | 对 SSL cookies 要求安全标志。默认设置为 off |
| -forms_options | 使用基于表单的选项菜单。默认设置为 on |
| -from | 启用表单头部传输。默认设置为 on |
| -ftp | 禁用 ftp 访问。默认设置为 off |
| -get_data | 为从 STDIN 获取表单而读取数据，以---结束 |
| -head | 发送 HEAD 请求。默认设置为 off |
| -help | 打印用法信息 |
| -hiddenlinks=*option* | 指定如何处理隐藏链接。*Option* 可以是 merge、listonly 或 ignore |
| -historical | 使用“>”代替“-->”作为注释的结束。默认设置为 off |
| -homepage=URL | 将主页与始页区分开 |
| -image_links | 启用包含所有图像的链接。默认设置为 off |
| -index=URL | 设置默认的索引文件名 |
| -ismap | 当客户端 MAP 出现时，包含 ISMAP 链接。默认设置为 off |
| -link=n | 设置-crawl 产生的 lnk#.dat 文件的开始计数。默认设置为 0 |
| -localhost | 禁用指向远程主机的 URL。默认设置为 off |
| -locexec | 启用仅来自本地文件的本地程序的执行。默认设置为 off |
| -mime_header | 设置 mime 头并强制源转储 |
| -minimal | 使用最小模式代替有效的注释解析。默认设置为 off |
| -nested_tables | 使用 nested_table 逻辑。默认设置为 off |
| -newschunksize=n | 设置大块新闻列表的条目数 |
| -newsmaxchunk=n | 设置列表中新闻条目组成块之前的最大数目 |
| -nobold | 禁用明显的视频属性 |
| -nobrowse | 禁用目录浏览 |
| -nocc | 禁用邮件自我复制的 Cc:提示。默认设置为 off |
| -nocolor | 禁用颜色支持 |
| -noexec | 禁用本地程序执行。默认设置为 on |
| -nofilereferer | 禁用文件 URL 的引用头的传输。默认设置为 on |
| -nolist | 禁用转储中的链接列表功能。默认设置为 off |
| -nolog | 禁止将错误消息通过邮件发送给所有者。默认设置为 on |
| -nonrestarting_sigwinch | 使窗口大小更改处理器为非重新启动。默认设置为 off |
| -nopause | 禁止 statusline 消息的强制暂停 |
| -noprint | 禁用一些打印函数，如-restructions=print。默认设置为 off |
| -noredir | 不跟踪位置：重定向。默认设置为 off |
| -noreferer | 禁止引用头部传输。默认设置为 off |
| -noreverse | 禁止反向视频属性 |
| -nostatus | 禁止其他信息消息。默认设置为 off |
| -nounderline | 禁止强调视频属性 |
| -number_fields | 强制对链接编号和表单输入字段。默认设置为 off |

续表

| 参数 | 描述 |
| --- | --- |
| -number_links | 强制对链接编号。默认设置为 off |
| -partial | 在下载的同时显示页面的一部分。默认设置为 on |
| -partial_thres=*n* | 在使用部分显示逻辑的重新显示之前设置呈现的行数。默认设置为-1，即禁用此功能 |
| -pauth=id:pw | 为受保护的端口服务器设置授权信息 |
| -popup | 通过弹出窗口处理单个选择的 SELECT 选项，而不是通过单选按钮列表。默认设置为 off |
| -post_data | 为从 stdin 发布表单而读取数据，以---结束 |
| -preparsed | 使用-source 显示解析的 text/html 并以源视图可视化 lynx 对无效 HTML 的处理。默认设置为 off |
| -prettysrc | 使用语法在源视图中突出显示和进行超链接处理。默认设置为 off |
| -print | 启用打印函数，与-noprint 相反。默认设置为 on |
| -pseudo_inlines | 使用 pseudo-ALT 与非 ALT 字符串联机。默认设置为 on |
| -raw | 使用默认的 8 字节字符翻译设置或 CJK 模式开启字符集。默认设置为 off |
| -realm | 限制访问开始域中的 URL。默认设置为 off |
| -reload | 清空代理服务器上的缓存（仅涉及第一个文档）。默认设置为 off |
| -restrictions=*options* | 设置限制选项。使用不带参数的-restrictrions 查看列表 |
| -resubmit_posts | 当返回的文档使用 PREV_DOC 命令可见或从历史列表中可见时，使用方法 POST 强制重复提交（不缓存）。默认设置为 off |
| -rlogin | 禁用 rlogin 功能。默认设置为 off |
| -selective | 要求.www_browsable 文件以浏览目录 |
| -short_url | 启用在状态栏中检查长 URL 的开始和结尾。默认设置为 off |
| -show_cursor | 当设置为 off 时，在右下角处隐藏光标，否则显示光标。默认设置为 on |
| -show_rate | 显示传输速率。默认设置为 on |
| -soft_dquotes | 使用旧 Netscape 和 Mosaic bug 的效仿，即将‘>’当作双引号标识或标记。默认设置为 off |
| -source | 将第一个 URL 的源代码转储到 STDOUT 并退出 |
| -stack_dump | 禁用 SIGINT 清除处理器。默认设置为 off |
| -startfile_ok | 允许非 HTTP startfile 和带-validate 的 homepage。默认设置为 off |
| -stdin | 从 STDIN 读取 startfile。默认设置为 off |
| -tagsoup | 使用 TagSoup 而不是 SortaSGML 解析器。默认设置为 off |
| -telnet | 禁用 Telnet 会话。默认设置为 off |
| -term=term | 指定终端类型以进行模拟 |
| -tlog | 对当前会话使用 Lynx 跟踪日志。默认设置为 on |
| -tna | 使用“文本字段需要激活”模式。默认设置为 off |
| -trace | 使用 Lynx 跟踪模式。默认设置为 off |
| -trace_mask | 自定义 Lynx 的跟踪模式。默认设置为 0 |
| -traversal | 检索所有来自 startfile 的 HTTP 链接 |
| -trim_input_fields | 整理表单中的输入 text/textarea 字段。默认设置为 off |
| -underline_links | 使用链接的 underline/bold 属性。默认设置为 off |
| -underscore | 在转储中使用下划线格式。默认设置为 off |
| -use_mouse | 启用鼠标支持。默认设置为 off |
| -useragent=Name | 设置可选 Lynx 用户代理头部 |
| -validate | 仅接受 http URL（针对验证）实现，比-anonymous 更加严格，但允许对 http 和 https 使用 goto。默认设置为 off |

续表

| 参数 | 描述 |
| --- | --- |
| -verbose | 使用这些图像的带文件名的[LINK]、[IMAGE]和[INLINE]注释。默认设置为on |
| -version | 显示Lynx版本信息 |
| -vikeys | 启用类似vi的键移动。默认设置为off |
| -width=n | 设置格式化转储的屏幕宽度。默认为80列 |
| -with_backspaces | 如果在输出中使用-dump或-crawl参数，则显示括号。默认设置为off |

如您所见，可以直接从命令行控制任何类型的HTTP或HTML设置。例如，想要通过使用HTTP POST方法将数据发布到Web，则只需要将数据包括在-post-data参数中。如果想要将通过Web站点检索到的cookies保存到特殊位置，则可以使用-cookie_save_file参数。

许多在全屏模式中使用的、用于定义控制Lynx行为的命令行参数允许在扫描Web页面时自定义Lynx的行为。

在常规浏览环境中，经常有许多非常有用的成组的命令行参数。为了不用通过Lynx每次将这些参数输入到命令行上，Lynx提供了一个总配置文件来配置使用Lynx时的基本行为。下一节将讨论这个配置文件。

### 25.1.3 Lynx配置文件

lynx命令为它的许多参数设置读取配置文件。默认情况下，此文件位于/usr/local/lib/lynx.cfg，但也有许多Linux部署将此位置更改到/etc目录（/etc/lynx.cfg）。

lynx.cfg配置文件将相关参数分组为各个部分以便查找。配置文件中项的格式为：

*PARAMETER:value*

其中*PARAMETER*是参数的全称（通常，但不总是全大写字母），而*value*是与参数关联的值。

细读此文件，可以发现许多参数与命令行参数类似，如ACCEPT_ALL_COOKIES参数，它等价于-accept_all_cookies命令行参数。

还有一些配置参数与函数类似，但名称不同。FORCE_SSL_COOKIES_SECURE配置文件参数设置可以被-force_secure命令行参数覆盖。

然而，还有许多参数与命令行参数匹配不上。这些值只能在配置文件中设置。

最常见的、不能在命令行上设置的配置参数是用于端口服务器的参数。一些网络（特别是公司网络）使用端口服务器作为客户端浏览器和目标Web站点服务器的中间件。客户端浏览器不是将HTTP请求直接发送到远程Web服务器，而是必须将请求发送给端口服务器。端口服务器转而将请求发送给远程Web服务器，检索结果并将结果回复给客户端浏览器。

然而这看起来有些浪费时间，但对于保护客户端免受Internet的威胁却是非常重要的。端口服务器可以过滤不当的内容、恶意代码或者甚至是探测用于Internet钓鱼模式（伪装成其他服务器的流氓服务器以捕获客户端数据）的网站。端口服务器可以帮助减少Internet带宽的使用，因为它可以缓存经常查看的Web页面，将这些网页返回给客户端而不是再次下载原始的页面。

用于定义端口服务器的配置参数是：

```
http_proxy:http://some.server.dom:port/
https_proxy:http://some.server.dom:port/
ftp_proxy:http://some.server.dom:port/
gopher_proxy:http://some.server.dom:port/
news_proxy:http://some.server.dom:port/
newspost_proxy:http://some.server.dom:port/
newsreply_proxy:http://some.server.dom:port/
snews_proxy:http://some.server.dom:port/
snewspost_proxy:http://some.server.dom:port/
```

```
snewsreply_proxy:http://some.server.dom:port/
nntp_proxy:http://some.server.dom:port/
wais_proxy:http://some.server.dom:port/
finger_proxy:http://some.server.dom:port/
cso_proxy:http://some.server.dom:port/
no_proxy:host.domain.dom
```

可以为任何 Lynx 支持的网络协议定义不同的端口服务器。NO_PROXY 参数是以逗号分隔的希望直接访问而不使用端口服务器的 Web 站点。通常是不需要过滤的内部 Web 站点。

### 25.1.4 Lynx 环境变量

从丰富的命令行选项和配置文件参数可以看出，Lynx 程序的自定义程度很高。然而，定制范围不止如此。可以使用环境变量覆盖许多配置文件参数。如果您的工作环境不能访问 `lynx.cfg` 配置文件，则可以通过设置本地环境变量来覆盖一些默认参数。表 25-2 列出了更多常用的 Lynx 环境变量，在受限制的环境中使用 Lynx 时可能会用到这些变量。

表 25-2 Lynx 环境变量

| 变量 | 描述 |
|---|---|
| LYNX_CFG | 指定替代配置文件的位置 |
| LYNX_LSS | 指定默认 Lynx 字符集样式表的位置 |
| LYNX_SAVE_SPACE | 指定将文件保存到磁盘的位置 |
| NNTPSERVER | 指定检索和发布 USENET 新闻的服务器 |
| PROTOCOL_PROXY | 为特定的协议覆盖端口服务器 |
| SSL_CERT_DIR | 指定包含信任证书以访问受信任站点的目录 |
| SSL_CERT_FILE | 指定包含信任证书的文件 |
| WWW_HOME | 定义 Lynx 在开始时使用的默认 URL |

在使用 Lynx 程序之前，像设置其他环境变量一样设置这些环境变量：

```
$ http proxy=http://myproxy.com:8080
$ lynx
```

要指定端口服务器，必须提供协议、服务器名称和与端口服务器通信的端口。如果需要此变量设置，最好将它包含在常用的 shell 开始文件中（如 bash shell 的 `.bashrc` 文件），以便不必每次都进行输入。

### 25.1.5 从 Lynx 捕获数据

在 shell 脚本中使用 Lynx 时，很有可能试图从 Web 页面获取一小段（或几段）信息。完成此操作的技术称为**屏幕抓取**（screen scraping），在屏幕抓取过程中，试图以编程方式找到图形屏幕上特定位置的数据，以便捕获并在 shell 脚本中使用。

执行屏幕抓取最简单的办法是使用带 `-dump` 选项的 `lynx`。此选项不会在终端屏幕上显示 Web 页面，而是将文本数据直接显示到 STDOUT 上：

```
$ lynx -dump http://localhost/RecipeCenter/

The Recipe Center

        "Just like mom use to make"
Welcome

   [1]Home
   [2]Login to post
   [3]Register for free login

[4]Post a new recipe
```

每个链接被定义为标记号，Lynx 在 Web 页面数据后显示所有标记引用的列表。

当拥有 Web 页面的所有文本数据时，您可能已经猜到我们将要从工具箱中使用什么工具来开始提取数据的工作了。对了，就是 sed 和 gawk 程序（参阅第 16 章）。

首先，让我们找出一些需要收集的数据。Yahoo!天气 Web 页面包含有关世界城市当前天气的丰富资源。每个地点使用单独的 URL 来显示该城市的天气信息（通过使用常规浏览器访问站点，可以找到特定城市的特定 URL）。找出伊利诺斯州的芝加哥天气的 Lynx 命令是：

```
lynx -dump http://weather.yahoo.com/forecast/USIL0225.html
```

此命令从 Web 页面转储出大量的数据。第一步是找到我们想要的数据。要完成此操作，请将 lynx 命令的输出重定向到一个文件，然后搜索文件以获得数据。执行完上面的命令后，可以在输出文件中发现如下文本：

```
    Current conditions as of 1:54 pm EDT

Light Drizzle

   Feels Like:
          50°

   Barometer:
          29.34 in and falling

   Humidity:
          93%

   Visibility:
          4 mi

   Dewpoint:
          48°

   Wind:
          S 16 mph
```

这就是我们需要的关于当前天气情况的所有信息。这个输出只有一个小问题。您可能已经注意到数字位于标题的下面一行。只提取出单独的数字将非常困难。第 18 章讨论了如何处理此类问题。

解决此问题的关键是编写 sed 脚本先搜索数据标题。当找到数据标题时，就可以转到正确的行提取数据了。幸运的是，在本例中，所有数据都位于自己的行上。只用 sed 脚本就可以解决此问题。如果同一行上还有其他文本，则需要使用 gawk 工具筛选出需要的数据。

首先，需要创建 sed 脚本查找**当前条件**（current condition）文本，跳过两行获取描述当前天气的文本并打印出来。如下所示：

```
$ cat sedcond
/Current conditions/{
n
n
p
}
$ .
```

地址指定查找所需文本的行。如果 sed 找到需要的文本，则两个 n 命令将跳过接下来的两行，然后 p 命令打印出下面的行，即描述城市当前天气条件的文本。

接下来，需要 sed 脚本搜索 Feels Like:文本，然后转到下一行打印气温：

```
$ cat sedtemp
/Feels Like:/{
n
p
}
$
```

非常好。现在，可以在 shell 脚本中使用这两个 sed 脚本首先捕获 Web 页面的 lynx 输出到临时文件，然后将两个 sed 脚本应用到 Web 页面数据以仅提取需要查找的数据。如下面的示例所示：

```
$ cat weather
#!/bin/bash
# extract the current weather for Chicago, IL

URL="http://weather.yahoo.com/forecast/USIL0225.html"
LYNX='which lynx'
TMPFILE='mktemp tmpXXXXXX'
$LYNX -dump $URL > $TMPFILE
conditions='cat $TMPFILE | sed -n -f sedcond'
temp='cat $TMPFILE | sed -n -f sedtemp'
rm -f $TMPFILE
echo "Current conditions: $conditions"
echo The current temp outside is: $temp
$ ./weather
Current conditions: Light Rain
The current temp outside is: 49°
$
```

天气脚本连接到 Yahoo!天气目标城市的 Web 页面，将 Web 页面保存到临时文件，提取适当的文本，删除临时文件，然后显示天气信息。这个脚本的好处在于，一旦从 Web 站点提取了数据，便可以任意操作这些数据，如创建气温表。然后还可以创建每天运行的 cron 作业（参阅第 13 章）来跟踪日常的气温。

警告：Internet 是一个充满活力的地方。如果在提取数据在 Web 页面上的精确位置上花上几个小时，才发现它已经运行了几个星期，不要觉得吃惊，请终止脚本。事实上，在您阅读本书时我们的示例可能已经不能正常工作了。重点是了解从 Web 页面提取数据的过程。然后将这些原则应用到其他情况中。

## 25.2 cURL 程序

Lynx 的流行催生了另一个类似的称之为 cURL 的程序。cURL 程序允许自动从命令行使用特定的 URL 传输文件。它目前支持 URL 中指定的 FTP、FTPS、HTTP、HTTPS、SCP、SFTP、TFTP、Telnet、DICT、LDAP、LDAPS 和 FILE 协议。

虽然 cURL 之前没有被用作 Web 页面浏览器，但它允许从命令行或 shell 脚本以无人值守模式、需使用简单的命令直接轻松发送和检索数据。这为我们的 shell 编程工具箱又添加了一个很棒的工具。

警告：还有一种编程语言称之为 curl，由 Sumisho Computor System Corporation 所有并发布到市场。请不要混淆 cURL 和 curl 程序语言。

### 25.2.1 安装 cURL

随着 cURL 越来越流行，它已经成为许多 Linux 发行版的默认安装内容。如果您的 Linux 发行版没有此程序，或者希望使用最新的版本，则可以下载源代码并在自己的系统上进行编译。另外，必须在 Linux 系统上安装 C 开发库才能完成此操作。

下载和安装 cURL 的步骤如下所示：

（1）转到主 cURL 的 Web 站点 curl.haxx.se，然后单击此页左侧的下载链接。

（2）单击 cURL 最新版本的链接并选择归档类型（如.tar.gz）。

（3）更改到结果工作目录（本例中为 curl-7.18.0）：

```
cd curl-7.18.0
```

（4）在目录中执行配置脚本：

```
./configure
```

（5）执行 make 工具以编译源代码和创建可执行文件：

```
make
```

（6）作为 root 用户，将 curl 可执行文件复制到 PATH 常规目录中，如/usr/local/bin 目录。下一节将描述如何从命令行使用 curl 程序。

## 25.2.2　cURL 命令行

与 Lynx 程序命令行类似，curl 命令行也非常复杂。有许多选项可用于精确控制 cURL 与远程服务器交互的方式。表 25-3 显示了 cURL 可用的命令行参数。

表 25-3　　cURL 命令行参数

| 长参数 | 短参数 | 描述 |
|---|---|---|
| --append | -a | 在上传时追加到目标文件 |
| --user-agent *name* | -A | 用户代理发送到服务器 |
| --anyauth | | 使用 ANY 验证模式 |
| --cookie *name=string* | -b | Cookie 字符串或读取 cookies 的文件 |
| --basic | | 使用 HTTP 基本验证 |
| --use-ascii | -B | 使用 ASCII 传输数据 |
| --cookie-jar *file* | -c | 操作后将 cookies 写入到指定的文件 |
| --continue-at offset | -C | 在 offset 上恢复文件传输 |
| --data data | -d | 使用 HTTP POST 方法发送数据 |
| --data-ascii *data* | | 使用 HTTP POST 方法发送 ASCII 数据 |
| --data-binary *data* | | 使用 HTTP POST 方法发送二进制数据 |
| --negotiate | | 使用 HTTP 谈判验证 |
| --digest | | 使用 HTTP 摘要验证 |
| --disable-eprt | | 禁止使用 DPRT 或 LPRT |
| --disable-epsv | | 禁止使用 EPSV |
| --dump-header *file* | -D | 将 HTTP 会话报头写入到指定文件 |
| --egd-file *file* | | 为随机数据指定 EGD 套接字路径 |
| --tcp-nodelay | | 使用 TCP 的 NODELAY 选项 |
| --referer | -e | 引用 URL |
| --cert cert:passwd | -E | 指定客户端证书文件和密码 |
| --cert-type *type* | | 指定证书文件类型(DER/PEM/ENG) |
| --key key | | 指定私钥文件 |
| --key-type *type* | | 指定私钥文件类型(DER/PEM/ENG) |
| --pass *pass* | | 指定私钥的 passphrase |
| --engine *eng* | | 指定要使用的 crypto 引擎 |
| --cacert *file* | | 指定 CA 证书以验证远程峰值 |
| --capath *dir* | | 指定 CA 目录以验证远程峰值 |
| --ciphers *list* | | 指定 SSL 密码列表以便在 SSL 连接中使用 |
| --compressed | | 请求压缩的响应 |
| --connect-timeout *sec* | | 指定连接的最大时间（以秒为单位） |
| --create-dirs | | 创建需要的本地目录层次结构 |
| --crlf | | 在上传时将 LF 转换为 CRLF |
| -fail | -f | 遇到 HTTP 错误时以静默方式失败 |
| --ftp-create-dirs | | 如果不存在，则创建远程目录 |
| --ftp-pasv | | 在传输中使用 PASV/EPSV，而不是 PORT |

续表

| 长参数 | 短参数 | 描述 |
| --- | --- | --- |
| --ftp-skip-pasv-ip | | 跳过 PASV 的 IP 地址 |
| --ftp-ssl | | 启用 FTP 传输的 SSL/TLS |
| --form name-*content* | -F | 指定 HTTP 多部件 POST 数据 |
| --form-string name-*string* | | 指定 HTTP 多部件 POST 数据 |
| --globoff | -g | 使用{}和[]禁用 URL 序列和范围 |
| -get | -G | 使用 HTTP GET 发送-d 数据 |
| --help | -h | 显示帮助文本文件 |
| --header *header* | -H | 指定要传递给服务器的自定义 HTTP 头部 |
| --ignore-content-length | | 忽略 HTTP 内容长度头部 |
| --include | -i | 在输出中包括协议的头部 |
| --head | -I | 只显示文档信息 |
| --junk-session-cookies | -j | 忽略从文件读取的会话 cookie |
| --interface *int* | | 指定要使用的网络接口 |
| --krb4 level | | 以指定的安全级别启用 krb4 |
| --insecure | -k | 不经确定而允许连接到 SSL 站点 |
| --config | -K | 指定读取的配置文件 |
| --list-only | -l | 只列出 FTP 目录的名称 |
| --limit-rate *rate* | | 将传输速度限制为 rate 字节/秒 |
| --location | -L | 遵循 HTTP 位置：头部 |
| --location-trusted | | 遵循位置：甚至将验证发送给其他主机名 |
| --max-time sec | -m | 允许文件传输的最大时间（以秒为单位） |
| --max-redirs *num* | | 最大重定向次数 |
| --max-filesize *bytes* | | 下载的最大文件大小（以字节为单位） |
| --manual | -M | 显示完整手册 |
| --netrc | -n | 必须读取.netrc 获得用户名和密码 |
| --netrc-optional | | 使用.netrc 或者 UR。覆盖-n |
| --ntlm | | 使用 HTTP NTLM 验证 |
| --no-buffer | -N | 禁用输出流的缓存 |
| --output *file* | -o | 将输出写到文件而不是 STDOUT |
| --remote-name | -O | 将输出写到以远程文件命令的文件 |
| --proxytunnel | -p | 通过 HTTP 代理通道操作 |
| --proxy-anyauth | | 使用任意代理验证方法 |
| --proxy-basic | | 在代理上使用 Basic 验证 |
| --proxy-digest | | 在代理上使用 Digest 验证 |
| --proxy-ntlm | | 在代理上使用 NTLM 验证 |
| --ftp-port *address* | -P | 为 FTP 使用 TCP 端口地址而不是 PASV |
| | -q | 如果用作第一个参数，则禁止读取.curlrc 文件 |
| --quote *cmd* | -Q | 在文件传输之前，将命令 cmd 发送到服务器 |
| --range *range* | -r | 检索来自 HTTP/1.1 或 FTP 服务器的字节范围 |
| --random-file *file* | | 指定文件用来为 SSL 读取随机数据 |
| --remote-time | -R | 在本地输出上设置远程文件的时间 |
| --retry *num* | | 如果发生短暂问题，则再尝试 num 次 |

续表

| 长参数 | 短参数 | 描述 |
|---|---|---|
| --retry-delay *sec* | | 在重新尝试时，在每次尝试之前等待 sec 秒 |
| --retry-max-time *sec* | | 只在 sec（秒）之内尝试 |
| --silent | -s | 静默模式。不输出任何内容 |
| --show-error | -S | 显示错误。使用-S 选项，在错误发生时显示错误 |
| --socks host:port | | 在指定的主机和端口上使用 SOCKS5 代理 |
| --stderr *file* | | 指定文件以重定向 STDERR。使用-redirects 到 STDOUT |
| --telnet-option OPT=val | -t | 设置 Telnet 选项 |
| --trace *file* | | 将错误跟踪写入指定的文件 |
| --trace-ascii *file* | | 类似于--trace，但无十六进制输出 |
| --trace-time | | 为跟踪/冗长输出添加时间戳 |
| --upload-file *file* | -T | 指定文件以传输到远程站点 |
| --url *URL* | | 指定要连接的 URL |
| --user user:password | -u | 指定远程服务器的用户 ID 和密码 |
| --proxy-user user:password | -U | 指定代理服务器要求的用户 ID 和密码 |
| --verbose | -v | 如果可用，则显示更多输出信息 |
| --version | -V | 显示版本号并退出 |
| --write-out *format* | -w | 指定完成后显示的文本 |
| --proxy *host:port* | -x | 指定 HTTP 代理服务器的主机名和端口 |
| --request *cmd* | -X | 指定要使用的请求命令 |
| --speed-time | -y | 激发一个--speed-limit 忽略需要的时间（以秒为单位）。默认为 30 秒 |
| --speed-limit | -Y | 如果--Speed-time 设置小于 Speed-limit，则停止传输 |
| --time-cond *time* | -z | 为传输设置时间条件 |
| --http1.0 | -0 | 使用 HTTP 1.0 |
| --tlsv1 | -1 | 使用 TLSv1 |
| --sslv2 | -2 | 使用 SSLv2 |
| --sslv3 | -3 | 使用 SSLv3 |
| --3p-quote | | 类似于源用于第三方传输的 URL 的-Q |
| --3p-url | | 激活第三方传输的源 URL |
| --3p-user | | 用于源第三方传输的用户和密码 |
| --ipv4 | -4 | 将名称解析为 IPv4 地址 |
| --ipv6 | -6 | 将名称解析为 IPv6 地址 |
| --progress-bar | -# | 将传输进程显示为进度条 |

命令行参数的列表与 Lynx 程序类似。大多数参数定义了如何与远程服务器接口。

现在，让我们看看 curl 命令能做些什么。

### 25.2.3　研究 curl

默认情况下，cURL 将 Web 页面的完整 HTML 代码返回给 STDOUT：

```
$ curl http://localhost/RecipeCenter/
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<link rel="stylesheet" type="text/css" href="mystyle.css" />
```

```
<link rel="stylesheet" media="print" type="text/css" href="print.css"
/>
<title>The Recipe Center</title>
</head>

<body>
<table width="100%" border="0">
  <tr>
    <td id="header" height="90" colspan="3">
        <h1><br>
  The Recipe Center</h1>
<h4>          
<em> "Just like mom use to make"</em></h4></td>
  </tr>
  <tr>
    <td id="nav" width="15%" valign="top">
        <table width="100%" cellpadding="3">
  <tr>
    <td><h3>Welcome</h3></td>
  </tr>
  <tr>
[ listing truncated ]
```

与 Lynx 程序类似，可以使用标准的 shell 编程技术从转储的 Web 页面提取各个数据元素。

我之所以喜欢 cURL 是因为它可以批量下载文件。这些天我就在下载最新的 Linux 发行版 ISO 文件。因为 ISO 文件非常大，所以我需要在下载时做些别的事情。只要知道 ISO 文件的 URL，就可以使用 cURL 创建简单的 shell 脚本自动执行该进程：

```
$ cat downld
#!/bin/bash
# download latest cURL file automatically
curl -s -o /home/rich/curl-7.18.0.tar.gz
http://curl.haxx.se/downloads/curl-7.18.0.tar.gz
$
```

请注意，`curl` 命令在脚本中都在同一行上。此脚本访问并直接从 cURL Web 站点下载文件。现在，可以使用 `at` 或 `cron` 命令（参阅第 13 章）将下载设置在晚间不使用 PC 或网络时。

## 25.3 使用 zsh 连接网络

第 23 章描述了所有 zsh shell 中可用的功能。zsh shell 是 Linux 和 Unix 环境中一种较新的 shell。zsh shell 的功能之一就是插件模块。它不是将许多功能都结合到核心 zsh shell 中，而是使用特殊的模块，以便用户可以获取和选择需要加载的命令，而其中就有 TCP 模块。

zsh shell 中的 TCP 模块提供了许多可以直接从命令行使用的、非常好的网络功能。可以从命令行（或 shell 脚本）直接创建完整的 TPC 网络与另一个网络设备的会话。本节将讨论 zsh TCP 模块的功能，并展示一个简单的使用 zsh shell 脚本构建的客户端/服务器应用程序。

### 25.3.1 TCP 模块

zsh shell 使用模块为核心 zsh shell 添加其他功能。每个模块包含特定于专项领域的内置命令。TCP 模块提供了大量用于网络功能的内置命令。

要将 TCP 模块安装到 zsh shell 中，请执行下面的命令：

```
% zmodload zsh/net/tcp
%
```

添加模块就是这么简单！如果在 shell 脚本中使用 TCP 模块，请记住在脚本中包括此行。模块只应用于当前 shell。

装载 TCP 模块后，需要获取 ztcp 命令。ztcp 命令的格式为：

```
ztcp [-acflLtv] [ -d fd] [args]
```

可用的命令行选项有以下几种。

- -a：接受新的连接。
- -c：关闭现有连接。
- -d：为连接使用特殊文件描述符。
- -f：强制连接关闭。
- -l：打开新的套接字以进行监听。
- -L：列出目前已连接的套接字。
- -t：如果没有连接被挂起则退出。
- -v：显示关于连接的详细信息。

ztcp 命令使用文件描述符与打开的 TCP 连接交互。默认情况下，zsh 使用环境变量$RESULT 引用文件描述符，而 TCP 模块会将其转发给远程主机。类似地，如果远程主机发送了任何数据，则需要做的只是从$RESULT 变量指定的文件描述符中读取内容。网络编程不过如此。

### 25.3.2　客户端/服务器模型

在投入使用 zsh shell 创建客户端/服务器程序的工作之前，最好先了解客户端和服务器程序的工作方式。显然，它们在连接和传输数据方面各有各的分工。

**服务器**程序侦听网络上来自**客户端**的请求。客户端向服务器发起连接请求。一旦服务器接受连接请求，则两端的设备便可以使用双向的通信通道发送和接收数据。此过程如图 25-2 所示。

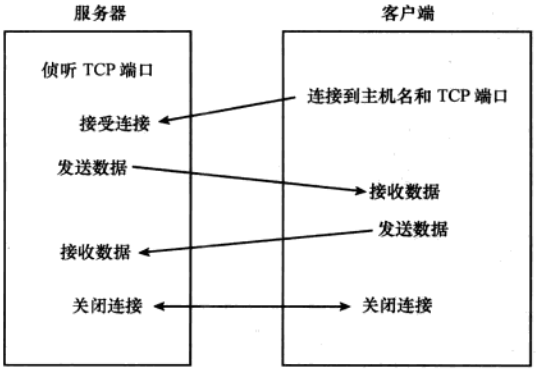



图 25-2　客户端/服务器通信图

如图 25-2 所示，在与客户端通信之前，服务器必须执行两个功能。首选，它必须建立特定 TCP 端口以侦听到来的请求。当连接请求到达时，它必须接受连接。

客户端的任务就简单得多。它所要做的只是试图连接到服务器上正在进行侦听的特定 TCP 端口。如果服务器接受连接，则双向通信可用并可以发送数据。

服务器和客户端建立起连接后，两端的设备之间必须有某种通信方法或规则。如果两个设备同时侦听一个消息，则将死锁且不会有任何反应。类似地，如果它们试图同时发送一个消息，则双方都不会成功。

网络程序员的任务就是决定客户端和服务器程序必须遵循的协议规则。

### 25.3.3　使用 zsh 进行客户端/服务器编程

要演示使用 zsh 创建客户端/服务器程序，我们使用一个简单的网络应用程序。我们创建的服务器

程序将侦听 TCP 端口 5150 上的连接请求。当连接请求到达时，服务器将接受它，然后向客户端发送一条欢迎消息。

然后，服务器程序将等待收取来自客户端的消息。如果收到消息，则服务器将显示消息，然后将相同的消息发送给客户端。发送消息后，服务器将循环回到侦听另一则消息。循环将持续，直到服务器收到由文本 exit 组成的消息。此时，服务器将终止会话。

我们创建的客户端程序将向服务器的 TCP 端口 5150 发送连接请求。建立起连接后，客户端需要接收服务器的欢迎消息。

收到消息后，客户端将显示消息，然后向用户询问要发送到服务器的数据。从用户处得到消息后，客户端程序将其发送给服务器并等待收取返回消息。如果消息返回，则客户端将显示消息并循环回到询问用户的另一则消息中。此循环将持续，直到用户输入文本 exit。当客户端将退出文本发送给服务器后，会话终止。

下一节将介绍服务器和客户端程序。

**1. 服务器程序**

服务器程序的代码如下：

```
% cat server
#!/bin/zsh
# zsh TCP server script
zmodload zsh/net/tcp
ztcp -l 5150
fd=$REPLY

echo "Waiting for a client..."
ztcp -a $fd
clientfd=$REPLY
echo "client connected"

echo "Welcome to my server" >& $clientfd

while [ 1 ]
do
   read line <& $clientfd
   if [[ $line = "exit" ]]
   then
      break
   else
      echo Received: $line
      echo $line >& $clientfd
   fi
done
echo "Client disconnected session"
ztcp -c $fd
ztcp -c $clientfd
%
```

服务器程序遵循图 25-2 所示的客户端/服务器模型。它首先使用-l 参数侦听特定的端口（5150）。$RESULT 变量包含 Linux 系统返回以识别连接的文件描述符。服务器使用-a 参数接受新的连接请求。此命令将执行等待，直到新的请求到达（称为**阻塞**）。脚本不会继续执行直至请求被接受。

每个客户端连接使用来自侦听端口文件描述符的单独文件描述符。这允许在需要的情况下维护多个客户端连接（此简单示例未涉及此内容）。

接受连接后，服务器将欢迎消息重定向到客户端文件描述符：

```
echo "Welcome to my server" >& $clientfd
```

zsh shell TCP 模块处理所能确定数据发送到远程客户端的任务。

接下来，服务器程序进入无限循环。它使用 read 命令等待数据从客户端返回：

```
read line <& $clientfd
```

此命令还阻止脚本执行，直到从客户端收到数据为止。如果客户端失去与网络的连接，那么这将变得非常糟糕。要防止此问题发生，可以在 read 行上使用-t 选项指定超时值（以秒为单位）。如果服务器在超时期限内未从客户端收到数据，则将继续执行。

如果服务器从客户端收到了数据，则在 STDOUT 上显示数据，然后将它发送回给客户端。如果数据等于文本字符串 exit，则服务器退出循环，然后使用 ztcp 中的-c 参数关闭客户端的文件描述符和侦听端口的文件描述符。如果想让服务器侦听另一个连接，那么在关闭客户端的文件描述符后，可以返回等待接受新的连接。

**2. 客户端程序**

客户端 shell 脚本程序的代码如下：

```
% cat client
#!/bin/zsh
# zsh TCP client program
zmodload zsh/net/tcp

ztcp localhost 5150
hostfd=$REPLY

read line <& $hostfd
echo $line

while [ 1 ]
do
   echo -n "Enter text: "
   read phrase
   echo Sending $phrase to remote host...
   echo $phrase >& $hostfd
   if [[ $phrase = "exit" ]]
   then
      break
   fi
   read line <& $hostfd
   echo " Received: $line"
done
ztcp -c $hostfd
%
```

客户端程序必须指定服务器程序所运行的系统的 IP 地址（或主机名），以及服务器正在侦听的适当的 TCP 端口号。服务器接受连接时，ztcp 程序为连接设置文件描述符并保存为$REPLY 变量中的值。客户端程序读取服务器的欢迎消息并进行显示：

```
read line <& $hostfd
echo $line
```

接下来，客户端程序进入 while 循环，要求用户输入发送到服务器的文本，读取输入的文本，然后将文本发送到服务器。在发送文本后，它查看输入的文本是否是 exit。如果是，则退出循环并关闭文件描述符，即关闭 TCP 连接。如果文本不是 exit，则等待服务器的响应并显示它。

**3. 运行程序**

可以在网络上两个独立的 Linux 系统上运行这些程序，或者从同一系统的两个终端会话上运行。首先必须运行服务器程序，以便在客户端启动时侦听进入的连接：

```
% ./server
Waiting for a client...
```

然后启动客户端：

```
% ./client
Welcome to my server
Enter text: test
Sending test to remote host...
   Received: test
```

客户端连接时，可以在服务器上看到如下内容：

```
client connected
```

```
Received: test
```

这将持续到用户在客户端上输入 exit 为止：

```
Enter text: exit
Sending exit to remote host...
%
```

然后应该看到服务器自动退出：

```
Client disconnected session
%
```

现在，我们已经能够开始创建完整的网络程序了！通过使用 zsh shell 和 TCP 模块，可以轻松地在网络上各个独立系统的 shell 脚本操作之间发送数据。

## 25.4 小结

本章介绍了 shell 脚本与 Internet 交互的方法。最受欢迎的工具是 Lynx 程序。Lynx 是可以使用文本模式图形在终端会话中显示 Web 站点信息的命令行程序。除了这些功能，Lynx 还提供了从 Web 站点检索原数据并显示到 STDOUT 的方法。还可以使用 Lynx 从 Web 站点提取数据，然后使用标准的文本处理工作如 sed 和 gawk 解析数据以查找特定信息。

cURL 程序是又一个与 Internet 交互的便利工具。cURL 程序还允许从 Web 站点转储数据，它还提供了在脚本中从许多不同类型的服务器上下载文件的方法。

最后，本章介绍了如何使用 zsh shell 的 TCP 模块编写自己的网络程序。zsh shell 为网络中独立系统上的 shell 脚本之间的通信提供了一种简单方法，就像对一个文件读取和写入数据一样简单。

下一章我们将研究如何在 shell 脚本中使用电子邮件。通常，使用 shell 脚本创建自动化过程时，如果能收到脚本是否成功的消息则会非常有帮助。通过了解系统上安装的电子邮件软件的工作方式，可以轻松地向世界上的任何人发送自动消息。

CHAPTER 26

# 第 26 章 使用电子邮件

本章内容：

- 电子邮件和 Linux；
- 设置电子邮件服务器；
- 发送简单的消息；
- 使用 Mutt 发送附件。

随着电子邮件的流行，如今几乎每个人都有一个电子邮件地址。因此，有些人更喜欢通过电子邮件接收数据，而不是查看文件或输出。在 shell 脚本中也是这样。如果从 shell 脚本生成任何类型的报告，则很有可能需要将其通过电子邮件发送给某人。本章将介绍如何设置 Linux 系统，以使其支持直接从 shell 脚本发送电子邮件。本章还将介绍如何确保 Linux 系统可以将邮件消息发送到系统之外，以及如何确保邮件客户端支持从命令行发送邮件。但首先，本章将从整体上介绍 Linux 处理电子邮件的方式。

## 26.1 Linux 电子邮件基础知识

有时，在 shell 脚本中使用电子邮件最难的部分是理解电子邮件系统在 Linux 上如何工作。弄清楚什么软件包执行什么任务是从 shell 脚本获取电子邮件到邮箱的关键。本节将介绍 Linux 系统如何使用电子邮件，以及在使用前需要具备什么条件。

### 26.1.1 Linux 中的电子邮件

Linux 系统的电子邮件系统源于 Unix 环境。Unix 操作系统的主要目标之一就是将软件模块化。与可以处理所有必要功能的大型程序相比，Unix 开发者创建较小的程序，每个小程序处理系统总功能的一小块。

Unix 在实现电子邮件系统时使用了这一原理，现在它被移植到了 Linux 环境中。在 Linux 中，电子邮件功能被分割为独立的小块，每一块分配给独立的程序。图 26-1 展示了大多数开源电子邮件软件如何在 Linux 环境下将电子邮件功能模块化。

如图 26-1 所示，在 Linux 环境中，电子邮件进程通常分为三块功能：

- 邮件传输代理（MTA）；
- 邮件分发代理（MDA）；
- 邮件用户代理（MUA）。

这三个功能之间的界线通常非常模糊。有些电子邮件软件包将 MDA 和 MTA 功能结合在一起，而一些其他的包将 MDA

和 MUA 功能结合起来。下面几节将详细介绍这些基本电子邮件组件以及它们在 Linux 系统上的实现方式。

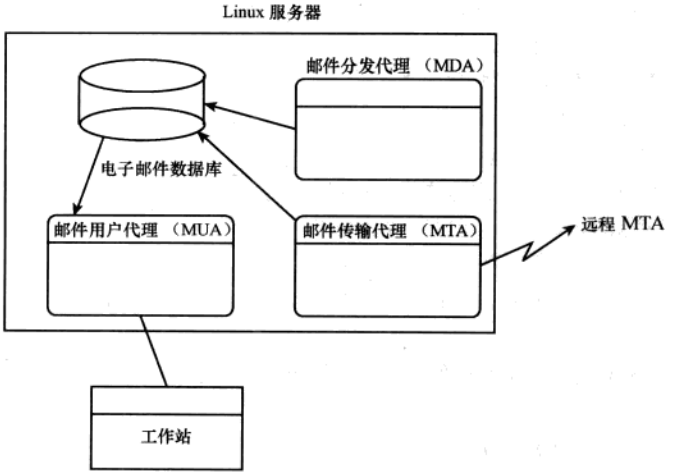



图 26-1 Linux 模的电子邮件环境

## 26.1.2 邮件传输代理

MTA 软件是 Linux 电子邮件系统的核心软件。它负责系统上进入和发出的邮件消息。对于每个发出的消息，MTA 必须决定收件地址。如果目标主机是本地系统，则 MTA 可以直接将其发送到本地邮箱或将消息交给本地 MDA 去发送。

然而，如果目标主机是远程邮件服务器，则 MTA 必须建立与远程主机的 MTA 软件的连接才能传送消息。MTA 软件包将邮件发送到远程主机的方法有两种：

- 直接发送；
- 代理发送。

如果 Linux 系统直接连接到 Internet，则总可以将消息发送给远程主机上的收件人。MTA 软件使用域名系统（Domain Name System，NDS）解析相应的网络 IP 地址以分发邮件消息，然后使用简单邮件传输协议（Simple Mail Transfer Protocol，SMTP）建立 TCP 连接。

大多数情况下，主机未直接连接到 Internet，或者不直接与其他远程主机通信。在这些情况下，通常使用**智能主机**（smart host）。智能主机是一台代理服务器，用于从 Linux 系统接收邮件消息，然后尝试直接将其分发给收件人。

注意：由于应答垃圾邮件的涌现，智能主机越来越难以在 Internet 上应用。无赖服务器会发送应答垃圾邮件，成千上万的自动商业广告邮件消息从智能主机以匿名方式发送出来。现在，大多数智能主机在将消息转发给其他主机时都要求一定形式的身份验证。

对于进入的信息，MTA 能够接收来自远程邮件服务器的请求，并接收发送给本地用户的消息。同样，此过程使用的最常用协议是 SMTP。

Linux 环境有许多不同类型的开源 MTA 程序。每种程序都提供了区别于其他程序的不同功能。目前为止，您可能遇到的最流行的两个程序是：

- sendmail；
- Postfix。

我们将在“设置服务器”一节详细介绍这两个电子邮件 MTA 包。

### 26.1.3　邮件分发代理

MDA 程序的责任是将消息分发给本地用户。它从 MTA 程序接收信息并且必须清楚地判断以何种方式分发消息以及分发消息的地址。图 26-2 展示了 MDA 程序与 MTA 程序交互以分发邮件的方式。

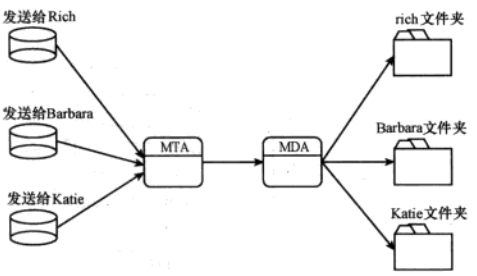



图 26-2　在邮件服务器上使用 MDA 程序

虽然有时 MDA 功能在 MTA 程序自身内部执行，但通常 Linux 电子邮件的实现依赖一个单独的独立 MDA 程序将消息发送给本地用户。因为这些 MDA 程序只集中将邮件发送给本地用户，所以可以为包括 MDA 功能的 MTA 程序添加其所不具备的额外的功能。这使邮件管理员可以向邮件用户提供额外的邮件功能，如过滤垃圾邮件，外出时的自动重定向和自动邮件排序等。

MDA 程序收到消息时，它必须确保消息发送到正确的位置，即本地用户的邮箱或者是本地用户定义的其他位置。

目前，在 Linux 系统上通常使用的 3 种邮箱类型是：

- `/var/spool/mail` 或`/var/mail` 文件；
- `$HOME/mail` 文件；
- Maildir 样式的邮箱目录。

每种邮箱都有自己独特的功能。大多数 Linux 发行版使用`/var/spool/mail` 或`/var/mail` 目录包含各个邮箱文件，每个文件对应系统上的一个用户账户。这是所有邮箱文件集中的位置，这样一来 MUA 程序就知道到哪里去查找每个人的邮箱文件了。

少数 Linux 发行版允许将邮箱文件移动到每个用户的`$HOME` 目录中。这提供了更高的安全性，因为每个邮箱文件位于已经设置好安全访问权限的区域中。

Maildir 样式的邮箱是受一些更高级的 MTA、MDA 和 MUA 应用程序支持的新型功能。不像每个消息是邮箱文件的一部分，它的邮箱是一个目录，而每个消息是该目录的单独的文件。这可以帮助削减邮箱破坏程度，因为单个文件不会破坏整个邮箱。

虽然 Maildir 样式的邮箱提供了改进的性能、安全性和容错功能，但许多流行的 MDA 和 MUA 程序仍无法使用它。然而，所有 MDA 和 MUA 程序都可以使用`/var/spool/mail` 邮箱文件。

注意：最初在 Unix 中邮箱位置是`/var/spool/mail`。大多数 Linux 发行版都使用此文件命名惯例；但少数 Linux 发行版使用`/var/mail`。

如果系统使用特殊的 MDA 程序处理接收的邮件消息，则很可能是流行的 Procmail 程序。Procmail 允许每个用户创建自定义的配置文件来定义邮件过滤，外出时的自动重定向和单独的邮箱。

### 26.1.4　邮件用户代理

目前为止，我们已经跟随电子邮件的足迹从远程主机到达本地主机独立用户的邮箱。下一步是允许各个用户查看他们的电子邮件消息。

Linux 电子邮件模型为每个用户使用本地的邮箱文件或目录来保存用户的消息。MUA 程序的任务是提供一个让用户与邮箱互动的方法，以读取他们的消息。

需要记住的重点是，MUA 不接收信息；它们只显示已经在邮箱中的消息。许多 MUA 程序也提供创建单独邮件文件夹的功能，以便用户可以将邮件从默认邮箱移动到单独的文件夹中进行管理。

大多数 MUA 程序还提供发送消息的功能。这一部分可能有一些费解，因为我们前面已经介绍说

发送邮件消息是 MTA 程序的任务。

要执行此功能，大多数 MUA 程序都利用 SMTP 中的智能主机功能。要么 MUA 程序自动将消息发送到本地 MTA 程序以进行发送，要么必须在 MUA 配置中定义远程智能主机，让它发送消息以进行分发。

多年以来，Linux 平台上涌现出许多不同的开源 MUA 程序。下面几节将介绍一些您在 Linux 中会遇到的常见的 MUA 程序。

### 1. Mailx

Mailx 程序是 Linux 环境中最受欢迎的命令行 MUA 程序。之所以取名为 Mailx，是因为它是原本为 Unix 开发的邮件程序的改进版。在所有的安装中，Mailx 程序以可执行文件 `mail` 进行安装，而不是单独的程序进行安装，这表示它是邮件程序的替代者。

Mailx 程序允许用户访问他们的邮箱以阅读保存的消息，以及将消息发送给其他邮件用户，一切都在命令行上完成。下面是一个 Mailx 会话示例。

```
$ mail
Mail version 8.1.2 01/15/2001. Type ? for help.
"/var/mail/rich": 2 messages 2 new
>N 1 atest@testbox        Fri Feb 1 17:42   16/664    This is a test
 N 2 atest@testbox        Fri Feb 1 17:43   16/676    This is another
test
& 1
Message 1:
From atest@testbox Fri Feb 1 17:42:56 2008
Date: Fri, 1 Feb 2008 17:42:56 -0500
From: atest@testbox
To: rich@localhost.localdomain
Subject: This is a test

This is a test message.

& d
& q
Held 1 message in /var/mail/rich
$mail Barbara
Subject: This is a test message
This is a test message that I'm sending to Barbara.
.
Cc:
$
```

第一行表示不带命令行选项运行 Mailx 程序。即默认允许用户查看邮箱中的消息。输入 `mail` 命令后，显示用户邮箱中的所有消息的汇总信息。Mailx 程序只能读取`/var/mail` 格式或`$HOME/mail` 格式的消息。它不能处理使用 Maildir 邮件文件夹格式的邮件。

每个用户都拥有包含自己所有消息的单独文件。文件名通常是用户的系统登录名并位于系统的邮箱目录中。因此，用户名为 rich 的所有消息都保存在 Linux 系统的`/var/mail/rich` 文件中。如果用户收到新的消息，则新消息将被追加到文件的结尾。

第二次使用 `mail` 命令演示了从命令行将邮件消息发送给另一个用户。收件人的名称与程序名一起包括在命令行中。Mailx 程序要求输入消息主题，然后允许输入消息文件。要结束消息输入，在一行上输入一个句点即可。作为结束，Mailx 程序询问是否还有其他收件人应该收到消息的副本，然后终止并尝试将消息传递给 MTA 程序以进行发送。

### 2. Mutt

随着 Unix 环境的不断改进，MUA 程序变得更高级。Unix 系统上的第一个图形尝试是 ncruses 图形库。使用 ncruses 的程序可以处理光标在终端屏幕上的位置并将字符放置在终端上的几乎任何位置。

一个利用 ncruses 库的 MUA 程序就是 Mutt 程序。启动 Mutt 时，它将在终端显示上绘制用户友好的

菜单，列出与 Mailx 程序的输出类似的消息。可以选择一则消息并在显示中进行查看，如图 26-3 所示。

Mutt 程序使用组合键执行标准函数，如读取消息和撰写新消息。对于 shell 脚本程序员来说，最有用的功能可能就是直接从命令行发送消息，而不必借助文本图形模式。我们将在后面的“Mutt 程序”一节详细讨论这些内容。

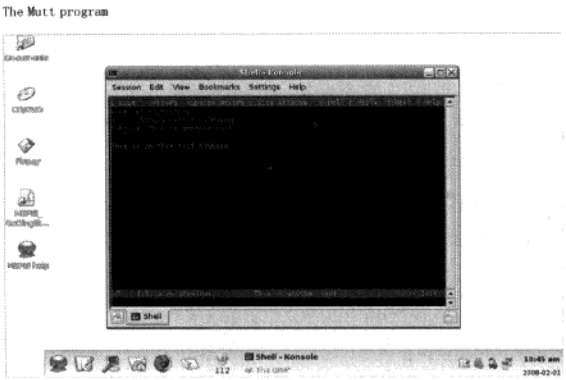



图 26-3　Mutt 程序

### 3. 图形电子邮件客户端

几乎所有 Linux 系统都支持图形 X Window 环境。有许多电子邮件 MUA 程序都使用 X Window 系统显示消息信息。最流行的两个图形 MUA 程序是：

- KDE 窗口环境的 KMail；
- GNOME 窗口环境的 Evolution。

两种软件包都允许与本地 Linux 邮箱交互，以及连接到远程邮件服务器读取邮件消息。图 26-4 展示了一个 KMail 会话界面的示例。

要与远程服务器连接，KMail 和 Evolution 都支持邮局协议（POP）和更高级的 Internet 消息访问协议（IMAP）。尽管对于桌面 Linux 来说，KMail 和 Evolution MUA 程序很好，但在 shell 脚本中它们却不是很有用。

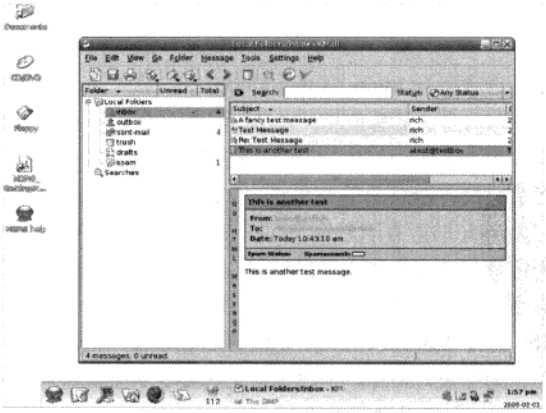


图 26-4　KMail MUA 程序主界面

## 26.2　设置服务器

在将自动的邮件消息发送到全域之前，需要确保 Linux 系统上具备 MTA 包以及其配置正确。这并

不是简单的任务，但幸运的是，一些 Linux 发行版提供了一些基本工具来帮助解决此问题。

本节将介绍在 Linux 中最常用的两种流行电子邮件 MTA 程序（sendmail 和 Postfix）的基础知识。虽然已经有完整的书籍撰写了如何正确配置这两种包，但我们将只介绍基本的内容，即如何将电子邮件从 Linux 系统发送出去并分发到用户的邮箱中。

## 26.2.1 sendmail

sendmail MTA 包是 Internet 邮件服务器使用的最为流行的开源 MTA 包之一。过去，它经常受到后门和安全漏洞等传言的困扰，然而，它已经被重新编写，不仅去除了安全漏洞，更融合了许多新型的 MTA 功能，如垃圾邮件控制。sendmail 程序的新版本已经证明是安全的且功能强大。

### 1. sendmail 程序的部件

主执行程序称为 `sendmail`。它通常以后台模式运行，侦听 SMTP 从远程邮件服务器的连接，以及为本地用户转发消息。

除了主 `sendmail` 程序以外，有一个配置文件和几个用于包含在处理接收和送出的邮件消息时所使用信息的表。表 26-1 列出了常规 sendmail 安装使用的部件。

表 26-1 sendmail 配置文件

| 文件 | 描述 |
|---|---|
| `sendmail.cf` | 控制 sendmail 程序行为的文本文件 |
| `sendmail.cw` | 包含 sendmail 程序为其接收信息的域名称列表的文本文件 |
| `sendmail.ct` | 包含可以控制 sendmail 操作的信任用户列表的文本文件 |
| `aliases` | 二进制文件，包含可以将邮件重定向到另一用户、文件或程序的有效本地邮件地址列表 |
| `newaliases` | 从文本文件创建新的别名数据库的可执行程序 |
| `mailq` | 检查邮件队列和打印任何消息的可执行程序 |
| `mqueue` | 用于保存待发送消息的目录 |
| `mailertable` | 用于指定特定域的路由路径的文本文件 |
| `domaintable` | 用于将旧域名映射到新域的文本文件 |
| `virtuesrtable` | 用于将用户和域映射到其他地址的文本文件 |
| `relay-domains` | 用于列出允许通过 sendmail 程序回复邮件的特定主机列表的文本文件 |
| `access` | 列出特定域的文本文件，从这些域收到的消息将被通过或拒绝 |

除非是为公司或 Internet 服务提供商运行邮件服务器，否则不需要担心 `sendmail.cf` 配置文件。事实上，许多使用 sendmail 的 Linux 发行版能自动为用户创建和配置核心 `sendmail.cf` 配置文件，此默认配置可在大多数简单应用中正常工作。

### 2. sendmail.cf 文件

需要告诉 sendmail 程序在服务器收到消息时应该如何处理这些消息。作为 MTA，sendmail 处理接收的消息并将其重定向到远程系统或本地系统上的另一邮件包。配置文件用于指导 sendmail 如何处理目标邮件地址，以便决定将消息发送到哪里以及如何发送。配置文件的默认位置是 `/etc/mail/sendmail.cf`。

`sendmail.cf` 文件由一些规则集构成，这些规则解析收到的邮件消息并决定执行何种操作。每个规则集对应于特定的邮件格式并指导 sendmail 如何处理该消息。

sendmail 程序收到消息时，将解析消息的头部并将消息传递给各个规则集以决定在该消息上执行什么操作。sendmail 配置文件包含一些规则，这些规则允许 sendmail 以多种不同格式处理邮件。从 SMTP 主机收到的邮件具有与从本地用户收到的邮件不同的头部字段。sendmail 程序必须知道如何处理每种格式的邮件。

配置文件中也定义了规则的一些帮助函数。可以定义三种类型的帮助函数：

- `Classes` 定义用于帮助规则集识别特定消息类型的普遍语句；

- Macros 是一些值，设置这些值以简化在配置文件中键入长字符串；
- 定义 Options 以设置 sendmail 程序的操作参数。

配置文件由一系列类、宏、选项和规则集组成。每个函数都定义为配置文件中的单个文本行。

配置文件的每个行都以单个字符开始，该字符定义该行的操作。以空格或制表符开头的行表示继续上一个操作行。以英镑符号(#)开头的行表示注释且不会被 sendmail 处理。

文本行开头的操作定义了该行的用途。表 26-2 展示了标准的 sendmail 操作以及它们代表的意义。

**表 26-2　　sendmail 配置文件行**

| 配置行 | 描述 | 配置行 | 描述 |
|---|---|---|---|
| C | 定义文本的类 | M | 定义邮件程序 |
| D | 定义宏 | O | 定义 sendmail 选项 |
| F | 定义包含文本类的文件 | P | 定义 sendmail 优先值 |
| H | 定义头部字段和操作 | R | 定义规则集以解析地址 |
| K | 定义包含要搜索的文本的数据库 | S | 定义规则集组 |

如前所述，不需要重新编写 sendmail.cf 配置文件，Linux 发行版应该已经为您创建了标准的模板。图 26-5 展示了来自基于 Debian 的 Linux 系统的 sendmail.cf 配置文件的一部分。

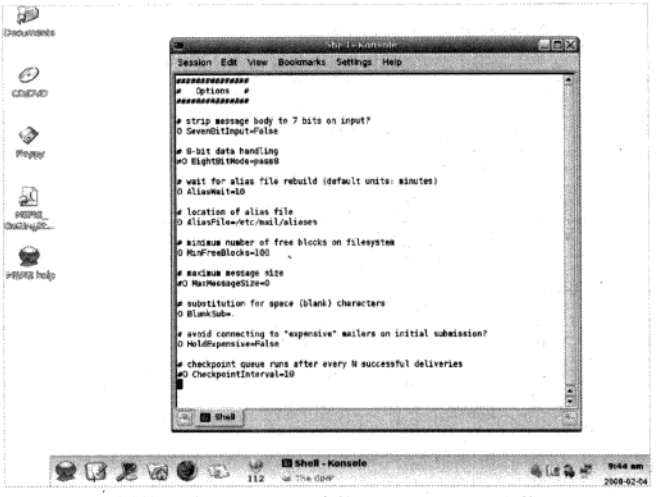



图 26-5　Debian Linux 中的 sendmail.cf 配置文件

通常，唯一需要关注的是是否需要使用智能主机来转发邮件。DS 配置行控制此功能：

```
DSmyisp.com
```

只需将智能主机的主机名添加到 DS 标记后面即可。

## 26.2.2　Postfix

Postfix 软件包迅速成为 Unix 和 Linux 系统上最受欢迎的电子邮件包之一。Postfix 由 Wietse Venema 开发，旨在为标准 Unix 类型的服务器提供一种 MTA 选择。Postfix 软件能够将任何 Unix 或 Linux 系统转变为功能完全的电子邮件服务器。

MTA 包的任务是管理来到或离开邮件服务器的消息。Postfix 通过使用几个不同的模块程序和一个邮件队列目录系统来完成此任务。每个程序处理通过各种消息队列的消息，直至消息被发送到最终的目标地址。如果某个时刻邮件服务器在传输消息的过程中崩溃，则 Postfix 可以判断出消息上一次被成功放置的是哪个队列并尝试继续消息处理。

### 1. Postfix 系统的部件

Postfix 系统由若干个邮件队列目录和可执行程序构成，它们互相交互以提供邮件服务。图 26-6 展示了核心 Postfix 部件的结构图。

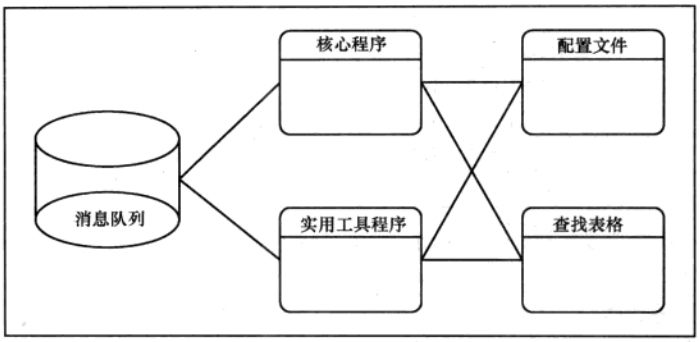



图 26-6 Postfix 的结构图

Postfix 结构图的每一部分提供整个电子邮件过程的不同功能。下面几节将详细介绍 Postfix 结构图的这些不同部分。

Postfix 包使用始终运行在后台进程的 `master` 程序。主程序允许 Postfix 产生程序以描述新消息的邮件队列并将它们发送到适当的地址。在装备核心程序后，可以将其配置为在设置的时间运行。这允许主程序在需要时重新利用运行的帮助程序，节省了处理时间。到设置的时间限制后，帮助程序以静默方式停止。

主程序用于控制 Postfix 的全部操作。它负责开启需要开启的其他 Postfix 进程。qmgr 和 pickup 程序被配置为比其他核心程序停驻时间更长的后台进程。pickup 程序决定消息何时可供 Postfix 系统发送。qmgr 程序负责 Postfix 的中心消息路由系统。

表 26-3 展示了 Postfix 用来传输邮件消息的其他核心程序。

表 26-3 Postfix 核心程序

| 程序 | 描述 |
|---|---|
| bounce | 将日志发送到退还消息的 bounce 消息队列并将返回的消息退还给发送者 |
| cleanup | 处理接收邮件的头部并将消息放到接收队列 |
| error | 处理来自 qmgr 的消息分发请求，强制消息退还 |
| flush | 处理等待被远程邮件服务器检索的消息 |
| local | 分发到本地用户的消息 |
| pickup | 等待 maildrop 队列中的消息并将它们发送到 cleanup 程序以开始处理 |
| pipe | 将来自队列管理器程序的消息转发给外部程序 |
| postdrop | 当队列不能被普通用户写入时，将接收消息移动到 maildrop 队列 |
| qmgr | 处理接收队列中的消息，并决定何处以及如何分发它们，并产生程序去分发它们 |
| sendmail | 为程序提供 sendmail 可兼容接口以将消息发送到 maildrop 队列 |
| showq | 报告 Postfix 邮件队列状态 |
| smtp | 使用 SMTP 协议将消息转发给外部邮件主机 |
| smtpd | 使用 SMTP 协议从外部邮件主机接收信息 |
| trivial-rewrite | 从 cleanup 程序接收信息以确保头部地址符合 qmgr 程序的标准格式，然后使用 qmgr 程序解析远程主机地址 |

与其他 MTA 包不同，Postfix 使用若干个不同的消息队列来管理处理的电子邮件消息。每个消息

队列包含 Postfix 系统中不同消息状态的消息。表 26-4 列出了 Postfix 使用的消息队列。

**表 26-4　　Postfix 消息队列**

| 队列 | 描述 |
|---|---|
| maildrop | 等待处理的新消息，收取自本地用户 |
| incoming | 等待处理的新消息，收取自远程主机，还有已处理的来自本地用户的消息 |
| active | 已经准备好由 Postfix 分发的消息 |
| deferred | 初始分发尝试失败并等待再次尝试分发的消息 |
| flush | 准备发送给远程主机的消息，远程主机将连接到邮件服务器以检索这些消息 |
| mail | 分发为本地用户保存的消息以供其阅读 |

如果 Postfix 系统应该在某个时间关闭，则消息将停留在最后一个它们应该被放置的队列中。当 Postfix 重新启动时，将自动开始处理队列中的消息。

**2. Postfix 配置文件**

结构图中的下一部分是 Postfix 配置文件。配置文件包含 Postfix 程序处理消息时所使用的信息。与其他一些 MTA 程序不同，可以在 Postfix 服务器运行时更改配置信息并发出命令让 Postfix 装载新信息，而不必完全停止邮件服务器。

有 3 个 Postfix 配置文件，它们位于常规 Postfix 目录中。通常此目录的默认位置是 /etc/postfix。一般情况下，所有用户都有权查看配置文件，但只有 root 用户有权更改文件中的值。当前，您可以根据自己的安全性情况进行更改。表 26-5 列出了 Postfix 配置文件。

**表 26-5　　Postfix 配置文件**

| 文件 | 描述 |
|---|---|
| install.cf | 包含安装 Postfix 时使用的安装参数信息 |
| main.cf | 包含处理信息时 Postfix 使用的参数 |
| master.cf | 包含运行核心程序时 Postfix master 程序使用的参数 |

install.cf 配置文件允许检索在 Postfix 软件第一次被安装到系统上时所使用的安装参数。这为确定在软件安装时哪些功能可用提供了一种简单方法。

master.cf 配置文件控制核心 Postfix 程序的行为。每个程序及其控制行为的参数都列在一个单独的行上。下面是 master.cf 文件默认设置的示例：

```
# ==========================================================================
#service type private unpriv chroot wakeup maxproc command + args
#              (yes)   (yes)  (yes)   (never)  (50)
# ==========================================================================
smtp      inet  n   -   n   -   -   smtpd
pickup    fifo  n   -   n   60  1   pickup
cleanup   unix  -   -   n   -   0   cleanup
qmgr      fifo  n   -   n   300 1   qmgr
rewrite   unix  -   -   n   -   -   trivial-rewrite
bounce    unix  -   -   n   -   0   bounce
defer     unix  -   -   n   -   0   bounce
trac      unix  -   -   n   -   0   bounce
verify    unix  -   -   n   -   1   verify
flush     unix  n   -   n   1000 0  flush
proxymap  unix  -   -   n   -   -   proxymap
smtp      unix  -   -   n   -   -   smtp
relay     unix  -   -   n   -   -   smtp-ofallback_relay=
showq     unix  n   -   n   -   -   showq
error     unix  -   -   n   -   -   error
local     unix  -   n   n   -   -   local
virtual   unix  -   n   n   -   -   virtual
```

```
lmtp      unix  -  -  n  -  -  lmtp
anvil     unix  -  -  n  -  1  anvil
scache    unix  -  -  n  -  1  scache
```

master.cf 配置文件还包括指示 Postfix 如何与外部 MDA 软件（如 Procmail）进行连接的行。

Postfix 的操作性参数设置在 main.cf 文件中。所有 Postfix 的操作性参数都具有已经应用到 Postfix 系统的默认值。如果参数值存在于 main.cf 文件，则其内容将覆盖默认值。

每个 Postfix 参数及其值都列在配置文件的一个单独的行上，如下所示：

```
parameter = value
```

parameter 和 value 都是纯文本字符串，在需要时可以轻松地读取和更改。Postfix master 程序在 Postfix 第一次启动时和发出 reload 命令时读取 main.cf 文件中的参数值。

myhostname 和 mydomain 参数是 Postfix 参数的两个例子。如果在 main.cf 配置文件中没有指定它们，则 myhostname 参数呈现 Linux 系统上的 gethostname() 命令，而 mydomain 呈现默认 myhostname 参数的域部分。通常，一个邮件服务器处理整个域的邮件。下面是 Postfix 配置文件中的一个简单设置：

```
myhostname = mailserver.smallorg.org
mydomain = smallorg.org
```

Postfix 启动时，它将本地邮件服务器识别为 mailserver.smallorg.org，将本地域识别为 smallorg.org 并忽略任何系统设置值。

如果需要指定智能主机，则可以使用 relayhost 参数：

```
relayhost=myisp.com
```

还可以指定 IP 地址，但必须使用方括号将其括起来。

## 26.3 使用 Mailx 发送消息

从 shell 脚本发送电子邮件的主要工具是 Mailx 程序。不仅可以用它以交互方式读取和发送消息，还可以使用命令行参数指定发送消息的方式。

Mailx 程序发送消息的命令行格式为：

```
mail [-eIinv] [-a header] [-b addr] [-c addr] [-s subj] to-addr
```

mail 命令使用表 26-6 所示的命令行参数。

表 26-6 Mailx 命令行参数

| 参数 | 描述 | 参数 | 描述 |
|---|---|---|---|
| -a | 指定其他 SMTP 头部行 | -I | 强制 Mailx 以交互模式运行 |
| -b | 将 BCC：收件人添加到消息中 | -n | 不读取/etc/mail.rc 启动文件 |
| -c | 将 CC：收件人添加到消息中 | -s | 指定主题行 |
| -e | 如果为空，则不发送消息 | -v | 在终端上显示分发详细信息 |
| -i | 忽略 TTY 中断信号 | | |

如表 26-6 所示，可以从命令行参数创建整个电子邮件消息。所需添加的只是消息主体。

要完成此操作，需要将文本重定向到 mail 命令。下面的示例将展示如何直接从命令行创建和发送电子邮件消息：

```
$ echo "This is a test message" | mail -s "Test message" rich
$ mail
Mail version 8.1.2 01/15/2001. Type ? for help.
"/var/mail/rich": 1 message 1 new
>N 1 rich@testbox        Fri Feb  1 19:12   16/664   Test message
&
Message 1:
From rich@testbox Fri Feb 1 14:12:03 2008
```

```
Date: Fri, 1 Feb 2008 19:12:02 -0500
From: rich <rich@testbox>
To: rich@localhost.localdomain
Subject: Test message

This is a test message

&
```

Mailx 程序将来自 echo 命令的文本作为消息主体发送。这为从 shell 脚本发送消息提供了一种简单方法。下面是一个简单示例：

```
$ cat factmail
#!/bin/bash
# mailing the answer to a factorial

MAIL='which mail'

factorial=1
counter=1

read -p "Enter the number: " value
while [ $counter -le $value ]
do
   factorial=$[$factorial * $counter]
   counter=$[$counter + 1]
done

echo The factorial of $value is $factorial | mail -s "Factorial
answer" $USER
echo "The result has been mailed to you."
```

脚本做的第一件事不是假设 Mailx 程序位于标准位置，而是使用 which 命令确定邮件程序在何处。

经过计算阶乘函数的结果，shell 脚本使用 mail 命令将消息发送给用户定义的$USER 环境变量，这应该是执行脚本的人。

```
$ ./factmail
Enter the number: 5
The result has been mailed to you.
$
```

所需做的只是查看邮件，查看结果是否已经到达：

```
$ mail
Mail version 8.1.2 01/15/2001. Type ? for help.
"/var/mail/rich": 1 message 1 new
>N 1 rich@testbox     Fri Feb 1 19:24     16/671     Factorial answer
&
Message 1:
From rich@testbox Fri Feb 1 14:24:33 2008
Date: Fri, 1 Feb 2008 19:24:33 -0500
From: rich <rich@testbox>
To: rich@localhost.localdomain
Subject: Factorial answer

The factorial of 5 is 120

&
```

通常在消息主体中只发送一行文本，而是需要将整个输出作为电子邮件进行发送。在这种情况下，可以将文本重定向到临时文件，然后使用 cat 命令并将输出重定向到邮件程序。

下面是在电子邮件消息中发送大量数据的示例：

```
$ cat diskmail
#!/bin/bash
# sending the current disk statistics in an e-mail message
```

```
date='date +%m/%d/%Y'
MAIL='which mail'
TEMP='mktemp tmp.XXXXXX'

df -k > $TEMP
cat $TEMP | $MAIL -s "Disk stats for $date" $1
rm -f $TEMP
$ ./diskmail rich
$ mail
Mail version 8.1.1 6/6/93.  Type ? for help.
"/var/spool/mail/rich": 1 message 1 new
>N 1 rich@test2.dfas.mil    Mon Feb   3 14:15   15/594   "Disk stats
for 02/03/"
&
Message 1:
From rich Mon Feb 3 14:15:57 2008
Delivered-To: rich@test2.dfas.mil
To: rich@test2.dfas.mil
Subject: Disk stats for 02/03/2008
Date: Mon, 3 Feb 2008 14:15:57 -0500 (EST)
From: rich@test2.dfas.mil (Rich)

Filesystem          1k-blocks       Used Available Use% Mounted on
/dev/hda1             3526172    1464476   1882576  44% /
/dev/hda6            16002168    6570168   8619116  43% /home

&
```

diskmail 程序使用 date 命令获取当前日期（以某些特殊的格式），查找 Mailx 程序所处的位置，然后创建临时文件。所有这些都完成之后，使用 df 命令显示当前磁盘空间统计信息（参阅第 4 章），将输出重定向到临时文件。

然后，将临时文件重定向到 mail 命令，将第一个命令行参数用作目标地址，然后是标题头部的当前日期。

## 26.4 Mutt 程序

Mutt 程序是 Linux 命令行又一流行的电子邮件客户软件包，在 1995 年由 Michael Elkins 开发。它具有 Mailx 程序所不具备的一种功能，这使得它在 shell 脚本中使用起来非常便利。

Mutt 程序可以将文件发送为电子邮件消息的附件。它不是像 Mailx 那样将很长的文本文件表现为电子邮件消息的主体，而是将文本文件作为主消息体的单独附件。此功能非常有利于发送长文件，如日志文件。

本节将介绍在 Linux 系统上安装 Mutt，并使用它在 shell 脚本中为电子邮件消息附加文件。

### 26.4.1 安装 Mutt

在如今高级的图形电子邮件客户端（如 KMail 和 Evolution）时代，Mutt 程序已经不是一个流行的软件包了，所以您的 Linux 发行版很有可能没有默认安装。然而，大多数 Linux 发行版将其包括在了常规发行版文件中，以便使用标准软件安装方法进行安装。

如果您的 Linux 发行版未包括 Mutt 软件包，或者希望下载最新的版本，则可以执行下面的步骤：

（1）转到 Mutt 软件包 Web 站点，地址为 www.mutt.org。从该站点选择下载链接以转到下载页面。

（2）在下载页面上，具有特定于 Linux 发行版的已编译的二进制可执行包，还有源代码发行包。如果 Linux 发行版具有已编译的二进制可执行包，则可以下载并按照下面适当的步骤将软件安装到

Linux 发行版上。否则，请下载当前源代码发版（有两个软件包：一个稳定版本和一个开发版本）。

（3）下载源代码软件包后，使用 tar 命令解压缩到工作目录：`tar -zxvf mutt-1.4.2.3.tar.gz`

（4）转到新创建的目录：`cd mutt-1.4.2.3`

（5）运行配置脚本以构建在系统上编译所需要的文件：`./configure`

（6）作为 root 用户，运行带 install 选项的 make 实用工具以创建和安装 Mutt：`make install`

这将在 Linux 系统上安装 Mutt 程序以便在命令行上使用。

## 26.4.2　Mutt 命令行

mutt 命令提供了用于控制 Mutt 操作的参数。表 26-7 展示了可用的命令行参数。

表 26-7　　Mutt 命令行参数

| 参数 | 描述 |
|---|---|
| -A *alias* | 将指定别名的扩展版本传递给 STDOUT |
| -a *file* | 使用 MIME 协议将指定的文件附加到消息中 |
| -b *address* | 指定秘送（BCC）收件人 |
| -c *address* | 指定抄送（CC）收件人 |
| -D | 打印所有配置选项的值到 STDOUT |
| -e *command* | 指定处理初始化文件之后要运行的配置命令 |
| -f *mailbox* | 指定要装载的邮箱文件 |
| -F *muttrc* | 指定除$HOME/.muttrc 以外的要读取的初始化文件 |
| -h | 显示帮助文本 |
| -H *draft* | 指定包含头部和主体的草稿文件以用于发送消息 |
| -i *include* | 指定要包括在消息主体中的文件 |
| -m *type* | 指定默认邮箱类型 |
| -n | 忽略系统配置文件 |
| -p | 恢复暂停的消息 |
| -Q *query* | 查询配置变量值。在解析所有配置文件和任何命令行上的命令被执行后，执行查询 |
| -R | 以只读模式打开邮箱 |
| -s *subject* | 指定消息的主题 |
| -v | 显示 Mutt 版本号和编译时间定义 |
| -x | 模拟 Mailx 撰写模式 |
| -y | 以列出 mailboxes 命令指定的所有邮箱开始 |
| -z | 当与 f 一起使用时，如果邮箱中没有消息则不启动 |
| -Z | 打开 mailboxes 命令指定的包含新邮件的第一个邮箱 |

通过无数的命令行参数可以直接从命令行自定义电子邮件消息，这正是我们想要在 shell 脚本中做的。

与 Mailx 程序非常类似，有一个不能使用 Mutt 程序在命令行上指定的东西，即消息主体文本。如果未将文本重定向到 Mutt 程序，则它将以文本图形模式启动，同时显示编辑器窗口以供输入消息主体。

这不利于 shell 脚本，所以需要总是将一些类型的文本重定向到消息主体，甚至在使用附件选项指定要附加的选项时也是如此。下一节将描述如何完成此操作。

## 26.4.3　使用 Mutt

现在，我们已经准备好在 shell 脚本中使用 Mutt 程序了。要在 shell 脚本中创建基本的 Mutt 命令，则可能希望包括指定消息主题、附加文件和所有消息收件人的命令行选项：

```
mutt -s Subject -a file recipients
```

recipients（收件人）列表是一系列要将消息发送到的电子邮件地址（以空格分隔）。如果想附加多个文件，则需要使用多个-a 选项，因为一个选项只能声明一个文件名。file 参数必须是绝对路径名或者与运行 mutt 命令的相对于当前工作目录的相对路径名。

使用 mutt 命令时还要注意一点，如果不将文本重定向到消息主体，则 Mutt 将自动转到全屏模式以供在编辑器窗口输入文本。大多数情况下这不是我们想要的，所以请确保将一些文本用作消息主体，甚至用空文件也无妨：

```
# echo "Here's the log file" | mutt -s "Log file" -a
/var/log/messages rich
```

此命令将系统日志文件作为附件发送到本地系统上的电子邮件地址 rich。请注意，必须具有要附加文件的适当访问权限。图 26-7 展示了在 KMail 邮件客户端收到电子邮件消息的情况。

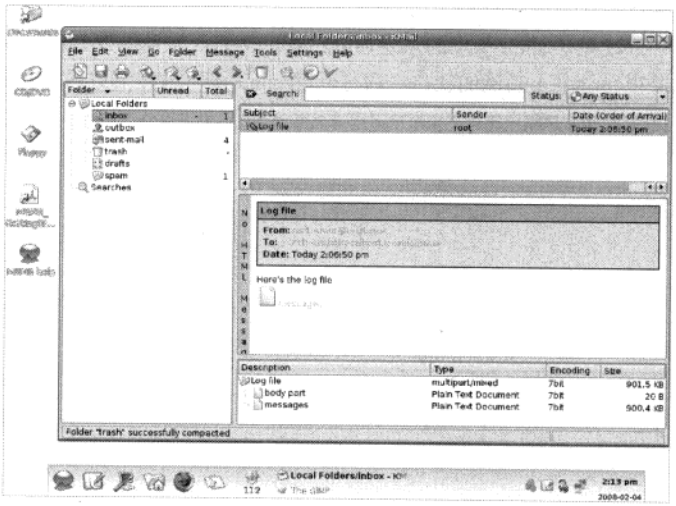



图 26-7 使用 KMail 查看带附件的消息

注意，消息包括 echo 语句的主体文本以及附加文件的单独图标。可以直接从 KMail 客户端保存附加文件。

警告：如果查看附加文件的名称，则会发现 Mutt 使用附加文件的基名称作为附件的文件名。请小心使用临时文件，因为 Mutt 将使用临时文件名作为附件文件名。最好在保存临时文件时使用更具描述性的名称，而不是使用临时文件名。

## 26.5 小结

本章讨论了在 shell 脚本中如何使用电子邮件。有规律地向客户发送电子邮件报告是 shell 脚本可以提供的一项非常棒的功能。

在使用命令行发送电子邮件消息之前，需要知道电子邮件在 Linux 环境中的工作方式，以及需要安装和配置什么样的应用程序。Linux 电子邮件环境由 3 个元素构成：邮件传输代理（MTA）程序、邮件分发代理（MDA）程序和邮件用户代理（MUA）程序。

MTA 程序负责为 Linux 系统发送和接收邮件消息。它必须知道如何将接收到的邮件消息传递给适当的用户邮箱，还要知道如何将邮件消息发送给远程邮件服务器上的用户。通常，MTA 程序使用代理服务器（也称为智能主机）执行具体的邮件分发。它将目标地址为远程邮件服务器用户的任何消息转发给智能主机进行分发。它依靠智能主机自行执行分发。

MDA 程序负责确保发送给本地用户的邮件可以准确到达本地邮箱。有时此功能直接由 MTA 程序执行。然而，如果需要高级邮件分发功能，如外出时的通知或垃圾邮件过滤，则可以配置 MTA 程序以将消息传递到 MDA 程序，后者通常内置了这些功能。

MUA 程序允许各个系统用户访问自己邮箱中的消息并将消息传递给其他用户让 MTA 执行分发。这些程序可以从简单的命令行程序（如 Mailx 和 Mutt）到高级的图形程序（如 KMail 和 Evolution）。

从 shell 脚本发送电子邮件的最简单的方法是使用 Mailx 程序。此程序允许在命令行上指定主题头部和一个或多个收件人。可以通过将文本重定向到 Mailx 程序来创建消息主体。可以使用用于单个文本行的 `echo` 命令，也可以使用将文件内容重定向到邮件消息的 `cat` 命令来完成此操作。

Mutt 程序是一种更高级的命令行 MUA 程序，它提供将文件附加到邮件消息的功能，而不是将文本包括到消息主体中。这允许用户附加大的文本文件以便客户可以轻松保存到磁盘再使用其他程序（如电子表格和文字处理软件包）仔细查看。

本部分的最后一章涵盖了 shell 脚本的重要内容：管理功能。如果作为 Linux 系统管理员，则很有可能需要有规律地监视系统某一功能的状态。通过创建 shell 脚本并将其放入 cron 作业，可以轻松地监视 Linux 系统上正在发生的事情。

# 第 27 章
# 管理员使用的 shell 脚本

27 CHAPTER

本章内容：

- 管理统计信息；
- 执行备份；
- 管理用户。

对于 Linux 系统管理员来说，没有比 shell 脚本编程更有用的了。通常，Linux 系统管理员每天需要完成无数项任务，从监视系统磁盘空间和系统用户到备份重要文件。shell 脚本可以让系统管理员的工作变得非常轻松！使用简单的 shell 脚本就可以自动执行所有基本的系统管理任务。本章将介绍一些可以使用 shell 脚本完成的任务。

## 27.1 监视系统统计信息

Linux 系统管理员的核心任务之一就是确保系统正常运行。要完成这一任务，必须监视多种不同系统的统计信息。创建 shell 脚本来监视特定情况是必不可少的。

本节将介绍如何创建简单的 shell 脚本来进行监视和立即报告问题，这甚至不需要登录到 Linux 系统。

### 27.1.1 监视磁盘空闲空间

多用户 Linux 系统的最大问题之一就是可用磁盘空间的量。在某些情况下，如在文件共享服务器上，仅由于一位用户粗心的操作，磁盘空间就可能被立即填满。

此 shell 脚本将监视特定卷上的可用磁盘空间，并在可用磁盘空间低于设置的阈值时发送一则电子邮件消息。

**1. 必需函数**

要自动监视可用的磁盘空间，首先需要使用可以显示这些值的命令。最佳的监视磁盘空间命令是 df 命令（参阅第 4 章）。df 命令的基本输出如下所示：

```
$ df
Filesystem  1K-blocks     Used Available Use% Mounted on
/dev/hda1      3197228  2453980   580836  81% /
varrun          127544      100   127444   1% /var/run
varlock         127544        4   127540   1% /var/lock
udev            127544       44   127500   1% /dev
devshm          127544        0   127544   0% /dev/shm
/dev/hda3       801636   139588   621328  19% /home
$
```

df 命令显示系统上所有真实磁盘和虚拟磁盘的当前磁盘空间的统计信息。对于这个练习，我们将监视根文件系统的大小。我们首先需要解析 df 命令的结果，以便只提取出根文件系统所在的行。

有多种方法可以完成此操作。最简单的办法是使用 sed 命令搜索以正斜杠结尾的行：

```
/dev/hda1          3197228       2453980       580836        81% /
```

sed 脚本的构建方法是使用美元符号（表示行的结尾）和正斜杠（需要进行转义，因为它是 sed 的特殊字符）。具体如下所示：

```
$ df | sed -n '/\/$/p'
/dev/hda1          3197228       2453980       580836        81% /
$
```

现在，我们得到了根文件系统卷的统计信息，下一步是分离出此行上的百分比值。要完成此操作，需要使用 gawk 命令：

```
$ df | sed -n '/\/$/p' | gawk '{print $5}'
81%
$
```

这已经很接近了，但还是有一个小问题。使用 gawk 命令可以过滤出第 5 个数据字段，但这个值还是包含百分号。必须删除百分号才能够在一个数学等式中使用此值。使用 sed 命令可以轻松删除百分号：

```
$ df | sed -n '/\/$/p' | gawk '{print $5}' | sed 's/%//'
81
$
```

成功了！下一步是使用此信息创建脚本。

**2. 创建脚本**

现在已经了解了如何提取已使用的磁盘空间值，可以使用此公式将值保存到变量中。然后，可以将此值与一个预定值比较，就可以知道使用的空间是否超过了预定的极限。

下面是使用此技术的代码示例：

```
$ cat diskmon
#!/bin/bash
# monitor available disk space

SPACE='df | sed -n '/\/$/p' | gawk '{print $5}' | sed 's/%//'

if [ $SPACE -ge 90 ]
then
   echo "Disk space on root at $SPACE% used" | mail -s "Disk warning"
rich
fi
$
```

一个简单的 shell 脚本将检查根文件系统的磁盘空间，并在使用的空间超过 90%时发送一则电子邮件消息。

**3. 运行脚本**

让 diskmon 脚本自动运行之前，需要手动测试几次以确保它能够正确执行所需的操作。要测试它，可以将检查值更改为比当前磁盘使用百分比低的一个值：

```
if [ $SPACE -ge 40 ]
When you run the script, it should send you a mail message:
$ ./diskmon
$ mail
Mail version 8.1.2 01/15/2001. Type ? for help.
"/var/mail/rich": 1 message 1 new
>N 1 rich@testbox       Tue Feb 5 06:22   16/672   Disk warning
&
Message 1:
From rich@testbox Tue Feb 5 06:22:26 2008
Date: Tue, 5 Feb 2008 06:22:26 -0500
From: rich <rich@testbox>
To: rich@localhost.localdomain
```

```
Subject: Disk warning

Disk space on root at 81% used

&q
$
```

好了！现在可以设置 shell 脚本来设置它的执行次数以监视磁盘活动。可以通过时间表（参阅第 13 章）完成此操作。

运行此脚本的频率取决于文件服务器的活跃程度。对于空间较小的文件服务器，只需要一天运行一次：

```
30 0 * * * /home/rich/diskmon
```

此 cron 表项目在每天上午 12：30 运行 shell 脚本。对于大型文件服务器环境，可能需要一天监视几次：

```
30 0,8,12,16 * * * /home/rich/diskmon
```

此 cron 表项目每天运行 4 次，分别是凌晨 12：30、上午 8：30、下午 12：30 和下午 4：30。

## 27.1.2 谁在霸占磁盘资源

如果您负责的 Linux 服务器上有许多用户，则经常需要解决的一个问题就是谁在使用所有磁盘空间。这个老掉牙的问题有时比其他问题更难以弄清。

不幸的是，虽然跟踪用户磁盘空间使用情况非常重要，但却没有一个 Linux 命令可以提供此信息。因此，需要编写 shell 脚本来将其他命令拼凑起来以提取需要查找的信息。本节将演示这个过程。

**1. 必需函数**

第一个需要使用的工具是 du 命令（参阅第 4 章）。此命令显示各个文件和目录的磁盘使用情况。使用-s 选项可显示目录级别的概况。这在计算单个用户的磁盘使用情况时非常便利。对/home 目录内容使用此命令可总结出每个用户的$HOME 目录：

```
# du -s  /home/*
40       /home/barbara
9868     /home/jessica
40       /home/katie
40       /home/lost+found
107340   /home/rich
5124     /home/test
#
```

好了，这只是个开始。现在，可以查看$HOME 目录总计的列表了（以 KB 为单位）。根据挂载/home 目录的方式，也许能看到一个称为 lost+found 的特殊目录，这不是一个用户账户。

注意：要获取所有用户$HOME 目录的统计信息，在运行脚本时必须作为根用户登录。

要去掉这个目录，可以使用带-v 选项的 grep 命令，将打印除了包含指定文本行以外的所有行：

```
# du -s  /home/* | grep -v lost
40       /home/barbara
9868     /home/jessica
40       /home/katie
107340   /home/rich
5124     /home/test
#
```

接下来，去掉所有的路径名，以便查看用户名。这似乎是 sed 命令的工作：

```
# du -s  /home/* | grep -v lost | sed 's/\/home\//'
40       barbara
9868     jessica
40       katie
107340   rich
5124     test
#
```

好多了。现在，让我们对此输出按降序排序：

```
# du -s   /home/* | grep -v lost | sed 's/\/home\///' | sort -g -r
107340    rich
9868      jessica
5124      test
40        katie
40        barbara
#
```

sort 命令在使用-g 选项时将所有数字值排序，并在包括-r 选项时按降序排序。

我们感兴趣只有一条信息，即所有用户使用的总磁盘量。下面是获取此值的方法：

```
# du -s /home
122420 /home
#
```

以上就是创建磁盘使用信息报告需要的所有信息。现在，我们已经准备好将它放到 shell 脚本中。

**2. 创建脚本**

现在，我们知道了如何提取用于报告的原数据，接下来需要创建脚本来读取报告，解析原数据值，最后以可以呈现给用户的格式显示出来。

处理用于报告的数据的最简单方法是使用 gawk 命令。报告由 3 部分构成。

- 头部：用文字标识出各个列。
- 报告主体：显示用户、用户使用的总磁盘空间和用户消耗占总量的百分比。
- 尾部：显示所有用户的总磁盘使用情况。

gawk 命令可以将所有这些函数作为单个命令执行，只需使用 BEGIN 和 END 标记（参阅第 16 章）。下面是将所有元素综合在一起的磁盘使用情况的跟踪信息脚本：

```
# cat diskhogs
#!/bin/bash
# calculate disk usage and report per user

TEMP='mktemp -t tmp.XXXXXX'
du -s /home/* | grep -v lost | sed 's/\/home\///' | sort -g -r >
$TEMP
TOTAL='du -s /home | gawk '{print $1}''
cat $TEMP | gawk -v n="$TOTAL" '
BEGIN {
        print "Total Disk Usage by User";
        print "User\tSpace\tPercent"
}

{
      printf "%s\t%d\t%6.2f%\n", $2, $1, ($1/n)*100
}

END {
      print "-------------------------";
      printf "Total\t%d\n", n
}'
rm -f $TEMP
#
```

脚本将用于产生原数据的公式得出的结果发送到一个临时文件，然后将总磁盘空间公式的结果保存在$TOTAL 中。

然后，脚本在临时文件中检索原数据，并将其发送给 gawk 命令。gawk 命令检索$TOTAL 值并将其分配给本地变量 n。

gawk 命令中的代码首先在 BEGIN 部分创建报告的头部：

```
BEGIN {
       print "Total Disk Usage by user";
```

```
        print User\tSpace\tPercent"
}
```

然后，使用 printf 命令格式化和重新调整原数据：

```
{
    printf "%s\t%d\t%6.2f%\n", $2, $1, ($1/n)*100
}
```

此部分的作用是处理 du 命令输出的每个行。prinf 命令允许将输出格式化为一个很好的表。如果愿意在系统上使用长用户名，则可能需要一些格式变换。

此处的另一个技巧是磁盘使用情况脚本将$TOTAL 变量值通过 gawk 命令行参数传递给 gawk 脚本：

```
-v n=$TOTAL
```

现在，gawk 的变量 *n* 等于用户磁盘空间的总和，可以在 gawk 脚本中的任意位置使用该值。

最后，脚本在结尾处显示用户已使用的磁盘空间的总和：

```
END {
      print "--------------------------";
      printf "Total\t%d\n", n
}'
```

此处使用了变量 n，它包含来自$TOTAL shell 变量的值。

**3. 运行脚本**

都放在一起后，运行脚本时应该得到一个很好的报告：

```
# ./diskhogs
Total Disk Usage   by user
User      Space    Percent
rich      107340   87.68%
jessica   9868      8.06%
test      5124      4.19%
katie     40        0.03%
barbara   40        0.03%
-------------------------
Total     122420
#
```

现在，我们就得到了一个很有成就感的报告，可以将它发送给老板！

注意：如果您的系统上有磁盘占用大户，则 du 命令在计算所有使用的空间时可能会消耗很长时间。这对看到 diskhogs 脚本的输出造成了一定时间的延迟，请耐心等待！

### 27.1.3 监视 CPU 和内存使用情况

任何 Linux 系统的核心统计信息应该是 CPU 和内存使用情况。如果这些值开始失控，那么系统将很快崩溃。本节将演示如何使用几个基本的 shell 脚本来编写监视和跟踪 Linux 系统上的 CPU 和内存使用情况的脚本。

**1. 必需函数**

与本章前面介绍的其他脚本一样，第一步是决定脚本生成的数据。有几个不同的命令都可以用于从系统提取 CPU 和内存使用信息。

最基本的系统统计信息命令是 uptime 命令：

```
$ uptime
 09:57:15 up 3:22, 3 users, load average: 0.00, 0.08, 0.28
$
```

uptime 命令给出了一些我们可以使用的不同的基本信息片段：

- 当前时间；
- 系统运行的天、小时和分钟数；
- 当前已登录到系统的用户数；
- 1、5 和 15 分钟的装载平均值。

另一个提取系统信息的命令是 vmstat 命令。下面是 vmstat 命令输出的示例：

```
$ vmstat
procs- ----memory--------- ---swap-- --io-- --system-- -----cpu------
r  b  swpd   free  buff cache si so bi bo in  cs us sy id  wa st
0  0 178660 13524 4316 72076  8 10 80 22 127 124  3  1  92  4  0
$
```

第一次运行 vmstat 命令时，将显示自上次重新引导的平均值。要获取当前统计信息，必须使用命令行参数运行 vmstat 命令：

```
$ vmstat 1 2
procs- ----memory--------- ---swap-- --io-- --system-- -----cpu------
r  b   swpd  free  buff cache si so bi bo in   cs  us sy id  wa st
0  0 178660 13524 4316 72076 8  10  80 22 127 124   3  1 92   4  0
0  0 178660 12845 4245 71890 8  10  80 22 127 124   3  1 86  10  0
$
```

第二行包含 Linux 系统的当前统计信息。如您所见，vmstat 命令的输出有一些隐含的意义。表 27-1 解释了每个符号的意思。

**表 27-1　　vmstat 输出符号**

| 符号 | 描述 | 符号 | 描述 |
|---|---|---|---|
| r | 等待 CPU 时间的进程数 | bo | 发送到块设备的块数 |
| b | 不间断休眠中的进程数 | in | 每秒 CPU 的中断数 |
| swpd | 使用的虚拟内存量（以 MB 为单位） | cs | 每秒 CPU 的上下文交换数 |
| free | 未使用的物理内存量（以 MB 为单位） | us | CPU 消耗在运行非内核代码上的时间百分比 |
| buff | 用作缓存空间的内存量（以 MB 为单位） | sy | CPU 消耗在运行内核代码上的时间百分比 |
| cache | 用作高速缓存空间的内存量（以 MB 为单位） | id | CPU 空间的时间百分比 |
| si | 从磁盘交换的内存量（以 MB 为单位） | wa | CPU 消耗在等待 I/O 上的时间百分比 |
| so | 交换到磁盘的内存量（以 MB 为单位） | st | 从虚拟机窃取的 CPU 时间百分比 |
| bi | 从块设备收到的块数 | | |

信息量很大。通常不需要记录来自 vmstat 命令的所有值，只需要记录各别值。通过空闲内存和空闲的 CPU 时间百分比就可以了解系统状态的大致情况（目前为止，您应该大概知道我们将如何从输出中提取这些信息了）。

您可能已经注意到 vmstat 命令的输出包含表头信息，这不是我们想要的数据。要解决这个问题，可以使用 sed 命令显示只包括数字值的行：

```
$ vmstat 1 2 | sed -n '/[0-9]/p'
1 0 172028 8564 5276 62244 9 13  98 28 160 157 5 2 89 5 0
0  0 178660 12845 4245 71890  8  10 80 22 127 124 3 1 86 10 0
$
```

现在好多了，但现在我们只需要第二行的数据。再一次调用 sed 编辑器可解决这个问题：

```
$ vmstat | sed -n '/[0-9]/p' | sed -n '2p'
0  0 178660 12845 4245 71890  8   10  80   22  127  124  3 1  86  10  0
$
```

现在可以轻松地使用 gawk 程序提取需要的数据值了。

最后，还需要用日期和时间戳来标记每个数据记录，以便指示抓取快照的时间。date 命令非常便利，但 date 命令的默认输出也许有些不便于处理。可以通过指定另一种格式来简化 date 命令的输出：

```
$ date +"%m/%d/%Y %k:%M:%S"
02/05/2008 19:19:26
$
```

这在输出中看起来好多了。说到输出，还应该考虑如何记录数据值。

对于有规律抽样的数据，通常最好直接将数据输出到日志文件。可以在$HOME 目录中创建日志文件，每次运行 shell 脚本时再追加数据。当想要查看结果时，只需查看日志文件。

还需要花些时间在思考日志文件的格式上。您可能希望确保日志文件中的数据能够便于阅读，这也是此脚本的主要目的。

还有许多不同的方法可以格式化日志文件中的数据。最流行的格式是逗号分隔值（comma-separated values，CSV）。这种格式将每个记录的数据分隔到一行中，用逗号分隔记录中的数据字段。由于它便于导入到电子表格，所以这种格式适合热衷于使用电子表格的用户。

然而，创建一个 CSV 数据文件并不是世界上最好的事情。如果想要提供更美观的报告，可以创建 HTML 文档。

多年来，HTML 早已成为格式化 Web 页面的标准方法。它使用简单的标记在 Web 页面中描绘数据类型。然而，HTML 并不只用于 Web 页面。HTML 还用于电子邮件消息。取决于 MUA 客户端，您也许能在电子邮件文档中嵌入 HTML。更好的方法是创建 HTML 报告，然后将其添加为电子邮件附件。

脚本将数据保存在 CSV 格式的文件中，以便总能够获取原数据并将其导入到电子表格中。当系统管理员运行报告脚本时，报告将被重新格式化为 HTML 报告，多棒啊！

**2. 创建捕获脚本**

由于需要在固定时间间隔对系统数据进行抽样，所以需要两个独立的脚本。一个脚本用于捕获数据并将其保存到日志文件中。此脚本应该按时间表有规律地运行。其运行频率取决于 Linux 系统的繁忙程度。对于大多数系统，每个小时运行一次即可。

第二个脚本应该输出报告数据并以电子邮件方式发送给相应的接收人。通常不希望在每次获取新数据样本时都发送新的电子邮件报告，而是希望第二个脚本以较低的频率按 cron 作业运行，如每天一次，或者一天中的第一个任务。

用于捕获数据的第一个脚本称为 capstats，脚本如下所示：

```
$ cat capstats
#!/bin/bash
# script to capture system statistics

OUTFILE=/home/rich/capstats.csv
DATE='date +%m/%d/%Y'
TIME='date +%k:%M:%S'

TIMEOUT='uptime'
VMOUT='vmstat 1 2'

USERS='echo $TIMEOUT | gawk '{print $4}''
LOAD='echo $TIMEOUT | gawk '{print $9}' | sed 's/,//''
FREE='echo $VMOUT |sed -n '/[0-9]/p' |sed -n '2p' |gawk '{print $4}''
IDLE='echo $VMOUT |sed -n '/[0-9]/p' |sed -n '2p'|gawk '{print $15}''

echo "$DATE,$TIME,$USERS,$LOAD,$FREE,$IDLE" >> $OUTFILE
$
```

此脚本从 uptime 和 vmstat 命令挖掘统计信息并将它们保存到变量中。然后将值写入到由 $OUTFILE 变量定义的文件中。对于本示例，我将文件保存到$HOME 目录。可以更改此位置以使其适用于您的环境。

创建 capstats 脚本后，在将其从时间表按规律投入运行之前应该在命令行上进行测试：

```
$ ./capstats
$ cat capstats.csv
02/06/2008,10:39:57,4,0.26,57076,87
$
```

脚本创建了新的文件，并将统计信息值放在适当的位置。为了确保后续运行脚本时不会覆盖文件，需要再次测试：

```
$ ./capstats
$ cat capstats.csv
```

```
02/06/2008,10:39:57,4,0.26,57076,87
02/06/2008,10:41:52,4,0.14,46292,88
$
```

正如我们希望的那样，第二次运行脚本时，新的统计信息数据被追加到了文件的结尾。现在，可以将些脚本按 cron 作业运行了。要使它每小时运行一次，可以创建此 cron 表项：

```
0 * * * * /home/rich/capstats
```

在 cron 表中使用 capstats shell 脚本的地方需要使用其全路径名。现在，可以每小时获取一次统计信息，而无需任何其他操作！

**3. 生成报告脚本**

现在，我们有了一个具有许多原数据的文件，可以开始编写生成提供给老板的报告脚本了。完成此任务的最佳工具是 gawk 命令。

gawk 命令允许从文件提取原数据并以任何形式显示。首先，从命令行进行测试，使用 capstats 脚本创建的新 capstats.csv 文件：

```
$ cat capstats.csv | gawk -F, '{printf "%s %s - %s\n", $1, $2, $4}'
02/06/2008 10:39:57 - 0.26
02/06/2008 10:41:52 - 0.14
02/06/2008 10:50:01 - 0.06
02/06/2008 11:00:01 - 0.18
02/06/2008 11:10:01 - 0.03
02/06/2008 11:20:01 - 0.07
02/06/2008 11:30:01 - 0.03
$
```

需要使用 gawk 命令的-F 选项将逗号定义为数据中的字段分隔符。然后，可以按需要检索各个数据字段并进行显示。

对于报告，我们将使用 HTML 格式。只需要少量的工作就可以创建格式很好的报告。显示报告的浏览器将进行格式化和显示报告的艰巨工作。我们所需要做的只是插入适当的 HTML 标记以格式化数据。

在 HTML 中显示电子表格数据的最简单方法就是使用<table>标记。表格标记允许创建带有行和单元格（在 HTML 中称为区）的表。使用<tr>标记定义行的开始。同样，使用<td>和</td>标记对定义单元格。

完整表格的 HTML 标记如下所示：

```
<html>
<body>
<h2>Report title</h2>
<table border="1">
<tr>
   <td>Date</td><td>Time</td><td>Users</td>
   <td>Load</td><td>Free Memory</td><td>%CPU Idle</td>
</tr>
<tr>
   <td>02/05/2008</td><td>11:00:00</td><td>4</td>
   <td>0.26</td><td>57076</td><td>87</td>
</tr>
</table>
</body>
</html>
```

每个数据记录是<tr></tr>标记对的一部分。每个数据字段都位于自己的<td></td>标记对中。

在浏览器中显示 HTML 报告时，浏览器将自动创建表，如图 27-1 所示。

对于脚本，所需做的只是使用 echo 命令生成 HTML 头部的代码，使用 gawk 命令生成 HTML 代码数据，然后再次使用 echo 命令关闭表。

生成 HTML 报告后，希望将它重定向或通过邮件发送出去。使用 mutt 命令（参阅第 26 章）可以轻松发送电子邮件附件。

下面是 reportstats 脚本，它将生成 HTML 报告并通过邮件发送它：

```
$ cat reportstats
#!/bin/bash
# parse capstats data into daily report

FILE=/home/rich/capstats.csv
TEMP=/home/rich/capstats.html
MAIL='which mutt'
DATE='date +"%A, %B %d, %Y"'

echo "<html><body><h2>Report for $DATE</h2>" > $TEMP
echo "<table border=\"1\">" >> $TEMP
echo "<tr><td>Date</td><td>Time</td><td>Users</td>" >> $TEMP
echo "<td>Load</td><td>Free Memory</td><td>%CPU Idle</td></tr>" >>
$TEMP
cat $FILE | gawk -F, '{
   printf "<tr><td>%s</td><td>%s</td><td>%s</td>", $1, $2, $3;
   printf "<td>%s</td><td>%s</td><td>%s</td>\n</tr>\n", $4, $5, $6;
    }' >> $TEMP
echo "</table></body></html>" >> $TEMP
$MAIL -a $TEMP -s "Stat report for $DATE" rich < /dev/null
rm -f $TEMP
$
```

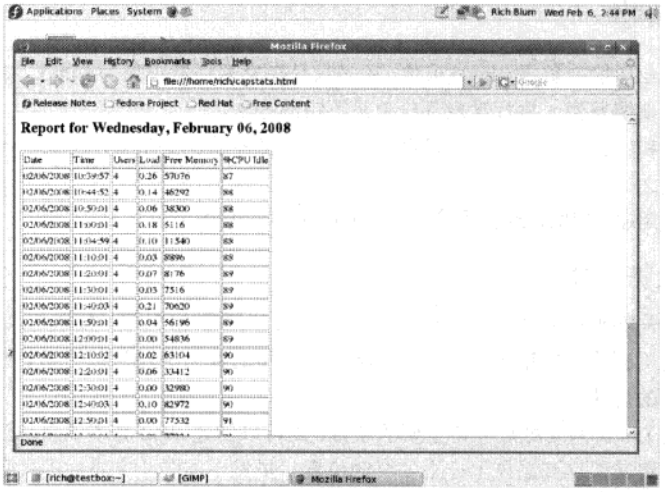



图 27-1 在 HTML 表中显示数据

由于 mutt 命令将附加文件的名称用作附件文件名，因此使用 mktemp 命令创建报告文件并不好。相反，我希望报告的名称更具描述性。脚本的最后将删除文件，所以在何处创建文件并不重要。

警告：大多数电子邮件客户端可以通过文件扩展名自动探测附件文件的类型。因此，应该让报告文件以 .html 结尾。

**4. 运行脚本**

创建 reportstats 脚本后，从命令行进行测试并查看结果：

```
$ ./reportstats
$
```

这没有什么特别之处。现在真正的测试是查看邮件消息，最好使用图形电子邮件客户端，如 KMail 或 Evolution。图 27-2 显示了从 Evolution 电子邮件客户端查看消息

Evolution 电子邮件客户端提供了在客户端窗口查看附件或不在客户端窗口查看附件的选项。图 27-2 显示了在客户端窗口查看附加的报告。注意，使用 HTML 表格组织数据非常好，与在浏览器中查看没有任何区别！

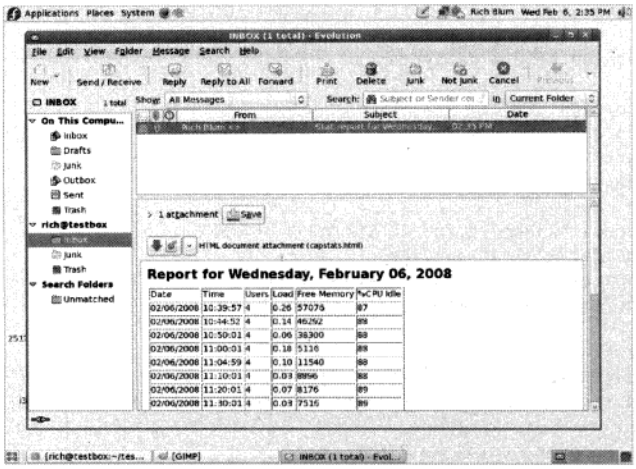



图 27-2　在 Evolution 中查看报告附件

## 27.2　执行备份

不管您是在商业环境中负责 Linux 系统，还是只是在家庭环境中使用 Linux，数据丢失都是一个灾难。为了防止此类事情发生，执行常规备份总是不错的办法。

然而，好办法和好的执行办法是两码事。制作时间表将重要文件保存起来是一个难题。这是 shell 脚本的另一个可取之处。

本节将描述使用 shell 脚本备份系统上数据的两种不同方法。

### 27.2.1　归档数据文件

通常，导致丢失数据的原因不是灾难性的系统崩溃。大多数情况下，数据是因为不小心而丢失的。十有八九是在单击“删除”按钮之后。因此，将正在使用的数据保存为归档副本并放置在系统的另一个位置是一个不错的想法。

如果正在使用 Linux 系统处理非常重要的项目，则可以创建 shell 脚本以自动获取正在使用的工作目录的快照，然后将其保存到系统上一个安全的位置。虽然不能防止灾难性的硬盘崩溃，但至少能防止文件被破坏或意外删除。

本节将描述如何创建自动 shell 脚本以获取正在使用的工作目录快照和保存以往数据版本的归档文件。

**1. 必需函数**

在 Linux 环境中，归档数据时通常使用 tar 命令（参阅第 4 章）。tar 命令用于将整个目录归档为单个文件。下面是使用 tar 命令创建工作目录归档文件的示例：

```
$ tar -cf archive.tar /home/rich/test
tar: Removing leading '/' from member names
$
```

tar 命令返回了一条警告消息，它删除了路径名开始处的正斜杠，将其转换为绝对路径以进行检索。这允许在文件系统的任何位置提取 tar 归档文件。如果希望从脚本中删除那条消息，可以通过将 STDERR 重定向到/dev/null 文件（参阅第 12 章）完成此操作：

```
$ tar -cf archive.tar /home/rich/test 2> /dev/null
$
```

现在，已经准备好在脚本中使用了。

警告：如果使用包括图形桌面的 Linux 发行版，则请小心归档$HOME 目录。虽然只是尝试，但$HOME 目录包含许多与图形桌面相关的配置文件和临时文件，创建出的归档文件可能比预计的要大很多。请选择一个子目录保存工作文件，并使用该目录作为归档目录。

创建归档文件后，可以将其发送给最喜爱的压缩工具来缩减其大小：

```
$ ls -l archive.tar
-rw-rw-r-- 1 rich rich 30720 2008-02-06 12:26 archive.tar
$ gzip archive.tar
$ ls -l archive*
-rw-rw-r-- 1 rich rich 2928 2008-02-06 12:26 archive.tar.gz
$
```

现在，您的归档系统上有了第一个组件。

下一步是创建滚动的归档系统。如果按固定频率保存工作目录，则可能不希望最新的归档副本立即覆盖以前的归档副本。

要防止此类情况发生，需要创建一个方法来自动为每个归档副本提供唯一的文件名。最简单的办法是在文件名中使用日期和时间。

如“监视系统统计信息”一节所述，可以格式化 date 命令以产生需要的任意形式的信息。在创建唯一的文件名时这非常便利。

例如，假设想使用两位数年、月、日的文件名，可以依照下面的方法操作：

```
$ DATE='date +%y%m%d'
$ FILE=tmp$DATE
$ echo $FILE
tmp080206
$
```

虽然，在文本字符串的末尾挤压$DATE 变量看起来似乎很奇怪，但在 bash shell 却是合乎语法的。shell 将变量值追加到字符串，创建唯一的文件名。

注意：使用日志来唯一标识文件可便于将来查找最近的文件版本。请记住标识日期的格式（如月-日-年或年-月-日）。通常在对文件名排序时，最好可以首先指定年，然后是月，最后是日。

现在我们已经有足够的信息开始创建脚本了。下一节将介绍创建归档脚本。

**2. 创建日常归档脚本**

archdaily 脚本自动在单独的位置创建工作目录的归档版本，同时使用日期来唯一标识文件。下面是该脚本的代码：

```
$ cat archdaily
#!/bin/bash
# archive a working directory

DATE='date +%y%m%d'
FILE=archive$DATE
SOURCE=/home/rich/test
DESTINATION=/home/rich/archive/$FILE

tar -cf $DESTINATION $SOURCE 2> /dev/null
gzip $DESTINATION
$
```

archdaily 脚本基于运行时的年、月、日生成唯一的文件名。它还使用环境变量定义源目录以进行归档，所以可以轻松更改此源目录。甚至可以使用命令行参数为 archdaily 程序添加更多功能。

$DESTINATION 变量为归档文件追加了完整的路径名。如果需要，这也可以轻易更改为另一个目录。

测试 archdaily 脚本的方法非常简单：

```
$ ./archdaily
```

```
$ ls -al /home/rich/archive
total 24
drwxrwxr-x 2 rich rich 4096 2008-02-06 12:50 .
drwx------ 37 rich rich 4096 2008-02-06 12:50 ..
-rw-rw-r-- 1 rich rich 3046 2008-02-06 12:50 archive080206.gz
$
```

现在，数据被安全地归档在一边了。

### 3. 创建每小时的归档脚本

如果您处在大容量的生产环境中，而文件变更非常迅速，那么每天的归档操作可能远远不够。如果希望将归档频率增加到每小时，则需要再考虑一件事。

如果按每小时归档文件，且使用 date 命令在文件名中添加时间戳，那么可能很快会发现它们非常难看。您肯定不希望在目录中筛选像下面这样的文件名：

```
archive080206110233.gz
```

与其将所有归档文件放在同一文件夹下，不如为归档的文件单独创建一个目录层次结构。图 27-3 展示了这个原则。

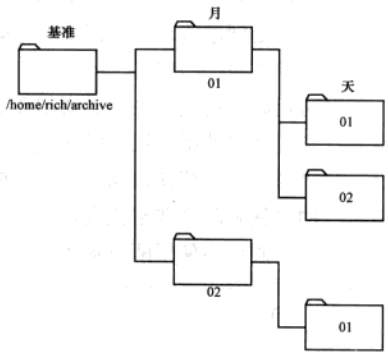



图 27-3　创建归档目录层次结构

归档目录包含每年的每个月的目录，将月份用作目录名。每个月的目录依次包含该月每天的目录（使用日期数值作为目录名）。这样可以只对各个归档文件添加时间戳，然后将其放在相应月份和日期下的目录中。

现在，有一个新的问题需要解决。脚本必须能够自动创建单独的月份和日期目录，然后知道如果它们已经存在，则不需要再创建。实际上，这没有想像的那么困难。

如果您细读过 mkdir 命令（参阅第 3 章）的命令行选项，则会发现-p 命令行选项。可以通过这一个命令创建目录和子目录，并且在目录已经存在的情况下不产生错误消息。完美极了！

现在我们准备创建 archhourly 脚本，如下所示：

```
$ cat archhourly
#!/bin/bash
# archive a working directory hourly

DAY='date +%d'
MONTH='date +%m'
TIME='date +%k%M'
SOURCE=/home/rich/test
BASEDEST=/home/rich/archive

mkdir -p $BASEDEST/$MONTH/$DAY

DESTINATION=$BASEDEST/$MONTH/$DAY/archive$TIME
tar -cf $DESTINATION $SOURCE 2> /dev/null
gzip $DESTINATION
$
```

此脚本从 date 命令检索日期和月份的值，以及用于唯一标识归档文件的时间戳。然后使用这些信息创建日期的归档目录（或者在已经存在的情况下以静默方式退出）。最后，脚本使用 tar 和 gzip 命令创建归档并进行压缩。

像 archdaily 脚本一样，在将 archhourly 脚本放到 cron 表之前应首先对其进行测试：

```
$ ./archhourly
$ ls -al /home/rich/archive/02/06
total 32
drwxrwxr-x 2 rich rich 4096 2008-02-06 13:20 .
drwxrwxr-x 3 rich rich 4096 2008-02-06 13:19 ..
```

```
-rw-rw-r-- 1 rich rich 3145 2008-02-06 13:19 archive1319.gz
$ ./archhourly
$ ls -al /home/rich/archive/02/06
total 32
drwxrwxr-x 2 rich rich 4096 2008-02-06 13:20 .
drwxrwxr-x 3 rich rich 4096 2008-02-06 13:19 ..
-rw-rw-r-- 1 rich rich 3145 2008-02-06 13:19 archive1319.gz
-rw-rw-r-- 1 rich rich 3142 2008-02-06 13:20 archive1320.gz
$
```

脚本第一次运行时很好，创建了相应的月份和日期目录，然后创建了归档文件。为了测试彻底，我又运行了一次以查看在处理已存在的目录上是否有问题。脚本再次正常运行，创建了第二个归档文件。现在可以放到时间表中了！

### 27.2.2 脱机存储备份文件

虽然对重要文件进行归档是一个很好的想法，但这无法取代保存在单独位置的完整备份。在大型的商务 Linux 环境中，系统管理员通常可以奢侈地使用磁带机将重要文件复制到服务器以外的位置进程安全储存。

对于小型环境，这并非可行之举。无法负担一种新奇的备份系统方法并不意味着您的重要数据就要承担风险。可以将 shell 脚本组合起来，即自动创建归档文件，然后发送到 Linux 系统以外的位置。本节将介绍这种方法。

**1. 必需函数**

我们已经学习了归档系统的核心内容，即 `tar` 和 `gzip` 命令。下一个关键就在于将压缩的归档文件放到 Linux 系统以外的位置。如果已经配置了电子邮件系统（参阅第 26 章），则可以将其作为通往外界的门户。

虽然系统管理员通常不将电子邮件用作归档数据的方式，但这确实是将归档数据保存在另一个安全位置的好方法。本书不建议将整个 Linux 系统归档并作为电子邮件的附件进行发送，但对工作目录归档并作为附件应该不成问题。

让我们看看这个脚本和它的工作方式。

**2. 创建脚本**

mailarch 脚本像 archdaily 和 archhourly 脚本那样创建了一个常规归档文件。创建归档文件后，使用 Mutt 电子邮件客户端将文件发送到远程电子邮件地址。

`mailarch` 脚本如下所示：

```
$ cat mailarch
#!/bin/bash
# archive a working directory and e-mail it out

MAIL='which mutt'
DATE='date +%y%m%d'
FILE=archive$DATE
SOURCE=/home/rich/test
DESTINATION=/home/rich/archive/$FILE
ZIPFILE=$DESTINATION.zip

tar -cf $DESTINATION $SOURCE 2> /dev/null
zip $ZIPFILE $DESTINATION
$MAIL -a $ZIPFILE -s "Archive for $DATE" rich@myhost.com < /dev/null
$
```

您可能已经注意到了，在 mailarch 脚本中，我使用了 zip 压缩工具，而非 `gzip` 命令。这可以创建.zip 文件，这样在非 Linux 系统（如 Microsoft Windows 工作站）上也可以轻松处理这些文件。

**3. 运行脚本**

mailarch 脚本的运行方式与 archdaily 脚本相同，即将其放到时间表中以便在一天的特定时间运行

（通常是在晚上）。脚本运行时，它将创建归档文件，使用 zip 命令压缩归档文件，然后将其通过电子邮件消息发送。图 27-4 展示了收到的电子邮件的形式。

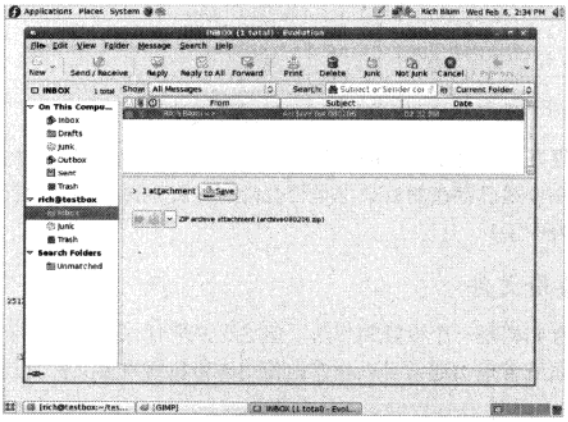


图 27-4　带有归档文件的电子邮件消息

祝贺您，您现在应该掌握了从 Linux 系统备份重要文件的方法了。

## 27.3　小结

本章将本书中出现的一些 shell 脚本信息很好地应用于 Linux 系统管理中。当您负责管理 Linux 系统时，不管是大型的多用户系统，还是自己的系统，都有许多需要监视的内容。与其大量搜索日志文件和手动运行命令，不如创建 shell 脚本来完成这些任务。

本章介绍了如何使用 df 命令确定可用的磁盘空间，然后使用 sed 和 gawk 命令从数据中检索特定的信息。将命令输出传递给 sed 和 gawk 来解析数据是 shell 脚本的常用方法，所以了解这些命令是非常必要的。

接下来，本章展示了如何使用 du 命令确定各个用户所使用的磁盘空间。du 命令的输出被传递给 sed 和 gawk 命令以帮助过滤出我们关心的数据。

接下来的一节演示了如何创建系统日志和报告。capstats shell 脚本有规律地捕获关键的系统统计信息，从命令中提取数据并保存在逗号分隔值文件中。可以将该文件导入到电子表格中，但本书的做法是创建另一个脚本从数据创建格式化的报告。reportstats shell 脚本使用 HTML 格式将原系统数据放到 HTML 表格中，然后使用 Mutt 电子邮件客户端将文件发送给远程用户。

本章最后讨论了使用 shell 脚本在 Linux 系统上进行归档和备份数据文件。tar 和 gzip 命令是两个在归档数据时非常受欢迎的命令。本章演示了如何使用它们在 shell 脚本中创建归档文件，以及在归档目录中管理归档文件。最后，演示了如何通过电子邮件将归档文件发送给远程用户，将其作为备份重要数据文件的一种方法。

感谢您和我一起分享 Linux 命令行和 shell 编程的学习之旅。希望您能享受这个过程并了解如何使用命令行，以及如何创建 shell 脚本来节省时间。但不要就此停止。在开源世界中总有新的东西开发出来，可能是新的命令行工具，也可能是完整的 shell。请多访问 Linux 社区以了解新的进展和功能。

# 附录 A
# bash 命令快速指南

APPENDIX A

如本书所述，bash shell 包含许多功能，也包含许多可用的命令。此附录为编写 bash 命令行或 bash shell 脚本时，快速查找某个功能或命令提供了一个简要的指南。

## A.1 内置命令

bash shell 将许多受欢迎的命令包含进了 shell 中。使用这些命令会更节省时间。表 A-1 显示了可以直接从 bash shell 获取的内置命令。

表 A–1 bash 内置命令

| 命令 | 描述 |
|---|---|
| alias | 定义指定命令的别名 |
| bg | 以后台方式恢复一个作业 |
| bind | 将键盘序列绑定到一个 readline 函数或宏 |
| break | 从 for、while、select 或 until 循环中退出 |
| builtin | 执行指定的 shell 内置命令 |
| cd | 将当前目录更改为指定目录 |
| caller | 返回任何活动的子例程调用的上下文 |
| command | 执行指定的命令，而不使用常规的 shell 查找 |
| compgen | 生成指定单词的可能的完整匹配 |
| complete | 显示指定的单词将如何完成 |
| continue | 恢复 for、while、select 或 until 循环的下一个递归 |
| declare | 声明变量或变量类型 |
| dirs | 显示当前记住的目录列表 |
| disown | 从作业表中删除指定作业的进程 |
| echo | 将指定的字符串显示到 STDOUT |
| enable | 启用或禁用指定的内置 shell 命令 |
| eval | 将指定的参数连结到单个命令中，然后执行该命令 |
| exec | 将 shell 进程替换为指定的命令 |
| exit | 以指定的退出状态强制 shell 退出 |
| export | 为子 shell 进程设置特定的变量 |
| fc | 从历史列表中选择命令列表 |
| fg | 以前台模式恢复一项作业 |
| getopts | 解析指定的位置参数 |
| hash | 查找和记住指定命令的全路径名 |
| help | 显示帮助文件 |

续表

| 命令 | 描述 |
| --- | --- |
| history | 显示命令历史记录 |
| jobs | 列表活动的作业 |
| kill | 将系统信号发送给指定的进程 ID（PID） |
| let | 计算在数学表达式中的每个参数 |
| local | 在函数中创建一定范围的变量 |
| logout | 退出登录的 shell |
| popd | 从目录堆栈中删除条目 |
| printf | 使用格式化的字符串显示文本 |
| pushd | 将目录添加到目录堆栈中 |
| pwd | 显示当前工作目录的路径名 |
| read | 从 STDIN 读取一行数据并分配给一个变量 |
| readonly | 从 STDIN 读取一行数据并分配给一个不可更改的变量 |
| return | 强制函数以某个值退出，这个值是执行调用脚本中检索不到的值 |
| set | 设置和显示环境变量值和 shell 属性 |
| shift | 将参数轮换一个位置 |
| shopt | 切换控制着可选 shell 行为的变量值 |
| suspend | 挂起 shell 的执行，直至收到 SIGCONT 信号 |
| test | 基于指定的条件，返回退出值 0 或 1 |
| times | 显示累积的用户和系统 |
| trap | 如果收到指定的系统信号，则执行指定的命令 |
| type | 显示指定的单词将如何被解析并作为命令进行使用 |
| ulimit | 为系统用户设置特定资源的限制 |
| umask | 为新创建的文件和目录设置默认权限 |
| unalias | 删除指定的别名 |
| unset | 删除指定的环境变量或 shell 属性 |
| wait | 等候指定的进程完成并返回退出状态 |

内置命令比外部命令性能更高，但添加到 shell 的内置命令越多，可能不再需要使用的命令所消耗的内存也越多。bash shell 还包含了用于 shell 的扩展功能的外部命令。我们将在下一节中讨论这些内容。

## A.2　bash 命令

除了内置命令外，bash shell 利用外部命令来允许用户管理文件系统以及操作文件和目录。表 A-2 展示了在 bash shell 中常用的外部命令。

表 A-2　　bash shell 外部命令

| 命令 | 描述 |
| --- | --- |
| bzip2 | 使用 Burrows-Wheeler 块排序算法和 Huffman 编码进行压缩 |
| cat | 列出指定文件的内容 |
| chage | 更改指定系统用户账户的密码过期日期 |

续表

| 命令 | 描述 |
| --- | --- |
| chfn | 更改指定用户账户的注释信息 |
| chgrp | 更改指定文件或目录的默认组 |
| chmod | 更改指定文件或目录的系统安全权限 |
| chown | 更改指定文件或目录的默认所有者 |
| chpasswd | 读取包含登录名和密码对的文件并更新密码 |
| chsh | 更改指定用户账户的默认 shell |
| compress | 原始的 Unix 文件压缩实用程序 |
| cp | 将指定的文件复制到另一个位置 |
| df | 显示所有已装载设备的硬盘使用情况统计信息 |
| du | 显示指定文件路径的磁盘使用情况统计信息 |
| file | 显示指定文件的文件类型 |
| finger | 显示 Linux 系统或远程系统的用户账户信息 |
| grep | 在文件中搜索指定的文本字符串 |
| groupadd | 创建新系统组 |
| groupmod | 修改现有系统组 |
| gzip | GNU 项目的压缩（使用 Lempel-Ziv 压缩） |
| head | 显示指定文件内容的第一部分 |
| killall | 基于进程名称将系统信号发送到一个正在运行的进程 |
| less | 查看文件内容的高级方式 |
| link | 使用别名创建到文件的链接 |
| ls | 列出目录内容 |
| mkdir | 在当前目录下创建指定的目录 |
| more | 列出指定文件的内容，并在显示一屏数据后暂停 |
| mount | 将磁盘设备显示或挂载到虚拟文件系统 |
| passwd | 更改系统用户账户的密码 |
| ps | 显示系统上正在运行的进程的信息 |
| pwd | 显示当前目录 |
| mv | 重命名文件 |
| rm | 删除指定的文件 |
| rmdir | 删除指定的目录 |
| sort | 基于指定的顺序在数据文件中组织数据 |
| stat | 查看指定文件的统计信息 |
| tail | 显示指定文件内容的最后一部分 |
| tar | 将数据和目录归档到单个文件中 |
| touch | 创建新的空文件，或者更新现有文件的时间戳 |
| umount | 从虚拟文件系统中删除已挂载的磁盘设备 |
| useradd | 创建新的系统用户账户 |
| userdel | 删除现有的系统用户账户 |
| usermod | 修改现有的系统用户账户 |
| zip | Windows PKZIP 程序的 Unix 版本 |

## A.3 环境变量

bash shell 还利用了许多环境变量。虽然环境变量不是命令，但它们会影响 shell 命令的操作，所

以了解 shell 环境变量是非常重要的。表 A-3 显示了 bash shell 中可用的默认环境变量。

表 A-3　bash shell 环境变量

| 变量 | 描述 |
| --- | --- |
| $* | 将所有命令行参数包含在单个文本值中 |
| $@ | 将所有命令行参数包含在分隔的文本值中 |
| $# | 命令行参数的个数 |
| $? | 最近使用过的前台进程的退出状态 |
| $- | 当前命令行选项的标记 |
| $$ | 当前 shell 的进程 ID（PID） |
| $! | 最近在后台执行的 PID |
| $0 | 命令行上的命令名称 |
| $_ | shell 的绝对路径名 |
| BASH | 用于调用 shell 的全文件名 |
| BASH_ARGC | 当前子例程中的参数数量 |
| BASH_ARGV | 包含所有指定命令行参数的数组 |
| BASH_COMMAND | 当前正在执行的命令名称 |
| BASH_ENV | 在设置时，每个 bash 脚本在运行前都试图执行由此变量定义的启动文件 |
| BASH_EXECUTION_STRING | 在-c 命令行选项中使用的命令 |
| BASH_LINENO | 脚本中包含每个命令的行号的数组 |
| BASH_REMATCH | 包含与指定正则表达式匹配的文本元素的数组 |
| BASH_SOURCE | shell 中包含已声明函数的源文件名的数组 |
| BASH_SUBSHELL | 当前 shell 中出现的子 shell 数量 |
| BASH_VERSION | bash shell 的当前实例的版本号 |
| BASH_VERSINFO | 一个可变数组，它包含 bash shell 当前实例的主要版本和次要版本 |
| COLUMNS | 用于当前 bash shell 实例的终端的宽度 |
| COMP_CWORD | 变量 COMP_WORDS 的索引，包含当前的光标位置 |
| COMP_LINE | 当前命令行 |
| COMP_POINT | 当前光标位置的索引（与当前命令的开始处相关） |
| COM_WORDBREAKS | 一系列字符，在执行单词完成时用作单词的分隔符 |
| COMP_WORDS | 一个变量数组，包含当前命令行上的各个单词 |
| COMPREPLY | 一个变量数组，包含 shell 函数生成的可能的完成代码 |
| DIRSTACK | 包含当前目录堆栈内容的变量数组 |
| EUID | 当前用户的有效数字用户 ID |
| FCEDIT | fc 命令的默认编辑器 |
| FIGNORE | 以冒号分隔的前缀列表，用于执行文件名完成时要忽略的项 |
| FUNCNAME | 当前正在执行的函数的名称 |
| GLOBIGNORE | 以冒号分隔的模式列表，它定义了一系列被文件扩展忽略的文件名 |
| GROUPS | 包含组列表的变量数组，当前用户是组成员之一 |
| histchars | 控制历史扩展的字符（不超过 3 个字符） |
| HISTCMD | 当前命令的历史号 |
| HISTCONTROL | 控制输入到 shell 历史列表的命令 |
| HISTFILE | 保存 shell 历史列表的文件名称（默认是.bash_history） |
| HISTFILESIZE | 保存到历史文件的最大行数 |
| HISTIGNORE | 以冒号分隔的模式列表，用于决定历史文件中应忽略哪些命令 |
| HISTSIZE | 保存到历史文件的最大命令个数 |
| HOSTFILE | 在 shell 需要完成主机名时，此变量包含应该读取的文件名称 |

续表

| 变量 | 描述 |
| --- | --- |
| HOSTNAME | 当前的主机名 |
| HOSTTYPE | 描述 bash shell 所运行在的计算机的字符串 |
| IGNOREEOF | shell 在退出前必须收到的连续 EOF 字符的个数 如果此值不存在，则默认为 1 |
| INPUTRC | Readline 初始化文件的名称（默认为 .inputrc） |
| LANG | shell 的本地类别 |
| LC_ALL | 定义本地类别，覆盖 LANG 变量 |
| LC_COLLATE | 在对字符串值排序时，设置要使用的排序顺序 |
| LC_CTYPE | 决定要在文件名扩展和模式匹配中使用的字符解释 |
| LC_MESSAGES | 在解析前缀为美元符号的带双引号的字符串时，决定要使用的本地设置 |
| LC_NUMERIC | 决定格式化数字时使用的本地设置 |
| LINENO | 当前正在执行的脚本行号 |
| LINES | 定义终端上可用的行数 |
| MACHTYPE | 以 cpu-company-system 格式定义系统类型的字符串 |
| MAILCHECK | shell 检查新邮件的时间间隔（以秒为单位，默认为 60 秒） |
| OLDPWD | shell 中使用的上一个工作目录 |
| OPTERR | 如果设置为 1，则 bash shell 显示由 getopts 命令产生的错误 |
| OSTYPE | 定义运行 shell 的操作系统的字符串 |
| PIPESTATUS | 一个变量数组，包含来自前台进程的进程退出状态值的列表 |
| POSIXLY_CORRECT | 如果设置此变量，则 bash 以 POSIX 模式启动 |
| PPID | bash shell 父进程的进程 ID(PID) |
| PROMPT_COMMAND | 如果设置此变量，则命令将在显示主提示之前得以执行 |
| PS1 | 主命令行提示字符串 |
| PS2 | 次命令行提示字符串 |
| PS3 | select 命令使用的提示 |
| PS4 | 如果使用了 bash 的-x 参数，则在显示命令行之前显示提示 |
| PWD | 当前的工作目录 |
| RANDOM | 返回一个 0～32767 之间的随机数。将值分配给此变量作为随机数生成器的种子 |
| REPLY | read 命令的默认变量 |
| SECONDS | shell 启动后的秒数。为此变量分配值时，计时器将被重置为此值 |
| SHELLOPTS | 以冒号分隔的已启用的 bash shell 选项的列表 |
| SHLVL | 指示 shell 的级别，每当启动新的 bash shell 时加 1 |
| TIMEFORMAT | 指定 shell 以何种格式显示时间值 |
| TMOUT | select 和 read 命令等待输入的秒数，默认是 0 秒，即无限等待 |
| UID | 当前用户的数字实际用户 ID |

可以使用 set 内置命令显示环境变量。默认的环境变量是在启动时设置的，而且在不同的 Linux 发行版之间会有所不同。

APPENDIX B

# 附录 B
# sed 和 gawk 快速指南

如果需要在 shell 脚本中执行任何类型的数据处理，则很可能要用到 sed 或者 gawk 程序（或同时用到二者）。此附录部分提供了 sed 和 gawk 命令的快速参考，在 shell 脚本中处理数据时会发现这些命令非常便利。

## B.1 sed 编辑器

sed 编辑器可以基于输入到命令行上的命令或保存在文本文件中的命令在数据流中处理数据。它一次从输入中读入一行数据，并将此数据与提供的编辑器命令匹配，按照命令中指定的方式更改数据流中的数据，然后将新数据输出到 STDOUT。

### B.1.1 启动 sed 编辑器

使用 sed 命令的格式为：

```
sed options script file
```

*options* 参数允许定义 sed 命令的行为，它包括表 B-1 中所示的选项。

表 B–1 sed 命令选项

| 选项 | 描述 |
| --- | --- |
| -e *script* | 将 script 中指定的命令添加到处理输入时运行的命令中 |
| -f *file* | 将 file 中指定的命令添加到处理输入时运行的命令中 |
| -n | 不产生每个命令的输出，但等待打印命令 |

*script* 参数指定应用到流数据上的单个命令。如果需要多个命令，则必须使用-e 选项在命令行中进行指定，或-f 选项在单独的文件中指定它们。

### B.1.2 sed 命令

sed 编辑器脚本包含一些命令，sed 将用这些命令处理输入流中的每一行数据。本节描述了可能会用到的更多常用 sed 命令。

**1. 替换**

s 命令替换输入流中的文本。s 命令的格式为：

```
s/pattern/replacement/flags
```

其中，*pattern* 是要替换的文本，而 *replacement* 是 sed 将插入的新文本。

*flags* 参数控制如何替换。有 4 种类型的替换标志。

- 数字，表示应该替换第几次出现的模式。
- g：表示所有出现的实例都应该替换。
- P：表示原始行的内容应该打印。
- w file：表示替换结果应该写入到文件中。

在第一种替换类型中，可以指定 sed 编辑器应该替换第几次匹配的模式，如 2 表示仅替换第二次出现的模式。

### 2. 寻址

默认情况下，在 sed 编辑器中使用的命令将应用于文本数据的所有行。如果想要命令只应用于某个特定的行或几个行，则必须使用**行寻址**。

在 sed 编辑器中，有两种形式的行寻址：

- 行数字范围；
- 过滤出一行的文本模式。

两种模式都使用相同的格式来指定地址：

```
[address] command
```

在使用数字行寻址时，通过行在文本流中的位置来引用行。sed 编辑器将文本流中的第一行指定为第一行并对每个新行依此类推。

```
$ sed '2,3s/dog/cat/' data1
```

另一种用于限制命令应用于哪些行的方法会更复杂一些。sed 编辑器允许指定一个文本模式来过滤要应用命令的行。格式为：

```
/pattern/command
```

必须用正斜杠包围指定的模式。sed 编辑器只将命令应用到包含指定文本模式的行。

```
$ sed '/rich/s/bash/csh/' /etc/passwd
```

此过滤器首先找到包含文本 rich 的行，然后使用文本 csh 替换 bash。

还可以对一个地址组合使用多个命令：

```
address {
    command1
    command2
    command3
}
```

sed 编辑器将每个指定的命令应用到与指定地址匹配的行上。sed 编辑器将处理每个列在地址行中的命令：

```
$ sed '2{
> s/fox/elephant/
> s/dog/cat/
> }' data1
```

sed 编辑器将每个替换命令应用于数据文件的第二行。

### 3. 删除行

删除命令 d 即执行删除。此命令将删除与提供的寻址模式匹配的任何文本行。使用删除命令时需要特别注意，因为如果忘记包含寻址模式，则将删除流中的所有行：

```
$ sed 'd' data1
```

显然，删除命令在指定地址时使用是非常有用的。它允许从数据流中删除特定的文本行，也可以通过行号删除：

```
$ sed '3d' data6
```

或删除特定范围的行：

```
$ sed '2,3d' data6
```

sed 编辑器的模式匹配功能也可用于删除命令：

```
$ sed '/number 1/d' data6
```

将只从流中删除与指定的文本匹配的行。

**4. 插入和追加文本**

您可能已经想到了，像许多其他编辑器一样，sed 编辑器也允许向数据流插入和追加文本行。两个操作之间的区别也许容易混淆：

- 插入命令(i)在指定的行之前添加一个新行。
- 追加命令(a)在指定的行之后添加一个新行。

两者容易混淆之处在于它们的格式。不能在单个命令行上使用这些命令，必须在单独的行上指定要插入或追加的行。其格式为：

```
sed '[address]command\
new line'
```

sed 编辑器中 *new line* 中的文本将在指定的位置输出。请切记，在使用插入命令时，文本出现在数据流文本之前：

```
$ echo "testing" | sed 'i\
> This is a test'
This is a test
testing
$
```

而在使用追加命令时，文本出现在数据流文本之后：

```
$ echo "testing" | sed 'a\
> This is a test'
testing
This is a test
$
```

此命令允许在常规的文本之后插入文本。

**5. 更改行**

更改命令允许更改数据流中整个文本行的内容。其工作方式与插入和追加命令相同，即必须与 sed 命令其余部分分开单独指定新行：

```
$ sed '3c\
> This is a changed line of text.' data6
```

反斜杠字符用于在脚本中指示数据的新行。

**6. 转换命令**

转换命令（y）是 sed 编辑器中唯一只操作单个字符的命令。转换命令的使用格式为：

```
[address]y/inchars/outchars/
```

转换命令执行 *inchars* 和 *outchars* 值的一对一映射。*inchars* 中的第一个字符转换为 *outchars* 中的第一个字符。*inchars* 中的第二个转换为 *outchars* 中的第二个字符。此映射将在指定的字符长度上持续。如果 *inchars* 和 *outchars* 的长度不同，则 sed 编辑器将产生一条错误消息。

**7. 打印行**

与替换命令中的 p 标志类似，p 命令在 sed 编辑器的输出中打印一个行。打印命令最常见的用法是打印所有包含与某个文本模式匹配的行。

```
$ sed -n '/number 3/p' data6
This is line number 3.
$
```

打印命令允许从输入流中过滤出特定的数据行。

**8. 写入到文件**

w 命令用于将行写入到文件。w 命令的格式为：

```
[address]w filename
```

*filename* 可以是相对路径，也可以是绝对路径，但不管哪种形式，运行 sed 编辑器的用户必须具有文件的写权限。*address* 可以是 sed 使用的任何寻址方式，如单个行号、文本模式或一定范围的

行数或文本模式。

下面的示例将数据流的前两行打印到一个文本文件中：

```
$ sed '1,2w test' data6
```

输出文件 *test* 只包含输入流的前两行。

**9. 从文件读入**

我们已经介绍了如何从 *sed* 命令行上将文本插入和追加到数据流。读取命令（*r*）允许插入包含在单独文件中的数据。

读取命令的格式为：

```
[address]r filename
```

*filename* 参数用于指定包含数据的文件的相对或绝对路径。不能对读取命令使用范围地址，只能指定单个行号或文本模式地址。sed 编辑器将文件中的文本插入到地址之后。

```
$ sed '3r data' data2
```

sed 编辑器从 *data2* 文件的第 3 行开始，将 *data* 文件的全部文本插入到 *data2* 文件。

## B.2 gawk 程序

gawk 是原始 Unix 平台上 awk 程序的 GUN 版本。awk 程序在流编辑上比 sed 编辑器更进一步，它提供了一种编程语言，而不仅仅是编辑器命令。本节将介绍 gawk 程序的基本知识，可以将本节内容作为其功能的快速参考。

### B.2.1 gawk 命令格式

gawk 程序的基本格式为：

```
gawk options program file
```

表 B-2 展示了 gawk 程序可用的选项。

**表 B-2 gawk 选项**

| 选项 | 描述 |
|---|---|
| -F *fs* | 指定用于描述行中的数据字段的文件分隔符 |
| -f *file* | 指定从中读取程序的文件名 |
| -v *var=value* | 定义用于 gawk 程序的变量及其默认值 |
| -mf *N* | 指定在数据文件中处理的最大字段数 |
| -mr *N* | 指定数据文件中的最大记录大小 |
| -W *keyword* | 指定 gawk 的兼容模式或警告级别。使用 help 可列出所有的关键字 |

命令行选项提供了一种在 gawk 程序中自定义功能的简单方法。

### B.2.2 使用 gawk

可以从命令行直接使用 gawk，也可以在 shell 脚本中使用它。本节展示了如何使用 gawk 程序，以及如何输入供 gawk 处理的脚本。

**1. 从命令行读取程序脚本**

gawk 程序脚本定义在一对大括号中。必须将脚本命令放在一对大括号之间。因为 gawk 命令行假设脚本是单个文本字符串，所以还必须将脚本用一对单引号括起来。下面是一个简单的在命令行上指定的 gawk 程序脚本示例：

```
$ gawk '{print $1}'
```

此脚本显示输入流中每一行的第一个数据字段。

**2. 在程序脚本中使用多个命令**

只能执行一个命令的编程语言没有多大用处。gawk 编程语言允许将几个命令合并到一个常规的程

序中。要在命令行上指定的程序脚本中使用多个命令，只需要在命令之间放置一个分号：

```
$ echo "My name is Rich" | gawk '{$4="Dave"; print $0}'
My name is Dave
$
```

此脚本执行两个命令：第一个命令将第四个数据字段替换为不同的值，然后显示流中的整个数据行。

**3. 从文件读取程序**

与使用 sed 编辑器类似，gawk 编辑器允许将程序保存在文件中并在命令行上进行引用：

```
$ cat script2
{ print $5 "'s userid is " $1 }
$ gawk -F: -f script2 /etc/passwd
```

gawk 程序将使用文件中指定的所有命令处理输入数据流。

**4. 处理数据前运行脚本**

gawk 程序还允许指定运行脚本的时间。默认情况下，gawk 从输入中读取一行文件，然后在文本行的数据上执行程序脚本。有时，可能需要在处理数据之前运行脚本，如创建一个报告的头部分。此时，需要使用 BEGIN 关键字。这可以强制 gawk 在读取数据前，BEGIN 关键字之后执行指定的程序脚本。

```
$ gawk 'BEGIN {print "This is a test report"}'
This is a test report
$
```

可以在 BEGIN 部分放置任何类型的 gawk 命令，如赋予变量默认值。

**5. 处理数据之后运行脚本**

与 BEGIN 关键字类似，END 关键字允许指定 gawk 在读取数据之后执行的程序脚本：

```
$ gawk 'BEGIN {print "Hello World!"} {print $0} END {print
    "byebye"}'
Hello World!
This is a test
This is a test
This is another test.
This is another test.
byebye
$
```

gawk 程序首先执行 BEGIN 部分的代码，然后处理输入流中的任何数据，最后执行 END 部分的代码。

## B.2.3　gawk 变量

gawk 程序不仅是一个编辑器，它还是一个完整的编程环境。因此，存在着许多与 gawk 相关的命令和功能。本节将展示使用 gawk 编程时需要了解的主要功能。

**1. 内置变量**

gawk 使用内置变量引用程序数据内的特定功能。本节讲述可以在 gawk 程序中使用的内置变量，并演示如何使用这些内置变量。

gawk 程序将数据定义为记录和数据字段。一个记录是一行数据（默认情况下由换行符号分隔），而数据字段是行中独立的数据元素（默认由空白符号分隔，如空格、制表符）。

gawk 程序使用数据字段变量引用每个记录中的数据元素。表 B-3 描述了这些变量。

**表 B-3　gawk 数据字段和记录变量**

| 变量 | 描述 |
|---|---|
| $0 | 整个数据记录 |
| $1 | 记录中的第一个数据字段 |
| $2 | 记录中的第二个数据字段 |
| $n | 记录中的第 *n* 个数据字段 |
| FIELDWIDTHS | 由空格分隔的数字列表，它定义每个数据字段的精确宽度（空格数） |

续表

| 变量 | 描述 |
| --- | --- |
| FS | 输入字段分隔符号 |
| RS | 输入记录分隔符号 |
| OFS | 输出字段分隔符号 |
| ORS | 输出记录分隔符号 |

除了可用的字段和记录分隔符外，gawk 还提供了一些内置变量来帮助了解数据的状况和 shell 环境中的信息。表 B-4 展示了 gawk 中的其他内置变量。

表 B-4 更多 gawk 内置变量

| 变量 | 描述 |
| --- | --- |
| ARGC | 出现的命令行参数个数 |
| ARGIND | 正在处理的当前文件的 ARGV 中的索引 |
| ARGV | 命令行参数数组 |
| CONVFMT | 数字的转换格式（见 printf 语句）。默认值为%.6g |
| ENVIRON | 当前 shell 环境变量和其值的关联数组 |
| ERRNO | 读取或关闭输入文件发生错误时的系统错误 |
| FILENAME | 用于 gawk 程序输入的数据文件的文件名 |
| FNR | 数据文件中的当前记录号 |
| IGNORECASE | 如果值设置为非 0，gawk 所有字符串函数（包括正则表达式）将忽略字符的大小写 |
| NF | 数据文件中数据字段的总数 |
| NR | 已处理的输入记录个数 |
| OFMT | 显示数字的输出格式。默认为%.6g |
| RLENGTH | 匹配函数中匹配上的子字符串长度 |
| RSTART | 匹配函数中匹配上的子字符串的启始索引 |

可以在 gawk 程序脚本的任意地方使用内置变量，包括 BEGIN 和 END 部分。

**2. 在脚本中分配变量**

在 gawk 程序中将值分配给变量类似于在 shell 脚本中的处理方式，即使用**分配语句**：

```
$ gawk '
> BEGIN{
> testing="This is a test"
> print testing
> }'
This is a test
$
```

将值分配给变量后，可以在 gawk 脚本中的任意地方使用该变量。

**3. 在命令行中分配变量**

还可以使用 gawk 命令行为 gawk 程序赋值变量。这允许在常规代码之外设置值，动态更改值。下面的示例展示了使用命令行变量显示文件中特定数据字段：

```
$ cat script1
BEGIN{FS=","}
{print $n}
$ gawk -f script1 n=2 data1
$ gawk -f script1 n=3 data1
```

此功能是在 gawk 脚本中处理来自 shell 脚本的数据的好方法。

## B.2.4 gawk 程序功能

还有一些功能使得 gawk 程序在处理数据时变得非常便利，它允许创建可以处理任何类型的文本

文件（如日志文件）的 gawk 脚本：

### 1. 正则表达式

可以使用基本正则表达式（BRE）或扩展的正则表达式（ERE）过滤程序脚本应用的数据流中的行。在使用正则表达式时，正则表达式必须出现在它控制的程序脚本的左大括号之前：

```
$ gawk 'BEGIN{FS=","} /test/{print $1}' data1
This is a test
$
```

### 2. 匹配操作符

匹配操作符允许限制表达式来指定记录中的特定数据字段。匹配操作符是波浪线（~）。使用时指定匹配操作符和可用的数据字段，然后是要匹配的正则表达式：

```
$1 ~ /^data/
```

此表达式过滤出第一个数据字段以文本 data 开始的记录。

### 3. 数学表达式

除了正则表达式外，还可以在匹配模式中使用数学表达式。此功能在匹配数据字段中的数字值时非常有用。例如，如果要显示属于根用户组（组号是 0）的所有系统用户，则可以使用下面的脚本：

```
$ gawk -F: '$4 == 0{print $1}' /etc/passwd
```

此脚本显示所有第一个数据字段的值，条件是这些行的第四个数据字段为 0。

### 4. 结构化命令

gawk 程序支持以下结构化命令：

if-then-else 语句：

```
if (condition) statement1; else statement2
```

while 语句：

```
while (condition)
{
   statements
}
```

do-while 语句：

```
do
{
   statements
} while (condition)
```

for 语句：

```
for ( variable assignment; condition; iteration process)
```

这为 gawk 脚本开发者提供了丰富的编程选择。编写的 gawk 程序的功能可以与任何高级语言程序相匹敌。

# 附录 C
# shell 版本比较

APPENDIX C

Linux 环境中有丰富的 shell 版本，有时可能难以选择哪种 shell 是最适合您的。有时，不得不使用某个特定的 shell，而您需要判断在默认 shell 脚本中使用的功能在新的 shell 中是否还受支持。本附录对 Linux 环境中常用的 shell 版本之间的不同功能进行了简要的比较。

## C.1 变量

每个 shell 都支持环境变量和用户定义的变量，但它们支持变量的方式是不同的。本节讲述在 shell 脚本中使用变量时需要注意的事项。

### C.1.1 环境变量

bash、dash、ksh93 和 zsh 都使用大写字符表示环境变量。zsh shell 为常规变量提供了大写和小写两种环境变量，因此，如果在脚本中引用大写环境变量，那么它们也可以在 zsh shell 中正常使用。

不幸的是，tcsh shell 对于特殊的 shell 变量使用小写变量。然而，它还是提供了一定数量的与某些基本的 Bourne shell 环境变量匹配的大写环境变量以提供一定的兼容性。如果您的脚本对 bash shell 环境变量有很强的依赖性，则需要再次检查以确保那些变量在 tcsh shell 中可用。

需要注意的是，tcsh shell 设置环境变量的方式与其他 shell 不同。它使用 setenv 命令设置环境变量：

```
setenv variable value
```

这与其他 shell 使用 set 命令（将在下一节“用户定义的变量”中描述）设置环境变量不同。

### C.1.2 用户定义的变量

所有的 shell 都允许用户在命令行和 shell 脚本中定义自己的变量。除 tcsh shell 以外的所有 shell 都允许直接从命令行提示符设置变量而不需要特别的命令：

```
$ testing=100
```

在tcsh shell中必须使用*set*命令：

```
$ set testing=100
```

也可以在其他shell中使用*set*命令，但这不是必须的。tcsh shell还可以使用@符号来设置变量：

```
$ @ testing=100
```

在所有shell版本中，要让子shell和进程获取变量，必须使用*export*命令：

```
$ export testing
```

在所有shell版本中，可以使用*typeset*命令在使用变量前声明变量的属性。在ksh93和zsh shell中，必须使用*typeset*命令定义关联数组变量：

```
$ typeset -A testing
```

ksh93和zsh shell还允许使用*typeset*命令定义浮点变量。

### C.1.3 数组变量

所有shell都支持一维数字数组：

```
$ mytest=(one two three four five)
$ echo $mytest
one
$ echo ${mytest[2]}
three
$ echo ${mytest[*]}
one two three four five
$
```

在bash、ksh93和tcsh shell中，数字数组都是将0做为第一个元素。zsh shell以1作为索引开始，而不是0。

ksh93和zsh shell还支持关联数组，允许使用文本值作为数组的索引。在ksh93 shell中，可以按下面的方式定义关联数组：

```
$ typeset -A test2
$ test2=( [fruit]=banana [vegetable]=carrot [age]=18)
```

然而，zsh shell中的方式为：

```
% typeset -A test
% test=( fruit banana vegetable carrot age 18 )
```

注意，它们在定义数据元素时的微小差别。

## C.2 结构化命令

所有的shell都支持基本的结构化命令以处理shell脚本流。然而，有些shell使用的语法略有不同。本节讨论了这些shell之间的不同之处。

### C.2.1 if-then、while和until语句

bash、dash、ksh93、和zsh shell使用相同的`if-then`、`while`和`until`语句格式。`if-then`语句的格式为：

```
if command
then
   commands
else
   commands
fi
```

同样的格式也可以应用于`while`和`until`语句。每条语句执行一个命令，然后检查命令的退出状态。如果退出状态为零，则`if-then`语句执行第一个命令集；如果退出状态不为零，则执行第二个命令集。

`while`和`until`语句在指定的命令中循环，直至评估语句的退出状态为零。

tcsh shell 对于这些语句使用的格式略有不同：

```
if (expression) then
   commands
else
   commands
endif
```

while 和 until 语句使用的格式类似，它们都使用表达式而不是命令进行评估。在这两种格式中，表达式的结果决定 shell 将执行哪个部分。如果表达式以退出状态 0 退出，则 shell 执行 then 部分的命令。如果表达式以非 0 退出状态退出，则 shell 执行 else 部分的命令。

bash、dash、ksh93 和 zsh shell 使用 test 命令（或它的别名，[]）评估值和文件：

```
if [ -d $HOME ]
then
   echo "Your HOME directory exists"
else
   echo "There's a problem with your HOME directory"
fi
```

tcsh shell 使用这样一种表达式，它与测试表达式结合，与用于 test 命令的方式类似。

```
if ( -d $HOME ) then
   echo "The HOME directory exists"
endif
```

这一微小的差异可能为 shell 脚本造成各种各样的问题。

### C.2.2 for 语句

for 语句又是一个问题。bash、dash、ksh93 和 zsh shell 对于基本的 for 语句使用的格式相同：

```
for var in list
do
   commands
done
```

bash 和 zsh shell 还支持 C 语言风格的 for 语句：

```
for (( variable assignment ; condition ; iteration process ))
```

然而，tcsh shell 不支持 for 语句，而是使用 foreach 语句：

```
foreach var1 (wordlist)
   statements
end
```

在将 shell 脚本迁移到 tcsh 环境中时，这是一个非常严重的兼容性问题。

## C.3 数学计算

所有的 shell 都允许在其中执行一定级别的数学计算。ksh93 和 zsh shell 在这一方面尤其突出。

bash 和 dash shell 使用 expr 命令计算数学表达式：

```
$ expr 1 + 5
6
```

它们都为定义数学计算提供了$[]和$(())快捷方式：

```
$ var1=$[1 + 5]
$ echo $var1
6
```

tcsh shell 使用@符号定义数学计算：

```
$ set test1 = 10
$ set test2 = 15
$ @ test3 = ( $test1 * $test2 )
$ echo $test3
150
$
```

bash、dash和tcsh shell只能执行整数计算。

ksh93和zsh都可以使用两种方法定义数学计算。let命令：

```
% let value1=" 4 * 5.1 / 3.2 "
% echo $value1
6.3749999999999991
```

和$(())快捷方式：

```
% value1=$(( 4 * 5.1 ))
% (( value2 = 4 * 5.1 ))
% printf "%6.3f\n" $value1 $value2
20.400
20.400
%
```

另外，ksh93和zsh shell在所有的数学计算中都支持浮点运算。

ksh93 shell将许多常用的数学函数内置到了shell中：

```
$ value1=$(( sqt(9) ))
$ echo $value1
3
```

zsh shell未包括内置的数学函数，但将它们作为单独的可装载的模块进行提供：

```
% zmodload zsh/mathfunc
% value1=$(( sqrt(9) ))
% echo $value1
3.
```

显然，如果对编写带有高级数学函数的shell脚本感兴趣，则可以优先考虑使用ksh93或zsh shell。